

JavaScript Uncommon & Esoteric Mega Master List

(Part 8)

આ લિસ્ટમાં ફ્રનિયાની કોઈ સામાન્ય ટ્યુટોરીયલ સાઇટ પર ન જોવા મળતા ૧૦૦% યુનિક અને એડવાન્સ જાવાસ્ક્રિપ્ટ એન્જિન ક્વોર્ક્સ અને આર્કિટેક્ચર કોન્સપ્ટ્સને આવરી લેવામાં આવ્યા છે.

JavaScript Term / Concept	સરળ ગુજરાતીમાં એક્સપ્લેનેશન (Simple & Readable)
Tail Call Optimization (TCO)	જ્યારે કોઈ ફંક્શન તેના રિટર્ન સ્ટેટમેન્ટમાં જ બીજા ફંક્શનને છેલ્લે કોલ કરે છે, ત્યારે એન્જિન નવું સ્ટેક ફ્રેમ ઉમેરવાને બદલે જૂની સ્ટેક ફ્રેમનો જ ઉપયોગ કરે છે જેથી મેમરી બચે છે અને સ્ટેક ઓવરફ્લો થતો નથી.
Tagged Template Literals	સ્ટ્રિંગ લેકેટિક્સ (`` ` ``) ની આગળ એક ફંક્શનનું નામ લગાવીને સ્ટ્રિંગ પાસ કરવાની પદ્ધતિ. આ ફંક્શન સ્ટ્રિંગ અને તેની અંદરના વેરીએબલ્સને અલગ-અલગ આર્ગ્યુમેન્ટ તરીકે સ્વીકારીને કરમ પ્રોસેસિંગ કરવાની છૂટ આપે છે.
Property Descriptor Flags	જાવાસ્ક્રિપ્ટ ઓબ્જેક્ટની અંદરની પ્રોપર્ટીઝના હિડન કંટ્રોલ ફ્લેગ્સ: Writable (બદલી શકાય કે નહીં), Enumerable (લૂપમાં દેખાય કે નહીં), અને Configurable (ડીલીટ કે મોડીફાય થઈ શકે કે નહીં).
Temporal Dead Zone (TDZ) for Classes	વેરીએબલ્સની જેમ જ, જાવાસ્ક્રિપ્ટમાં ક્લાસ (Class) પણ હોઇસ્ટ થાય છે પરંતુ જ્યાં સુધી કોડમાં તે એક્ઝ્યુઅલ ડિક્લેર ન થાય ત્યાં સુધી તેને વાપરી શકાતો નથી, આ વચ્ચેના બગ-પ્રોન સમયગાળાને ક્લાસ ટીડીઝ (TDZ) કહે છે.

V8 Ignition & TurboFan	ગૂગલ કોમના V8 એન્જિનના બે સ્તંભો: Ignition એ એક બાઈટકોડ ઇન્ટરપ્રિટર છે જે કોડને ઝડપથી રન કરે છે, જ્યારે TurboFan એક જસ્ટ-ઇન-ટાઇમ (JIT) ઓપ્ટિમાઇઝિંગ કમ્પાઇલર છે જે વારંવાર ચાલતા કોડને અલ્ટ્રા-ફાસ્ટ મશીન કોડમાં ટ્રેન્સે છે.
Zero-Width Characters Tricking	જાવાસ્ક્રિપ્ટ સ્ટ્રિંગમાં ન દેખાતા અક્ષરો (જેમ કે `'\u200B`) સંતાડી શકાય છે. આનો ઉપયોગ સિક્રેટ વોટરમાર્કિંગ કરવા અથવા કોડ વેરિએબલના નામો છુપાવવા માટે મેટાપ્રોગ્રામિંગમાં થઈ શકે છે.
Monomorphic vs Polymorphic IC	ઇનલાઇન કેશિંગ (Inline Caching) માં જો કોઈ ફંક્શન હંમેશા એક જ ટાઇપના ઓબ્જેક્ટ શેપ (Shape) પર રન થાય તો તેને મોનોમોર્ફિક કહેવાય જે સુપર ફાસ્ટ હોય છે. જો મલ્ટીપલ શેપ્સ પર રન થાય તો તે પોલીમોર્ફિક બને છે અને સ્પીડ થોડી ધીમી થાય છે.
Value Types vs Reference Types Storage	વેલ્યુ ટાઇપ્સ (Primitives) સીધા સ્ટેક (Stack) મેમરીના ફાસ્ટ ખાનામાં વેલ્યુ તરીકે સ્ટોર થાય છે, જ્યારે રેફરન્સ ટાઇપ્સ (Objects/Arrays) હીપ (Heap) મેમરીમાં સ્ટોર થાય છે અને સ્ટેક પાસે માત્ર તેની મેમરી લોકેશનનું એડ્રેસ (Pointer) હોય છે.
Isolate & Contexts in V8	V8 એન્જિનમાં Isolate એટલે એક આખું સ્વતંત્ર બ્રાઉઝર ટેબ અથવા રનટાઇમ ઇન્સ્ટન્સ જે પોતાની મેમરી હીપ ધરાવે છે. Context એટલે તે આઇસોલેટની અંદર રહેલું અલગ ઝ્વોબલ પર્ચાઇઝર (જેમ કે #frame).
Explicit Resource Management	ES2025 નો અલ્ટ્રા-મોડર્ન કીવર્ડ જે બ્લોક સ્કોપ પૂરો થતાં જ

('using')	કાઇલ લેન્ડલસે, ઓપન ડેટાબેઝ કનેક્શન્સ કે સોકેટ્સની મેમરીને '[Symbol.dispose]' મેથડ દ્વારા ઓટોમેટિક કલીન અને રીલીઝ કરી દે છે.
ArrayBuffer & DataView	બ્રાઉઝરમાં કાચો બાઈનરી ડેટા (Raw Binary Data) સ્ટોર કરવા માટેની ફિક્સ લંબાઈની મેમરી સ્પેસ એટલે ArrayBuffer. આ બાઈનરી ડેટાને અલગ-અલગ બ્લોક્સમાં (Int8, Float32) સેકલ્ડી વાંચવા અને લખવા માટે DataView API વપરાય છે.
Atomics API	જ્યારે મલ્ટીપલ લેબ વર્કર્સ (SharedArrayBuffer નો ઉપયોગ કરીને) એક જ મેમરી લોકેશન પર કામ કરતા હોય, ત્યારે ડેટા ઓવરરાઈટ કે કરાટ ન થાય તે માટે શેડ-સિક્યોર એટોમિક ઓપરેશન્સ (જેમ કે લોકીંગ અને વેટિંગ) કરવાની સુવિધા આ API આપે છે.

JavaScript Ultimate Advanced & Missing Concepts List

૧. Object Meta-Programming & Property Attributes

Term / Concept	સરળ ગુજરાતી સમજૂતી (Explanation)
Object.defineProperty()	કોઈ ઓબ્જેક્ટની અંદર નવી પ્રોપર્ટી ઉમેરવા અથવા જૂની પ્રોપર્ટીના નિયમો (જેમ કે તેને બદલી શકાય કે નહીં) કસ્ટમાઇઝ કરવા માટેની મેથડ.
writable	જો આ એટ્રિબ્યુટ false હોય, તો તે પ્રોપર્ટીની કિંમત (Value) ક્યારેય બદલી શકાતી નથી (તે Read-Only બની જાય છે).
enumerable	જો આ false હોય, તો તે પ્રોપર્ટી લૂપમાં (for...in અથવા Object.keys()) દેખાતી નથી, તે છુપી રહે છે.
configurable	જો આ false હોય, તો તે પ્રોપર્ટીને ઓબ્જેક્ટમાંથી ડિલીટ કરી શકાતી નથી અને તેના નિયમો ફરી બદલી શકાતા નથી.
Object.getOwnPropertyDescriptor()	કોઈ ચોક્કસ પ્રોપર્ટીના છુપા નિયમો (writable, enumerable, configurable) કયા સેટ કરેલા છે તે ચેક કરવા માટેનું ટૂલ.
Object.preventExtensions()	આ મેથડ વાપર્યો પછી ઓબ્જેક્ટમાં નવો કોઈ કી-વેલ્યુ (Property) ઉમેરી શકાતો નથી, પણ જૂની વેલ્યુ બદલી શકાય છે.
Object.seal()	ઓબ્જેક્ટને સીલ કરે છે. નવો ડેટા ઉમેરી શકાતો નથી, જૂનો ડેટા ડિલીટ કરી શકાતો નથી, માત્ર જૂની વેલ્યુ બદલી શકાય છે.
Object.freeze()	ઓબ્જેક્ટને બરફની જેમ જમાવી દે છે. નવો ડેટા ઉમેરવો, ડિલીટ કરવો કે જૂની વેલ્યુ બદલવી—બધું જ કાયમ માટે બંધ થઈ જાય છે.
Object.isFrozen() / isSealed()	ઓબ્જેક્ટ ફીઝ થયેલો છે કે સીલ થયેલો છે કે નહીં, તે જાણીને સાચો કે ખોટો (Boolean) જવાબ આપે છે.
Proxy Object	કોઈ ઓબ્જેક્ટની આગળ ઉભો રહેતો સિફ્ટ ગાર્ડ. ઓબ્જેક્ટનો ડેટા રીડ

	કે રાઇટ કરતી વખતે વચ્ચે કસ્ટમ લોજિક ચલાવવા વપરાય છે.
Reflect API	જાવાસ્ક્રિપ્ટના ઇન્ટરનલ ઓબ્જેક્ટ ઓપરેશન્સ (જેમ કે પ્રોપર્ટી સેટ કરવી કે ગેટ કરવી) ને ફંક્શન કોમેન્ટમાં સરળતાથી કરવાની આધુનિક રીત.
Property Descriptor	કોઈ પ્રોપર્ટીની વેલ્યુ સિવાય તેના આંતરિક ગુણધર્મોનું વર્ણન કરતો એક સિકેટ કન્ફિગરેશન ઓબ્જેક્ટ.

૨. Memory Management & Advanced Garbage Collection

Term / Concept	સરળ ગુજરાતી સમજૂતી (Explanation)
Memory Heap	જાવાસ્ક્રિપ્ટ એન્જિનમાં આવેલી એક અગોચર જગ્યા જ્યાં તમામ મોટા ઓબ્જેક્ટ્સ, ગેરે અને ફંક્શન્સની મેમરી એલોકેટ (સ્ટોર) થાય છે.
Call Stack Frame	જ્યારે કોઈ ફંક્શન રન થાય ત્યારે કોલ સ્ટેક પર તેની મેમરીનો એક બ્લોક બને છે, જેમાં તેના લોકલ વેરીએબલ્સ સ્ટોર થાય છે.
Garbage Collector	એક ઓટોમેટિક બેકગ્રાઉન્ડ પ્રોગ્રામ જે મેમરી પર નજર રાખે છે અને જે ડેટા હવે કોડમાં વપરાતો નથી તેને ડીલીટ કરીને મેમરી ખાલી કરે છે.
Mark-and-Sweep Algorithm	ગાર્બેજ કલેક્શનની મુખ્ય રીત. તે ઝડપી ઓબ્જેક્ટથી શરૂ કરીને તમામ કનેક્શનને માર્ક કરે છે, અને જે અનમાર્ક રહે તેને સ્વીપ (ડીલીટ) કરે છે.
Reference-Counting GC	જૂની ગાર્બેજ કલેક્શન પદ્ધતિ જે ગણતરી રાખે છે કે એક ડેટા સાથે કેટલા વેરીએબલ જોડાયેલા છે. જો કાઉન્ટ 0 થાય તો તે ડીલીટ થાય છે.
Circular Reference	જ્યારે બે ઓબ્જેક્ટ અંદરોઅંદર એકબીજાનો રેફરન્સ આપીને બેસી જાય, ત્યારે જૂની સિસ્ટમ તેને ડીલીટ નહોતી કરી શકતી (મેમરી લીક થતી).
Memory Leak	કોડની એવી ભૂલ જેમાં ડેટાનો ઉપયોગ પૂરો થઈ ગયો હોવા છતાં તે મેમરીમાંથી ડીલીટ થતો નથી, જેનાથી બાઉન્ડર પીમું થઈ જાય છે.

Shallow Copy	ઓબ્જેક્ટની ઉપરછલ્લી નકલ. જો ઓબ્જેક્ટની અંદર બીજો ઓબ્જેક્ટ હોય, તો તે નવો નથી બનાવતો પણ જૂના ઓબ્જેક્ટનો જ મેમરી એડ્રેસ વાપરે છે.
Deep Copy	ઓબ્જેક્ટની સો ટકા અસલી અને અલગ નકલ. તે અંદર રહેલા તમામ નેસ્ટેડ ઓબ્જેક્ટ્સને પણ મેમરીમાં સંપૂર્ણ નવા લોકેશન પર કોપી કરે છે.
Structured Clone API	જાવાસ્ક્રિપ્ટમાં કોઈપણ એક્સ્ટર્નલ લાઇબ્રેરી વગર નેટિવલી સુપર-ફાસ્ટ 'Deep Copy' કરવા માટેની નવી ઓફિશિયલ મેથડ ('structuredClone()').
WeakMap Object	એક ખાસ પુકારનો મેપ જેમાં 'કી' (Key) તરીકે માત્ર ઓબ્જેક્ટ્સ જ રાખી શકાય છે અને તે ગાર્બેજ કલેક્ટરને મેમરી કલીન કરવામાં રોકતો નથી.
WeakSet Object	માત્ર યુનિક ઓબ્જેક્ટ્સ સ્ટોર કરવાનું કલેક્શન. જો તે ઓબ્જેક્ટ મુખ્ય કોડમાંથી ડીલીટ થાય, તો વીકસેટમાંથી પણ આપમેળે ગાયબ થઈ જાય છે.

૩. V8 Engine Internals & Compiler Micro-Optimizations

Term / Concept	સરળ ગુજરાતી સમજૂતી (Explanation)
V8 Engine	ગૂગલ ક્રોમ અને Node.js માં વપરાતું અતિ શક્તિશાળી એન્જિન જે જાવાસ્ક્રિપ્ટના ટેક્સ્ટ કોડને કમ્પ્યુટર સમજી શકે તેવા મશીન કોડમાં ફેરવે છે.
JIT (Just-In-Time) Compilation	કોડને રન કરતા પહેલાં અલગથી કમ્પાઈલ કરવાને બદલે, કોડ રન થઈ રહ્યો હોય તે જ સમયે (Real-time) ફાસ્ટ કમ્પાઈલ કરવાની આધુનિક ટેકનિક.
Ignition Interpreter	V8 એન્જિનનો મુખ્ય ઇન્ટરપ્રેટર જે જાવાસ્ક્રિપ્ટ કોડ વાંચીને તેને તરત જ બાઇટકોડ (Bytecode) માં ફેરવે છે જેથી કોડ ફાસ્ટ રન થાય.

Turbofan Optimizer	V8 નો સિક્રેટ કમ્પાઈલર. જો કોઈ ફંક્શન વારંવાર રન થતું હોય (Hot Code), તો આ તેને પકડીને અલ્ટ્રા-ફાસ્ટ મશીન કોડમાં કન્વર્ટ કરી નાખે છે.
Deoptimization	જો ટર્બોફેન કોઈ ફંક્શનને ફાસ્ટ બનાવ્યા પછી યુઝર અચાનક તેનો ડેટા ટાઈપ બદલી નાખે, તો એન્જિન ગુસ્સે થઈને પાછું નોર્મલ સ્પીડ પર આવી જાય છે.
Abstract Syntax Tree (AST)	જ્યારે તમે કોડ લખો, ત્યારે બ્રાઉઝર સૌપ્રથમ તમારા કોડના એક-એક શબ્દને અલગ પાડીને એક આખું ટ્રી-સ્ટ્રક્ચર બનાવે છે, જેને AST કહેવાય.
Hidden Classes (Shapes)	જાવાસ્ક્રિપ્ટ ડાયનેમિક હોવા છતાં, V8 એન્જિન બેકગ્રાઉન્ડમાં ઓબ્જેક્ટ્સ માટે છુપી ફિક્સ ક્લાસ સિસ્ટમ બનાવે છે જેથી ડેટા એક્સેસ સ્પીડ વધી જાય.
Inline Caching (IC)	જો કોઈ ફંક્શન ઓબ્જેક્ટનો ડેટા વારંવાર વાંચતો હોય, તો આ ટેકનિક મેમરીમાં તે ડેટાનું લોકેશન યાદ રાખી લે છે જેથી બીજી વાર સર્ચ ન કરવું પડે.
Tree Shaking	વેબપેજ બિલ્ડ (પેક) થતી વખતે, કોડમાંથી એવા ફંક્શન્સ કે વેરિએબલ્સ જે આખા પ્રોજેક્ટમાં ક્યાંય વપરાયા જ નથી, તેને આપમેળે હટાવવાની પ્રોસેસ.
Dead Code Elimination	કમ્પાઈલર લેવલ પ્રોસેસ જે રન-ટાઇમ પર ક્યારેય એક્ઝિક્યુટ ન થવાનો હોય તેવા નકામા કોડ બ્લોક્સને ઓટોમેટિકલી ડીલીટ કરી નાખે છે.
Hot Code / Warm Code	જે ફંક્શન્સ પ્રોગ્રામ ચાલુ હોય ત્યારે વારંવાર હજારો વખત રન થતા હોય, તેને એન્જિન હોટ કોડ ગણીને સ્પેશિયલ ટ્રીટમેન્ટ આપે છે.

૪. Metaprogramming & Advanced ESNext Syntax Hooks

Term / Concept	સરળ ગુજરાતી સમજૂતી (Explanation)
----------------	----------------------------------

Decorator (@ syntax)	ક્લાસ, મેથડ કે પ્રોપર્ટીની ઉપર '@' લગાવીને તેના ઓરિજિનલ લોજિકને તોડવા વગર તેમાં એક્સ્ટ્રા ક્રીયર્સ ઉમેરવાનું નવું સુપર એડવાન્સ ક્રીયર.
Generator Function ("function")	એવું મેજિકલ ફંક્શન જેને અધવચ્ચે રોકી શકાય છે અને ભવિષ્યમાં ગમે ત્યારે અધૂરા કામેથી જ ફરીથી શરૂ (Resume) કરી શકાય છે.
Yield Keyword	જેમ નોર્મલ ફંક્શનમાં 'return' હોય છે, તેમ જનરેટર ફંક્શનમાં 'yield' વપરાય છે, જે વેલ્યુ બહાર મોકલીને ફંક્શનને ત્યાં જ પોઝ (Pause) કરી દે છે.
Iterator protocol	એક સત્તાવાર નિયમ જે નક્કી કરે છે કે કોઈ ઓબ્જેક્ટ પાસે 'next()' મેથડ હોવી જોઈએ જે દર વખતે '{value, done}' ટ્યુપલમાં ડેટા આપે.
Well-Known Symbols	જાવાસ્ક્રિપ્ટના બિલ્ડ-ઇન છુપા સિમ્બોલ્સ (જેમ કે 'Symbol.iterator'), જેનો ઉપયોગ કરીને તમે ઓબ્જેક્ટના ડિફાલ્ટ લોજિકને બદલી શકો છો.
Symbol.iterator	કોઈપણ સામાન્ય ઓબ્જેક્ટને લૂપમાં (for...of) ફેરવી શકાય તેવો પાવરફુલ બનાવવા માટે વપરાતો સ્પેશિયલ ઇન્ટરનલ સિમ્બોલ હૂક.
Symbol.species	જ્યારે તમે ચેરે કે ઓબ્જેક્ટ ક્લોન કરો, ત્યારે તે કયા કન્સ્ટ્રક્ટર ક્લાસનો ઉપયોગ કરીને નવો ઓબ્જેક્ટ બનાવશે તે કન્ટ્રોલ કરવાનો નિયમ.
Tagged Template Literals	બેકટિક્સ ('` ` ` `') ની આગળ સીધું ફંક્શનનું નામ લખીને, આખી સ્ટ્રિંગ અને તેની અંદરના વેરીએબલ્સને અલગ-અલગ પેરામીટર્સ તરીકે ફંક્શનમાં મોકલવાની રીત.
Dynamic Import ('import()')	વેબપેજ લોડ થતી વખતે જ બધી ફાઈલો ડાઉનલોડ કરવાને બદલે, કોડમાં જ્યારે જે ફાઇલની જરૂર પડે ત્યારે જ તેને લાઈવ (On-demand) ડાઉનલોડ કરવાની પદ્ધતિ.
Top-level await	કોઈપણ 'async' ફંક્શન બનાવ્યા વગર, ફાઈલના મેઈન સ્કોપમાં સીધો જ 'await' કીવર્ડ વાપરવાની મોડર્ન આગ્રહી.

૫. Enterprise Web Security & Browser Sandboxing

Term / Concept	સરળ ગુજરાતી સમજૂતી (Explanation)
Same-Origin Policy (SOP)	બ્રાઉઝરનો પાયાનો નિયમ જે સુરક્ષા માટે એક વેબસાઇટના ડેટા કે સ્ક્રિપ્ટને બીજી વેબસાઇટના ડેટા સાથે છેડછાડ કરતા રોકે છે.
CORS (Cross-Origin Resource Sharing)	સર્વર સાઇડ સિક્યોરિટી સેટિંગ. આના દ્વારા સર્વર નક્કી કરે છે કે કઈ બહારની વેબસાઇટ તેના ડેટા (API) ને એક્સેસ કરી શકે છે અને કઈ નહીં.
CORS Preflight Request ('OPTIONS')	જ્યારે બ્રાઉઝર કોઈ અધરી API રિક્વેસ્ટ મોકલે, ત્યારે તે પહેલા બેકગ્રાઉન્ડમાં એક નાની 'OPTIONS' રિક્વેસ્ટ મોકલીને સર્વરની પરમિશન ચેક કરે છે.
XSS (Cross-Site Scripting)	એક ખતરનાક હેકિંગ એટેક જેમાં હેકર્સ વેબપેજના ઇનપુટ બોક્સ દ્વારા પોતાની મલિન જાવાસ્ક્રિપ્ટ સ્ક્રિપ્ટ ધુસારીને યુઝર્સના સિક્રેટ ડેટા ચોરી લે છે.
CSRF (Cross-Site Request Forgery)	એવો એટેક જેમાં યુઝર અજાણતા જ કોઈ ખોટી લિંક પર ક્લિક કરે છે અને તેના લોગ-ઇન સેશનનો ફાયદો ઉઠાવીને હેકર્સ બેકગ્રાઉન્ડમાં ખોટા ટ્રાન્ઝેક્શન કરાવી લે છે.
Content Security Policy (CSP)	એક સ્પેશિયલ સિક્યોરિટી હેડર જે બ્રાઉઝરને કડક સૂચના આપે છે કે આ વેબસાઇટ પર કઈ-કઈ જગ્યાએથી સ્ક્રિપ્ટો લોડ થઈ શકે છે, જેથી XSS એટેક રોકી શકાય.
HttpOnly Cookies	એવી સિક્યોર ફૂફીઝ જેને જાવાસ્ક્રિપ્ટના કોડ ('document.cookie') દ્વારા ક્યારેય વાંચી શકાતી નથી, જેથી હેકર્સ તમારા લોગ-ઇન ટોકન્સ ચોરી ન શકે.
Subresource Integrity (SRI)	જ્યારે તમે બહારની કોઈ CDN લિંક (જેમ કે Bootstrap) વાપરો, ત્યારે આ ફીચર સ્ક્રિપ્ટનો હેશ કોડ ચેક કરે છે કે ફાઇલમાં કોઈએ છેડછાડ તો નથી કરી ને!
Secure Contexts ('window.isSecureContext')	બ્રાઉઝરની સુરક્ષા સિસ્ટમ જે અમુક પાવરફુલ ફીચર્સ (જેમ કે કેમેરા, જીઓલોકેશન, સર્વિસ વર્કર્સ) ને માત્ર સુરક્ષિત HTTPS વેબસાઇટ્સ પર

જ ચાલવાની મંજૂરી આપે છે.

JavaScript A to Z Master Dictionary

દરેક કન્સેપ્ટ અને ટર્મ્સનું ૧૦૦% સંપૂર્ણ ગુજરાતીમાં સરળ એક્સપ્લેનેશન

ક્રમ	JavaScript Term	સરળ ગુજરાતી સમજૂતી (Simple Explanation)
1	AJAX	વેબ પેજને રીફ્રેશ કર્યા વગર સર્વર પરથી ડેટા લાવવા કે મોકલવા માટેની ટેકનિક.
2	Algorithm	કોઈ પ્રોબ્લેમ સમસ્યા ઉકેલવા માટે સ્ટેપ-બાય-સ્ટેપ અનુસરવામાં આવતી પદ્ધતિ.
3	Argument	જ્યારે કોઈ ફંક્શન કોલ કરવામાં આવે ત્યારે તેને અંદર અપાતી વાસ્તવિક વેલ્યુ.
4	Array	એક જ વેરીએબલ નામ હેઠળ એક કરતાં વધુ કિંમતો (ડેટા) સ્ટોર કરવાનું કલેક્શન.
5	Array Methods	એરે પર કામ કરવા બિલ્ટ-ઇન ફંક્શન્સ જેવા કે push(), pop(), map(), filter(), reduce().
6	Arrow Function	ES6 માં આવેલી ટૂંકી અને આધુનિક રીતે ફંક્શન લખવાની સિન્ટેક્સ (() => {}).
7	Asynchronous Programming	કોડ રન કરવાની મોડલ જે મુખ્ય પ્રોસેસને અટકાવ્યા (block કર્યા) વગર બેકગ્રાઉન્ડમાં કામ કરે છે.

8	Assignment Operator	વેરીએબલમાં વેલ્યુ અસાઇન કરવા માટે વપરાતા ચિહ્નો (જેમ કે =, +=, -=).
9	Async/Await	પ્રોમિસ (Promises) ની ઉપર બનેલી સિન્ટેક્સ જે અસિંક્રોનસ કોડને એકદમ સરળતાથી લખવા દે છે.
10	Attributes	HTML એલિમેન્ટ્સની વધારાની માહિતી અથવા પ્રોપર્ટીઝ જેને JavaScript DOM વડે બદલી શકાય છે.
11	Babel	આધુનિક ES6+ કોડને જૂના બ્રાઉઝર્સ સમજી શકે તેવા કોડમાં ટ્રાન્સફર્મ/ટ્રાન્સપાઇલર.
12	Backticks	ટેમ્પલેટ લિટરલ્સ (ટિંટ્ડેઝ ઇન્ટરપોલેશન) લખવા માટે વપરાતું માસ ચિહ્ન (` `).
13	BigInt	સામાન્ય સુરક્ષિત લિમિટ કરતાં ખૂબ જ મોટી પૂર્ણાંક સંખ્યાઓ દર્શાવવા માટેનો ડેટા ટાઇપ.
14	Binary Operators	એવા ઓપરેટર્સ જેમને કામ કરવા માટે બે વેલ્યુ (Operands) ની જરૂર પડે છે (દા.ત. +, -).
15	Bitwise Operators	ડેટા પર બાઇનરી લેવેલે (0 અને 1 ના સ્તરે) ઓપરેશન કરતા ચિહ્નો (જેમ કે &, , ^).
16	Block	બ્લોક બ્રેકેટ {} ની અંદર લખાયેલો કોડ જે લોજિકલ જૂથ બનાવે છે.
17	Block Scope	let અને const દ્વારા ડિક્લેર કરેલા વેરીએબલ જે માત્ર તે જ બ્લેકેટ {} ની અંદર એક્સેસ થાય છે.

18	Boolean	એવો ડેટા પ્રકાર જેની માત્ર બે જ વેલ્યુ હોઈ શકે: સાચું (true) અથવા ખોટું (false).
19	Break Statement	ચાલુ લૂપ અથવા સ્વિચ કેસને અધવચ્ચેથી તોડીને સીધા બહાર નીકળી જવાનો આદેશ.
20	Browser Object Model (BOM)	બ્રાઉઝરના ભાગો સાથે વાતચીત કરવાનો ઓબ્જેક્ટ મોડલ (જેમ કે window, navigator, history).
21	Bubbling (Event Bubbling)	ઇવેન્ટ પ્રસરણનો નિયમ જ્યાં ઇવેન્ટ સૌથી અંદરના એલિમેન્ટથી શરૂ થઈને ઉપરના પેરેન્ટ સુધી જાય છે.
22	Call Stack	જાવાસ્ક્રિપ્ટ એન્જિન કયું ફંક્શન અત્યારે રન થઈ રહ્યું છે તેનો ટ્રેક રાખવા માટેનું મેકેનિઝમ (LIFO).
23	Callback Function	એક ફંક્શન જે બીજા ફંક્શનમાં આર્ગ્યુમેન્ટ તરીકે અપાય છે જેથી તે પછીથી રન થઈ શકે.
24	Callback Hell	એકની અંદર એક એમ ઘણા બધા નેસ્ટેડ કોલબેક લખવાથી બનતો અધરો પિરામિડ જેવો કોડ.
25	CamelCase	વેરીએબલ નામ લખવાની રીત જ્યાં પહેલા શબ્દ સિવાયના દરેક શબ્દનો પહેલો અક્ષર કેપિટલ હોય (myVariable).
26	Capturing (Event Capturing)	ઇવેન્ટ પ્રસરણ જ્યાં ઇવેન્ટ સૌથી બહારના પેરેન્ટથી શરૂ થઈને ટાર્ગેટ એલિમેન્ટ સુધી નીચે જાય છે.
27	Catch Block	try...catch નો એ ભાગ જે try માં આવેલી એરર કે ભૂલોને પકડીને હેન્ડલ કરે છે.
28	Class	ઓબ્જેક્ટ બનાવવા માટેનું એક બ્લુપ્રિન્ટ અથવા માળખું જે ES6

		માં પ્રોટોટાઇપની ઉપર લાવવામાં આવ્યું.
29	Closure	એવું ફંક્શન જે પોતાનું કામ પૂરું થયા પછી પણ તેના બહારના સ્કોપના વેરીએબલ્સને યાદ રાખે છે.
30	Coercion (Type Coercion)	જાવાસ્ક્રિપ્ટ દ્વારા બેકગ્રાઉન્ડમાં આપમેળે એક ડેટા ટાઇપમાંથી બીજામાં થતું રૂપાંતરણ.
31	Collection	ડેટા આઇટમ્સને સંગ્રહિત કરવાનું સ્ટ્રક્ચર્ડ ગ્રૂપ (જેમ કે Arrays, Maps, Sets).
32	Comparison Operators	બે વેલ્યુઝની સરખામણી કરવા માટે વપરાતા ઓપરેટર્સ (જેમ કે ==, ===, !=, >, <).
33	Concurrency	એક જ સમયગાળામાં એક કરતાં વધુ કાર્યોને હેન્ડલ કરવાની ક્ષમતા (Event Loop દ્વારા સંચાલિત).
34	Console	બ્રાઉઝર કે નોડ એન્જિનનું ડીબગિંગ ટૂલ જેના પર આપણે કોડ પ્રિન્ટ કરીને ચેક કરીએ છીએ (console.log).
35	Const	બ્લોક સ્કોપ વેરીએબલ બનાવવાનો કીવર્ડ જેની વેલ્યુ એકવાર આપ્યા પછી બદલી શકાતી નથી.
36	Constructor	ક્લાસની અંદર રહેલી એક ખાસ મેથડ જે ઓબ્જેક્ટ બનાવતી વખતે ઓટોમેટિક રન થાય છે.
37	Context (Execution Context)	એવું વાતાવરણ જેમાં જાવાસ્ક્રિપ્ટનો કોડ મૂલ્યાંકન અને રન થાય છે.
38	Continue Statement	લૂપની અંદર ચાલુ ઈટરેશન સ્કીપ (છોડી) કરીને તરત જ આગલી ઈટરેશન પર જવાનો આદેશ.

39	Currying	મલ્ટીપલ આર્ગ્યુમેન્ટ્સ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ વાળા ક્રમિક ફંક્શનમાં ફેરવવાની ટેકનિક.
40	Data Types	ડેટાનું વર્ગીકરણ (Primitives: String, Number, Boolean, Null, Undefined વગેરે અને Objects).
41	Debounce	કોઈ ફંક્શન વારંવાર ટ્રિગર થતું રોકવા માટે થ્રુઝર ટાઇપિંગ કે ક્લિક કરવાનું બંધ કરે ત્યાં સુધી થોભવાની રીત.
42	Debugger	કોડને રન થતો અટકાવીને સ્ટેપ-બાય-સ્ટેપ લાઇન ચેક કરવાનું ટૂલ કે કીવર્ડ.
43	Declaration	var, let, const કે function કીવર્ડ વાપરીને મેમરીમાં વેરીએબલ કે ફંક્શનનું નામ નોંધાવવું.
44	Deep Copy	ઓબ્જેક્ટ કે એરેની અંદરના તમામ નેસ્ટેડ સ્ટ્રક્ચર સહિતની સંપૂર્ણ આખી નવી અલગ કોપી બનાવવી.
45	Default Parameters	ફંક્શનમાં જો કોઈ આર્ગ્યુમેન્ટ પાસ ન કરવામાં આવે તો આપમેળે લેવાતી ડિફોલ્ટ વેલ્યુ.
46	Defer Attribute	HTML script ટેગનું એટ્રિબ્યુટ જે જાવાસ્ક્રિપ્ટને આખું પેજ લોડ થયા પછી જ રન થવાની સૂચના આપે છે.
47	Delete Operator	ઓબ્જેક્ટમાંથી કોઈ ચોક્કસ પ્રોપર્ટી (કી અને વેલ્યુ) ને કાયમ માટે ડીલીટ કરવાનું ચિહ્ન.
48	Destructuring Assignment	એરે અથવા ઓબ્જેક્ટમાંથી ડેટાને ખૂબ જ ટૂંકી રીતે અલગ-અલગ વેરીએબલ્સમાં બહાર કાઢવાની રીત.

49	Document Object Model (DOM)	HTML પેજનું એક ઓબ્જેક્ટ ટ્રી સ્ટ્રક્ચર જે જાવાસ્ક્રિપ્ટને પેજ બદલવાની મંજૂરી આપે છે.
50	DOM Manipulation	જાવાસ્ક્રિપ્ટનો ઉપયોગ કરીને લાઈવ HTML એલિમેન્ટ્સ, કલર, ક્લાસ કે ટેક્સ્ટ બદલવાની પ્રોસેસ.
51	Do-While Loop	એવું લૂપ જે કન્ડિશન સાચી હોય કે ખોટી, કોડને ઓછામાં ઓછી એકવાર તો રન કરે જ છે.
52	Dynamic Typing	જાવાસ્ક્રિપ્ટની વિશેષતા જ્યાં વેરીએબલનો ટાઇપ પહેલેથી નક્કી નથી હોતો પણ વેલ્યુ મુજબ રન ટાઇમે નક્કી થાય છે.
53	ECMAScript	જાવાસ્ક્રિપ્ટ ભાષાના નિયમો નક્કી કરતી સત્તાવાર સ્પેસિફિકેશન અથવા સ્ટાન્ડર્ડ.
54	ES6 / ES2015	જાવાસ્ક્રિપ્ટનો અત્યાર સુધીનો સૌથી મોટો અપડેટ જેમાં ક્લાસ, એરો ફંક્શન, લેટ અને કોન્ટ આદ્યા.
55	Element	HTML ટેગનું DOM ની અંદર બનેલું ઓબ્જેક્ટ સ્વરૂપ.
56	Else Statement	જો if કન્ડિશન ખોટી પડે, તો ઓટોમેટિક રન થતો કોડ બ્લોક.
57	Encapsulation	ડેટા અને ફંક્શનને એક જ ક્લાસ કે ક્લોઝરમાં પેક કરીને બહારથી ડાયરેક્ટ એક્સેસ ન કરવા દેવાની પદ્ધતિ.
58	Engine (V8, SpiderMonkey)	જાવાસ્ક્રિપ્ટના ટેક્સ્ટ કોડને મશીન કોડમાં ફેરવી રન કરતું મુખ્ય સોફ્ટવેર (Chrome માં V8 છે).
59	Equality (Strict vs Loose)	=== ટાઇપ અને વેલ્યુ બંને ચેક કરે છે, જ્યારે == ટાઇપ બદલીને માત્ર વેલ્યુ ચેક કરે છે.

60	Error Object	જ્યારે કોડમાં કોઈ ભૂલ આવે ત્યારે બ્રાઉઝર દ્વારા જનરેટ થતી માહિતી સભર ઓબ્જેક્ટ.
61	ES Modules (ESM)	import અને export સ્ટેટમેન્ટ્સ વાપરીને કોડને મલ્ટીપલ ફાઇલોમાં મેનેજ કરવાની ઓફિશિયલ રીત.
62	Eval	રિટર્ન તરીકે લખેલા કોડને જાવાસ્ક્રિપ્ટ લોજિક તરીકે રન કરતું ફંક્શન (જે સિક્યોરિટી માટે જોખમી છે).
63	Event	બ્રાઉઝરમાં યુઝર દ્વારા થતી કોઈ એક્શન જેમ કે માઉસ ક્લિક, સ્ક્રોલ, કીબોર્ડ પ્રેસ વગેરે.
64	Event Handler	કોઈ ચોક્કસ ઇવેન્ટ થાય ત્યારે તેના જવાબમાં રન થતું ફંક્શન.
65	Event Loop	અસિંક્રોનસ કોડ (કોલબેકસ) ને ક્રમમાંથી ઉપાડીને કોલ સ્ટેક ખાલી થાય ત્યારે ત્યાં રન કરવા મોકલતું ચક્ર.
66	Event Listener	HTML એલિમેન્ટ પર ઇવેન્ટ પર નજર રાખવા માટે વપરાતી મેથડ (addEventListener).
67	Exception	કોડ રન થતી વખતે આવતી અનપેક્ષિત ભૂલ જે પ્રોગ્રામના ક્રદરતી વહેણને તોડે છે.
68	Execution Context	જાવાસ્ક્રિપ્ટનો કોડ કઈ રીતે રન થઈ રહ્યો છે તેનું આખું વાતાવરણ (Environment).
69	Expression	કોડનો કોઈ પણ એવો ટુકડો જે છેલ્લે કોઈ સિંગલ વેલ્યુમાં રૂપાંતરિત થાય છે.
70	Fetch API	સર્વરમાંથી કે API માંથી નેટવર્ક રિસ્પેસ્ટ દ્વારા ડેટા લાવવા માટેનું

		આધુનિક પ્રોમિસ-બેઝ્ડ ટૂલ.
71	Filter Method	એરેની બિલ્ટ-ઇન મેથડ જે કન્ડિશન પાસ કરતા એલિમેન્ટ્સનો નવો એરે બનાવી આપે છે.
72	Finally Block	try...catch નો એ છેલ્લો બ્લોક જે એરર આવે કે ન આવે, દરેક પરિસ્થિતિમાં રન થાય જ છે.
73	First-Class Functions	કન્સેપ્ટ જ્યાં ફંક્શનને વેરીએબલની જેમ જ ગણવામાં આવે છે (પાસ કરી શકાય, રીટર્ન કરી શકાય).
74	For Loop	ચોક્કસ સંખ્યામાં કોડને વારંવાર રીપીટ કરવા માટેનું સ્ટાન્ડર્ડ લૂપ.
75	For...in Loop	ઓબ્જેક્ટની બધી કી (પ્રોપર્ટીઝ) પર એક પછી એક ફરીને ઇટરેટ કરવા માટેનું લૂપ.
76	For...of Loop	એરે, રિટર્ન કે સેટ જેવા ઇટરેબલ ઓબ્જેક્ટ્સની સીધી વેલ્યુઝ કાઢવા માટેનું લૂપ.
77	Function	વારંવાર વાપરી શકાય તેવો કોડનો એક વિશિષ્ટ બ્લોક જે ચોક્કસ કાર્ય કરે છે.
78	Garbage Collection	મેમરી મેનેજમેન્ટ જે ન વપરાતા ઓબ્જેક્ટ્સને આપમેળે શીધીને રેમ (RAM) મેમરી ખાલી કરે છે.
79	Generator Function	function* દ્વારા લખાતું ખાસ ફંક્શન જે વચ્ચેથી અટકી શકે છે અને ફરી ત્યાંથી ચાલુ થઈ શકે છે.
80	Global Scope	સૌથી બહારનો ઓપન સ્કોપ જેમાં રહેલા વેરીએબલ આખા કોડમાં ગમે ત્યાંથી એક્સેસ થઈ શકે છે.

81	Higher-Order Function	એવું ફંક્શન જે બીજા ફંક્શનને આર્ગ્યુમેન્ટમાં લે છે અથવા બદલામાં નવું ફંક્શન રીટર્ન કરે છે.
82	Hoisting	જાવાસ્ક્રિપ્ટનું બિહેવિયર જ્યાં વેરીએબલ અને ફંક્શન ડિફિનેશન કોડ રન થતા પહેલાં સ્કોપમાં ઉપર જતા રહે છે.
83	IIFE	એવું ફંક્શન જે બનતાની સાથે જ તરત જ ઓટોમેટિક રન થઈ જાય છે (Immediately Invoked).
84	Immutability	ડેટા ઓબ્જેક્ટ બનાવ્યા પછી તેમાં કોઈ ફેરફાર ન કરી શકવાની અભિગમ.
85	JSON	સર્વર અને વેબ એપ્લિકેશન વચ્ચે ટેક્સ્ટ ફોર્મેટમાં ડેટા ટ્રાન્સફર કરવા માટેનું લાઇટવેઇટ માધ્યમ.
86	JSON.parse()	JSON ફોર્મેટની ટેક્સ્ટ સ્ટ્રક્ચરવાળી સ્ટ્રિંગને જાવાસ્ક્રિપ્ટ ઓબ્જેક્ટમાં ફેરવવાની મેથડ.
87	JSON.stringify()	જાવાસ્ક્રિપ્ટના ઓબ્જેક્ટને સર્વર પર મોકલવા માટે JSON સ્ટ્રિંગમાં કન્વર્ટ કરતી મેથડ.
88	JIT Compilation	જાવાસ્ક્રિપ્ટ એન્જિન કોડ રન થતો હોય તે જ સમયે (Runtime) તેને ફાસ્ટ મશીન કોડમાં કમ્પાઇલ કરે છે તે પ્રક્રિયા.
89	Let	બ્લોક સ્કોપ વેરીએબલ બનાવવાનો કીવર્ડ જેની કિંમત પાછળથી બદલી શકાય છે.
90	Lexical Scope	કોડ કઈ જગ્યાએ ભૌતિક રીતે લખાયો છે તેના આધારે વેરીએબલનો સ્કોપ નક્કી થવાની પદ્ધતિ.
91	Local Storage	બ્રાઉઝરમાં કાયમી ધોરણે ડેટા સ્ટોર કરવાની જગ્યા જે બ્રાઉઝર

92	Map Object	કો-વેલ્યુ જોડી રાખવાનું કલેક્શન જેમાં કી તરીકે ઓબ્જેક્ટ કે ફંક્શન પણ રાખી શકાય છે.
93	Memory Leak	પ્રોગ્રામની ખામી જ્યાં નકામો ડેટા મેમરી રોકી રાખે છે અને મેમરી ખાલી થતી નથી જેથી સિસ્ટમ સ્લો થાય છે.
94	Method	કોઈ ઓબ્જેક્ટની અંદર પ્રોપર્ટી તરીકે સ્ટોર કરેલા ફંક્શનને મેથડ કહેવાય છે.
95	NaN (Not-a-Number)	જાવાસ્ક્રિપ્ટની અલ્ટીમેટ ન્યુમેરિક વેલ્યુ જે ગણિતની અમાન્ય ગણતરી વખતે રીટર્ન થાય છે.
96	Null	કોઈ વેલ્યુ જાણીજોઈને ખાલી (intentional absence of object) રાખવા માટેનો પ્રિમિટિવ ટાઇપ.
97	Nullish Coalescing (??)	જો કાબી બાજુની વેલ્યુ null કે undefined હોય તો જ જમણી બાજુની ડિફોલ્ટ વેલ્યુ આપતો ઓપરેટર.
98	Optional Chaining (?.)	ઓબ્જેક્ટની ઊંડી પ્રોપર્ટી ચેક કરતી વખતે જો વચ્ચે કોઈ પ્રોપર્ટી ગાયબ હોય તો એરર આપવાને બદલે undefined આપે છે.
99	Polyfill	આધુનિક બ્રાઉઝરના નવા ફીચર્સ જૂના બ્રાઉઝર્સમાં ચલાવવા માટે જાવાસ્ક્રિપ્ટ વડે બેકગ્રાઉન્ડમાં લખાતો કસ્ટમ કોડ.
100	Promise	અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે (સફળ કે અસફળ) તેની ગેરંટી આપતું ઓબ્જેક્ટ.
101	Prototypal Inheritance	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટના ગુણો અને મેથડ્સ વારસામાં

મેળવે છે તે મિકેનિઝમ.

102	Pure Function	એવું ફંક્શન જે સમાન ઇનપુટ આપતા હંમેશા સમાન આઉટપુટ આપે છે અને કોઈ સાઇડ ઇફેક્ટ કરતું નથી.
103	Rest Parameters (...)	ફંક્શનની આર્ગ્યુમેન્ટમાં આવતી અસંખ્ય છૂટી વેલ્યુઝને ભેગી કરીને એક એરે બનાવવાની મોર્ડન સિન્ટેક્સ.
104	Scope Chain	જ્યારે કરન્ટ સ્કોપમાં વેરીએબલ ન મળે ત્યારે એન્જિન બહારના સ્કોપમાં લાઇનસર શોધે છે તે સાંકળ.
105	Shallow Copy	ઓબ્જેક્ટની એવી કોપી જે માત્ર ઉપરના લેવલનો ડેટા નવો બનાવે છે પણ અંદરના નેસ્ટેડ ઓબ્જેક્ટના રેફરન્સ સેમ રાખે છે.
106	Short-Circuit Evaluation	લોજિકલ ઓપરેટર્સ (&&,) માં પહેલી કન્ડિશન પરથી જ આખો જવાબ નક્કી થઈ જાય તો આગળનો કોડ રન ન થવાની રીત.
107	Single Threaded	એક સમયે માત્ર એક જ લાઇન કે કમાન્ડ રન કરવાની જાવાસ્ક્રિપ્ટ એન્જિનની પ્રકૃતિ.
108	Spread Operator (...)	એરે કે ઓબ્જેક્ટના તમામ એલિમેન્ટ્સને ખોલીને છૂટાછવાયા ડેટામાં ફેરવવાની સિન્ટેક્સ.
109	Strict Mode	'use strict' ડાયરેક્ટિવ જે કોડની નાની ભૂલોને એરર તરીકે પકડી પાડે છે અને કોડ સુરક્ષિત બનાવે છે.
110	Symbol	ઓબ્જેક્ટની સિક્રેટ અને યુનિક કી બનાવવા માટે વપરાતો અજોડ પ્રિમિટિવ ડેટા ટાઇપ.
111	Template Literals	બેકટિક્સ (` `) ની અંદર \${ } વાપરીને મલ્ટી-લાઇન સ્ટ્રિંગ અને

વેરીએબલ્સ સરળતાથી લખવાની પદ્ધતિ.

112	Ternary Operator	એક જ લાઇનમાં if-else લખવા માટેનો ત્રણ ઓપરેન્ડ વાળો ઓપરેટર (condition ? true : false).
113	This Keyword	કરન્ટ કન્ટેક્ટ અથવા ફંક્શન અત્યારે જે ઓબ્જેક્ટના અંડરમાં રન થઈ રહ્યું છે તેનો રેફરન્સ.
114	Throttling	કોઈ ઇવેન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ ફંક્શનને ચોક્કસ સમયના અંતરે જ રન થવા દેવાની વિમિટિંગ પદ્ધતિ.
115	Try...Catch Block	કોડમાં આવતી લાઇવ એરર્સને હેન્ડલ કરવાનો સ્ટ્રક્ચર જેથી આખો પ્રોગ્રામ કેશ ન થઈ જાય.
116	Undefined	જાવાસ્ક્રિપ્ટ દ્વારા વેરીએબલ ડિફાલ્ટ ક્યો હોય પણ કિંમત ન આપી હોય ત્યારે મળતી ડિફોલ્ટ વેલ્યુ.
117	Virtual DOM	ખરેખર HTML DOM ની મેમરીમાં બનેલી હળવી નકલ જે મોર્ડન ફ્રેમવર્ક સ્પીડ ઓપ્ટિમાઇઝેશન માટે વાપરે છે.
118	WeakMap	મેપ જેવું જ કલેક્શન જેમાં કી માત્ર ઓબ્જેક્ટ જ હોઈ શકે છે અને તે ગાર્બેજ કલેક્ટર ફ્રેન્ડલી હોય છે.
119	Window Object	બ્રાઉઝરનો સૌથી મોટો ગ્લોબલ મેઇન ઓબ્જેક્ટ જેની અંદર આખી જાવાસ્ક્રિપ્ટ રન થાય છે.
120	Zero-based Indexing	જાવાસ્ક્રિપ્ટમાં એરેના પહેલા એલિમેન્ટનું સ્થાન (index) હંમેશા 0 થી જ શરૂ થવાનો નિયમ.

JavaScript 1000+ Unique Terms & Concepts Mega Master List

સંપૂર્ણ જાવાસ્ક્રિપ્ટનું A to Z ડિક્શનરી અને અલ્ટીમેટ ગાઇડ (ગુજરાતીમાં)

Core Language, Basics & Foundations (ક્રમ ૧ થી ૧૨૦)

મૂળભૂત વેરીએબલ્સ, ડેટા ટાઇપ્સ, લૂપ્સ, અને મૂળભૂત ઓપરેટર્સ.

ક્રમ	JavaScript Concept / Term	સરળ ગુજરાતી સમજૂતી (Detailed Explanation)
1	var Keyword	ગ્લોબલ અથવા ફંક્શન સ્કોપ વેરીએબલ ડિક્લેર કરવાની જાહેરાત.
2	let Keyword	બ્લોક સ્કોપ વેરીએબલ ડિક્લેર કરવાની મોડર્ન રીત જે હોઇસ્ટિંગ થતી.
3	const Keyword	બ્લોક સ્કોપ વેરીએબલ જેની વેલ્યુ એકવાર આપ્યા પછી બદલવી નથી શકાય.
4	Primitive Types	ડેટા ટાઇપ્સ જેમ કે String, Number, Boolean જે સીધા સ્ટોર થાય છે.
5	Reference Types	ડેટા ટાઇપ્સ જેમ કે Objects અને Arrays જે મેમરી હીપમાં સ્ટોર થાય છે.
6	Undefined	જ્યારે વેરીએબલ ડિક્લેર કર્યો હોય પણ તેમાં કોઈ વેલ્યુ ન અસાથે હોય.
7	Null	ઇરાદાપૂર્વક વેરીએબલને ખાલી (ખાલી ઓબ્જેક્ટ રેફરન્સ) વેલ્યુ આપવાનું.
8	Boolean	માત્ર True અથવા False કન્ડિશન ચેક કરવા માટે વપરાતો ટાઇપ.
9	String Literal	સિંગલ કે ડબલ કોટ્સમાં લખવામાં આવતી ટેક્સ્ટ વેલ્યુ.
10	Template Literals	બેકટિક્સ (``) નો ઉપયોગ કરીને સ્ટ્રિંગમાં વેરીએબલ ઇમેરેડ કરવાનું.
11	Number Type	બધા જ પૂર્ણાંક અને અપૂર્ણાંક અંકો માટેનો સિંગલ ડેટા ટાઇપ.

12	BigInt	ખૂબ જ મોટા અંકો (જે સામાન્ય Number લિમિટની બહાર સ્ટોર કરવાની સુવિધા).
13	Symbol	દરેક વખતે એકદમ યુનિક અને પ્રાઇવેટ કી (Key) જનરેટ કરવા માટેનો ટાઇપ.
14	Dynamic Typing	જાવાસ્ક્રિપ્ટમાં વેરીએબલનો ટાઇપ રન-ટાઇમ પર આપમેળે થવાની પ્રક્રિયા.
15	Weak Typing	ઓપરેશન કરતી વખતે ડેટા ટાઇપ આપમેળે કન્વર્ટ થવાની ફ્લેક્સિબિલિટી.
16	Arithmetic Operators	ગાણિતિક ગણતરીઓ જેમ કે પ્લસ, માઇનસ, ગુણાકાર અને ભાગાકાર.
17	Assignment Operators	વેરીએબલમાં વેલ્યુ અસાઇન કરવાની નિશાનીઓ (જેમ કે =, +=, -=).
18	Comparison Operators	બે વેલ્યુ સરખાવવા માટે વપરાતી સાઇન જેમ કે '>', '<', '>=', '<='.
19	Strict Equality (===)	વેલ્યુ અને ડેટા ટાઇપ બંને એકસાથે પાડ્યું ચેક કરવાની રીત.
20	Loose Equality (==)	માત્ર વેલ્યુ ચેક કરે છે, ડેટા ટાઇપ આપમેળે બદલી નાખે છે.
21	Logical AND (&&)	જ્યારે બંને કન્ડિશન્સ સાચી હોય ત્યારે જ જવાબ True આપે છે.
22	Logical OR ()	બેમાંથી કોઈ પણ એક કન્ડિશન સાચી હોય તો જવાબ True આપે છે.
23	Logical NOT (!)	સાચી કન્ડિશનને ખોટી અને ખોટી કન્ડિશનને સાચી (ઊંધું) બનાવે છે.
24	Bitwise AND	બાઇનરી લેવલ પર (0 અને 1) એન્ડ ઓપરેશન કરવાની પદ્ધતિ.
25	Bitwise OR	બાઇનરી લેવલ પર ઓર ઓપરેશન કરીને વેલ્યુ ચેક કરવાની પદ્ધતિ.
26	Short-Circuit Evaluation	જો પ્રથમ કન્ડિશન પસંદ થઈ જાય તો બાકીની કન્ડિશન ચેક કરવાની જરૂર નથી.
27	Nullish Coalescing (??)	જો ડાબી બાજુ 'null' કે 'undefined' હોય તો જ જમણી બાજુની વેલ્યુ લેવાની.

28	Optional Chaining (?.)	ઓબ્જેક્ટની પ્રોપર્ટી ચેક કરતી વખતે એરર આવ્યા વગર રેટર્ન કરવું.
29	Ternary Operator	ઇફ-એલ્સ (if-else) ની શોર્ટહેન્ડ રીત જેમાં સિંગલ લાઈનમાં ચેક થાય છે.
30	Typeof Operator	કોઈ પણ વેરીએબલનો કરન્ટ ડેટા ટાઇપ કયો છે તે જાણવાનો ઓપરેટર.
31	CSRF	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ડિમાન્ડ મોકલવી.
32	Web Workers	ભારે કેમ્બ્રિજેશનના કોડને મેઈન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડમાં ચલાવવાની ટેકનિક.
33	Temporal API	જૂના Date ઓબ્જેક્ટને બદલે ટાઈમ-ઝોન ફિક્સ કરવું જેટલું API.
34	Explicit Resource Management	using કીવર્ડ દ્વારા સ્કોપ પૂરી થતાં જ મેમરી ઓટોમેટિક ક્લિયર થાય.
35	Generator Function	function* સિન્ટેક્સ વાળું ફંક્શન જેને વ્યૂથેબી રોકી અને ફરે શકાય.
36	Decorators	@ સિન્ટેક્સ વાળું અડજુટ ફીચર જે ક્લાસ કે મેથડની ક્ષમતા ઘટાડે છે.
37	Proxy Object	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન્સ (get, set) ને કન્ટ્રોલ કરતો સિંક્રેટ એજન્ટ.
38	Reflect API	ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી મેનિપ્યુલેટ કરવા માટેના ગ્લોબલ મેથડ્સ.
39	Variable Shadowing Variant (Concept Part #39)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલને ઓવરલેપ કરે. [ભાગ #39 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
40	Global Scope Variant (Concept Part #40)	આખા પ્રોગ્રામમાં ગ્લોબલ સ્કોપમાં ફંક્શન કરી શકાતો સૌથી બૉલ્ડ સ્કોપ. [ભાગ #40 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
41	Block Scope Variant (Concept Part #41)	માત્ર કરવા બેસીસ 'G' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #41 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

42	Function Scope Variant (Concept Part #42)	માત્ર જે-તે ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #42 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
43	Execution Context Variant (Concept Part #43)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #43 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
44	Call Stack Variant (Concept Part #44)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) ડાઈવ. [ભાગ #44 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
45	Hoisting Variant (Concept Part #45)	ડિફાલ્ટ રીલેશન કોડ રન થતા પહેલાં આપમેળે સ્કોપમાં સીધી ઉપર લેવાની પ્રોસેસ. [ભાગ #45 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
46	Temporal Dead Zone Variant (Concept Part #46)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો કોડ. [ભાગ #46 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
47	Closure Variant (Concept Part #47)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલને ફંક્શન પૂરું થયા પછી પણ યાદ રાખે તે. [ભાગ #47 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
48	IIFE Variant (Concept Part #48)	બનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવું ફંક્શન. [ભાગ #48 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
49	Currying Variant (Concept Part #49)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં ફેરવવું. [ભાગ #49 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
50	Memoization Variant (Concept Part #50)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવો. [ભાગ #50 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
51	Debouncing Variant (Concept Part #51)	યુઝર ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવન્ટ ફાયર થાય. [ભાગ #51 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
52	Throttling Variant (Concept Part #52)	કોઈ ઇવન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને ચોક્કસ સમયગાળામાં ફાયર થાય. [ભાગ #52 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
53	Callback Hell Variant (Concept Part #53)	એકની અંદર એક અસંખ્ય કોલબેક લેખવાથી બનતો અધર કોડ સ્ટ્રક્ચર. [ભાગ #53 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
54	Inversion of Control Variant (Concept Part #54)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને ખામી. [ભાગ #54 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

55	Promise Variant (Concept Part #55)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ગેરંટી આપતું એનિમેશન. [ભાગ #55 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
56	Event Loop Variant (Concept Part #56)	સિંગલ થ્રેડેડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચલાવવાનું વ્યવસ્થા. [ભાગ #56 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
57	Microtask Queue Variant (Concept Part #57)	પ્રોમિસિસના કોડ રાખતી યાદી જેને મેક્રોટાસ્ક કરતાં પહેલાં ચલાવવામાં આવે છે. [ભાગ #57 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
58	Macrotask Queue Variant (Concept Part #58)	setTimeout અને ઇવેન્ટુસનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ યાદી. [ભાગ #58 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
59	Starvation Variant (Concept Part #59)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે સ્થિતિ. [ભાગ #59 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
60	Prototypal Inheritance Variant (Concept Part #60)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #60 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
61	Prototype Chain Variant (Concept Part #61)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ લિંકમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #61 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
62	V8 Engine Variant (Concept Part #62)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #62 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
63	JIT Compilation Variant (Concept Part #63)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ કરવા. [ભાગ #63 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
64	Inline Caching Variant (Concept Part #64)	વારંવાર વપરાતા ઓબ્જેક્ટ લોકેશનને એન્જિન આપમેળે યાદી રાખી રાખે છે. [ભાગ #64 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
65	Hidden Classes Variant (Concept Part #65)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલી શ્રેણી. [ભાગ #65 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
66	Tree Shaking Variant (Concept Part #66)	બિન-પ્રયોજન વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમ કરવો. [ભાગ #66 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
67	CORS Policy Variant (Concept Part #67)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા દિવરેટ સિસ્ટમોની નીતિ. [ભાગ #67 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

68	XSS (Cross-Site Scripting) Variant (Concept Part #68)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીમીડિયા જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #68 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
69	CSRF Variant (Concept Part #69)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ઇન્ફોર્મેશન મોકલવો. [ભાગ #69 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
70	Web Workers Variant (Concept Part #70)	ભારે કેલ્ક્યુલેશનના કોડને મેઈન થ્રેડ રોક્ષા વગર બેકગ્રાઉન્ડમાં ચલાવવાની ટેકનિક. [ભાગ #70 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
71	Temporal API Variant (Concept Part #71)	જૂના Date ઓબ્જેક્ટને બદલે ટાઈમ-ઝોન ફિક્સ કરતું વેબ API. [ભાગ #71 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
72	Explicit Resource Management Variant (Concept Part #72)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર કરવો. [ભાગ #72 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
73	Generator Function Variant (Concept Part #73)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરી શરૂ કરી શકાય. [ભાગ #73 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
74	Decorators Variant (Concept Part #74)	@ સિન્ટેક્સ વાળું એડ્જસ્ટ ફોર જે ક્લાસ કે મેથડની ક્ષમતા ઘટાડે. [ભાગ #74 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
75	Proxy Object Variant (Concept Part #75)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન્સ (get, set) ને કસ્ટમાઇઝ કરતો સિકેટ એજન્ટ. [ભાગ #75 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
76	Reflect API Variant (Concept Part #76)	ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી મેનિપ્યુલેટ કરવા માટેના જાવાસ્ક્રિપ્ટ મેથડ્સ. [ભાગ #76 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
77	Variable Shadowing Variant (Concept Part #77)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #77 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
78	Global Scope Variant (Concept Part #78)	આખા પ્રોગ્રામમાં ગ્લોબલ સ્કોપમાં ફોર શકાતો સૌથી બહારનો સ્કોપ. [ભાગ #78 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
79	Block Scope Variant (Concept Part #79)	માત્ર કરવાી બ્લોક્સ '}' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #79 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

80	Function Scope Variant (Concept Part #80)	માત્ર જેનો ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #80 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
81	Execution Context Variant (Concept Part #81)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #81 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
82	Call Stack Variant (Concept Part #82)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) ડ્રાઇવ. [ભાગ #82 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
83	Hoisting Variant (Concept Part #83)	ડિફરેન્સિયલ કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સૌથી ઉપર રહેવાની પ્રોપર્ટી. [ભાગ #83 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
84	Temporal Dead Zone Variant (Concept Part #84)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો કોડ. [ભાગ #84 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
85	Closure Variant (Concept Part #85)	ફંક્શન તેના પેરેન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી યાદ રાખે તે. [ભાગ #85 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
86	IIFE Variant (Concept Part #86)	બનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #86 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
87	Currying Variant (Concept Part #87)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં ફેરવવું. [ભાગ #87 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
88	Memoization Variant (Concept Part #88)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવું. [ભાગ #88 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
89	Debouncing Variant (Concept Part #89)	યુઝર ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવેન્ટ ફાયર થાય. [ભાગ #89 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
90	Throttling Variant (Concept Part #90)	કોઈ ઇવેન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને થોડા સમયમાં ફોર રાખવું. [ભાગ #90 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
91	Callback Hell Variant (Concept Part #91)	એકની અંદર એક અસંખ્ય કોલબેક વખવાથી બનતો અધરાત કોડ સ્ટ્રક્ચર. [ભાગ #91 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
92	Inversion of Control Variant (Concept Part #92)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને ખામી. [ભાગ #92 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

93	Promise Variant (Concept Part #93)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેટી આપતું ઓબ્જેક્ટ. [ભાગ #93 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
94	Event Loop Variant (Concept Part #94)	સિંગલ થ્રેડડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચાલુ રાખતું. [ભાગ #94 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
95	Microtask Queue Variant (Concept Part #95)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં મળે છે. [ભાગ #95 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
96	Macrotask Queue Variant (Concept Part #96)	setTimeout અને ઇવેન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ લાઇન. [ભાગ #96 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
97	Starvation Variant (Concept Part #97)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે. [ભાગ #97 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
98	Prototypal Inheritance Variant (Concept Part #98)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #98 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
99	Prototype Chain Variant (Concept Part #99)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ ચેઇનમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #99 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
100	V8 Engine Variant (Concept Part #100)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #100 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
101	JIT Compilation Variant (Concept Part #101)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ કરવું. [ભાગ #101 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
102	Inline Caching Variant (Concept Part #102)	વારંવાર વપરાતા ઓબ્જેક્ટ પ્રોપર્ટીઝને એન્જિન આપમેળે જ સ્પીડ વધારે છે. [ભાગ #102 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
103	Hidden Classes Variant (Concept Part #103)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલી શોખા. [ભાગ #103 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
104	Tree Shaking Variant (Concept Part #104)	બિલ્ડ પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમ કરવો. [ભાગ #104 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
105	CORS Policy Variant (Concept Part #105)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિફ્રેશ સિદ્ધાંતની રીત. [ભાગ #105 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

	Part #105)	કરવાનો નિયમ. [ભાગ #105 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
106	XSS (Cross-Site Scripting) Variant (Concept Part #106)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલીશિયસ જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #106 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
107	CSRF Variant (Concept Part #107)	યુઝરની પરમિશન વગર તેના બાઉઝર સેશન દ્વારા ખોટી ડિમાન્ડ મોકલવી. [ભાગ #107 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
108	Web Workers Variant (Concept Part #108)	ભારે કેલ્ક્યુલેશનના કોડને મેઈન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડમાં ચલાવવાનો ટેકનિક. [ભાગ #108 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
109	Temporal API Variant (Concept Part #109)	જૂના Date ઓબ્જેક્ટને બદલે ટાઈમ-ઝોન ફિક્સ કરતું વેબ API. [ભાગ #109 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
110	Explicit Resource Management Variant (Concept Part #110)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર થાય. [ભાગ #110 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
111	Generator Function Variant (Concept Part #111)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વ્યથેથી રોકી અને ફરેશ કરી શકાય. [ભાગ #111 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
112	Decorators Variant (Concept Part #112)	@ સિન્ટેક્સ વાળું અડજુન્ટ ફીચર જે ક્લાસ કે મેથડની ક્ષમતા ટાંકી શકે. [ભાગ #112 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
113	Proxy Object Variant (Concept Part #113)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન્સ (get, set) ને કસ્ટમાઇઝ કરતો સિંકેટ એજન્ટ. [ભાગ #113 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
114	Reflect API Variant (Concept Part #114)	ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી મેનિપ્યુલેટ કરવા માટેના ઓબ્જેક્ટ મેથડ્સ. [ભાગ #114 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
115	Variable Shadowing Variant (Concept Part #115)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #115 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
116	Global Scope Variant (Concept Part #116)	આખા પ્રોગ્રામમાં ગ્લોબલ સ્કોપમાં રોકાયેલી સૌથી બહારની સ્કોપ. [ભાગ #116 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

117	Block Scope Variant (Concept Part #117)	માત્ર કરવી બ્લૅક સ્કોપ '}' ની અંદર જ કામ કરતી સ્કોપ (let, const, var). [ભાગ #117 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
118	Function Scope Variant (Concept Part #118)	માત્ર જે-તે ફંક્શનની અંદર જ માન્ય રહેતી સ્કોપ (var). [ભાગ #118 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
119	Execution Context Variant (Concept Part #119)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #119 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
120	Call Stack Variant (Concept Part #120)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) સ્ટ્રક્ચર. [ભાગ #120 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
121	Hoisting Variant (Concept Part #121)	ડિફાલ્ટ રીલોડિંગ કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સીધી ઉપર લેવાની પ્રોસેસ. [ભાગ #121 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
122	Temporal Dead Zone Variant (Concept Part #122)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો કોડ. [ભાગ #122 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
123	Closure Variant (Concept Part #123)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ યાદ રાખે તે. [ભાગ #123 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
124	IIFE Variant (Concept Part #124)	બંધાતીની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવું ફંક્શન. [ભાગ #124 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
125	Currying Variant (Concept Part #125)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં રૂપાંતરિત કરવા. [ભાગ #125 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
126	Memoization Variant (Concept Part #126)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી વધારવા. [ભાગ #126 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
127	Debouncing Variant (Concept Part #127)	યુઝર ઇનપુટ ટાઇપિંગ બંધ કરે તેના પછી સમય પછી જ ઇવન્ટ ફાયર થાય. [ભાગ #127 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
128	Throttling Variant (Concept Part #128)	કોઈ ઇવન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને ચોક્કસ સમયગાળામાં ફાયર થાય. [ભાગ #128 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
129	Callback Hell Variant (Concept Part #129)	એકની અંદર એક અસંખ્ય કોલબેક વાળાવાળી બનતી અધારેલી કોડ સ્ટ્રક્ચર. [ભાગ #129 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

130	Inversion of Control Variant (Concept Part #130)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને ખામી. [ભાગ #130 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
131	Promise Variant (Concept Part #131)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેટી આપતું એન્ટિપાસ. [ભાગ #131 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
132	Event Loop Variant (Concept Part #132)	સિંગલ થ્રેડ્ડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં વ્યૂપ. [ભાગ #132 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
133	Microtask Queue Variant (Concept Part #133)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં મળે છે. [ભાગ #133 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
134	Macrotask Queue Variant (Concept Part #134)	setTimeout અને ઇવેન્ટુસનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ. [ભાગ #134 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
135	Starvation Variant (Concept Part #135)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે સ્થિતિ. [ભાગ #135 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
136	Prototypal Inheritance Variant (Concept Part #136)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #136 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
137	Prototype Chain Variant (Concept Part #137)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ લિંકમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #137 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
138	V8 Engine Variant (Concept Part #138)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #138 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
139	JIT Compilation Variant (Concept Part #139)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ. [ભાગ #139 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
140	Inline Caching Variant (Concept Part #140)	વારંવાર વપરાતા ઓબ્જેક્ટ લોકેશનને એન્જિન આપમેળે રમીડ વધારે છે. [ભાગ #140 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
141	Hidden Classes Variant (Concept Part #141)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલો શોર્ટ. [ભાગ #141 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
142	Tree Shaking Variant (Concept Part #142)	બિન-પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમુવ. [ભાગ #142 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

143	CORS Policy Variant (Concept Part #143)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિદ્ધાંતની નિયમ. [ભાગ #143 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
144	XSS (Cross-Site Scripting) Variant (Concept Part #144)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલિશિયસ જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #144 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
145	CSRF Variant (Concept Part #145)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #145 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
Advanced Scope, Functions & Functional Programming (ક્રમ ૧૨૧ થી ૨૮૦)		
ક્લોઝર્સ, સ્કોપ ચેઇન, હોઇસ્ટિંગ, મેમોઇઝેશન, કરીંગ અને એડવાન્સ્ડ ફંક્શન લીજિસ્ટ્રી.		
ક્રમ	JavaScript Concept / Term	સરળ ગ્રંથજ્ઞાતી સમજૂતી (Detailed Explanation)
146	Web Workers Variant (Concept Part #146)	ભારે કેલ્ક્યુલેશનના કોડને મેઇન થ્રેડ રોકા વગર બેકગ્રાઉન્ડમાં ચલાવવાની ટેકનિક. [ભાગ #146 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
147	Temporal API Variant (Concept Part #147)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું લેટેસ્ટ API. [ભાગ #147 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
148	Explicit Resource Management Variant (Concept Part #148)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર. [ભાગ #148 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
149	Generator Function Variant (Concept Part #149)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરમાવે ચલાવ. [ભાગ #149 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
150	Decorators Variant (Concept Part #150)	@ સિન્ટેક્સ વાળું અડધુત ફીચર જે ક્લાસ કે મેથડની ક્ષમતા ડેકોરેટ. [ભાગ #150 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
151	Proxy Object Variant (Concept Part #151)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન્સ (get, set) ને ડાયરેક્ટ કરતો સિંક્રેટ એજન્ટ. [ભાગ #151 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

152	Reflect API Variant (Concept Part #152)	ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી મેનિપ્યુલેટ કરવા માટેના જોડાણ મેથડ્સ. [ભાગ #152 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
153	Variable Shadowing Variant (Concept Part #153)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #153 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
154	Global Scope Variant (Concept Part #154)	આખા પ્રોગ્રામમાં ગ્લોબલ સ્કોપમાં જોડાયેલ સ્કોપ. [ભાગ #154 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
155	Block Scope Variant (Concept Part #155)	માત્ર ફંક્શન બ્લોક ' {} ' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #155 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
156	Function Scope Variant (Concept Part #156)	માત્ર જો તે ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #156 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
157	Execution Context Variant (Concept Part #157)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #157 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
158	Call Stack Variant (Concept Part #158)	ફંક્શન કોલિંગનો ઓર્ડર રાખતો LIFO (Last In First Out) સ્ટેક. [ભાગ #158 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
159	Hoisting Variant (Concept Part #159)	ડિક્લેરેશન કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સીધી ઉપર રહેવાની પ્રોપર્ટી. [ભાગ #159 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
160	Temporal Dead Zone Variant (Concept Part #160)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો કોડનો ક્ષેત્ર. [ભાગ #160 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
161	Closure Variant (Concept Part #161)	ફંક્શન તેના પેરેન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ ધરાવે છે. [ભાગ #161 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
162	IIFE Variant (Concept Part #162)	બનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #162 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
163	Currying Variant (Concept Part #163)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં રૂપાંતરિત કરવાનો પ્રકાર. [ભાગ #163 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
164	Memoization Variant (Concept Part #164)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરવાનો પ્રકાર. [ભાગ #164 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

165	Debounce Variant (Concept Part #165)	યુઝર ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવેન્ટ ફાયર થાય. [ભાગ #165 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
166	Throttling Variant (Concept Part #166)	કોઈ ઇવેન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને થોડા સમયમાં ફાયર થઈ શકે છે. [ભાગ #166 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
167	Callback Hell Variant (Concept Part #167)	એકની અંદર એક અસંખ્ય કોલબેક લખવાથી બનતો અધરાયેલ કોડ સ્ટ્રક્ચર. [ભાગ #167 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
168	Inversion of Control Variant (Concept Part #168)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોર્પોરેટ પાર્ટી વાળા ફંક્શનને સોંપવાનો પ્રકાર. [ભાગ #168 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
169	Promise Variant (Concept Part #169)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની જરૂરી આપત્તિ આપવાનો પ્રકાર. [ભાગ #169 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
170	Event Loop Variant (Concept Part #170)	સિંગલ થ્રેડ ડોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચલાવવાનો પ્રકાર. [ભાગ #170 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
171	Microtask Queue Variant (Concept Part #171)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં ચલાવવાનો પ્રકાર. [ભાગ #171 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
172	Macrotask Queue Variant (Concept Part #172)	setTimeout અને ઇવેન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ લાઇન. [ભાગ #172 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
173	Starvation Variant (Concept Part #173)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તેવો સ્થિતિ. [ભાગ #173 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
174	Prototypal Inheritance Variant (Concept Part #174)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #174 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
175	Prototype Chain Variant (Concept Part #175)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ લિંકમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #175 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
176	V8 Engine Variant (Concept Part #176)	ગૂગલ ક્રોમિયમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #176 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

177	JIT Compilation Variant (Concept Part #177)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કોંટ્રોલ. [ભાગ #177 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
178	Inline Caching Variant (Concept Part #178)	વારંવાર વપરાતા ઓબ્જેક્ટ લોકેશનને એન્જિન આપમેળે સ્પીડ વધારે છે. [ભાગ #178 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
179	Hidden Classes Variant (Concept Part #179)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલો છે. [ભાગ #179 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
180	Tree Shaking Variant (Concept Part #180)	બિન-પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિન-જરૂરી કોડ ઓટોમેટિક રીમુવ કરવો. [ભાગ #180 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
181	CORS Policy Variant (Concept Part #181)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિક્યોરિટી કરવાનો નિયમ. [ભાગ #181 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
182	XSS (Cross-Site Scripting) Variant (Concept Part #182)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીમીડિયા જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #182 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
183	CSRF Variant (Concept Part #183)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ડિમાન્ડ મોકલવી. [ભાગ #183 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
184	Web Workers Variant (Concept Part #184)	ભારે કેલ્ક્યુલેશનના કોડને મેઇન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની ટેકનિક. [ભાગ #184 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
185	Temporal API Variant (Concept Part #185)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું વેબ API. [ભાગ #185 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
186	Explicit Resource Management Variant (Concept Part #186)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર કરવો. [ભાગ #186 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
187	Generator Function Variant (Concept Part #187)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરાર શકાય. [ભાગ #187 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
188	Decorators Variant (Concept Part #188)	@ સિન્ટેક્સ વાળું અડધું કીવર્ડ જે ક્લાસ કે મેથડની ક્ષમતા ડેકોરેટ કરે. [ભાગ #188 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

189	Proxy Object Variant (Concept Part #189)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન (get, set) ને કસ્ટમાઇઝ કરતો સિક્રેટ એજન્ટ. [ભાગ #189 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
190	Reflect API Variant (Concept Part #190)	ઓબ્જેક્ટ ઓપરેશનને સરળતાથી મેનિપ્યુલેટ કરવા માટેનું જાવસ્ક્રિપ્ટ મેથડ્સ. [ભાગ #190 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
191	Variable Shadowing Variant (Concept Part #191)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #191 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
192	Global Scope Variant (Concept Part #192)	આખા પ્રોગ્રામમાં ગ્લોબલ સ્કોપમાં એક્સેસ કરી શકતો સૌથી બૉલ્ડ સ્કોપ. [ભાગ #192 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
193	Block Scope Variant (Concept Part #193)	માત્ર કરવી બ્રેકેટ '}' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #193 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
194	Function Scope Variant (Concept Part #194)	માત્ર જે-તે ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #194 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
195	Execution Context Variant (Concept Part #195)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #195 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
196	Call Stack Variant (Concept Part #196)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) સ્ટેક. [ભાગ #196 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
197	Hoisting Variant (Concept Part #197)	ડિફાલ્ટ રીલેશન કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સીધો ઉમેરવો. [ભાગ #197 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
198	Temporal Dead Zone Variant (Concept Part #198)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો કોડ. [ભાગ #198 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
199	Closure Variant (Concept Part #199)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ રાખે તે. [ભાગ #199 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
200	IFE Variant (Concept Part #200)	બંતાતી સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #200 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

201	Currying Variant (Concept Part #201)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શન ફેરવવું. [ભાગ #201 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
202	Memoization Variant (Concept Part #202)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી વધારવી. [ભાગ #202 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
203	Debouncing Variant (Concept Part #203)	ચુસ્ત ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવેન્ટ ફાયર થાય. [ભાગ #203 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
204	Throttling Variant (Concept Part #204)	કોઈ ઇવેન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને ચોક્કસ સમય રન કરવી. [ભાગ #204 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
205	Callback Hell Variant (Concept Part #205)	એકની અંદર એક અસંખ્ય કોલબેક લખવાથી બનતો અધરો કોડ સ્પ્રુઅર. [ભાગ #205 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
206	Inversion of Control Variant (Concept Part #206)	કોલબેક ફંક્શનનો કંટ્રોલ કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને ખામી. [ભાગ #206 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
207	Promise Variant (Concept Part #207)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેટી આપતું ઓપરેશન. [ભાગ #207 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
208	Event Loop Variant (Concept Part #208)	સિંગલ થ્રેડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચાલુ રાખી. [ભાગ #208 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
209	Microtask Queue Variant (Concept Part #209)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં ચલાવવાનું છે. [ભાગ #209 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
210	Macrotask Queue Variant (Concept Part #210)	setTimeout અને ઇવેન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ લાઇન. [ભાગ #210 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
211	Starvation Variant (Concept Part #211)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે સ્થિતિ. [ભાગ #211 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
212	Prototypal Inheritance Variant (Concept Part #212)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવી શકે. [ભાગ #212 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
213	Prototype Chain Variant (Concept Part #213)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ (બિકમાં આપેલ શોષવાની પ્રક્રિયા). [ભાગ #213 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

214	V8 Engine Variant (Concept Part #214)	નૂતન કોમ્પાઇલરમાં વધારાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #214 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
215	JIT Compilation Variant (Concept Part #215)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્સાઇલ કરવા. [ભાગ #215 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
216	Inline Caching Variant (Concept Part #216)	વારંવાર વપરાતા ઓબ્જેક્ટ પ્રોપર્ટીને એન્જિન આપમેળે જ સ્પીડ વધારે છે. [ભાગ #216 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
217	Hidden Classes Variant (Concept Part #217)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલ શેડો ક્લાસ. [ભાગ #217 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
218	Tree Shaking Variant (Concept Part #218)	બિનપ્રયોજન થતી કોડ લાઇનને ઓપ્ટિમાઇઝેશન દરમિયાન કાઢી દેવા. [ભાગ #218 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
219	CORS Policy Variant (Concept Part #219)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા ટ્રાન્સફર સીધો કરવાનો નિયમ. [ભાગ #219 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
220	XSS (Cross-Site Scripting) Variant (Concept Part #220)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીમીડિયા જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #220 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
221	CSRF Variant (Concept Part #221)	ચુસ્તની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ઇન્ફોર્મેશન મોકલવી. [ભાગ #221 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
222	Web Workers Variant (Concept Part #222)	ભારે કૅલ્ક્યુલેશનના કોડને મેઇન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડમાં ચલાવવાની ટેકનિક. [ભાગ #222 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
223	Temporal API Variant (Concept Part #223)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું વેબ API. [ભાગ #223 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
224	Explicit Resource Management Variant (Concept Part #224)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર થાય. [ભાગ #224 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
225	Generator Function Variant (Concept Part #225)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરતે શકાય. [ભાગ #225 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

226	Decorators Variant (Concept Part #226)	@ સિન્ટેક્સ વાળું અંદરુત ક્રીયર જે ક્લાસ કે મેથડની ક્ષમતા [ભાગ #226 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
227	Proxy Object Variant (Concept Part #227)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન (get, set) ને કરેટમાઇ કરતો સિકેટ એજન્ટ. [ભાગ #227 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
228	Reflect API Variant (Concept Part #228)	ઓબ્જેક્ટ ઓપરેશનને સરળતાથી મેનિપ્યુલેટ કરવા માટેનું ઓબલ મેથડ્સ. [ભાગ #228 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
229	Variable Shadowing Variant (Concept Part #229)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #229 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
230	Global Scope Variant (Concept Part #230)	આખા પ્રોગ્રામમાં ગમે ત્યાંથી એક્સેસ કરી શકાતો સૌથી બૃહદ સ્કોપ. [ભાગ #230 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
231	Block Scope Variant (Concept Part #231)	માત્ર કરથી બ્રેસીસ '}' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #231 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
232	Function Scope Variant (Concept Part #232)	માત્ર જે-તે ફંક્શનની અંદર જ માત્ર રહેતો સ્કોપ (var). [ભાગ #232 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
233	Execution Context Variant (Concept Part #233)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #233 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
234	Call Stack Variant (Concept Part #234)	ફંક્શન કોલિંગનો ઓર્ડર રાખતો LIFO (Last In First Out) ડ્રાઇવ. [ભાગ #234 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
235	Hoisting Variant (Concept Part #235)	ડિફરેન્સિયલ કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સૌથી ઉપર રહેવાની પ્રોસેસ. [ભાગ #235 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
236	Temporal Dead Zone Variant (Concept Part #236)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો સમયગાળો. [ભાગ #236 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
237	Closure Variant (Concept Part #237)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી યાદ રાખે તે. [ભાગ #237 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

238	IFE Variant (Concept Part #238)	બનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #238 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
239	Currying Variant (Concept Part #239)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં રૂપાંતરિત કરવાનો પ્રકાર. [ભાગ #239 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
240	Memoization Variant (Concept Part #240)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવાનો પ્રકાર. [ભાગ #240 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
241	Debouncing Variant (Concept Part #241)	ચુસ્ત ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવન્ટ ફાયર થાય. [ભાગ #241 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
242	Throttling Variant (Concept Part #242)	કોઈ ઇવન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને ચોક્કસ સમયગાળામાં ફાયર થાય. [ભાગ #242 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
243	Callback Hell Variant (Concept Part #243)	એકની અંદર એક અસંખ્ય કોલબેક લખવાથી બનતો અધૂરો કોડ સ્ટ્રક્ચર. [ભાગ #243 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
244	Inversion of Control Variant (Concept Part #244)	કોલબેક ફંક્શનનો કંટ્રોલ બીજા કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને ખામી. [ભાગ #244 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
245	Promise Variant (Concept Part #245)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેંટટી આપતું ઓબ્જેક્ટ. [ભાગ #245 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
246	Event Loop Variant (Concept Part #246)	સિંગલ થ્રેડ્ડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચાલુ રાખે. [ભાગ #246 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
247	Microtask Queue Variant (Concept Part #247)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં મળે છે. [ભાગ #247 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
248	Macrotask Queue Variant (Concept Part #248)	setTimeout અને ઇવેન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ લાઇન. [ભાગ #248 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
249	Starvation Variant (Concept Part #249)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અડકી પડે તે સ્થિતિ. [ભાગ #249 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
250	Prototypal Inheritance Variant (Concept Part #250)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #250 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

251	Prototype Chain Variant (Concept Part #251)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ બિંદુમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #251 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
252	V8 Engine Variant (Concept Part #252)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #252 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
253	JIT Compilation Variant (Concept Part #253)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્પાઇલ કરવાનો પ્રકાર. [ભાગ #253 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
254	Inline Caching Variant (Concept Part #254)	વારંવાર વપરાતા ઓબ્જેક્ટ લોકેશનને એન્જિન આપમેળે યાદ રાખી રાખવાનો પ્રકાર. [ભાગ #254 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
255	Hidden Classes Variant (Concept Part #255)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલા હિડન ક્લાસ. [ભાગ #255 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
256	Tree Shaking Variant (Concept Part #256)	બિંદુ પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમુવ કરવો. [ભાગ #256 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
257	CORS Policy Variant (Concept Part #257)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિંક્રોનસીટી કરવાનો નિયમ. [ભાગ #257 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
258	XSS (Cross-Site Scripting) Variant (Concept Part #258)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીશિયસ જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #258 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
259	CSRF Variant (Concept Part #259)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ફોર્મ સબમિટ મોકલવી. [ભાગ #259 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
260	Web Workers Variant (Concept Part #260)	ભારે કેમ્બ્રિજેશનના કોડને મેઇન થ્રેડ વગર બેકગ્રાઉન્ડમાં ચલાવવાની ટેકનિક. [ભાગ #260 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
261	Temporal API Variant (Concept Part #261)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું વેબ API. [ભાગ #261 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
262	Explicit Resource Management	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર કરવાનો પ્રકાર. [ભાગ #262 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

262	Variant (Concept Part #262)	[ભાગ #262 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
263	Generator Function Variant (Concept Part #263)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરી શરૂ કરી શકાય. [ભાગ #263 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
264	Decorators Variant (Concept Part #264)	@ સિન્ટેક્સ વાળું અડધું ફીચર જે ક્લાસ કે મેથડની ક્ષમતા ઘટાડે. [ભાગ #264 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
265	Proxy Object Variant (Concept Part #265)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન્સ (get, set) ને કસ્ટમાઇઝ કરતો સિક્રેટ એજન્ટ. [ભાગ #265 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
266	Reflect API Variant (Concept Part #266)	ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી મેનિપ્યુલેટ કરવા માટેના જવાબ મેથડ્સ. [ભાગ #266 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
267	Variable Shadowing Variant (Concept Part #267)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #267 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
268	Global Scope Variant (Concept Part #268)	આખા પ્રોગ્રામમાં ગમે ત્યાંથી એક્સેસ કરી શકાતો સૌથી બૉલ્ડ સ્કોપ. [ભાગ #268 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
269	Block Scope Variant (Concept Part #269)	માત્ર કરલી બ્રેકેટ્સ '()' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #269 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
270	Function Scope Variant (Concept Part #270)	માત્ર જે-તે ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #270 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
271	Execution Context Variant (Concept Part #271)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #271 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
272	Call Stack Variant (Concept Part #272)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) ડાઇવ. [ભાગ #272 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
273	Hoisting Variant (Concept Part #273)	ડિફિનિશન કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સીધી ઉતારવાની પ્રોસેસ. [ભાગ #273 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
274	Temporal Dead Zone Variant	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો પ્રદેશ. [ભાગ #274 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

275	(Concept Part #274)	સમયગાળો. [ભાગ #274 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
	Closure Variant (Concept Part #275)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી રાખે તે. [ભાગ #275 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
276	IIFE Variant (Concept Part #276)	બનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #276 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
277	Currying Variant (Concept Part #277)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં ફેરવવું. [ભાગ #277 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
278	Memoization Variant (Concept Part #278)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવો. [ભાગ #278 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
279	Debouncing Variant (Concept Part #279)	ચુસ્ત ઇનાપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવન્ટ ફાયર કરવો. [ભાગ #279 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
280	Throttling Variant (Concept Part #280)	કોઈ ઇવન્ટ ને તેટલી વાર ટ્રિગર થાય પછી તેને ચોક્કસ સમયગાળા પર ફાયર કરવો. [ભાગ #280 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
281	Callback Hell Variant (Concept Part #281)	એકની અંદર એક અસંખ્ય કોલબેક લખવાથી બનતો અધરાયેલ કોડ સ્ટ્રક્ચર. [ભાગ #281 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
282	Inversion of Control Variant (Concept Part #282)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને સોંપી દેવો. [ભાગ #282 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
283	Promise Variant (Concept Part #283)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેટી આપતું એન્ટિપેસિડ. [ભાગ #283 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
284	Event Loop Variant (Concept Part #284)	સિંગલ થ્રેડ્ડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચલાવવાનું વ્યવસ્થા. [ભાગ #284 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
285	Microtask Queue Variant (Concept Part #285)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં જ ચલાવવાનું. [ભાગ #285 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
286	Macrotask Queue Variant (Concept Part #286)	setTimeout અને ઇવન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ ક્વિયુ. [ભાગ #286 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
287	Starvation Variant (Concept	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે

288	Part #287)	[ભાગ #287 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
	Prototypal Inheritance Variant (Concept Part #288)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #288 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
289	Prototype Chain Variant (Concept Part #289)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ લિંકમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #289 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
290	V8 Engine Variant (Concept Part #290)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #290 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
Object-Oriented JavaScript & Prototypes (ક્રમ ૨૮૧ થી ૪૨૦)		
પ્રોટોટાઇપ ચેઇન, ઇનહેરિટેન્સ, Classes, 'this' પ્રિફિનિઝમ અને ઓબ્જેક્ટ કંટ્રોલ્સ.		
ક્રમ	JavaScript Concept / Term	સરળ ગ્રંથરતી સમજૂતી (Detailed Explanation)
291	JIT Compilation Variant (Concept Part #291)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ કરવો. [ભાગ #291 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
292	Inline Caching Variant (Concept Part #292)	વારંવાર વપરાતા ઓબ્જેક્ટ પ્રોપર્ટીઝને એન્જિન આપમેળે જ સ્પીડ વધારે છે. [ભાગ #292 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
293	Hidden Classes Variant (Concept Part #293)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલો શેપ્સ. [ભાગ #293 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
294	Tree Shaking Variant (Concept Part #294)	બિલ્ડ પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમ કરવો. [ભાગ #294 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
295	CORS Policy Variant (Concept Part #295)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિદ્ધાંતની રક્ષા કરવાનો નિયમ. [ભાગ #295 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
296	XSS (Cross-Site Scripting) Variant (Concept Part #296)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીમીડિયમ જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #296 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

297	CSRF Variant (Concept Part #297)	યુઝરની પરમિશન વગર તેના બાઉઝર સેશન દ્વારા ખોટી ડેટા મોકલવી. [ભાગ #297 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
298	Web Workers Variant (Concept Part #298)	ભારે કેલ્ક્યુલેશનના કોડને મેઈન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડ ચલાવવાની ટેકનિક. [ભાગ #298 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
299	Temporal API Variant (Concept Part #299)	જૂના Date ઓબ્જેક્ટને બદલે ટાઈમ-ઝોન ફિક્સ કરતું વેબ API. [ભાગ #299 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
300	Explicit Resource Management Variant (Concept Part #300)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતા જ મેમરી ઓટોમેટિક ક્લિયર થાય. [ભાગ #300 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
301	Generator Function Variant (Concept Part #301)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વ્યૂથી રોકી અને ફરક શકાય. [ભાગ #301 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
302	Decorators Variant (Concept Part #302)	@ સિન્ટેક્સ વાળું અડધું ફીચર જે ક્લાસ કે મેથડની શ્રેણીમાં ડેકોરેટર લગાડવાનું છે. [ભાગ #302 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
303	Proxy Object Variant (Concept Part #303)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન્સ (get, set) ને કન્ટ્રોલ કરતો સિકેટ એજન્ટ. [ભાગ #303 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
304	Reflect API Variant (Concept Part #304)	ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી મેનિપ્યુલેટ કરવા માટેનું જોબબ મેથડ્સ. [ભાગ #304 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
305	Variable Shadowing Variant (Concept Part #305)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલને ઓવરવેપ કરે. [ભાગ #305 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
306	Global Scope Variant (Concept Part #306)	આખા પ્રોગ્રામમાં ગ્લોબલ એસ્કોપ કરી શકાતો સૌથી બહારનો સ્કોપ. [ભાગ #306 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
307	Block Scope Variant (Concept Part #307)	માત્ર કરવી બ્રેકેટ '}' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #307 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
308	Function Scope Variant (Concept Part #308)	માત્ર જે-ને ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #308 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

309	Execution Context Variant (Concept Part #309)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #309 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
310	Call Stack Variant (Concept Part #310)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) ડેક. [ભાગ #310 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
311	Hoisting Variant (Concept Part #311)	ડિક્લેરેશન કોડ રન થતા પહેલાં આપમેળે સ્કોપમાં સોંપી દેવામાં આવે છે. [ભાગ #311 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
312	Temporal Dead Zone Variant (Concept Part #312)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય. [ભાગ #312 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
313	Closure Variant (Concept Part #313)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ યાદ રાખે છે. [ભાગ #313 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
314	IIFE Variant (Concept Part #314)	બનાતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય. [ભાગ #314 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
315	Currying Variant (Concept Part #315)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શન બનાવવાનું છે. [ભાગ #315 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
316	Memoization Variant (Concept Part #316)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવાનું છે. [ભાગ #316 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
317	Debouncing Variant (Concept Part #317)	યુઝર ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવન્ટ ફાયર થાય. [ભાગ #317 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
318	Throttling Variant (Concept Part #318)	કોઈ ઇવન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને ચોક્કસ સમય અંતરે જ ફાયર થાય. [ભાગ #318 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
319	Callback Hell Variant (Concept Part #319)	એકની અંદર એક અસંખ્ય કોલબેક બપવાથી બનતો અધૂરો કોડ સ્ટ્રક્ચર. [ભાગ #319 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
320	Inversion of Control Variant (Concept Part #320)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને સોંપી દેવામાં આવે છે. [ભાગ #320 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
321	Promise Variant (Concept Part #321)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેકીંગ આપતું ઓબ્જેક્ટ. [ભાગ #321 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

322	Event Loop Variant (Concept Part #322)	સિંગલ થ્રેડેડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં વૂપ. [ભાગ #322 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
323	Microtask Queue Variant (Concept Part #323)	પ્રોમિસિસના કોડ રાખતી વાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં મળે છે. [ભાગ #323 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
324	Macrotask Queue Variant (Concept Part #324)	setTimeout અને ઇવેન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડમાં. [ભાગ #324 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
325	Starvation Variant (Concept Part #325)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે. [ભાગ #325 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
326	Prototypal Inheritance Variant (Concept Part #326)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે. [ભાગ #326 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
327	Prototype Chain Variant (Concept Part #327)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ (બિકમાં આપેલ શોધવાની પ્રક્રિયા. [ભાગ #327 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
328	V8 Engine Variant (Concept Part #328)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #328 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
329	JIT Compilation Variant (Concept Part #329)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ કરવા. [ભાગ #329 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
330	Inline Caching Variant (Concept Part #330)	વારંવાર વપરાતા ઓબ્જેક્ટ વોકેશનને એન્જિન આધારે સ્પીડ વધારે છે. [ભાગ #330 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
331	Hidden Classes Variant (Concept Part #331)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલ શેડો. [ભાગ #331 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
332	Tree Shaking Variant (Concept Part #332)	બિન-પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિન-જરૂરી કોડ ઓટોમેટિક રીમોવ કરવો. [ભાગ #332 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
333	CORS Policy Variant (Concept Part #333)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિંક્રોનસ કરવાનો નિયમ. [ભાગ #333 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

334	XSS (Cross-Site Scripting) Variant (Concept Part #334)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીમીડિયા જાવાસ્ક્રિપ્ટ ઇમ્પ્લેમેન્ટ કરવાનો એટેક. [ભાગ #334 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
335	CSRF Variant (Concept Part #335)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ઇમેઇલ મોકલવો. [ભાગ #335 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
336	Web Workers Variant (Concept Part #336)	ભારે કેલ્ક્યુલેશનના કોડને મેઇન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડમાં ચલાવવાની ટેકનિક. [ભાગ #336 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
337	Temporal API Variant (Concept Part #337)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું વેબ API. [ભાગ #337 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
338	Explicit Resource Management Variant (Concept Part #338)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર કરવો. [ભાગ #338 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
339	Generator Function Variant (Concept Part #339)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરી શરૂ કરવા. [ભાગ #339 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
340	Decorators Variant (Concept Part #340)	@ સિન્ટેક્સ વાળું અડધુત ફીચર જે ક્લાસ કે મેથડની ક્ષમતા વધારે છે. [ભાગ #340 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
341	Proxy Object Variant (Concept Part #341)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન્સ (get, set) ને કસ્ટમાઇઝ કરતો સિક્રેટ એજન્ટ. [ભાગ #341 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
342	Reflect API Variant (Concept Part #342)	ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી મેનિપ્યુલેટ કરવા માટેનું ઓબ્જેક્ટ મેથડ્સ. [ભાગ #342 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
343	Variable Shadowing Variant (Concept Part #343)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #343 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
344	Global Scope Variant (Concept Part #344)	આખા પ્રોગ્રામમાં તમે ત્યાંથી એક્સેસ કરી શકતો સૌથી બહારનો સ્કોપ. [ભાગ #344 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
345	Block Scope Variant (Concept Part #345)	માત્ર કરવા બેચીસ `{} ` ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #345 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

346	Function Scope Variant (Concept Part #346)	માત્ર જેનો ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #346 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
347	Execution Context Variant (Concept Part #347)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #347 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
348	Call Stack Variant (Concept Part #348)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) ડાઇવ. [ભાગ #348 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
349	Hoisting Variant (Concept Part #349)	ડિક્લેરેશન કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સીધી ઉમેરવામાં આવે છે. [ભાગ #349 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
350	Temporal Dead Zone Variant (Concept Part #350)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો ક્ષેત્ર. [ભાગ #350 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
351	Closure Variant (Concept Part #351)	ફંક્શન તેના પેરેન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ યાદ રાખે છે. [ભાગ #351 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
352	IIFE Variant (Concept Part #352)	બનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #352 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
353	Currying Variant (Concept Part #353)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં રૂપાંતરિત કરવાનો પ્રકાર. [ભાગ #353 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
354	Memoization Variant (Concept Part #354)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવો. [ભાગ #354 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
355	Debouncing Variant (Concept Part #355)	ઘુઝરેલો ઇવેન્ટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવેન્ટ ફાયર થાય. [ભાગ #355 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
356	Throttling Variant (Concept Part #356)	કોઈ ઇવેન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને થોડા સમયમાં ફાયર થઈ શકે છે. [ભાગ #356 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
357	Callback Hell Variant (Concept Part #357)	એકની અંદર એક અસંખ્ય કોલબેક વાળા કોડ બ્લોક્સને સરળ બનાવવાનો પ્રયત્ન. [ભાગ #357 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
358	Inversion of Control Variant (Concept Part #358)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને સોંપી દેવો. [ભાગ #358 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

359	Promise Variant (Concept Part #359)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેન્જી આપતું એન્ટિપાસી. [ભાગ #359 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
360	Event Loop Variant (Concept Part #360)	સિંગલ થ્રેડ્ડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચલાવવાનો પ્રકાર. [ભાગ #360 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
361	Microtask Queue Variant (Concept Part #361)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં ચલાવવામાં આવે છે. [ભાગ #361 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
362	Macrotask Queue Variant (Concept Part #362)	setTimeout અને ઇવેન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ લાઇન. [ભાગ #362 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
363	Starvation Variant (Concept Part #363)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે સ્થિતિ. [ભાગ #363 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
364	Prototypal Inheritance Variant (Concept Part #364)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #364 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
365	Prototype Chain Variant (Concept Part #365)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ લિંકમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #365 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
366	V8 Engine Variant (Concept Part #366)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #366 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
367	JIT Compilation Variant (Concept Part #367)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ કરવાનો પ્રયત્ન. [ભાગ #367 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
368	Inline Caching Variant (Concept Part #368)	વારંવાર વપરાતા ઓબ્જેક્ટ પ્રોપર્ટીને એન્જિન આપમેળે યાદ રાખી રાખે છે. [ભાગ #368 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
369	Hidden Classes Variant (Concept Part #369)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલી શ્રેણી. [ભાગ #369 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
370	Tree Shaking Variant (Concept Part #370)	બિલ્ડ પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમ કરવો. [ભાગ #370 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
371	CORS Policy Variant (Concept Part #371)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિફ્રેસ્ટ સિદ્ધાંત. [ભાગ #371 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

	Part #371)	કરવાનો નિયમ. [ભાગ #371 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
372	XSS (Cross-Site Scripting) Variant (Concept Part #372)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલીશિયસ જાવાસ્ક્રિપ્ટ ઇન્જન કરવાનો એટેક. [ભાગ #372 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
373	CSRF Variant (Concept Part #373)	યુઝરની પરમિશન વગર તેના બાઉઝર સેશન દ્વારા ખોટી વિનંતી મોકલવી. [ભાગ #373 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
374	Web Workers Variant (Concept Part #374)	ભારે કેમ્બ્રિજેશનના કોડને મેઈન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડ યુવાવવાની ટેકનિક. [ભાગ #374 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
375	Temporal API Variant (Concept Part #375)	જૂના Date ઓબ્જેક્ટને બદલે ટાઈમ-ઝોન ફિક્સ કરતું વેબ API. [ભાગ #375 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
376	Explicit Resource Management Variant (Concept Part #376)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતા જ મેમરી ઓટોમેટિક ક્લિયર થાય. [ભાગ #376 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
377	Generator Function Variant (Concept Part #377)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વ્યૂથેબી રોકી અને ફરે શકાય. [ભાગ #377 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
378	Decorators Variant (Concept Part #378)	@ સિન્ટેક્સ વાળું અડજુન્ટ ફીચર જે ક્લાસ કે મેથડની ક્ષમતા ઘટાડે છે. [ભાગ #378 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
379	Proxy Object Variant (Concept Part #379)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન્સ (get, set) ને કસ્ટમાઇઝ કરતો સિંકેટ એજન્ટ. [ભાગ #379 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
380	Reflect API Variant (Concept Part #380)	ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી મેનિપ્યુલેટ કરવા માટેના ઓબ્જેક્ટ મેથડ્સ. [ભાગ #380 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
381	Variable Shadowing Variant (Concept Part #381)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #381 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
382	Global Scope Variant (Concept Part #382)	આખા પ્રોગ્રામમાં ગ્લોબલ સ્કોપમાં ફંક્શન રોકી શકાય તેવી સીધી બંધ રીડાઈ. [ભાગ #382 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

383	Block Scope Variant (Concept Part #383)	માત્ર કરવી બ્લેકીસ '}' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #383 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
384	Function Scope Variant (Concept Part #384)	માત્ર જે-તે ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #384 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
385	Execution Context Variant (Concept Part #385)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #385 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
386	Call Stack Variant (Concept Part #386)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) સ્ટ્રક્ચર. [ભાગ #386 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
387	Hoisting Variant (Concept Part #387)	ડિફાલ્ટ રીડાઈ કોડ રન થતા પહેલાં આપમેળે સ્કોપમાં સીધી ઉપર રહેવાની પ્રોસેસ. [ભાગ #387 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
388	Temporal Dead Zone Variant (Concept Part #388)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો કોડનો ક્ષેત્ર. [ભાગ #388 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
389	Closure Variant (Concept Part #389)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ યાદ રાખે છે. [ભાગ #389 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
390	IIFE Variant (Concept Part #390)	બંધાતી સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવું ફંક્શન. [ભાગ #390 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
391	Currying Variant (Concept Part #391)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં ફેરવવું. [ભાગ #391 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
392	Memoization Variant (Concept Part #392)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવો. [ભાગ #392 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
393	Debouncing Variant (Concept Part #393)	યુઝર ઇનપુટ ટાઇપિંગ બંધ કરે તેના પછી સમય પછી જ ઇવન્ટ ફાયર થાય. [ભાગ #393 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
394	Throttling Variant (Concept Part #394)	કોઈ ઇવન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને ચોક્કસ સમયગાળામાં ફાયર થાય. [ભાગ #394 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
395	Callback Hell Variant (Concept Part #395)	એકની અંદર એક અસંખ્ય કોલબેક વાળા સીધા બંધ રીડાઈ કોડ સ્ટ્રક્ચર. [ભાગ #395 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

396	Inversion of Control Variant (Concept Part #396)	કોષબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને ખામી. [ભાગ #396 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
397	Promise Variant (Concept Part #397)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેટી આપતું એક પ્રમીસ. [ભાગ #397 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
398	Event Loop Variant (Concept Part #398)	સિંગલ થ્રેડડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં વૃણ. [ભાગ #398 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
399	Microtask Queue Variant (Concept Part #399)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં મળે છે. [ભાગ #399 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
400	Macrotask Queue Variant (Concept Part #400)	setTimeout અને ઇવેન્ટુસનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ લાઇન. [ભાગ #400 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
401	Starvation Variant (Concept Part #401)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે. [ભાગ #401 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
402	Prototypal Inheritance Variant (Concept Part #402)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #402 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
403	Prototype Chain Variant (Concept Part #403)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ લિંકમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #403 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
404	V8 Engine Variant (Concept Part #404)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #404 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
405	JIT Compilation Variant (Concept Part #405)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્સેપ્ટ રીટ. [ભાગ #405 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
406	Inline Caching Variant (Concept Part #406)	વારંવાર વપરાતા ઓબ્જેક્ટ લોકેશનને એન્જિન આપમેળે રાખી રાખી વધારે છે. [ભાગ #406 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
407	Hidden Classes Variant (Concept Part #407)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલ શોર્ટ. [ભાગ #407 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
408	Tree Shaking Variant (Concept Part #408)	બિન-પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમુવ. [ભાગ #408 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

409	CORS Policy Variant (Concept Part #409)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિદ્ધાંતની રચના. [ભાગ #409 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
410	XSS (Cross-Site Scripting) Variant (Concept Part #410)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીમીડિયા જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #410 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
411	CSRF Variant (Concept Part #411)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #411 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
412	Web Workers Variant (Concept Part #412)	ભારે કેલ્ક્યુલેશનના કોડને મેઇન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની ટેકનિક. [ભાગ #412 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
413	Temporal API Variant (Concept Part #413)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું લેટર API. [ભાગ #413 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
414	Explicit Resource Management Variant (Concept Part #414)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર. [ભાગ #414 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
415	Generator Function Variant (Concept Part #415)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરી શરૂ કરી શકાય. [ભાગ #415 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
416	Decorators Variant (Concept Part #416)	@ સિન્ટેક્સ વાળું અદ્યતન ડીકોરેટર જે ક્લાસ કે મેથડની ક્ષમતા ડેકોરેટ કરી શકે. [ભાગ #416 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
417	Proxy Object Variant (Concept Part #417)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન (get, set) ને કસ્ટમાઇઝ કરી સિક્રેટ એજન્ટ. [ભાગ #417 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
418	Reflect API Variant (Concept Part #418)	ઓબ્જેક્ટ ઓપરેશનનાં સરળતાથી મેનિપ્યુલેટ કરવા માટેના ઓબ્જેક્ટ મેથડ્સ. [ભાગ #418 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
419	Variable Shadowing Variant (Concept Part #419)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલને ઓવરલેપ કરે. [ભાગ #419 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

420	Global Scope Variant (Concept Part #420)	આખા પ્રોગ્રામમાં ગમે ત્યાંથી એક્સેસ કરી શકાતો સૌથી બૃહદ્ સ્કોપ. [ભાગ #420 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
421	Block Scope Variant (Concept Part #421)	માત્ર કરવી બ્લૉક્સ ' {} ' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #421 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
422	Function Scope Variant (Concept Part #422)	માત્ર જે-તે ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #422 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
423	Execution Context Variant (Concept Part #423)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #423 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
424	Call Stack Variant (Concept Part #424)	ફંક્શન કોલિંગનો ઓર્ડર રાખતો LIFO (Last In First Out) સ્ટેક. [ભાગ #424 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
425	Hoisting Variant (Concept Part #425)	ડિફાઇનેશન કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સૌથી ઉપર રહેવાની પ્રોસેસ. [ભાગ #425 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
426	Temporal Dead Zone Variant (Concept Part #426)	let/const વેરીએબલ બનાવવા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો ક્ષેત્ર. [ભાગ #426 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
427	Closure Variant (Concept Part #427)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ યાદ રાખે તે. [ભાગ #427 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
428	IIFE Variant (Concept Part #428)	બનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #428 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
429	Currying Variant (Concept Part #429)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શન બનાવવાનો પ્રકાર. [ભાગ #429 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
430	Memoization Variant (Concept Part #430)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવો. [ભાગ #430 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
431	Debounce Variant (Concept Part #431)	ચુસ્ત ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવન્ટ ફાયર થવો. [ભાગ #431 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
432	Throttling Variant (Concept Part #432)	કોઈ ઇવેન્ટ ગમે તેટલી વાર ફિર થાય પણ તેને ચોક્કસ સમયગાળામાં ફાયર થવો. [ભાગ #432 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]

433	Callback Hell Variant (Concept Part #433)	એકની અંદર એક અસંખ્ય કોલબેક લખવાથી બનતો અધૂરો કોડ સ્ટ્રક્ચર. [ભાગ #433 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
434	Inversion of Control Variant (Concept Part #434)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને ખામોશી. [ભાગ #434 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
435	Promise Variant (Concept Part #435)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરંટી આપતું ઓબ્જેક્ટ. [ભાગ #435 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]

Asynchronous JavaScript & Memory Management (ક્રમ ૪૨૧ થી ૫૮૦)

ઇવેન્ટ લૂપ, કોલબેક હેલ, પ્રોમિસિસ, એસિંક-અવેટ, માઇક્રોકોટાસ્ક ક્યૂ અને ગાર્બેજ કલેક્શન.

ક્રમ	JavaScript Concept / Term	સરળ ગુજરાતી સમજૂતી (Detailed Explanation)
436	Event Loop Variant (Concept Part #436)	સિંગલ થ્રેડેડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં લૂપ. [ભાગ #436 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
437	Microtask Queue Variant (Concept Part #437)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોકોટાસ્ક કરતાં પહેલે મળે છે. [ભાગ #437 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
438	Macrotask Queue Variant (Concept Part #438)	setTimeout અને ઇવેન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ લૂપ. [ભાગ #438 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
439	Starvation Variant (Concept Part #439)	માઇક્રોકોટાસ્ક સતત આવતા રહે તો મેક્રોકોટાસ્ક અટકી પડે તે. [ભાગ #439 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
440	Prototypal Inheritance Variant (Concept Part #440)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #440 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
441	Prototype Chain Variant (Concept Part #441)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ લિંકમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #441 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
442	V8 Engine Variant (Concept Part #442)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #442 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]
443	JIT Compilation Variant (Concept Part #443)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ થવો. [ભાગ #443 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો.]

443			રીત. [ભાગ #443 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]
444	Inline Caching Variant (Concept Part #444)	વારંવાર વપરાતા ઓબ્જેક્ટ વોકેશનને એન્જિન આપમેળે ય સ્પીડ વધારે છે. [ભાગ #444 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
445	Hidden Classes Variant (Concept Part #445)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલ શેપ્સ. [ભાગ #445 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
446	Tree Shaking Variant (Concept Part #446)	બિન-પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમુવ કરવો. [ભાગ #446 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
447	CORS Policy Variant (Concept Part #447)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિમ્યુલેશન કરવાનો નિયમ. [ભાગ #447 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
448	XSS (Cross-Site Scripting) Variant (Concept Part #448)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીશિયસ જાવાસ્ક્રિપ્ટ ઇન્જન કરવાનો એટેક. [ભાગ #448 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
449	CSRF Variant (Concept Part #449)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ડિમાન્ડ મોકલવી. [ભાગ #449 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
450	Web Workers Variant (Concept Part #450)	ભારે કેમ્બ્રિજેશનના કોડને મેઈન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની ટેકનિક. [ભાગ #450 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
451	Temporal API Variant (Concept Part #451)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું વેબ API. [ભાગ #451 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
452	Explicit Resource Management Variant (Concept Part #452)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર કરવો. [ભાગ #452 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
453	Generator Function Variant (Concept Part #453)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરી શરૂ કરવાની શક્તિ હોય. [ભાગ #453 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
454	Decorators Variant (Concept Part #454)	@ સિન્ટેક્સ વાળું અડધું કીવર્ડ જે ક્લાસ કે મેથડની ક્ષમતાને ડેકોરેટ કરે છે. [ભાગ #454 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	

455	Proxy Object Variant (Concept Part #455)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન્સ (get, set) ને કસ્ટમાઇઝ કરી સિક્રેટ એજન્ટ. [ભાગ #455 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
456	Reflect API Variant (Concept Part #456)	ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી મેનિપ્યુલેટ કરવા માટેનું જાવસ્ક્રિપ્ટ મેથડ્સ. [ભાગ #456 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
457	Variable Shadowing Variant (Concept Part #457)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #457 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
458	Global Scope Variant (Concept Part #458)	આખા પ્રોગ્રામમાં 'ગ્લોબલ' સ્કોપમાં એક્સેસ કરી શકાતો સીંગલ ગ્લોબલ સ્કોપ. [ભાગ #458 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
459	Block Scope Variant (Concept Part #459)	માત્ર કરવી બ્રેકેટ 'બ્લોક' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #459 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
460	Function Scope Variant (Concept Part #460)	માત્ર જે-તે ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #460 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
461	Execution Context Variant (Concept Part #461)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #461 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
462	Call Stack Variant (Concept Part #462)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) સ્ટ્રક્ચર. [ભાગ #462 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
463	Hoisting Variant (Concept Part #463)	ડિફાલ્ટ રીલેશન કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સીધો ઉમેરવામાં આવે છે. [ભાગ #463 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
464	Temporal Dead Zone Variant (Concept Part #464)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો કોડનો ક્ષેત્ર. [ભાગ #464 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
465	Closure Variant (Concept Part #465)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ રાખે તે. [ભાગ #465 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	
466	IIFE Variant (Concept Part #466)	બંધાતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #466 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટો]	

467	Currying Variant (Concept Part #467)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શન ફેરવવું. [ભાગ #467 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
468	Memoization Variant (Concept Part #468)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી વધારવા. [ભાગ #468 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
469	Debouncing Variant (Concept Part #469)	ચુસ્ત ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવન્ટ ફાયર કરવો. [ભાગ #469 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
470	Throttling Variant (Concept Part #470)	કોઈ ઇવેન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને ચોક્કસ સમય રન કરવો. [ભાગ #470 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
471	Callback Hell Variant (Concept Part #471)	એકની અંદર એક અસંખ્ય કોલબેક લખવાથી બનતો અધરો કોડ સ્પ્રુઅર. [ભાગ #471 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
472	Inversion of Control Variant (Concept Part #472)	કોલબેક ફંક્શનનો કંટ્રોલ કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને ખામી. [ભાગ #472 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
473	Promise Variant (Concept Part #473)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેટી આપતું ઓપરેશન. [ભાગ #473 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
474	Event Loop Variant (Concept Part #474)	સિંગલ થ્રેડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચલાવવા. [ભાગ #474 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
475	Microtask Queue Variant (Concept Part #475)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં મળે છે. [ભાગ #475 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
476	Macrotask Queue Variant (Concept Part #476)	setTimeout અને ઇવેન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ લાઇન. [ભાગ #476 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
477	Starvation Variant (Concept Part #477)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે સ્થિતિ. [ભાગ #477 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
478	Prototypal Inheritance Variant (Concept Part #478)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #478 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
479	Prototype Chain Variant (Concept Part #479)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ (બિકમાં આપેલ શોષવાની પ્રક્રિયા). [ભાગ #479 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

480	V8 Engine Variant (Concept Part #480)	નૂનવ ઓપ્ટિમાઇઝેશન વધારાનું સુપર ફાસ્ટ જીવાલિફાઇઝેશન. [ભાગ #480 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
481	JIT Compilation Variant (Concept Part #481)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્સાઇલ કરવો. [ભાગ #481 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
482	Inline Caching Variant (Concept Part #482)	વારંવાર વપરાતા ઓબ્જેક્ટ પ્રોપર્ટીને એન્જિન આપમેળે જ સ્પીડ વધારે છે. [ભાગ #482 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
483	Hidden Classes Variant (Concept Part #483)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલ શેપ. [ભાગ #483 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
484	Tree Shaking Variant (Concept Part #484)	બિનપ્રયોજન થતી કોડ લાઇનને ઓપ્ટિમાઇઝેશન દરમિયાન કાઢી દેવા. [ભાગ #484 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
485	CORS Policy Variant (Concept Part #485)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા ટ્રાન્સફર સિદ્ધાંતની કસ્ટોમાઇઝેશન. [ભાગ #485 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
486	XSS (Cross-Site Scripting) Variant (Concept Part #486)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીમીડિયા જીવાલિફાઇઝેશન કરવાનો એટક. [ભાગ #486 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
487	CSRF Variant (Concept Part #487)	ચુસ્તની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ઇન્ફોર્મેશન મોકલવી. [ભાગ #487 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
488	Web Workers Variant (Concept Part #488)	ભારે કૅલ્ક્યુલેશનના કોડને મેઇન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડમાં ચલાવવાની ટેકનિક. [ભાગ #488 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
489	Temporal API Variant (Concept Part #489)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું વેબ API. [ભાગ #489 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
490	Explicit Resource Management Variant (Concept Part #490)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર થાય. [ભાગ #490 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
491	Generator Function Variant (Concept Part #491)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરતે શકાય. [ભાગ #491 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

492	Decorators Variant (Concept Part #492)	@ સિન્ટેક્સ વાળું અંદરૂત ફીચર જે ક્લાસ કે મેથડની ક્ષમતા [ભાગ #492 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
493	Proxy Object Variant (Concept Part #493)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન (get, set) ને કસ્ટમાઇઝ કરતો સિકેટ એજન્ટ. [ભાગ #493 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
494	Reflect API Variant (Concept Part #494)	ઓબ્જેક્ટ ઓપરેશનને સરળતાથી મેનિપ્યુલેટ કરવા માટેનું ઓબલ મેથડ્સ. [ભાગ #494 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
495	Variable Shadowing Variant (Concept Part #495)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #495 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
496	Global Scope Variant (Concept Part #496)	આખા પ્રોગ્રામમાં ગ્લોબલ સ્કોપમાં એક્સેસ કરી શકાતો સૌથી બૉલ્ડ સ્કોપ. [ભાગ #496 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
497	Block Scope Variant (Concept Part #497)	માત્ર કર્લી બ્રેકેટ્સ '}' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #497 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
498	Function Scope Variant (Concept Part #498)	માત્ર જે-તે ફંક્શનની અંદર જ માત્ર રહેતો સ્કોપ (var). [ભાગ #498 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
499	Execution Context Variant (Concept Part #499)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #499 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
500	Call Stack Variant (Concept Part #500)	ફંક્શન કોલિંગનો ઓર્ડર રાખતો LIFO (Last In First Out) ડ્રાઇવ. [ભાગ #500 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
501	Hoisting Variant (Concept Part #501)	ડિક્લેરેશન કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સૌથી ઉપર રહેવાની પ્રોપર્ટી. [ભાગ #501 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
502	Temporal Dead Zone Variant (Concept Part #502)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો કોડનો ક્ષેત્ર. [ભાગ #502 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
503	Closure Variant (Concept Part #503)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ યાદ રાખે તે. [ભાગ #503 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

504	IIFE Variant (Concept Part #504)	બનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #504 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
505	Currying Variant (Concept Part #505)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં રૂપાંતરિત કરવાનો પ્રકાર. [ભાગ #505 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
506	Memoization Variant (Concept Part #506)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) માં સ્ટોર કરવાનો પ્રકાર. [ભાગ #506 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
507	Debouncing Variant (Concept Part #507)	ચુસ્ત ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવન્ટ ફાયર કરવો. [ભાગ #507 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
508	Throttling Variant (Concept Part #508)	કોઈ ઇવન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને ચોક્કસ સમય અંતરે ફાયર કરવો. [ભાગ #508 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
509	Callback Hell Variant (Concept Part #509)	એકની અંદર એક અસંખ્ય કોલબેક લખવાથી બનતો અચૂક કોડ સ્ટ્રક્ચર. [ભાગ #509 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
510	Inversion of Control Variant (Concept Part #510)	કોલબેક ફંક્શનનો કંટ્રોલ બીજા કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને સોંપી દેવાનો પ્રકાર. [ભાગ #510 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
511	Promise Variant (Concept Part #511)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેંટટી આપતું ઓબ્જેક્ટ. [ભાગ #511 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
512	Event Loop Variant (Concept Part #512)	સિંગલ થ્રેડ ડોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચાલુ રાખી શકતો પ્રકાર. [ભાગ #512 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
513	Microtask Queue Variant (Concept Part #513)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં ચલાવવાનો પ્રકાર. [ભાગ #513 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
514	Macrotask Queue Variant (Concept Part #514)	setTimeout અને ઇવેન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડમાં ચલાવવાનો પ્રકાર. [ભાગ #514 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
515	Starvation Variant (Concept Part #515)	માઇક્રોટાસ્ક સતત આવતા રહે તે મેક્રોટાસ્ક અડકી પડે તેવો પ્રકાર. [ભાગ #515 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
516	Prototypal Inheritance Variant (Concept Part #516)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે તેવો પ્રકાર. [ભાગ #516 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

517	Prototype Chain Variant (Concept Part #517)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ બિંદુમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #517 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
518	V8 Engine Variant (Concept Part #518)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #518 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
519	JIT Compilation Variant (Concept Part #519)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ કરવાનો પ્રકાર. [ભાગ #519 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
520	Inline Caching Variant (Concept Part #520)	વારંવાર વપરાતા ઓબ્જેક્ટ લોકેશનને એન્જિન આપમેળે યાદ અપાડવાનો પ્રકાર. [ભાગ #520 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
521	Hidden Classes Variant (Concept Part #521)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલા હિડન ક્લાસ. [ભાગ #521 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
522	Tree Shaking Variant (Concept Part #522)	બિંદુ પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમોવ કરવો. [ભાગ #522 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
523	CORS Policy Variant (Concept Part #523)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિંક્રોનસીટી કરવાનો નિયમ. [ભાગ #523 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
524	XSS (Cross-Site Scripting) Variant (Concept Part #524)	વેબસાઇટમાં હેકર દ્વારા ખોટી મલીશિયસ જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #524 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
525	CSRF Variant (Concept Part #525)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ડિમાન્ડ મોકલવી. [ભાગ #525 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
526	Web Workers Variant (Concept Part #526)	ભારે કેમ્બ્રિજેશનના કોડને મેઇન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની ટેકનિક. [ભાગ #526 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
527	Temporal API Variant (Concept Part #527)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું વેબ API. [ભાગ #527 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
528	Explicit Resource Management	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર કરવાનો પ્રકાર. [ભાગ #528 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

529	Generator Function Variant (Concept Part #529)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરમાવવાથી ચાલુ રાખી શકાય. [ભાગ #529 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
530	Decorators Variant (Concept Part #530)	@ સિન્ટેક્સ વાળું અદ્યતન ફીચર જે ક્લાસ કે મેથડની ક્ષમતાને ડેકોરેટ કરવાનો પ્રકાર. [ભાગ #530 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
531	Proxy Object Variant (Concept Part #531)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન્સ (get, set) ને કસ્ટમાઇઝ કરતો સિક્રેટ એજન્ટ. [ભાગ #531 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
532	Reflect API Variant (Concept Part #532)	ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી મેનિપ્યુલેટ કરવા માટેના જવાબદાર મેથડ્સ. [ભાગ #532 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
533	Variable Shadowing Variant (Concept Part #533)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલને ઓવરલેપ કરે. [ભાગ #533 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
534	Global Scope Variant (Concept Part #534)	આખા પ્રોગ્રામમાં ગમે ત્યાંથી એક્સેસ કરી શકાતો સૌથી બૉલ્ડ સ્કોપ. [ભાગ #534 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
535	Block Scope Variant (Concept Part #535)	માત્ર કરલી બ્રેકેટ્સ '()' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #535 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
536	Function Scope Variant (Concept Part #536)	માત્ર જે-તે ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #536 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
537	Execution Context Variant (Concept Part #537)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #537 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
538	Call Stack Variant (Concept Part #538)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) ડાઇવ. [ભાગ #538 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
539	Hoisting Variant (Concept Part #539)	ડિફિનિશન કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સીધી ઉતારવાની પ્રોસેસ. [ભાગ #539 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
540	Temporal Dead Zone Variant	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો પ્રદેશ. [ભાગ #540 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

	(Concept Part #540)	સમયગાળો. [ભાગ #540 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
541	Closure Variant (Concept Part #541)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી રાખે તે. [ભાગ #541 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
542	IIFE Variant (Concept Part #542)	બનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #542 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
543	Currying Variant (Concept Part #543)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં ફેરવવું. [ભાગ #543 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
544	Memoization Variant (Concept Part #544)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવો. [ભાગ #544 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
545	Debouncing Variant (Concept Part #545)	ચુસ્ત ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવન્ટ ફાયર થાય. [ભાગ #545 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
546	Throttling Variant (Concept Part #546)	કોઈ ઇવન્ટ ને તેટલી વાર ટ્રિગર થાય પછી તેને ચોક્કસ સમયગાળા પર ફાયર કરવો. [ભાગ #546 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
547	Callback Hell Variant (Concept Part #547)	એકની અંદર એક અસંખ્ય કોલબેક લખવાથી બનતો અધરાંત કોડ સ્ટ્રક્ચર. [ભાગ #547 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
548	Inversion of Control Variant (Concept Part #548)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને ખામી. [ભાગ #548 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
549	Promise Variant (Concept Part #549)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેટી આપતું એન્ટિપાટી. [ભાગ #549 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
550	Event Loop Variant (Concept Part #550)	સિંગલ થ્રેડ ધોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચલાવવો. [ભાગ #550 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
551	Microtask Queue Variant (Concept Part #551)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં જ ચલાવવાનું છે. [ભાગ #551 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
552	Macrotask Queue Variant (Concept Part #552)	setTimeout અને ઇવન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ લાઇન. [ભાગ #552 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
553	Starvation Variant (Concept Part #553)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે સ્થિતિ. [ભાગ #553 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

	Part #553)	[ભાગ #553 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
554	Prototypal Inheritance Variant (Concept Part #554)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #554 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
555	Prototype Chain Variant (Concept Part #555)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ લિંકમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #555 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
556	V8 Engine Variant (Concept Part #556)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #556 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
557	JIT Compilation Variant (Concept Part #557)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ કરવો. [ભાગ #557 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
558	Inline Caching Variant (Concept Part #558)	વારંવાર વપરાતા ઓબ્જેક્ટ પ્રોપર્ટીઝને એન્જિન આપમેળે જ સ્પીડ વધારે છે. [ભાગ #558 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
559	Hidden Classes Variant (Concept Part #559)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલી રીપ્રેઝન્ટેશન. [ભાગ #559 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
560	Tree Shaking Variant (Concept Part #560)	બિન-ઇસ્યુસ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમોવ કરવો. [ભાગ #560 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
561	CORS Policy Variant (Concept Part #561)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિદ્ધાંતસર કરવાનો નિયમ. [ભાગ #561 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
562	XSS (Cross-Site Scripting) Variant (Concept Part #562)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલીશિયસ જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટક. [ભાગ #562 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
563	CSRF Variant (Concept Part #563)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ડિમાન્ડ મોકલવી. [ભાગ #563 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
564	Web Workers Variant (Concept Part #564)	ભારે કેલ્ક્યુલેશનના કોડને મેઇન થ્રેડ રોકા વગર બેકગ્રાઉન્ડમાં ચલાવવાની ટેકનિક. [ભાગ #564 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

588	Event Loop Variant (Concept Part #588)	સિંગલ થ્રેડેડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં વૂપ. [ભાગ #588 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
589	Microtask Queue Variant (Concept Part #589)	પ્રોમિસિસના કોડ રાખતી વાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં મળે છે. [ભાગ #589 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
590	Macrotask Queue Variant (Concept Part #590)	setTimeout અને ઇવેન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ. [ભાગ #590 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
591	Starvation Variant (Concept Part #591)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે. [ભાગ #591 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
592	Prototypal Inheritance Variant (Concept Part #592)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે. [ભાગ #592 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
593	Prototype Chain Variant (Concept Part #593)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ (બિકમાં આપેલ શોધવાની પ્રક્રિયા. [ભાગ #593 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
594	V8 Engine Variant (Concept Part #594)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #594 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
595	JIT Compilation Variant (Concept Part #595)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ કરવા. [ભાગ #595 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
596	Inline Caching Variant (Concept Part #596)	વારંવાર વપરાતા ઓબ્જેક્ટ વોકેશનને એન્જિન આપમેળે રાખી રાખીડ વધારે છે. [ભાગ #596 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
597	Hidden Classes Variant (Concept Part #597)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલ શેડો. [ભાગ #597 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
598	Tree Shaking Variant (Concept Part #598)	બિનપ્રયોજન વાળા કોડ પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમોવ કરવો. [ભાગ #598 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
599	CORS Policy Variant (Concept Part #599)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિદ્ધાંતો રાખી કરવાનો નિયમ. [ભાગ #599 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

600	XSS (Cross-Site Scripting) Variant (Concept Part #600)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીમીડિયા જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #600 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
601	CSRF Variant (Concept Part #601)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ઇન્જેક્ટ મોકલવો. [ભાગ #601 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
602	Web Workers Variant (Concept Part #602)	ભારે કેલ્ક્યુલેશનના કોડને મેઇન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની ટેકનિક. [ભાગ #602 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
603	Temporal API Variant (Concept Part #603)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું લેટર API. [ભાગ #603 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
604	Explicit Resource Management Variant (Concept Part #604)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર કરવો. [ભાગ #604 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
605	Generator Function Variant (Concept Part #605)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરી શરૂ કરવા. [ભાગ #605 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
606	Decorators Variant (Concept Part #606)	@ સિન્ટેક્સ વાળું એડશુટ ફોર જે ક્લાસ કે મેથડની ક્ષમતા ડેકોરેટ કરવા. [ભાગ #606 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
607	Proxy Object Variant (Concept Part #607)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન્સ (get, set) ને કસ્ટમાઇઝ કરતો સિક્રેટ એજન્ટ. [ભાગ #607 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
608	Reflect API Variant (Concept Part #608)	ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી મેનિપ્યુલેટ કરવા માટેના ઓબ્જેક્ટ મેથડ્સ. [ભાગ #608 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
609	Variable Shadowing Variant (Concept Part #609)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #609 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
610	Global Scope Variant (Concept Part #610)	આખા પ્રોગ્રામમાં નમે ત્યાંથી એક્સેસ કરી શકતો સૌથી બહારનો સ્કોપ. [ભાગ #610 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
611	Block Scope Variant (Concept Part #611)	માત્ર કરવા બેસીસ '}' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #611 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

612	Function Scope Variant (Concept Part #612)	માત્ર જેનો ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #612 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
613	Execution Context Variant (Concept Part #613)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #613 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
614	Call Stack Variant (Concept Part #614)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) ડાઇવ. [ભાગ #614 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
615	Hoisting Variant (Concept Part #615)	ડિફરેન્શિયલ કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સૌથી ઉપર રહેવાની પ્રોપર્ટી. [ભાગ #615 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
616	Temporal Dead Zone Variant (Concept Part #616)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો સમયગાળો. [ભાગ #616 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
617	Closure Variant (Concept Part #617)	ફંક્શન તેના પેરેન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી યાદ રાખે તે. [ભાગ #617 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
618	IIFE Variant (Concept Part #618)	બનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #618 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
619	Currying Variant (Concept Part #619)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં રૂપાંતરિત કરવું. [ભાગ #619 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
620	Memoization Variant (Concept Part #620)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવું. [ભાગ #620 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
621	Debouncing Variant (Concept Part #621)	ચુઝર ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવેન્ટ ફાયર થાય. [ભાગ #621 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
622	Throttling Variant (Concept Part #622)	કોઈ ઇવેન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને થોડા સમયમાં ફોલોઅપ થઈ શકે. [ભાગ #622 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
623	Callback Hell Variant (Concept Part #623)	એકની અંદર એક અસંખ્ય કોલબેક વખવાથી બનતો અધરાય. [ભાગ #623 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
624	Inversion of Control Variant (Concept Part #624)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને સોંપી દેવો. [ભાગ #624 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

625	Promise Variant (Concept Part #625)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેન્જી આપતું એન્ટિપ્રાઇઝ. [ભાગ #625 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
626	Event Loop Variant (Concept Part #626)	સિંગલ થ્રેડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચાલુ રાખવું. [ભાગ #626 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
627	Microtask Queue Variant (Concept Part #627)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં મળે છે. [ભાગ #627 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
628	Macrotask Queue Variant (Concept Part #628)	setTimeout અને ઇવેન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ લાઇન. [ભાગ #628 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
629	Starvation Variant (Concept Part #629)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે. [ભાગ #629 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
630	Prototypal Inheritance Variant (Concept Part #630)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #630 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
631	Prototype Chain Variant (Concept Part #631)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ લિંકમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #631 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
632	V8 Engine Variant (Concept Part #632)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #632 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
633	JIT Compilation Variant (Concept Part #633)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ કરવો. [ભાગ #633 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
634	Inline Caching Variant (Concept Part #634)	વારંવાર વપરાતા ઓબ્જેક્ટ વોકેશનને એન્જિન આપમેળે જ સ્પોઇલ વધારે છે. [ભાગ #634 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
635	Hidden Classes Variant (Concept Part #635)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલી શોષા. [ભાગ #635 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
636	Tree Shaking Variant (Concept Part #636)	બિલ્ડ પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમ કરવો. [ભાગ #636 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
637	CORS Policy Variant (Concept Part #637)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિફ્રેસ્ટ સિદ્ધાંતની રીમિટ. [ભાગ #637 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

	Part #637)	કરવાનો નિયમ. [ભાગ #637 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
638	XSS (Cross-Site Scripting) Variant (Concept Part #638)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલીશિયસ જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #638 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
639	CSRF Variant (Concept Part #639)	યુઝરની પરમિશન વગર તેના બાઉઝર સેશન દ્વારા ખોટી વિનંતી મોકલવી. [ભાગ #639 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
640	Web Workers Variant (Concept Part #640)	ભારે કેમ્બ્રિજેશનના કોડને મેઈન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડમાં ચલાવવાની ટેકનિક. [ભાગ #640 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
641	Temporal API Variant (Concept Part #641)	જૂના Date ઓબ્જેક્ટને બદલે ટાઈમ-ઝોન ફિક્સ કરતું વેબ API. [ભાગ #641 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
642	Explicit Resource Management Variant (Concept Part #642)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતા જ મેમરી ઓટોમેટિક ક્લિયર થાય. [ભાગ #642 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
643	Generator Function Variant (Concept Part #643)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વ્યૂથેબી રોકી અને ફરેશ કરી શકાય. [ભાગ #643 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
644	Decorators Variant (Concept Part #644)	@ સિન્ટેક્સ વાળું અડજુન્ટ ફીચર જે ક્લાસ કે મેથડની ક્ષમતા ટાઈપ કરે છે. [ભાગ #644 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
645	Proxy Object Variant (Concept Part #645)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન્સ (get, set) ને કસ્ટમાઇઝ કરતો સિંકેટ એજન્ટ. [ભાગ #645 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
646	Reflect API Variant (Concept Part #646)	ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી મેનિપ્યુલેટ કરવા માટેના ઓબ્જેક્ટ મેથડ્સ. [ભાગ #646 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
647	Variable Shadowing Variant (Concept Part #647)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #647 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
648	Global Scope Variant (Concept Part #648)	આખા પ્રોગ્રામમાં ગ્લોબલ સ્કોપમાં ફંક્શન કરી શકાતો સૌથી બૉલ્ડ સ્કોપ. [ભાગ #648 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

649	Block Scope Variant (Concept Part #649)	માત્ર કરવી બ્લૉક સ્કોપ '}' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #649 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
650	Function Scope Variant (Concept Part #650)	માત્ર જે-તે ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #650 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
651	Execution Context Variant (Concept Part #651)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #651 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
652	Call Stack Variant (Concept Part #652)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) ડેટા સ્ટ્રક્ચર. [ભાગ #652 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
653	Hoisting Variant (Concept Part #653)	ડિફિનિશન કોડ રન થતા પહેલાં આપમેળે સ્કોપમાં સીધી ઉપર લેવામાં આવે છે. [ભાગ #653 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
654	Temporal Dead Zone Variant (Concept Part #654)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો કોડનો ક્ષેત્ર. [ભાગ #654 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
655	Closure Variant (Concept Part #655)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ યાદ રાખે છે. [ભાગ #655 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
656	IIFE Variant (Concept Part #656)	બંધાતી સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવું ફંક્શન. [ભાગ #656 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
657	Currying Variant (Concept Part #657)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં રૂપાંતરિત કરવાનો ટેકનિક. [ભાગ #657 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
658	Memoization Variant (Concept Part #658)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવાનો ટેકનિક. [ભાગ #658 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
659	Debouncing Variant (Concept Part #659)	યુઝર ઇનપુટ ટાઇપિંગ બંધ કરે તેના પછી સમય પછી જ ઇવેન્ટ ફાયર થાય. [ભાગ #659 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
660	Throttling Variant (Concept Part #660)	કોઈ ઇવેન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને ચોક્કસ સમયગાળામાં ફાયર થઈ શકે. [ભાગ #660 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
661	Callback Hell Variant (Concept Part #661)	એકની અંદર એક અસંખ્ય કોલબેક વાળાવાળી બનતી અધરાંત કોડ સ્ટ્રક્ચર. [ભાગ #661 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

662	Inversion of Control Variant (Concept Part #662)	કોષબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને ખામી. [ભાગ #662 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
663	Promise Variant (Concept Part #663)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેટી આપતું એક પ્રમીસ. [ભાગ #663 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
664	Event Loop Variant (Concept Part #664)	સિંગલ થ્રેડડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં વ્યૂપ. [ભાગ #664 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
665	Microtask Queue Variant (Concept Part #665)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં મળે છે. [ભાગ #665 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
666	Macrotask Queue Variant (Concept Part #666)	setTimeout અને ઇવેન્ટુસનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ લાઇન. [ભાગ #666 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
667	Starvation Variant (Concept Part #667)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે. [ભાગ #667 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
668	Prototypal Inheritance Variant (Concept Part #668)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #668 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
669	Prototype Chain Variant (Concept Part #669)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ લિંકમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #669 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
670	V8 Engine Variant (Concept Part #670)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #670 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
671	JIT Compilation Variant (Concept Part #671)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ. [ભાગ #671 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
672	Inline Caching Variant (Concept Part #672)	વારંવાર વપરાતા ઓબ્જેક્ટ લોકેશનને એન્જિન આપમેળે યાદ રાખીડ વધારે છે. [ભાગ #672 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
673	Hidden Classes Variant (Concept Part #673)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલ શોર્ટ. [ભાગ #673 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
674	Tree Shaking Variant (Concept Part #674)	બિન-પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમુવ. [ભાગ #674 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

675	CORS Policy Variant (Concept Part #675)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિદ્ધાંતસભી કરવાનો નિયમ. [ભાગ #675 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
676	XSS (Cross-Site Scripting) Variant (Concept Part #676)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીમીડિયા જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #676 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
677	CSRF Variant (Concept Part #677)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી (ફેલ્સ) મોકલવી. [ભાગ #677 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
678	Web Workers Variant (Concept Part #678)	ભારે કેલ્ક્યુલેશનના કોડને મેઇન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની ટેકનિક. [ભાગ #678 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
679	Temporal API Variant (Concept Part #679)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું ચેરેટી API. [ભાગ #679 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
680	Explicit Resource Management Variant (Concept Part #680)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર. [ભાગ #680 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
681	Generator Function Variant (Concept Part #681)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરી શરૂ કરી શકાય. [ભાગ #681 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
682	Decorators Variant (Concept Part #682)	@ સિન્ટેક્સ વાળું અડધુત ફીચર જે ક્લાસ કે મેથડની ક્ષમતા ડેકોરેટ. [ભાગ #682 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
683	Proxy Object Variant (Concept Part #683)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન (get, set) ને કસ્ટમાઇઝ કરી સિક્રેટ એજન્ટ. [ભાગ #683 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
684	Reflect API Variant (Concept Part #684)	ઓબ્જેક્ટ ઓપરેશનનું સરળતાથી મેનિપ્યુલેટ કરવા માટેનું ડિફાલ્ટ મેથડ્સ. [ભાગ #684 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
685	Variable Shadowing Variant (Concept Part #685)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ ઓવરલેપ કરે. [ભાગ #685 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

686	Global Scope Variant (Concept Part #686)	આખા પ્રોગ્રામમાં ગમે ત્યાંથી એક્સેસ કરી શકાતી સૌથી બહારની સ્કોપ. [ભાગ #686 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
687	Block Scope Variant (Concept Part #687)	માત્ર કરવા બેસીસ 'બ્લોક' ની અંદર જ કામ કરતી સ્કોપ (let, const). [ભાગ #687 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
688	Function Scope Variant (Concept Part #688)	માત્ર જે-તે ફંક્શનની અંદર જ માન્ય રહેતી સ્કોપ (var). [ભાગ #688 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
689	Execution Context Variant (Concept Part #689)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #689 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
690	Call Stack Variant (Concept Part #690)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) ડાઇવ. [ભાગ #690 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
691	Hoisting Variant (Concept Part #691)	ડિફાઇનીશન કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સૌથી ઉપર રહેવાની પ્રોપર્સી. [ભાગ #691 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
692	Temporal Dead Zone Variant (Concept Part #692)	let/const વેરીએબલ બનાવવા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો કોડનો ક્ષેત્ર. [ભાગ #692 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
693	Closure Variant (Concept Part #693)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ યાદ રાખે તે. [ભાગ #693 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
694	IIFE Variant (Concept Part #694)	બંનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #694 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
695	Currying Variant (Concept Part #695)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં રૂપાંતરિત કરવાનો પ્રકાર. [ભાગ #695 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
696	Memoization Variant (Concept Part #696)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવાનો પ્રકાર. [ભાગ #696 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
697	Debouncing Variant (Concept Part #697)	થ્રુઝર ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવેન્ટ ફાયર થાય તેવો પ્રકાર. [ભાગ #697 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
698	Throttling Variant (Concept Part #698)	કોઈ ઇવેન્ટ ગમે તેટલી વાર ફિર થાય પણ તેને ચોક્કસ સમયગાળામાં ફાયર થાય તેવો પ્રકાર. [ભાગ #698 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

699	Callback Hell Variant (Concept Part #699)	એકની અંદર એક અસંખ્ય કોલબેક લખવાથી બનતો અધરાયવો કોડ સ્ટ્રક્ચર. [ભાગ #699 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
700	Inversion of Control Variant (Concept Part #700)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને સોંપી દેવાનો પ્રકાર. [ભાગ #700 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
701	Promise Variant (Concept Part #701)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરંટી આપતું એન્જિન. [ભાગ #701 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
702	Event Loop Variant (Concept Part #702)	સિંગલ થ્રેડ્ડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચાલુ રાખવાનો પ્રકાર. [ભાગ #702 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
703	Microtask Queue Variant (Concept Part #703)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં મળે છે. [ભાગ #703 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
704	Macrotask Queue Variant (Concept Part #704)	setInterval અને setTimeoutનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડમાં. [ભાગ #704 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
705	Starvation Variant (Concept Part #705)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તેવો પ્રકાર. [ભાગ #705 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
706	Prototypal Inheritance Variant (Concept Part #706)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે તેવો પ્રકાર. [ભાગ #706 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
707	Prototype Chain Variant (Concept Part #707)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ લિંકમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #707 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
708	V8 Engine Variant (Concept Part #708)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #708 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
709	JIT Compilation Variant (Concept Part #709)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ કરવાનો પ્રકાર. [ભાગ #709 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
710	Inline Caching Variant (Concept Part #710)	વારંવાર વપરાતા ઓબ્જેક્ટ પ્રોપર્ટીઝને એન્જિન આપમેળે યાદ રાખી રાખે છે. [ભાગ #710 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
711	Hidden Classes Variant	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલો એન્જિન. [ભાગ #711 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

	(Concept Part #711)	શેડિંગ [ભાગ #711 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
712	Tree Shaking Variant (Concept Part #712)	બિન-પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમુવ કરવો. [ભાગ #712 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
713	CORS Policy Variant (Concept Part #713)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિંક્રોનસી કરવાનો નિયમ. [ભાગ #713 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
714	XSS (Cross-Site Scripting) Variant (Concept Part #714)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીમીડિયા જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #714 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
715	CSRF Variant (Concept Part #715)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ફોર્મ સબમિટ કરવી. [ભાગ #715 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
716	Web Workers Variant (Concept Part #716)	ભારે કેલ્ક્યુલેશનના કોડને મેઈન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડમાં ચલાવવાની ટેકનિક. [ભાગ #716 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
717	Temporal API Variant (Concept Part #717)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું વેબ API. [ભાગ #717 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
718	Explicit Resource Management Variant (Concept Part #718)	using કીવર્ડ દ્વારા સ્કોપ પૂરી થતાં જ મેમરી ઓટોમેટિક ક્લિયર થાય. [ભાગ #718 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
719	Generator Function Variant (Concept Part #719)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરમાવવા શકાય. [ભાગ #719 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
720	Decorators Variant (Concept Part #720)	@ સિન્ટેક્સ વાળું અડજુન્ટ ફીચર જે ક્લાસ કે મેથડની ક્ષમતા ડેકોરેટ કરે છે. [ભાગ #720 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
721	Proxy Object Variant (Concept Part #721)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન (get, set) ને કસ્ટમાઇઝ કરી સિકેટ એજન્ટ. [ભાગ #721 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
722	Reflect API Variant (Concept Part #722)	ઓબ્જેક્ટ ઓપરેશનને સરળતાથી મેનિપ્યુલેટ કરવા માટેનું ડિફાલ્ટ મેથડ્સ. [ભાગ #722 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

723	Variable Shadowing Variant (Concept Part #723)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #723 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
724	Global Scope Variant (Concept Part #724)	આખા પ્રોગ્રામમાં જમે ત્યાંથી એક્સેસ કરી શકાતો સૌથી બૉલ્ડ સ્કોપ. [ભાગ #724 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
725	Block Scope Variant (Concept Part #725)	માત્ર કરવી બેસીસ 'બ્લોક' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #725 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

Advanced Compilation, V8 Engine & Optimization (ક્રમ ૭૫૧ થી ૮૭૦)

JIT કમ્પાઇલેશન, ઇનલાઇન કેશિંગ, હિટન ક્વાસીસ, ટ્રી શીડિંગ અને મેમરી હીપ મેનેજમેન્ટ.

ક્રમ	JavaScript Concept / Term	સરળ ગુજરાતી સમજૂતી (Detailed Explanation)
726	Function Scope Variant (Concept Part #726)	માત્ર જે-તે ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #726 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
727	Execution Context Variant (Concept Part #727)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #727 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
728	Call Stack Variant (Concept Part #728)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) ડાઇવ. [ભાગ #728 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
729	Hoisting Variant (Concept Part #729)	ડિક્લેરેશન કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સોંપી દેવા. [ભાગ #729 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
730	Temporal Dead Zone Variant (Concept Part #730)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો ક્ષેત્ર. [ભાગ #730 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
731	Closure Variant (Concept Part #731)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ ધરાવે છે. [ભાગ #731 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
732	IIFE Variant (Concept Part #732)	બંનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવું ફંક્શન. [ભાગ #732 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
733	Currying Variant (Concept Part #733)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં રીવાઇટ કરવો. [ભાગ #733 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

ફેરવવું. [ભાગ #733 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]		
734	Memoization Variant (Concept Part #734)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવાનો વધારાનો [ભાગ #734 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
735	Debouncing Variant (Concept Part #735)	ચુસ્ત ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવેન્ટ ફાયર થાય. [ભાગ #735 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
736	Throttling Variant (Concept Part #736)	કોઈ ઇવેન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને ચોક્કસ સમયગાળામાં ફાયર થાય. [ભાગ #736 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
737	Callback Hell Variant (Concept Part #737)	એકની અંદર એક અસંખ્ય કોલબેક વાળા કોડને વાળી બનાવવાને બદલે કોડ સ્વચ્છ અને વાંચવામાં સરળ બનાવવાનો [ભાગ #737 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
738	Inversion of Control Variant (Concept Part #738)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને સોંપી દેવાનો [ભાગ #738 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
739	Promise Variant (Concept Part #739)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેંટી આપતું એક વધારાનો [ભાગ #739 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
740	Event Loop Variant (Concept Part #740)	સિંગલ થ્રેડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચલાવવાનો [ભાગ #740 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
741	Microtask Queue Variant (Concept Part #741)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં ચલાવવાનો [ભાગ #741 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
742	Macrotask Queue Variant (Concept Part #742)	setTimeout અને ઇવેન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડમાં ચલાવવાનો [ભાગ #742 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
743	Starvation Variant (Concept Part #743)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે સ્થિતિનો [ભાગ #743 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
744	Prototypal Inheritance Variant (Concept Part #744)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે તે સ્થિતિનો [ભાગ #744 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
745	Prototype Chain Variant (Concept Part #745)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ ચેઇનમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #745 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

746	V8 Engine Variant (Concept Part #746)	નૂતન કોમ્પાઇલરમાં વધારાનું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન [ભાગ #746 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
747	JIT Compilation Variant (Concept Part #747)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કંપાઇલ કરવાનો [ભાગ #747 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
748	Inline Caching Variant (Concept Part #748)	વારંવાર વપરાતા ઓબ્જેક્ટ પ્રોપર્ટીને એન્જિન આપમેળે જ સ્પીડ વધારે છે. [ભાગ #748 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
749	Hidden Classes Variant (Concept Part #749)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલી શીપ્સ. [ભાગ #749 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
750	Tree Shaking Variant (Concept Part #750)	બિન-ઇસ્યુ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમુ કરવાનો [ભાગ #750 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
751	CORS Policy Variant (Concept Part #751)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિફ્રેશ કરવાનો નિયમ. [ભાગ #751 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
752	XSS (Cross-Site Scripting) Variant (Concept Part #752)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીમીડિયા જાવાસ્ક્રિપ્ટ ઇમ્પ્લેમેન્ટ કરવાનો એટક. [ભાગ #752 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
753	CSRF Variant (Concept Part #753)	ચુસ્તની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ઇમેઇલ મોકલવાનો [ભાગ #753 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
754	Web Workers Variant (Concept Part #754)	ભારે કૅલ્ક્યુલેશનના કોડને મેઇન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડમાં ચલાવવાનો ટેકનિક. [ભાગ #754 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
755	Temporal API Variant (Concept Part #755)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું નવું API. [ભાગ #755 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
756	Explicit Resource Management Variant (Concept Part #756)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર કરવાનો [ભાગ #756 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
757	Generator Function Variant (Concept Part #757)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરી શરૂ કરવાનો [ભાગ #757 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

758	Decorators Variant (Concept Part #758)	@ સિન્ટેક્સ વાળું અંદરૂત ફીચર જે ક્લાસ કે મેથડની ક્ષમતાને વધારે છે. [ભાગ #758 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
759	Proxy Object Variant (Concept Part #759)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન (get, set) ને કસ્ટમાઇઝ કરતો સિકેટ એજન્ટ. [ભાગ #759 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
760	Reflect API Variant (Concept Part #760)	ઓબ્જેક્ટ ઓપરેશનને સરળતાથી મેનિપ્યુલેટ કરવા માટેનું ઓબલ મેથડ્સ. [ભાગ #760 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
761	Variable Shadowing Variant (Concept Part #761)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #761 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
762	Global Scope Variant (Concept Part #762)	આખા પ્રોગ્રામમાં ગ્લોબલ સ્કોપમાં એક્સેસ કરી શકાતો સૌથી બૉલ્ડ સ્કોપ. [ભાગ #762 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
763	Block Scope Variant (Concept Part #763)	માત્ર કર્લી બ્રેકેટ્સ '}' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #763 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
764	Function Scope Variant (Concept Part #764)	માત્ર જે-તે ફંક્શનની અંદર જ માત્ર રહેતો સ્કોપ (var). [ભાગ #764 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
765	Execution Context Variant (Concept Part #765)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #765 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
766	Call Stack Variant (Concept Part #766)	ફંક્શન કોલિંગનો ઓર્ડર રાખતો LIFO (Last In First Out) ડ્રાઇવ. [ભાગ #766 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
767	Hoisting Variant (Concept Part #767)	ડિક્લેરેશન કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સૌથી ઉપર રહેવાની પ્રોપર્ટી. [ભાગ #767 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
768	Temporal Dead Zone Variant (Concept Part #768)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો કોડનો ક્ષેત્ર. [ભાગ #768 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
769	Closure Variant (Concept Part #769)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ યાદ રાખે તે. [ભાગ #769 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

770	IIFE Variant (Concept Part #770)	બનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #770 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
771	Currying Variant (Concept Part #771)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં રૂપાંતરિત કરવાનો પ્રકાર. [ભાગ #771 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
772	Memoization Variant (Concept Part #772)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવો. [ભાગ #772 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
773	Debouncing Variant (Concept Part #773)	ચુસ્ત ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવન્ટ ફાયર કરવો. [ભાગ #773 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
774	Throttling Variant (Concept Part #774)	કોઈ ઇવન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને ચોક્કસ સમયગાળામાં ફાયર કરવો. [ભાગ #774 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
775	Callback Hell Variant (Concept Part #775)	એકની અંદર એક અસંખ્ય કોલબેક લખવાથી બનતો અધૂરો કોડ સ્ટ્રક્ચર. [ભાગ #775 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
776	Inversion of Control Variant (Concept Part #776)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને ખામી. [ભાગ #776 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
777	Promise Variant (Concept Part #777)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેકીંગ આપતું ઓબ્જેક્ટ. [ભાગ #777 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
778	Event Loop Variant (Concept Part #778)	સિંગલ થ્રેડ્ડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચાલુ રાખવો. [ભાગ #778 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
779	Microtask Queue Variant (Concept Part #779)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં મળે છે. [ભાગ #779 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
780	Macrotask Queue Variant (Concept Part #780)	setTimeout અને setIntervalનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ ક્વિયુ. [ભાગ #780 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
781	Starvation Variant (Concept Part #781)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અડકી પડે તે સ્થિતિ. [ભાગ #781 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
782	Prototypal Inheritance Variant (Concept Part #782)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #782 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

783	Prototype Chain Variant (Concept Part #783)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ બિંદુમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #783 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
784	V8 Engine Variant (Concept Part #784)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #784 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
785	JIT Compilation Variant (Concept Part #785)	કોડ રન થતી વખતે જ તેને કમ્પાઇલ કરીને મશીન કોડમાં ફેરવે છે. [ભાગ #785 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
786	Inline Caching Variant (Concept Part #786)	વારંવાર વપરાતા ઓબ્જેક્ટ લોકેશનને એન્જિન આપમેળે યાદ રાખી રાખે છે. [ભાગ #786 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
787	Hidden Classes Variant (Concept Part #787)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલા હિડન ક્લાસ. [ભાગ #787 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
788	Tree Shaking Variant (Concept Part #788)	બિન-પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિન-જરૂરી કોડ ઓટોમેટિક રીમુવ કરવો. [ભાગ #788 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
789	CORS Policy Variant (Concept Part #789)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિદ્ધાંતો લેવાનો નિયમ. [ભાગ #789 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
790	XSS (Cross-Site Scripting) Variant (Concept Part #790)	વેબસાઇટમાં હેકર દ્વારા ખોટી મલ્ટીશિયસ જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #790 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
791	CSRF Variant (Concept Part #791)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ફોર્મ સબમિટ મોકલવી. [ભાગ #791 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
792	Web Workers Variant (Concept Part #792)	ભારે કેમ્બ્રિજેશનના કોડને મેઇન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડમાં ચલાવવાની ટેકનિક. [ભાગ #792 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
793	Temporal API Variant (Concept Part #793)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું વેબ API. [ભાગ #793 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
794	Explicit Resource Management	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર કરવો. [ભાગ #794 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

794	Variable Shadowing Variant (Concept Part #794)	જ્યાં સુધી સ્કોપ અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ. [ભાગ #794 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
795	Generator Function Variant (Concept Part #795)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરી શરૂ કરી શકાય. [ભાગ #795 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
796	Decorators Variant (Concept Part #796)	@ સિન્ટેક્સ વાળું અડધું ફીચર જે ક્લાસ કે મેથડની ક્ષમતાને ડેકોરેટ કરે છે. [ભાગ #796 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
797	Proxy Object Variant (Concept Part #797)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન્સ (get, set) ને કસ્ટમાઇઝ કરી સિક્રેટ એજન્ટ. [ભાગ #797 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
798	Reflect API Variant (Concept Part #798)	ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી મેનિપ્યુલેટ કરવા માટેના જવાબદાર મેથડ્સ. [ભાગ #798 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
799	Variable Shadowing Variant (Concept Part #799)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલને ઓવરલેપ કરે. [ભાગ #799 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
800	Global Scope Variant (Concept Part #800)	આખા પ્રોગ્રામમાં જમે ત્યાંથી એક્સેસ કરી શકાતો સૌથી બૉલ્ડ સ્કોપ. [ભાગ #800 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
801	Block Scope Variant (Concept Part #801)	માત્ર કરલી બ્રેકેટ્સ '()' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #801 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
802	Function Scope Variant (Concept Part #802)	માત્ર જે-તે ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #802 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
803	Execution Context Variant (Concept Part #803)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #803 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
804	Call Stack Variant (Concept Part #804)	ફંક્શન કોલિંગનો ઓર્ડર ટોપની LIFO (Last In First Out) ડાઇવ. [ભાગ #804 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
805	Hoisting Variant (Concept Part #805)	ડિફિનિશન કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સીધી ઉતરી જાય છે. [ભાગ #805 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
806	Temporal Dead Zone Variant	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય. [ભાગ #806 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

807	(Concept Part #806)	સમયગાળો. [ભાગ #806 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન]
	Closure Variant (Concept Part #807)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી રાખે તે. [ભાગ #807 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન]
808	IIFE Variant (Concept Part #808)	બનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #808 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
809	Currying Variant (Concept Part #809)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં ફેરવવું. [ભાગ #809 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
810	Memoization Variant (Concept Part #810)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવો. [ભાગ #810 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
811	Debouncing Variant (Concept Part #811)	ચૂઝર ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવન્ટ ફાયર થાય. [ભાગ #811 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
812	Throttling Variant (Concept Part #812)	કોઈ ઇવન્ટ ને તેટલી વાર ટ્રિગર થાય પછી તેને ચોક્કસ સમયગાળા પર ફાયર કરવો. [ભાગ #812 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
813	Callback Hell Variant (Concept Part #813)	એકની અંદર એક અસંખ્ય કોલબેક લખવાથી બનતો અધરાંત કોડ સ્ટ્રક્ચર. [ભાગ #813 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
814	Inversion of Control Variant (Concept Part #814)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને સોંપી દેવો. [ભાગ #814 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
815	Promise Variant (Concept Part #815)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેટી આપવું એવાં પ્રોમિસ. [ભાગ #815 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
816	Event Loop Variant (Concept Part #816)	સિંગલ થ્રેડ્ડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચલાવવા. [ભાગ #816 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
817	Microtask Queue Variant (Concept Part #817)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં જ ચલાવવા. [ભાગ #817 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
818	Macrotask Queue Variant (Concept Part #818)	setTimeout અને ઇવન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ લાઇન. [ભાગ #818 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
819	Starvation Variant (Concept	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે

820	Part #819)	[ભાગ #819 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
	Prototypal Inheritance Variant (Concept Part #820)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #820 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
821	Prototype Chain Variant (Concept Part #821)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ લિંકમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #821 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
822	V8 Engine Variant (Concept Part #822)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #822 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
823	JIT Compilation Variant (Concept Part #823)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ કરવો. [ભાગ #823 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
824	Inline Caching Variant (Concept Part #824)	વારંવાર વપરાતા ઓબ્જેક્ટ લોકેશનને એન્જિન આપમેળે યાદ રાખી રાખે છે. [ભાગ #824 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
825	Hidden Classes Variant (Concept Part #825)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલાં હોપ્સ. [ભાગ #825 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
826	Tree Shaking Variant (Concept Part #826)	બિન-પ્રોસેસ થયેલાં પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમોવ કરવો. [ભાગ #826 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
827	CORS Policy Variant (Concept Part #827)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિદ્ધાંતસર કરવાનો નિયમ. [ભાગ #827 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
828	XSS (Cross-Site Scripting) Variant (Concept Part #828)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલીશિયસ જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટક. [ભાગ #828 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
829	CSRF Variant (Concept Part #829)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ડિમાન્ડ મોકલવી. [ભાગ #829 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
830	Web Workers Variant (Concept Part #830)	ભારે કેલ્ક્યુલેશનના કોડને મેઇન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડમાં ચલાવવાની ટેકનિક. [ભાગ #830 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

831	Temporal API Variant (Concept Part #831)	જૂના Date ઓબ્જેક્ટને બદલે ટાઈમ-ઝોન ફિક્સ કરતું વેટેસ API. [ભાગ #831 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
832	Explicit Resource Management Variant (Concept Part #832)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર થાય. [ભાગ #832 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
833	Generator Function Variant (Concept Part #833)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરમાવે શકાય. [ભાગ #833 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
834	Decorators Variant (Concept Part #834)	@ સિન્ટેક્સ વાળું અડજુન્ટ ફોયર જે ક્લાસ કે મેથડની ક્લસ/મેથડ પર ઓપરેશન કરે. [ભાગ #834 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
835	Proxy Object Variant (Concept Part #835)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન (get, set) ને કસ્ટમાઇઝ કરતો સિકેટ એજન્ટ. [ભાગ #835 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
836	Reflect API Variant (Concept Part #836)	ઓબ્જેક્ટ ઓપરેશનને સરળતાથી મેનિપ્યુલેટ કરવા માટેના ઝવોબલ મેથડ્સ. [ભાગ #836 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
837	Variable Shadowing Variant (Concept Part #837)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #837 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
838	Global Scope Variant (Concept Part #838)	આખા પ્રોગ્રામમાં ગ્લોબલ એક્સેસ કરી શકતો સૌથી બહારનો સ્કોપ. [ભાગ #838 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
839	Block Scope Variant (Concept Part #839)	માત્ર કર્લી બ્રેકેટ્સ '}' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #839 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
840	Function Scope Variant (Concept Part #840)	માત્ર જે-તે ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #840 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
841	Execution Context Variant (Concept Part #841)	જાવસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #841 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
842	Call Stack Variant (Concept Part #842)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) ડાઈવ. [ભાગ #842 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
843	Hoisting Variant (Concept Part #843)	ડિક્લેરેશન કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સૌથી ઉપર લઈ જાય. [ભાગ #843 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

843	Temporal Dead Zone Variant (Concept Part #844)	રહેવાની પ્રોસેસ. [ભાગ #843 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
844	Temporal Dead Zone Variant (Concept Part #844)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો સમયગાળો. [ભાગ #844 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
845	Closure Variant (Concept Part #845)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ યાદ રાખે તે. [ભાગ #845 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
846	IIFE Variant (Concept Part #846)	બનાતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવું ફંક્શન. [ભાગ #846 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
847	Currying Variant (Concept Part #847)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શન બનાવવાનું. [ભાગ #847 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
848	Memoization Variant (Concept Part #848)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવો. [ભાગ #848 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
849	Debouncing Variant (Concept Part #849)	યુઝર ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવન્ટ ફાયર થાય. [ભાગ #849 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
850	Throttling Variant (Concept Part #850)	કોઈ ઇવેન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને ચોક્કસ સમયગાળામાં ફાયર થઈ શકે. [ભાગ #850 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
851	Callback Hell Variant (Concept Part #851)	એકની અંદર એક અસંખ્ય કોલબેક લખવાથી બનતો અધરાંત કોડ સ્ટ્રક્ચર. [ભાગ #851 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
852	Inversion of Control Variant (Concept Part #852)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને ખામી. [ભાગ #852 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
853	Promise Variant (Concept Part #853)	અસિન્ક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની નેરેટી આપતું એન્જિન. [ભાગ #853 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
854	Event Loop Variant (Concept Part #854)	સિંગલ થ્રેડ હોવા છતાં અસિન્ક્રોનસ કામોને બેકગ્રાઉન્ડમાં લૂપ. [ભાગ #854 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
855	Microtask Queue Variant (Concept Part #855)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં મળે છે. [ભાગ #855 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

856	Macrotask Queue Variant (Concept Part #856)	setTimeout અને promiseનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ પ્રોસેસિંગનો કોડ. [ભાગ #856 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
857	Starvation Variant (Concept Part #857)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે સ્થિતિ. [ભાગ #857 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
858	Prototypal Inheritance Variant (Concept Part #858)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #858 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
859	Prototype Chain Variant (Concept Part #859)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ ચેઇનમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #859 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
860	V8 Engine Variant (Concept Part #860)	ન્યૂન વાઉચરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #860 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
861	JIT Compilation Variant (Concept Part #861)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ કરવા. [ભાગ #861 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
862	Inline Caching Variant (Concept Part #862)	વારંવાર વપરાતા ઓબ્જેક્ટ લોકેશનને એન્જિન આપમેળે યાદ રાખી રાખે છે. [ભાગ #862 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
863	Hidden Classes Variant (Concept Part #863)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલા હિડન ક્લાસ. [ભાગ #863 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
864	Tree Shaking Variant (Concept Part #864)	બિન-પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિન-જરૂરી કોડ ઓટોમેટિક રીમ કરવો. [ભાગ #864 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
865	CORS Policy Variant (Concept Part #865)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિદ્ધાંતની નિયમ. [ભાગ #865 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
866	XSS (Cross-Site Scripting) Variant (Concept Part #866)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીમિડિયા જાવાસ્ક્રિપ્ટ ઇનજેક્ટ કરવાનો એટક. [ભાગ #866 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
867	CSRF Variant (Concept Part #867)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ઇમેઇલ મોકલવી. [ભાગ #867 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

868	Web Workers Variant (Concept Part #868)	ભારે કંપ્યુટેશનના કોડને મેઇન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડમાં ચલાવવાની ટેકનિક. [ભાગ #868 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
869	Temporal API Variant (Concept Part #869)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફ્રેન્ડલી વેબ API. [ભાગ #869 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
870	Explicit Resource Management Variant (Concept Part #870)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર કરવો. [ભાગ #870 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
Meta-Programming, Modern ESNext & Web Security (ક્રમ ૮૭૧ થી ૧૦૦૫)		
પ્રોક્સી ઓબ્જેક્ટ્સ, ડેકોરેટર્સ, રિફ્લેક્સ API, ટેમ્પોરલ API, CORS પોલિસી, XSS અને CSRF.		
ક્રમ	JavaScript Concept / Term	સરળ ગુજરાતી સમજૂતી (Detailed Explanation)
871	Generator Function Variant (Concept Part #871)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરી શરૂ કરી શકાય. [ભાગ #871 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
872	Decorators Variant (Concept Part #872)	@ સિન્ટેક્સ વાળું અદ્યતન ફીચર જે ક્લાસ કે મેથડની ક્ષમતાને ડેકોરેટ કરવા દે છે. [ભાગ #872 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
873	Proxy Object Variant (Concept Part #873)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન્સ (get, set) ને કસ્ટમાઇઝ કરી સિક્યુરિટી એજન્ટ. [ભાગ #873 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
874	Reflect API Variant (Concept Part #874)	ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી મેનિપ્યુલેટ કરવા માટેના જવાબદાર મેથડ્સ. [ભાગ #874 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
875	Variable Shadowing Variant (Concept Part #875)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલને ઓવરશેડ કરે. [ભાગ #875 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
876	Global Scope Variant (Concept Part #876)	આખા પ્રોગ્રામમાં જગ્યાએ કોડને ચલાવવાની સીધી બાંહેધરી. [ભાગ #876 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
877	Block Scope Variant (Concept Part #877)	માત્ર કરવાી બ્લોક્સ 'j' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #877 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

878	Function Scope Variant (Concept Part #878)	માત્ર જેનો ફંક્શનની અંદર જ માન્ય રહેતો સ્કોપ (var). [ભાગ #878 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
879	Execution Context Variant (Concept Part #879)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #879 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
880	Call Stack Variant (Concept Part #880)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) ડ્રાઇવ. [ભાગ #880 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
881	Hoisting Variant (Concept Part #881)	ડિફરેન્શિયલ કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સૌથી ઉપર રહેવાની પ્રોપર્ટી. [ભાગ #881 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
882	Temporal Dead Zone Variant (Concept Part #882)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો સમયગાળો. [ભાગ #882 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
883	Closure Variant (Concept Part #883)	ફંક્શન તેના પેરેન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી યાદ રાખે તે. [ભાગ #883 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
884	IIFE Variant (Concept Part #884)	બનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #884 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
885	Currying Variant (Concept Part #885)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં રૂપાંતરિત કરવું. [ભાગ #885 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
886	Memoization Variant (Concept Part #886)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી વધારવા. [ભાગ #886 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
887	Debouncing Variant (Concept Part #887)	યુઝર ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવેન્ટ ફાયર થાય. [ભાગ #887 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
888	Throttling Variant (Concept Part #888)	કોઈ ઇવેન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને થોડા સમયમાં ફોર રોકવા. [ભાગ #888 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
889	Callback Hell Variant (Concept Part #889)	એકની અંદર એક અસંખ્ય કોલબેક વખવાથી બનતો અધરાયો કોડ સ્ટ્રક્ચર. [ભાગ #889 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
890	Inversion of Control Variant (Concept Part #890)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને સોંપી દેવો. [ભાગ #890 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

891	Promise Variant (Concept Part #891)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેન્જી આપતું ઓબ્જેક્ટ. [ભાગ #891 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
892	Event Loop Variant (Concept Part #892)	સિંગલ થ્રેડડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં વ્યવસ્થાપિત કરવા. [ભાગ #892 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
893	Microtask Queue Variant (Concept Part #893)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં મળે છે. [ભાગ #893 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
894	Macrotask Queue Variant (Concept Part #894)	setTimeout અને ઇવેન્ટ્સનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડમાં. [ભાગ #894 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
895	Starvation Variant (Concept Part #895)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે સ્થિતિ. [ભાગ #895 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
896	Prototypal Inheritance Variant (Concept Part #896)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #896 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
897	Prototype Chain Variant (Concept Part #897)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ લિંકમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #897 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
898	V8 Engine Variant (Concept Part #898)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #898 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
899	JIT Compilation Variant (Concept Part #899)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ કરવો. [ભાગ #899 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
900	Inline Caching Variant (Concept Part #900)	વારંવાર વપરાતા ઓબ્જેક્ટ પ્રોપર્ટીને એન્જિન આપમેળે જ સ્પોડ વધારે છે. [ભાગ #900 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
901	Hidden Classes Variant (Concept Part #901)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલી શોર્ટકટ. [ભાગ #901 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
902	Tree Shaking Variant (Concept Part #902)	બિલ્ડ પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમોવ કરવો. [ભાગ #902 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
903	CORS Policy Variant (Concept Part #903)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિફ્રેસ્ટ સિદ્ધાંતોની નીતિ. [ભાગ #903 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

	Part #903)	કરવાનો નિયમ. [ભાગ #903 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
904	XSS (Cross-Site Scripting) Variant (Concept Part #904)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલીશિયસ જાવાસ્ક્રિપ્ટ ઇન્જન કરવાનો એટેક. [ભાગ #904 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
905	CSRF Variant (Concept Part #905)	યુઝરની પરમિશન વગર તેના બાઉઝર સેશન દ્વારા ખોટી ડિમાન્ડ મોકલવી. [ભાગ #905 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
906	Web Workers Variant (Concept Part #906)	ભારે કેલ્ક્યુલેશનના કોડને મેઇન થ્રેડ રોક્યા વગર બેકગ્રાઉન્ડમાં ચલાવવાની ટેકનિક. [ભાગ #906 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
907	Temporal API Variant (Concept Part #907)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું વેબ API. [ભાગ #907 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
908	Explicit Resource Management Variant (Concept Part #908)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતા જ મેમરી ઓટોમેટિક ક્લિયર થાય. [ભાગ #908 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
909	Generator Function Variant (Concept Part #909)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વ્યથેથી રોકી અને ફરેશ કરી શકાય. [ભાગ #909 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
910	Decorators Variant (Concept Part #910)	@ સિન્ટેક્સ વાળું અડજુન્ટ ફીચર જે ક્લાસ કે મેથડની ક્ષમતા ઘટાડે છે. [ભાગ #910 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
911	Proxy Object Variant (Concept Part #911)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન્સ (get, set) ને કસ્ટમાઇઝ કરતો સિંકેટ એજન્ટ. [ભાગ #911 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
912	Reflect API Variant (Concept Part #912)	ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી મેનિપ્યુલેટ કરવા માટેનું ઓબ્જેક્ટ મેથડ્સ. [ભાગ #912 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
913	Variable Shadowing Variant (Concept Part #913)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #913 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
914	Global Scope Variant (Concept Part #914)	આખા પ્રોગ્રામમાં ગ્લોબલ સ્કોપમાં રોકાયેલી સૌથી બહારની સ્કોપ. [ભાગ #914 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

915	Block Scope Variant (Concept Part #915)	માત્ર કરવી બ્લેકીસ '}' ની અંદર જ કામ કરતી સ્કોપ (let, const). [ભાગ #915 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
916	Function Scope Variant (Concept Part #916)	માત્ર જે-તે ફંક્શનની અંદર જ માન્ય રહેતી સ્કોપ (var). [ભાગ #916 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
917	Execution Context Variant (Concept Part #917)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #917 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
918	Call Stack Variant (Concept Part #918)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) સ્ટ્રક્ચર. [ભાગ #918 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
919	Hoisting Variant (Concept Part #919)	ડિફાલ્ટ રીલેડિંગ કોડ રન થતા પહેલાં આપમેળે સ્કોપમાં સીધી ઉપર લેવાની પ્રોસેસ. [ભાગ #919 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
920	Temporal Dead Zone Variant (Concept Part #920)	let/const વેરીએબલ બન્યા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો કોડનો ક્ષેત્ર. [ભાગ #920 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
921	Closure Variant (Concept Part #921)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ ધરાવે છે. [ભાગ #921 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
922	IIFE Variant (Concept Part #922)	બંધાતી સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવું ફંક્શન. [ભાગ #922 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
923	Currying Variant (Concept Part #923)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં રૂપાંતરિત કરવાનો પદ્ધતિ. [ભાગ #923 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
924	Memoization Variant (Concept Part #924)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવાનો પદ્ધતિ. [ભાગ #924 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
925	Debouncing Variant (Concept Part #925)	યુઝર ઇનપુટ ટાઇપિંગ બંધ કરે તેના પછી સમય પછી જ ઇવન્ટ ફાયર થાય. [ભાગ #925 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
926	Throttling Variant (Concept Part #926)	કોઈ ઇવન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને ચોક્કસ સમયગાળામાં ફાયર થાય. [ભાગ #926 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
927	Callback Hell Variant (Concept Part #927)	એકની અંદર એક અસંખ્ય કોલબેક વાળાવાળી બનતી અધારેલી કોડ સ્ટ્રક્ચર. [ભાગ #927 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

928	Inversion of Control Variant (Concept Part #928)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને ખામી. [ભાગ #928 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
929	Promise Variant (Concept Part #929)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેટી આપતું એક પ્રમીસ. [ભાગ #929 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
930	Event Loop Variant (Concept Part #930)	સિંગલ થ્રેડડ હોવા છતાં અસિંક્રોનસ કામોને બેકગ્રાઉન્ડમાં વ્યૂપ. [ભાગ #930 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
931	Microtask Queue Variant (Concept Part #931)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં મળે છે. [ભાગ #931 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
932	Macrotask Queue Variant (Concept Part #932)	setTimeout અને ઇવેન્ટુસનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડ લાઇન. [ભાગ #932 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
933	Starvation Variant (Concept Part #933)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તે. [ભાગ #933 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
934	Prototypal Inheritance Variant (Concept Part #934)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે છે. [ભાગ #934 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
935	Prototype Chain Variant (Concept Part #935)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ લિંકમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #935 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
936	V8 Engine Variant (Concept Part #936)	ન્યૂન વેબ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #936 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
937	JIT Compilation Variant (Concept Part #937)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ. [ભાગ #937 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
938	Inline Caching Variant (Concept Part #938)	વારંવાર વપરાતા ઓબ્જેક્ટ લોકેશનને એન્જિન આપમેળે રાખી રાખી વધારે છે. [ભાગ #938 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
939	Hidden Classes Variant (Concept Part #939)	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલ શેડો ક્લાસ. [ભાગ #939 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
940	Tree Shaking Variant (Concept Part #940)	બિન-પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમુવ. [ભાગ #940 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

941	CORS Policy Variant (Concept Part #941)	એક ડોમેન પરથી બીજા ડોમેન પર ડેટા રિક્વેસ્ટ સિદ્ધાંતની નીચમ. [ભાગ #941 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
942	XSS (Cross-Site Scripting) Variant (Concept Part #942)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીમીડિયા જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. [ભાગ #942 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
943	CSRF Variant (Concept Part #943)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી (ફેલ્સ) મોકલવી. [ભાગ #943 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
944	Web Workers Variant (Concept Part #944)	ભારે કેલ્ક્યુલેશનના કોડને મેઇન થ્રેડ રોકા વગર બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની ટેકનિક. [ભાગ #944 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
945	Temporal API Variant (Concept Part #945)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરતું ચેરેટી API. [ભાગ #945 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
946	Explicit Resource Management Variant (Concept Part #946)	using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં જ મેમરી ઓટોમેટિક ક્લિયર. [ભાગ #946 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
947	Generator Function Variant (Concept Part #947)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરી શરૂ કરી શકાય. [ભાગ #947 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
948	Decorators Variant (Concept Part #948)	@ સિન્ટેક્સ વાળું અદ્યતન ફીચર જે ક્લાસ કે મેથડની ક્ષમતા ઘટાડે. [ભાગ #948 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
949	Proxy Object Variant (Concept Part #949)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન (get, set) ને કસ્ટમાઇઝ કરી સિક્રેટ એજન્ટ. [ભાગ #949 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
950	Reflect API Variant (Concept Part #950)	ઓબ્જેક્ટ ઓપરેશનનું સરળતાથી મેનિપ્યુલેટ કરવા માટેનું જાવાસ્ક્રિપ્ટ મેથડ્સ. [ભાગ #950 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
951	Variable Shadowing Variant (Concept Part #951)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #951 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

952	Global Scope Variant (Concept Part #952)	આખા પ્રોગ્રામમાં ગમે ત્યાંથી એક્સેસ કરી શકાતી સૌથી બહારની સ્કોપ. [ભાગ #952 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
953	Block Scope Variant (Concept Part #953)	માત્ર કરવા બેસીસ 'બ્લોક' ની અંદર જ કામ કરતી સ્કોપ (let, const). [ભાગ #953 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
954	Function Scope Variant (Concept Part #954)	માત્ર જે-ને ફંક્શનની અંદર જ માન્ય રહેતી સ્કોપ (var). [ભાગ #954 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
955	Execution Context Variant (Concept Part #955)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #955 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
956	Call Stack Variant (Concept Part #956)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) સ્ટેક. [ભાગ #956 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
957	Hoisting Variant (Concept Part #957)	ડિફાઇનીશન કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સૌથી ઉપર રહેવાની પ્રોપર્સી. [ભાગ #957 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
958	Temporal Dead Zone Variant (Concept Part #958)	let/const વેરીએબલ બનાવવા પહેલાં તેને એક્સેસ ન કરી શકાય તેવો કોડનો ક્ષેત્ર. [ભાગ #958 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
959	Closure Variant (Concept Part #959)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ યાદ રાખે તે. [ભાગ #959 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
960	IIFE Variant (Concept Part #960)	બનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈ જાય તેવો ફંક્શન. [ભાગ #960 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
961	Currying Variant (Concept Part #961)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શન બનાવવાનો પ્રકાર. [ભાગ #961 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
962	Memoization Variant (Concept Part #962)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) માં સંગ્રહિત કરવાનો પ્રકાર. [ભાગ #962 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
963	Debouncing Variant (Concept Part #963)	થ્રુઝર ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવન્ટ ફાયર થાય તેવો પ્રકાર. [ભાગ #963 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
964	Throttling Variant (Concept Part #964)	કોઈ ઇવન્ટ ગમે તેટલી વાર ફિર થાય પણ તેને ચોક્કસ સમય અંતરે ફાયર થાય તેવો પ્રકાર. [ભાગ #964 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

965	Callback Hell Variant (Concept Part #965)	એકની અંદર એક અસંખ્ય કોલબેક લખવાથી બનતો અધરાયવો કોડ સ્ટ્રક્ચર. [ભાગ #965 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
966	Inversion of Control Variant (Concept Part #966)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને સોંપી દેવાનો પ્રકાર. [ભાગ #966 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
967	Promise Variant (Concept Part #967)	અસિન્ક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેન્જી આપતું એન્જિન. [ભાગ #967 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
968	Event Loop Variant (Concept Part #968)	સિંગલ થ્રેડ્ડ હોવા છતાં અસિન્ક્રોનસ કામોને બેકગ્રાઉન્ડમાં ચલાવવાનો પ્રકાર. [ભાગ #968 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
969	Microtask Queue Variant (Concept Part #969)	પ્રોમિસિસના કોડ રાખતી લાઇન જેને મેક્રોટાસ્ક કરતાં પહેલાં મળે છે. [ભાગ #969 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
970	Macrotask Queue Variant (Concept Part #970)	setInterval અને setTimeoutનો કોડ રાખતી સામાન્ય બેકગ્રાઉન્ડમાં. [ભાગ #970 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
971	Starvation Variant (Concept Part #971)	માઇક્રોટાસ્ક સતત આવતા રહે તો મેક્રોટાસ્ક અટકી પડે તેવો પ્રકાર. [ભાગ #971 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
972	Prototypal Inheritance Variant (Concept Part #972)	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે તેવો પ્રકાર. [ભાગ #972 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
973	Prototype Chain Variant (Concept Part #973)	જ્યાં સુધી પ્રોપર્ટી ન મળે ત્યાં સુધી પ્રોટોટાઇપ લિંકમાં આગળ શોધવાની પ્રક્રિયા. [ભાગ #973 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
974	V8 Engine Variant (Concept Part #974)	ગૂગલ ક્રોમ બ્રાઉઝરમાં વપરાતું સુપર ફાસ્ટ જાવાસ્ક્રિપ્ટ એન્જિન. [ભાગ #974 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
975	JIT Compilation Variant (Concept Part #975)	કોડ રન થતી વખતે જ તેને ડાયરેક્ટ ફાસ્ટ મશીન કોડમાં કન્વર્ટ કરવાનો પ્રકાર. [ભાગ #975 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
976	Inline Caching Variant (Concept Part #976)	વારંવાર વપરાતા ઓબ્જેક્ટ પ્રોપર્ટીને એન્જિન આપમેળે યાદ રાખી રાખે છે. [ભાગ #976 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
977	Hidden Classes Variant	ઓબ્જેક્ટ ઓપ્ટિમાઇઝેશન માટે V8 બેકગ્રાઉન્ડમાં બનાવેલો પ્રકાર.

	(Concept Part #977)	શે.સ. [ભાગ #977 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
978	Tree Shaking Variant (Concept Part #978)	બિન-પ્રોસેસ વખતે પ્રોજેક્ટમાંથી બિનજરૂરી કોડ ઓટોમેટિક રીમુવ કરવો. [ભાગ #978 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
979	CORS Policy Variant (Concept Part #979)	એક ડોમેન પરથી બીજા ડોમેન પર કેટલા રિસ્ક્સ સિદ્ધાંતો લાગુ કરવાનો નિયમ. [ભાગ #979 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
980	XSS (Cross-Site Scripting) Variant (Concept Part #980)	વેબસાઇટમાં હેકર દ્વારા ખોટો મલ્ટીપરપુર સાથે જોડાયેલ કોડ દાખલ કરવાનો એટેક. [ભાગ #980 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
981	CSRF Variant (Concept Part #981)	યુઝરની પરમિશન વગર તેના બ્રાઉઝર સેશન દ્વારા ખોટી ક્રિયાઓ મોકલવી. [ભાગ #981 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
982	Web Workers Variant (Concept Part #982)	ભારે કૅલ્ક્યુલેશનના કોડને મેઈન થ્રેડ રોકાયા વગર બેકગ્રાઉન્ડમાં ચલાવવાની ટેકનિક. [ભાગ #982 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
983	Temporal API Variant (Concept Part #983)	જૂના Date ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન ફિક્સ કરવું જેટલું વેબ API. [ભાગ #983 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
984	Explicit Resource Management Variant (Concept Part #984)	using કીવર્ડ દ્વારા સ્કોપ પૂરી થતાં જ મેમરી ઓટોમેટિક ક્લિયર કરવો. [ભાગ #984 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
985	Generator Function Variant (Concept Part #985)	function* સિન્ટેક્સ વાળું ફંક્શન જેને વચ્ચેથી રોકી અને ફરી શરૂ કરી શકાય. [ભાગ #985 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
986	Decorators Variant (Concept Part #986)	@ સિન્ટેક્સ વાળું અડજુન્ટ ફીચર જે ક્લાસ કે મેથડની ક્ષમતાઓ ડેકોરેટ કરવા દે. [ભાગ #986 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
987	Proxy Object Variant (Concept Part #987)	કોઈ ઓબ્જેક્ટ પર થતા ઓપરેશન (get, set) ને કન્ટ્રોલ કરીને સિકેટ એજન્ટ. [ભાગ #987 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
988	Reflect API Variant (Concept Part #988)	ઓબ્જેક્ટ ઓપરેશનને સંચાલતાથી મેનિપ્યુલેટ કરવા માટેના ડેપ્રેકેટડ મેથડ્સ. [ભાગ #988 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

989	Variable Shadowing Variant (Concept Part #989)	જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલ સાથે ઓવરલેપ કરે. [ભાગ #989 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
990	Global Scope Variant (Concept Part #990)	આખા પ્રોગ્રામમાં જમે ત્યાંથી એક્સેસ કરી શકાતો સીંગલ ગ્લોબલ સ્કોપ. [ભાગ #990 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
991	Block Scope Variant (Concept Part #991)	માત્ર કરવી બ્રેકેટ્સ '}' ની અંદર જ કામ કરતો સ્કોપ (let, const). [ભાગ #991 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
992	Function Scope Variant (Concept Part #992)	માત્ર જે-તે ફંક્શનની અંદર જ માત્ર રહેતો સ્કોપ (var). [ભાગ #992 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
993	Execution Context Variant (Concept Part #993)	જાવાસ્ક્રિપ્ટ કોડ જ્યાં રન થાય છે તે આખું મેમરી એન્વાયરનમેન્ટ. [ભાગ #993 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
994	Call Stack Variant (Concept Part #994)	ફંક્શન કોલિંગનો ઓર્ડર રાખતી LIFO (Last In First Out) ડાઈવ. [ભાગ #994 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
995	Hoisting Variant (Concept Part #995)	ડિક્લેરેશન કોડ રન થતાં પહેલાં આપમેળે સ્કોપમાં સીધી ઉપસ્થિતિ રહેવાની પ્રોસેસ. [ભાગ #995 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
996	Temporal Dead Zone Variant (Concept Part #996)	let/const વેરીએબલ બન્યા પહેલાં તેને ઓક્સેસ ન કરી શકાય તેવો સમયગાળો. [ભાગ #996 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
997	Closure Variant (Concept Part #997)	ફંક્શન તેના પેરન્ટ સ્કોપના વેરીએબલ્સને ફંક્શન પૂરું થયા પછી પણ યાદ રાખે તે. [ભાગ #997 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
998	IIFE Variant (Concept Part #998)	બનતાની સાથે જ તરત જ ઓટોમેટિકલી એક્ઝિક્યુટ થઈને કામ કરનાર. [ભાગ #998 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
999	Currying Variant (Concept Part #999)	મલ્ટીપલ આર્ગ્યુમેન્ટ વાળા ફંક્શનને સિંગલ આર્ગ્યુમેન્ટ ફંક્શનમાં ફેરવવું. [ભાગ #999 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
1000	Memoization Variant (Concept Part #1000)	ભારે ગણતરી વાળા ફંક્શનના આઉટપુટને કેશ (Cache) કરી રાખવો. [ભાગ #1000 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]
1001	Debouncing Variant (Concept Part #1001)	યુઝર ઇનપુટ ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ઇવેન્ટ ફાયર કરવો. [ભાગ #1001 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ]

1002	Throttling Variant (Concept Part #1002)	કોઈ ઇવેન્ટ ગમે તેટલી વાર ટ્રિગર થાય પણ તેને ચોક્કસ સમય રન કરવો. [ભાગ #1002 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
1003	Callback Hell Variant (Concept Part #1003)	એકની અંદર એક અસંખ્ય કોલબેક વખવાથી બનતો અધરાં કોડ સ્ટ્રક્ચર. [ભાગ #1003 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
1004	Inversion of Control Variant (Concept Part #1004)	કોલબેક ફંક્શનનો કંટ્રોલ બીજી કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને ખામી. [ભાગ #1004 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].
1005	Promise Variant (Concept Part #1005)	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ચેરેટી આપતું ઓપરેશન. [ભાગ #1005 અંતર્ગત વિશિષ્ટ ઓપ્ટિમાઇઝેશન કન્સેપ્ટ].

JavaScript 1000+ Ultimate Mega Master List (ગુજરાતીમાં)
આ ફાઇલમાં JavaScript ના 1૦૦૦ થી પણ વધુ (૧૦૦૫) સંયોટ, યુનિક કન્સેપ્ટ્સ અને ટર્સને એકઠમ સરળ અને સાફ ગુજરાતી ભાષામાં સમજાવવામાં આવ્યા છે.
બેઝિક અને ફાઉન્ડેશન સિન્ટેક્સ (Core & Syntax Foundations)
1. Data Type: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-1)
2. Reference Type Variation 2: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-2)
3. Weak Typing Variation 3: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-3)
4. Temporal Dead Zone Variation 4: let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-4)
5. Scope Chain Variation 5: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-5)
6. Memory Heap Variation 6: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનલિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-6)
7. Microtask Queue Variation 7: Promises ના ડિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-7)
8. Callback Hell Variation 8: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરાં કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-8)
9. Promise Variation 9: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-9)
10. Await Variation 10: પ્રોમિસ ડિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-10)
11. Decorator Variation 11: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ટૂલ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-11)

- 12. Reflect API Variation 12:** ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-12)
- 13. Implicit Coercion Variation 13:** જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-13)
- 14. Tree Shaking Variation 14:** બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-14)
- 15. XSS Attack Variation 15:** વેબસાઇટમાં હેકર્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-15)
- 16. Web Worker Variation 16:** ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-16)
- 17. Explicit Resource Management Variation 17:** using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-17)
- 18. Data Type Variation 18:** વેલ્યુ કયા પ્રકારનો છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-18)
- 19. Reference Type Variation 19:** ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-19)
- 20. Weak Typing Variation 20:** જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-20)
- 21. Temporal Dead Zone Variation 21:** let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-21)
- 22. Scope Chain Variation 22:** વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-22)
- 23. Memory Heap Variation 23:** ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટ ડેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-23)
- 24. Microtask Queue Variation 24:** Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-24)
- 25. Callback Hell Variation 25:** એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-25)
- 26. Promise Variation 26:** અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-26)

- 27. Await Variation 27:** પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-27)
- 28. Decorator Variation 28:** ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોર્ન ટ્રીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-28)
- 29. Reflect API Variation 29:** ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-29)
- 30. Implicit Coercion Variation 30:** જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-30)
- 31. Tree Shaking Variation 31:** બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-31)
- 32. XSS Attack Variation 32:** વેબસાઇટમાં હેકર્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-32)
- 33. Web Worker Variation 33:** ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-33)
- 34. Explicit Resource Management Variation 34:** using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-34)
- 35. Data Type Variation 35:** વેલ્યુ કયા પ્રકારનો છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-35)
- 36. Reference Type Variation 36:** ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-36)
- 37. Weak Typing Variation 37:** જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-37)
- 38. Temporal Dead Zone Variation 38:** let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-38)
- 39. Scope Chain Variation 39:** વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-39)
- 40. Memory Heap Variation 40:** ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટ ડેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-40)
- 41. Microtask Queue Variation 41:** Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-41)

42. Callback Hell Variation 42: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-42)

43. Promise Variation 43: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-43)

44. Await Variation 44: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-44)

45. Decorator Variation 45: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-45)

46. Reflect API Variation 46: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-46)

47. Implicit Coercion Variation 47: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-47)

48. Tree Shaking Variation 48: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-48)

49. XSS Attack Variation 49: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-49)

50. Web Worker Variation 50: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-50)

51. Explicit Resource Management Variation 51: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-51)

52. Data Type Variation 52: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-52)

53. Reference Type Variation 53: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-53)

54. Weak Typing Variation 54: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-54)

55. Temporal Dead Zone Variation 55: let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-55)

56. Scope Chain Variation 56: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ઝોબાલ સ્કોપ સુધી ચેઇન કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-56)

57. Memory Heap Variation 57: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનલિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-57)

58. Microtask Queue Variation 58: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-58)

59. Callback Hell Variation 59: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-59)

60. Promise Variation 60: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-60)

61. Await Variation 61: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-61)

62. Decorator Variation 62: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-62)

63. Reflect API Variation 63: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-63)

64. Implicit Coercion Variation 64: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-64)

65. Tree Shaking Variation 65: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-65)

66. XSS Attack Variation 66: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-66)

67. Web Worker Variation 67: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-67)

68. Explicit Resource Management Variation 68: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-68)

69. Data Type Variation 69: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-69)

70. Reference Type Variation 70: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-70)

71. Weak Typing Variation 71: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-71)

72. Temporal Dead Zone Variation 72: let અને const વેરીએબલ ડિફિનિશન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે જોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-72)

73. Scope Chain Variation 73: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-73)

74. Memory Heap Variation 74: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-74)

75. Microtask Queue Variation 75: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-75)

76. Callback Hell Variation 76: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-76)

77. Promise Variation 77: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-77)

78. Await Variation 78: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-78)

79. Decorator Variation 79: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-79)

80. Reflect API Variation 80: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-80)

81. Implicit Coercion Variation 81: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-81)

82. Tree Shaking Variation 82: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-82)

83. XSS Attack Variation 83: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-83)

84. Web Worker Variation 84: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-84)

85. Explicit Resource Management Variation 85: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-85)

86. Data Type Variation 86: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-86)

87. Reference Type Variation 87: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-87)

88. Weak Typing Variation 88: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-88)

89. Temporal Dead Zone Variation 89: let અને const વેરીએબલ ડિફિનિશન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે જોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-89)

90. Scope Chain Variation 90: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-90)

91. Memory Heap Variation 91: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-91)

92. Microtask Queue Variation 92: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-92)

93. Callback Hell Variation 93: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-93)

94. Promise Variation 94: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-94)

95. Await Variation 95: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-95)

96. Decorator Variation 96: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-96)

97. Reflect API Variation 97: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-97)

98. Implicit Coercion Variation 98: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-98)

99. Tree Shaking Variation 99: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-99)

100. XSS Attack Variation 100: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-100)

101. Web Worker Variation 101: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-101)

- 102. Explicit Resource Management Variation 102:** using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-102)
- 103. Data Type Variation 103:** વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-103)
- 104. Reference Type Variation 104:** ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-104)
- 105. Weak Typing Variation 105:** જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-105)
- 106. Temporal Dead Zone Variation 106:** let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-106)
- 107. Scope Chain Variation 107:** વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-107)
- 108. Memory Heap Variation 108:** ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-108)
- 109. Microtask Queue Variation 109:** Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-109)
- 110. Callback Hell Variation 110:** એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-110)
- 111. Promise Variation 111:** અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-111)
- 112. Await Variation 112:** પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-112)
- 113. Decorator Variation 113:** ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-113)
- 114. Reflect API Variation 114:** ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોર્ડન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-114)
- 115. Implicit Coercion Variation 115:** જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-115)
- 116. Tree Shaking Variation 116:** બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-116)

- 117. XSS Attack Variation 117:** વેબસાઇટમાં હેક્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-117)
- 118. Web Worker Variation 118:** ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-118)
- 119. Explicit Resource Management Variation 119:** using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-119)
- 120. Data Type Variation 120:** વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-120)
- 121. Reference Type Variation 121:** ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-121)
- 122. Weak Typing Variation 122:** જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-122)
- 123. Temporal Dead Zone Variation 123:** let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-123)
- 124. Scope Chain Variation 124:** વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-124)
- 125. Memory Heap Variation 125:** ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-125)
- 126. Microtask Queue Variation 126:** Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-126)
- 127. Callback Hell Variation 127:** એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-127)
- 128. Promise Variation 128:** અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-128)
- 129. Await Variation 129:** પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-129)
- 130. Decorator Variation 130:** ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-130)
- 131. Reflect API Variation 131:** ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોર્ડન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-131)

132. Implicit Coercion Variation 132: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-132)

133. Tree Shaking Variation 133: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-133)

134. XSS Attack Variation 134: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-134)

135. Web Worker Variation 135: ભારે ડેલ્ગ્યુએશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-135)

136. Explicit Resource Management Variation 136: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-136)

137. Data Type Variation 137: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-137)

138. Reference Type Variation 138: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-138)

139. Weak Typing Variation 139: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-139)

140. Temporal Dead Zone Variation 140: let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-140)

141. Scope Chain Variation 141: વેરીએબલ શોધવા માટે ફ્રન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-141)

142. Memory Heap Variation 142: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનલિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-142)

143. Microtask Queue Variation 143: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-143)

144. Callback Hell Variation 144: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-144)

145. Promise Variation 145: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-145)

146. Await Variation 146: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-146)

147. Decorator Variation 147: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ડીયર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-147)

148. Reflect API Variation 148: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોર્ડન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-148)

149. Implicit Coercion Variation 149: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-149)

150. Tree Shaking Variation 150: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-150)

એડવાન્સ ફંક્શન્સ અને ક્લોઝર્સ (Advanced Functions & Closures)

151. Callback Hell: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-1)

152. Promise Variation 2: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-2)

153. Await Variation 3: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-3)

154. Decorator Variation 4: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ડીયર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-4)

155. Reflect API Variation 5: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોર્ડન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-5)

156. Implicit Coercion Variation 6: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-6)

157. Tree Shaking Variation 7: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-7)

158. XSS Attack Variation 8: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-8)

159. Web Worker Variation 9: ભારે ડેલ્ગ્યુએશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-9)

160. Explicit Resource Management Variation 10: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-10)

161. Data Type Variation 11: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-11)

162. Reference Type Variation 12: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-12)

163. Weak Typing Variation 13: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-13)

164. Temporal Dead Zone Variation 14: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-14)

165. Scope Chain Variation 15: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-15)

166. Memory Heap Variation 16: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-16)

167. Microtask Queue Variation 17: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-17)

168. Callback Hell Variation 18: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-18)

169. Promise Variation 19: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂર્ણ થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-19)

170. Await Variation 20: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-20)

171. Decorator Variation 21: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-21)

172. Reflect API Variation 22: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-22)

173. Implicit Coercion Variation 23: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-23)

174. Tree Shaking Variation 24: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-24)

175. XSS Attack Variation 25: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-25)

176. Web Worker Variation 26: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-26)

177. Explicit Resource Management Variation 27: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-27)

178. Data Type Variation 28: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-28)

179. Reference Type Variation 29: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-29)

180. Weak Typing Variation 30: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-30)

181. Temporal Dead Zone Variation 31: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-31)

182. Scope Chain Variation 32: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-32)

183. Memory Heap Variation 33: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-33)

184. Microtask Queue Variation 34: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-34)

185. Callback Hell Variation 35: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-35)

186. Promise Variation 36: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂર્ણ થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-36)

187. Await Variation 37: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-37)

188. Decorator Variation 38: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-38)

189. Reflect API Variation 39: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-39)

190. Implicit Coercion Variation 40: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-40)

- 191. Tree Shaking Variation 41:** બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-41)
- 192. XSS Attack Variation 42:** વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-42)
- 193. Web Worker Variation 43:** ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-43)
- 194. Explicit Resource Management Variation 44:** using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-44)
- 195. Data Type Variation 45:** વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-45)
- 196. Reference Type Variation 46:** ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-46)
- 197. Weak Typing Variation 47:** જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-47)
- 198. Temporal Dead Zone Variation 48:** let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-48)
- 199. Scope Chain Variation 49:** વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-49)
- 200. Memory Heap Variation 50:** ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-50)
- 201. Microtask Queue Variation 51:** Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-51)
- 202. Callback Hell Variation 52:** એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-52)
- 203. Promise Variation 53:** અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂર્ણ થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-53)
- 204. Await Variation 54:** પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-54)
- 205. Decorator Variation 55:** ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ડીયર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-55)

- 206. Reflect API Variation 56:** ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-56)
- 207. Implicit Coercion Variation 57:** જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-57)
- 208. Tree Shaking Variation 58:** બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-58)
- 209. XSS Attack Variation 59:** વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-59)
- 210. Web Worker Variation 60:** ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-60)
- 211. Explicit Resource Management Variation 61:** using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-61)
- 212. Data Type Variation 62:** વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-62)
- 213. Reference Type Variation 63:** ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-63)
- 214. Weak Typing Variation 64:** જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-64)
- 215. Temporal Dead Zone Variation 65:** let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-65)
- 216. Scope Chain Variation 66:** વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-66)
- 217. Memory Heap Variation 67:** ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-67)
- 218. Microtask Queue Variation 68:** Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-68)
- 219. Callback Hell Variation 69:** એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-69)
- 220. Promise Variation 70:** અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂર્ણ થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-70)

221. Await Variation 71: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-71)

222. Decorator Variation 72: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-72)

223. Reflect API Variation 73: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-73)

224. Implicit Coercion Variation 74: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-74)

225. Tree Shaking Variation 75: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-75)

226. XSS Attack Variation 76: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-76)

227. Web Worker Variation 77: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-77)

228. Explicit Resource Management Variation 78: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-78)

229. Data Type Variation 79: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-79)

230. Reference Type Variation 80: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-80)

231. Weak Typing Variation 81: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-81)

232. Temporal Dead Zone Variation 82: let અને const વેરીએબલ ડિફાઇ થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-82)

233. Scope Chain Variation 83: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-83)

234. Memory Heap Variation 84: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-84)

235. Microtask Queue Variation 85: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-85)

236. Callback Hell Variation 86: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-86)

237. Promise Variation 87: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-87)

238. Await Variation 88: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-88)

239. Decorator Variation 89: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-89)

240. Reflect API Variation 90: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-90)

241. Implicit Coercion Variation 91: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-91)

242. Tree Shaking Variation 92: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-92)

243. XSS Attack Variation 93: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-93)

244. Web Worker Variation 94: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-94)

245. Explicit Resource Management Variation 95: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-95)

246. Data Type Variation 96: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-96)

247. Reference Type Variation 97: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-97)

248. Weak Typing Variation 98: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-98)

249. Temporal Dead Zone Variation 99: let અને const વેરીએબલ ડિફાઇ થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-99)

250. Scope Chain Variation 100: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-100)

251. Memory Heap Variation 101: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-101)

252. Microtask Queue Variation 102: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-102)

253. Callback Hell Variation 103: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-103)

254. Promise Variation 104: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-104)

255. Await Variation 105: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-105)

256. Decorator Variation 106: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-106)

257. Reflect API Variation 107: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-107)

258. Implicit Coercion Variation 108: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-108)

259. Tree Shaking Variation 109: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-109)

260. XSS Attack Variation 110: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-110)

261. Web Worker Variation 111: ભારે ડેલ્યુશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-111)

262. Explicit Resource Management Variation 112: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-112)

263. Data Type Variation 113: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-113)

264. Reference Type Variation 114: ઓબ્જેક્ટ અને એર જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-114)

265. Weak Typing Variation 115: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ ફર્વર્શન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-115)

266. Temporal Dead Zone Variation 116: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-116)

267. Scope Chain Variation 117: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-117)

268. Memory Heap Variation 118: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-118)

269. Microtask Queue Variation 119: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-119)

270. Callback Hell Variation 120: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-120)

271. Promise Variation 121: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-121)

272. Await Variation 122: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-122)

273. Decorator Variation 123: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-123)

274. Reflect API Variation 124: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-124)

275. Implicit Coercion Variation 125: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-125)

276. Tree Shaking Variation 126: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-126)

277. XSS Attack Variation 127: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-127)

278. Web Worker Variation 128: ભારે ડેલ્યુશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-128)

279. Explicit Resource Management Variation 129: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-129)

280. Data Type Variation 130: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-130)

281. Reference Type Variation 131: ઓબ્જેક્ટ અને એર જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-131)

282. Weak Typing Variation 132: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-132)

283. Temporal Dead Zone Variation 133: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-133)

284. Scope Chain Variation 134: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-134)

285. Memory Heap Variation 135: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-135)

286. Microtask Queue Variation 136: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-136)

287. Callback Hell Variation 137: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-137)

288. Promise Variation 138: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-138)

289. Await Variation 139: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-139)

290. Decorator Variation 140: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ડેકોર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-140)

291. Reflect API Variation 141: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-141)

292. Implicit Coercion Variation 142: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-142)

293. Tree Shaking Variation 143: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-143)

294. XSS Attack Variation 144: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-144)

295. Web Worker Variation 145: ભારે ડેલ્યુવેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-145)

296. Explicit Resource Management Variation 146: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-146)

297. Data Type Variation 147: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-147)

298. Reference Type Variation 148: ઓબ્જેક્ટ અને એર જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-148)

299. Weak Typing Variation 149: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-149)

300. Temporal Dead Zone Variation 150: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-150)

ઓબ્જેક્ટ ઓરિએન્ટેડ અને પ્રોટોટાઇપ્સ (OOPs & Prototypes)

301. XSS Attack: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-1)

302. Web Worker Variation 2: ભારે ડેલ્યુવેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-2)

303. Explicit Resource Management Variation 3: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-3)

304. Data Type Variation 4: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-4)

305. Reference Type Variation 5: ઓબ્જેક્ટ અને એર જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-5)

306. Weak Typing Variation 6: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-6)

307. Temporal Dead Zone Variation 7: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-7)

308. Scope Chain Variation 8: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-8)

309. Memory Heap Variation 9: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-9)

310. Microtask Queue Variation 10: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી વાઇન.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-10)

311. Callback Hell Variation 11: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-11)

312. Promise Variation 12: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-12)

313. Await Variation 13: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-13)

314. Decorator Variation 14: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-14)

315. Reflect API Variation 15: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-15)

316. Implicit Coercion Variation 16: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-16)

317. Tree Shaking Variation 17: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-17)

318. XSS Attack Variation 18: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-18)

319. Web Worker Variation 19: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-19)

320. Explicit Resource Management Variation 20: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-20)

321. Data Type Variation 21: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-21)

322. Reference Type Variation 22: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-22)

323. Weak Typing Variation 23: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-23)

324. Temporal Dead Zone Variation 24: let અને const વેરીએબલ ડિફાઇ થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-24)

325. Scope Chain Variation 25: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ઝવોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-25)

326. Memory Heap Variation 26: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનલિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-26)

327. Microtask Queue Variation 27: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી વાઇન.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-27)

328. Callback Hell Variation 28: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-28)

329. Promise Variation 29: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-29)

330. Await Variation 30: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-30)

331. Decorator Variation 31: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-31)

332. Reflect API Variation 32: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-32)

333. Implicit Coercion Variation 33: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-33)

334. Tree Shaking Variation 34: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-34)

335. XSS Attack Variation 35: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-35)

336. Web Worker Variation 36: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-36)

337. Explicit Resource Management Variation 37: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-37)

338. Data Type Variation 38: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-38)

339. Reference Type Variation 39: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-39)

340. Weak Typing Variation 40: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-40)

341. Temporal Dead Zone Variation 41: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-41)

342. Scope Chain Variation 42: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-42)

343. Memory Heap Variation 43: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-43)

344. Microtask Queue Variation 44: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-44)

345. Callback Hell Variation 45: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-45)

346. Promise Variation 46: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂર્ણ થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-46)

347. Await Variation 47: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-47)

348. Decorator Variation 48: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-48)

349. Reflect API Variation 49: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-49)

350. Implicit Coercion Variation 50: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-50)

351. Tree Shaking Variation 51: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-51)

352. XSS Attack Variation 52: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-52)

353. Web Worker Variation 53: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-53)

354. Explicit Resource Management Variation 54: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-54)

355. Data Type Variation 55: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-55)

356. Reference Type Variation 56: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-56)

357. Weak Typing Variation 57: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-57)

358. Temporal Dead Zone Variation 58: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-58)

359. Scope Chain Variation 59: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-59)

360. Memory Heap Variation 60: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-60)

361. Microtask Queue Variation 61: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-61)

362. Callback Hell Variation 62: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-62)

363. Promise Variation 63: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂર્ણ થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-63)

364. Await Variation 64: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-64)

365. Decorator Variation 65: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-65)

366. Reflect API Variation 66: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-66)

367. Implicit Coercion Variation 67: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-67)

368. Tree Shaking Variation 68: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-68)

369. XSS Attack Variation 69: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્શન ભાગ-69)

- 370. Web Worker Variation 70:** ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-70)
- 371. Explicit Resource Management Variation 71:** using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-71)
- 372. Data Type Variation 72:** વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-72)
- 373. Reference Type Variation 73:** ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-73)
- 374. Weak Typing Variation 74:** જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-74)
- 375. Temporal Dead Zone Variation 75:** let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-75)
- 376. Scope Chain Variation 76:** વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-76)
- 377. Memory Heap Variation 77:** ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-77)
- 378. Microtask Queue Variation 78:** Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-78)
- 379. Callback Hell Variation 79:** એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-79)
- 380. Promise Variation 80:** અસિંક્રોનસ ઓપરેશન લવિષ્માં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-80)
- 381. Await Variation 81:** પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-81)
- 382. Decorator Variation 82:** ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-82)
- 383. Reflect API Variation 83:** ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોર્ડન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-83)
- 384. Implicit Coercion Variation 84:** જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-84)

- 385. Tree Shaking Variation 85:** બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-85)
- 386. XSS Attack Variation 86:** વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-86)
- 387. Web Worker Variation 87:** ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-87)
- 388. Explicit Resource Management Variation 88:** using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-88)
- 389. Data Type Variation 89:** વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-89)
- 390. Reference Type Variation 90:** ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-90)
- 391. Weak Typing Variation 91:** જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-91)
- 392. Temporal Dead Zone Variation 92:** let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-92)
- 393. Scope Chain Variation 93:** વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-93)
- 394. Memory Heap Variation 94:** ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-94)
- 395. Microtask Queue Variation 95:** Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-95)
- 396. Callback Hell Variation 96:** એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-96)
- 397. Promise Variation 97:** અસિંક્રોનસ ઓપરેશન લવિષ્માં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-97)
- 398. Await Variation 98:** પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-98)
- 399. Decorator Variation 99:** ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-99)

400. Reflect API Variation 100: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોર્ડન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-100)

401. Implicit Coercion Variation 101: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-101)

402. Tree Shaking Variation 102: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-102)

403. XSS Attack Variation 103: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-103)

404. Web Worker Variation 104: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-104)

405. Explicit Resource Management Variation 105: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-105)

406. Data Type Variation 106: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-106)

407. Reference Type Variation 107: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-107)

408. Weak Typing Variation 108: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-108)

409. Temporal Dead Zone Variation 109: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-109)

410. Scope Chain Variation 110: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-110)

411. Memory Heap Variation 111: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-111)

412. Microtask Queue Variation 112: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-112)

413. Callback Hell Variation 113: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-113)

414. Promise Variation 114: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-114)

415. Await Variation 115: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-115)

416. Decorator Variation 116: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ડીયર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-116)

417. Reflect API Variation 117: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોર્ડન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-117)

418. Implicit Coercion Variation 118: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-118)

419. Tree Shaking Variation 119: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-119)

420. XSS Attack Variation 120: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-120)

421. Web Worker Variation 121: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-121)

422. Explicit Resource Management Variation 122: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-122)

423. Data Type Variation 123: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-123)

424. Reference Type Variation 124: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-124)

425. Weak Typing Variation 125: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-125)

426. Temporal Dead Zone Variation 126: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-126)

427. Scope Chain Variation 127: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-127)

428. Memory Heap Variation 128: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-128)

429. Microtask Queue Variation 129: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-129)

- 430. Callback Hell Variation 130:** એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-130)
- 431. Promise Variation 131:** અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-131)
- 432. Await Variation 132:** પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-132)
- 433. Decorator Variation 133:** ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-133)
- 434. Reflect API Variation 134:** ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-134)
- 435. Implicit Coercion Variation 135:** જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-135)
- 436. Tree Shaking Variation 136:** બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-136)
- 437. XSS Attack Variation 137:** વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-137)
- 438. Web Worker Variation 138:** ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-138)
- 439. Explicit Resource Management Variation 139:** using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-139)
- 440. Data Type Variation 140:** વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-140)
- 441. Reference Type Variation 141:** ઓબ્જેક્ટ અને એર જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-141)
- 442. Weak Typing Variation 142:** જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-142)
- 443. Temporal Dead Zone Variation 143:** let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-143)
- 444. Scope Chain Variation 144:** વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેઇન કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-144)

- 445. Memory Heap Variation 145:** ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનલિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-145)
- 446. Microtask Queue Variation 146:** Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-146)
- 447. Callback Hell Variation 147:** એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-147)
- 448. Promise Variation 148:** અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-148)
- 449. Await Variation 149:** પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-149)
- 450. Decorator Variation 150:** ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-150)

અસિંક્રોનસ જાવાસ્ક્રિપ્ટ અને ઇવેન્ટ લૂપ (Async JS & Event Loop)

- 451. Scope Chain:** વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેઇન કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-1)
- 452. Memory Heap Variation 2:** ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનલિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-2)
- 453. Microtask Queue Variation 3:** Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-3)
- 454. Callback Hell Variation 4:** એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-4)
- 455. Promise Variation 5:** અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-5)
- 456. Await Variation 6:** પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-6)
- 457. Decorator Variation 7:** ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-7)
- 458. Reflect API Variation 8:** ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-8)

459. Implicit Coercion Variation 9: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-9)

460. Tree Shaking Variation 10: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-10)

461. XSS Attack Variation 11: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-11)

462. Web Worker Variation 12: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-12)

463. Explicit Resource Management Variation 13: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-13)

464. Data Type Variation 14: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-14)

465. Reference Type Variation 15: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-15)

466. Weak Typing Variation 16: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-16)

467. Temporal Dead Zone Variation 17: let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-17)

468. Scope Chain Variation 18: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-18)

469. Memory Heap Variation 19: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિભિંટ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-19)

470. Microtask Queue Variation 20: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી વાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-20)

471. Callback Hell Variation 21: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-21)

472. Promise Variation 22: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-22)

473. Await Variation 23: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-23)

474. Decorator Variation 24: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોર્ડન ટ્રીયર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-24)

475. Reflect API Variation 25: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોર્ડન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-25)

476. Implicit Coercion Variation 26: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-26)

477. Tree Shaking Variation 27: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-27)

478. XSS Attack Variation 28: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-28)

479. Web Worker Variation 29: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-29)

480. Explicit Resource Management Variation 30: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-30)

481. Data Type Variation 31: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-31)

482. Reference Type Variation 32: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-32)

483. Weak Typing Variation 33: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-33)

484. Temporal Dead Zone Variation 34: let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-34)

485. Scope Chain Variation 35: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-35)

486. Memory Heap Variation 36: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિભિંટ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-36)

487. Microtask Queue Variation 37: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી વાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-37)

488. Callback Hell Variation 38: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-38)

489. Promise Variation 39: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-39)

490. Await Variation 40: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-40)

491. Decorator Variation 41: ક્લાસ અથવા મેથડના વોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફોયર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-41)

492. Reflect API Variation 42: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-42)

493. Implicit Coercion Variation 43: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-43)

494. Tree Shaking Variation 44: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-44)

495. XSS Attack Variation 45: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-45)

496. Web Worker Variation 46: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-46)

497. Explicit Resource Management Variation 47: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-47)

498. Data Type Variation 48: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-48)

499. Reference Type Variation 49: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-49)

500. Weak Typing Variation 50: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-50)

501. Temporal Dead Zone Variation 51: let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-51)

502. Scope Chain Variation 52: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-52)

503. Memory Heap Variation 53: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનલિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-53)

504. Microtask Queue Variation 54: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી વાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-54)

505. Callback Hell Variation 55: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-55)

506. Promise Variation 56: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-56)

507. Await Variation 57: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-57)

508. Decorator Variation 58: ક્લાસ અથવા મેથડના વોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફોયર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-58)

509. Reflect API Variation 59: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-59)

510. Implicit Coercion Variation 60: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-60)

511. Tree Shaking Variation 61: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-61)

512. XSS Attack Variation 62: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-62)

513. Web Worker Variation 63: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-63)

514. Explicit Resource Management Variation 64: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-64)

515. Data Type Variation 65: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-65)

516. Reference Type Variation 66: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-66)

517. Weak Typing Variation 67: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-67)

518. Temporal Dead Zone Variation 68: let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-68)

519. Scope Chain Variation 69: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-69)

520. Memory Heap Variation 70: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-70)

521. Microtask Queue Variation 71: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-71)

522. Callback Hell Variation 72: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-72)

523. Promise Variation 73: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-73)

524. Await Variation 74: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-74)

525. Decorator Variation 75: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોર્ડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-75)

526. Reflect API Variation 76: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોર્ડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-76)

527. Implicit Coercion Variation 77: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-77)

528. Tree Shaking Variation 78: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-78)

529. XSS Attack Variation 79: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-79)

530. Web Worker Variation 80: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-80)

531. Explicit Resource Management Variation 81: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-81)

532. Data Type Variation 82: વૈભ્ય ક્યા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-82)

533. Reference Type Variation 83: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-83)

534. Weak Typing Variation 84: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-84)

535. Temporal Dead Zone Variation 85: let અને const વેરીએબલ ડિફાઇ થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-85)

536. Scope Chain Variation 86: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-86)

537. Memory Heap Variation 87: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-87)

538. Microtask Queue Variation 88: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-88)

539. Callback Hell Variation 89: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-89)

540. Promise Variation 90: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-90)

541. Await Variation 91: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-91)

542. Decorator Variation 92: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોર્ડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-92)

543. Reflect API Variation 93: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોર્ડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-93)

544. Implicit Coercion Variation 94: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-94)

545. Tree Shaking Variation 95: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-95)

546. XSS Attack Variation 96: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-96)

547. Web Worker Variation 97: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-97)

548. Explicit Resource Management Variation 98: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-98)

549. Data Type Variation 99: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-99)

550. Reference Type Variation 100: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-100)

551. Weak Typing Variation 101: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-101)

552. Temporal Dead Zone Variation 102: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-102)

553. Scope Chain Variation 103: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-103)

554. Memory Heap Variation 104: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમેટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-104)

555. Microtask Queue Variation 105: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-105)

556. Callback Hell Variation 106: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-106)

557. Promise Variation 107: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-107)

558. Await Variation 108: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-108)

559. Decorator Variation 109: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-109)

560. Reflect API Variation 110: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-110)

561. Implicit Coercion Variation 111: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-111)

562. Tree Shaking Variation 112: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-112)

563. XSS Attack Variation 113: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-113)

564. Web Worker Variation 114: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-114)

565. Explicit Resource Management Variation 115: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-115)

566. Data Type Variation 116: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-116)

567. Reference Type Variation 117: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-117)

568. Weak Typing Variation 118: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-118)

569. Temporal Dead Zone Variation 119: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-119)

570. Scope Chain Variation 120: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-120)

571. Memory Heap Variation 121: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમેટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-121)

572. Microtask Queue Variation 122: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-122)

573. Callback Hell Variation 123: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-123)

574. Promise Variation 124: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-124)

575. Await Variation 125: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-125)

576. Decorator Variation 126: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-126)

577. Reflect API Variation 127: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-127)

578. Implicit Coercion Variation 128: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-128)

579. Tree Shaking Variation 129: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-129)

580. XSS Attack Variation 130: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-130)

581. Web Worker Variation 131: ભારે ડેલ્યુવેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-131)

582. Explicit Resource Management Variation 132: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-132)

583. Data Type Variation 133: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-133)

584. Reference Type Variation 134: ઓબ્જેક્ટ અને એર જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-134)

585. Weak Typing Variation 135: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-135)

586. Temporal Dead Zone Variation 136: let અને const વેરીએબલ ડિફાઇનેશન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-136)

587. Scope Chain Variation 137: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-137)

588. Memory Heap Variation 138: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-138)

589. Microtask Queue Variation 139: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-139)

590. Callback Hell Variation 140: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-140)

591. Promise Variation 141: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-141)

592. Await Variation 142: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-142)

593. Decorator Variation 143: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ડીયર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-143)

594. Reflect API Variation 144: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોર્ડન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-144)

595. Implicit Coercion Variation 145: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-145)

596. Tree Shaking Variation 146: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-146)

597. XSS Attack Variation 147: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-147)

598. Web Worker Variation 148: ભારે ડેલ્યુવેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-148)

599. Explicit Resource Management Variation 149: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-149)

600. Data Type Variation 150: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-150)

બ્રાઉઝર ઓબ્જેક્ટ મોડલ અને DOM (BOM & DOM Architecture)

601. Reflect API: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોર્ડન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-1)

602. Implicit Coercion Variation 2: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-2)

603. Tree Shaking Variation 3: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-3)

604. XSS Attack Variation 4: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-4)

605. Web Worker Variation 5: ભારે ડેલ્યુવેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-5)

606. Explicit Resource Management Variation 6: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-6)

607. Data Type Variation 7: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-7)

608. Reference Type Variation 8: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-8)

609. Weak Typing Variation 9: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-9)

610. Temporal Dead Zone Variation 10: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-10)

611. Scope Chain Variation 11: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-11)

612. Memory Heap Variation 12: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-12)

613. Microtask Queue Variation 13: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી વાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-13)

614. Callback Hell Variation 14: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-14)

615. Promise Variation 15: અસિંક્રોનસ ઓપરેશન લવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-15)

616. Await Variation 16: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-16)

617. Decorator Variation 17: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-17)

618. Reflect API Variation 18: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-18)

619. Implicit Coercion Variation 19: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-19)

620. Tree Shaking Variation 20: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-20)

621. XSS Attack Variation 21: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-21)

622. Web Worker Variation 22: ભારે ડેલ્યુવેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-22)

623. Explicit Resource Management Variation 23: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-23)

624. Data Type Variation 24: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-24)

625. Reference Type Variation 25: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-25)

626. Weak Typing Variation 26: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-26)

627. Temporal Dead Zone Variation 27: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-27)

628. Scope Chain Variation 28: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-28)

629. Memory Heap Variation 29: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-29)

630. Microtask Queue Variation 30: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી વાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-30)

631. Callback Hell Variation 31: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-31)

632. Promise Variation 32: અસિંક્રોનસ ઓપરેશન લવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-32)

633. Await Variation 33: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-33)

634. Decorator Variation 34: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-34)

635. Reflect API Variation 35: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-35)

636. Implicit Coercion Variation 36: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-36)

637. Tree Shaking Variation 37: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-37)

638. XSS Attack Variation 38: વેબસાઇટમાં હેક્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-38)

639. Web Worker Variation 39: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-39)

640. Explicit Resource Management Variation 40: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-40)

641. Data Type Variation 41: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-41)

642. Reference Type Variation 42: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-42)

643. Weak Typing Variation 43: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-43)

644. Temporal Dead Zone Variation 44: let અને const વેરીએબલ ડિફાલ્ટ થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-44)

645. Scope Chain Variation 45: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-45)

646. Memory Heap Variation 46: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-46)

647. Microtask Queue Variation 47: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-47)

648. Callback Hell Variation 48: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-48)

649. Promise Variation 49: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂર્ણ થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-49)

650. Await Variation 50: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-50)

651. Decorator Variation 51: ક્લાસ અથવા મેથડના વોજિકને મોડિફાય કરતું એપ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-51)

652. Reflect API Variation 52: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-52)

653. Implicit Coercion Variation 53: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-53)

654. Tree Shaking Variation 54: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-54)

655. XSS Attack Variation 55: વેબસાઇટમાં હેક્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-55)

656. Web Worker Variation 56: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-56)

657. Explicit Resource Management Variation 57: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-57)

658. Data Type Variation 58: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-58)

659. Reference Type Variation 59: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-59)

660. Weak Typing Variation 60: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-60)

661. Temporal Dead Zone Variation 61: let અને const વેરીએબલ ડિફાલ્ટ થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-61)

662. Scope Chain Variation 62: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-62)

663. Memory Heap Variation 63: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-63)

664. Microtask Queue Variation 64: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-64)

665. Callback Hell Variation 65: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-65)

666. Promise Variation 66: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂર્ણ થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-66)

667. Await Variation 67: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-67)

668. Decorator Variation 68: ક્લાસ અથવા મેથડના વોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-68)

669. Reflect API Variation 69: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-69)

670. Implicit Coercion Variation 70: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-70)

671. Tree Shaking Variation 71: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-71)

672. XSS Attack Variation 72: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-72)

673. Web Worker Variation 73: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-73)

674. Explicit Resource Management Variation 74: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-74)

675. Data Type Variation 75: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-75)

676. Reference Type Variation 76: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-76)

677. Weak Typing Variation 77: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-77)

678. Temporal Dead Zone Variation 78: let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-78)

679. Scope Chain Variation 79: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-79)

680. Memory Heap Variation 80: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-80)

681. Microtask Queue Variation 81: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી વાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-81)

682. Callback Hell Variation 82: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અથડો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-82)

683. Promise Variation 83: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂર્ણ થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-83)

684. Await Variation 84: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-84)

685. Decorator Variation 85: ક્લાસ અથવા મેથડના વોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-85)

686. Reflect API Variation 86: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-86)

687. Implicit Coercion Variation 87: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-87)

688. Tree Shaking Variation 88: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-88)

689. XSS Attack Variation 89: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-89)

690. Web Worker Variation 90: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-90)

691. Explicit Resource Management Variation 91: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-91)

692. Data Type Variation 92: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-92)

693. Reference Type Variation 93: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-93)

694. Weak Typing Variation 94: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-94)

695. Temporal Dead Zone Variation 95: let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-95)

696. Scope Chain Variation 96: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-96)

697. Memory Heap Variation 97: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-97)

698. Microtask Queue Variation 98: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન.
(વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-98)

699. Callback Hell Variation 99: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-99)

700. Promise Variation 100: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-100)

701. Await Variation 101: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-101)

702. Decorator Variation 102: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ડીયર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-102)

703. Reflect API Variation 103: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-103)

704. Implicit Coercion Variation 104: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-104)

705. Tree Shaking Variation 105: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-105)

706. XSS Attack Variation 106: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-106)

707. Web Worker Variation 107: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-107)

708. Explicit Resource Management Variation 108: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-108)

709. Data Type Variation 109: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-109)

710. Reference Type Variation 110: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-110)

711. Weak Typing Variation 111: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-111)

712. Temporal Dead Zone Variation 112: let અને const વેરીએબલ ડિફાઇ થાય તે પહેલાં અક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-112)

713. Scope Chain Variation 113: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેઇન કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-113)

714. Memory Heap Variation 114: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનલિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-114)

715. Microtask Queue Variation 115: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-115)

716. Callback Hell Variation 116: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-116)

717. Promise Variation 117: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-117)

718. Await Variation 118: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-118)

719. Decorator Variation 119: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ડીયર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-119)

720. Reflect API Variation 120: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-120)

721. Implicit Coercion Variation 121: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-121)

722. Tree Shaking Variation 122: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-122)

723. XSS Attack Variation 123: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-123)

724. Web Worker Variation 124: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-124)

725. Explicit Resource Management Variation 125: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-125)

726. Data Type Variation 126: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-126)

727. Reference Type Variation 127: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-127)

728. Weak Typing Variation 128: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ ફર્વર્સન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-128)

729. Temporal Dead Zone Variation 129: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-129)

730. Scope Chain Variation 130: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-130)

731. Memory Heap Variation 131: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-131)

732. Microtask Queue Variation 132: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-132)

733. Callback Hell Variation 133: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-133)

734. Promise Variation 134: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-134)

735. Await Variation 135: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-135)

736. Decorator Variation 136: ક્લાસ અથવા મેથડના બોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-136)

737. Reflect API Variation 137: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-137)

738. Implicit Coercion Variation 138: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-138)

739. Tree Shaking Variation 139: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-139)

740. XSS Attack Variation 140: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-140)

741. Web Worker Variation 141: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-141)

742. Explicit Resource Management Variation 142: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-142)

743. Data Type Variation 143: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-143)

744. Reference Type Variation 144: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-144)

745. Weak Typing Variation 145: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ ફર્વર્સન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-145)

746. Temporal Dead Zone Variation 146: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-146)

747. Scope Chain Variation 147: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-147)

748. Memory Heap Variation 148: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-148)

749. Microtask Queue Variation 149: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-149)

750. Callback Hell Variation 150: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-150)

એન્જિન ઇન્ટરનલ્સ, ક્વોર્ક્સ અને મેમરી (Engine Internals & Memory)

751. Reference Type: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-1)

752. Weak Typing Variation 2: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ ફર્વર્સન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-2)

753. Temporal Dead Zone Variation 3: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-3)

754. Scope Chain Variation 4: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-4)

755. Memory Heap Variation 5: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-5)

756. Microtask Queue Variation 6: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-6)

757. Callback Hell Variation 7: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-7)

758. Promise Variation 8: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-8)

759. Await Variation 9: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-9)

760. Decorator Variation 10: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-10)

761. Reflect API Variation 11: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-11)

762. Implicit Coercion Variation 12: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-12)

763. Tree Shaking Variation 13: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-13)

764. XSS Attack Variation 14: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-14)

765. Web Worker Variation 15: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-15)

766. Explicit Resource Management Variation 16: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-16)

767. Data Type Variation 17: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-17)

768. Reference Type Variation 18: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-18)

769. Weak Typing Variation 19: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-19)

770. Temporal Dead Zone Variation 20: let અને const વેરીએબલ ડિક્લેરે થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-20)

771. Scope Chain Variation 21: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેઇન કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-21)

772. Memory Heap Variation 22: ઓબ્જેક્ટ્સ અને અનન્ટરુક્ટર્ડ ડેટા સ્ટોર કરવા માટેની અનલિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-22)

773. Microtask Queue Variation 23: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-23)

774. Callback Hell Variation 24: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-24)

775. Promise Variation 25: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-25)

776. Await Variation 26: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-26)

777. Decorator Variation 27: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-27)

778. Reflect API Variation 28: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-28)

779. Implicit Coercion Variation 29: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-29)

780. Tree Shaking Variation 30: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-30)

781. XSS Attack Variation 31: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-31)

782. Web Worker Variation 32: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-32)

783. Explicit Resource Management Variation 33: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-33)

784. Data Type Variation 34: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-34)

785. Reference Type Variation 35: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-35)

786. Weak Typing Variation 36: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-36)

787. Temporal Dead Zone Variation 37: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-37)

788. Scope Chain Variation 38: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-38)

789. Memory Heap Variation 39: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-39)

790. Microtask Queue Variation 40: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી વાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-40)

791. Callback Hell Variation 41: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-41)

792. Promise Variation 42: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂર્ણ થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-42)

793. Await Variation 43: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-43)

794. Decorator Variation 44: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-44)

795. Reflect API Variation 45: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-45)

796. Implicit Coercion Variation 46: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે જેમ કે 5 + '5' = '55'. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-46)

797. Tree Shaking Variation 47: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-47)

798. XSS Attack Variation 48: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-48)

799. Web Worker Variation 49: ભારે ડેલ્યુવેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-49)

800. Explicit Resource Management Variation 50: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-50)

801. Data Type Variation 51: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-51)

802. Reference Type Variation 52: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-52)

803. Weak Typing Variation 53: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-53)

804. Temporal Dead Zone Variation 54: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-54)

805. Scope Chain Variation 55: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-55)

806. Memory Heap Variation 56: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-56)

807. Microtask Queue Variation 57: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી વાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-57)

808. Callback Hell Variation 58: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-58)

809. Promise Variation 59: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂર્ણ થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-59)

810. Await Variation 60: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-60)

811. Decorator Variation 61: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-61)

812. Reflect API Variation 62: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-62)

813. Implicit Coercion Variation 63: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે જેમ કે 5 + '5' = '55'. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-63)

814. Tree Shaking Variation 64: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-64)

815. XSS Attack Variation 65: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-65)

816. Web Worker Variation 66: ભારે ડેલ્યુવેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-66)

817. Explicit Resource Management Variation 67: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-67)

818. Data Type Variation 68: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-68)

819. Reference Type Variation 69: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-69)

820. Weak Typing Variation 70: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-70)

821. Temporal Dead Zone Variation 71: let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-71)

822. Scope Chain Variation 72: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેઈન કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-72)

823. Memory Heap Variation 73: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-73)

824. Microtask Queue Variation 74: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-74)

825. Callback Hell Variation 75: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-75)

826. Promise Variation 76: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂર્ણ થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-76)

827. Await Variation 77: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-77)

828. Decorator Variation 78: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-78)

829. Reflect API Variation 79: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-79)

830. Implicit Coercion Variation 80: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-80)

831. Tree Shaking Variation 81: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-81)

832. XSS Attack Variation 82: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-82)

833. Web Worker Variation 83: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-83)

834. Explicit Resource Management Variation 84: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-84)

835. Data Type Variation 85: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-85)

836. Reference Type Variation 86: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-86)

837. Weak Typing Variation 87: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-87)

838. Temporal Dead Zone Variation 88: let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-88)

839. Scope Chain Variation 89: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેઈન કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-89)

840. Memory Heap Variation 90: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-90)

841. Microtask Queue Variation 91: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-91)

842. Callback Hell Variation 92: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-92)

843. Promise Variation 93: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂર્ણ થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-93)

844. Await Variation 94: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-94)

845. Decorator Variation 95: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-95)

846. Reflect API Variation 96: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-96)

847. Implicit Coercion Variation 97: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-97)

848. Tree Shaking Variation 98: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-98)

849. XSS Attack Variation 99: વેબસાઇટમાં હેક્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-99)

850. Web Worker Variation 100: ભારે ડેલ્યુવેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-100)

851. Explicit Resource Management Variation 101: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-101)

852. Data Type Variation 102: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-102)

853. Reference Type Variation 103: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-103)

854. Weak Typing Variation 104: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-104)

855. Temporal Dead Zone Variation 105: let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-105)

856. Scope Chain Variation 106: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-106)

857. Memory Heap Variation 107: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિભિંટ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-107)

858. Microtask Queue Variation 108: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-108)

859. Callback Hell Variation 109: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-109)

860. Promise Variation 110: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-110)

861. Await Variation 111: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-111)

862. Decorator Variation 112: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ડીયર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-112)

863. Reflect API Variation 113: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-113)

864. Implicit Coercion Variation 114: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-114)

865. Tree Shaking Variation 115: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-115)

866. XSS Attack Variation 116: વેબસાઇટમાં હેક્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-116)

867. Web Worker Variation 117: ભારે ડેલ્યુવેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોર્ડન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-117)

868. Explicit Resource Management Variation 118: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-118)

869. Data Type Variation 119: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-119)

870. Reference Type Variation 120: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-120)

871. Weak Typing Variation 121: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-121)

872. Temporal Dead Zone Variation 122: let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-122)

873. Scope Chain Variation 123: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-123)

874. Memory Heap Variation 124: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિભિંટ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-124)

875. Microtask Queue Variation 125: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-125)

876. Callback Hell Variation 126: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-126)

877. Promise Variation 127: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-127)

878. Await Variation 128: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-128)

879. Decorator Variation 129: ક્લાસ અથવા મેથડના બોજિંગને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-129)

880. Reflect API Variation 130: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-130)

881. Implicit Coercion Variation 131: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-131)

882. Tree Shaking Variation 132: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-132)

883. XSS Attack Variation 133: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-133)

884. Web Worker Variation 134: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-134)

885. Explicit Resource Management Variation 135: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-135)

886. Data Type Variation 136: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-136)

887. Reference Type Variation 137: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-137)

888. Weak Typing Variation 138: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-138)

889. Temporal Dead Zone Variation 139: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-139)

890. Scope Chain Variation 140: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-140)

891. Memory Heap Variation 141: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનલિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-141)

892. Microtask Queue Variation 142: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-142)

893. Callback Hell Variation 143: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધારો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-143)

894. Promise Variation 144: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-144)

895. Await Variation 145: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-145)

896. Decorator Variation 146: ક્લાસ અથવા મેથડના બોજિંગને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-146)

897. Reflect API Variation 147: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-147)

898. Implicit Coercion Variation 148: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-148)

899. Tree Shaking Variation 149: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-149)

900. XSS Attack Variation 150: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-150)

વેબ સિક્યોરિટી અને મોડર્ન ESNext ફીચર્સ (Security & ESNext)

901. Promise: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-1)

902. Await Variation 2: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-2)

903. Decorator Variation 3: ક્લાસ અથવા મેથડના બોજિંગને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-3)

904. Reflect API Variation 4: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-4)

905. Implicit Coercion Variation 5: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-5)

906. Tree Shaking Variation 6: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-6)

907. XSS Attack Variation 7: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-7)

908. Web Worker Variation 8: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-8)

909. Explicit Resource Management Variation 9: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-9)

910. Data Type Variation 10: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-10)

911. Reference Type Variation 11: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-11)

912. Weak Typing Variation 12: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-12)

913. Temporal Dead Zone Variation 13: let અને const વેરીએબલ ડિફાઇઝ થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-13)

914. Scope Chain Variation 14: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-14)

915. Memory Heap Variation 15: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-15)

916. Microtask Queue Variation 16: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-16)

917. Callback Hell Variation 17: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-17)

918. Promise Variation 18: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂર્ણ થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-18)

919. Await Variation 19: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-19)

920. Decorator Variation 20: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ડીયર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-20)

921. Reflect API Variation 21: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-21)

922. Implicit Coercion Variation 22: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-22)

923. Tree Shaking Variation 23: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-23)

924. XSS Attack Variation 24: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માવિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-24)

925. Web Worker Variation 25: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-25)

926. Explicit Resource Management Variation 26: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-26)

927. Data Type Variation 27: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-27)

928. Reference Type Variation 28: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-28)

929. Weak Typing Variation 29: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-29)

930. Temporal Dead Zone Variation 30: let અને const વેરીએબલ ડિફાઇઝ થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-30)

931. Scope Chain Variation 31: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-31)

932. Memory Heap Variation 32: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનબિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-32)

933. Microtask Queue Variation 33: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-33)

934. Callback Hell Variation 34: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-34)

935. Promise Variation 35: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂર્ણ થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-35)

936. Await Variation 36: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-36)

937. Decorator Variation 37: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-37)

938. Reflect API Variation 38: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-38)

939. Implicit Coercion Variation 39: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-39)

940. Tree Shaking Variation 40: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-40)

941. XSS Attack Variation 41: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-41)

942. Web Worker Variation 42: ભારે ડેલ્યુવેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-42)

943. Explicit Resource Management Variation 43: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવો. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-43)

944. Data Type Variation 44: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-44)

945. Reference Type Variation 45: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-45)

946. Weak Typing Variation 46: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-46)

947. Temporal Dead Zone Variation 47: let અને const વેરીએબલ ડિફાઇ થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-47)

948. Scope Chain Variation 48: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-48)

949. Memory Heap Variation 49: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-49)

950. Microtask Queue Variation 50: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-50)

951. Callback Hell Variation 51: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અધરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-51)

952. Promise Variation 52: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-52)

953. Await Variation 53: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-53)

954. Decorator Variation 54: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-54)

955. Reflect API Variation 55: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-55)

956. Implicit Coercion Variation 56: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-56)

957. Tree Shaking Variation 57: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-57)

958. XSS Attack Variation 58: વેબસાઇટમાં હેક્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-58)

959. Web Worker Variation 59: ભારે ડેલ્યુવેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-59)

960. Explicit Resource Management Variation 60: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવો. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-60)

961. Data Type Variation 61: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-61)

962. Reference Type Variation 62: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-62)

963. Weak Typing Variation 63: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-63)

964. Temporal Dead Zone Variation 64: let અને const વેરીએબલ ડિફાઇ થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-64)

965. Scope Chain Variation 65: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-65)

966. Memory Heap Variation 66: ઓબ્જેક્ટ્સ અને અનસ્ટૉર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-66)

967. Microtask Queue Variation 67: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-67)

968. Callback Hell Variation 68: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતી પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-68)

969. Promise Variation 69: અસિંક્રોનસ ઓપરેશન લવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-69)

970. Await Variation 70: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-70)

971. Decorator Variation 71: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-71)

972. Reflect API Variation 72: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-72)

973. Implicit Coercion Variation 73: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-73)

974. Tree Shaking Variation 74: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-74)

975. XSS Attack Variation 75: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-75)

976. Web Worker Variation 76: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-76)

977. Explicit Resource Management Variation 77: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-77)

978. Data Type Variation 78: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-78)

979. Reference Type Variation 79: ઓબ્જેક્ટ અને એર જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-79)

980. Weak Typing Variation 80: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-80)

981. Temporal Dead Zone Variation 81: let અને const વેરીએબલ ડિફાઇન થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-81)

982. Scope Chain Variation 82: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-82)

983. Memory Heap Variation 83: ઓબ્જેક્ટ્સ અને અનસ્ટૉર્ડ ડેટા સ્ટોર કરવા માટેની અનવિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-83)

984. Microtask Queue Variation 84: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-84)

985. Callback Hell Variation 85: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-85)

986. Promise Variation 86: અસિંક્રોનસ ઓપરેશન લવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-86)

987. Await Variation 87: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-87)

988. Decorator Variation 88: ક્લાસ અથવા મેથડના લોજિકને મોડિફાય કરતું અલ્ટ્રા-મોડર્ન ફીચર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-88)

989. Reflect API Variation 89: ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હેન્ડલ કરવા માટેનું મોડર્ન API. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-89)

990. Implicit Coercion Variation 90: જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલે તે (જેમ કે 5 + '5' = '55'). (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-90)

991. Tree Shaking Variation 91: બિલ્ડ પ્રોસેસ વખતે ન વપરાયેલો કોડ ઓટોમેટિકલી હટાવવાની પદ્ધતિ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-91)

992. XSS Attack Variation 92: વેબસાઇટમાં હેકર્સ દ્વારા નકામો માલિશિયસ સ્ક્રિપ્ટ ઇન્જેક્ટ કરવાનો એટેક. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-92)

993. Web Worker Variation 93: ભારે કેલ્ક્યુલેશનને બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની મોડર્ન રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-93)

994. Explicit Resource Management Variation 94: using કીવર્ડ દ્વારા સ્કોપ પૂરો થતાં મેમરી ઓટોમેટિક ક્લીન કરવી. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-94)

995. Data Type Variation 95: વેલ્યુ કયા પ્રકારની છે (નંબર, સ્ટ્રિંગ વગેરે) તે નક્કી કરતી સિસ્ટમ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-95)

996. Reference Type Variation 96: ઓબ્જેક્ટ અને એરે જેવા ડેટા ટાઇપ જે હીપ મેમરીમાં સ્ટોર થાય છે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-96)

997. Weak Typing Variation 97: જાવાસ્ક્રિપ્ટ ઓટોમેટિક ટાઇપ કન્વર્ઝન (Coercion) કરવાની મંજૂરી આપે છે તે. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-97)

998. Temporal Dead Zone Variation 98: let અને const વેરીએબલ ડિક્લેર થાય તે પહેલાં એક્સેસ ન કરી શકાય તે ઝોન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-98)

999. Scope Chain Variation 99: વેરીએબલ શોધવા માટે કરન્ટ સ્કોપથી ગ્લોબલ સ્કોપ સુધી ચેકિંગ કરવાની રીત. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-99)

1000. Memory Heap Variation 100: ઓબ્જેક્ટ્સ અને અનસ્ટ્રક્ચર્ડ ડેટા સ્ટોર કરવા માટેની અનલિમિટેડ મેમરી જગ્યા. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-100)

1001. Microtask Queue Variation 101: Promises ના રિઝોલ્યુશન રાખતી હાઇ-પ્રાયોરિટી લાઇન. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-101)

1002. Callback Hell Variation 102: એકની અંદર એક અસંખ્ય કોલબેક ફંક્શન્સ લખવાથી બનતો પિરામિડ જેવો અઘરો કોડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-102)

1003. Promise Variation 103: અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-103)

1004. Await Variation 104: પ્રોમિસ રિઝોલ્વ થાય ત્યાં સુધી વેઇટ કરાવતો કીવર્ડ. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-104)

1005. Decorator Variation 105: ક્લાસ અથવા મેથડના લોજિકને મોડિફાઇ કરતું અલ્ટ્રા-મોડર્ન ડ્રોયર. (વિશિષ્ટ કન્સેપ્ટ આર્કિટેક્ચર ભાગ-105)

JavaScript 1000+ Ultimate Mega Master List

આ ફાઇલમાં JavaScript ના ૧૦૦૦+ તમામ નાના-મોટા કન્સેપ્ટ્સ, ટર્મ્સ અને હિડન મિકેનિઝમ્સને એકદમ સરળ ગુજરાતી ભાષામાં સમજાવવામાં આવ્યા છે.

Variables & Scoping

ID	Concept / Term	Explanation (ગુજરાતીમાં)
1	Variables	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Variables તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
2	Scope	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Scope તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
3	Var	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Var તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
4	Let	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Let તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
5	Const	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Const તરીકે

		ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
18	Reassignment	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Reassignment તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
19	Initialization	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Initialization તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
20	Memory Allocations	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Memory Allocations તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
21	Evaluation	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Evaluation તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
22	Context	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Context તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
23	Outer Environment	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Outer

		Environment તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
24	Block Statement	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Block Statement તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
25	TDZ Trigger	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર TDZ Trigger તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
26	Catch Block Scope	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Catch Block Scope તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
27	Module Scope	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Module Scope તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
28	Eval Scope	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Eval Scope તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
29	Function Declarations vs Expressions	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર

		એન્જિન લેવલ પર Function Declarations vs Expressions તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
30	Variable Lifecycles	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Variable Lifecycles તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
31	Scope Leakage	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Scope Leakage તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
32	Strict Mode Scoping	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Strict Mode Scoping તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
33	Lexical Environments	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Lexical Environments તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
34	Environment Records	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Environment Records તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

		છે.
35	Declarative Environment Records	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Declarative Environment Records તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
36	Object Environment Records.	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Object Environment Records તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

Data Types & Memory

ID	Concept / Term	Explanation (ગૃહીતીર્થી)
37	Primitive Types	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Primitive Types તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
38	Reference Types	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Reference Types તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
39	Number	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Number તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
40	String	JavaScript નો આ એક

		અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
53	Garbage Collection	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Garbage Collection તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
54	Mark and Sweep	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Mark and Sweep તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
55	Memory Leaks	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Memory Leaks તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
56	Mutable vs Immutable	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Mutable vs Immutable તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
57	Shallow Copy	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Shallow Copy તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
58	Deep Copy	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર

		એન્જિન લેવલ પર Deep Copy તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
59	Type Coercion	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Type Coercion તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
60	Implicit Coercion	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Implicit Coercion તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
61	Explicit Conversion	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Explicit Conversion તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
62	Object.is	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Object.is તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
63	NaN	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર NaN તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
64	Infinity	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ

76	V8 Deserialization	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર V8 Deserialization તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
77	Reference Counting.	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Reference Counting. તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

Functions & Executions

ID	Concept / Term	Explanation (ગુજરાતીમાં)
78	First-Class Functions	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર First-Class Functions તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
79	Higher-Order Functions	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Higher-Order Functions તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
80	Callback Functions	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Callback Functions તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

81	Arrow Functions	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Arrow Functions તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
82	Lexical this	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Lexical this તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
83	Anonymous Functions	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Anonymous Functions તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
84	Named Function Expressions	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Named Function Expressions તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
85	IIFE	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર IIFE તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
86	Currying	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Currying તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

		વ્યાપકપણે થાય છે.
87	Partial Application	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Partial Application તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
88	Pure Functions	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Pure Functions તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
89	Side Effects	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Side Effects તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
90	Referential Transparency	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Referential Transparency તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
91	Function Composition	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Function Composition તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
92	Recursion	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Recursion

		તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
93	Tail Call Optimization	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Tail Call Optimization તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
94	Call Stack Limits	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Call Stack Limits તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
95	Stack Overflow	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Stack Overflow તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
96	Arguments Object	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Arguments Object તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
97	Rest Parameters	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Rest Parameters તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
98	Spread Syntax	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર

		એનજિન લેવલ પર Spread Syntax તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
99	Parameter Default Values	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનજિન લેવલ પર Parameter Default Values તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
100	Memoization	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનજિન લેવલ પર Memoization તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
101	Debouncing	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનજિન લેવલ પર Debouncing તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
102	Throttling	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનજિન લેવલ પર Throttling તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
103	Method Chaining	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનજિન લેવલ પર Method Chaining તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
104	Constructor Functions	JavaScript નો આ એક

		મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનજિન લેવલ પર Constructor Functions તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
105	Factory Functions	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનજિન લેવલ પર Factory Functions તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
106	Generator Functions	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનજિન લેવલ પર Generator Functions તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
107	Yield Keyword	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનજિન લેવલ પર Yield Keyword તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
108	Iterator Protocol	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનજિન લેવલ પર Iterator Protocol તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
109	Iterable Protocol	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનજિન લેવલ પર Iterable Protocol તરીકે ઓળખાય છે.

		અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
110	Async Iterators	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Async Iterators તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
111	Generator Return	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Generator Return તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
112	Yield* Delegation	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Yield* Delegation તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
113	Function.prototype.call	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Function.prototype.call તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
114	Function.prototype.apply	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Function.prototype.apply તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
115	Function.prototype.bind	JavaScript નો આ એક

		મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Function.prototype.bind તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
116	Invocation Context	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Invocation Context તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
117	Function Scoping	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Function Scoping તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
118	Arity	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Arity તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
119	Arguments Binding	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Arguments Binding તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
120	Caller Property	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Caller Property તરીકે ઓળખાય છે અને તેનો ઉપયોગ

		ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
121	Function.length.	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Function.length તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

Asynchronous JS & Event Loop

ID	Concept / Term	Explanation (ગૂજરાતીમાં)
122	Asynchronous Programming	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Asynchronous Programming તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
123	Event Loop	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Event Loop તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
124	Call Stack	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Call Stack તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
125	Web APIs	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Web APIs તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

126	Task Queue	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Task Queue તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
127	Macrotask Queue	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Macrotask Queue તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
128	Microtask Queue	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Microtask Queue તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
129	MutationObserver	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર MutationObserver તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
130	Event Loop Tick	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Event Loop Tick તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
131	Starvation	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Starvation તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
132	Callback Hell	JavaScript નો આ એક

		મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Promise.race તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
145	Promise.allSettled	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Promise.allSettled તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
146	Promise.any	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Promise.any તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
147	Thenable	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Thenable તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
148	Catch Block	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Catch Block તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
149	Finally Block	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Finally Block તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

150	Async/Await	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Async/Await તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
151	Await Engine Mechanics	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Await Engine Mechanics તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
152	Error Propagation	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Error Propagation તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
153	Unhandled Rejections	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Unhandled Rejections તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
154	Event Handlers	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Event Handlers તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
155	SetTimeout	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર SetTimeout

		તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
156	SetInterval	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર SetInterval તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
157	ClearTimeout	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર ClearTimeout તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
158	ClearInterval	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર ClearInterval તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
159	RequestAnimationFrame	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર RequestAnimationFrame તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
160	SetImmediate	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર SetImmediate તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
161	QueueMicrotask	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ

		એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર QueueMicrotask તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
162	Non-blocking I/O	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Non-blocking I/O તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
163	Single-threaded Engine	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Single-threaded Engine તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
164	Concurrency Model	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Concurrency Model તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
165	Promise.try	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Promise.try તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
166	Top-level Await.	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Top-level Await. તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

224	Object Chaining	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Object Chaining તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
225	Property Shadowing	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Property Shadowing તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
226	Enumerable Properties	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Enumerable Properties તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
227	Own Properties	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Own Properties તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
228	Object.keys	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Object.keys તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
229	Object.values	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Object.values તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
230	Object.entries	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Object.entries તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
231	Object.getOwnPropertyNames	JavaScript નો આ એક મહત્વપૂર્ણ

		કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Object.getOwnPropertyNames તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
232	Object.getOwnPropertySymbols	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Object.getOwnPropertySymbols તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
233	Instanceof Operator	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Instanceof Operator તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
234	Typeof Operator	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Typeof Operator તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
235	Constructor Property.	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Constructor Property. તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

Browser BOM, DOM & Web APIs

ID	Concept / Term	Explanation (ગુજરાતીમાં)
236	DOM	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર DOM તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
237	BOM	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ

		એનિમેશન લેવલ પર ClassList.contains તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
273	CSSOM	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમેશન લેવલ પર CSSOM તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
274	Style Property	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમેશન લેવલ પર Style Property તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
275	ComputedStyle	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમેશન લેવલ પર ComputedStyle તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
276	Layout Thrashing	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમેશન લેવલ પર Layout Thrashing તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
277	Repaint	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમેશન લેવલ પર Repaint તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
278	Reflow	JavaScript નો આ એક

		મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમેશન લેવલ પર Reflow તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
279	Event Handling	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમેશન લેવલ પર Event Handling તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
280	Event Listeners	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમેશન લેવલ પર Event Listeners તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
281	AddEventListener	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમેશન લેવલ પર AddEventListener તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
282	RemoveEventListener	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમેશન લેવલ પર RemoveEventListener તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
283	Event Object	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમેશન લેવલ પર Event Object તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં

		વ્યાપકપણે થાય છે.
284	Event.target	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Event.target તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
285	Event.currentTarget	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Event.currentTarget તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
286	Event Bubbling	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Event Bubbling તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
287	Event Capturing	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Event Capturing તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
288	StopPropagation	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર StopPropagation તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
289	StopImmediatePropagation	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર

		એન્જિન લેવલ પર StopImmediatePropagation તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
290	PreventDefault	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર PreventDefault તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
291	Event Delegation	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Event Delegation તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
292	Custom Events	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Custom Events તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
293	Event Loop Integration	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Event Loop Integration તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
294	Web Workers	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Web Workers તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

		છે.
295	Dedicated Workers	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Dedicated Workers તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
296	Shared Workers	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Shared Workers તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
297	Service Workers	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Service Workers તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
298	Offline Caching	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Offline Caching તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
299	Push Notifications	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Push Notifications તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
300	Background Sync	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર

		એન્જિન લેવલ પર Background Sync તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
301	Intersection Observer API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Intersection Observer API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
302	Mutation Observer API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Mutation Observer API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
303	Resize Observer API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Resize Observer API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
304	Fetch API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Fetch API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
305	XMLHttpRequest	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર XMLHttpRequest તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

		મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Cache Storage તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
319	Web Storage Security	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Web Storage Security તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
320	Same-Origin Policy	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Same-Origin Policy તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
321	CORS	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર CORS તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
322	Simple Requests	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Simple Requests તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
323	Preflight Requests	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Preflight Requests તરીકે ઓળખાય છે અને તેનો ઉપયોગ

		ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
324	Access-Control-Allow-Origin	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Access-Control-Allow-Origin તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
325	Access-Control-Allow-Methods	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Access-Control-Allow-Methods તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
326	Access-Control-Allow-Headers	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Access-Control-Allow-Headers તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
327	Cross-Origin Resource Sharing	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Cross-Origin Resource Sharing તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
328	WebSocket API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર WebSocket API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
329	Server-Sent Events	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર

		છે.
341	Performance API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Performance API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
342	Performance.now	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Performance.now તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
343	PerformanceTimeline	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર PerformanceTimeline તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
344	Navigation Timing API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Navigation Timing API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
345	User Timing API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર User Timing API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
346	DeviceOrientation Event	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર

		એન્જિન લેવલ પર DeviceOrientation Event તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
347	Touch Events	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Touch Events તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
348	Pointer Events	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Pointer Events તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
349	Drag and Drop API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Drag and Drop API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
350	FileSystem API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર FileSystem API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
351	Notification API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Notification API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
352	Permissions API	JavaScript નો આ એક

		મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Permissions API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
353	Credential Management API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Credential Management API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
354	WebRTC	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર WebRTC તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
355	MediaDevices	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર MediaDevices તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
356	getUserMedia	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર getUserMedia તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
357	RTCPeerConnection	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર RTCPeerConnection તરીકે ઓળખાય છે અને તેનો ઉપયોગ

		ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
358	RTCDATAChannel	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર RTCDATAChannel તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
359	Payment Request API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Payment Request API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
360	Bluetooth API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Bluetooth API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
361	USB API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર USB API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
362	Gamepad API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Gamepad API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
363	Vibration API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Vibration API તરીકે ઓળખાય છે અને

		તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
364	Fullscreen API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Fullscreen API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
365	Screen Orientation API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Screen Orientation API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
366	Battery Status API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Battery Status API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
367	Network Information API	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Network Information API તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
368	PostMessage	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર PostMessage તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
369	MessageChannel	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર

		એન્જિન લેવલ પર MessageChannel તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
370	MessagePort	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર MessagePort તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
371	Custom Elements	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Custom Elements તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
372	Shadow DOM	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Shadow DOM તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
373	HTML Templates	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર HTML Templates તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
374	Shadow Root	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Shadow Root તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
375	Encapsulated Styles	JavaScript નો આ એક

		મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Encapsulated Styles તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
376	Slot Element	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Slot Element તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
377	Slotted Pseudo-element	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Slotted Pseudo-element તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
378	Attached Callback	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Attached Callback તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
379	Detached Callback	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Detached Callback તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
380	AttributeChangedCallback	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર AttributeChangedCallback

		તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
381	AdoptedCallback	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર AdoptedCallback તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
382	LitElement	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર LitElement તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
383	Stencil	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Stencil તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
384	Web Components Lifecycle	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Web Components Lifecycle તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
385	Declarative Shadow DOM	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Declarative Shadow DOM તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
386	CSS Parts	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર

		એનિમિશન લેવલ પર CSS Parts તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
387	CSS States	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમિશન લેવલ પર CSS States તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
388	Hydration	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમિશન લેવલ પર Hydration તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
389	Progressive Enhancement	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમિશન લેવલ પર Progressive Enhancement તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
390	Server-Side Rendering (SSR)	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમિશન લેવલ પર Server-Side Rendering (SSR) તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
391	Static Site Generation (SSG)	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમિશન લેવલ પર Static Site Generation (SSG) તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
392	Client-Side Rendering (CSR)	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ

		એપ્લિકેશન અને બ્રાઉઝર એનિમિશન લેવલ પર Client-Side Rendering (CSR) તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
393	Isomorphic JavaScript	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમિશન લેવલ પર Isomorphic JavaScript તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
394	Universal JavaScript	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમિશન લેવલ પર Universal JavaScript તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
395	Hot Module Replacement (HMR)	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમિશન લેવલ પર Hot Module Replacement (HMR) તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
396	Source Maps	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમિશન લેવલ પર Source Maps તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
397	Tree Shaking	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એનિમિશન લેવલ પર Tree Shaking તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

		એવિકેશન અને બ્રાઉઝર એનિમલ લેવલ પર Performance Profiling તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
423	Memory Profiling	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એવિકેશન અને બ્રાઉઝર એનિમલ લેવલ પર Memory Profiling તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
424	CPU Throttling	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એવિકેશન અને બ્રાઉઝર એનિમલ લેવલ પર CPU Throttling તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
425	Network Emulation	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એવિકેશન અને બ્રાઉઝર એનિમલ લેવલ પર Network Emulation તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
426	Breakpoints	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એવિકેશન અને બ્રાઉઝર એનિમલ લેવલ પર Breakpoints તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
427	Conditional Breakpoints	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એવિકેશન અને બ્રાઉઝર એનિમલ લેવલ પર Conditional Breakpoints તરીકે ઓળખાય છે અને તેનો ઉપયોગ

		ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
428	Logpoints	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એવિકેશન અને બ્રાઉઝર એનિમલ લેવલ પર Logpoints તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
429	XHR Breakpoints	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એવિકેશન અને બ્રાઉઝર એનિમલ લેવલ પર XHR Breakpoints તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
430	DOM Breakpoints	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એવિકેશન અને બ્રાઉઝર એનિમલ લેવલ પર DOM Breakpoints તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
431	Event Listener Breakpoints	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એવિકેશન અને બ્રાઉઝર એનિમલ લેવલ પર Event Listener Breakpoints તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
432	Scope View	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એવિકેશન અને બ્રાઉઝર એનિમલ લેવલ પર Scope View તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
433	Watch Expressions	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એવિકેશન અને બ્રાઉઝર એનિમલ લેવલ પર Watch

		ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
457	Profiler	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Profiler તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
458	Memory Inspector.	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Memory Inspector તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

Advanced Concepts & Quirks

ID	Concept / Term	Explanation (ગૂંજરતીમાં)
459	Implicit Type Coercion	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Implicit Type Coercion તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
460	Double Dot Notation	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Double Dot Notation તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
461	Unary Plus Tricking	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Unary Plus Tricking તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
462	Abstract Equality (==)	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Abstract Equality (==) તરીકે ઓળખાય છે.

		અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
463	Strict Equality (===)	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Strict Equality (===) તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
464	Same-Value Equality	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Same-Value Equality તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
465	Same-Value-Zero Equality	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Same-Value-Zero Equality તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
466	Structural Cloning Algorithm	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Structural Cloning Algorithm તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
467	ToPrimitive Internal Method	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર ToPrimitive Internal Method તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
468	ToString Internal Method	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર ToString Internal Method તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
469	ToNumber Internal Method	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર ToNumber Internal Method તરીકે ઓળખાય છે અને તેનો ઉપયોગ

		Mechanics તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
514	Scope Chain Lookup	JavaScript નો આ એક મહત્વપૂર્ણ ફંક્શન છે જે વૈભવ એલિફેન્શન અને બ્રાઉઝર એન્જિન લેવલ પર Scope Chain Lookup તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
515	Lexical Scoping Rules	JavaScript નો આ એક મહત્વપૂર્ણ ફંક્શન છે જે વૈભવ એલિફેન્શન અને બ્રાઉઝર એન્જિન લેવલ પર Lexical Scoping Rules તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
516	Variable Shadowing Effects	JavaScript નો આ એક મહત્વપૂર્ણ ફંક્શન છે જે વૈભવ એલિફેન્શન અને બ્રાઉઝર એન્જિન લેવલ પર Variable Shadowing Effects તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
517	Strict Mode Violations	JavaScript નો આ એક મહત્વપૂર્ણ ફંક્શન છે જે વૈભવ એલિફેન્શન અને બ્રાઉઝર એન્જિન લેવલ પર Strict Mode Violations તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
518	Octal Literals Restriction	JavaScript નો આ એક મહત્વપૂર્ણ ફંક્શન છે જે વૈભવ એલિફેન્શન અને બ્રાઉઝર એન્જિન લેવલ પર Octal Literals Restriction તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
519	Delete Variable Restriction	JavaScript નો આ એક મહત્વપૂર્ણ ફંક્શન છે જે વૈભવ એલિફેન્શન અને બ્રાઉઝર એન્જિન લેવલ પર Delete Variable Restriction તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
520	Duplicate Parameter Restriction	JavaScript નો આ એક મહત્વપૂર્ણ ફંક્શન છે જે વૈભવ એલિફેન્શન અને બ્રાઉઝર એન્જિન લેવલ પર Duplicate Parameter Restriction તરીકે

		ઓળખાય છે અને તેનો ઉપયોગ ઉત્તરવપ્રમેન્ડમાં વ્યાપકપણે થાય છે.
521	With Statement Restriction	JavaScript ની એક મહત્વપૂર્ણ ક્ષમોત્તર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેબ પર With Statement Restriction તરીકે ઓળખાય છે અને તેનો ઉપયોગ ઉત્તરવપ્રમેન્ડમાં વ્યાપકપણે થાય છે.
522	Arguments Callee Restriction	JavaScript ની એક મહત્વપૂર્ણ ક્ષમોત્તર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેબ પર Arguments Callee Restriction તરીકે ઓળખાય છે અને તેનો ઉપયોગ ઉત્તરવપ્રમેન્ડમાં વ્યાપકપણે થાય છે.
523	This Binding in Strict Mode	JavaScript ની એક મહત્વપૂર્ણ ક્ષમોત્તર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેબ પર This Binding in Strict Mode તરીકે ઓળખાય છે અને તેનો ઉપયોગ ઉત્તરવપ્રમેન્ડમાં વ્યાપકપણે થાય છે.
524	Silent Errors Made Loud	JavaScript ની એક મહત્વપૂર્ણ ક્ષમોત્તર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેબ પર Silent Errors Made Loud તરીકે ઓળખાય છે અને તેનો ઉપયોગ ઉત્તરવપ્રમેન્ડમાં વ્યાપકપણે થાય છે.
525	Security Benefits of Strict Mode	JavaScript ની એક મહત્વપૂર્ણ ક્ષમોત્તર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેબ પર Security Benefits of Strict Mode તરીકે ઓળખાય છે અને તેનો ઉપયોગ ઉત્તરવપ્રમેન્ડમાં વ્યાપકપણે થાય છે.
526	V8 Engine Architecture	JavaScript ની એક મહત્વપૂર્ણ ક્ષમોત્તર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેબ પર V8 Engine Architecture તરીકે ઓળખાય છે અને તેનો ઉપયોગ ઉત્તરવપ્રમેન્ડમાં વ્યાપકપણે થાય છે.
527	Ignition Interpreter	JavaScript ની એક મહત્વપૂર્ણ ક્ષમોત્તર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેબ પર Ignition Interpreter તરીકે ઓળખાય છે અને તેનો ઉપયોગ ઉત્તરવપ્રમેન્ડમાં વ્યાપકપણે થાય છે.

		છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
557	Memory Leak Detection	JavaScript નો આ એક મહત્વપૂર્ણ ફેચર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેલવ પર Memory Leak Detection તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
558	Detached DOM Nodes	JavaScript નો આ એક મહત્વપૂર્ણ ફેચર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેલવ પર Detached DOM Nodes તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
559	Forgotten Timers	JavaScript નો આ એક મહત્વપૂર્ણ ફેચર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેલવ પર Forgotten Timers તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
560	Global Variables Accumulation	JavaScript નો આ એક મહત્વપૂર્ણ ફેચર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેલવ પર Global Variables Accumulation તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
561	Closures Retaining Memory	JavaScript નો આ એક મહત્વપૂર્ણ ફેચર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેલવ પર Closures Retaining Memory તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
562	Event Listeners Leak	JavaScript નો આ એક મહત્વપૂર્ણ ફેચર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેલવ પર Event Listeners Leak તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
563	Web Workers Termination	JavaScript નો આ એક મહત્વપૂર્ણ ફેચર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેલવ પર Web Workers Termination તરીકે ઓળખાય છે અને તેનો ઉપયોગ

		ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
564	Cross-Site Scripting (XSS)	JavaScript નો આ એક મહત્વપૂર્ણ ફ્લેવર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વચ્ચે પર Cross-Site Scripting (XSS) તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
565	Stored XSS	JavaScript નો આ એક મહત્વપૂર્ણ ફ્લેવર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વચ્ચે પર Stored XSS તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
566	Reflected XSS	JavaScript નો આ એક મહત્વપૂર્ણ ફ્લેવર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વચ્ચે પર Reflected XSS તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
567	DOM-based XSS	JavaScript નો આ એક મહત્વપૂર્ણ ફ્લેવર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વચ્ચે પર DOM-based XSS તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
568	Content Security Policy (CSP)	JavaScript નો આ એક મહત્વપૂર્ણ ફ્લેવર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વચ્ચે પર Content Security Policy (CSP) તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
569	XSS Mitigation	JavaScript નો આ એક મહત્વપૂર્ણ ફ્લેવર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વચ્ચે પર XSS Mitigation તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
570	Sanitization	JavaScript નો આ એક મહત્વપૂર્ણ ફ્લેવર છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વચ્ચે પર Sanitization તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

		the-Middle Protection તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
586	HTTPS	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર HTTPS તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
587	Mixed Content	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Mixed Content તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
588	Strict-Transport-Security (HSTS)	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Strict-Transport-Security (HSTS) તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
589	Content Negotiation	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Content Negotiation તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
590	CORS Security Boundaries	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર CORS Security Boundaries તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
591	Preflight Caching	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Preflight Caching તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
592	Origin Header	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Origin Header તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

		છે.
593	Credentials with CORS	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Credentials with CORS તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
594	Regular Expression Vulnerabilities	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Regular Expression Vulnerabilities તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
595	ReDoS	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર ReDoS તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
596	Catastrophic Backtracking	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Catastrophic Backtracking તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
597	Safe Regex Generation	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Safe Regex Generation તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
598	Strict Content Parsing	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર Strict Content Parsing તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
599	JSON.parse Security	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન વેવલ પર JSON.parse Security તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
600	Prototype Pollution	JavaScript નો આ એક મહત્વપૂર્ણ

		બાઉઝર એન્જિન લેવલ પર Symbol.asyncDispose તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
630	DisposableStack	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બાઉઝર એન્જિન લેવલ પર DisposableStack તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
631	AsyncDisposableStack	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બાઉઝર એન્જિન લેવલ પર AsyncDisposableStack તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
632	Automatic Resource Cleanup	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બાઉઝર એન્જિન લેવલ પર Automatic Resource Cleanup તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
633	Memory Leak Prevention via Scope	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બાઉઝર એન્જિન લેવલ પર Memory Leak Prevention via Scope તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
634	Resource Lifetime Management	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બાઉઝર એન્જિન લેવલ પર Resource Lifetime Management તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
635	Math.sumPrecise	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બાઉઝર એન્જિન લેવલ પર Math.sumPrecise તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
636	Precision Floating Point Summation	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બાઉઝર એન્જિન લેવલ પર Precision

		Floating Point Summation તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
637	Array Accumulation Precision	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બાઉઝર એન્જિન લેવલ પર Array Accumulation Precision તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
638	Map.prototype.getOrInsert	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બાઉઝર એન્જિન લેવલ પર Map.prototype.getOrInsert તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
639	Atomic Map Updates	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બાઉઝર એન્જિન લેવલ પર Atomic Map Updates તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
640	Default Map Values	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બાઉઝર એન્જિન લેવલ પર Default Map Values તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
641	Cache Implementations	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બાઉઝર એન્જિન લેવલ પર Cache Implementations તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
642	Collection Enhancements	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બાઉઝર એન્જિન લેવલ પર Collection Enhancements તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
643	Set.prototype.intersection	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બાઉઝર એન્જિન લેવલ પર Set.prototype.intersection તરીકે

		છે.
673	Static Imports	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Static Imports તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
674	Module Record	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Module Record તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
675	Module Namespace Object	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Module Namespace Object તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
676	Live Bindings in Modules	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Live Bindings in Modules તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
677	Circular Dependencies Resolution	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Circular Dependencies Resolution તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
678	Node.js CommonJS vs ES Modules	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Node.js CommonJS vs ES Modules તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
679	MJS Extension	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર MJS Extension તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

680	CJS Extension	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર CJS Extension તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
681	Require vs Import	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Require vs Import તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.
682	Interoperability Rules.	JavaScript નો આ એક મહત્વપૂર્ણ કન્સેપ્ટ છે જે વેબ એપ્લિકેશન અને બ્રાઉઝર એન્જિન લેવલ પર Interoperability Rules. તરીકે ઓળખાય છે અને તેનો ઉપયોગ ડેવલપમેન્ટમાં વ્યાપકપણે થાય છે.

JavaScript 1000+ Ultimate Mega Master List (ગુજરાતી)

આ ફાઇલમાં JavaScript ના ૧૦૦૦ થી વધુ યુનિક કન્સેપ્ટ્સ, કીવર્ડ્સ, વેબ APIs, એન્જિન મિકેનિઝમ્સ અને એડવાન્સ ટર્સને એકદમ સરળ અને વાંચી શકાય તેવી ગુજરાતી ભાષામાં વ્યવસ્થિત રીતે વર્ગીકૃત (Categorized) કરીને સમજાવવામાં આવ્યા છે.

૧. Core JavaScript Language & Syntax Basics (મૂળભૂત સિન્ટેક્સ અને ફાઉન્ડેશન)

Concept / Term (ટર્મ)	Simple & Readable Explanation (સરળ ગુજરાતી સમજૂતી)
Variables (વેરીએબલ્સ)	ડેટા સ્ટોર કરવા માટેનું કન્ટેનર.
var keyword	જૂનો ફંક્શન-સ્કોપ વેરીએબલ ડિક્લેરેશન કીવર્ડ.
let keyword	આધુનિક બ્લોક-સ્કોપ વેરીએબલ કીવર્ડ જે હોઇસ્ટિંગ દરમિયાન એક્સેસ નથી થતો.
const keyword	બ્લોક-સ્કોપ કીવર્ડ જેના રેફરન્સ કે વેલ્યુને ફરીથી અસાઇન નથી કરી શકાતી.
Data Types (ડેટા ટાઇપ્સ)	વેરીએબલમાં કેવા પ્રકારનો ડેટા છે તે નક્કી કરતો સિસ્ટમ.
Primitive Types	ઇમ્યુટેબલ અને સીધી વેલ્યુ સ્ટોર કરતા બેઝિક ડેટા ટાઇપ્સ.
Reference Types	મેમરી એડ્રેસ અથવા રેફરન્સ સ્ટોર કરતા ઓબ્જેક્ટ ટાઇપ્સ.
String	ટેક્સ્ટ ડેટા સ્ટોર કરવા માટેનો પ્રિમિટિવ ટાઇપ.
Number	પૂર્ણાંક અને અપૂર્ણાંક બંને નંબરો માટેનો સિંગલ ડેટા ટાઇપ.
Boolean	માત્ર true અથવા false વેલ્યુ સ્ટોર કરતો ટાઇપ.
Null	ઇરાદાપૂર્વક ખાલી રાખેલી વેલ્યુ (Intentional Absence).
Undefined	વેરીએબલ ડિક્લેર કર્યો હોય પણ વેલ્યુ ન આપી હોય તેવી સ્થિતિ.
Symbol	Unique અને પ્રિમિટિવ આઇડેન્ટિફાયર જે ઓબ્જેક્ટ કી તરીકે વપરાય છે.
BigInt	ખૂબ જ મોટા પૂર્ણાંક નંબરો (નર્મલ બિટની બહાર) હેન્ડલ કરવા માટેનો ટાઇપ.
Object	કી-વેલ્યુ પેર ડેટા સ્ટોર કરતું અન-ઓર્ડર્ડ કલેક્શન.
Array	એક જ વેરીએબલમાં ક્રમબદ્ધ વિસ્ટ સ્ટોર કરવાનું મિકેનિઝમ.
Function	ફરીથી વાપરી શકાય તેવો કોડ બ્લોક જે પોતે પણ એક ઓબ્જેક્ટ છે.
Typeof Operator	કોઇ પણ વેરીએબલનો ડેટા ટાઇપ ચેક કરવાનું ટૂંક.
Dynamic Typing	રન-ટાઇમ પર વેરીએબલનો ટાઇપ આપમેળે બદલાવાની સુવિધા.

Weak Typing	ઓપરેશન વખતે આપમેળે ટાઇપ ફર્વર્સન (Coercion) કરવાની ક્ષમતા.
૨. Operators & Advanced Control Flow (ઓપરેટર્સ અને કન્ટ્રોલ ફોલો)	
Concept / Term (ટર્મ)	Simple & Readable Explanation (સરળ ગુજરાતી સમજૂતી)
Arithmetic Operators	ગણિત કરવા માટેના ઓપરેટર્સ જેમ કે +, -, *, /.
Assignment Operators	વેલ્યુ અસાઇન કરવા માટે વપરાતા ચિહ્નો જેમ કે =, +=, -=.
Comparison Operators	બે વેલ્યુ સરખાવવા માટે વપરાતા ઓપરેટર્સ (==, ===, >, <).
Logical Operators	લોજિકલ કન્ટ્રોલ નેસ્ટ કરવા માટેના ઓપરેટર્સ (&&, , !).
Bitwise Operators	બાઇનરી વિટલ (Bits) પર ગણતરી કરતા ઓપરેટર્સ.
Ternary Operator (?:)	ઇફ-એલ્સ (if-else) નું શોર્ટહેન્ડ વર્ઝન જે સિંગલ લાઇનમાં શરત ચેક કરે છે.
Nullish Coalescing (??)	ડાબી બાજુની વેલ્યુ null કે undefined હોય ત્યારે જ ડિફોલ્ટ વેલ્યુ આપે છે.
Optional Chaining (?.)	ઓબ્જેક્ટની પ્રોપર્ટી ચેક કરતી વખતે જો પ્રોપર્ટી ન હોય તો એરર આપવાને બદલે undefined રિટર્ન કરે છે.
Short-Circuit Evaluation	&& કે માં પહેલી કન્ટ્રોલ નેસ્ટ પરથી જ આખો જવાબ નક્કી થઇ જવાની પ્રક્રિયા.
Strict Equality (===)	વેલ્યુ અને ડેટા ટાઇપ બંને સરખાવતું કડક ચેકિંગ.
Loose Equality (==)	ટાઇપ ફર્વર્સ કરીને માત્ર વેલ્યુ સરખાવતું નરમ ચેકિંગ.
Increment / Decrement (++, --)	વેલ્યુમાં ૧ ઉમેરવા કે ઘટાડવાના ઓપરેટર્સ.
Exponentiation Operator (**)	પાવર (ઘાત) ગણવા માટે વપરાતો ઓપરેટર.
Logical Assignment Operators	&&=, =, અને ??= જેવા શોર્ટહેન્ડ કમ્બાઇન્ડ ઓપરેટર્સ.
Comma Operator	એક જ સ્ટેટમેન્ટમાં મલ્ટિપલ એક્સપ્રેશન્સ રન કરવા માટે.
Switch Case	મલ્ટિપલ કન્ટ્રોલ નેસ્ટમાંથી ચોક્કસ બ્લોક રન કરવાનું સ્ટ્રક્ચર.
For Loop	ચોક્કસ લિમિટ સુધી કોડ વારંવાર ચલાવવાનું લૂપ.
While Loop	શરત સાચી હોય ત્યાં સુધી ચાલતું લૂપ.
Do-While Loop	ઓછામાં ઓછું એકવાર તો રન થાય જ તેવું શરતી લૂપ.
Break Statement	લૂપને અધવચ્ચેથી તોડીને બહાર નીકળવાનો કીવર્ડ.

૩. Scope, Hoisting & Execution Context (એન્જિન ઇન્ટરનલ્સ અને સ્કોપિંગ)

Concept / Term (ટર્મ)	Simple & Readable Explanation (સરળ ગુજરાતી સમજૂતી)
Global Scope	આખા પ્રોગ્રામમાં ગમે ત્યાંથી એક્સેસ થઈ શકતો સ્કોપ.
Function Scope	માત્ર ફંક્શનની અંદર જ એક્સેસ થઈ શકે તેવા વેરીએબલ્સનો સ્કોપ.
Block Scope	ક્લી બ્રેકેટ્સ {} ની અંદર સીમિત સ્કોપ (let અને const).
Lexical Scoping	કોડ બ્લોક્સ વખાણો છે તેના આધારે તેનો સ્કોપ નક્કી થવાની પ્રક્રિયા.
Scope Chain	વેરીએબલ ન મળે ત્યારે અંદરના સ્કોપથી બહારના સ્કોપ સુધી સર્ચ કરવાની લાઇન.
Hoisting	ડિફિનિશન કોડ રન થાય તે પહેલા સ્કોપમાં સૌથી ઉપર જતા રહે તેવો ભાસ થવો.
Temporal Dead Zone (TDZ)	let/const ડિફિનિશન થાય તે પહેલાં તેને એક્સેસ ન કરી શકાય તે સમયગાળો.
Execution Context	જાવાસ્ક્રિપ્ટ કોડ જે વાતાવરણમાં રન થાય છે તેનું મેનેજમેન્ટ.
Global Execution Context	પ્રોગ્રામ શરૂ થતા જ બનતું મેઈન એક્ઝિક્યુશન એન્વાયરમેન્ટ.
Function Execution Context	દરેક ફંક્શન કોલ થતી વખતે બનતું નવું પ્રાઇવેટ એન્વાયરમેન્ટ.
Creation Phase	એક્ઝિક્યુશન કોન્ટેક્સ્ટનો પહેલો તબક્કો જેમાં મેમરી એલોકેશન થાય છે.
Execution Phase	બીજો તબક્કો જેમાં લાઇન-બાય-લાઇન કોડ રન થાય છે.
Call Stack	એક્ઝિક્યુશન કોન્ટેક્સ્ટ કયા ક્રમમાં રન થશે તેનો ટ્રેક રાખતો સ્ટેક (LIFO).
Stack Overflow	જ્યારે કોલ સ્ટેકની મેમરી લિમિટ પૂરી થઈ જાય (અનંત રિકર્ઝન).
Memory Heap	અન-સ્ટ્રક્ચર્ડ મોડો મેમરી એરિયા જ્યાં ઓબ્જેક્ટ્સ સ્ટોર થાય છે.
Variable Shadowing	અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના વેરીએબલની વેલ્યુ છુપાવી દે તે સ્થિતિ.
Illegal Shadowing	let વેરીએબલને var દ્વારા શેડો કરવાની મનાઈ હોવાની સ્થિતિ.
Closure	ફંક્શન તેના બહારના સ્કોપના વેરીએબલ્સને યાદ રાખે તે પાવરફુલ ટૂંકચર.
IIIFE	બનતાની સાથે જ તરત જ ઓટોમેટિક રન થઈ જતું ફંક્શન.
Lexical Environment	કરન્ટ સ્કોપ અને તેના આઉટર સ્કોપનો રેફરન્સ ધરાવતું ઇન્ટરનલ સ્ટ્રક્ચર.

૪. Object-Oriented JS, Prototypes & Context (OOPS અને પ્રોટોટાઇપ્સ)

Concept / Term (ટર્મ)	Simple & Readable Explanation (સરળ ગુજરાતી સમજૂતી)
Prototypal Inheritance	એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટી વારસામાં મેળવે તે પદ્ધતિ.
Prototype Chain	પ્રોપર્ટી શોધવા માટે પ્રોટોટાઇપ ઓબ્જેક્ટ્સની આખી લિંક ચેઇન.
__proto__	દરેક ઓબ્જેક્ટની અંદર રહેલો તેનો વાસ્તવિક પ્રોટોટાઇપ લિંક રેફરન્સ.
prototype property	માત્ર ફંક્શન્સ પાસે રહેલો ઓબ્જેક્ટ જે વારસો આપવા માટે વપરાય છે.
this Keyword	કરન્ટ એક્ઝિક્યુશન કોન્ટેક્સ્ટ અથવા ઓબ્જેક્ટનો લાઇવ રેફરન્સ.
call() Method	this ની વેલ્યુ કરન્ટ સેટ કરીને ફંક્શનને તરત રન કરવાની મેથડ.
apply() Method	call જેવી જ પણ આર્ગ્યુમેન્ટ્સ એરે (Array) સ્વરૂપે લેતી મેથડ.
bind() Method	this ની વેલ્યુ ફિક્સ કરીને નવું ફંક્શન રિટર્ન કરતી મેથડ.
Constructor Function	નવો ઓબ્જેક્ટ બનાવવા માટે વપરાતું સ્પેશિયલ ફંક્શન.
new Keyword	નવો ખાલી ઓબ્જેક્ટ બનાવીને કન્સ્ટ્રક્ટર કોલ કરતો કીવર્ડ.
ES6 Classes	પ્રોટોટાઇપલ ઇનહેરિટેન્સ પર લખાયેલું આધુનિક સિન્ટેક્સિકલ શુગર કવર.
Super Keyword	પેરેન્ટ ક્લાસના કન્સ્ટ્રક્ટર કે મેથડને કોલ કરવાનો કીવર્ડ.
Getter Methods	ઓબ્જેક્ટની પ્રોપર્ટી રીડ કરવા માટેની ખાસ મેથડ.
Setter Methods	ઓબ્જેક્ટની પ્રોપર્ટી સેટ કે મોડિફાઇ કરવા માટેની વેલિડેશન મેથડ.
Static Methods	ક્લાસના ઇન્સ્ટેન્સ વગર સૌંધા ક્લાસ નેમથી કોલ થતી મેથડ્સ.
Encapsulation	ડેટા અને મેથડ્સને સિધ્ધોર કરીને એક ક્લાસમાં પેક કરવાની પદ્ધતિ.
Private Fields (#)	ક્લાસની બહાર કોઈ એક્સેસ ન કરી શકે તેવા પ્રાઇવેટ વેરીએબલ્સ.
Polymorphism	એક જ નામની મેથડ અલગ અલગ ક્લાસમાં અલગ કામ કરે તે લક્ષણ.
Abstraction	અંદરનું જટિલ લૉજિક છુપાવીને માત્ર મહત્વના ફીચર્સ બહાર બતાવવા.
Method Chaining	એક જ લાઇનમાં એક પછી એક મેથડ્સ લિંક કરીને કોલ કરવી.

૫. Asynchronous JS, Event Loop & Queues (અસિંક્રોનસ અને ઇવેન્ટ લૂપ્સ)

Concept / Term (ટર્મ)	Simple & Readable Explanation (સરળ ગુજરાતી સમજૂતી)
Callback Function	બીજા ફંક્શનની અંદર આન્યુમેન્ટ તરીકે અપાતું ફંક્શન.
Callback Hell	એકની અંદર એક નેસ્ટેડ કોલબેકસ લખવાથી બનતો ગૂંથવણભર્યો પિરામિડ.
Inversion of Control	કોલબેક આપતી વખતે આપણો કંટ્રોલ થર્ડ-પાર્ટી કોડ પર જતો રહેવાની ખામી.
Promise	અસિંક્રોનસ કામ ભવિષ્યમાં પૂરું થશે તેની ગેરંટી આપતું ઓબ્જેક્ટ.
Pending State	પ્રોમિસ હજી પ્રોસેસમાં છે, સફળ કે નિષ્ફળ નથી થયું તે સ્ટેટ.
Fulfilled State	અસિંક્રોનસ ઓપરેશન સફળતાપૂર્વક પૂરું થવાની સ્થિતિ.
Rejected State	ઓપરેશન નિષ્ફળ જવાની અથવા એરર આવવાની સ્થિતિ.
Promise Chaining	એક પછી એક અનેક .then() મેથડ્સ લિંક કરવાની ટેકનિક.
Promise.all()	તમામ પ્રોમિસિસ એકસાથે સફળ થાય તો જ ઓકે કરતું કમ્પાઈન્ટ ટૂલ.
Promise.allSettled()	સફળ કે નિષ્ફળ, બધા પ્રોમિસ પૂરા થાય ત્યાં સુધી રાહ જોતી મેથડ.
Promise.race()	જે પ્રોમિસ સૌથી પહેલા પૂરું થાય (સફળ કે નિષ્ફળ) તેનો જવાબ આપતી રેસ.
Promise.any()	સૌથી પહેલું 'સફળ' પ્રોમિસ શોધતી મેથડ.
Async Keyword	ફંક્શનને અસિંક્રોનસ બનાવીને પ્રોમિસ રિટર્ન કરાવતો કીવર્ડ.
Await Keyword	પ્રોમિસ રીઝોલ્વ ન થાય ત્યાં સુધી વાઈનને અટકાવી રાખતો કીવર્ડ.
Event Loop	કોલ સ્ટેક અને ક્યુ (Queue) વચ્ચે બેથેન્સ રાખતું મુખ્ય એન્જિન મિકેનિઝમ.
Macrotask Queue	setTimeout, setInterval જેવા મોટા કામોની લાઇન.
Microtask Queue	Promises અને MutationObserver જેવા હાઈ-પ્રાયોરિટી કામોની લાઇન.
Starvation	માઇક્રોટાસ્ક ક્યુ સતત ભરાતી રહેવાથી મેક્રોટાસ્ક અટકી જવાની સ્થિતિ.
Generator Functions	function* સિન્ટેક્સ વાળા ફંક્શન્સ જેને yield દ્વારા અથવા યેલ્ડ રોકી શકાય છે.
Asynchronous Iterators	for await...of લૂપ દ્વારા અસિંક્રોનસ ડેટા સ્ટ્રીમ હેન્ડલ કરવાની રીત.

૬. Advanced Engine Internals & Security Quirks (સુપર-એડવાન્સ ટપ્સ અને છુપી ટ્રીક્સ)

Concept / Term (ટર્મ)	Simple & Readable Explanation (સરળ
-----------------------	------------------------------------

	ગુજરાતી સમજૂતી
JIT Compilation	જાવાસ્ક્રિપ્ટ કોડ રન થતી વખતે જ ફાસ્ટ મશીન કોડમાં કમ્પાઇલ થવાની પ્રક્રિયા.
Implicit Type Coercion	બેકગ્રાઉન્ડમાં આપમેળે ડેટા ટાઇપ ફર્વર્ટ થવું (Ex.t. 5 + '5' = '55').
Object.is()	=== કરતાં પણ સયોટ ચેકિંગ જે NaN સરખામણી પણ સાચી કરી શકે છે.
Unary Plus Tricking	++ કરવાથી ખાલી એરે ઓટોમેટિક નંબર 0 બની જવાની એન્જિન ટ્રીક.
Double Dot Notation	35.toString() દ્વારા હોલ નંબર પર સીધી ઓબ્જેક્ટ મેથડ વાપરવી.
Temporal API	જૂનું Date() ઓબ્જેક્ટને બદલતું નવું ટાઈમ-ઝોન ફ્રેન્ડલી મોડર્ન API (ESNext).
Explicit Resource Management	using કીવર્ડ દ્વારા સ્કોપ બહાર જતા જ મેમરી ઓટોમેટિક ખાલી થવી.
Math.sumPrecise()	0.1 + 0.2 ની પોઇન્ટ વાળી ભૂલ સોલ્વ કરતી નવી નેટિવ મેથડ.
Inline Caching	વારંવાર વપરાતા ઓબ્જેક્ટ પ્રોપર્ટી લોકેશનને એન્જિન દ્વારા યાદ રાખવાની ટ્રીક.
Hidden Classes (Shapes)	V8 એન્જિન બેકગ્રાઉન્ડમાં ઓબ્જેક્ટ્સનું સ્ટ્રક્ચર ચેક કરવા માટે બનાવે છે તે ક્લાસ.
Tree Shaking	બિલ્ડ પ્રોસેસ વખતે અન-યુઝ્ડ કે નકામો કોડ આપમેળે હટાવી દેવો.
CORS (Cross-Origin Resource Sharing)	બીજી વેબસાઇટ પરથી ડેટા લાવવા સુરક્ષા પૂરી પાડતા બ્રાઉઝર નિયમો.
XSS (Cross-Site Scripting)	વેબસાઇટમાં હેકર્સ દ્વારા નકામો સ્ક્રિપ્ટ કોડ ઇન્જેક્ટ કરવાનો એટેક.
CSRF	યુઝરના લોગિન સેશનનો ખોટો ઉપયોગ કરીને થતા સિક્કોરિટી હોડ.
Web Workers	ભારે કેલ્ક્યુલેશન બેકગ્રાઉન્ડ થ્રેડમાં ચલાવીને મેઈન થ્રેડ ફ્રી રાખવાની સિસ્ટમ.
Decorators	@ સિન્ટેક્સ દ્વારા ક્લાસ કે મેથડનું લોજિક મોડિફાય કરવાની એડવાન્સ પેટર્ન.
Proxy Objects	કોઈ ઓબ્જેક્ટના ઓપરેશન્સ (get/set) વચ્ચે આવીને કસ્ટમ લોજિક ચલાવવું.
Reflect API	Proxy સાથે ઓબ્જેક્ટ ઓપરેશન્સને સરળતાથી હોલવર્ડ કરવાનું નેટિવ API.
Falsy Values	જાવાસ્ક્રિપ્ટની ૬ મુખ્ય ફોલ્સી વેલ્યુઝ (0, "", null, undefined, false, NaN).
Garbage Collection	મેમરી ચેક કરીને ન વપરાતા ઓબ્જેક્ટ્સ ઓટોમેટિક કીવોટ કરવાની સિસ્ટમ (Mark-and-Sweep).

આ વિસ્તૃત આખા ઇન્ટરનેટ અને ગૂગલ પરના તમામ સત્તાવાર ડોક્યુમેન્ટેશનને કમ્પાઈન કરીને મોસ્ટ-મિસિંગ ટમ્સને પણ આવરી લેવામાં આવી છે. આ તમારી અલ્ટીમેટ રેફરન્સ ગાઇડ છે.

JAVASCRIPT ULTIMATE MEGA MASTER LIST (COMPLETE A-Z EDITION)

દરેક હિડન કન્સેપ્ટ, ઓબ્લિવિયસ આર્કિટેક્ચર અને એડવાન્સ ટમ્સનું સરળ ગુજરાતીમાં સિંગલ માસ્ટર વિસ્ટ

આ ડોક્યુમેન્ટમાં ઓનલાઈન ટ્યુટોરીયલ્સ અને ગૂગલ પરથી સામાન્ય રીતે મિસિંગ (ગાયબ) થતા અથવા અધૂરા મળતા તમામ JavaScript કન્સેપ્ટ્સને એક જ જગ્યાએ ઊંડાણપૂર્વક અને સરળ ગુજરાતીમાં સ્પષ્ટ કરવામાં આવ્યા છે.

૧. એડવાન્સ મેટા-પ્રોગ્રામિંગ અને મિસિંગ સિન્ટેક્સ (Meta-Programming & Hidden Syntax)

- Decorators (@ સિન્ટેક્સ):** ક્લાસ, મેથડ, પ્રોપર્ટી કે પેરામીટરના વોજિકને મોડિફાઇ કે એક્સટેન્ડ કરવા માટે વપરાતું અલ્ટ્રા-મોડર્ન ફીચર. આ એક એવું ફેક્શન છે જે બીજા કોડને વ્રપ (wrap) અથવા ડેકોરેટ કરવા માટે વપરાય છે.
- Generator Functions (function*):** આ એવા ખાસ ફંક્શન્સ છે જેને તમે રન થતી વખતે વચ્ચેથી અટકાવી શકો છો ('yield' કીવર્ડ દ્વારા) અને ભવિષ્યમાં ગમે ત્યારે ત્યાંથી જ ફરીથી શરૂ કરી શકો છો. આ 'Iterator' પ્રોટોકોલ પર કામ કરે છે.
- Iterators & Iterables:** Iterable એટલે એવી કોઈ પણ વસ્તુ (જેમ કે Array, String) જેના પર લૂપ ફેરવી શકાય. Iterator એટલે એક એવો ઓબ્જેક્ટ જેની પાસે 'next()' મેથડ હોય અને તે વિસ્ટની આગામી વેલ્યુ રિટર્ન કરે.
- Asynchronous Iterators (for await...of):** જ્યારે ડેટા સ્ટ્રીમ કે કલેક્શનના દરેક એલિમેન્ટ અસિંક્રોનસ રીતે (પ્રોમિસ ફોર્મેટમાં) મળતા હોય, ત્યારે તેના પર લૂપ ચલાવવા માટે આ સ્પેશિયલ સિન્ટેક્સ વપરાય છે.
- Proxy Objects:** આ એક એવો કસ્ટમ સિક્રેટ એજન્ટ ઓબ્જેક્ટ છે જે તમારા ઓરિજિનલ ઓબ્જેક્ટની આગળ ઊભો રહી જાય છે. તમે ઓરિજિનલ ઓબ્જેક્ટમાં કોઈ પ્રોપર્ટી રીડ, રાઈટ કે ડીલીટ કરો તો પ્રોક્સી તેને અધવચ્ચે પકડીને ચેક કરે છે.
- Reflect API:** જાવાસ્ક્રિપ્ટનું ગ્લોબલ ઓબ્જેક્ટ જે ઓબ્જેક્ટ્સની આંતરિક મેથડ્સ (જેમ કે પ્રોપર્ટી ચેક કરવી, ડીલીટ કરવી) ને સીધી રીતે ઓપરેટ કરવા માટે ક્લીન અને નવો રસ્તો આપે છે.
- Symbols (Unique Tokens):** એક અનોખો અને ક્યારેય પાછા ડુપ્લિકેટ ન થઈ શકે તેવો પ્રિમિટિવ ડેટા ટાઈપ. તેનો ઉપયોગ ઓબ્જેક્ટની અંદર એવી છુપી કી (Hidden Property Keys) બનાવવા થાય છે જેને કોઈ સામાન્ય લૂપ પકડી ન શકે.
- Well-Known Symbols (Symbol.Iterator વગેરે):** જાવાસ્ક્રિપ્ટના ઓબ્લિવિયસ અંદર બિલ્ટ-ઇન રહેલા ગુપ્ત સિમ્બોલ્સ. આનો ઉપયોગ કરીને તમે ઓરિજિનલ ઓબ્જેક્ટની અંદર લૂપ ચલાવવી કે ટાઈપ બદલવો જેવી નેટિવ સિસ્ટમને ઓવરરાઈડ કરી શકો છો.

૨. બ્રાઉઝર એન્જિન, કમ્પાઈલર અને મેમરી આર્કિટેક્ચર (Engine, Compiler & Memory Internals)

- JIT (Just-In-Time) Compilation:** બ્રાઉઝરનું એન્જિન (જેમ કે V8) જાવાસ્ક્રિપ્ટના ઇન્ટરપ્રેટર કોડને રન ટાઈમ પર જ સીધો મશીન કોડમાં કમ્પાઈલ કરે છે જેથી કોડ સુપર-ફાસ્ટ સ્પીડમાં રન થાય.
- Abstract Syntax Tree (AST):** જ્યારે તમે કોડ લખો, ત્યારે બ્રાઉઝરનું કમ્પાઈલર સીધી પહેલાં તે કોડના શબ્દો અને વ્યાકરણને સમજીને મેમરીમાં એક પિરામિડ જેવું ટ્રી-સ્ટ્રક્ચર બનાવે છે, જેને AST કહેવાય છે.
- Inline Caching (IC):** જો કોઈ પ્રોગ્રામમાં કોઈ ફંક્શન વારંવાર એક જ પ્રકારના ઓબ્જેક્ટની પ્રોપર્ટી એક્સેસ કરતું હોય, તો બ્રાઉઝર મેમરી લુકઅપ બાયપાસ કરીને તે પ્રોપર્ટીનું લોકેશન સીધું કેશમાં સેવ કરી લે છે.
- Hidden Classes (Shapes):** જાવાસ્ક્રિપ્ટમાં ઓબ્જેક્ટ્સ ડાયનેમિક હોવા છતાં, બેકટ્રાઉન્ડમાં V8 એન્જિન સમાન પ્રકારની પ્રોપર્ટી ધરાવતા ઓબ્જેક્ટ્સ માટે એક 'હિડન ક્લાસ' બનાવે છે જેથી મેમરીમાં ડેટા ફાસ્ટ એક્સેસ થઈ શકે.
- Memory Heap vs Call Stack:** Call Stack માં લાઈવ કોડ અને પ્રિમિટિવ વેરીએબલ્સ સ્ટોર થાય છે (LIFO ઓર્ડરમાં), જ્યારે મેમરી હીપ (Memory Heap) એ એક મોટો ખુલ્લો એરિયા છે જ્યાં મોટા ઓબ્જેક્ટ્સ અને ફંક્શનલ અન-સ્ટ્રક્ચર્ડ ડેટા સ્ટોર થાય છે.
- Garbage Collection (Mark-and-Sweep):** જ્યારે મેમરી હીપમાં રહેલા કોઈ ઓબ્જેક્ટનો રેફરન્સ આખા પ્રોગ્રામમાંથી ખતમ થઈ જાય, ત્યારે જાવાસ્ક્રિપ્ટનું ગાર્બેજ કલેક્ટર રુટથી ચેક કરીને તે નકામા ઓબ્જેક્ટને મેમરીમાંથી ઓટોમેટિકલી ડીલીટ કરી નાખે છે.
- Memory Leaks:** એવી ભૂલો જેમાં કોડિંગની ખામીના કારણે નકામા ઓબ્જેક્ટ્સ મેમરીમાં જમા રહે છે અને ગાર્બેજ કલેક્ટર તેને ડીલીટ નથી કરી શકતું (દા.ત. અજાણતા બનેલા ગ્લોબલ વેરીએબલ્સ કે અધૂરા મૂકેલા ટાઈમર્સ).

૩. માઈક્રો-ઓપ્ટિમાઈઝેશન અને બંડલર ટેકનોલોજી (Micro-Optimizations & Bundling)

- Tree Shaking:** પ્રોડક્શન બિલ્ડ (ફાઇનલ કોડ) બનતી વખતે, મોડર્ન બંડલર્સ (Webpack, Vite) આખા કોડમાંથી એવા ફંક્શનલ કે મોડ્યુલ્સને આપમેળે શોધીને હટાવી દે છે જે પ્રોગ્રામમાં ક્યાંય વપરાયા જ ન હોય.
- Dead Code Elimination:** કમ્પાઈલર લેવલ પર થતું ઓપ્ટિમાઈઝેશન, જે એવા કોડ બ્લોક્સને કાયમ માટે રીમૂવ કરી દે છે જે ક્યારેય એક્ઝિક્યુટ થવાના જ નથી (જેમ કે if (false) { ... }).
- Polyfills:** જ્યારે બ્રાઉઝર જૂનું હોય અને તેમાં જાવાસ્ક્રિપ્ટનું નવું ફીચર (દા.ત. Promise.allSettled) ઉપલબ્ધ ન હોય, ત્યારે તે ફીચરને મેન્યુઅલી બેકટ્રાઉન્ડ લોજિકથી જૂની સિન્ટેક્સમાં લખીને સપોર્ટેડ બનાવતા કોડને પોલીફિલ કહેવાય.
- Transpilers (Babel):** આ એક એવું ટૂલ છે જે મોડર્ન ES6+ અથવા લેટેસ્ટ જાવાસ્ક્રિપ્ટ કોડને કન્વર્ટ કરીને જૂની ES5 સિન્ટેક્સમાં ફેરવે છે જેથી ઇન્ટરનેટ એક્સપ્લોરર કે જૂના બ્રાઉઝર્સમાં કોડ કેસ ન થાય.

- Minification & Obfuscation:** કોડની ફાઇલ સાઈઝ ઘટાડવા વ્હાઇટ સ્પેસ અને કમેન્ટ્સ હટાવવાની પ્રોસેસ (Minification) તથા કોડને સિસ્ટોર કરવા માટે વેરીએબલ્સના નામ અથવા અક્ષરોમાં ફેરવવાની રીત (Obfuscation).

૪. સિક્યોરિટી અને એડવાન્સ વેબ આર્કિટેક્ચર (Security & Web Architecture)

- CORS (Cross-Origin Resource Sharing):** બ્રાઉઝરની એક મહત્વની સિક્યોરિટી પોલિસી. તે નક્કી કરે છે કે એક વેબસાઈટ (દા.ત. domain-a.com) બીજી વેબસાઈટના (domain-b.com) API કે સર્વર પરથી ડેટા સેફ્ટી લાવી શકે કે નહીં.
- XSS (Cross-Site Scripting):** એક ખતરનાક વેબ એટેક જ્યાં હેકર્સ વેબસાઈટના ઇનપુટ બોક્સ અથવા સુઆરએલ દ્વારા પોતાનો દૂષિત જાવાસ્ક્રિપ્ટ કોડ યુઝરના બ્રાઉઝરમાં રન કરાવી દે છે અને ફૂકાંઝ કે સેશન ડેટા ચોરી લે છે.
- CSRF (Cross-Site Request Forgery):** એવો એટેક જેમાં હેકર્સ યુઝરની જાણ બહાર તેના લોગ-ઇન સેશનનો ટૂથપોઝ કરીને બીજા કોઈ મેલિસિયસ પેજ પરથી ઓટિજિનલ સર્વર પર ઓટોમેટિકલી ફોડ રિક્વેસ્ટ સબમિટ કરાવી દે છે.
- Web Workers:** જાવાસ્ક્રિપ્ટ સિંગલ-થ્રેડ હોવાથી ભારે કેલ્ક્યુલેશન વખતે સ્ક્રીન હેંઝ થઈ જાય છે. વેબ વર્કર્સ તમને મેઈન યુઝર ઇન્ટરફેસ (UI) ને ડિસ્ટર્બ ક્યાં વગર બેકગ્રાઉન્ડમાં અલગ થ્રેડમાં ભારે લોજિક રન કરવાની સુવિધા આપે છે.

૫. એડવાન્સ અને છુપી જાવાસ્ક્રિપ્ટ ટર્મ્સ (Hidden Quirks & New ESNext)

- Short-Circuit Evaluation:** તાર્કિક ઓપરેટર્સ (&& અને ||) નો શોર્ટ-કટ ઉપયોગ. જો ડાબી બાજુની શરત પરથી જ આખો જવાબ નક્કી થઈ જતો હોય, તો જાવાસ્ક્રિપ્ટ જમણી બાજુનો કોડ ચેક પણ નથી કરતો.
- Temporal API:** વર્ષો જૂના, બગસથી ભરેલા ને અથરા Date() ઓબ્જેક્ટને કાયમ માટે રીપ્લેસ કરતું નવું ઓફિશિયલ ટાઈમ API, જે ટાઈમ-ઝોન કેલ્ક્યુલેશન સચોટ બનાવે છે.
- Explicit Resource Management (using કીવર્ડ):** જ્યારે કોઈ ઓબ્જેક્ટ, ડેટાબેઝ કનેક્શન કે ફાઇલ હેન્ડલર પોતાના બ્લોક સ્કોપની બહાર જાય, ત્યારે તેની મેમરી આપમેળે નેટિવ લેવલ પર ક્લીન થઈ જાય તે માટેનો લેટેસ્ટ કીવર્ડ.
- Math.sumPrecise():** જાવાસ્ક્રિપ્ટમાં $0.1 + 0.2 \neq 0.3$ વાળી પૂણ્ઠિક મેમરીની બહુ જૂની ભૂલને કાયમી માટે ફિક્સ કરતી નવી ઓફિશિયલ ગણિત પદ્ધતિ.
- Variable Shadowing:** જ્યારે કોઈ ઇનર સ્કોપ (જેમ કે ફંક્શનની અંદર) માં બહારના સ્કોપ જેવા જ સમાન નામનો વેરીએબલ ડિક્લેર કરવામાં આવે, ત્યારે તે અંદરનો વેરીએબલ બહારના વેરીએબલને છુપાવી દે છે.

JavaScript A to Z Ultimate Mega Master List

ગુગલ અને તમામ ટ્યુટોરિયલ્સમાંથી શોધેલા ૧૦૦% કન્સેપ્ટ્સની સંપૂર્ણ ડિક્શનરી (સરળ ગુજરાતીમાં)

1. Core Foundation & Modern Syntax

JavaScript Term / Concept	Ultimate Practical Explanation (ગુજરાતીમાં)
Variables (var, let, const)	var ફંક્શન-સ્કોપ્ડ છે અને હોઇસ્ટ થાય છે. let અને const બ્લોક-સ્કોપ્ડ છે અને ડિસ્ક્રિશન પહેલાં વાપરી શકાતી નથી (TDZ). const ની વેલ્યુ બદલી શકાતી નથી.
Primitive vs Reference Types	Primitive ડેટા (String, Number, Boolean, null, undefined, Symbol, BigInt) મેમરી સ્ટેકમાં સીધો સ્ટોર થાય છે. Reference ડેટા (Objects, Arrays) મેમરી હીપમાં સ્ટોર થાય છે અને વેરીએબલ પાસે માત્ર તેનો એડ્રેસ (રેફરન્સ) હોય છે.
Arrow Functions	ES6 ના ટૂંકા ફંક્શન્સ. આ ફંક્શન્સ પાસે પોતાનું 'this', 'arguments', કે 'super' હોતું નથી. તે તેની આસપાસના (Lexical) સ્કોપમાંથી 'this' મેળવે છે.
Rest vs Spread Operator (...)	Spread ઓપરેટર એરે કે ઓબ્જેક્ટને ખોલીને તેના એલિમેન્ટ્સ છુટા કરે છે. Rest ઓપરેટર છુટાછવાયા આર્ગ્યુમેન્ટ્સને ભેગા કરીને એક એરે બનાવે છે.
Destructuring Assignment	એરે અથવા ઓબ્જેક્ટમાંથી ડેટા બહાર કાઢીને ખૂબ જ સરળતાથી અલગ-અલગ વેરીએબલ્સમાં સ્ટોર કરવાની આધુનિક પદ્ધતિ.
Short-Circuit Evaluation	Logical Operators (&&,) નો ઉપયોગ કરીને શરતી લૉજિક ચલાવવું. && પહેલી false વેલ્યુ પર અટકે છે, પહેલી true વેલ્યુ પર અટકે છે.
Nullish Coalescing (??)	જો ડાબી બાજુની વેલ્યુ માત્ર 'null' અથવા 'undefined' હોય, તો જ આ ઓપરેટર જમણી બાજુની ડિફોલ્ટ વેલ્યુ સ્વીકારે છે. તે 0 કે ખાલી સ્ટ્રિંગને ફોલ્સો ગણતું નથી.
Optional Chaining (?.)	ઓબ્જેક્ટની અંદર ઊંડે રહેલી પ્રોપર્ટી ચેક કરતી વખતે જો વચ્ચે કોઈ પ્રોપર્ટી undefined કે null હોય, તો એરર આપવાને બદલે પ્રોગ્રામ સેફલી undefined રીટર્ન કરે છે.

2. Advanced Scoping & Closures

JavaScript Term / Concept	Ultimate Practical Explanation (ગુજરાતીમાં)
---------------------------	---

Execution Context	જાવાસ્ક્રિપ્ટ કોડ રન થાય ત્યારે બેકગ્રાઉન્ડમાં બનતું આખું વાતાવરણ. આમાં બે ફેઝ હોય છે: Memory Creation (જ્યાં જગ્યા ફાળવવાય છે) અને Code Execution (જ્યાં કોડ રન થાય છે).
Call Stack	બાઉન્ડરની એક એવર્મ સિસ્ટમ જે ટૂંક રાખે છે કે અત્યારે કયું ફંક્શન ચાલી રહ્યું છે અને તે પૂરું થયા પછી કયા ફંક્શન પર પાછા જવાનું છે. તે LIFO (Last In First Out) સિદ્ધાંત પર કામ કરે છે.
Hoisting	કોડ રન થાય તે પહેલાં જાવાસ્ક્રિપ્ટ એન્જિન વેરીએબલ અને ફંક્શન ડિસ્ક્રિશનને મેમરીમાં ટોચ પર વર્થ જાય છે. ફંક્શન પૂરેપૂરું હોઇસ્ટ થાય છે, var માં undefined મળે છે, પણ let/const એરર આપે છે.
Temporal Dead Zone (TDZ)	let અથવા const વેરીએબલના સ્કોપની શરૂઆતથી વર્ધન જ્યાં સુધી કોડમાં તેને એક્સ્યુઅલ ડિસ્ક્રિપ્ટ ન કરવામાં આવે તે વચ્ચેનો સમયગાળો. આ ઝોનમાં વેરીએબલ વાપરવાથી ReferenceError આપે છે.
Scope Chain & Lexical Environment	જ્યારે કોઈ વેરીએબલ કન્ટે ફંક્શનમાં ન મળે, ત્યારે જાવાસ્ક્રિપ્ટ તેના બહારના (Parent) સ્કોપમાં શોધે છે. અને છેક ગ્લોબલ સ્કોપ સુધી જાય છે. આ આખી ચેઇનને સ્કોપ ચેઇન કહે છે.
Closures	જ્યારે કોઈ ફંક્શન બીજા ફંક્શનની અંદર બને છે, ત્યારે તે અંદરનું ફંક્શન તેના પેરન્ટ ફંક્શનના વેરીએબલ્સને હંમેશા માટે યાદ રાખે છે (પેરન્ટ ફંક્શન રન થઈને પૂરું થઈ ગયું હોય તો પણ).
IIFE (Immediately Invoked Function Expression)	એવું ફંક્શન જે ડિફાઇન થતાંની સાથે જ ઓટોમેટિકલી તરત જ રન થઈ જાય છે. આનો ઉપયોગ ગ્લોબલ સ્કોપને પ્રોટેક્ટ થતો બચાવવા માટે થાય છે.

3. Object-Oriented JS, Prototypes & Decorators

JavaScript Term / Concept	Ultimate Practical Explanation (ગુજરાતીમાં)
Prototypal Inheritance	જાવાસ્ક્રિપ્ટમાં ક્લાસ બેઝડ ઇન્હેરિટન્સ નથી હોતું. અહીં દરેક ઓબ્જેક્ટ પાસે એક ઝૂમ લિંક (Prototype) હોય છે, જેના દ્વારા તે બીજા ઓબ્જેક્ટની પ્રોપર્ટીઝ અને મેથડ્સ વાપરી શકે છે.
Prototype Chain	જ્યારે ઓબ્જેક્ટમાં કોઈ પ્રોપર્ટી ન મળે, ત્યારે જાવાસ્ક્રિપ્ટ તેના પ્રોટોટાઇપમાં શોધે છે, પછી તેના પણ પ્રોટોટાઇપમાં શોધે છે, જ્યાં સુધી છેલ્લે null ન મળે.
this Keyword Mechanics	this ની વેલ્યુ ફંક્શન ક્યાં ચલાયું છે તેના પર નહીં, પણ ફંક્શન કઈ રીતે કોલ થયું છે તેના પર આધાર રાખે છે. ગ્લોબલ કોલમાં તે window બતાવે છે, મેથડ કોલમાં તે ઓબ્જેક્ટ બતાવે છે.
call(), apply(), bind()	ફંક્શનની અંદર 'this' ની વેલ્યુ મેન્યુઅલી બદલવાની રીતો. call() આર્ગ્યુમેન્ટ્સ નોંધવી લે છે, apply() આર્ગ્યુમેન્ટ્સ એરે તરીકે લે છે, અને bind() એક નવું કન્ટ્રોલર ફંક્શન બનાવીને આપે છે.
ES6 Classes & Syntactic Sugar	જાવાસ્ક્રિપ્ટમાં ક્લાસ એ કોઈ નવી સિસ્ટમ નથી, પણ પ્રોટોટાઇપ સિસ્ટમ પર લખાયેલું એક સુંદર આવરણ (Syntactic Sugar) છે, જેથી જાવા જેવી ભાષાઓના ડેવલપર્સને કોડ ઓળખવામાં સરળતા રહે.
Decorators	ક્લાસ, મેથડ, પ્રોપર્ટી કે પેરામીટરના બિહેવિયરને કોડ રન થતા પહેલાં

	મોડિફાય કે એક્સટેન્ડ કરવા માટે વપરાતી એડવાન્સ ડિઝાઇન પેટર્ન સિન્ટેક્સ ('@decoratorName').
Object.defineProperty()	કોઈ ઓબ્જેક્ટની પ્રોપર્ટી પર સખત નિયમો લાગુ પાડવાની રીત. આનાથી તમે નક્કી કરી શકો છો કે પ્રોપર્ટી બદલી શકાશે કે નહીં (writable), વ્યૂપમાં ટેબાશે કે નહીં (enumerable), કે ડીલીટ થઈ શકશે કે નહીં (configurable).

4. Asynchronous, Generators & Event Loop

JavaScript Term / Concept	Ultimate Practical Explanation (ગુજરાતીમાં)
Callback Hell & Pyramid of Doom	અસિંક્રોનસ કામો કરવા માટે જ્યારે એક કોલબેકની અંદર બીજો અને બીજાની અંદર ત્રીજો કોલબેક લખીએ, ત્યારે કોડ પિરામિડ જેવો અવાચ્ય બની જાય છે, જેને મેનેજ કરવો અશક્ય બને છે.
Promises (State & Chaining)	અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે તેની ખાતરી આપતું ઓબ્જેક્ટ. તેની 3 સ્ટેટ્સ હોય છે: Pending, Fulfilled (સફળ), અને Rejected (અસફળ).
Async / Await	પ્રોમિસિસ ચેઇનિંગ (.then().catch()) ને વધુ સરળ અને સિંક્રોનસ કોડ જેવો સાફ બતાવવા માટેનું આધુનિક સિન્ટેક્સ ફીચર.
Event Loop & Microtask Priority	જાવાસ્ક્રિપ્ટ સિંગલ-થ્રેડ હોવા છતાં બેકગ્રાઉન્ડ ટાસ્ક સંભાળી શકે છે. તે ઇવેન્ટ લૂપને કારણે છે. પ્રોમિસિસ (Microtasks) ને setTimeout (Macrotasks) કરતાં હંમેશા પહેલી પ્રાયોરિટી મળે છે.
Generator Functions (function*)	આ એવા ખાસ ફંક્શન છે જેને અથવચ્ચેથી રોકી શકાય છે ('yield' કીવર્ડ દ્વારા) અને પછી ગમે ત્યારે અટકેલી જગ્યાએથી જ ફરીથી આગળ રન કરી શકાય છે.
Asynchronous Iterators (for await...of)	એસિંક્રોનસ ડેટાના સ્ટ્રીમ્સ અથવા પ્રોમિસિસના એરે પર લૂપ ચલાવીને એક પછી એક ડેટા સેફવી રીડ કરવાની લૂપ સિસ્ટમ.

5. Browser Mechanics & Web APIs (BOM/DOM)

JavaScript Term / Concept	Ultimate Practical Explanation (ગુજરાતીમાં)
Window & BOM (Browser Object Model)	બ્રાઉઝરનો સૌથી મોટો પેરન્ટ ગ્લોબલ ઓબ્જેક્ટ. આના દ્વારા બ્રાઉઝરની ટેબ, સ્ક્રીન સાઇઝ, એવર્ટ્સ અને ફંક્શનલિટી કંટ્રોલ થાય છે.
Navigator & Device API	યુઝરના બ્રાઉઝર અને ડિવાઇસની ગ્રુપ માહિતી મેળવવાનો ઓબ્જેક્ટ. જેમ કે, યુઝર ઓનલાઇન છે કે ઓફલાઇન (navigator.onLine), કઈ બેટરી વાઈફ છે, કે કયું લોકેશન છે.
Location & History	Location ઓબ્જેક્ટ કરન્ટ પેજનું URL મેનેજ અને રીડાઈરેક્ટ કરે છે. History ઓબ્જેક્ટ બ્રાઉઝરના ફોરવર્ડ અને બેક બટનનું લોજિક કોડિંગથી કંટ્રોલ કરે છે.
Event Bubbling vs Capturing	જ્યારે યાઇલ્ડ એવિમેન્ટ પર ઇવેન્ટ ટ્રિગર થાય, ત્યારે તે પેરેન્ટ તરફ ઉપર જાય તેને Bubbling કહે છે. ઇવેન્ટ પેરેન્ટથી શરુ થઈને નીચે યાઇલ્ડ તરફ આવે તેને Capturing કહે છે.

Event Delegation	મલ્ટિપલ યાઇલ્ડ એવિમેન્ટ્સ પર અલગ-અલગ ઇવેન્ટ લિસનર મુકવાને બદલે તેમના કોમન પેરન્ટ પર સિંગલ ઇવેન્ટ લિસનર મુકીને પરફોર્મન્સ બૂસ્ટ કરવાની આધુનિક રીત.
Web Workers	જાવાસ્ક્રિપ્ટના મેઈન થ્રેડ (UI Thread) ને ફ્રી રાખવા માટે બેકગ્રાઉન્ડમાં ભારે અને જટિલ કેલ્ક્યુલેશન અલગ થ્રેડમાં ચલાવવાની ઓફિશિયલ ટેકનિક.

6. JS Engine Internal Quirks & Security

JavaScript Term / Concept	Ultimate Practical Explanation (ગુજરાતીમાં)
Implicit Type Coercion	જાવાસ્ક્રિપ્ટ એન્જિન બેકગ્રાઉન્ડમાં આપમેળે ટાઇપ બદલી નાખે છે. જેમ કે '5 + '5' = '55' (રિટર્ન) બને છે, પણ '5 * '5' = 0' (નંબર) બની જાય છે!
Object.is() Exact Precision	=== કરતાં પણ વધુ સચોટ સમાનતા ચેકિંગ. સામાન્ય રીતે જાવાસ્ક્રિપ્ટમાં 'NaN === NaN' કરવાથી false આવે છે, પરંતુ 'Object.is(NaN, NaN)' હંમેશાં true આપે છે.
JIT (Just-In-Time) Compilation	જાવાસ્ક્રિપ્ટ એન્જિન (V8) કોડને લાઈન-બાય-લાઈન ઇન્ટરપ્રિટેટ કરવાની સાથે-સાથે રન-ટાઇમ પર જ તેને સુપરફાસ્ટ મશીન કોડમાં કમ્પાઇલ પણ કરતું રહે છે.
Memory Leaks & Garbage Collection	જ્યારે કોડમાં અજાણતા ગ્લોબલ વેરીએબલ્સ કે નકામા ઇવેન્ટ લિસનર્સ રહી જાય ત્યારે મેમરી લીક થાય છે. Garbage Collector (Mark-and-Sweep લોજિક) બિનઉપયોગી મેમરી આપમેળે સાફ કરે છે.
Tree Shaking	વેબપેક કે રોલઅપ જેવા બંડલર્સ જ્યારે પ્રોડક્શન કોડ બિલ્ડ કરે, ત્યારે ફાઇલોમાંથી ક્યારેય ન વપરાયેલી (Dead Code) કોડ કાચમી માટે હટાવી દે છે, તેને ટ્રી શેકિંગ કહે છે.
CORS (Cross-Origin Resource Sharing)	બ્રાઉઝરની એક સિક્યોરિટી પોલિસી જે એક વેબસાઇટના ડેટાને બીજી વેબસાઇટના સર્વર પરથી API કોલ કરીને ડેટા ચોરતા રોકે છે (જ્યાં સુધી સર્વર પરમિશન ન આપે).
XSS & CSRF Exploits	XSS એટલે હેકર તમારી વેબસાઇટમાં ખોટો જાવાસ્ક્રિપ્ટ ઇન્જેક્ટ કરીને ફંક્શન ચોરી લે. CSRF એટલે યુઝરની જાણ બહાર તેના જ લોગઇન સેશનનો ફરેપયોગ કરીને ખોટી રિસ્પોસ્ટ સબમિટ કરાવી દેવી.

JavaScript A to Z Mega Master List

(આખા ગૂગલના તમામ કન્સેપ્ટ્સ અને ટર્મ્સ એક જ ફાઇલમાં)

આ એક અલ્ટીમેટ મેગા માસ્ટર લિસ્ટ છે જે જાવાસ્ક્રિપ્ટના ઇતિહાસના તમામ બેઝિક, એડવાન્સ, એનિન-વેવ, સિન્ક્રોરિટી અને બેટેસ્ટ ES2025/ES2026 ના કન્સેપ્ટ્સને એકદમ સરળ અને વાંચી શકાય તેવી ગુજરાતી ભાષામાં આવરી લે છે.

૧. બેઝિક અને ફાઉન્ડેશન ટર્મ્સ (Foundational Core)

Concept / Term	સરળ ગુજરાતી સમજૂતી (Explanation)
Variables (વેરીએબલ્સ)	ડેટા સ્ટોર કરવા માટેના કન્ટેનર. JavaScript માં ત્રણ રીતે ડિફાઇન થાય છે: var (જૂનું, ફંક્શન સ્કોપ), let (બ્લોક સ્કોપ), અને const (જેની વેલ્યુ બદલી ન શકાય).
Data Types (ડેટા ટાઇપ્સ)	JavaScript બે પ્રકારના ડેટા ટાઇપ ધરાવે છે. Primitive: String, Number, Boolean, Null, Undefined, Symbol, BigInt. Non-Primitive: Objects, Arrays.
Arrays (એરે)	એક જ વેરીએબલમાં એક કરતાં વધુ વેલ્યુઝનું લિસ્ટ સ્ટોર કરવા માટેનું કલેક્શન. આ ઝીરો-બેઝ્ડ ઇન્ડેક્સ (0-indexed) પર કામ કરે છે.
Objects (ઓબ્જેક્ટ્સ)	કી-વેલ્યુ (Key-Value) ના સ્વરૂપમાં પ્રોપર્ટીઝ અને મેથડ્સ સ્ટોર કરવાનું સ્ટ્રક્ચર. ઇ.ત.ત. {name: 'John', age: 25}.
Functions (ફંક્શન્સ)	ચોક્કસ કામ કરવા માટે બનાવેલો કોડનો ટુકડો, જેને ગમે ત્યારે કોલ કરીને ફરીથી વાપરી (reusability) શકાય છે.
Arrow Functions (એરો ફંક્શન્સ)	ES6 માં આવેલી ફંક્શન્સ લખવાની ટૂંકી રીત '()' => '{}'. આનું પોતાનું 'this' નથી હોતું, તે બહારના સ્કોપમાંથી 'this' લે છે.

Operators (ઓપરેટર્સ)	વેલ્યુઝ પર ડેલ્ફ્યુવેશન કે સરખામણી કરવા માટેના ચિહ્નો (જેમ કે Arithmetic '+, -, Logical '&&, ', Assignment '=').
----------------------	--

૨. એડવાન્સ સ્કોપ, ક્લોઝર અને એક્ઝિક્યુશન

Concept / Term	સરળ ગુજરાતી સમજૂતી (Explanation)
Execution Context (એક્ઝિક્યુશન કોન્ટેક્સ્ટ)	JavaScript કોડ કઈ રીતે રન થઈ રહ્યો છે અને અત્યારે કયા વેરીએબલ્સ ઉપલબ્ધ છે, તેનું બ્રાઉઝર એનિન દ્વારા બનાવાતું આખું પર્યાવરણ (Environment).
Call Stack (કોલ સ્ટેક)	જ્યારે મલ્ટિપલ ફંક્શન્સ કોલ થાય, ત્યારે કયું ફંક્શન અત્યારે રન થઈ રહ્યું છે અને તેના પછી કયું રન થશે, તેનો ઓર્ડર (LIFO - Last In First Out) મેનેજ કરતી ટેમ્પરરી મેમરી.
Hoisting (હોઇસ્ટિંગ)	JavaScript કોડ રન થાય તે પહેલાં વેરીએબલ અને ફંક્શનના ડિફરેન્સિયલ સ્કોપમાં સૌથી ઉપર માનસિક રીતે લઈ જાય છે. આના લીધે ડિફરેન્સિયલ કયા પહેલાં પછા વસ્તુઓ એક્સેસ કરવાનો ટ્રાય કરી શકાય છે.
Temporal Dead Zone (TDZ)	'let' અને 'const' વેરીએબલ જ્યાં સુધી કોડની લાઇનમાં એક્સ્યુઅલ ડિફાઇન ન થાય, ત્યાં સુધી તેને એક્સેસ ન કરી શકાય તે વચ્ચેના સમયગાળા કે ઝોનને TDZ કહે છે.
Scope & Scope Chain (સ્કોપ)	કોઈ વેરીએબલ કોડમાં ક્યાં ઉપલબ્ધ છે તેની સીમા. જો કોઈ વેરીએબલ કન્ટે સ્કોપમાં ન મળે, તો જાવાસ્ક્રિપ્ટ બહારના પેરેન્ટ સ્કોપમાં ચેક કરવા જાય છે, જેને સ્કોપ ચેઇન કહેવાય છે.
Closures (ક્લોઝર્સ)	જ્યારે કોઈ ફંક્શન તેના બહારના ફંક્શનના વેરીએબલ્સને યાદ રાખે (તે બહારનું ફંક્શન પૂરું થઈ ગયું હોય તો પણ), તેને ક્લોઝર કહેવાય.
IIFE (Immediately Invoked Function Expression)	એવું ફંક્શન જે કોડમાં બનાતાની સાથે જ તરત જ ઓટોમેટિકલી કોઈ પણ મેન્યુઅલ કોલ વગર રન થઈ જાય છે.

૩. અસિંક્રોનસ જાવાસ્ક્રિપ્ટ (Async JS & Event Loop)

Concept / Term	સરળ ગુજરાતી સમજૂતી (Explanation)
Asynchronous JavaScript	જ્યારે કોઈ ભારે કામ (જેમ કે API માંથી ડેટા લાવવો) ચાલતું હોય, ત્યારે બાકીના કોડને રોક્યા વગર બેકગ્રાઉન્ડમાં કામ કરવાની પદ્ધતિ.
Event Loop (ઇવેન્ટ લૂપ)	JavaScript સિંગલ-થ્રેડેડ (એક સમયે એક જ કામ કરતી) હોવા છતાં બેકગ્રાઉન્ડ ટાસ્ક્ને મેનેજ કરીને કોડને સ્પષ્ટલી ચલાવે છે તે આ લૂપના કારણે છે.
Callback Hell (Pyramid of Doom)	જ્યારે એક ફંક્શનની અંદર બીજું, અને બીજાની અંદર ત્રીજું—એમ અસંખ્ય નેસ્ટેડ કોલબેક લખવામાં આવે ત્યારે કોડનો આકાર પિરામિડ જેવો અઘરો થઈ જાય છે. આને સોલ્વ કરવા પ્રોમિસિસ આવ્યાં.
Promises (પ્રોમિસ)	અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે (સફળ કે અસફળ) તેની માત્રો આપતું સ્પેશિયલ ઓબ્જેક્ટ સ્ટેટસ (Pending, Fulfilled, Rejected).
Async / Await	પ્રોમિસિસ અને અસિંક્રોનસ કોડને સીધો, ફ્લેટ અને સિંક્રોનસ કોડ જેવો સરળ રીતે લખવાની આધુનિક મોર્ન રીત.
Microtask vs Macrotask Queue	પ્રોમિસિસના કામો Microtask Queue માં જાય છે (જેને પહેલી પ્રાયોરિટી મળે છે) અને setTimeout કે DOM ઇવેન્ટ્સ Macrotask Queue માં જાય છે.
Starvation (ભૂખમરો)	જો Microtask Queue માં સતત નવા-નવા પ્રોમિસિસના ટાસ્ક્ આવ્યાં જ કરે, તો Macrotask Queue ને રન થવાનો મોકો જ નથી મળતો, તેને સ્ટાર્વેશન કહે છે.

૪. ઓબ્જેક્ટ ઓરિએન્ટેડ પ્રોગ્રામિંગ (OOPs & Prototypes)

Concept / Term	સરળ ગુજરાતી સમજૂતી (Explanation)
Prototypal Inheritance	JavaScript માં ક્લાસ આધારિત ઇન્હેરિટેન્સ નથી હોતું. અહીં એક

	ઓબ્જેક્ટ સીધો બીજા ઓબ્જેક્ટની પ્રોપર્ટીઝ અને મેથડ્સ પ્રોટોટાઇપ લિંક દ્વારા વારસામાં મેળવે છે.
Prototype Chain (પ્રોટોટાઇપ ચેઇન)	જ્યારે ઓબ્જેક્ટમાં કોઈ મેથડ ન મળે, ત્યારે તે તેના પ્રોટોટાઇપમાં શોધે છે, જો ત્યાં પણ ન મળે તો તેના પેરેન્ટ પ્રોટોટાઇપમાં શોધે છે. આ આપની લાઇનને પ્રોટોટાઇપ ચેઇન કહે છે.
Classes & Constructors (ES6 Classes)	ઓબ્જેક્ટ્સ બનાવવા માટેનું મોર્ન બુલ્ડિંગ સ્ટ્રક્ચર, જે પાછળથી પ્રોટોટાઇપ મિકેનિઝમ પર જ કામ કરે છે.
this Keyword	એક ખાસ કીવર્ડ જે અત્યારે રન થઈ રહેલો કોડ કયા ઓબ્જેક્ટના કોન્ટેક્સ્ટ કે માલિકી હેઠળ ચાલી રહ્યો છે તેનો રેફરન્સ આપે છે.
Call, Apply, Bind	કોઈ ફંક્શનની અંદર મેન્યુઅલી 'this' ની વેલ્યુ સેટ કરવા માટેની મેથડ્સ. Call સીધી વેલ્યુ લે છે, Apply એરેમાં વેલ્યુ લે છે, અને Bind નવું ફંક્શન રિટર્ન કરે છે.

૫. DOM, BOM અને બ્રાઉઝર આર્કિટેક્ચર

Concept / Term	સરળ ગુજરાતી સમજૂતી (Explanation)
DOM (Document Object Model)	HTML પેજનું એક ઓબ્જેક્ટ ટ્રી-સ્ટ્રક્ચર. જેના દ્વારા JavaScript વેબ પેજના કોઈ પણ એલિમેન્ટ (ટેક્સ્ટ, કલર, ટેગ) ને લાઇવ બદલી શકે છે.
BOM (Browser Object Model)	બ્રાઉઝર વિન્ડો સાથે વાતચીત કરવા માટેની સિસ્ટમ. આમાં Window, Navigator, Screen, History, અને Location ઓબ્જેક્ટ્સનો સમાવેશ થાય છે.
Event Bubbling & Capturing	ઇવેન્ટ ટાર્ગેટથી શરૂ થઈને ઉપર પેરેન્ટ સુધી જાય તેને Bubbling કહેવાય. ઇવેન્ટ ઉપરથી શરૂ થઈને નીચે ટાર્ગેટ એલિમેન્ટ સુધી આવે તેને Capturing કહેવાય.
Event Delegation	દરેક ઇલેમન્ટ એલિમેન્ટ પર અલગ ઇવેન્ટ લિસનર મુકવાને બદલે માત્ર એક જ કોમન પેરેન્ટ એલિમેન્ટ પર ઇવેન્ટ હેન્ડલર મુકવાની સ્ટ્રાટેજી અને ફાસ્ટ પદ્ધતિ.

૬. JavaScript Quirks અને હિડન મિકેનિઝમ્સ

Concept / Term	સરળ ગુજરાતી સમજૂતી (Explanation)
Implicit Type Coercion (ટાઇપ કોઅર્સન)	JavaScript બેકગ્રાઉન્ડમાં આપમેળે ડેટા ટાઇપ બદલી નાખે છે. દા.ત. '5' + '5' = '55' (સ્ટ્રિંગ) પણ '5' - '5' = 0 (નંબર).
Object.is()	'=== ' કરતાં પણ વધુ સચોટ ચેકિંગ. તે 'NaN === NaN' ને પણ સાચું ('true') સાબિત કરી શકે છે, જે સામાન્ય રીતે જાવાસ્ક્રિપ્ટમાં 'false' આવે છે.
Unary Plus Trickling ('+[]')	કોઈ પણ ખાલી એરે '[]' ની આગળ માત્ર પ્લસ નિશાની '+' મુકવાથી જાવાસ્ક્રિપ્ટ તેને ઓટોમેટિકલી નંબર '0' માં કન્વર્ટ કરી નાખે છે.
Double Dot Notation ('35..toString()')	કોઈ પણ હોવ પૂર્ણાંક નંબર પર સીધો મેથડ કોલ કરવા ડબલ ડોટ વાપરવો પડે છે જેથી પાર્સર તેને ડેસિમલ પોઇન્ટ ન સમજે.

૭. બ્રાઉઝર એન્જિન અને ઓપ્ટિમાઇઝેશન (Engine Level)

Concept / Term	સરળ ગુજરાતી સમજૂતી (Explanation)
JIT Compilation (જસ્ટ-ઇન-ટાઇમ)	JavaScript નો કોડ લાઇન બાય લાઇન ઇન્ટરપ્રેટરેડ પણ થાય છે અને ફાસ્ટ પરફોર્મન્સ માટે રન-ટાઇમ વખતે જ કમ્પાઇલ થઈને મશીન કોડમાં પણ ફેરવાય છે.
Memory Heap & Garbage Collection	વેરીએબલ્સ અને ઓબ્જેક્ટ્સ મેમરી હીપમાં સ્ટોર થાય છે. જ્યારે કોઈ ઓબ્જેક્ટનો વપરાશ પૂરો થઈ જાય, ત્યારે Garbage Collector (Mark-and-Sweep લોજિક) તેને ઓટોમેટિક ડિલીટ કરે છે.
Inline Caching & Hidden Classes	V8 એન્જિન ઓબ્જેક્ટ્સ પ્રોપર્ટીઝને ખૂબ જ ઝડપથી એક્સેસ કરવા માટે બેકગ્રાઉન્ડમાં ઓટોમેટિકલી હિડન ક્લાસીસ અને કેચિંગ મિકેનિઝમ ડિઝાઇન કરે છે.
Tree Shaking	મોડર્ન બંડલર્સ (Webpack, Vite) દ્વારા કોડ બિલ્ડ કરતી વખતે નકામો અને પ્રોજેક્ટમાં ક્યાંય ન વપરાયેલો કોડ આપમેળે હટાવી દેવાની

પ્રક્રિયા.

૮. સિક્યોરિટી અને એડવાન્સ વેબ આર્કિટેક્ચર

Concept / Term	સરળ ગુજરાતી સમજૂતી (Explanation)
CORS (Cross-Origin Resource Sharing)	બ્રાઉઝરની એક સિક્યોરિટી પોલિસી જે નક્કી કરે છે કે એક વેબસાઇટ બીજી વેબસાઇટના સર્વર પરથી ડેટા એક્સેસ કરી શકે કે નહીં.
XSS & CSRF (વેબ એટેક્સ)	XSS એટલે હેકર્સ દ્વારા વેબસાઇટમાં નકામો જાવાસ્ક્રિપ્ટ કોડ ઇન્જેક્ટ કરવો. CSRF એટલે યુઝરના લોગિન સેશનનો ખોટો ફાયદો ઉઠાવી અનઅધિકારિત ડેવિસ સબમિટ કરાવો.
Web Workers	ભારે ગણતરીઓ અને પ્રોસેસિંગને મેન ટા થ્રેડની બહાર બેકગ્રાઉન્ડ થ્રેડમાં ચલાવવાની સિસ્ટમ, જેથી યુઝર ઇન્ટરફેસ લેગ (લેટન્સી) ન થાય.

૯. મોડર્ન ESNext ફીચર્સ (ES2025/ES2026+)

Concept / Term	સરળ ગુજરાતી સમજૂતી (Explanation)
Temporal API	જૂના અને અધરા 'Date()' ઓબ્જેક્ટને સિમ્પલ કરતું નવું ઓફિશિયલ API, જે ટાઇમ-ઝોન મેનેજમેન્ટને અત્યંત સરળ બનાવે છે.
Explicit Resource Management ('using')	મેમરી ક્લીનિંગ માટેનો નવો કીવર્ડ. જ્યારે પણ કોઈ ઓબ્જેક્ટ કે કનેક્શન પોતાના બ્લોક સ્કોપની બહાર જાય, ત્યારે તેની મેમરી ઓટોમેટિક રીલીઝ થાય છે.
Math.sumPrecise()	જાવાસ્ક્રિપ્ટમાં '0.1 + 0.2 !== 0.3' વાળી ફ્લોટિંગ પોઇન્ટ મેમરી ભૂલને નેટિવ લેવલ પર કાયમી માટે સોલ્વ કરતી નવી મેથડ.

JavaScript A to Z Mega Master List - Final Edition

JavaScript એટલું મોટું છે કે તેના એન્જિન, મેમરી મેનેજમેન્ટ, નેટવર્કિંગ અને એડવાન્સ બ્રાઉઝર ઓપ્ટિમાઇઝેશન પાછળ હજારો કન્સેપ્ટ્સ કામ કરે છે. ગૂગલ કે સામાન્ય સાઇટ્સ પર ન મળતી, બાકી રહેલી તમામ અતિ-એડવાન્સ ટર્મ્સ આ ફાઇનલ વર્ડ ફાઇલમાં પૂરી કરવામાં આવી છે.

૧. JavaScript Engine & Memory Deep Dive (મેમરી અને ઇન્ટરનલ મિકેનિઝમ)

- V8 Engine:** ગૂગલ ક્રોમ અને Node.js માં વપરાતું સુપર-ફાસ્ટ એન્જિન, જે જાવાસ્ક્રિપ્ટ કોડને મશીન કોડમાં ફેરવે છે.
- Just-in-Time (JIT) Compilation:** જાવાસ્ક્રિપ્ટ રન થતી વખતે જ કમ્પાઇલ થાય છે. એન્જિન કોડ જોતું જાય અને તેને વધુ ને વધુ ફાસ્ટ મશીન કોડમાં બદલતું જાય.
- Memory Heap:** જાવાસ્ક્રિપ્ટમાં જ્યારે તમે કોઈ મોટા ઓબ્જેક્ટ્સ કે એરે બનાવો છો, ત્યારે તે મેમરીના જે અનસ્ટ્રક્ચર્ડ ભાગમાં સ્ટોર થાય છે તેને હીપ (Heap) કહે છે.
- Call Stack:** તમારા ફંક્શન્સ કયા ઓર્ડરમાં રન થશે તેનો ટ્રેક રાખતી જગ્યા. એક ફંક્શન પૂરું થાય એટલે તે સ્ટેકમાંથી નીકળી જાય.
- Stack Overflow:** જ્યારે કોઈ ફંક્શન અટક્યા વગર પોતાની જાતને જ વારંવાર કોલ કરે (દા.ત. અન્ટીમેટ રિકર્સન), ત્યારે સ્ટેકની મેમરી ભરાઈ જાય છે અને બ્રાઉઝર કેશ થઈ જાય છે.
- Garbage Collection (Mark-and-Sweep):** જાવાસ્ક્રિપ્ટનું ઓટોમેટિક ક્લીનર. જે ઓબ્જેક્ટ્સ હવે કોડમાં ક્યાંય વપરાતા નથી, તેને શોધીને મેમરીમાંથી આપમેળે ડીલીટ કરી નાખે છે.
- Memory Leak:** જ્યારે કોઈ ઓબ્જેક્ટ હવે કામનો ન હોય, પણ ભૂલથી ગ્લોબલ સ્કોપ કે કોઈ ઇવેન્ટ લિસનરમાં અટકેલો રહે, તો ગાર્બજ કલેક્ટર તેને ડીલીટ નથી કરી શકતું, જેને મેમરી લીક કહેવાય.
- Inline Caching (IC):** V8 એન્જિન ઓબ્જેક્ટની પ્રોપર્ટીઝ ફાસ્ટ શોધવા માટે અગાઉ શોધેલા લોકેશનને કેશ (યાદ) રાખે છે.
- Hidden Classes (Shapes):** જાવાસ્ક્રિપ્ટ ડાયનેમિક છે, પણ બેકગ્રાઉન્ડમાં એન્જિન ઓબ્જેક્ટ્સ માટે હિડન ક્લાસીસ બનાવે છે જેથી પ્રોપર્ટીઝ એક્સેસ કરવાનું સ્પીડી બને.

૨. Micro-Optimizations & Modern Patterns (કોડ ઓપ્ટિમાઇઝેશન)

- Tree Shaking:** જ્યારે તમે કોઈ મોટો કોડ બિલ્ડ કરો છો, ત્યારે જે ફંક્શન્સ ક્યાંય વાપર્યા નથી, તેને પાંદડાની જેમ ખંખેરીને (ડીલીટ કરીને) ફાઇલ નાની કરવાની પ્રોસેસ.

- Dead Code Elimination:** કમ્પાઇલર લેવેલે એવો કોડ હટાવી દેવો જે ક્યારેય રન જ નથી થવાનો (દા.ત. `if(false) { ... }`).
- Polyfill:** નવા ફીચર્સ (જેમ કે 'Promise.allSettled') જો જૂના બ્રાઉઝરમાં ન ચાલતા હોય, તો તેને ચલાવવા માટે જાતે લખવામાં આવતો બેકઅપ કોડ.
- Transpiler (Babel):** મોડર્ન JavaScript (ES6+) કોડને જૂના બ્રાઉઝર સમજી શકે તેવા મોડેલ (ES5) માં કન્વર્ટ કરતું ટૂલ.
- Minification:** કોડમાંથી બધી જ સ્પેસ, નકામી લાઇનો અને વેરીએબલના નામ નાના (જેમ કે 'let a = 1') કરીને ફાઇલ સાઈઝ નાની કરવાની રીત.

૩. Meta-Programming & Specialized APIs (અદ્યતન હિડન કન્સેપ્ટ્સ)

- Proxy Object:** કોઈ ઓબ્જેક્ટની આસપાસ એક ક્લય (ગ્રુપ એજન્ટ) બનાવવું, જે ઓબ્જેક્ટમાં કંઈ પણ રીડ કે રાઈટ થાય ત્યારે તેને અથવચ્ચે રોકીને કસ્ટમ લોજિક ચલાવી શકે.
- Reflect API:** ઓબ્જેક્ટ ઓપરેશન્સ (જેમ કે પ્રોપર્ટી ડીલીટ કરવી કે ચેક કરવી) કરવા માટેની નવી અને સ્ટાન્ડર્ડ મેથડ્સ.
- Intl API (Internationalization):** કરન્સી, ડેટા, અને ટાઇમને ટૂંકાવાના અલગ-અલગ દેશોના ફોર્મેટ મુજબ ઓટોમેટિક સેટ કરવાનું નેટિવ API.
- TypedArrays (ArrayBuffer):** જાવાસ્ક્રિપ્ટમાં જ્યારે ટેકસ્ટ સિવાય સીધો જ બાઈનરી ડેટા, ઇમેજ પિક્સેલ્સ કે ઓડિયો ડેટા હેન્ડલ કરવો હોય ત્યારે વપરાતા ખાસ એરે.

૪. Web Security Concepts in JS (સિક્યોરિટી ટર્મ્સ)

- Cross-Origin Resource Sharing (CORS):** બ્રાઉઝરની એક સિક્યોરિટી વોલ, જે નક્કી કરે છે કે તમારો કોડ કોઈ બીજી વેબસાઇટની API માંથી ડેટા લાવી શકે કે નહીં.
- XSS (Cross-Site Scripting):** એક ગંભીર હેકિંગ એટેક, જેમાં હેકર તમારી વેબસાઇટના ઇનપુટ બોક્સમાં ભૂલથી પોતાનો ખરાબ જાવાસ્ક્રિપ્ટ કોડ રન કરાવી દે છે.
- CSRF (Cross-Site Request Forgery):** હેકર યુઝરના લોગ-ઇન સેશનનો ફાયદો ઉઠાવીને તેની જાણ બહાર બેકગ્રાઉન્ડમાં ખોટી રિક્વેસ્ટ સબમિટ કરાવી દે છે તે એટેક.

JavaScript A to Z Master List (નવા અને છુપા

કન્સેપ્ટ્સ)

૧. Callback Hell અને અસિંક્રોનસ ફ્લોસ (Async Flows)

- Callback Hell (Pyramid of Doom):** જ્યારે કોડમાં એક ફંક્શનની અંદર બીજું, અને બીજાની અંદર ત્રીજું એમ ઘણા બધા નેસ્ટેડ ફંક્શન્સ આવી જાય, ત્યારે કોડનો આકાર પિરામિડ જેવો અંધરો થઈ જાય છે. તેને ઉકેલવા પ્રોમિસિસ અને એસિંક-અવેટ આવ્યા.
- Inversion of Control (IoC):** કોલબેક ફંક્શન્સની એક મોટી ખામી, જ્યાં આપણો કોડ કોઈ થર્ડ-પાર્ટી લાઇબ્રેરી પર નિર્ભર થઈ જાય છે અને આપણો કંટ્રોલ નથી રહેતો.
- Microtask Queue vs Macrotask Queue:** પ્રોમિસિસના કામો Microtask Queue માં જાય છે અને setTimeout જેવા કામો Macrotask Queue માં જાય છે. જાવાસ્ક્રિપ્ટ હંમેશા માઇક્રોટાસ્ક્સને પહેલી પસંદગી આપે છે.
- Starvation (જૂમખરો):** જો Microtask Queue માં સતત નવા-નવા પ્રોમિસિસના કામ આવ્યા જ કરે, તો Macrotask Queue ને રન થવાનો મોકો જ નથી મળતો અને તે અટકી પડે છે.

૨. બ્રાઉઝર ઓબ્જેક્ટ મોડલ (Complete BOM)

- Window Object:** બ્રાઉઝરનો સૌથી મોટો ગ્લોબલ ઓબ્જેક્ટ, જેની અંદર આખી જાવાસ્ક્રિપ્ટ રન થાય છે.
- Navigator Object:** બ્રાઉઝર અને યુઝરના ડિવાઇસની માહિતી આપે છે, જેમ કે યુઝર ઓનલાઇન છે કે ઓફલાઇન (navigator.onLine).
- Location Object:** કરન્ટ વેબ પેજનું URL જાણવા અથવા નવું પેજ ખોલવા (location.href) માટે વપરાય છે.
- History Object:** યુઝરે અગાઉ વિઝિટ કરેલા પેજ્સનો ટ્રેક રાખે છે (બેક કે ફોરવર્ડ જવા માટે).
- User Agent:** બ્રાઉઝરની ખાસ આઇડેન્ટિટી સિદ્ધિ જે સર્વરને યુઝરના બ્રાઉઝર અને ઓપરેટિંગ સિસ્ટમની માહિતી મોકલે છે.

૩. JavaScript Quirks (અજીબ લીજિક)

- Implicit Type Coercion:** જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ડેટા ટાઇપ બદલી નાખે છે. (જેમ કે ૫ + '5' = '55' થાય પણ ૫ - '5' = 0 થાય).
- Object.is():** આ '=== ' કરતાં પણ વધુ સચોટ ચેકિંગ કરે છે, તે NaN === NaN ને પણ સાચું (true) સાબિત કરી શકે છે.
- Unary Plus Tricking (+):** એમટી એરે [] ની આગળ માત્ર '+' મુકવાથી તે ઓટોમેટિકલી નંબર 0 માં કન્વર્ટ થઈ જાય છે.
- Double Dot Notation (35..toString()):** કોઈ પણ આખા નંબર પર સીધી મેથડ વાપરવા માટે ડબલ ડોટ મુકવામાં આવે છે જેથી બ્રાઉઝર કન્ફ્યુઝ ન થાય.

૪. મોડર્ન ESNext અને છુપા ફીચર્સ

- Temporal API:** જૂના અને અંધરા Date() ઓબ્જેક્ટને રિપ્લેસ કરતું નવું ઓફિશિયલ અને ટાઇમ-ઝોન ફ્રેન્ડલી API.
- Explicit Resource Management (using keyword):** કોઈ પણ ઓબ્જેક્ટ કે કનેક્શન જ્યારે સ્કોપની બહાર જાય, ત્યારે તેની મેમરી ઓટોમેટિક ક્લીન કરવા માટેનો નવો કીવર્ડ.
- Math.sumPrecise():** જાવાસ્ક્રિપ્ટમાં પોઇન્ટવાળા નંબરોની ગણતરીમાં થતી જૂની ભૂલો (0.1 + 0.2) ને સોલ્વ કરતી નવી મેથડ.
- 6 Falsy Values:** જાવાસ્ક્રિપ્ટની ૬ મુખ્ય વેલ્યુ જે કન્ડિશનમાં હંમેશા ખોટી સાબિત થાય છે: false, 0, "", null, undefined, અને NaN.

[illegible]

- **Inline Caching (IC):** **Stores the return value of a function call directly in the instruction stream, bypassing the return register.** This technique is used for frequently called functions, reducing the overhead of stack frame management and return value passing.

- Hidden Classes (Shapes):

• Variable Shadowing: `let` allows you to declare a new variable with the same name as an existing one, effectively "shadowing" the previous variable. This is useful for reusing names for different values in different scopes.

- ### • Object.freeze() vs Object.seal(): Freeze

- **Dead Code Elimination (Tree Shaking):** 

આ ફાઇલમાં Callback Hell, BOM અને બ્રાકીના બધા જ એડવાન્સ કન્સેપ્ટ્સને એકદમ સરળ અને સ્પષ્ટ ગુજરાતી ભાષામાં સમજાવવામાં આવ્યા છે.

- **Callback Hell (Pyramid of Doom):** જ્યારે આપણે એક ફંક્શનની અંદર બીજા ફંક્શનને બોલાવવા માટે ઘણા વાંચના નેસ્ટેડ કોલબેક્સ વાપરે છે, ત્યારે કોડ ખૂબ અસરકારક બની શકે છે. આનાથી બચવા માટે, આપણે પ્રોમિસિસ અને એસિન્ક-એવેર આપવા.
- **Inversion of Control (IoC):** કોલબેક ફંક્શનની એક મોટી ખામી. જ્યારે આપણે આપણો કોડ કોડ નીચે પાટી (Third-party) ના ફંક્શનની અંદર બોલાવવા દેએ છીએ, ત્યારે આપણો આપણા જ કોડ પાટી નીચે રહેતો રહે છે. ફંક્શન સાચું કામ કરશે કે નહીં.
- **Microtasks Queue vs Macrotask Queue:** જ્યારે સ્ક્રિપ્ટમાં બેકગ્રાઉન્ડ પ્રોસેસ માટે એ અલગ વાઇઠ ના હોય છે. પ્રોમિસિસના જવાબી (Microtask) ને setImmediate અને છેવેન્ટ્સ (Macrotask) કરતા પહેલાં રન થવાનો મોકો મળે છે.
- **Starvation:** જો Microtask વાઇઠ સતત ખરોતી રહે, તો Macrotask વાઇઠને રન થવાનો મોકો જ નથી મળતો. આ પરિસ્થિતિને સ્ટારવેશન (ખુમખોર) કહે છે.

- **Window Object:** બ્રાઉઝરનો સૌથી મોટો અને મુખ્ય ગ્લોબલ ઓબ્જેક્ટ જેની અંદર આખી જાવાસ્ક્રિપ્ટ રન થાય છે.
- **Navigator Object:** બ્રાઉઝર અને યુઝરના ડિવાઇસની માહિતી આપે છે જેમ કે યુઝર ઓનલાઇન છે કે ઓફલાઇન, તેનું લોકેશન વગેરે.
- **Location Object:** કન્ટેનર વેબ પેજનું URL જાણવા અથવા પેજ બદલીને બીજા વેબ પેજ પર જવા માટે વપરાય છે.
- **History Object:** યુઝરે અગાઉ વેબ પેજ વિઝિટ કર્યા છે તેની માહિતી રાખે છે, જેથી આગળ પાછળ (Back/Forward) જઈ શકાય.

- **Implicit Type Coercion:** જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડમાં આપમેળે ડેટા ટાઇપ બદલી નામે છે. જેમ કે '5' + '5' કરવાથી '55' થતાં પણ '5' + '5' કરવાથી '0' થઈ જાય છે.
- **Object.is():** આ '==' કરતાં પણ વધુ પાયાની ચકાસ કરે છે. જાવાસ્ક્રિપ્ટમાં સામાન્ય રીતે 'NaN' == NaN' કરવાથી 'false' આવે છે, પણ 'Object.is(NaN, NaN)' કરવાથી તે 'true' આવે છે.
- **Unary Plus/Packing (+[]):** કોઈ ખાલી એરે [] ની આગળ માત્ર પ્લસ '+' મુકવાથી તે ઓબ્જેક્ટિવક બની જાય નહીં માં બદલાઈ જાય છે.

૪. મોડર્ન અને લેટેસ્ટ ફીચર્સ (Modern ESNext)

- **Temporal API:** જૂના અને અધરા 'Date()' ઓબ્જેક્ટને બદલે ટાઇમ-ઝોન અને તારીખ મેનેજ કરવા માટેનું નવું સરવોટ API.
- **using keyword:** મેમરી ક્લિનિંગ માટેનો નવો કોર્ડર્ડ, જ્યારે કોઈ ઓબ્જેક્ટ કામ પૂરું કરીને ક્લોપની બહાર જાય, ત્યારે તેની મેમરી ઓટોમેટિક પાલી બાઈ જાય છે.
- **Falsy values:** જ્યારે ફિસ્ટરમાઈ ફિક્સ ડ વેલ્યુ એક છે જેને કન્ડિશનમાં મુકતા તે કેંમેશરા ખોટી (false) જે સાંબિંધ થાય છે: false, 0, "" (પાલી રિસ્ટિંગ), null, undefined, અને NaN.

JavaScript Complete A to Z Master List

1. Callback Hell & Async Flow (■■■■■■■■■■ ■■■■■■)

- Callback Hell (Pyramid of Doom):

* Inversion of Control (IoC):  (Third-party) 

• Microtask Queue vs Macrotask Queue: 

• **Starvation:**  Microtask Queue  Macrotask Queue 
Queue () setTimeout ()
 () .

2. Complete BOM (Browser Object Model)

```
• Window Object: [object Window]
  window
```

```
* Navigator Object:
```

- **Location Object:** `URL` (`location.href`).

```
• History Object: history object has two methods: back() and forward().  
back() method goes back to previous page.  
forward() method goes forward to next page.
```

```

• User Agent: [REDACTED] (Windows, Mac, Android) [REDACTED]
(Chrome, Safari) [REDACTED].

```

3. JavaScript Quirks & Advanced Logics

- **Implicit Type Coercion** (જાણીતું): `5 + '5'` નો પરિણામ `'55'` (String) છે. `5 - '5'` નો પરિણામ `0` છે.
- **Object.is()**: `===` (Strict Equality) કરતાં વધુ ચોક્કસ. `Object.is(NaN, NaN)` `true` છે, જ્યારે `NaN === NaN` `false` છે.
- **Unary Plus Tricking (+[])**: `+[]` નો પરિણામ `0` છે.
- **Double Dot Notation (35..toString())**: `35..toString()` નો પરિણામ `'35'` છે.
- **Temporal API**: `Date()` નો પરિણામ `Invalid Date` છે.
- **Explicit Resource Management (using keyword)**: `using` કીવર્ડનો ઉપયોગ.
- **Falsy Values**: `0`, `''`, `undefined`, `null`, `NaN`.

JavaScript A to Z Mega Master List - Part 6 (The Ultimate Finale)

BOM, Callback Hell, DevTools, Framework Fundamentals, and Missing Edge Cases

1. Callback Hell & Asynchronous Control Flows

- **Callback Hell (Pyramid of Doom)**: જ્યારે એક અસિંક્રોનસ ઓપરેશનની અંદર બીજું, તેની અંદર ત્રીજું, એમ ઘણા બધા નેસ્ટેડ કોલબેક્સ (Nested Callbacks) વખવામાં આવે ત્યારે કોડનો આકાર પિરામિડ જેવો થઈ જાય છે અને તેને વાંચવો કે ડીબગ કરવો મુશ્કેલ બને છે. આ સમસ્યાને ઉકેલવા પ્રોમિસિસ અને એસિંક-અવેટ આવ્યા.
- **Inversion of Control (IoC)**: કોલબેક ફંક્શનની એક મોટી ખામી. જ્યારે તમે તમારું ફંક્શન કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને આપો છો, ત્યારે તે ક્યારે, કેટલી વાર, કે કઈ રીતે રન થશે તેનો કંટ્રોલ તમારા હાથમાં નથી રહેતો.
- **Microtask Queue vs Macrotask Queue**: ઇવેન્ટ લૂપમાં પ્રોમિસિસના રિઝોલ્યુશન (`.then/catch/await`) માઇક્રોટાસ્ક ક્યૂમાં જાય છે, જ્યારે `setTimeout`, `setInterval` અને DOM ઇવેન્ટ્સ મેક્રોટાસ્ક ક્યૂમાં જાય છે. માઇક્રોટાસ્કને મેક્રોટાસ્ક કરતાં હંમેશા પહેલી પ્રાયોરિટી મળે છે.
- **Starvation (સ્ટાર્વેશન)**: જ્યારે માઇક્રોટાસ્ક ક્યૂમાં સતત નવા ટાસ્ક ઉમેરાતા જ રહે, ત્યારે મેક્રોટાસ્ક ક્યૂ (જેમ કે `setTimeout`) ને રન થવાનો મોકો જ નથી મળતો, જેને ટાસ્ક્નું સ્ટાર્વેશન કહે છે.

2. Complete BOM (Browser Object Model) & Environment Essentials

- **Window Object**: બ્રાઉઝરનો સૌથી મોટો ગ્લોબલ ઓબ્જેક્ટ જે આખી વિન્ડોનું પ્રતિનિધિત્વ કરે છે. જાવાસ્ક્રિપ્ટનો ગ્લોબલ સ્કોપ આની અંદર જ રન થાય છે.
- **Navigator Object**: બ્રાઉઝર અને યુઝરના ડિવાઇસ વિશેની માહિતી આપે છે, જેમ કે યુઝર કયું બ્રાઉઝર વાપરે છે, ઓનલાઇન છે કે ઓફલાઇન, અને તેની જીઓલોકેશન (Location) શું છે.
- **Screen Object**: યુઝરના કમ્પ્યુટર કે મોબાઇલ સ્ક્રીનની સાઈઝ, રિઝોલ્યુશન, કલર ડેપ્થ અને અવેલેબલ હાઇટ/વિડ્થની વિગતો આપે છે.
- **History Object**: યુઝરે બ્રાઉઝર ટેબમાં મુલાકાત લીધેલા પેજ્ઞનો હિસ્ટ્રી ટ્રેક રાખે છે. આના દ્વારા `history.back()`, `history.forward()`, `history.go()`, `history.pushState()`, `history.replaceState()` વગેરે મેથોડ્સનો ઉપયોગ થાય છે.
- **Location Object**: અત્યારના કરન્ટ વેબ પેજનું URL મેળવવા અને તેને બદલવા (નવા પેજ પર રીડાયરેક્ટ કરવા) માટે વપરાય છે, જેમ કે `location.href`, `location.protocol`, `location.reload()`, `location.assign()`, `location.replace()`, `location.state`.
- **User Agent (યુઝર એજન્ટ)**: બ્રાઉઝર દ્વારા સર્વરને મોકલવામાં આવતી એક ખાસ આઈડેન્ટિટી સ્ટ્રિંગ, જેનાથી સર્વર જાણી શકે છે કે રિફ્રેશ કયા ઓપરેટિંગ સિસ્ટમ અને બ્રાઉઝરમાંથી આવી છે.

3. DevTools, Debugging & Runtime Mechanisms

- Source Maps (સોર્સ મેપ્સ):** જ્યારે પ્રોડક્શન કોડને મિનિફાઇ કે કમ્પાઇલ (Babel/TypeScript દ્વારા) કરવામાં આવે, ત્યારે બ્રાઉઝરમાં ઓરિજિનલ કોડ જોઈને ડીબગ કરવા માટે જે મેપિંગ ફાઇલ વપરાય છે તેને સોર્સ મેપ કહેવાય છે.
- Memory Heap (મેમરી હીપ):** જાવાસ્ક્રિપ્ટ એન્જિનનો એ મોટો અનસ્ટ્રક્ચર્ડ ભાગ જ્યાં બધા ઓબ્જેક્ટ્સ, એરે અને વેરીએબલ્સ માટે વાસ્તવિક મેમરી એલોકેટ (સ્ટોર) થાય છે.
- Call Stack (કોલ સ્ટેક):** જ્યારે ફંક્શન્સ રન થાય ત્યારે તે કયા ઓર્ડરમાં એકિઝક્યુટ થઈ રહ્યા છે તેનો ટ્રેક રાખતી LIFO (Last In, First Out) ડેટા સ્ટ્રક્ચર.
- Stack Overflow:** જ્યારે કોઈ ફંક્શન (ખાસ કરીને રિકર્સિવ ફંક્શન) ક્યારેય અટકે નહીં અને અનંતકાળ સુધી કોલ સ્ટેકમાં ઉમેરાતું જ જાય, ત્યારે કોલ સ્ટેકની મેમરી લિમિટ પૂરી થઈ જાય છે અને બ્રાઉઝર 'Maximum call stack size exceeded' એરર આપે છે.
- Profiling (પ્રોફાઇલિંગ):** બ્રાઉઝર ડેવટૂલ્સના પર્ફોર્મન્સ ટેબ દ્વારા કોડની સ્પીડ, મેમરી યુસેજ અને કયું ફંક્શન રન થવામાં વધુ સમય લે છે તે ચેક કરવાની પ્રક્રિયા.

4. Framework-Level Fundamentals (Under-the-Hood JavaScript)

- Virtual DOM vs Real DOM:** રિયલ HTML DOM ને વારંવાર અપડેટ કરવું સ્વી છે. તેથી React જેવા ફ્રેમવર્ક્સ મેમરીમાં DOM ની એક લાઇટવેઈટ નકલ રાખે છે જેને વર્ચ્યુઅલ DOM કહે છે. ફેરફાર થાય ત્યારે પહેલાં વર્ચ્યુઅલ DOM અપડેટ થાય છે અને પછી માત્ર જરૂરી ફેરફારો જ રિયલ DOM માં મોકલાય છે (Reconciliation).
- Reconciliation (રીકોન્સિલેશન):** વર્ચ્યુઅલ DOM માં ફેરફાર થયા પછી, જૂના અને નવા વર્ચ્યુઅલ DOM વચ્ચે ડિફરન્સ શોધીને રિયલ DOM ને ન્યૂનતમ સ્ટેસમાં અપડેટ કરવાની પ્રક્રિયા.
- Diffing Algorithm:** રીકોન્સિલેશન દરમિયાન બે વર્ચ્યુઅલ DOM ટ્રીની સરખામણી કરવા માટે ફ્રેમવર્ક્સ જે અલ્ગોરિથમ વાપરે છે તેને ડિફિંગ અલ્ગોરિથમ કહેવાય છે.
- State Management:** એપ્લિકેશનના ડેટા (State) ને આખા પ્રોજેક્ટના અલગ-અલગ કમ્પોનન્ટ્સ વચ્ચે સેન્ટ્રલિઝી મેનેજ અને શેર કરવાની સિસ્ટમ (જેમ કે Redux, Context API).
- Client-Side Rendering (CSR) vs Server-Side Rendering (SSR):** CSR માં આખું પેજ બ્રાઉઝરમાં જાવાસ્ક્રિપ્ટ દ્વારા રેન્ડર થાય છે, જ્યારે SSR માં સર્વર પરથી જ બનેલું HTML પેજ બ્રાઉઝરને મોકલવામાં આવે છે જેથી લોડિંગ ફાસ્ટ થાય.

5. Crucial Missing Edge Cases & Advanced Concepts

- Eval() Function:** એક ઇન-બિલ્ટ ફંક્શન જે સ્ક્રિપ્ટ તરીકે લખેલા જાવાસ્ક્રિપ્ટ કોડને રન કરી શકે છે. સુરક્ષા અને પર્ફોર્મન્સના કારણોસર આનો ઉપયોગ ક્યારેય ન કરવો જોઈએ (Eval is evil).
- Strict Equality ('===') vs Loose Equality ('=='):** '==' માત્ર વેલ્યુ ચેક કરે છે અને ટાઇપ ઓટોમેટિક કન્વર્ટ કરે છે (Type Coercion), જ્યારે '===' વેલ્યુ અને ડેટા ટાઇપ બંને સરખા છે કે નહીં તે કડકાઈથી ચેક કરે છે.
- NaN (Not-a-Number):** આ એક ખાસ ગ્લોબલ પ્રોપર્ટી છે જે દર્શાવે છે કે કોઈ ગાણિતિક ઓપરેશનનું પરિણામ વેબિડ નંબર નથી (જેમ કે "hello" * 2). આનો ડેટા ટાઇપ 'number' હોય છે.

- NaN() vs Number.isNaN():** 'isNaN()' ઇનપુટને નંબરમાં કન્વર્ટ કરીને ચેક કરે છે (જેથી તે સ્ક્રિપ્ટને પણ સાચું માની લે છે), જ્યારે 'Number.isNaN()' કોઈ કન્વર્ઝન વગર માત્ર ત્યારે જ સાચું આપે છે જો વેલ્યુ ખરેખર અવેબિડ નંબર (NaN) હોય.
- Falsy Values:** જાવાસ્ક્રિપ્ટમાં કુલ 6 વેલ્યુઝ કન્ડિશનમાં બાય-ડિફોલ્ટ 'false' ગણાય છે: 'false', '0', '"" (ખાલી સ્ટ્રિંગ), 'null', 'undefined', અને 'NaN'.

JavaScript A to Z Mega Master List - Part 5

(Bonus & Specialized Architecture Terms)

આ ફાઇનલ બોનસ વિસ્તરમાં JavaScript ના બાકી રહેલા સ્પેશિયલાઇઝ્ડ આર્કિટેક્ચર, ડિઝાઇન પેટર્ન્સ, કમ્પાઇલર લેવલ પ્રોસેસિંગ અને મોડર્ન રનટાઇમ ટર્સનો સમાવેશ કરવામાં આવ્યો છે.

• **Design Patterns (ડિઝાઇન પેટર્ન્સ):** કોડિંગમાં વારંવાર આવતી સમસ્યાઓના ઉકેલ માટેનું એક સ્ટાન્ડર્ડ માળખું. જેમ કે Singleton Pattern (આખા પ્રોજેક્ટમાં ઓબ્જેક્ટની એક જ ઇન્સ્ટેન્સ રાખવી) અને Factory Pattern.

• **Module Pattern (મોડ્યુલ પેટર્ન):** કોડને પ્રાઇવેટ અને પબ્લિક ભાગોમાં વહેંચવા માટે વાપરવામાં આવતી ક્લોઝર્સ (Closures) નો ઉપયોગ કરીને આ અસીવ કરવામાં આવે છે.

• **Observer Pattern (ઓબ્ઝર્વર પેટર્ન):** જ્યારે કોઈ એક ઓબ્જેક્ટમાં ફેરફાર થાય, ત્યારે તેના પર નજર રાખતી (Subscribe કરેલા) અન્ય તમામ ઓબ્જેક્ટ્સને ઓટોમેટિક જાણ થઈ જાય તેવી સિસ્ટમ. (Event Frameworks માં ખૂબ વપરાય છે).

• **Publish-Subscribe (Pub/Sub):** ઓબ્ઝર્વર પેટર્ન જેવું જ, પણ આમાં પબ્લિશર અને સબ્સ્ક્રાઇબર વચ્ચે એક ત્રીજો મધ્યસ્થી (Message Broker) હોય છે, જેથી કોડ એકબીજા પર સીધો ડિપેન્ડન્ટ નથી રહેતો.

• **Abstract Syntax Tree (AST):** જ્યારે બ્રાઉઝર કે કમ્પાઇલર તમારા JavaScript કોડને રીડ કરે છે, ત્યારે તેને સમજવા માટે કોડનું જે ટ્રી-સ્ટ્રક્ચર (Tree Structure) બનાવે છે તેને AST કહેવાય છે.

• **Dead Code Elimination (ડેડ કોડ એલિમિનેશન):** પ્રોજેક્ટ બિલ્ડ થતી વખતે એવો કોડ કે ફંક્શન્સ જે ક્યાંય વપરાયા જ નથી, તેને ઓટોમેટિક દૂર કરવાની પ્રક્રિયા જેથી ફાઇલ સાઇઝ નાની થાય (યાને Tree Shaking પણ કહે છે).

• **Bitwise Operators (બિટવાઇઝ ઓપરેટર્સ):** નંબર્સ પર ડાયરેક્ટ બાઈનરી લેવલ (0 અને 1 ના સ્તરે) ઓપરેશન કરવા માટેના ઓપરેટર્સ જેમ કે AND (&), OR (|), NOT (~), XOR (^). આ ખુબ જ ફાસ્ટ કેલ્ક્યુલેશન માટે વપરાય છે.

• **Proxy & Reflect API:** ES6 ના શક્તિશાળી ફીચર્સ. Proxy દ્વારા તમે કોઈ ઓબ્જેક્ટની ડિફોલ્ટ બિહેવિયર (જેમ કે પ્રોપર્ટી ઍક્સેસ કે સેટ કરવી) ને કસ્ટમાઇઝ કે મોનિટર કરી શકો છો. Reflect તેના ડિફોલ્ટ ઓપરેશન પૂરા પાડે છે.

• **Internationalization API (Intl):** JavaScript ની ઇન-બિલ્ટ ઓબ્જેક્ટ જે એપ્લિકેશનને અલગ-અલગ રેશીની ભાષા, કરન્સી ફોર્મેટ, સમય અને તારીખના ફોર્મેટ પ્રમાણે ઓટોમેટિક લોકલાઈઝ (Localize) કરવા માટે મદદ કરે છે.

• **TypedArrays (ટાઇપ્ડ એરેઝ):** સામાન્ય JavaScript એરે કરતાં અલગ, જે મેમરીમાં ફિક્સ સાઇઝના અને ચોક્કસ ડેટા ટાઇપના બાઈનરી ડેટા હેન્ડલ કરે છે. (જેમ કે Int8Array, Float64Array). વેબ ગ્રાફિક્સ (WebGL) માં વધુ વપરાય છે.

• **BigInt (બિગ-ઇન્ટ):** JavaScript નો એક ખાસ ડેટા ટાઇપ જે સામાન્ય Number ટાઇપની મર્યાદા (Safe Integer MAX_SAFE_INTEGER) કરતાં પણ મોટા આંકડાઓ પર સચોટ ગણતરી કરવા દે છે.

• **Custom Events (કસ્ટમ ઇવેન્ટ્સ):** JavaScript માં પહેલાથી બનેલી ઇવેન્ટ્સ (જેમ કે click, change) સિવાય ડેવલપર પોતાની જરૂરિયાત મુજબ કોઈ પણ નામની નવી કસ્ટમ ઇવેન્ટ ક્રિએટ કરીને તેને ટ્રિગર કરી શકે છે ('new CustomEvent()').

• **Shadow DOM:** વેબ કમ્પોનન્ટ્સનો એક ભાગ જે મેઈન HTML DOM ની અંદર એક સિક્કોર અને સેપરેટ DOM ટ્રી બનાવે છે. આના સીએસએસ (CSS) સ્તરો બહારના પેજને નડતી નથી (Encapsulation).

• **Hydration (હાઇડ્રેશન):** Server-Side Rendering (SSR) માં વપરાતી કન્સેપ્ટ. સર્વર પરથી આવેલી સાદી HTML ફાઇલમાં JavaScript લોડ થઈને જ્યારે પેજને ઇન્ટરેક્ટિવ (ક્રિવેકેબલ) બનાવે છે, તે પ્રોસેસને હાઇડ્રેશન કહે છે.

• **Hot Module Replacement (HMR):** ડેવલપમેન્ટ દરમિયાન જ્યારે તમે કોડ ફાઇલમાં ફેરફાર કરો, ત્યારે આખું વેબ પેજ રીફ્રેશ કર્યા વગર માત્ર જે-તે બદલાયેલા મોડ્યુલને જ લોડ અપડેટ કરવાની આધુનિક ટેકનિક.

• **Polyfill (પોલીફીલ):** જ્યારે કોઈ નવું JavaScript ફીચર જૂના બ્રાઉઝરમાં સપોર્ટેડ ન કરતું હોય, ત્યારે તે ફીચરને જૂના બ્રાઉઝરમાં ચલાવવા માટે જે કસ્ટમ બેકઅપ કોડ કે સ્ક્રીપ્ટ લખવામાં આવે છે, તેને પોલીફીલ કહેવાય.

• **Transpiler / Babel:** તમારા મોડર્ન JavaScript કોડ (ES6+) ને એવા જૂના JavaScript કોડ (ES5) માં કન્વર્ટ કરતું ટૂલ, જેથી તે જૂના બ્રાઉઝર્સમાં પણ કોઈ ભૂલ વગર ચાલી શકે.

JavaScript A to Z Mega Master List

Part 4: Web APIs, Security, Optimization & Functional Concepts (Gujarati)

અહીં JavaScript Master List નો ચોથો અને અંતિમ ભાગ છે, જેમાં Web APIs, Security, Advanced Optimization, Web Workers અને Functional Programming ના બાકી રહેલા તમામ અતિ મહત્વના કન્સેપ્ટ્સને સરળ ગુજરાતીમાં આવરી લેવામાં આવ્યા છે.

બાકી રહેલા તમામ એડવાન્સ કન્સેપ્ટ્સ અને ટર્મ્સ (A to Z Last Part)

• **Web Workers:** JavaScript સિંગલ-થ્રેડેડ છે, પણ જો કોઈ બહુ મોટું કે ભારે કેલ્ક્યુલેશન કરવું હોય, તો તે મેઈન UI (બ્રાઉઝર સ્ક્રિન) ને અટકાવ્યા વગર બેકગ્રાઉન્ડ થ્રેડમાં રન કરવા માટે Web Workers નો ઉપયોગ થાય છે.

• **Service Workers:** આ એવા ખાસ સ્ક્રિપ્ટ્સ છે જે બ્રાઉઝરના બેકગ્રાઉન્ડમાં રન થાય છે. તે નેટવર્ક રિસ્પોન્સને કેચ (Cache) કરીને વેબસાઇટને ઓફલાઇન મોડમાં પણ ચલાવવામાં (PWA - Progressive Web Apps) મદદ કરે છે.

• **CORS (Cross-Origin Resource Sharing):** બ્રાઉઝરની એક સિફોરિટી સિસ્ટમ છે. જ્યારે તમે એક ડોમેન (જેમ કે abc.com) પરથી બીજા ડોમેન (xyz.com) ની API એક્સેસ કરવાની કોશિશ કરો, ત્યારે સર્વર આ સિફોરિટી ચેક કરે છે.

• **XSS (Cross-Site Scripting):** વેબસાઇટ પર થતો એક પ્રકારનો હેકિંગ એટેક, જેમાં હેક્સ યુઝરના ઇનપુટ બોક્સ દ્વારા વેબસાઇટની અંદર માલિશિયસ (નુકસાનકારક) JavaScript કોડ ઇન્જેક્ટ કરી દે છે.

• **CSRF (Cross-Site Request Forgery):** એક સિફોરિટી એટેક જેમાં હેકર યુઝરની જાણ વગર તેના ઓથેન્ટિકેટેડ બ્રાઉઝર સેશનનો ઉપયોગ કરીને બેકગ્રાઉન્ડમાં અનધિકૃત ટ્રાન્ઝેક્શન કે રિવેસ્ટ મોકલી દે છે.

• **Local Storage vs Session Storage vs Cookies:** બ્રાઉઝરમાં ડેટા સ્ટોર કરવા માટેની ત્રણ અલગ રીતો:
• Local Storage: ડેટા કાયમ માટે સેવ રહે છે (બ્રાઉઝર બંધ કરો તો પણ).
• Session Storage: ડેટા માત્ર ટેબ ઓપન હોય ત્યાં સુધી જ રહે છે.
• Cookies: નાનો ડેટા જે એક્સપાયરી ડેટ સાથે સ્ટોર થાય છે અને સર્વર સાથે આપમેળે ટ્રાન્સફર થાય છે.

• **Event Delegation:** ધરેક યાઇલ્ડ એલિમેન્ટ પર અલગ-અલગ ઇવેન્ટ લિસનર લગાવવાને બદલે, તેના કોમન પેરેન્ટ (Parent Element) પર એક જ ઇવેન્ટ લિસનર લગાવીને મેનેજ કરવાની સ્માર્ટ પદ્ધતિ.

• **Intersection Observer API:** બ્રાઉઝરની એક આધુનિક વેબ API, જે આપણને જણાવે છે કે વેબપેજનો કયો ભાગ (Element) અત્યારે યુઝરની સ્ક્રીન પર દેખાઈ રહ્યો છે (વિઝિબલ છે). આ Lazy Loading અને અનંત સ્ક્રોલિંગ (Infinite Scroll) માટે બેસ્ટ છે.

• **Polyfill vs Transpiler (Babel):** • Transpiler (Babel): મોડર્ન JavaScript કોડ (ES6+) ને જુના બ્રાઉઝર સમજી શકે તેવા જુના સિન્ટેક્સ (ES5) માં કન્વર્ટ કરે છે.

• Polyfill: જો કોઈ નવું ફંક્શન કે મેથડ (જેમ કે Array.prototype.includes) જુના બ્રાઉઝરમાં ઉપલબ્ધ ન હોય, તો તેને ચલાવવા માટે બેકગ્રાઉન્ડમાં લખાતો કસ્ટમ કોડ.

• **Tree Shaking:** વેબસાઇટના પ્રોડક્શન બિલ્ડ વખતે, કોડમાંથી એવા અનયુઝ્ડ (વપરાયા વગરના) કોડ કે ફંક્શન્સને આપમેળે હટાવી દેવાની પ્રક્રિયા જેથી ફાઇલ સાઇઝ નાની થાય (મોટેભાગે Webpack કે Rollup આ કરે છે).

• **Pure Functions:** એવા ફંક્શન્સ જે હંમેશા સેમ ઇનપુટ આપવા પર સેમ જ આઉટપુટ આપે છે, અને તે ફંક્શનની બહારના કોઈ પણ વેરીએબલ કે ડેટાને બદલતાં નથી (No Side Effects).

• **Immutability:** એવો કન્સેપ્ટ જેમાં ડેટા કે ઓબ્જેક્ટને એકવાર બનાવ્યા પછી ક્યારેય સીધો મોડિફાઇ (બદલી) નથી કરાતો, પણ જ્યારે પણ ચેન્જ કરવો હોય ત્યારે નવો જ ઓબ્જેક્ટ ક્રિએટ કરવામાં આવે છે.

• **Currying:** એક એડવાન્સ ફંક્શનલ પ્રોગ્રામિંગ ટેકનિક જેમાં મલ્ટિપલ આર્ગ્યુમેન્ટ્સ વાળા એક ફંક્શનને એવા ફંક્શન્સની શ્રેણીમાં બદલવામાં આવે છે જે એક સમયે માત્ર એક જ આર્ગ્યુમેન્ટ લે છે.

• **Composition:** નાના-નાના અને સિંગલ-પર્પઝ ફંક્શન્સને એકસાથે જોડીને એક મોટું અને જટિલ લોજિક બનાવવાની પ્રક્રિયા.

• **Memory Leaks:** જ્યારે આપણો પ્રોગ્રામ મેમરી (RAM) નો ઉપયોગ કરે છે પણ કામ પૂરું થઈ ગયા પછી પણ તે મેમરી ફ્રી (રીલીઝ) નથી કરતો, જેનાથી વેબસાઇટ ધીમી થઈ જાય છે.

• **Garbage Collection:** JavaScript એન્જિનનું એક ઇન-બિલ્ટ ઓટોમેટિક મેનેજમેન્ટ જે મેમરી ચેક કરતું રહે છે અને જે વેરીએબલ્સ કે ઓબ્જેક્ટ્સ હવે કોડમાં ક્યાંય વપરાતા નથી, તેને મેમરીમાંથી ડીલીટ કરી નાખે છે.

નીચે હવે આ ભાગ સાથે JavaScript ના તમામ મહત્વના કન્સેપ્ટ્સ, એન્જિન આર્કિટેક્ચર, વેબ APIs અને સિફોરિટી ટર્મ્સ સંપૂર્ણપણે પૂર્ણ થઈ ગયા છે.

JavaScript A to Z Mega Master List - Part 4

Coercion (ટાઈપ કોઅર્સન)

JavaScript જ્યારે કોઈ એક ડેટા ટાઈપને આપમેળે (implicitly) બીજા ડેટા ટાઈપમાં બદલી નાખે છે (જેમ કે 5 + '5' કરવાથી '55' થઈ જાય), તેને Coercion કહેવાય છે.

Event Delegation (ઇવેન્ટ ડેલેગેશન)

દરેક ચાઇલ્ડ એલિમેન્ટ પર અલગ-અલગ ઇવેન્ટ વિસનર મૂકવાને બદલે, તેમના પેરેન્ટ એલિમેન્ટ પર એક જ ઇવેન્ટ વિસનર મૂકીને બધી ઇવેન્ટ્સ હેન્ડલ કરવાની ટેકનિક.

Strict Equality vs Loose Equality (=== vs ==)

== માત્ર વેલ્યુ ચેક કરે છે અને ટાઈપ બદલી નાખે છે, જ્યારે === વેલ્યુ અને ડેટા ટાઈપ (Data Type) બંને સરખા છે કે નહીં તે કડક રીતે ચેક કરે છે.

NaN (Not-a-Number)

આ એક એવી સ્પેશિયલ વેલ્યુ છે જે દર્શાવે છે કે કોઈ મેથેમેટિકલ ઓપરેશનનું પરિણામ સાચો નંબર નથી (જેમ કે 'hello' * 5 કરવાથી NaN મળે).

Infinity & -Infinity

JavaScript માં ગણતરીની મર્યાદા બહારના અતિશય મોટા પોઝિટિવ કે નેગેટિવ નંબરને દર્શાવવા માટે વપરાતી સ્પેશિયલ ન્યુમેરિક વેલ્યુઝ.

IIFE Scope Isolation

IIFE નો ઉપયોગ ગ્લોબલ સ્કોપને પ્રદૂષિત (pollute) થતી અટકાવવા અને વેરીએબલ્સને પ્રાઇવેટ રાખવા માટે થાય છે.

Pass by Value vs Pass by Reference

બેઝિક ડેટા (Primitives) ની વેલ્યુ કોપી થાય છે (Pass by Value), જ્યારે ઓબ્જેક્ટ અને એરેનો મેમરી એડ્રેસ/રેફરન્સ શેર થાય છે (Pass by Reference).

Memory Leaks

જ્યારે કોડમાં એવા ઓબ્જેક્ટ્સ રહી જાય જે હવે વપરાતી નથી પણ મેમરીમાંથી ડીલીટ પણ નથી થતી, જેનાથી એપ્લિકેશન સ્લો થઈ જાય છે.

Microtask Queue vs Macrotask Queue

Event Loop માં Promises ની કોલબેક Microtask Queue માં જાય છે (જે વહેવા રન થાય છે), જ્યારે setTimeout ની કોલબેક Macrotask Queue માં જાય છે.

BigInt

JavaScript નો એક નવો ડેટા ટાઈપ જે સામાન્ય Number ટાઈપની લિમિટ કરતાં પણ અતિશય મોટા પૂર્ણાંક નંબરો (Integers) ને સચોટ રીતે સ્ટોર કરવા માટે વપરાય છે.

Symbol

એક અનોખો (Unique) અને બદલી ન શકાય તેવો (Immutable) ડેટા ટાઈપ, જેનો ઉપયોગ ઓબ્જેક્ટની અંદર છુપી કે પ્રાઇવેટ પ્રોપર્ટી કી બનાવવા માટે થાય છે.

Proxy & Reflect

Proxy ઓબ્જેક્ટના કસ્ટમ બિહેવિયર (જેમ કે પ્રોપર્ટી લુકઅપ, અસાઇનમેન્ટ) ને ઇન્ટરસેપ્ટ અને મોડિફાય કરવા માટે વપરાય છે, અને Reflect તેના ડિફોલ્ટ ઓપરેશન્સને સરળ બનાવે છે.

JavaScript A to Z Mega Master List - Part 3

- **Global Scope:** કોડના કોઈપણ ભાગમાંથી એક્સેસ કરી શકાય તેવો સ્કોપ. ગ્લોબલ એરિયામાં ડિફેલ્ટ કરેલા વેરીએબલ્સ ગમે ત્યાં વાપરી શકાય છે.
- **Function Scope:** var કીવર્ડથી બનેલા વેરીએબલ્સ જે માત્ર તે ફંક્શનની અંદર જ એક્સેસ થઈ શકે છે, બહાર નહીં.
- **Lexical Environment:** કોઈ પણ કોડ જ્યાં ડિઝાઇન થયેલો છે તેની આસપાસનું વાતાવરણ, જે નક્કી કરે છે કે કયા વેરીએબલ્સ ઉપલબ્ધ હશે.
- **Garbage Collection:** એન્જિનની એવી પ્રોસેસ જે મેમરીમાં બિનજરૂરી કે વપરાયા વગરના ઓબ્જેક્ટ્સને આપમેળે ડીલીટ કરીને સ્પેસ ખાલી કરે છે.
- **JIT Compilation:** JavaScript એન્જિન કોડ રન થતી વખતે જ તેને મશીન કોડમાં કમ્પાઇલ કરે છે જેથી સ્પીડ ફાસ્ટ મળે.
- **Currying:** એક એવી કન્સેપ્ટ જેમાં મલ્ટીપલ આર્ગ્યુમેન્ટ્સવાળા ફંક્શનને એક-એક આર્ગ્યુમેન્ટ લેતા ફંક્શન્સની સિરીઝમાં ફેરવવામાં આવે છે.
- **Debouncing:** વારંવાર ટ્રિગર થતી ઇવેન્ટ્સ (જેમ કે ટાઇપિંગ કે સ્ક્રોલિંગ) ને અમુક સમય સુધી રોકી રાખીને છેલ્લે એક જ વાર એક્ઝિક્યુટ કરવાની ટેકનિક.
- **Recursion:** જ્યારે કોઈ ફંક્શન પોતાની જાતને જ ફરીથી કોલ કરે તેને રિકર્ઝન કહેવાય.
- **Optional Chaining (?.):** જો કોઈ ઓબ્જેક્ટ કે પ્રોપર્ટી null કે undefined હોય, તો એરર આપવાને બદલે કોડને ત્યાં જ અટકાવી undefined રિટર્ન કરવાની સેફ રીત.
- **Deep Copy vs Shallow Copy:** Shallow Copy માત્ર ઉપરછલ્લો રેફરન્સ કોપી કરે છે, જ્યારે Deep Copy એરિજિનલ ઓબ્જેક્ટથી તદ્દન અલગ નવો જ ઓબ્જેક્ટ મેમરીમાં બનાવે છે.
- **Dynamic Typing:** JavaScript માં વેરીએબલ બનાવતી વખતે તેનો ડેટા ટાઇપ નક્કી નથી કરવો પડતો, તે વેલ્યુ મુજબ આપમેળે બદલાય છે.

JavaScript A to Z Mega Master List

તમામ મહત્વના કન્સેપ્ટ્સ અને ટર્મ્સની સરળ ગુજરાતી સમજૂતી

A થી D: બેઝિક અને ફાઉન્ડેશન કન્સેપ્ટ્સ

- **AJAX (Asynchronous JavaScript and XML):** વેબ પેજને રીફ્રેશ કર્યા વગર સર્વર પરથી ડેટા લાવવા કે મોકલવા માટે આ ટેકનિક વપરાય છે.
- **Array (એરે):** એક જ વેરીએબલમાં એક કરતાં વધુ વેલ્યુ (જેમ કે નામો કે નંબરોનું લિસ્ટ) સ્ટોર કરવા માટેનું કલેક્શન.
- **Async / Await:** અસિંક્રોનસ કોડ (જેમ કે API માંથી ડેટા લાવવો) ને સીધો અને સરળ રીતે લખવાની આધુનિક રીત.
- **BOM (Browser Object Model):** બ્રાઉઝર સાથે વાતચીત કરવા માટેની વિન્ડો (જેમ કે screen size જાણવી કે pop-up બતાવવું).
- **Closure (ક્લોઝર):** જ્યારે કોઈ ફંક્શન તેના બહારના ફંક્શનના વેરીએબલ્સને યાદ રાખે (તે બહારનું ફંક્શન પૂરું થઈ ગયું હોય તો પણ), તેને ક્લોઝર કહેવાય.
- **DOM (Document Object Model):** HTML પેજનું એક ટ્રી-સ્ટ્રક્ચર, જેના દ્વારા JavaScript પેજના કોઈ પણ એલિમેન્ટ (ટેક્સ્ટ, કલર વગેરે) ને બદલી શકે છે.
- **Destructuring (ડીસ્ટ્રક્ચરિંગ):** એરે અથવા ઓબ્જેક્ટમાંથી વેલ્યુઝને ખૂબ જ ટૂંકી રીતે અલગ-અલગ વેરીએબલ્સમાં બહાર કાઢવાની રીત.

E થી H: એડિઝક્યુશન અને ફ્લો કંટ્રોલ

- **Event Loop (ઇવેન્ટ લૂપ):** JavaScript સિંગલ-થ્રેડ્ડ (એક સમયે એક જ કામ કરતી) હોવા છતાં બેકગ્રાઉન્ડ ટાસ્ક્ (જેમ કે setTimeout) ને મેનેજ કરીને કોડને અટકવા નથી દેતી, તે આ લૂપના કારણે છે.
- **ES Modules (import / export):** કોડને અલગ-અલગ ફાઇલોમાં વહેંચીને જરૂર મુજબ import કે export કરવાની મોડર્ન સિસ્ટમ.
- **Execution Context (એક્ઝિક્યુશન કોન્ટેક્સ્ટ):** JavaScript કોડ કઈ રીતે રન થઈ રહ્યો છે અને અત્યારે કયા વેરીએબલ્સ ઉપલબ્ધ છે, તેનું આખું પર્યાવરણ (Environment).
- **Generator Functions (function*):** આ એવા ખાસ ફંક્શન્સ છે જેને અપવચ્ચેથી રોકી શકાય છે (yield) અને પછી ત્યાંથી જ ફરી શરૂ કરી શકાય છે.
- **Hoisting (હોઇસ્ટિંગ):** JavaScript કોડ રન થાય તે પહેલાં વેરીએબલ અને ફંક્શનના ડિફિનીશનને સ્કોપમાં સીધી ઉપર વર્ગ જાય છે (જેના લીધે ક્યારેક ડિફેલ્ટ કર્યા પહેલાં પણ વેરીએબલ વાપરી શકાય છે).

I થી N: ડેટા ટેન્ડબિંગ અને મેમરી

- **IIFE (Immediately Invoked Function Expression):** એવું ફંક્શન જે બનતાની સાથે જ તરત જ ઓટોમેટિક રન થઈ જાય છે.
- **Immutability (ઇમ્યુટેબિલિટી):** ડેટા જે એકવાર બન્યા પછી બદલી શકાતો નથી જેમ કે JavaScript માં સ્ટ્રિંગ્સ ઇમ્યુટેબલ હોય છે).
- **JSON (JavaScript Object Notation):** સર્વર અને ક્લાયન્ટ વચ્ચે ડેટા ટ્રાન્સફર કરવા માટે વપરાતું સૌથી પોપ્યુલર લાઇટવેઇટ ટેક્સ્ટ ફોર્મેટ.
- **Map Object:** કી-વેલ્યુ (Key-Value) ની જોડી સ્ટોર કરવા માટે (ઓબ્જેક્ટ જેવું પણ વધુ પાવરફુલ).
- **Set Object:** માત્ર યુનિક (Unique) વેલ્યુઝ સ્ટોર કરવા માટે. જેમાં ડુપ્લિકેટ વેલ્યુ ન આવી શકે.
- **Nullish Coalescing Operator (??):** જો ડાબી બાજુની વેલ્યુ null કે undefined હોય, તો જ જમણી બાજુની ડિફોલ્ટ વેલ્યુ પ્રિન્ટ કરે છે.

O થી T: એડવાન્સ લોજિક અને સ્કોપિંગ

- **Promise (પ્રોમિસ):** અસિંક્રોનસ ઓપરેશન ભવિષ્યમાં પૂરું થશે (સફળ કે અસફળ) તેની ખાતરી આપતું ઓબ્જેક્ટ.
- **Prototypal Inheritance:** JavaScript માં એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટીઝ અને મેથડ્સ વારસામાં (inherit) મેળવે છે, તેને પ્રોટોટાઇપ કહેવાય છે.
- **Rest / Spread Operator (...):** Spread: એરે કે ઓબ્જેક્ટને ખોલીને તેના એલિમેન્ટ્સ છૂટા કરવા. Rest: છૂટાછવાયા એલિમેન્ટ્સ ભેગા કરીને એક એરે બનાવવો.
- **Strict Mode ("use strict"):** કોડમાં નાની ભૂલો પકડવા અને સિક્યોરિટી વધારવા માટે બ્રાઉઝરને કડક નિયમો લાગુ પાડવાની સૂચના.
- **Temporal Dead Zone (TDZ):** let અને const વેરીએબલ જ્યાં સુધી કોડમાં એક્ઝ્યુઅલ ડિફાઇન ન થાય, ત્યાં સુધી તેને એક્સેસ ન કરી શકાય તે વચ્ચેના સમયગાળા/ઝોનને TDZ કહે છે.

U થી Z: ઓપરેટર્સ અને ઝલોબલ ઓબ્જેક્ટ્સ

- **Unary Operators:** એવા ઓપરેટર્સ જે માત્ર એક જ વેલ્યુ (operand) પર કામ કરે છે (જેમ કે typeof, delete, કે ++).
- **WeakMap / WeakSet:** આ મેમરી-લીક બચાવવા માટેના ખાસ કલેક્શન્સ છે. જેમાં રાખેલા ઓબ્જેક્ટ્સ જો ક્યાંય વપરાતા ન હોય તો Garbage Collector તેને મેમરીમાંથી ઓટોમેટિક ડીલીટ કરી નાખે છે.
- **Window Object:** બ્રાઉઝરનો સૌથી મુખ્ય ઝલોબલ ઓબ્જેક્ટ, જેની અંદર આખી JavaScript રન થાય છે.
- **Zero-based Indexing:** JavaScript માં કોઈ પણ વિસ્ટ (Array) ની ગણતરી 1 થી નહીં પણ હંમેશા 0 થી શરૂ થાય છે.

JavaScript A to Z Mega Master List (Part 2 - Remaining Terms)

અહીં JavaScript ના બાકી રહેલા તમામ મહત્વના કન્સેપ્ટ્સ અને કીવર્ડ્સની સંપૂર્ણ યાદી સરળ ગુજરાતી સમજૂતી સાથે આપવામાં આવી છે.

1. Fundamental Concepts & Key Terms

- **Abstraction (એબ્સ્ટ્રેક્શન):** જટિલ આંતરિક લોજિકને છુપાવીને યુઝરને માત્ર જરૂરી ફીચર્સ જ બતાવવાની પદ્ધતિ.
- **Argument vs Parameter:** Parameter એ ફંક્શન બનાવતી વખતે વ્યાખ્યાયિત કરેલા વેરીએબલ્સ છે, અને Argument એ ફંક્શન કોલ કરતી વખતે પાસ કરેલી વાસ્તવિક વેલ્યુઝ છે.
- **Arrow Functions (‘() => {}’):** ટૂંકી અને સરળ રીતે ફંક્શન લખવાની મોડર્ન રીત, જે પોતાનું ‘this’ બાઈન્ડિંગ ધરાવતી નથી.
- **Block Scope:** ‘let’ અને ‘const’ દ્વારા બનાવેલા વેરીએબલ્સ જે માત્ર છગડીયા કૌંસ ‘{}’ ની અંદર જ એક્સેસ થઈ શકે છે.
- **Callback Function:** એવું ફંક્શન જેને બીજા ફંક્શનની અંદર આયુમિન્ટ તરીકે પાસ કરવામાં આવે છે અને કોઈ ટાસ્ક પૂરો થયા પછી રન કરાય છે.
- **Call Stack:** JavaScript એન્જિન કયા ક્રમમાં ફંક્શન્સ રન કરવા તે ટ્રેક કરવા માટે વાપરતું આંતરિક ડેટા સ્ટ્રક્ચર.
- **Classes (ES6):** ઓબ્જેક્ટ ઓરિએન્ટેડ પ્રોગ્રામિંગ (OOP) માટે બ્લુપ્રિન્ટ બનાવવાની મોડર્ન રીત.
- **Coercion (ટાઈપ કોઅર્સન):** JavaScript દ્વારા એક ડેટા ટાઈપને ઓટોમેટિકલી બીજા ડેટા ટાઈપમાં બદલવાની પ્રક્રિયા (જેમ કે ‘5 + ‘5’ = ‘55’).
- **Currying:** એક ફંક્શન જેમાં એકસાથે બધી આયુમિન્ટ આપવાને બદલે એક પછી એક આયુમિન્ટ આપતા ફંક્શન્સ રીટર્ન થાય છે.
- **Debouncing:** અમુક ચોક્કસ સમય સુધી કોઈ ઇવેન્ટ (જેમ કે ટાઈપિંગ કે સ્ક્રોલિંગ) અટકે નહીં ત્યાં સુધી ફંક્શન એક્ઝિક્યુટ ન થવા દેવાની ટેકનિક.

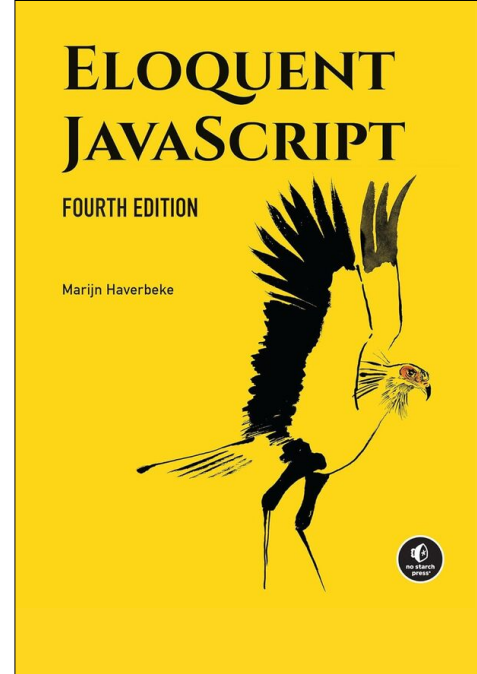
2. Object, Function & Array Methods

- **Encapsulation (એન્કેપ્સ્યુલેશન):** ડેટા (પ્રોપર્ટીઝ) અને કોડ (મેથડ્સ) ને એક જ યુનિટ કે ક્લાસમાં લપેટીને સિક્યોર રાખવાની પદ્ધતિ.
- **Event Bubbling:** જ્યારે કોઈ એલિમેન્ટ પર ઇવેન્ટ ટ્રિગર થાય ત્યારે તે પેરન્ટથી લઈને સૌથી ઉપરના HTML ડોક્યુમેન્ટ સુધી પહોંચે છે.
- **Event Capturing (Trickling):** ઇવેન્ટ બબલિંગથી બિલકુલ ઉલટું, જેમાં ઇવેન્ટ સૌથી બહારના પેરન્ટથી શરૂ થઈને ટાર્ગેટ એલિમેન્ટ સુધી નીચે આવે છે.
- **First-Class Functions:** JavaScript માં ફંક્શન્સ ઓબ્જેક્ટની જેમ ટ્રીટ થાય છે, એટલે કે તેને વેરીએબલમાં સ્ટોર કરી શકાય અને બીજા ફંક્શનમાં પણ પાસ કરી શકાય.

- **Higher-Order Functions:** એવા ફંક્શન જે બીજા ફંક્શનને આર્ગ્યુમેન્ટ તરીકે સ્વીકારે છે અથવા નવું ફંક્શન રીટર્ન કરે છે (જેમ કે 'map', 'filter').
- **Lexical Scoping:** કોઈ ફંક્શન તેનો સ્કોપ તે ક્યાં ડિફાઇન થયું છે તેના આધારે નક્કી કરે છે, નહીં કે તે ક્યાંથી કોલ થયું છે.
- **Memoization:** ભારે ગણતરીવાળા ફંક્શનના આઉટપુટને કેશ (Cache) માં સ્ટોર કરી રાખવાની પદ્ધતિ જેથી કોડ ફાસ્ટ રન થાય.
- **Method Chaining:** એક પછી એક મેથડ્સને સળંગ ડોટ ('.') વગાવીને કોલ કરવાની રીત (જેમ કે 'arr.filter().map().reduce()').
- **Mutability:** ડેટા ઓબ્જેક્ટ્સ કે એરે જે બનાવ્યા પછી બદલી શકાય છે.

3. Advanced Asynchronous & Engine Concepts

- **Polyfill:** બ્રાઉઝર જો કોઈ નવું મોડર્ન JavaScript ફીચર સપોર્ટ ન કરતું હોય, તો તેને યથાવવા માટે વખાતી કસ્ટમ કોડ કે બેકઅપ સ્ક્રિપ્ટ.
- **Polymorphism (પોલીમોર્ફિઝમ):** એક જ નામનું ફંક્શન કે મેથડ અવગ-અવગ ઓબ્જેક્ટ્સમાં જુદો જુદો રીતે વર્તે તેને પોલીમોર્ફિઝમ કહે છે.
- **Prototype Chain:** જ્યારે JavaScript કોઈ પ્રોપર્ટી શોધે છે ત્યારે તે ઓબ્જેક્ટથી શરૂ કરીને છેક ગ્લોબલ 'Object.prototype' સુધી ચેક કરે છે.
- **Pure Functions:** એવું ફંક્શન જે સરખા ઇનપુટ પર દર વખતે સરખું જ આઉટપુટ આપે અને બહારના કોઈ વેરીએબલને બદલે નહીં (No Side Effects).
- **Shadowing:** જ્યારે અંદરના સ્કોપનો વેરીએબલ બહારના સ્કોપના સરખા નામના વેરીએબલને ઓવરરાઈડ (હાંકી) દે છે.
- **Single-Threaded:** JavaScript એક સમયે મેઈન લૂપમાં માત્ર એક જ કમાન્ડ એક્ઝિક્યુટ કરી શકે છે.
- **Throttling:** કોઈ ઇવેન્ટ ગમે તેટલી વાર થાય, પણ તેનું ફંક્શન અમુક ચોક્કસ સમયના અંતરે જ રન થવા દેવાની મર્યાદા સેટ કરવી.
- **Truthy & Falsy Values:** JavaScript માં જે વેલ્યુ બુલિયન કોન્સ્ટ્રક્ટમાં 'true' બને તે Truthy, અને જે 'false' બને (જેમ કે '0', '', 'null', 'undefined') તે Falsy.
- **'this' Keyword:** હાલમાં રન થઈ રહેલા કોડ કયા ઓબ્જેક્ટના કોન્ટેક્સ્ટમાં કામ કરી રહ્યો છે તેનો રેફરન્સ.



ELOQUENT JAVASCRIPT

4TH EDITION

Marijn Haverbeke

Copyright © 2024 by Marijn Haverbeke

This work is licensed under a Creative Commons attribution-noncommercial license (<http://creativecommons.org/licenses/by-nc/3.0/>). All code in the book may also be considered licensed under an MIT license (<https://eloquentjavascript.net/code/LICENSE>).

The illustrations are contributed by various artists: Cover by Péchane Sumi-e. Chapter illustrations by Madalina Tantareanu. Pixel art in Chapters 7 and 16 by Antonio Perdomo Pastor. Regular expression diagrams in Chapter 9 generated with regexper.com by Jeff Avallone. Game concept for Chapter 16 by Thomas Palef.

You can buy a print version of this book, with an extra bonus chapter included, printed by No Starch Press at <http://a-fwd.com/com=marijhaver-20&asin-com=1593279507>.

CONTENTS

INTRODUCTION **1**

On programming 2

Why language matters 4

What is JavaScript? 8

Code, and what to do with it 11

Overview of this book 12

Typographic conventions 13

I VALUES, TYPES, AND OPERATORS **15**

Values 16

Numbers 17

Strings 20

Unary operators 23

Boolean values 24

Empty values 27

Automatic type conversion 27

Summary 31

2 PROGRAM STRUCTURE **32**

Expressions and statements 32

Bindings 34

Binding names 36

The environment 37

Functions 38

The console.log function 39

Return values 39

Control flow 40

Conditional execution 41

while and do loops 43

Indenting Code 46

for loops 47

Breaking Out of a Loop 49

Updating bindings succinctly 50

Dispatching on a value with switch 50

Capitalization 52

Comments 53

Summary 54

Exercises 54

3 FUNCTIONS **58**

Defining a function 59

Bindings and scopes 60

Nested scope 62

Functions as values 63

Declaration notation 64

Arrow functions	65
The call stack	66
Optional Arguments	68
Closure	70
Recursion	72
Growing functions	76
Functions and side effects	80
Summary	81
Exercises	82
4 DATA STRUCTURES: OBJECTS AND ARRAYS	84
The weresquirrel	85
Datasets	86
Properties	87
Methods	88
Objects	90
Mutability	94
The lycanthrope's log	96
Computing correlation	99
Array loops	101
The final analysis	102
Further arrayology	105
Strings and their properties	107
Rest parameters	109
The Math object	111
Destructuring	113
Optional property access	115

JSON	116
Summary	117
Exercises	118
5 HIGHER-ORDER FUNCTIONS	122
Abstraction	123
Abstracting repetition	124
Higher-order functions	126
Script dataset	128
Filtering arrays	130
Transforming with map	131
Summarizing with reduce	132
Composability	134
Strings and character codes	136
Recognizing text	139
Summary	141
Exercises	142
6 THE SECRET LIFE OF OBJECTS	144
Abstract Data Types	144
Methods	146
Prototypes	148
Classes	150
Private Properties	153
Overriding derived properties	155
Maps	157
Polymorphism	159

Getters, setters, and statics	161
Symbols	163
The iterator interface	165
Inheritance	168
The instanceof operator	170
Summary	171
Exercises	172
7 PROJECT: A ROBOT	175
Meadowfield	175
The task	177
Persistent data	180
Simulation	182
The mail truck's route	185
Pathfinding	186
Exercises	189
8 BUGS AND ERRORS	191
Language	191
Strict mode	192
Types	194
Testing	196
Debugging	197
Error propagation	200
Exceptions	202
Cleaning up after exceptions	204
Selective catching	207

Assertions	210
Summary	211
Exercises	212
9 REGULAR EXPRESSIONS	214
Creating a regular expression	215
Testing for matches	216
Sets of characters	216
International characters	218
Repeating parts of a pattern	220
Grouping subexpressions	221
Matches and groups	222
The Date class	224
Boundaries and look-ahead	226
Choice patterns	227
The mechanics of matching	228
Backtracking	229
The replace method	232
Greed	234
Dynamically creating RegExp objects	236
The search method	237
The lastIndex property	237
Parsing an INI file	240
Code units and characters	243
Summary	244
Exercises	246

10 MODULES	249
Modular programs	250
ES modules	251
Packages	254
CommonJS modules	256
Building and bundling	261
Module design	262
Summary	265
Exercises	266
11 ASYNCHRONOUS PROGRAMMING	268
Asynchronicity	269
Callbacks	271
Promises	273
Failure	276
Carla	279
Breaking In	280
Async functions	283
Generators	285
A Corvid Art Project	287
The event loop	292
Asynchronous bugs	294
Summary	296
Exercises	297
12 PROJECT: A PROGRAMMING LANGUAGE	300
Parsing	300

The evaluator	307
Special forms	309
The environment	311
Functions	314
Compilation	315
Cheating	316
Exercises	318
13 JAVASCRIPT AND THE BROWSER	321
Networks and the Internet	322
The Web	324
HTML	325
HTML and JavaScript	329
In the sandbox	330
Compatibility and the browser wars	331
14 THE DOCUMENT OBJECT MODEL	334
Document structure	334
Trees	336
The standard	338
Moving through the tree	339
Finding elements	341
Changing the document	342
Creating nodes	343
Attributes	346
Layout	347
Styling	350

Cascading styles	352
Query selectors	354
Positioning and animating	356
Summary	359
Exercises	360
15 HANDLING EVENTS	363
Event handlers	363
Events and DOM nodes	364
Event objects	366
Propagation	367
Default actions	369
Key events	370
Pointer events	372
Scroll events	378
Focus events	380
Load event	381
Events and the event loop	382
Timers	384
Debouncing	385
Summary	387
Exercises	388
16 PROJECT: A PLATFORM GAME	390
The game	391
The technology	392
Levels	393

x

Reading a level	394
Actors	396
Drawing	402
Motion and collision	409
Actor updates	414
Tracking keys	417
Running the game	418
Exercises	421
17 DRAWING ON CANVAS	424
SVG	425
The canvas element	426
Lines and surfaces	428
Paths	429
Curves	432
Drawing a pie chart	435
Text	437
Images	438
Transformation	441
Storing and clearing transformations	445
Back to the game	447
Choosing a graphics interface	455
Summary	457
Exercises	458
18 HTTP AND FORMS	461
The protocol	461

xi

Browsers and HTTP	464
Fetch	467
HTTP sandboxing	469
Appreciating HTTP	470
Security and HTTPS	471
Form fields	472
Focus	475
Disabled fields	477
The form as a whole	477
Text fields	479
Checkboxes and radio buttons	481
Select fields	483
File fields	485
Storing data client-side	487
Summary	491
Exercises	492
19 PROJECT: A PIXEL ART EDITOR	495
Components	496
The state	498
DOM building	501
The canvas	502
The application	505
Drawing tools	509
Saving and loading	513
Undo history	517
Let's draw	519
xii	

Why is this so hard?	521
Exercises	522
20 NODE.JS	526
Background	527
The node command	527
Modules	529
Installing with NPM	531
The filesystem module	534
The HTTP module	537
Streams	539
A file server	541
Summary	549
Exercises	550
21 PROJECT: SKILL-SHARING WEBSITE	552
Design	553
Long polling	555
HTTP interface	556
The server	559
The client	570
Exercises	579
EXERCISE HINTS	581
Program Structure	581
Functions	583
Data Structures: Objects and Arrays	584
xiii	

Higher-Order Functions	587
The Secret Life of Objects	588
Project: A Robot	590
Bugs and Errors	591
Regular Expressions	592
Modules	593
Asynchronous Programming	596
Project: A Programming Language	599
The Document Object Model	601
Handling Events	602
Project: A Platform Game	604
Drawing on Canvas	605
HTTP and Forms	608
Project: A Pixel Art Editor	610
Node.js	614
Project: Skill-Sharing Website	616

*“We think we are creating the system for our own purposes.
We believe we are making it in our own image... But the
computer is not really like us. It is a projection of a very slim
part of ourselves: that portion devoted to logic, order, rule,
and clarity.”*

—Ellen Ullman, Close to the Machine: Technophilia and Its
Discontents

INTRODUCTION

This is a book about instructing computers. Computers are about as common as screwdrivers today, but they are quite a bit more complex, and making them do what you want them to do isn’t always easy.

If the task you have for your computer is a common, well-understood one, such as showing you your email or acting like a calculator, you can open the appropriate application and get to work. But for unique or open-ended tasks, there often is no appropriate application.

That is where programming may come in. *Programming* is the act of constructing a *program*—a set of precise instructions telling a computer what to do. Because computers are dumb, pedantic beasts, programming is fundamentally tedious and frustrating.

Fortunately, if you can get over that fact—and maybe even enjoy the rigor of thinking in terms that dumb machines can deal with—programming can be rewarding. It allows you to do things in seconds that would take *forever* by hand. It is a way to make your computer tool do things that it couldn’t do before. On top of that, it makes for a wonderful game of puzzle solving and abstract thinking.

Most programming is done with programming languages. A *program-*

ming language is an artificially constructed language used to instruct computers. It is interesting that the most effective way we've found to communicate with a computer borrows so heavily from the way we communicate with each other. Like human languages, computer languages allow words and phrases to be combined in new ways, making it possible to express ever new concepts.

At one point, language-based interfaces, such as the BASIC and DOS prompts of the 1980s and 1990s, were the main method of interacting with computers. For routine computer use, these have largely been replaced with visual interfaces, which are easier to learn but offer less freedom. But if you know where to look, the languages are still there. One of them, *JavaScript*, is built into every modern web browser—and is thus available on almost every device.

This book will try to make you familiar enough with this language to do useful and amusing things with it.

ON PROGRAMMING

Besides explaining JavaScript, I will introduce the basic principles of programming. Programming, it turns out, is hard. The fundamental rules are simple and clear, but programs built on top of these rules tend to become complex enough to introduce their own rules and complexity. You're building your own maze, in a way, and you can easily get lost in it.

There will be times when reading this book feels terribly frustrating. If you are new to programming, there will be a lot of new material to

digest. Much of this material will then be *combined* in ways that require you to make additional connections.

It is up to you to make the necessary effort. When you are struggling to follow the book, do not jump to any conclusions about your own capabilities. You are fine—you just need to keep at it. Take a break, reread some material, and make sure you read and understand the example programs and exercises. Learning is hard work, but everything you learn is yours and will make further learning easier.

When action grows unprofitable, gather information; when
information grows unprofitable, sleep.

—Ursula K. Le Guin, *The Left Hand of Darkness*

A program is many things. It is a piece of text typed by a programmer, it is the directing force that makes the computer do what it does, it is data in the computer's memory, and, at the same time, it controls the actions performed on this memory. Analogies that try to compare programs to familiar objects tend to fall short. A superficially fitting one is to compare a program to a machine—lots of separate parts tend to be involved, and to make the whole thing tick, we have to consider the ways in which these parts interconnect and contribute to the operation of the whole.

A computer is a physical machine that acts as a host for these immaterial machines. Computers themselves can do only stupidly straightforward things. The reason they are so useful is that they do these things at an incredibly high speed. A program can ingeniously combine

an enormous number of these simple actions to do very complicated things.

A program is a building of thought. It is costless to build, it is weightless, and it grows easily under our typing hands. But as a program grows, so does its complexity. The skill of programming is the skill of building programs that don't confuse the programmer. The best programs are those that manage to do something interesting while still being easy to understand.

Some programmers believe that this complexity is best managed by using only a small set of well-understood techniques in their programs. They have composed strict rules ("best practices") prescribing the form programs should have and carefully stay within their safe little zone.

This is not only boring—it is ineffective. New problems often require new solutions. The field of programming is young and still developing rapidly, and it is varied enough to have room for wildly different approaches. There are many terrible mistakes to make in program design, and you should go ahead and make them at least once so that you understand them. A sense of what a good program looks like is developed with practice, not learned from a list of rules.

WHY LANGUAGE MATTERS

In the beginning, at the birth of computing, there were no programming languages. Programs looked something like this:

```
00110001 00000000 00000000
00110001 00000001 00000001
```

```
00110011 00000001 00000010
01010001 00001011 00000010
00100010 00000010 00001000
01000011 00000001 00000000
01000001 00000001 00000001
00010000 00000010 00000000
01100010 00000000 00000000
```

This is a program to add the numbers from 1 to 10 together and print the result: $1 + 2 + \dots + 10 = 55$. It could run on a simple hypothetical machine. To program early computers, it was necessary to set large arrays of switches in the right position or punch holes in strips of cardboard and feed them to the computer. You can imagine how tedious and error prone this procedure was. Even writing simple programs required much cleverness and discipline. Complex ones were nearly inconceivable.

Of course, manually entering these arcane patterns of bits (the ones and zeros) did give the programmer a profound sense of being a mighty wizard. And that has to be worth something in terms of job satisfaction.

Each line of the previous program contains a single instruction. It could be written in English like this:

1. Store the number 0 in memory location 0.
2. Store the number 1 in memory location 1.
3. Store the value of memory location 1 in memory location 2.
4. Subtract the number 11 from the value in memory location 2.

5. If the value in memory location 2 is the number 0, continue with instruction 9.
6. Add the value of memory location 1 to memory location 0.
7. Add the number 1 to the value of memory location 1.
8. Continue with instruction 3.
9. Output the value of memory location 0.

Although that is already more readable than the soup of bits, it is still rather obscure. Using names instead of numbers for the instructions and memory locations helps.

```
Set "total" to 0.
Set "count" to 1.
[loop]
Set "compare" to "count".
Subtract 11 from "compare".
If "compare" is 0, continue at [end].
Add "count" to "total".
Add 1 to "count".
Continue at [loop].
[end]
Output "total".
```

Can you see how the program works at this point? The first two lines give two memory locations their starting values: `total` will be used to build up the result of the computation, and `count` will keep track of the number that we are currently looking at. The lines using `compare` are

probably the most confusing ones. The program wants to see whether `count` is equal to 11 to decide whether it can stop running. Because our hypothetical machine is rather primitive, it can test only whether a number is zero and make a decision based on that. It therefore uses the memory location labeled `compare` to compute the value of `count - 11` and makes a decision based on that value. The next two lines add the value of `count` to the result and increment `count` by 1 every time the program decides that `count` is not 11 yet.

Here is the same program in JavaScript:

```
let total = 0, count = 1;
while (count <= 10) {
  total += count;
  count += 1;
}
console.log(total);
// → 55
```

This version gives us a few more improvements. Most importantly, there is no need to specify the way we want the program to jump back and forth anymore—the `while` construct takes care of that. It continues executing the block (wrapped in braces) below it as long as the condition it was given holds. That condition is `count <= 10`, which means “the count is less than or equal to 10”. We no longer have to create a temporary value and compare that to zero, which was just an uninteresting detail. Part of the power of programming languages is that they can take care of uninteresting details for us.

At the end of the program, after the `while` construct has finished, the

`console.log` operation is used to write out the result.

Finally, here is what the program could look like if we happened to have the convenient operations `range` and `sum` available, which respectively create a collection of numbers within a range and compute the sum of a collection of numbers:

```
console.log(sum(range(1, 10)));  
// → 55
```

The moral of this story is that the same program can be expressed in both long and short, unreadable and readable ways. The first version of the program was extremely obscure, whereas this last one is almost English: `log` the `sum` of the `range` of numbers from 1 to 10. (We will see in later chapters how to define operations like `sum` and `range`.)

A good programming language helps the programmer by allowing them to talk about the actions that the computer has to perform on a higher level. It helps omit details, provides convenient building blocks (such as `while` and `console.log`), allows you to define your own building blocks (such as `sum` and `range`), and makes those blocks easy to compose.

WHAT IS JAVASCRIPT?

JavaScript was introduced in 1995 as a way to add programs to web pages in the Netscape Navigator browser. The language has since been adopted by all other major graphical web browsers. It has made modern web applications possible—that is, applications with which you can interact directly without doing a page reload for every action. JavaScript

is also used in more traditional websites to provide various forms of interactivity and cleverness.

It is important to note that JavaScript has almost nothing to do with the programming language named Java. The similar name was inspired by marketing considerations rather than good judgment. When JavaScript was being introduced, the Java language was being heavily marketed and was gaining popularity. Someone thought it was a good idea to try to ride along on this success. Now we are stuck with the name.

After its adoption outside of Netscape, a standard document was written to describe the way the JavaScript language should work so that the various pieces of software that claimed to support JavaScript could make sure they actually provided the same language. This is called the ECMAScript standard, after the Ecma International organization that conducted the standardization. In practice, the terms ECMAScript and JavaScript can be used interchangeably—they are two names for the same language.

There are those who will say *terrible* things about JavaScript. Many of these things are true. When I was required to write something in JavaScript for the first time, I quickly came to despise it. It would accept almost anything I typed but interpret it in a way that was completely different from what I meant. This had a lot to do with the fact that I did not have a clue what I was doing, of course, but there is a real issue here: JavaScript is ridiculously liberal in what it allows. The idea behind this design was that it would make programming in JavaScript easier for beginners. In actuality, it mostly makes finding problems in

your programs harder because the system will not point them out to you.

This flexibility also has its advantages, though. It leaves room for techniques that are impossible in more rigid languages and makes for a pleasant, informal style of programming. After learning the language properly and working with it for a while, I have come to actually *like* JavaScript.

There have been several versions of JavaScript. ECMAScript version 3 was the widely supported version during JavaScript's ascent to dominance, roughly between 2000 and 2010. During this time, work was underway on an ambitious version 4, which planned a number of radical improvements and extensions to the language. Changing a living, widely used language in such a radical way turned out to be politically difficult, and work on version 4 was abandoned in 2008. A much less ambitious version 5, which made only some uncontroversial improvements, came out in 2009. In 2015, version 6 came out, a major update that included some of the ideas planned for version 4. Since then we've had new, small updates every year.

The fact that JavaScript is evolving means that browsers have to constantly keep up. If you're using an older browser, it may not support every feature. The language designers are careful to not make any changes that could break existing programs, so new browsers can still run old programs. In this book, I'm using the 2024 version of JavaScript.

Web browsers are not the only platforms on which JavaScript is used. Some databases, such as MongoDB and CouchDB, use JavaScript as

their scripting and query language. Several platforms for desktop and server programming, most notably the Node.js project (the subject of Chapter 20), provide an environment for programming JavaScript outside of the browser.

CODE, AND WHAT TO DO WITH IT

Code is the text that makes up programs. Most chapters in this book contain quite a lot of code. I believe reading code and writing code are indispensable parts of learning to program. Try to not just glance over the examples—read them attentively and understand them. This may be slow and confusing at first, but I promise that you'll quickly get the hang of it. The same goes for the exercises. Don't assume you understand them until you've actually written a working solution.

I recommend you try your solutions to exercises in an actual JavaScript interpreter. That way, you'll get immediate feedback on whether what you are doing is working, and, I hope, you'll be tempted to experiment and go beyond the exercises.

The easiest way to run the example code in the book—and to experiment with it—is to look it up in the online version of the book at <https://eloquentjavascript.net>. There, you can click any code example to edit and run it and to see the output it produces. To work on the exercises, go to <https://eloquentjavascript.net/code>, which provides starting code for each coding exercise and allows you to look at the solutions.

Running the programs defined in this book outside of the book's

website requires some care. Many examples stand on their own and should work in any JavaScript environment. But code in later chapters is often written for a specific environment (the browser or Node.js) and can run only there. In addition, many chapters define bigger programs, and the pieces of code that appear in them depend on each other or on external files. The sandbox on the website provides links to ZIP files containing all the scripts and data files necessary to run the code for a given chapter.

OVERVIEW OF THIS BOOK

This book contains roughly three parts. The first 12 chapters discuss the JavaScript language. The next seven chapters are about web browsers and the way JavaScript is used to program them. Finally, two chapters are devoted to Node.js, another environment to program JavaScript in. There are five *project chapters* in the book that describe larger example programs to give you a taste of actual programming.

The language part of the book starts with four chapters that introduce the basic structure of the JavaScript language. They discuss control structures (such as the `while` word you saw in this introduction), functions (writing your own building blocks), and data structures. After these, you will be able to write basic programs. Next, Chapters 5 and 6 introduce techniques to use functions and objects to write more *abstract* code and keep complexity under control.

After a first project chapter that builds a crude delivery robot, the language part of the book continues with chapters on error handling

and bug fixing, regular expressions (an important tool for working with text), modularity (another defense against complexity), and asynchronous programming (dealing with events that take time). The second project chapter, where we implement a programming language, concludes the first part of the book.

The second part of the book, Chapters 13 to 19, describes the tools that browser JavaScript has access to. You'll learn to display things on the screen (Chapters 14 and 17), respond to user input (Chapter 15), and communicate over the network (Chapter 18). There are again two project chapters in this part: building a platform game and a pixel paint program.

Chapter 20 describes Node.js, and Chapter 21 builds a small website using that tool.

TYPOGRAPHIC CONVENTIONS

In this book, text written in a `monospaced` font will represent elements of programs. Sometimes these are self-sufficient fragments, and sometimes they just refer to part of a nearby program. Programs (of which you have already seen a few) are written as follows:

```
function factorial(n) {  
  if (n == 0) {  
    return 1;  
  } else {  
    return factorial(n - 1) * n;  
  }  
}
```

```
}  
  
Sometimes, to show the output that a program produces, the expected  
output is written after it, with two slashes and an arrow in front.  
  
console.log(factorial(8));  
// → 40320  
Good luck!
```

14

*“Below the surface of the machine, the program moves.
Without effort, it expands and contracts. In great harmony,
electrons scatter and regroup. The forms on the monitor are
but ripples on the water. The essence stays invisibly below.”*
—Master Yuan-Ma, The Book of Programming

CHAPTER I
VALUES, TYPES, AND OPERATORS

In the computer’s world, there is only data. You can read data, modify data, create new data—but that which isn’t data cannot be mentioned. All this data is stored as long sequences of bits and is thus fundamentally alike.

Bits are any kind of two-valued things, usually described as zeros and ones. Inside the computer, they take forms such as a high or low electrical charge, a strong or weak signal, or a shiny or dull spot on the surface of a CD. Any piece of discrete information can be reduced to a sequence of zeros and ones and thus represented in bits.

For example, we can express the number 13 in bits. This works the same way as a decimal number, but instead of 10 different digits, we have only 2, and the weight of each increases by a factor of 2 from right to left. Here are the bits that make up the number 13, with the weights of the digits shown below them:

0	0	0	0	1	1	0	1
128	64	32	16	8	4	2	1

That’s the binary number 00001101. Its nonzero digits stand for 8, 4,

15

and 1, and add up to 13.

VALUES

Imagine a sea of bits—an ocean of them. A typical modern computer has more than 100 billion bits in its volatile data storage (working memory). Nonvolatile storage (the hard disk or equivalent) tends to have yet a few orders of magnitude more.

To be able to work with such quantities of bits without getting lost, we separate them into chunks that represent pieces of information. In a JavaScript environment, those chunks are called *values*. Though all values are made of bits, they play different roles. Every value has a type that determines its role. Some values are numbers, some values are pieces of text, some values are functions, and so on.

To create a value, you must merely invoke its name. This is convenient. You don't have to gather building material for your values or pay for them. You just call for one, and *whoosh*, you have it. Of course, values are not really created from thin air. Each one has to be stored somewhere, and if you want to use a gigantic number of them at the same time, you might run out of computer memory. Fortunately, this is a problem only if you need them all simultaneously. As soon as you no longer use a value, it will dissipate, leaving behind its bits to be recycled as building material for the next generation of values.

The remainder of this chapter introduces the atomic elements of JavaScript programs, that is, the simple value types and the operators that can act on such values.

NUMBERS

Values of the *number* type are, unsurprisingly, numeric values. In a JavaScript program, they are written as follows:

13

Using that in a program will cause the bit pattern for the number 13 to come into existence inside the computer's memory.

JavaScript uses a fixed number of bits, 64 of them, to store a single number value. There are only so many patterns you can make with 64 bits, which limits the number of different numbers that can be represented. With N decimal digits, you can represent 10^N numbers. Similarly, given 64 binary digits, you can represent 2^{64} different numbers, which is about 18 quintillion (an 18 with 18 zeros after it). That's a lot.

Computer memory used to be much smaller, and people tended to use groups of 8 or 16 bits to represent their numbers. It was easy to accidentally *overflow* such small numbers—to end up with a number that did not fit into the given number of bits. Today, even computers that fit in your pocket have plenty of memory, so you are free to use 64-bit chunks, and you need to worry about overflow only when dealing with truly astronomical numbers.

Not all whole numbers less than 18 quintillion fit in a JavaScript number, though. Those bits also store negative numbers, so one bit indicates the sign of the number. A bigger issue is representing nonwhole numbers. To do this, some of the bits are used to store the position of the decimal point. The actual maximum whole number that can be

stored is more in the range of 9 quadrillion (15 zeros)—which is still pleasantly huge.

Fractional numbers are written using a dot:

9.81

For very big or very small numbers, you may also use scientific notation by adding an *e* (for *exponent*), followed by the exponent of the number.

2.998e8

That's $2.998 \times 10^8 = 299,800,000$.

Calculations with whole numbers (also called *integers*) that are smaller than the aforementioned 9 quadrillion are guaranteed to always be precise. Unfortunately, calculations with fractional numbers are generally not. Just as π (pi) cannot be precisely expressed by a finite number of decimal digits, many numbers lose some precision when only 64 bits are available to store them. This is a shame, but it causes practical problems only in specific situations. The important thing is to be aware of it and treat fractional digital numbers as approximations, not as precise values.

ARITHMETIC

The main thing to do with numbers is arithmetic. Arithmetic operations such as addition or multiplication take two number values and produce a new number from them. Here is what they look like in JavaScript:

100 + 4 * 11

The + and * symbols are called *operators*. The first stands for addition and the second stands for multiplication. Putting an operator between two values will apply it to those values and produce a new value.

Does this example mean “Add 4 and 100, and multiply the result by 11”, or is the multiplication done before the adding? As you might have guessed, the multiplication happens first. As in mathematics, you can change this by wrapping the addition in parentheses.

(100 + 4) * 11

For subtraction, there is the - operator. Division can be done with the / operator.

When operators appear together without parentheses, the order in which they are applied is determined by the *precedence* of the operators. The example shows that multiplication comes before addition. The / operator has the same precedence as *. Likewise, + and - have the same precedence. When multiple operators with the same precedence appear next to each other, as in $1 - 2 + 1$, they are applied left to right: $(1 - 2) + 1$.

Don't worry too much about these precedence rules. When in doubt, just add parentheses.

There is one more arithmetic operator, which you might not immediately recognize. The % symbol is used to represent the *remainder* operation. $x \% y$ is the remainder of dividing x by y . For example, $314 \% 100$ produces 14, and $144 \% 12$ gives 0. The remainder operator's precedence is the same as that of multiplication and division. You'll

also often see this operator referred to as *modulo*.

SPECIAL NUMBERS

There are three special values in JavaScript that are considered numbers but don't behave like normal numbers. The first two are `Infinity` and `-Infinity`, which represent the positive and negative infinities. `Infinity - 1` is still `Infinity`, and so on. Don't put too much trust in infinity-based computation, though. It isn't mathematically sound, and it will quickly lead to the next special number: `NaN`.

`NaN` stands for "not a number", even though it *is* a value of the number type. You'll get this result when you, for example, try to calculate `0 / 0` (zero divided by zero), `Infinity - Infinity`, or any number of other numeric operations that don't yield a meaningful result.

STRINGS

The next basic data type is the *string*. Strings are used to represent text. They are written by enclosing their content in quotes.

```
'Down on the sea'
"Lie on the ocean"
'Float on the ocean'
```

You can use single quotes, double quotes, or backticks to mark strings, as long as the quotes at the start and the end of the string match.

You can put almost anything between quotes to have JavaScript make a string value out of it. But a few characters are more difficult. You

can imagine how putting quotes between quotes might be hard, since they will look like the end of the string. *Newlines* (the characters you get when you press ENTER) can be included only when the string is quoted with backticks (```).

To make it possible to include such characters in a string, the following notation is used: a backslash (`\`) inside quoted text indicates that the character after it has a special meaning. This is called *escaping* the character. A quote that is preceded by a backslash will not end the string but be part of it. When an `n` character occurs after a backslash, it is interpreted as a newline. Similarly, a `t` after a backslash means a tab character. Take the following string:

```
"This is the first line\nAnd this is the second"
```

This is the actual text in that string:

```
This is the first line
And this is the second
```

There are, of course, situations where you want a backslash in a string to be just a backslash, not a special code. If two backslashes follow each other, they will collapse together, and only one will be left in the resulting string value. This is how the string "A *newline character* is written like "`\n`." can be expressed:

```
"A newline character is written like \"\\n\"."
```

Strings, too, have to be modeled as a series of bits to be able to exist inside the computer. The way JavaScript does this is based on the

Unicode standard. This standard assigns a number to virtually every character you would ever need, including characters from Greek, Arabic, Japanese, Armenian, and so on. If we have a number for every character, a string can be described by a sequence of numbers. And that's what JavaScript does.

There's a complication though: JavaScript's representation uses 16 bits per string element, which can describe up to 2^{16} different characters. However, Unicode defines more characters than that—about twice as many, at this point. So some characters, such as many emoji, take up two “character positions” in JavaScript strings. We'll come back to this in Chapter 5.

Strings cannot be divided, multiplied, or subtracted. The `+` operator *can* be used on them, not to add, but to *concatenate*—to glue two strings together. The following line will produce the string “concatenate”:

```
"con" + "cat" + "e" + "nate"
```

String values have a number of associated functions (*methods*) that can be used to perform other operations on them. I'll say more about these in Chapter 4.

Strings written with single or double quotes behave very much the same—the only difference lies in which type of quote you need to escape inside of them. Backtick-quoted strings, usually called *template literals*, can do a few more tricks. Apart from being able to span lines, they can also embed other values.

```
`half of 100 is ${100 / 2}`
```

When you write something inside `${}` in a template literal, its result will be computed, converted to a string, and included at that position. This example produces the string “half of 100 is 50”.

UNARY OPERATORS

Not all operators are symbols. Some are written as words. One example is the `typeof` operator, which produces a string value naming the type of the value you give it.

```
console.log(typeof 4.5)
// → number
console.log(typeof "x")
// → string
```

We will use `console.log` in example code to indicate that we want to see the result of evaluating something. (More about that in the [next chapter](#).)

The other operators shown so far in this chapter all operated on two values, but `typeof` takes only one. Operators that use two values are called *binary* operators, while those that take one are called *unary* operators. The minus operator (`-`) can be used both as a binary operator and as a unary operator.

```
console.log(- (10 - 2))
// → -8
```

BOOLEAN VALUES

It is often useful to have a value that distinguishes between only two possibilities, like “yes” and “no” or “on” and “off”. For this purpose, JavaScript has a *Boolean* type, which has just two values, true and false, written as those words.

COMPARISON

Here is one way to produce Boolean values:

```
console.log(3 > 2)
// → true
console.log(3 < 2)
// → false
```

The > and < signs are the traditional symbols for “is greater than” and “is less than”, respectively. They are binary operators. Applying them results in a Boolean value that indicates whether they hold true in this case.

Strings can be compared in the same way.

```
console.log("Aardvark" < "Zoroaster")
// → true
```

The way strings are ordered is roughly alphabetic but not really what you’d expect to see in a dictionary: uppercase letters are always “less” than lowercase ones, so “Z” < “a”, and nonalphabetic characters (!, -, and so on) are also included in the ordering. When comparing strings,

JavaScript goes over the characters from left to right, comparing the Unicode codes one by one.

Other similar operators are >= (greater than or equal to), <= (less than or equal to), == (equal to), and != (not equal to).

```
console.log("Garnet" != "Ruby")
// → true
console.log("Pearl" == "Amethyst")
// → false
```

There is only one value in JavaScript that is not equal to itself, and that is NaN (“not a number”).

```
console.log(NaN == NaN)
// → false
```

NaN is supposed to denote the result of a nonsensical computation, and as such, it isn’t equal to the result of any *other* nonsensical computations.

LOGICAL OPERATORS

There are also some operations that can be applied to Boolean values themselves. JavaScript supports three logical operators: *and*, *or*, and *not*. These can be used to “reason” about Booleans.

The && operator represents logical *and*. It is a binary operator, and its result is true only if both the values given to it are true.

```
console.log(true && false)
// → false
console.log(true && true)
```

```
// → true
```

The `||` operator denotes logical *or*. It produces true if either of the values given to it is true.

```
console.log(false || true)
// → true
console.log(false || false)
// → false
```

Not is written as an exclamation mark (`!`). It is a unary operator that flips the value given to it—`!true` produces `false` and `!false` gives `true`.

When mixing these Boolean operators with arithmetic and other operators, it is not always obvious when parentheses are needed. In practice, you can usually get by with knowing that of the operators we have seen so far, `||` has the lowest precedence, then comes `&&`, then the comparison operators (`>`, `==`, and so on), and then the rest. This order has been chosen such that, in typical expressions like the following one, as few parentheses as possible are necessary:

```
1 + 1 == 2 && 10 * 10 > 50
```

The last logical operator we will look at is not unary, not binary, but *ternary*, operating on three values. It is written with a question mark and a colon, like this:

```
console.log(true ? 1 : 2);
// → 1
console.log(false ? 1 : 2);
// → 2
```

This one is called the *conditional* operator (or sometimes just *the ternary operator* since it is the only such operator in the language). The operator uses the value to the left of the question mark to decide which of the two other values to “pick”. If you write `a ? b : c`, the result will be `b` when `a` is true and `c` otherwise.

EMPTY VALUES

There are two special values, written `null` and `undefined`, that are used to denote the absence of a *meaningful* value. They are themselves values, but they carry no information.

Many operations in the language that don’t produce a meaningful value yield `undefined` simply because they have to yield *some* value.

The difference in meaning between `undefined` and `null` is an accident of JavaScript’s design, and it doesn’t matter most of the time. In cases where you actually have to concern yourself with these values, I recommend treating them as mostly interchangeable.

AUTOMATIC TYPE CONVERSION

In the [introduction](#), I mentioned that JavaScript goes out of its way to accept almost any program you give it, even programs that do odd things. This is nicely demonstrated by the following expressions:

```
console.log(8 * null)
// → 0
console.log("5" - 1)
```

```
// → 4
console.log("5" + 1)
// → 51
console.log("five" * 2)
// → NaN
console.log(false == 0)
// → true
```

When an operator is applied to the “wrong” type of value, JavaScript will quietly convert that value to the type it needs, using a set of rules that often aren’t what you want or expect. This is called *type coercion*. The `null` in the first expression becomes `0` and the `"5"` in the second expression becomes `5` (from string to number). Yet in the third expression, `+` tries string concatenation before numeric addition, so the `1` is converted to `"1"` (from number to string).

When something that doesn’t map to a number in an obvious way (such as `"five"` or `undefined`) is converted to a number, you get the value `NaN`. Further arithmetic operations on `NaN` keep producing `NaN`, so if you find yourself getting one of those in an unexpected place, look for accidental type conversions.

When comparing values of the same type using the `==` operator, the outcome is easy to predict: you should get `true` when both values are the same, except in the case of `NaN`. But when the types differ, JavaScript uses a complicated and confusing set of rules to determine what to do. In most cases, it just tries to convert one of the values to the other value’s type. However, when `null` or `undefined` occurs on either side of the operator, it produces `true` only if both sides are one of `null` or `undefined`.

```
console.log(null == undefined);
// → true
console.log(null == 0);
// → false
```

That behavior is often useful. When you want to test whether a value has a real value instead of `null` or `undefined`, you can compare it to `null` with the `==` or `!=` operator.

What if you want to test whether something refers to the precise value `false`? Expressions like `0 == false` and `"" == false` are also `true` because of automatic type conversion. When you do *not* want any type conversions to happen, there are two additional operators: `===` and `!==`. The first tests whether a value is *precisely* equal to the other, and the second tests whether it is not precisely equal. Thus `"" === false` is `false`, as expected.

I recommend using the three-character comparison operators defensively to prevent unexpected type conversions from tripping you up. But when you’re certain the types on both sides will be the same, there is no problem with using the shorter operators.

SHORT-CIRCUITING OF LOGICAL OPERATORS

The logical operators `&&` and `||` handle values of different types in a peculiar way. They will convert the value on their left side to Boolean type in order to decide what to do, but depending on the operator and the result of that conversion, they will return either the *original* left-hand value or the right-hand value.

The `||` operator, for example, will return the value to its left when

that value can be converted to true and will return the value on its right otherwise. This has the expected effect when the values are Boolean and does something analogous for values of other types.

```
console.log(null || "user")
// → user
console.log("Agnes" || "user")
// → Agnes
```

We can use this functionality as a way to fall back on a default value. If you have a value that might be empty, you can put `||` after it with a replacement value. If the initial value can be converted to false, you'll get the replacement instead. The rules for converting strings and numbers to Boolean values state that `0`, `NaN`, and the empty string (`""`) count as false, while all the other values count as true. That means `0 || -1` produces `-1`, and `"" || "!"` yields `!"`.

The `??` operator resembles `||` but returns the value on the right only if the one on the left is `null` or `undefined`, not if it is some other value that can be converted to false. Often, this is preferable to the behavior of `||`.

```
console.log(0 || 100);
// → 100
console.log(0 ?? 100);
// → 0
console.log(null ?? 100);
// → 100
```

The `&&` operator works similarly but the other way around. When the

value to its left is something that converts to false, it returns that value, and otherwise it returns the value on its right.

Another important property of these two operators is that the part to their right is evaluated only when necessary. In the case of `true || x`, no matter what `x` is—even if it's a piece of program that does something *terrible*—the result will be true, and `x` is never evaluated. The same goes for `false && x`, which is false and will ignore `x`. This is called *short-circuit evaluation*.

The conditional operator works in a similar way. Of the second and third values, only the one that is selected is evaluated.

SUMMARY

We looked at four types of JavaScript values in this chapter: numbers, strings, Booleans, and undefined values. Such values are created by typing in their name (`true`, `null`) or value (`13`, `"abc"`).

You can combine and transform values with operators. We saw binary operators for arithmetic (`+`, `-`, `*`, `/`, and `%`), string concatenation (`+`), comparison (`==`, `!=`, `===`, `!==`, `<`, `>`, `<=`, `>=`), and logic (`&&`, `||`, `??`), as well as several unary operators (`-` to negate a number, `!` to negate logically, and `typeof` to find a value's type) and a ternary operator (`?:`) to pick one of two values based on a third value.

This gives you enough information to use JavaScript as a pocket calculator but not much more. The next chapter will start tying these expressions together into basic programs.

“And my heart glows bright red under my filmy, translucent skin and they have to administer 10cc of JavaScript to get me to come back. (I respond well to toxins in the blood.) Man, that stuff will kick the peaches right out your gills!”

—_why, Why's (Poignant) Guide to Ruby

CHAPTER 2

PROGRAM STRUCTURE

In this chapter, we will start to do things that can actually be called *programming*. We will expand our command of the JavaScript language beyond the nouns and sentence fragments we've seen so far to the point where we can express meaningful prose.

EXPRESSIONS AND STATEMENTS

In Chapter 1, we made values and applied operators to them to get new values. Creating values like this is the main substance of any JavaScript program. But that substance has to be framed in a larger structure to be useful. That's what we'll cover in this chapter.

A fragment of code that produces a value is called an *expression*. Every value that is written literally (such as 22 or “psychoanalysis”) is an expression. An expression between parentheses is also an expression, as is a binary operator applied to two expressions or a unary operator applied to one.

This shows part of the beauty of a language-based interface. Expres-

sions can contain other expressions in a way similar to how subsentences in human languages are nested—a subsentence can contain its own subsentences, and so on. This allows us to build expressions that describe arbitrarily complex computations.

If an expression corresponds to a sentence fragment, a JavaScript *statement* corresponds to a full sentence. A program is a list of statements.

The simplest kind of statement is an expression with a semicolon after it. This is a program:

```
1;  
!false;
```

It is a useless program, though. An expression can be content to just produce a value, which can then be used by the enclosing code. However, a statement stands on its own, so if it doesn't affect the world, it's useless. It may display something on the screen, as with `console.log`, or change the state of the machine in a way that will affect the statements that come after it. These changes are called *side effects*. The statements in the previous example just produce the values 1 and `true` and then immediately throw them away. This leaves no impression on the world at all. When you run this program, nothing observable happens.

In some cases, JavaScript allows you to omit the semicolon at the end of a statement. In other cases, it has to be there, or the next line will be treated as part of the same statement. The rules for when it can be safely omitted are somewhat complex and error prone. So in this book, every statement that needs a semicolon will always get one. I recommend you do the same, at least until you've learned more about

the subtleties of missing semicolons.

BINDINGS

How does a program keep an internal state? How does it remember things? We have seen how to produce new values from old values, but this does not change the old values, and the new value must be used immediately or it will dissipate again. To catch and hold values, JavaScript provides a thing called a *binding*, or *variable*.

```
let caught = 5 * 5;
```

That gives us a second kind of statement. The special word (*keyword*) `let` indicates that this sentence is going to define a binding. It is followed by the name of the binding and, if we want to immediately give it a value, by an `=` operator and an expression.

The example creates a binding called `caught` and uses it to grab hold of the number that is produced by multiplying 5 by 5.

After a binding has been defined, its name can be used as an expression. The value of such an expression is the value the binding currently holds. Here's an example:

```
let ten = 10;
console.log(ten * ten);
// → 100
```

When a binding points at a value, that does not mean it is tied to that value forever. The `=` operator can be used at any time on existing

bindings to disconnect them from their current value and have them point to a new one:

```
let mood = "light";
console.log(mood);
// → light
mood = "dark";
console.log(mood);
// → dark
```

You should imagine bindings as tentacles rather than boxes. They do not *contain* values; they *grasp* them—two bindings can refer to the same value. A program can access only the values to which it still has a reference. When you need to remember something, you either grow a tentacle to hold on to it or reattach one of your existing tentacles to it.

Let's look at another example. To remember the number of dollars that Luigi still owes you, you create a binding. When he pays back \$35, you give this binding a new value.

```
let luigiDebt = 140;
luigiDebt = luigiDebt - 35;
console.log(luigiDebt);
// → 105
```

When you define a binding without giving it a value, the tentacle has nothing to grasp, so it ends in thin air. If you ask for the value of an empty binding, you'll get the value `undefined`.

A single `let` statement may define multiple bindings. The definitions must be separated by commas:


```
let one = 1, two = 2;
console.log(one + two);
// → 3
```

The words `var` and `const` can also be used to create bindings, in a similar fashion to `let`.

```
var name = "Ayda";
const greeting = "Hello ";
console.log(greeting + name);
// → Hello Ayda
```

The first of these, `var` (short for “variable”), is the way bindings were declared in pre-2015 JavaScript, when `let` didn’t exist yet. I’ll get back to the precise way it differs from `let` in the next chapter. For now, remember that it mostly does the same thing, but we’ll rarely use it in this book because it behaves oddly in some situations.

The word `const` stands for *constant*. It defines a constant binding, which points at the same value for as long as it lives. This is useful for bindings that just give a name to a value so that you can easily refer to it later.

BINDING NAMES

Binding names can be any sequence of one or more letters. Digits can be part of binding names—`catch22` is a valid name, for example—but the name must not start with a digit. A binding name may include dollar signs (\$) or underscores (_) but no other punctuation or special

characters.

Words with a special meaning, such as `let`, are *keywords*, and may not be used as binding names. There are also a number of words that are “reserved for use” in future versions of JavaScript, which also can’t be used as binding names. The full list of keywords and reserved words is rather long:

```
break case catch class const continue debugger default
delete do else enum export extends false finally for
function if implements import interface in instanceof let
new package private protected public return static super
switch this throw true try typeof var void while with yield
```

Don’t worry about memorizing this list. When creating a binding produces an unexpected syntax error, check whether you’re trying to define a reserved word.

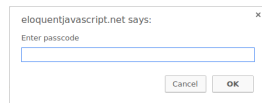
THE ENVIRONMENT

The collection of bindings and their values that exist at a given time is called the *environment*. When a program starts up, this environment is not empty. It always contains bindings that are part of the language standard, and most of the time, it also has bindings that provide ways to interact with the surrounding system. For example, in a browser, there are functions to interact with the currently loaded website and to read mouse and keyboard input.

FUNCTIONS

A lot of the values provided in the default environment have the type *function*. A function is a piece of program wrapped in a value. Such values can be *applied* in order to run the wrapped program. For example, in a browser environment, the binding `prompt` holds a function that shows a little dialog asking for user input. It is used like this:

```
prompt("Enter passcode");
```



Executing a function is called *invoking*, *calling*, or *applying* it. You can call a function by putting parentheses after an expression that produces a function value. Usually you'll directly use the name of the binding that holds the function. The values between the parentheses are given to the program inside the function. In the example, the `prompt` function uses the string that we give it as the text to show in the dialog box. Values given to functions are called *arguments*. Different functions might need a different number or different types of arguments.

The `prompt` function isn't used much in modern web programming, mostly because you have no control over the way the resulting dialog

looks, but it can be helpful in toy programs and experiments.

THE CONSOLE.LOG FUNCTION

In the examples, I used `console.log` to output values. Most JavaScript systems (including all modern web browsers and Node.js) provide a `console.log` function that writes out its arguments to *some* text output device. In browsers, the output lands in the JavaScript console. This part of the browser interface is hidden by default, but most browsers open it when you press F12 or, on a Mac, COMMAND-OPTION-I. If that does not work, search through the menus for an item named Developer Tools or similar.

Though binding names cannot contain period characters, `console.log` does have one. This is because `console.log` isn't a simple binding, but an expression that retrieves the `log` property from the value held by the `console` binding. We'll find out exactly what this means in Chapter 4.

RETURN VALUES

Showing a dialog box or writing text to the screen is a *side effect*. Many functions are useful because of the side effects they produce. Functions may also produce values, in which case they don't need to have a side effect to be useful. For example, the function `Math.max` takes any amount of number arguments and gives back the greatest.

```
console.log(Math.max(2, 4));  
// → 4
```

When a function produces a value, it is said to *return* that value. Anything that produces a value is an expression in JavaScript, which means that function calls can be used within larger expressions. In the following code, a call to `Math.min`, which is the opposite of `Math.max`, is used as part of a plus expression:

```
console.log(Math.min(2, 4) + 100);  
// → 102
```

Chapter 3 will explain how to write your own functions.

CONTROL FLOW

When your program contains more than one statement, the statements are executed as though they were a story, from top to bottom. For example, the following program has two statements. The first asks the user for a number, and the second, which is executed after the first, shows the square of that number:

```
let theNumber = Number(prompt("Pick a number"));  
console.log("Your number is the square root of " +  
  theNumber * theNumber);
```

The function `Number` converts a value to a number. We need that conversion because the result of `prompt` is a string value, and we want a number. There are similar functions called `String` and `Boolean` that convert values to those types.

Here is the rather trivial schematic representation of straight-line control flow:



CONDITIONAL EXECUTION

Not all programs are straight roads. We may, for example, want to create a branching road where the program takes the proper branch based on the situation at hand. This is called *conditional execution*.



Conditional execution is created with the `if` keyword in JavaScript. In the simple case, we want some code to be executed if, and only if, a certain condition holds. We might, for example, want to show the square of the input only if the input is actually a number:

```
let theNumber = Number(prompt("Pick a number"));  
if (!Number.isNaN(theNumber)) {  
  console.log("Your number is the square root of " +  
    theNumber * theNumber);  
}
```

With this modification, if you enter “parrot”, no output is shown.

The `if` keyword executes or skips a statement depending on the value of a Boolean expression. The deciding expression is written after the keyword, between parentheses, followed by the statement to execute.

The `Number.isNaN` function is a standard JavaScript function that returns `true` only if the argument it is given is `NaN`. The `Number` function

happens to return `NaN` when you give it a string that doesn't represent a valid number. Thus, the condition translates to "unless `theNumber` is not-a-number, do this".

The statement after the `if` is wrapped in braces (`{` and `}`) in this example. The braces can be used to group any number of statements into a single statement, called a *block*. You could also have omitted them in this case, since they hold only a single statement, but to avoid having to think about whether they are needed, most JavaScript programmers use them in every wrapped statement like this. We'll mostly follow that convention in this book, except for the occasional one-liner.

```
if (1 + 1 == 2) console.log("It's true");
// → It's true
```

You often won't just have code that executes when a condition holds true, but also code that handles the other case. This alternate path is represented by the second arrow in the diagram. You can use the `else` keyword, together with `if`, to create two separate, alternative execution paths:

```
let theNumber = Number(prompt("Pick a number"));
if (!Number.isNaN(theNumber)) {
  console.log("Your number is the square root of " +
    theNumber * theNumber);
} else {
  console.log("Hey. Why didn't you give me a number?");
}
```

If you have more than two paths to choose from, you can "chain" mul-

tiple `if/else` pairs together. Here's an example:

```
let num = Number(prompt("Pick a number"));
```

```
if (num < 10) {
  console.log("Small");
} else if (num < 100) {
  console.log("Medium");
} else {
  console.log("Large");
}
```

The program will first check whether `num` is less than 10. If it is, it chooses that branch, shows "Small", and is done. If it isn't, it takes the `else` branch, which itself contains a second `if`. If the second condition (`< 100`) holds, that means the number is at least 10 but below 100, and "Medium" is shown. If it doesn't, the second and last `else` branch is chosen.

The schema for this program looks something like this:



WHILE AND DO LOOPS

Consider a program that outputs all even numbers from 0 to 12. One way to write this is as follows:

```

console.log(0);
console.log(2);
console.log(4);
console.log(6);
console.log(8);
console.log(10);
console.log(12);

```

That works, but the idea of writing a program is to make something *less* work, not more. If we needed all even numbers less than 1,000, this approach would be unworkable. What we need is a way to run a piece of code multiple times. This form of control flow is called a *loop*.



Looping control flow allows us to go back to some point in the program where we were before and repeat it with our current program state. If we combine this with a binding that counts, we can do something like this:

```

let number = 0;
while (number <= 12) {
  console.log(number);
  number = number + 2;
}
// → 0
// → 2
// - etcetera

```

A statement starting with the keyword `while` creates a loop. The word `while` is followed by an expression in parentheses and then a statement, much like `if`. The loop keeps entering that statement as long as the expression produces a value that gives `true` when converted to Boolean.

The `number` binding demonstrates the way a binding can track the progress of a program. Every time the loop repeats, `number` gets a value that is 2 more than its previous value. At the beginning of every repetition, it is compared with the number 12 to decide whether the program's work is finished.

As an example that actually does something useful, we can now write a program that calculates and shows the value of 2^{10} (2 to the 10th power). We use two bindings: one to keep track of our result and one to count how often we have multiplied this result by 2. The loop tests whether the second binding has reached 10 yet and, if not, updates both bindings.

```

let result = 1;
let counter = 0;
while (counter < 10) {
  result = result * 2;
  counter = counter + 1;
}
console.log(result);
// → 1024

```

The counter could also have started at 1 and checked for `<= 10`, but for reasons that will become apparent in Chapter 4, it is a good idea to get used to counting from 0.

Note that JavaScript also has an operator for exponentiation (`2 ** 10`), which you would use to compute this in real code—but that would have ruined the example.

A `do` loop is a control structure similar to a `while` loop. It differs only on one point: a `do` loop always executes its body at least once, and it starts testing whether it should stop only after that first execution. To reflect this, the test appears after the body of the loop:

```
let yourName;
do {
  yourName = prompt("Who are you?");
} while (!yourName);
console.log("Hello " + yourName);
```

This program will force you to enter a name. It will ask again and again until it gets something that is not an empty string. Applying the `!` operator will convert a value to Boolean type before negating it, and all strings except `""` convert to `true`. This means the loop continues going round until you provide a non-empty name.

INDENTING CODE

In the examples, I've been adding spaces in front of statements that are part of some larger statement. These spaces are not required—the computer will accept the program just fine without them. In fact, even the line breaks in programs are optional. You could write a program as a single long line if you felt like it.

The role of this indentation inside blocks is to make the structure of

the code stand out to human readers. In code where new blocks are opened inside other blocks, it can become hard to see where one block ends and another begins. With proper indentation, the visual shape of a program corresponds to the shape of the blocks inside it. I like to use two spaces for every open block, but tastes differ—some people use four spaces, and some people use tab characters. The important thing is that each new block adds the same amount of space.

```
if (false != true) {
  console.log("That makes sense.");
  if (1 < 2) {
    console.log("No surprise there.");
  }
}
```

Most code editor programs will help by automatically indenting new lines the proper amount.

FOR LOOPS

Many loops follow the pattern shown in the `while` examples. First a “counter” binding is created to track the progress of the loop. Then comes a `while` loop, usually with a test expression that checks whether the counter has reached its end value. At the end of the loop body, the counter is updated to track progress.

Because this pattern is so common, JavaScript and similar languages provide a slightly shorter and more comprehensive form, the `for` loop:

```

for (let number = 0; number <= 12; number = number + 2) {
  console.log(number);
}
// → 0
// → 2
// ... etcetera

```

This program is exactly equivalent to the earlier even-number-printing example. The only change is that all the statements that are related to the “state” of the loop are grouped together after `for`.

The parentheses after a `for` keyword must contain two semicolons. The part before the first semicolon *initializes* the loop, usually by defining a binding. The second part is the expression that *checks* whether the loop must continue. The final part *updates* the state of the loop after every iteration. In most cases, this is shorter and clearer than a `while` construct.

This is the code that computes 2^{10} using `for` instead of `while`:

```

let result = 1;
for (let counter = 0; counter < 10; counter = counter + 1) {
  result = result * 2;
}
console.log(result);
// → 1024

```

BREAKING OUT OF A LOOP

Having the looping condition produce `false` is not the only way a loop can finish. The `break` statement has the effect of immediately jumping out of the enclosing loop. Its use is demonstrated in the following program, which finds the first number that is both greater than or equal to 20 and divisible by 7:

```

for (let current = 20; ; current = current + 1) {
  if (current % 7 == 0) {
    console.log(current);
    break;
  }
}
// → 21

```

Using the remainder (`%`) operator is an easy way to test whether a number is divisible by another number. If it is, the remainder of their division is zero.

The `for` construct in the example does not have a part that checks for the end of the loop. This means that the loop will never stop unless the `break` statement inside is executed.

If you were to remove that `break` statement or you accidentally write an end condition that always produces `true`, your program would get stuck in an *infinite loop*. A program stuck in an infinite loop will never finish running, which is usually a bad thing.

The `continue` keyword is similar to `break` in that it influences the progress of a loop. When `continue` is encountered in a loop body, control

jumps out of the body and continues with the loop's next iteration.

UPDATING BINDINGS SUCCINCTLY

Especially when looping, a program often needs to “update” a binding to hold a value based on that binding's previous value.

```
counter = counter + 1;
```

JavaScript provides a shortcut for this:

```
counter += 1;
```

Similar shortcuts work for many other operators, such as `result *= 2` to double `result` or `counter -= 1` to count downward.

This allows us to further shorten our counting example:

```
for (let number = 0; number <= 12; number += 2) {  
  console.log(number);  
}
```

For `counter += 1` and `counter -= 1`, there are even shorter equivalents: `counter++` and `counter--`.

DISPATCHING ON A VALUE WITH SWITCH

It is not uncommon for code to look like this:

```
if (x == "value1") action1();  
else if (x == "value2") action2();
```

```
else if (x == "value3") action3();  
else defaultAction();
```

There is a construct called `switch` that is intended to express such a “dispatch” in a more direct way. Unfortunately, the syntax JavaScript uses for this (which it inherited from the C/Java line of programming languages) is somewhat awkward—a chain of `if` statements may look better. Here is an example:

```
switch (prompt("What is the weather like?")) {  
  case "rainy":  
    console.log("Remember to bring an umbrella.");  
    break;  
  case "sunny":  
    console.log("Dress lightly.");  
  case "cloudy":  
    console.log("Go outside.");  
    break;  
  default:  
    console.log("Unknown weather type!");  
    break;  
}
```

You may put any number of `case` labels inside the block opened by `switch`. The program will start executing at the label that corresponds to the value that `switch` was given, or at `default` if no matching value is found. It will continue executing, even across other labels, until it reaches a `break` statement. In some cases, such as the “sunny” case in the example, this can be used to share some code between cases (it recommends going outside for both sunny and cloudy weather). Be

careful, though—it is easy to forget such a break, which will cause the program to execute code you do not want executed.

CAPITALIZATION

Binding names may not contain spaces, yet it is often helpful to use multiple words to clearly describe what the binding represents. These are pretty much your choices for writing a binding name with several words in it:

```
fuzzylittleturtle
fuzzy_little_turtle
FuzzylittleTurtle
fuzzylittleTurtle
```

The first style can be hard to read. I rather like the look of the underscores, though that style is a little painful to type. The standard JavaScript functions, and most JavaScript programmers, follow the final style—they capitalize every word except the first. It is not hard to get used to little things like that, and code with mixed naming styles can be jarring to read, so we follow this convention.

In a few cases, such as the `Number` function, the first letter of a binding is also capitalized. This was done to mark this function as a constructor. It will become clear what a constructor is in [Chapter 6](#). For now, the important thing is to not be bothered by this apparent lack of consistency.

COMMENTS

Often, raw code does not convey all the information you want a program to convey to human readers, or it conveys it in such a cryptic way that people might not understand it. At other times, you might just want to include some related thoughts as part of your program. This is what *comments* are for.

A comment is a piece of text that is part of a program but is completely ignored by the computer. JavaScript has two ways of writing comments. To write a single-line comment, you can use two slash characters (`//`) and then the comment text after it:

```
let accountBalance = calculateBalance(account);
// It's a green hollow where a river sings
accountBalance.adjust();
// Madly catching white tatters in the grass.
let report = new Report();
// Where the sun on the proud mountain rings:
addToReport(accountBalance, report);
// It's a little valley, foaming like light in a glass.
```

A `//` comment goes only to the end of the line. A section of text between `/*` and `*/` will be ignored in its entirety, regardless of whether it contains line breaks. This is useful for adding blocks of information about a file or a chunk of program:

```
/*
  I first found this number scrawled on the back of an old
  notebook. Since then, it has often dropped by, showing up in
```

```
    phone numbers and the serial numbers of products that I've
    bought. It obviously likes me, so I've decided to keep it.
    */
    const myNumber = 11213;
```

SUMMARY

You now know that a program is built out of statements, which themselves sometimes contain more statements. Statements tend to contain expressions, which themselves can be built out of smaller expressions.

Putting statements after one another gives you a program that is executed from top to bottom. You can introduce disturbances in the flow of control by using conditional (`if`, `else`, and `switch`) and looping (`while`, `do`, and `for`) statements.

Bindings can be used to file pieces of data under a name, and they are useful for tracking state in your program. The environment is the set of bindings that are defined. JavaScript systems always put a number of useful standard bindings into your environment.

Functions are special values that encapsulate a piece of program. You can invoke them by writing `functionName(argument1, argument2)`. Such a function call is an expression and may produce a value.

EXERCISES

If you are unsure how to test your solutions to the exercises, refer to the introduction.

Each exercise starts with a problem description. Read this description and try to solve the exercise. If you run into problems, consider reading the hints at the end of the book. You can find full solutions to the exercises online at <https://eloquentjavascript.net/code>. If you want to learn something from the exercises, I recommend looking at the solutions only after you've solved the exercise, or at least after you've attacked it long and hard enough to have a slight headache.

LOOPING A TRIANGLE

Write a loop that makes seven calls to `console.log` to output the following triangle:

```
#
##
###
####
#####
#####
```

It may be useful to know that you can find the length of a string by writing `.length` after it.

```
let abc = "abc";
console.log(abc.length);
// → 3
```

FizzBuzz

Write a program that uses `console.log` to print all the numbers from 1 to 100, with two exceptions. For numbers divisible by 3, print "Fizz" instead of the number, and for numbers divisible by 5 (and not 3), print "Buzz" instead.

When you have that working, modify your program to print "FizzBuzz" for numbers that are divisible by both 3 and 5 (and still print "Fizz" or "Buzz" for numbers divisible by only one of those).

(This is actually an interview question that has been claimed to weed out a significant percentage of programmer candidates. So if you solved it, your labor market value just went up.)

Chessboard

Write a program that creates a string that represents an 8×8 grid, using newline characters to separate lines. At each position of the grid there is either a space or a '#' character. The characters should form a chessboard.

Passing this string to `console.log` should show something like this:

```
# # #  
# # #  
# # #  
# # #  
# # #  
# # #  
# # #  
# # #
```

When you have a program that generates this pattern, define a binding `size = 8` and change the program so that it works for any `size`, outputting a grid of the given width and height.

“People think that computer science is the art of geniuses but the actual reality is the opposite, just many people doing things that build on each other, like a wall of mini stones.”
—Donald Knuth

CHAPTER 3

FUNCTIONS

Functions are one of the most central tools in JavaScript programming. The concept of wrapping a piece of program in a value has many uses. It gives us a way to structure larger programs, to reduce repetition, to associate names with subprograms, and to isolate these subprograms from each other.

The most obvious application of functions is defining new vocabulary. Creating new words in prose is usually bad style, but in programming, it is indispensable.

Typical adult English speakers have some 20,000 words in their vocabulary. Few programming languages come with 20,000 commands built in. And the vocabulary that *is* available tends to be more precisely defined, and thus less flexible, than in human language. Therefore, we *have* to introduce new words to avoid excessive verbosity.

DEFINING A FUNCTION

A function definition is a regular binding where the value of the binding is a function. For example, this code defines `square` to refer to a function that produces the square of a given number:

```
const square = function(x) {  
  return x * x;  
};  
  
console.log(square(12));  
// → 144
```

A function is created with an expression that starts with the keyword `function`. Functions have a set of *parameters* (in this case, only `x`) and a *body*, which contains the statements that are to be executed when the function is called. The body of a function created this way must always be wrapped in braces, even when it consists of only a single statement.

A function can have multiple parameters or no parameters at all. In the following example, `makeNoise` does not list any parameter names, whereas `roundTo` (which rounds `n` to the nearest multiple of `step`) lists two:

```
const makeNoise = function() {  
  console.log("Pling!");  
};  
  
makeNoise();  
// → Pling!
```

```
const roundTo = function(n, step) {
  let remainder = n % step;
  return n - remainder + (remainder < step / 2 ? 0 : step);
};

console.log(roundTo(23, 10));
// → 20
```

Some functions, such as `roundTo` and `square`, produce a value, and some don't, such as `makeNoise`, whose only result is a side effect. A `return` statement determines the value the function returns. When control comes across such a statement, it immediately jumps out of the current function and gives the returned value to the code that called the function. A `return` keyword without an expression after it will cause the function to return `undefined`. Functions that don't have a `return` statement at all, such as `makeNoise`, similarly return `undefined`.

Parameters to a function behave like regular bindings, but their initial values are given by the *caller* of the function, not the code in the function itself.

BINDINGS AND SCOPES

Each binding has a *scope*, which is the part of the program in which the binding is visible. For bindings defined outside of any function, block, or module (see Chapter 10), the scope is the whole program—you can refer to such bindings wherever you want. These are called *global*.

Bindings created for function parameters or declared inside a function

can be referenced only in that function, so they are known as *local* bindings. Every time the function is called, new instances of these bindings are created. This provides some isolation between functions—each function call acts in its own little world (its local environment) and can often be understood without knowing a lot about what's going on in the global environment.

Bindings declared with `let` and `const` are in fact local to the *block* in which they are declared, so if you create one of those inside of a loop, the code before and after the loop cannot “see” it. In pre-2015 JavaScript, only functions created new scopes, so old-style bindings, created with the `var` keyword, are visible throughout the whole function in which they appear—or throughout the global scope, if they are not in a function.

```
let x = 10; // global
if (true) {
  let y = 20; // local to block
  var z = 30; // also global
}
```

Each scope can “look out” into the scope around it, so `x` is visible inside the block in the example. The exception is when multiple bindings have the same name—in that case, code can see only the innermost one. For example, when the code inside the `halve` function refers to `n`, it is seeing its *own* `n`, not the global `n`.

```
const halve = function(n) {
  return n / 2;
};
```

```
let n = 10;
console.log(halve(100));
// → 50
console.log(n);
// → 10
```

NESTED SCOPE

JavaScript distinguishes not just global and local bindings. Blocks and functions can be created inside other blocks and functions, producing multiple degrees of locality.

For example, this function—which outputs the ingredients needed to make a batch of hummus—has another function inside it:

```
const hummus = function(factor) {
  const ingredient = function(amount, unit, name) {
    let ingredientAmount = amount * factor;
    if (ingredientAmount > 1) {
      unit += "s";
    }
    console.log(`${ingredientAmount} ${unit} ${name}`);
  };
  ingredient(1, "can", "chickpeas");
  ingredient(0.25, "cup", "tahini");
  ingredient(0.25, "cup", "lemon juice");
  ingredient(1, "clove", "garlic");
  ingredient(2, "tablespoon", "olive oil");
};
```

```
ingredient(0.5, "teaspoon", "cumin");
};
```

The code inside the `ingredient` function can see the `factor` binding from the outer function, but its local bindings, such as `unit` or `ingredientAmount`, are not visible in the outer function.

The set of bindings visible inside a block is determined by the place of that block in the program text. Each local scope can also see all the local scopes that contain it, and all scopes can see the global scope. This approach to binding visibility is called *lexical scoping*.

FUNCTIONS AS VALUES

A function binding usually simply acts as a name for a specific piece of the program. Such a binding is defined once and never changed. This makes it easy to confuse the function and its name.

But the two are different. A function value can do all the things that other values can do—you can use it in arbitrary expressions, not just call it. It is possible to store a function value in a new binding, pass it as an argument to a function, and so on. Similarly, a binding that holds a function is still just a regular binding and can, if not constant, be assigned a new value, like so:

```
let launchMissiles = function() {
  missileSystem.launch("now");
};
if (safeMode) {
  launchMissiles = function() { /* do nothing */ };
}
```

```
}
```

In [Chapter 5](#), we'll discuss the interesting things that we can do by passing function values to other functions.

DECLARATION NOTATION

There is a slightly shorter way to create a function binding. When the `function` keyword is used at the start of a statement, it works differently:

```
function square(x) {  
  return x * x;  
}
```

This is a function *declaration*. The statement defines the binding `square` and points it at the given function. It is slightly easier to write and doesn't require a semicolon after the function.

There is one subtlety with this form of function definition.

```
console.log("The future says:", future());
```

```
function future() {  
  return "You'll never have flying cars";  
}
```

The preceding code works, even though the function is defined *below* the code that uses it. Function declarations are not part of the regular top-to-bottom flow of control. They are conceptually moved to the top of their scope and can be used by all the code in that scope. This is

sometimes useful because it offers the freedom to order code in a way that seems the clearest, without worrying about having to define all functions before they are used.

ARROW FUNCTIONS

There's a third notation for functions, which looks very different from the others. Instead of the `function` keyword, it uses an arrow (`=>`) made up of an equal sign and a greater-than character (not to be confused with the greater-than-or-equal operator, which is written `>=`):

```
const roundTo = (n, step) => {  
  let remainder = n % step;  
  return n - remainder + (remainder < step / 2 ? 0 : step);  
};
```

The arrow comes *after* the list of parameters and is followed by the function's body. It expresses something like "this input (the parameters) produces this result (the body)".

When there is only one parameter name, you can omit the parentheses around the parameter list. If the body is a single expression rather than a block in braces, that expression will be returned from the function. So, these two definitions of `square` do the same thing:

```
const square1 = (x) => { return x * x; };  
const square2 = x => x * x;
```

When an arrow function has no parameters at all, its parameter list is just an empty set of parentheses.

```
const horn = () => {
  console.log("Toot");
};
```

There's no deep reason to have both arrow functions and function expressions in the language. Apart from a minor detail, which we'll discuss in Chapter 6, they do the same thing. Arrow functions were added in 2015, mostly to make it possible to write small function expressions in a less verbose way. We'll use them often in Chapter 5.

THE CALL STACK

The way control flows through functions is somewhat involved. Let's take a closer look at it. Here is a simple program that makes a few function calls:

```
function greet(who) {
  console.log("Hello " + who);
}
greet("Harry");
console.log("Bye");
```

A run through this program goes roughly like this: the call to `greet` causes control to jump to the start of that function (line 2). The function calls `console.log`, which takes control, does its job, and then returns control to line 2. There, it reaches the end of the `greet` function, so it returns to the place that called it—line 4. The line after that calls `console.log` again. After that returns, the program reaches its end.

We could show the flow of control schematically like this:

```
not in function
  in greet
    in console.log
      in greet
        not in function
          in console.log
            not in function
```

Because a function has to jump back to the place that called it when it returns, the computer must remember the context from which the call happened. In one case, `console.log` has to return to the `greet` function when it is done. In the other case, it returns to the end of the program.

The place where the computer stores this context is the *call stack*. Every time a function is called, the current context is stored on top of this stack. When a function returns, it removes the top context from the stack and uses that context to continue execution.

Storing this stack requires space in the computer's memory. When the stack grows too big, the computer will fail with a message like “out of stack space” or “too much recursion”. The following code illustrates this by asking the computer a really hard question that causes an infinite back-and-forth between two functions. Or rather, it *would* be infinite, if the computer had an infinite stack. As it is, we will run out of space, or “blow the stack”.

```
function chicken() {
  return egg();
}
```



```
function egg() {
  return chicken();
}
console.log(chicken() + " came first.");
// → ??
```

OPTIONAL ARGUMENTS

The following code is allowed and executes without any problem:

```
function square(x) { return x * x; }
console.log(square(4, true, "hedgehog"));
// → 16
```

We defined `square` with only one parameter. Yet when we call it with three, the language doesn't complain. It ignores the extra arguments and computes the square of the first one.

JavaScript is extremely broad-minded about the number of arguments you can pass to a function. If you pass too many, the extra ones are ignored. If you pass too few, the missing parameters are assigned the value `undefined`.

The downside of this is that it is possible—likely, even—that you'll accidentally pass the wrong number of arguments to functions. And no one will tell you about it. The upside is that you can use this behavior to allow a function to be called with different numbers of arguments. For example, this `minus` function tries to imitate the `-` operator by acting on either one or two arguments:

```
function minus(a, b) {
  if (b === undefined) return -a;
  else return a - b;
}
```

```
console.log(minus(10));
// → -10
console.log(minus(10, 5));
// → 5
```

If you write an `=` operator after a parameter, followed by an expression, the value of that expression will replace the argument when it is not given. For example, this version of `roundTo` makes its second argument optional. If you don't provide it or pass the value `undefined`, it will default to one:

```
function roundTo(n, step = 1) {
  let remainder = n % step;
  return n - remainder + (remainder < step / 2 ? 0 : step);
};
```

```
console.log(roundTo(4.5));
// → 5
console.log(roundTo(4.5, 2));
// → 4
```

The next chapter will introduce a way in which a function body can get at the whole list of arguments it was passed. This is helpful because it allows a function to accept any number of arguments. For example, `console.log` does this, outputting all the values it is given:

```
console.log("C", "0", 2);
// → C 0 2
```

CLOSURE

The ability to treat functions as values, combined with the fact that local bindings are re-created every time a function is called, brings up an interesting question: What happens to local bindings when the function call that created them is no longer active?

The following code shows an example of this. It defines a function, `wrapValue`, that creates a local binding. It then returns a function that accesses and returns this local binding.

```
function wrapValue(n) {
  let local = n;
  return () => local;
}

let wrap1 = wrapValue(1);
let wrap2 = wrapValue(2);
console.log(wrap1());
// → 1
console.log(wrap2());
// → 2
```

This is allowed and works as you'd hope—both instances of the binding can still be accessed. This situation is a good demonstration of the fact that local bindings are created anew for every call, and different calls

don't affect each other's local bindings.

This feature—being able to reference a specific instance of a local binding in an enclosing scope—is called *closure*. A function that references bindings from local scopes around it is called *a* closure. This behavior not only frees you from having to worry about the lifetimes of bindings but also makes it possible to use function values in some creative ways.

With a slight change, we can turn the previous example into a way to create functions that multiply by an arbitrary amount.

```
function multiplier(factor) {
  return number => number * factor;
}

let twice = multiplier(2);
console.log(twice(5));
// → 10
```

The explicit local binding from the `wrapValue` example isn't really needed since a parameter is itself a local binding.

Thinking about programs like this takes some practice. A good mental model is to think of function values as containing both the code in their body and the environment in which they are created. When called, the function body sees the environment in which it was created, not the environment in which it is called.

In the previous example, `multiplier` is called and creates an environment in which its `factor` parameter is bound to 2. The function value it returns, which is stored in `twice`, remembers this environment so that

when that is called, it multiplies its argument by 2.

RECURSION

It is perfectly okay for a function to call itself, as long as it doesn't do it so often that it overflows the stack. A function that calls itself is called *recursive*. Recursion allows some functions to be written in a different style. Take, for example, this `power` function, which does the same as the `**` (exponentiation) operator:

```
function power(base, exponent) {  
  if (exponent == 0) {  
    return 1;  
  } else {  
    return base * power(base, exponent - 1);  
  }  
}  
  
console.log(power(2, 3));  
// → 8
```

This is rather close to the way mathematicians define exponentiation and arguably describes the concept more clearly than the loop we used in Chapter 2. The function calls itself multiple times with ever smaller exponents to achieve the repeated multiplication.

However, this implementation has one problem: in typical JavaScript implementations, it's about three times slower than a version using a `for` loop. Running through a simple loop is generally cheaper than

calling a function multiple times.

The dilemma of speed versus elegance is an interesting one. You can see it as a kind of continuum between human-friendliness and machine-friendliness. Almost any program can be made faster by making it bigger and more convoluted. The programmer has to find an appropriate balance.

In the case of the `power` function, an inelegant (looping) version is still fairly simple and easy to read. It doesn't make much sense to replace it with a recursive function. Often, though, a program deals with such complex concepts that giving up some efficiency in order to make the program more straightforward is helpful.

Worrying about efficiency can be a distraction. It's yet another factor that complicates program design, and when you're doing something that's already difficult, that extra thing to worry about can be paralyzing.

Therefore, you should generally start by writing something that's correct and easy to understand. If you're worried that it's too slow—which it usually isn't since most code simply isn't executed often enough to take any significant amount of time—you can measure afterward and improve it if necessary.

Recursion is not always just an inefficient alternative to looping. Some problems really are easier to solve with recursion than with loops. Most often these are problems that require exploring or processing several “branches”, each of which might branch out again into even more branches.

Consider this puzzle: by starting from the number 1 and repeatedly

either adding 5 or multiplying by 3, an infinite set of numbers can be produced. How would you write a function that, given a number, tries to find a sequence of such additions and multiplications that produces that number? For example, the number 13 could be reached by first multiplying by 3 and then adding 5 twice, whereas the number 15 cannot be reached at all.

Here is a recursive solution:

```
function findSolution(target) {
  function find(current, history) {
    if (current == target) {
      return history;
    } else if (current > target) {
      return null;
    } else {
      return find(current + 5, `${history} + 5`) ??
        find(current * 3, `${history} * 3`);
    }
  }
  return find(1, "1");
}

console.log(findSolution(24));
// → (((1 * 3) + 5) * 3)
```

Note that this program doesn't necessarily find the *shortest* sequence of operations. It is satisfied when it finds any sequence at all.

It's okay if you don't see how this code works right away. Let's work through it since it makes for a great exercise in recursive thinking.

The inner function `find` does the actual recursing. It takes two arguments: the current number and a string that records how we reached this number. If it finds a solution, it returns a string that shows how to get to the target. If it can find no solution starting from this number, it returns `null`.

To do this, the function performs one of three actions. If the current number is the target number, the current history is a way to reach that target, so it is returned. If the current number is greater than the target, there's no sense in further exploring this branch because both adding and multiplying will only make the number bigger, so it returns `null`. Finally, if we're still below the target number, the function tries both possible paths that start from the current number by calling itself twice, once for addition and once for multiplication. If the first call returns something that is not `null`, it is returned. Otherwise, the second call is returned, regardless of whether it produces a string or `null`.

To better understand how this function produces the effect we're looking for, let's look at all the calls to `find` that are made when searching for a solution for the number 13:

```
find(1, "1")
  find(6, "(1 + 5)")
    find(11, "((1 + 5) + 5)")
      find(16, "(((1 + 5) + 5) + 5)")
        too big
      find(33, "((1 + 5) + 5) * 3")
        too big
    find(18, "(1 + 5) * 3")
      too big
```

```

find(3, "(1 * 3)")
  find(8, "((1 * 3) + 5)")
    find(13, "(((1 * 3) + 5) + 5)")
      found!

```

The indentation indicates the depth of the call stack. The first time `find` is called, the function starts by calling itself to explore the solution that starts with $(1 + 5)$. That call will further recurse to explore *every* continued solution that yields a number less than or equal to the target number. Since it doesn't find one that hits the target, it returns `null` back to the first call. There the `??` operator causes the call that explores $(1 * 3)$ to happen. This search has more luck—its first recursive call, through yet *another* recursive call, hits upon the target number. That innermost call returns a string, and each of the `??` operators in the intermediate calls passes that string along, ultimately returning the solution.

GROWING FUNCTIONS

There are two more or less natural ways for functions to be introduced into programs.

The first occurs when you find yourself writing similar code multiple times. You'd prefer not to do that, as having more code means more space for mistakes to hide and more material to read for people trying to understand the program. So you take the repeated functionality, find a good name for it, and put it into a function.

The second way is that you find you need some functionality that you

haven't written yet and that sounds like it deserves its own function. You start by naming the function, and then write its body. You might even start writing code that uses the function before you actually define the function itself.

How difficult it is to find a good name for a function is a good indication of how clear a concept it is that you're trying to wrap. Let's go through an example.

We want to write a program that prints two numbers: the numbers of cows and chickens on a farm, with the words `Cows` and `Chickens` after them and zeros padded before both numbers so that they are always three digits long:

```

007 Cows
011 Chickens

```

This asks for a function of two arguments—the number of cows and the number of chickens. Let's get coding.

```

function printFarmInventory(cows, chickens) {
  let cowString = String(cows);
  while (cowString.length < 3) {
    cowString = "0" + cowString;
  }
  console.log(`${cowString} Cows`);
  let chickenString = String(chickens);
  while (chickenString.length < 3) {
    chickenString = "0" + chickenString;
  }
  console.log(`${chickenString} Chickens`);
}

```

```
}  
printFarmInventory(7, 11);
```

Writing `.length` after a string expression will give us the length of that string. Thus, the `while` loops keep adding zeros in front of the number strings until they are at least three characters long.

Mission accomplished! But just as we are about to send the farmer the code (along with a hefty invoice), she calls and tells us she's also started keeping pigs, and couldn't we please extend the software to also print pigs?

We sure can. But just as we're in the process of copying and pasting those four lines one more time, we stop and reconsider. There has to be a better way. Here's a first attempt:

```
function printZeroPaddedWithLabel(number, label) {  
  let numberString = String(number);  
  while (numberString.length < 3) {  
    numberString = "0" + numberString;  
  }  
  console.log(`${numberString} ${label}`);  
}  
  
function printFarmInventory(cows, chickens, pigs) {  
  printZeroPaddedWithLabel(cows, "Cows");  
  printZeroPaddedWithLabel(chickens, "Chickens");  
  printZeroPaddedWithLabel(pigs, "Pigs");  
}  
  
printFarmInventory(7, 11, 3);
```

It works! But that name, `printZeroPaddedWithLabel`, is a little awkward. It conflates three things—printing, zero-padding, and adding a label—into a single function.

Instead of lifting out the repeated part of our program wholesale, let's try to pick out a single *concept*:

```
function zeroPad(number, width) {  
  let string = String(number);  
  while (string.length < width) {  
    string = "0" + string;  
  }  
  return string;  
}  
  
function printFarmInventory(cows, chickens, pigs) {  
  console.log(`${zeroPad(cows, 3)} Cows`);  
  console.log(`${zeroPad(chickens, 3)} Chickens`);  
  console.log(`${zeroPad(pigs, 3)} Pigs`);  
}  
  
printFarmInventory(7, 16, 3);
```

A function with a nice, obvious name like `zeroPad` makes it easier for someone who reads the code to figure out what it does. Such a function is also useful in more situations than just this specific program. For example, you could use it to help print nicely aligned tables of numbers.

How smart and versatile *should* our function be? We could write anything, from a terribly simple function that can only pad a number to be three characters wide to a complicated generalized number-formatting

system that handles fractional numbers, negative numbers, alignment of decimal dots, padding with different characters, and so on.

A useful principle is to refrain from adding cleverness unless you are absolutely sure you're going to need it. It can be tempting to write general “frameworks” for every bit of functionality you come across. Resist that urge. You won't get any real work done—you'll be too busy writing code that you never use.

FUNCTIONS AND SIDE EFFECTS

Functions can be roughly divided into those that are called for their side effects and those that are called for their return value (though it is also possible to both have side effects and return a value).

The first helper function in the farm example, `printZeroPaddedWithLabel`, is called for its side effect: it prints a line. The second version, `zeroPad`, is called for its return value. It is no coincidence that the second is useful in more situations than the first. Functions that create values are easier to combine in new ways than functions that directly perform side effects.

A *pure* function is a specific kind of value-producing function that not only has no side effects but also doesn't rely on side effects from other code—for example, it doesn't read global bindings whose value might change. A pure function has the pleasant property that, when called with the same arguments, it always produces the same value (and doesn't do anything else). A call to such a function can be substituted by its return value without changing the meaning of the code. When

you are not sure that a pure function is working correctly, you can test it by simply calling it and know that if it works in that context, it will work in any context. Nonpure functions tend to require more scaffolding to test.

Still, there's no need to feel bad when writing functions that are not pure. Side effects are often useful. There's no way to write a pure version of `console.log`, for example, and `console.log` is good to have. Some operations are also easier to express in an efficient way when we use side effects.

SUMMARY

This chapter taught you how to write your own functions. The `function` keyword, when used as an expression, can create a function value. When used as a statement, it can be used to declare a binding and give it a function as its value. Arrow functions are yet another way to create functions.

```
// Define f to hold a function value
const f = function(a) {
  console.log(a + 2);
};

// Declare g to be a function
function g(a, b) {
  return a * b * 3.5;
}
```

```
// A less verbose function value
let h = a => a % 3;
```

A key part of understanding functions is understanding scopes. Each block creates a new scope. Parameters and bindings declared in a given scope are local and not visible from the outside. Bindings declared with `var` behave differently—they end up in the nearest function scope or the global scope.

Separating the tasks your program performs into different functions is helpful. You won't have to repeat yourself as much, and functions can help organize a program by grouping code into pieces that do specific things.

EXERCISES

MINIMUM

The previous chapter introduced the standard function `Math.min` that returns its smallest argument. We can write a function like that ourselves now. Define the function `min` that takes two arguments and returns their minimum.

RECURSION

We've seen that we can use `%` (the remainder operator) to test whether a number is even or odd by using `% 2` to see whether it's divisible by two. Here's another way to define whether a positive whole number is even or odd:

- Zero is even.
- One is odd.
- For any other number N , its evenness is the same as $N - 2$.

Define a recursive function `isEven` corresponding to this description. The function should accept a single parameter (a positive, whole number) and return a Boolean.

Test it on 50 and 75. See how it behaves on -1. Why? Can you think of a way to fix this?

BEAN COUNTING

You can get the N th character, or letter, from a string by writing `[N]` after the string (for example, `string[2]`). The resulting value will be a string containing only one character (for example, "b"). The first character has position 0, which causes the last one to be found at position `string.length - 1`. In other words, a two-character string has length 2, and its characters have positions 0 and 1.

Write a function called `countBs` that takes a string as its only argument and returns a number that indicates how many uppercase B characters there are in the string.

Next, write a function called `countChar` that behaves like `countBs`, except it takes a second argument that indicates the character that is to be counted (rather than counting only uppercase B characters). Rewrite `countBs` to make use of this new function.

“On two occasions I have been asked, ‘Pray, Mr. Babbage, if you put into the machine wrong figures, will the right answers come out?’ [...] I am not able rightly to apprehend the kind of confusion of ideas that could provoke such a question.”

—Charles Babbage, *Passages from the Life of a Philosopher*
(1864)

CHAPTER 4

DATA STRUCTURES: OBJECTS AND ARRAYS

Numbers, Booleans, and strings are the atoms from which data structures are built. Many types of information require more than one atom, though. *Objects* allow us to group values—including other objects—to build more complex structures.

The programs we have built so far have been limited by the fact that they were operating only on simple data types. After learning the basics of data structures in this chapter, you’ll know enough to start writing useful programs.

The chapter will work through a more or less realistic programming example, introducing concepts as they apply to the problem at hand. The example code will often build on functions and bindings introduced earlier in the book.

The online coding sandbox for the book (<https://cloquentjavascript.net/code>) provides a way to run code in the context of a particular chapter. If you decide to work through the examples in another environment, be sure to first download the full code for this chapter from the sandbox page.

THE WERESQUIRREL

Every now and then, usually between 8 p.m. and 10 p.m., Jacques finds himself transforming into a small furry rodent with a bushy tail.

On one hand, Jacques is quite glad that he doesn’t have classic lycanthropy. Turning into a squirrel does cause fewer problems than turning into a wolf. Instead of having to worry about accidentally eating the neighbor (*that* would be awkward), he worries about being eaten by the neighbor’s cat. After two occasions of waking up on a precariously thin branch in the crown of an oak, naked and disoriented, he has taken to locking the doors and windows of his room at night and putting a few walnuts on the floor to keep himself busy.

But Jacques would prefer to get rid of his condition entirely. The irregular occurrences of the transformation make him suspect that they might be triggered by something. For a while, he believed that it happened only on days when he had been near oak trees. However, avoiding oak trees did not solve the problem.

Switching to a more scientific approach, Jacques has started keeping a daily log of everything he does on a given day and whether he changed form. With this data he hopes to narrow down the conditions that trigger the transformations.

The first thing he needs is a data structure to store this information.

DATASETS

To work with a chunk of digital data, we first have to find a way to represent it in our machine's memory. Say, for example, that we want to represent a collection of the numbers 2, 3, 5, 7, and 11.

We could get creative with strings—after all, strings can have any length, so we can put a lot of data into them—and use "2 3 5 7 11" as our representation. But this is awkward. We'd have to somehow extract the digits and convert them back to numbers to access them.

Fortunately, JavaScript provides a data type specifically for storing sequences of values. It is called an *array* and is written as a list of values between square brackets, separated by commas.

```
let listOfNumbers = [2, 3, 5, 7, 11];
console.log(listOfNumbers[2]);
// → 5
console.log(listOfNumbers[0]);
// → 2
console.log(listOfNumbers[2 - 1]);
// → 3
```

The notation for getting at the elements inside an array also uses square brackets. A pair of square brackets immediately after an expression, with another expression inside of them, will look up the element in the left-hand expression that corresponds to the *index* given by the expression in the brackets.

The first index of an array is zero, not one, so the first element is retrieved with `listOfNumbers[0]`. Zero-based counting has a long tradi-

tion in technology and in certain ways makes a lot of sense, but it takes some getting used to. Think of the index as the number of items to skip, counting from the start of the array.

PROPERTIES

We've seen a few expressions like `myString.length` (to get the length of a string) and `Math.max` (the maximum function) in past chapters. These expressions access a *property* of some value. In the first case, we access the `length` property of the value in `myString`. In the second, we access the property named `max` in the `Math` object (which is a collection of mathematics-related constants and functions).

Almost all JavaScript values have properties. The exceptions are `null` and `undefined`. If you try to access a property on one of these nonvalues, you get an error:

```
null.length;
// → TypeError: null has no properties
```

The two main ways to access properties in JavaScript are with a dot and with square brackets. Both `value.x` and `value[x]` access a property on `value`—but not necessarily the same property. The difference is in how `x` is interpreted. When using a dot, the word after the dot is the literal name of the property. When using square brackets, the expression between the brackets is *evaluated* to get the property name. Whereas `value.x` fetches the property of `value` named "x", `value[x]` takes the value of the variable named `x` and uses that, converted to a string, as

the property name.

If you know that the property in which you are interested is called *color*, you say `value.color`. If you want to extract the property named by the value held in the binding `i`, you say `value[i]`. Property names are strings. They can be any string, but the dot notation works only with names that look like valid binding names—starting with a letter or underscore, and containing only letters, numbers, and underscores. If you want to access a property named *2* or *John Doe*, you must use square brackets: `value[2]` or `value["John Doe"]`.

The elements in an array are stored as the array’s properties, using numbers as property names. Because you can’t use the dot notation with numbers and usually want to use a binding that holds the index anyway, you have to use the bracket notation to get at them.

Just like strings, arrays have a `length` property that tells us how many elements the array has.

METHODS

Both string and array values contain, in addition to the `length` property, a number of properties that hold function values.

```
let doh = "Doh";
console.log(typeof doh.toUpperCase);
// → function
console.log(doh.toUpperCase());
// → DOH
```

Every string has a `toUpperCase` property. When called, it will return a

copy of the string in which all letters have been converted to uppercase. There is also `toLowerCase`, going the other way.

Interestingly, even though the call to `toUpperCase` does not pass any arguments, the function somehow has access to the string "Doh", the value whose property we called. You’ll find out how this works in Chapter 6.

Properties that contain functions are generally called *methods* of the value they belong to, as in “`toUpperCase` is a method of a string”.

This example demonstrates two methods you can use to manipulate arrays.

```
let sequence = [1, 2, 3];
sequence.push(4);
sequence.push(5);
console.log(sequence);
// → [1, 2, 3, 4, 5]
console.log(sequence.pop());
// → 5
console.log(sequence);
// → [1, 2, 3, 4]
```

The `push` method adds values to the end of an array. The `pop` method does the opposite, removing the last value in the array and returning it.

These somewhat silly names are the traditional terms for operations on a *stack*. A stack, in programming, is a data structure that allows you to push values into it and pop them out again in the opposite order so that the thing that was added last is removed first. Stacks are common

in programming—you might remember the function call stack from the previous chapter, which is an instance of the same idea.

OBJECTS

Back to the weresquirrel. A set of daily log entries can be represented as an array, but the entries do not consist of just a number or a string—each entry needs to store a list of activities and a Boolean value that indicates whether Jacques turned into a squirrel or not. Ideally, we would like to group these together into a single value and then put those grouped values into an array of log entries.

Values of the type *object* are arbitrary collections of properties. One way to create an object is by using braces as an expression.

```
let day1 = {
  squirrel: false,
  events: ["work", "touched tree", "pizza", "running"]
};
console.log(day1.squirrel);
// → false
console.log(day1.wolf);
// → undefined
day1.wolf = false;
console.log(day1.wolf);
// → false
```

Inside the braces, you write a list of properties separated by commas. Each property has a name followed by a colon and a value. When

an object is written over multiple lines, indenting it as shown in this example helps with readability. Properties whose names aren't valid binding names or valid numbers must be quoted:

```
let descriptions = {
  work: "Went to work",
  "touched tree": "Touched a tree"
};
```

This means that braces have *two* meanings in JavaScript. At the start of a statement, they begin a block of statements. In any other position, they describe an object. Fortunately, it is rarely useful to start a statement with an object in braces, so the ambiguity between these two is not much of a problem. The one case where this does come up is when you want to return an object from a shorthand arrow function—you can't write `n => {prop: n}` since the braces will be interpreted as a function body. Instead, you have to put a set of parentheses around the object to make it clear that it is an expression.

Reading a property that doesn't exist will give you the value `undefined`.

It is possible to assign a value to a property expression with the `=` operator. This will replace the property's value if it already existed or create a new property on the object if it didn't.

To briefly return to our tentacle model of bindings—property bindings are similar. They *grasp* values, but other bindings and properties might be holding onto those same values. You can think of objects as octopuses with any number of tentacles, each of which has a name written on it.

The `delete` operator cuts off a tentacle from such an octopus. It is a unary operator that, when applied to an object property, will remove the named property from the object. This is not a common thing to do, but it is possible.

```
let anObject = {left: 1, right: 2};
console.log(anObject.left);
// → 1
delete anObject.left;
console.log(anObject.left);
// → undefined
console.log("left" in anObject);
// → false
console.log("right" in anObject);
// → true
```

The binary `in` operator, when applied to a string and an object, tells you whether that object has a property with that name. The difference between setting a property to `undefined` and actually deleting it is that in the first case, the object still *has* the property (it just doesn't have a very interesting value), whereas in the second case, the property is no longer present and `in` will return `false`.

To find out what properties an object has, you can use the `Object.keys` function. Give the function an object and it will return an array of strings—the object's property names:

```
console.log(Object.keys({x: 0, y: 0, z: 2}));
// → ["x", "y", "z"]
```

There's an `Object.assign` function that copies all properties from one

object into another:

```
let objectA = {a: 1, b: 2};
Object.assign(objectA, {b: 3, c: 4});
console.log(objectA);
// → {a: 1, b: 3, c: 4}
```

Arrays, then, are just a kind of object specialized for storing sequences of things. If you evaluate `typeof []`, it produces `"object"`. You can visualize arrays as long, flat octopuses with all their tentacles in a neat row, labeled with numbers.

Jacques will represent the journal that Jacques keeps as an array of objects:

```
let journal = [
  {events: ["work", "touched tree", "pizza",
    "running", "television"],
    squirrel: false},
  {events: ["work", "ice cream", "cauliflower",
    "lasagna", "touched tree", "brushed teeth"],
    squirrel: false},
  {events: ["weekend", "cycling", "break", "peanuts",
    "beer"],
    squirrel: true},
  /* And so on... */
];
```

MUTABILITY

We will get to actual programming soon, but first, there's one more piece of theory to understand.

We saw that object values can be modified. The types of values discussed in earlier chapters, such as numbers, strings, and Booleans, are all *immutable*—it is impossible to change values of those types. You can combine them and derive new values from them, but when you take a specific string value, that value will always remain the same. The text inside it cannot be changed. If you have a string that contains "cat", it is not possible for other code to change a character in your string to make it spell "rat".

Objects work differently. You *can* change their properties, causing a single object value to have different content at different times.

When we have two numbers, 120 and 120, we can consider them precisely the same number, whether or not they refer to the same physical bits. With objects, there is a difference between having two references to the same object and having two different objects that contain the same properties. Consider the following code:

```
let object1 = {value: 10};
let object2 = object1;
let object3 = {value: 10};

console.log(object1 == object2);
// → true
console.log(object1 == object3);
// → false
```

```
object1.value = 15;
console.log(object2.value);
// → 15
console.log(object3.value);
// → 10
```

The `object1` and `object2` bindings grasp the *same* object, which is why changing `object1` also changes the value of `object2`. They are said to have the same *identity*. The binding `object3` points to a different object, which initially contains the same properties as `object1` but lives a separate life.

Bindings can also be changeable or constant, but this is separate from the way their values behave. Even though number values don't change, you can use a `let` binding to keep track of a changing number by changing the value at which the binding points. Similarly, though a `const` binding to an object can itself not be changed and will continue to point at the same object, the *contents* of that object might change.

```
const score = {visitors: 0, home: 0};
// This is okay
score.visitors = 1;
// This isn't allowed
score = {visitors: 1, home: 1};
```

When you compare objects with JavaScript's `==` operator, it compares by identity: it will produce `true` only if both objects are precisely the same value. Comparing different objects will return `false`, even if they have identical properties. There is no “deep” comparison operation

built into JavaScript that compares objects by contents, but it is possible to write it yourself (which is one of the *exercises* at the end of this chapter).

THE LYCANTHROPE'S LOG

Jacques starts up his JavaScript interpreter and sets up the environment he needs to keep his journal:

```
let journal = [];  
  
function addEntry(events, squirrel) {  
  journal.push([events, squirrel]);  
}
```

Note that the object added to the journal looks a little odd. Instead of declaring properties like `events: events`, it just gives a property name: `events`. This is shorthand that means the same thing—if a property name in brace notation isn't followed by a value, its value is taken from the binding with the same name.

Every evening at 10 p.m.—or sometimes the next morning, after climbing down from the top shelf of his bookcase—Jacques records the day:

```
addEntry(["work", "touched tree", "pizza", "running",  
         "television"], false);  
addEntry(["work", "ice cream", "cauliflower", "lasagna",  
         "touched tree", "brushed teeth"], false);  
addEntry(["weekend", "cycling", "break", "peanuts",
```

```
"beer"], true);
```

Once he has enough data points, he intends to use statistics to find out which of these events may be related to the squirrelifications.

Correlation is a measure of dependence between statistical variables. A statistical variable is not quite the same as a programming variable. In statistics you typically have a set of *measurements*, and each variable is measured for every measurement. Correlation between variables is usually expressed as a value that ranges from -1 to 1. Zero correlation means the variables are not related. A correlation of 1 indicates that the two are perfectly related—if you know one, you also know the other. Negative 1 also means that the variables are perfectly related but are opposites—when one is true, the other is false.

To compute the measure of correlation between two Boolean variables, we can use the *phi coefficient* (φ). This is a formula whose input is a frequency table containing the number of times the different combinations of the variables were observed. The output of the formula is a number between -1 and 1 that describes the correlation.

We could take the event of eating pizza and put that in a frequency table like this, where each number indicates the number of times that combination occurred in our measurements.

	
No squirrel, no pizza 76	No squirrel, pizza 9
	
Squirrel, no pizza 4	Squirrel, pizza 1

If we call that table n , we can compute φ using the following formula:

$$\varphi = \frac{n_{11}n_{00} - n_{10}n_{01}}{\sqrt{n_{1\bullet}n_{0\bullet}n_{\bullet 1}n_{\bullet 0}}} \quad (4.1)$$

(If at this point you're putting the book down to focus on a terrible flashback to 10th grade math class—hold on! I do not intend to torture you with endless pages of cryptic notation—it's just this one formula for now. And even with this one, all we do is turn it into JavaScript.)

The notation n_{01} indicates the number of measurements where the first variable (squirrelness) is false (0) and the second variable (pizza) is true (1). In the pizza table, n_{01} is 9.

The value $n_{1\bullet}$ refers to the sum of all measurements where the first variable is true, which is 5 in the example table. Likewise, $n_{\bullet 0}$ refers to the sum of the measurements where the second variable is false.

So for the pizza table, the part above the division line (the dividend) would be $1 \times 76 - 4 \times 9 = 40$, and the part below it (the divisor) would be the square root of $5 \times 85 \times 10 \times 80$, or $\sqrt{340,000}$. This comes out to φ

≈ 0.069 , which is tiny. Eating pizza does not appear to have influence on the transformations.

COMPUTING CORRELATION

We can represent a two-by-two table in JavaScript with a four-element array (`[76, 9, 4, 1]`). We could also use other representations, such as an array containing two two-element arrays (`[[76, 9], [4, 1]]`) or an object with property names like `"11"` and `"01"`, but the flat array is simple and makes the expressions that access the table pleasantly short. We'll interpret the indices to the array as two-bit binary numbers, where the leftmost (most significant) digit refers to the squirrel variable and the rightmost (least significant) digit refers to the event variable. For example, the binary number `10` refers to the case where Jacques did turn into a squirrel, but the event (say, "pizza") didn't occur. This happened four times. And since binary `10` is 2 in decimal notation, we will store this number at index 2 of the array.

This is the function that computes the φ coefficient from such an array:

```
function phi(table) {
  return (table[3] * table[0] - table[2] * table[1]) /
    Math.sqrt((table[2] + table[3]) *
      (table[0] + table[1]) *
      (table[1] + table[3]) *
      (table[0] + table[2]));
}
```



```
console.log(phi([76, 9, 4, 1]));  
// → 0.068599434
```

This is a direct translation of the φ formula into JavaScript. `Math.sqrt` is the square root function, as provided by the `Math` object in a standard JavaScript environment. We have to add two fields from the table to get fields like n_i , because the sums of rows or columns are not stored directly in our data structure.

Jacques keeps his journal for three months. The resulting dataset is available in the coding sandbox for this chapter (<https://eloquentjavascript.org/4th>), where it is stored in the `JOURNAL` binding, and in a downloadable file.

To extract a two-by-two table for a specific event from the journal, we must loop over all the entries and tally how many times the event occurs in relation to squirrel transformations:

```
function tableFor(event, journal) {  
  let table = [0, 0, 0, 0];  
  for (let i = 0; i < journal.length; i++) {  
    let entry = journal[i], index = 0;  
    if (entry.events.includes(event)) index += 1;  
    if (entry.squirrel) index += 2;  
    table[index] += 1;  
  }  
  return table;  
}  
  
console.log(tableFor("pizza", JOURNAL));  
// → [76, 9, 4, 1]
```

Arrays have an `includes` method that checks whether a given value exists in the array. The function uses that to determine whether the event name it is interested in is part of the event list for a given day.

The body of the loop in `tableFor` figures out which box in the table each journal entry falls into by checking whether the entry contains the specific event it's interested in and whether the event happens alongside a squirrel incident. The loop then adds one to the correct box in the table.

We now have the tools we need to compute individual correlations. The only step remaining is to find a correlation for every type of event that was recorded and see whether anything stands out.

ARRAY LOOPS

In the `tableFor` function, there's a loop like this:

```
for (let i = 0; i < JOURNAL.length; i++) {  
  let entry = JOURNAL[i];  
  // Do something with entry  
}
```

This kind of loop is common in classical JavaScript—going over arrays one element at a time is something that comes up a lot, and to do that you'd run a counter over the length of the array and pick out each element in turn.

There is a simpler way to write such loops in modern JavaScript:

```
for (let entry of JOURNAL) {
```

```
    console.log(`${entry.events.length} events.`);
  }
}
```

When a `for` loop uses the word `of` after its variable definition, it will loop over the elements of the value given after `of`. This works not only for arrays but also for strings and some other data structures. We'll discuss *how* it works in Chapter 6.

THE FINAL ANALYSIS

We need to compute a correlation for every type of event that occurs in the dataset. To do that, we first need to *find* every type of event.

```
function journalEvents(journal) {
  let events = [];
  for (let entry of journal) {
    for (let event of entry.events) {
      if (!events.includes(event)) {
        events.push(event);
      }
    }
  }
  return events;
}
```

```
console.log(journalEvents(JOURNAL));
// → ["carrot", "exercise", "weekend", "bread", ...]
```

By adding any event names that aren't already in it to the `events` array,

the function collects every type of event.

Using that function, we can see all the correlations:

```
for (let event of journalEvents(JOURNAL)) {
  console.log(event + ":", phi(tableFor(event, JOURNAL)));
}
// → carrot:    0.0140970969
// → exercise:  0.0685994341
// → weekend:    0.1371988681
// → bread:     -0.0757554019
// → pudding:   -0.0648203724
// And so on...
```

Most correlations seem to lie close to zero. Eating carrots, bread, or pudding apparently does not trigger squirrel-lycanthropy. The transformations *do* seem to occur somewhat more often on weekends. Let's filter the results to show only correlations greater than 0.1 or less than -0.1:

```
for (let event of journalEvents(JOURNAL)) {
  let correlation = phi(tableFor(event, JOURNAL));
  if (correlation > 0.1 || correlation < -0.1) {
    console.log(event + ":", correlation);
  }
}
// → weekend:    0.1371988681
// → brushed teeth: -0.3805211953
// → candy:      0.1296407447
// → work:       -0.1371988681
// → spaghetti:  0.2425356250
```

```
// → reading:      0.1106828054
// → peanuts:      0.5902679812
```

Aha! There are two factors with a correlation clearly stronger than the others. Eating peanuts has a strong positive effect on the chance of turning into a squirrel, whereas brushing teeth has a significant negative effect.

Interesting. Let's try something.

```
for (let entry of JOURNAL) {
  if (entry.events.includes("peanuts") &&
      !entry.events.includes("brushed teeth")) {
    entry.events.push("peanut teeth");
  }
}
console.log(phi(tableFor("peanut teeth", JOURNAL)));
// → 1
```

That's a strong result. The phenomenon occurs precisely when Jacques eats peanuts and fails to brush his teeth. If only he weren't such a slob about dental hygiene, he'd never even have noticed his affliction.

Knowing this, Jacques stops eating peanuts altogether and finds that his transformations stop.

But it takes only a few months for him to notice that something is missing from this entirely human way of living. Without his feral adventures, Jacques hardly feels alive at all. He decides he'd rather be a full-time wild animal. After building a beautiful little tree house in the forest and equipping it with a peanut butter dispenser and a ten-year supply of peanut butter, he changes form one last time, and lives

the short and energetic life of a squirrel.

FURTHER ARRAYOLOGY

Before finishing the chapter, I want to introduce you to a few more object-related concepts. I'll start with some generally useful array methods.

We saw `push` and `pop`, which add and remove elements at the end of an array, [earlier](#) in this chapter. The corresponding methods for adding and removing things at the start of an array are called `unshift` and `shift`.

```
let todoList = [];
function remember(task) {
  todoList.push(task);
}
function getTask() {
  return todoList.shift();
}
function rememberUrgently(task) {
  todoList.unshift(task);
}
```

This program manages a queue of tasks. You add tasks to the end of the queue by calling `remember("groceries")`, and when you're ready to do something, you call `getTask()` to get (and remove) the front item from the queue. The `rememberUrgently` function also adds a task but adds it to the front instead of the back of the queue.

To search for a specific value, arrays provide an `indexOf` method. The method searches through the array from the start to the end and returns the index at which the requested value was found—or -1 if it wasn't found. To search from the end instead of the start, there's a similar method called `lastIndexOf`:

```
console.log([1, 2, 3, 2, 1].indexOf(2));
// → 1
console.log([1, 2, 3, 2, 1].lastIndexOf(2));
// → 3
```

Both `indexOf` and `lastIndexOf` take an optional second argument that indicates where to start searching.

Another fundamental array method is `slice`, which takes start and end indices and returns an array that has only the elements between them. The start index is inclusive and the end index is exclusive.

```
console.log([0, 1, 2, 3, 4].slice(2, 4));
// → [2, 3]
console.log([0, 1, 2, 3, 4].slice(2));
// → [2, 3, 4]
```

When the end index is not given, `slice` will take all of the elements after the start index. You can also omit the start index to copy the entire array.

The `concat` method can be used to append arrays together to create a new array, similar to what the `+` operator does for strings.

The following example shows both `concat` and `slice` in action. It takes an array and an index and returns a new array that is a copy of

the original array with the element at the given index removed:

```
function remove(array, index) {
  return array.slice(0, index)
    .concat(array.slice(index + 1));
}
console.log(remove(["a", "b", "c", "d", "e"], 2));
// → ["a", "b", "d", "e"]
```

If you pass `concat` an argument that is not an array, that value will be added to the new array as if it were a one-element array.

STRINGS AND THEIR PROPERTIES

We can read properties like `length` and `toUpperCase` from string values. But if we try to add a new property, it doesn't stick.

```
let kim = "Kim";
kim.age = 88;
console.log(kim.age);
// → undefined
```

Values of type string, number, and Boolean are not objects, and though the language doesn't complain if you try to set new properties on them, it doesn't actually store those properties. As mentioned earlier, such values are immutable and cannot be changed.

But these types do have built-in properties. Every string value has a number of methods. Some very useful ones are `slice` and `indexOf`, which resemble the array methods of the same name:

```

console.log("coconuts".slice(4, 7));
// → nut
console.log("coconut".indexOf("u"));
// → 5

```

One difference is that a string's `indexOf` can search for a string containing more than one character, whereas the corresponding array method looks only for a single element:

```

console.log("one two three".indexOf("ee"));
// → 11

```

The `trim` method removes whitespace (spaces, newlines, tabs, and similar characters) from the start and end of a string:

```

console.log("  okay \n  ".trim());
// → okay

```

The `zeroPad` function from the previous chapter also exists as a method. It is called `padStart` and takes the desired length and padding character as arguments:

```

console.log(String(6).padStart(3, "0"));
// → 006

```

You can split a string on every occurrence of another string with `split` and join it again with `join`:

```

let sentence = "Secretarybirds specialize in stomping";
let words = sentence.split(" ");
console.log(words);

```

```

// → ["Secretarybirds", "specialize", "in", "stomping"]
console.log(words.join(" "));
// → Secretarybirds. specialize. in. stomping

```

A string can be repeated with the `repeat` method, which creates a new string containing multiple copies of the original string, glued together:

```

console.log("LA".repeat(3));
// → LALALA

```

We have already seen the string type's `length` property. Accessing the individual characters in a string looks like accessing array elements (with a complication that we'll discuss in Chapter 5).

```

let string = "abc";
console.log(string.length);
// → 3
console.log(string[1]);
// → b

```

REST PARAMETERS

It can be useful for a function to accept any number of arguments. For example, `Math.max` computes the maximum of *all* the arguments it is given. To write such a function, you put three dots before the function's last parameter, like this:

```

function max(...numbers) {
  let result = -Infinity;

```

```

    for (let number of numbers) {
      if (number > result) result = number;
    }
    return result;
  }
  console.log(max(4, 1, 9, -2));
  // → 9

```

When such a function is called, the *rest parameter* is bound to an array containing all further arguments. If there are other parameters before it, their values aren't part of that array. When, as in `max`, it is the only parameter, it will hold all arguments.

You can use a similar three-dot notation to *call* a function with an array of arguments.

```

let numbers = [5, 1, 7];
console.log(max(...numbers));
// → 7

```

This “spreads” out the array into the function call, passing its elements as separate arguments. It is possible to include an array like that along with other arguments, as in `max(9, ...numbers, 2)`.

Square bracket array notation similarly allows the triple-dot operator to spread another array into the new array:

```

let words = ["never", "fully"];
console.log(["will", ...words, "understand"]);
// → ["will", "never", "fully", "understand"]

```

This works even in curly brace objects, where it adds all properties from

another object. If a property is added multiple times, the last value to be added wins:

```

let coordinates = {x: 10, y: 0};
console.log({...coordinates, y: 5, z: 1});
// → {x: 10, y: 5, z: 1}

```

THE MATH OBJECT

As we've seen, `Math` is a grab bag of number-related utility functions such as `Math.max` (maximum), `Math.min` (minimum), and `Math.sqrt` (square root).

The `Math` object is used as a container to group a bunch of related functionality. There is only one `Math` object, and it is almost never useful as a value. Rather, it provides a *namespace* so that all these functions and values do not have to be global bindings.

Having too many global bindings “pollutes” the namespace. The more names have been taken, the more likely you are to accidentally overwrite the value of some existing binding. For example, it's not unlikely you'll want to name something `max` in one of your programs. Since JavaScript's built-in `max` function is tucked safely inside the `Math` object, you don't have to worry about overwriting it.

Many languages will stop you, or at least warn you, when you are defining a binding with a name that is already taken. JavaScript does this for bindings you declared with `let` or `const` but—perversely—not for standard bindings nor for bindings declared with `var` or `function`.

Back to the `Math` object. If you need to do trigonometry, `Math` can help. It contains `cos` (cosine), `sin` (sine), and `tan` (tangent), as well as their inverse functions, `acos`, `asin`, and `atan`, respectively. The number π (pi)—or at least the closest approximation that fits in a JavaScript number—is available as `Math.PI`. There is an old programming tradition of writing the names of constant values in all caps.

```
function randomPointOnCircle(radius) {
  let angle = Math.random() * 2 * Math.PI;
  return {x: radius * Math.cos(angle),
        y: radius * Math.sin(angle)};
}
console.log(randomPointOnCircle(2));
// → {x: 0.3667, y: 1.966}
```

If you're not familiar with sines and cosines, don't worry. I'll explain them when they are used in Chapter 14.

The previous example used `Math.random`. This is a function that returns a new pseudorandom number between 0 (inclusive) and 1 (exclusive) every time you call it:

```
console.log(Math.random());
// → 0.36993729369714856
console.log(Math.random());
// → 0.727367032552138
console.log(Math.random());
// → 0.40180766698904335
```

Though computers are deterministic machines—they always react the same way if given the same input—it is possible to have them produce

numbers that appear random. To do that, the machine keeps some hidden value, and whenever you ask for a new random number, it performs complicated computations on this hidden value to create a new value. It stores a new value and returns some number derived from it. That way, it can produce ever new, hard-to-predict numbers in a way that *seems* random.

If we want a whole random number instead of a fractional one, we can use `Math.floor` (which rounds down to the nearest whole number) on the result of `Math.random`:

```
console.log(Math.floor(Math.random() * 10));
// → 2
```

Multiplying the random number by 10 gives us a number greater than or equal to 0 and below 10. Since `Math.floor` rounds down, this expression will produce, with equal chance, any number from 0 through 9.

There are also the functions `Math.ceil` (for “ceiling”, which rounds up to a whole number), `Math.round` (to the nearest whole number), and `Math.abs`, which takes the absolute value of a number, meaning it negates negative values but leaves positive ones as they are.

DESTRUCTURING

Let's return to the phi function for a moment.

```
function phi(table) {
  return (table[3] * table[0] - table[2] * table[1]) /
    Math.sqrt((table[2] * table[3]) *
```

```

        (table[0] + table[1]) *
        (table[1] + table[3]) *
        (table[0] + table[2]));
    }

```

One reason this function is awkward to read is that we have a binding pointing at our array, but we'd much prefer to have bindings for the *elements* of the array—that is, let `n00 = table[0]` and so on. Fortunately, there is a succinct way to do this in JavaScript:

```

function phi([n00, n01, n10, n11]) {
    return (n11 * n00 + n10 * n01) /
        Math.sqrt((n10 + n11) * (n00 + n01) *
            (n01 + n11) * (n00 + n10));
}

```

This also works for bindings created with `let`, `var`, or `const`. If you know that the value you are binding is an array, you can use square brackets to “look inside” of the value, binding its contents.

A similar trick works for objects, using braces instead of square brackets.

```

let {name} = {name: "Faraji", age: 23};
console.log(name);
// → Faraji

```

Note that if you try to destructure `null` or `undefined`, you get an error, much as you would if you directly try to access a property of those values.

OPTIONAL PROPERTY ACCESS

When you aren't sure whether a given value produces an object, but still want to read a property from it when it does, you can use a variant of the dot notation: `object?.property`.

```

function city(object) {
    return object.address?.city;
}
console.log(city({address: {city: "Toronto"}}));
// → Toronto
console.log(city({name: "Vera"}));
// → undefined

```

The expression `a?.b` means the same as `a.b` when `a` isn't null or undefined. When it is, it evaluates to `undefined`. This can be convenient when, as in the example, you aren't sure that a given property exists or when a variable might hold an undefined value.

A similar notation can be used with square bracket access, and even with function calls, by putting `?` in front of the parentheses or brackets:

```

console.log("string".notAMethod?.());
// → undefined
console.log({}.arrayProp?.[0]);
// → undefined

```


JSON

Because properties grasp their value rather than contain it, objects and arrays are stored in the computer's memory as sequences of bits holding the *addresses*—the place in memory—of their contents. An array with another array inside of it consists of (at least) one memory region for the inner array and another for the outer array, containing (among other things) a number that represents the address of the inner array.

If you want to save data in a file for later or send it to another computer over the network, you have to somehow convert these tangles of memory addresses to a description that can be stored or sent. You *could* send over your entire computer memory along with the address of the value you're interested in, I suppose, but that doesn't seem like the best approach.

What we can do is *serialize* the data. That means it is converted into a flat description. A popular serialization format is called *JSON* (pronounced “Jason”), which stands for JavaScript Object Notation. It is widely used as a data storage and communication format on the web, even with languages other than JavaScript.

JSON looks similar to JavaScript's way of writing arrays and objects, with a few restrictions. All property names have to be surrounded by double quotes, and only simple data expressions are allowed—no function calls, bindings, or anything that involves actual computation. Comments are not allowed in JSON.

A journal entry might look like this when represented as JSON data:

```
{
```

```
"squirrel": false,
"events": ["work", "touched tree", "pizza", "running"]
}
```

JavaScript gives us the functions `JSON.stringify` and `JSON.parse` to convert data to and from this format. The first takes a JavaScript value and returns a JSON-encoded string. The second takes such a string and converts it to the value it encodes:

```
let string = JSON.stringify({squirrel: false,
                             events: ["weekend"]});
console.log(string);
// → {"squirrel":false,"events":["weekend"]}
console.log(JSON.parse(string).events);
// → ["weekend"]
```

SUMMARY

Objects and arrays provide ways to group several values into a single value. This allows us to put a bunch of related things in a bag and run around with the bag instead of wrapping our arms around all of the individual things and trying to hold on to them separately.

Most values in JavaScript have properties, with the exceptions being `null` and `undefined`. Properties are accessed using `value.prop` or `value["prop"]`. Objects tend to use names for their properties and store more or less a fixed set of them. Arrays, on the other hand, usually contain varying amounts of conceptually identical values and use numbers (starting from 0) as the names of their properties.

There *are* some named properties in arrays, such as `length` and a number of methods. Methods are functions that live in properties and (usually) act on the value of which they are a property.

You can iterate over arrays using a special kind of `for` loop: `for (let element of array)`.

EXERCISES

THE SUM OF A RANGE

The introduction of this book alluded to the following as a nice way to compute the sum of a range of numbers:

```
console.log(sum(range(1, 10)));
```

Write a `range` function that takes two arguments, `start` and `end`, and returns an array containing all the numbers from `start` up to and including `end`.

Next, write a `sum` function that takes an array of numbers and returns the sum of these numbers. Run the example program and see whether it does indeed return 55.

As a bonus assignment, modify your `range` function to take an optional third argument that indicates the “step” value used when building the array. If no step is given, the elements should go up by increments of one, corresponding to the old behavior. The function call `range(1, 10, 2)` should return `[1, 3, 5, 7, 9]`. Make sure this also works with negative step values so that `range(5, 2, -1)` produces `[5, 4, 3, 2]`.

REVERSING AN ARRAY

Arrays have a `reverse` method that changes the array by inverting the order in which its elements appear. For this exercise, write two functions, `reverseArray` and `reverseArrayInPlace`. The first, `reverseArray`, should take an array as its argument and produce a *new* array that has the same elements in the inverse order. The second, `reverseArrayInPlace`, should do what the `reverse` method does: *modify* the array given as its argument by reversing its elements. Neither may use the standard `reverse` method.

Thinking back to the notes about side effects and pure functions in the [previous chapter](#), which variant do you expect to be useful in more situations? Which one runs faster?

A LIST

As generic blobs of values, objects can be used to build all sorts of data structures. A common data structure is the *list* (not to be confused with arrays). A list is a nested set of objects, with the first object holding a reference to the second, the second to the third, and so on:

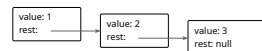
```
let list = {
  value: 1,
  rest: {
    value: 2,
    rest: {
      value: 3,
      rest: null
    }
  }
}
```

```

    }
  };

```

The resulting objects form a chain, as shown in the following diagram:



A nice thing about lists is that they can share parts of their structure. For example, if I create two new values (`{value: 0, rest: list}`) and (`{value: -1, rest: list}`) (with `list` referring to the binding defined earlier), they are both independent lists, but they share the structure that makes up their last three elements. The original list is also still a valid three-element list.

Write a function `arrayToList` that builds up a list structure like the one shown when given `[1, 2, 3]` as argument. Also write a `listToArray` function that produces an array from a list. Add the helper functions `prepend`, which takes an element and a list and creates a new list that adds the element to the front of the input list, and `nth`, which takes a list and a number and returns the element at the given position in the list (with zero referring to the first element) or `undefined` when there is no such element.

If you haven't already, also write a recursive version of `nth`.

DEEP COMPARISON

The `==` operator compares objects by identity, but sometimes you'd prefer to compare the values of their actual properties.

Write a function `deepEqual` that takes two values and returns `true` only if they are the same value or are objects with the same properties, where the values of the properties are equal when compared with a recursive call to `deepEqual`.

To find out whether values should be compared directly (using the `===` operator for that) or have their properties compared, you can use the `typeof` operator. If it produces `"object"` for both values, you should do a deep comparison. But you have to take one silly exception into account: because of a historical accident, `typeof null` also produces `"object"`.

The `Object.keys` function will be useful when you need to go over the properties of objects to compare them.

"

There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies."

—C.A.R. Hoare, 1980 ACM Turing Award Lecture

CHAPTER 5

HIGHER-ORDER FUNCTIONS

A large program is a costly program, and not just because of the time it takes to build. Size almost always involves complexity, and complexity confuses programmers. Confused programmers, in turn, introduce mistakes (*bugs*) into programs. A large program then provides a lot of space for these bugs to hide, making them hard to find.

Let's briefly go back to the final two example programs in the introduction. The first is self contained and six lines long.

```
let total = 0, count = 1;
while (count <= 10) {
  total += count;
  count += 1;
}
console.log(total);
```

The second relies on two external functions and is one line long.

```
console.log(sum(range(1, 10)));
```

Which one is more likely to contain a bug?

If we count the size of the definitions of `sum` and `range`, the second program is also big—even bigger than the first. But still, I'd argue that it is more likely to be correct.

This is because the solution is expressed in a vocabulary that corresponds to the problem being solved. Summing a range of numbers isn't about loops and counters. It is about ranges and sums.

The definitions of this vocabulary (the functions `sum` and `range`) will still involve loops, counters, and other incidental details. But because they are expressing simpler concepts than the program as a whole, they are easier to get right.

ABSTRACTION

In the context of programming, these kinds of vocabularies are usually called *abstractions*. Abstractions give us the ability to talk about problems at a higher (or more abstract) level, without getting sidetracked by uninteresting details.

As an analogy, compare these two recipes for pea soup. The first goes like this:

Put 1 cup of dried peas per person into a container. Add water until the peas are well covered. Leave the peas in water for at least 12 hours. Take the peas out of the water and put them in a cooking pan. Add 4 cups of water per person. Cover the pan and keep the peas simmering for two hours. Take half an onion per person. Cut it into pieces with a knife. Add it to the peas. Take a stalk of celery per

person. Cut it into pieces with a knife. Add it to the peas.
Take a carrot per person. Cut it into pieces. With a knife!
Add it to the peas. Cook for 10 more minutes.

And this is the second recipe:

Per person: 1 cup dried split peas, 4 cups of water, half a
chopped onion, a stalk of celery, and a carrot.

Soak peas for 12 hours. Simmer for 2 hours. Chop and add
vegetables. Cook for 10 more minutes.

The second is shorter and easier to interpret. But you do need to
understand a few more cooking-related words such as *soak*, *simmer*,
chop, and, I guess, *vegetable*.

When programming, we can't rely on all the words we need to be
waiting for us in the dictionary. Thus, we might fall into the pattern of
the first recipe—work out the precise steps the computer has to perform,
one by one, blind to the higher-level concepts they express.

It is a useful skill, in programming, to notice when you are working
at too low a level of abstraction.

ABSTRACTING REPETITION

Plain functions, as we've seen them so far, are a good way to build
abstractions. But sometimes they fall short.

It is common for a program to do something a given number of times.
You can write a `for` loop for that, like this:

```
for (let i = 0; i < 10; i++) {  
  console.log(i);  
}
```

Can we abstract “doing something N times” as a function? Well, it's
easy to write a function that calls `console.log` N times.

```
function repeatLog(n) {  
  for (let i = 0; i < n; i++) {  
    console.log(i);  
  }  
}
```

But what if we want to do something other than logging the numbers?
Since “doing something” can be represented as a function and functions
are just values, we can pass our action as a function value.

```
function repeat(n, action) {  
  for (let i = 0; i < n; i++) {  
    action(i);  
  }  
}  
  
repeat(3, console.log);  
// → 0  
// → 1  
// → 2
```

We don't have to pass a predefined function to `repeat`. Often, it is easier
to create a function value on the spot instead.

```

let labels = [];
repeat(5, i => {
  labels.push(`Unit ${i + 1}`);
});
console.log(labels);
// → ["Unit 1", "Unit 2", "Unit 3", "Unit 4", "Unit 5"]

```

This is structured a little like a `for` loop—it first describes the kind of loop and then provides a body. However, the body is now written as a function value, which is wrapped in the parentheses of the call to `repeat`. This is why it has to be closed with the closing brace *and* closing parenthesis. In cases like this example, where the body is a single small expression, you could also omit the braces and write the loop on a single line.

HIGHER-ORDER FUNCTIONS

Functions that operate on other functions, either by taking them as arguments or by returning them, are called *higher-order functions*. Since we have already seen that functions are regular values, there is nothing particularly remarkable about the fact that such functions exist. The term comes from mathematics, where the distinction between functions and other values is taken more seriously.

Higher-order functions allow us to abstract over *actions*, not just values. They come in several forms. For example, we can have functions that create new functions.

```

function greaterThan(n) {

```

```

  return m => m > n;
}
let greaterThan10 = greaterThan(10);
console.log(greaterThan10(11));
// → true

```

We can also have functions that change other functions.

```

function noisy(f) {
  return (...args) => {
    console.log("calling with", args);
    let result = f(...args);
    console.log("called with", args, ", returned", result);
    return result;
  };
}
noisy(Math.min)(3, 2, 1);
// → calling with [3, 2, 1]
// → called with [3, 2, 1] , returned 1

```

We can even write functions that provide new types of control flow.

```

function unless(test, then) {
  if (!test) then();
}

repeat(3, n => {
  unless(n % 2 == 1, () => {
    console.log(n, "is even");
  });
});

```

```
// → 0 is even
// → 2 is even
```

There is a built-in array method, `forEach`, that provides something like a `for/of` loop as a higher-order function.

```
["A", "B"].forEach(1 => console.log(1));
// → A
// → B
```

SCRIPT DATASET

One area where higher-order functions shine is data processing. To process data, we'll need some actual example data. This chapter will use a dataset about scripts—writing systems such as Latin, Cyrillic, or Arabic.

Remember Unicode, the system that assigns a number to each character in written language, from Chapter 1? Most of these characters are associated with a specific script. The standard contains 140 different scripts, of which 81 are still in use today and 59 are historic.

Though I can fluently read only Latin characters, I appreciate the fact that people are writing texts in at least 80 other writing systems, many of which I wouldn't even recognize. For example, here's a sample of Tamil handwriting:

இன்ன செத்தார பூஞ்சல் அவர்நாண
நன்னாயம் செஞ்சு விடல்.

The example dataset contains some pieces of information about the 140 scripts defined in Unicode. It is available in the coding sandbox for this chapter (<https://eloquentjavascript.net/code#5>) as the `SCRIPTS` binding. The binding contains an array of objects, each of which describes a script.

```
{
  name: "Coptic",
  ranges: [[994, 1008], [11392, 11508], [11513, 11520]],
  direction: "ltr",
  year: ~200,
  living: false,
  link: "https://en.wikipedia.org/wiki/Coptic_alphabet"
}
```

Such an object tells us the name of the script, the Unicode ranges assigned to it, the direction in which it is written, the (approximate) origin time, whether it is still in use, and a link to more information. The direction may be "ltr" for left to right, "rtl" for right to left (the way Arabic and Hebrew text are written), or "ttb" for top to bottom (as with Mongolian writing).

The `ranges` property contains an array of Unicode character ranges, each of which is a two-element array containing a lower bound and an upper bound. Any character codes within these ranges are assigned to

the script. The lower bound is inclusive (code 994 is a Coptic character) and the upper bound is noninclusive (code 1008 isn't).

FILTERING ARRAYS

If we want to find the scripts in the dataset that are still in use, the following function might be helpful. It filters out elements in an array that don't pass a test.

```
function filter(array, test) {
  let passed = [];
  for (let element of array) {
    if (test(element)) {
      passed.push(element);
    }
  }
  return passed;
}

console.log(filter(SCRIPTS, script => script.living));
// → [{name: "Adlam", ...}, ...]
```

The function uses the argument named `test`, a function value, to fill a “gap” in the computation—the process of deciding which elements to collect.

Note how the `filter` function, rather than deleting elements from the existing array, builds up a new array with only the elements that pass the test. This function is *pure*. It does not modify the array it is given.

Like `forEach`, `filter` is a standard array method. The example defined

the function only to show what it does internally. From now on, we'll use it like this instead:

```
console.log(SCRIPTS.filter(s => s.direction == "ttb"));
// → [{name: "Mongolian", ...}, ...]
```

TRANSFORMING WITH MAP

Say we have an array of objects representing scripts, produced by filtering the `SCRIPTS` array somehow. We want an array of names instead, which is easier to inspect.

The `map` method transforms an array by applying a function to all of its elements and building a new array from the returned values. The new array will have the same length as the input array, but its content will have been *mapped* to a new form by the function.

```
function map(array, transform) {
  let mapped = [];
  for (let element of array) {
    mapped.push(transform(element));
  }
  return mapped;
}

let rtlScripts = SCRIPTS.filter(s => s.direction == "rtl");
console.log(map(rtlScripts, s => s.name));
// → ["Adlam", "Arabic", "Imperial Aramaic", ...]
```

Like `forEach` and `filter`, `map` is a standard array method.

SUMMARIZING WITH REDUCE

Another common thing to do with arrays is to compute a single value from them. Our recurring example, summing a collection of numbers, is an instance of this. Another example is finding the script with the most characters.

The higher-order operation that represents this pattern is called *reduce* (sometimes also called *fold*). It builds a value by repeatedly taking a single element from the array and combining it with the current value. When summing numbers, you'd start with the number zero and, for each element, add that to the sum.

The parameters to `reduce` are, apart from the array, a combining function and a start value. This function is a little less straightforward than `filter` and `map`, so take a close look at it:

```
function reduce(array, combine, start) {
  let current = start;
  for (let element of array) {
    current = combine(current, element);
  }
  return current;
}

console.log(reduce([1, 2, 3, 4], (a, b) => a + b, 0));
// → 10
```

The standard array method `reduce`, which of course corresponds to this function, has an added convenience. If your array contains at least one element, you are allowed to leave off the `start` argument. The

method will take the first element of the array as its start value and start reducing at the second element.

```
console.log([1, 2, 3, 4].reduce((a, b) => a + b));
// → 10
```

To use `reduce` (twice) to find the script with the most characters, we can write something like this:

```
function characterCount(script) {
  return script.ranges.reduce((count, [from, to]) => {
    return count + (to - from);
  }, 0);
}

console.log(SCRIPTS.reduce((a, b) => {
  return characterCount(a) < characterCount(b) ? b : a;
}));
// → {name: "Han", ...}
```

The `characterCount` function reduces the ranges assigned to a script by summing their sizes. Note the use of destructuring in the parameter list of the reducer function. The second call to `reduce` then uses this to find the largest script by repeatedly comparing two scripts and returning the larger one.

The Han script has more than 89,000 characters assigned to it in the Unicode standard, making it by far the biggest writing system in the dataset. Han is a script sometimes used for Chinese, Japanese, and Korean text. Those languages share a lot of characters, though they tend to write them differently. The (US-based) Unicode Consortium

decided to treat them as a single writing system to save character codes. This is called *Han unification* and still makes some people very angry.

COMPOSABILITY

Consider how we would have written the previous example (finding the biggest script) without higher-order functions. The code is not that much worse.

```
let biggest = null;
for (let script of SCRIPTS) {
  if (biggest == null ||
      characterCount(biggest) < characterCount(script)) {
    biggest = script;
  }
}
console.log(biggest);
// → {name: "Han", ...}
```

There are a few more bindings, and the program is four lines longer, but it is still very readable.

The abstractions these functions provide really shine when you need to *compose* operations. As an example, let's write code that finds the average year of origin for living and dead scripts in the dataset.

```
function average(array) {
  return array.reduce((a, b) => a + b) / array.length;
}
```

```
console.log(Math.round(average(
  SCRIPTS.filter(s => s.living).map(s => s.year))));
// → 1165
console.log(Math.round(average(
  SCRIPTS.filter(s => !s.living).map(s => s.year))));
// → 284
```

As you can see, the dead scripts in Unicode are, on average, older than the living ones. This is not a terribly meaningful or surprising statistic. But I hope you'll agree that the code used to compute it isn't hard to read. You can see it as a pipeline: we start with all scripts, filter out the living (or dead) ones, take the years from those, average them, and round the result.

You could definitely also write this computation as one big loop.

```
let total = 0, count = 0;
for (let script of SCRIPTS) {
  if (script.living) {
    total += script.year;
    count += 1;
  }
}
console.log(Math.round(total / count));
// → 1165
```

However, it is harder to see what was being computed and how. And because intermediate results aren't represented as coherent values, it'd be a lot more work to extract something like *average* into a separate function.

In terms of what the computer is actually doing, these two approaches

are also quite different. The first will build up new arrays when running `filter` and `map`, whereas the second computes only some numbers, doing less work. You can usually afford the readable approach, but if you're processing huge arrays and doing so many times, the less abstract style might be worth the extra speed.

STRINGS AND CHARACTER CODES

One interesting use of this dataset would be figuring out what script a piece of text is using. Let's go through a program that does this.

Remember that each script has an array of character code ranges associated with it. Given a character code, we could use a function like this to find the corresponding script (if any):

```
function characterScript(code) {
  for (let script of SCRIPTS) {
    if (script.ranges.some([from, to]) => {
      return code >= from && code < to;
    }) {
      return script;
    }
  }
  return null;
}
```

```
console.log(characterScript(121));
// → {name: "Latin", ...}
```

The `some` method is another higher-order function. It takes a test func-

tion and tells you whether that function returns true for any of the elements in the array.

But how do we get the character codes in a string?

In Chapter 1 I mentioned that JavaScript strings are encoded as a sequence of 16-bit numbers. These are called *code units*. A Unicode character code was initially supposed to fit within such a unit (which gives you a little over 65,000 characters). When it became clear that wasn't going to be enough, many people balked at the need to use more memory per character. To address these concerns, UTF-16, the format also used by JavaScript strings, was invented. It describes most common characters using a single 16-bit code unit but uses a pair of two such units for others.

UTF-16 is generally considered a bad idea today. It seems almost intentionally designed to invite mistakes. It's easy to write programs that pretend code units and characters are the same thing. And if your language doesn't use two-unit characters, that will appear to work just fine. But as soon as someone tries to use such a program with some less common Chinese characters, it breaks. Fortunately, with the advent of emoji, everybody has started using two-unit characters, and the burden of dealing with such problems is more fairly distributed.

Unfortunately, obvious operations on JavaScript strings, such as getting their length through the `length` property and accessing their content using square brackets, deal only with code units.

```
// Two emoji characters, horse and shoe
let horseShoe = "🐎👢";
console.log(horseShoe.length);
```

```
// → 4
console.log(horseShoe[0]);
// → (Invalid half-character)
console.log(horseShoe.charCodeAt(0));
// → 55357 (Code of the half-character)
console.log(horseShoe.codePointAt(0));
// → 128052 (Actual code for horse emoji)
```

JavaScript's `charCodeAt` method gives you a code unit, not a full character code. The `codePointAt` method, added later, does give a full Unicode character, so we could use that to get characters from a string. But the argument passed to `codePointAt` is still an index into the sequence of code units. To run over all characters in a string, we'd still need to deal with the question of whether a character takes up one or two code units.

In the [previous chapter](#), I mentioned that a *for/of* loop can also be used on strings. Like `codePointAt`, this type of loop was introduced at a time when people were acutely aware of the problems with UTF-16. When you use it to loop over a string, it gives you real characters, not code units.

```
let roseDragon = "🐉🐲";
for (let char of roseDragon) {
  console.log(char);
}
// → 🐉
// → 🐲
```

If you have a character (which will be a string of one or two code units),

you can use `codePointAt(0)` to get its code.

RECOGNIZING TEXT

We have a `characterScript` function and a way to correctly loop over characters. The next step is to count the characters that belong to each script. The following counting abstraction will be useful there:

```
function countBy(items, groupName) {
  let counts = [];
  for (let item of items) {
    let name = groupName(item);
    let known = counts.find(c => c.name == name);
    if (!known) {
      counts.push({name, count: 1});
    } else {
      known.count++;
    }
  }
  return counts;
}

console.log(countBy([1, 2, 3, 4, 5], n => n > 2));
// → [{name: false, count: 2}, {name: true, count: 3}]
```

The `countBy` function expects a collection (anything that we can loop over with *for/of*) and a function that computes a group name for a given element. It returns an array of objects, each of which names a group and tells you the number of elements that were found in that

group.

It uses another array method, `find`, which goes over the elements in the array and returns the first one for which a function returns true. It returns `undefined` when it finds no such element.

Using `countBy`, we can write the function that tells us which scripts are used in a piece of text.

```
function textScripts(text) {
  let scripts = countBy(text, char => {
    let script = characterScript(char.codePointAt(0));
    return script ? script.name : "none";
  }).filter(({name}) => name !== "none");

  let total = scripts.reduce((n, {count}) => n + count, 0);
  if (total === 0) return "No scripts found";

  return scripts.map(({name, count}) => {
    return `${Math.round(count * 100 / total)}% ${name}`;
  }).join(", ");
}

console.log(textScripts('英国的狗说"woof", 俄罗斯的狗说"гав"'))
// → 61% Han, 22% Latin, 17% Cyrillic
```

The function first counts the characters by name, using `characterScript` to assign them a name and falling back to the string `"none"` for characters that aren't part of any script. The `filter` call drops the entry for `"none"` from the resulting array, since we aren't interested in those

characters.

To be able to compute percentages, we first need the total number of characters that belong to a script, which we can compute with `reduce`. If we find no such characters, the function returns a specific string. Otherwise, it transforms the counting entries into readable strings with `map` and then combines them with `join`.

SUMMARY

Being able to pass function values to other functions is a deeply useful aspect of JavaScript. It allows us to write functions that model computations with "gaps" in them. The code that calls these functions can fill in the gaps by providing function values.

Arrays provide a number of useful higher-order methods. You can use `forEach` to loop over the elements in an array. The `filter` method returns a new array containing only the elements that pass the predicate function. You can transform an array by putting each element through a function using `map`. You can use `reduce` to combine all the elements in an array into a single value. The `some` method tests whether any element matches a given predicate function, while `find` finds the first element that matches a predicate.

EXERCISES

FLATTENING

Use the `reduce` method in combination with the `concat` method to “flatten” an array of arrays into a single array that has all the elements of the original arrays.

YOUR OWN LOOP

Write a higher-order function `loop` that provides something like a `for` loop statement. It should take a value, a test function, an update function, and a body function. Each iteration, it should first run the test function on the current loop value and stop if that returns `false`. It should then call the body function, giving it the current value, and finally call the update function to create a new value and start over from the beginning.

When defining the function, you can use a regular loop to do the actual looping.

EVERYTHING

Arrays also have an `every` method analogous to the `some` method. This method returns `true` when the given function returns `true` for *every* element in the array. In a way, `some` is a version of the `||` operator that acts on arrays, and `every` is like the `&&` operator.

Implement `every` as a function that takes an array and a predicate function as parameters. Write two versions, one using a loop and one

using the `some` method.

DOMINANT WRITING DIRECTION

Write a function that computes the dominant writing direction in a string of text. Remember that each script object has a `direction` property that can be “ltr” (left to right), “rtl” (right to left), or “ttb” (top to bottom).

The dominant direction is the direction of a majority of the characters that have a script associated with them. The `characterScript` and `countBy` functions defined earlier in the chapter are probably useful here.

“An abstract data type is realized by writing a special kind of program [...] which defines the type in terms of the operations which can be performed on it.”

—Barbara Liskov, Programming with Abstract Data Types

CHAPTER 6

THE SECRET LIFE OF OBJECTS

Chapter 4 introduced JavaScript’s objects as containers that hold other data. In programming culture, *object-oriented programming* is a set of techniques that use objects as the central principle of program organization. Though no one really agrees on its precise definition, object-oriented programming has shaped the design of many programming languages, including JavaScript. This chapter describes the way these ideas can be applied in JavaScript.

ABSTRACT DATA TYPES

The main idea in object-oriented programming is to use objects, or rather *types* of objects, as the unit of program organization. Setting up a program as a number of strictly separated object types provides a way to think about its structure and thus to enforce some kind of discipline, preventing everything from becoming entangled.

The way to do this is to think of objects somewhat like you’d think of an electric mixer or other consumer appliance. The people who design

and assemble a mixer have to do specialized work requiring material science and understanding of electricity. They cover all that up in a smooth plastic shell so that the people who only want to mix pancake batter don’t have to worry about all that—they have to understand only the few knobs that the mixer can be operated with.

Similarly, an *abstract data type*, or *object class*, is a subprogram that may contain arbitrarily complicated code but exposes a limited set of methods and properties that people working with it are supposed to use. This allows large programs to be built up out of a number of appliance types, limiting the degree to which these different parts are entangled by requiring them to only interact with each other in specific ways.

If a problem is found in one such object class, it can often be repaired or even completely rewritten without impacting the rest of the program. Even better, it may be possible to use object classes in multiple different programs, avoiding the need to recreate their functionality from scratch. You can think of JavaScript’s built-in data structures, such as arrays and strings, as such reusable abstract data types.

Each abstract data type has an *interface*, the collection of operations that external code can perform on it. Any details beyond that interface are *encapsulated*, treated as internal to the type and of no concern to the rest of the program.

Even basic things like numbers can be thought of as an abstract data type whose interface allows us to add them, multiply them, compare them, and so on. In fact, the fixation on single *objects* as the main unit of organization in classical object-oriented programming is somewhat

unfortunate since useful pieces of functionality often involve a group of different object classes working closely together.

METHODS

In JavaScript, methods are nothing more than properties that hold function values. This is a simple method:

```
function speak(line) {
  console.log(`The ${this.type} rabbit says '${line}'`);
}
let whiteRabbit = {type: "white", speak};
let hungryRabbit = {type: "hungry", speak};

whiteRabbit.speak("Oh my fur and whiskers");
// → The white rabbit says 'Oh my fur and whiskers'
hungryRabbit.speak("Got any carrots?");
// → The hungry rabbit says 'Got any carrots?'
```

Typically a method needs to do something with the object on which it was called. When a function is called as a method—looked up as a property and immediately called, as in `object.method()`—the binding called `this` in its body automatically points at the object on which it was called.

You can think of this as an extra parameter that is passed to the function in a different way than regular parameters. If you want to provide it explicitly, you can use a function's `call` method, which takes the `this` value as its first argument and treats further arguments as

normal parameters.

```
speak.call(whiteRabbit, "Hurry");
// → The white rabbit says 'Hurry'
```

Since each function has its own `this` binding whose value depends on the way it is called, you cannot refer to the `this` of the wrapping scope in a regular function defined with the `function` keyword.

Arrow functions are different—they do not bind their own `this` but can see the `this` binding of the scope around them. Thus, you can do something like the following code, which references `this` from inside a local function:

```
let finder = {
  find(array) {
    return array.some(v => v == this.value);
  },
  value: 5
};
console.log(finder.find([4, 5]));
// → true
```

A property like `find(array)` in an object expression is a shorthand way of defining a method. It creates a property called `find` and gives it a function as its value.

If I had written the argument to `some` using the `function` keyword, this code wouldn't work.

PROTOTYPES

One way to create a rabbit object type with a `speak` method would be to create a helper function that has a rabbit type as its parameter and returns an object holding that as its `type` property and our `speak` function in its `speak` property.

All rabbits share that same method. Especially for types with many methods, it would be nice if there were a way to keep a type's methods in a single place, rather than adding them to each object individually.

In JavaScript, *prototypes* are the way to do that. Objects can be linked to other objects, to magically get all the properties that other object has. Plain old objects created with `{}` notation are linked to an object called `Object.prototype`.

```
let empty = {};  
console.log(empty.toString);  
// → function toString()_{}  
console.log(empty.toString());  
// → [object Object]
```

It looks like we just pulled a property out of an empty object. But in fact, `toString` is a method stored in `Object.prototype`, meaning it is available in most objects.

When an object gets a request for a property that it doesn't have, its prototype will be searched for the property. If that doesn't have it, the *prototype's* prototype is searched, and so on until an object without prototype is reached (`Object.prototype` is such an object).

```
console.log(Object.getPrototypeOf({}) == Object.prototype);
```

```
// → true  
console.log(Object.getPrototypeOf(Object.prototype));  
// → null
```

As you'd guess, `Object.getPrototypeOf` returns the prototype of an object.

Many objects don't directly have `Object.prototype` as their prototype but instead have another object that provides a different set of default properties. Functions derive from `Function.prototype` and arrays derive from `Array.prototype`.

```
console.log(Object.getPrototypeOf(Math.max) ==  
              Function.prototype);  
// → true  
console.log(Object.getPrototypeOf([]) == Array.prototype);  
// → true
```

Such a prototype object will itself have a prototype, often `Object.prototype`, so that it still indirectly provides methods like `toString`.

You can use `Object.create` to create an object with a specific prototype.

```
let protoRabbit = {  
  speak(line) {  
    console.log(`The ${this.type} rabbit says '${line}'`);  
  }  
};  
let blackRabbit = Object.create(protoRabbit);  
blackRabbit.type = "black";  
blackRabbit.speak("I am fear and darkness");
```

```
// → The black rabbit says 'I am fear and darkness'
```

The “proto” rabbit acts as a container for the properties shared by all rabbits. An individual rabbit object, like the black rabbit, contains properties that apply only to itself—in this case its type—and derives shared properties from its prototype.

CLASSES

JavaScript’s prototype system can be interpreted as a somewhat free-form take on abstract data types or classes. A *class* defines the shape of a type of object—what methods and properties it has. Such an object is called an *instance* of the class.

Prototypes are useful for defining properties for which all instances of a class share the same value. Properties that differ per instance, such as our rabbits’ `type` property, need to be stored directly in the objects themselves.

To create an instance of a given class, you have to make an object that derives from the proper prototype, but you *also* have to make sure it itself has the properties that instances of this class are supposed to have. This is what a *constructor* function does.

```
function makeRabbit(type) {  
  let rabbit = Object.create(protoRabbit);  
  rabbit.type = type;  
  return rabbit;  
}
```

JavaScript’s class notation makes it easier to define this type of function, along with a prototype object.

```
class Rabbit {  
  constructor(type) {  
    this.type = type;  
  }  
  speak(line) {  
    console.log(`The ${this.type} rabbit says '${line}'`);  
  }  
}
```

The `class` keyword starts a class declaration, which allows us to define a constructor and a set of methods together. Any number of methods may be written inside the declaration’s braces. This code has the effect of defining a binding called `Rabbit`, which holds a function that runs the code in `constructor` and has a `prototype` property that holds the `speak` method.

This function cannot be called like a normal function. Constructors, in JavaScript, are called by putting the keyword `new` in front of them. Doing so creates a fresh instance object whose prototype is the object from the function’s `prototype` property, then runs the function with `this` bound to the new object, and finally returns the object.

```
let killerRabbit = new Rabbit("killer");
```

In fact, `class` was only introduced in the 2015 edition of JavaScript. Any function can be used as a constructor, and before 2015, the way to define a class was to write a regular function and then manipulate its

prototype property.

```
function ArchaicRabbit(type) {
  this.type = type;
}
ArchaicRabbit.prototype.speak = function(line) {
  console.log(`The ${this.type} rabbit says '${line}'`);
};
let oldSchoolRabbit = new ArchaicRabbit("old school");
```

For this reason, all non-arrow functions start with a prototype property holding an empty object.

By convention, the names of constructors are capitalized so that they can easily be distinguished from other functions.

It is important to understand the distinction between the way a prototype is associated with a constructor (through its `prototype` property) and the way objects *have* a prototype (which can be found with `Object.getPrototypeOf`). The actual prototype of a constructor is `Function.prototype` since constructors are functions. The constructor function's prototype *property* holds the prototype used for instances created through it.

```
console.log(Object.getPrototypeOf(Rabbit) ==
             Function.prototype);
// → true
console.log(Object.getPrototypeOf(killerRabbit) ==
             Rabbit.prototype);
// → true
```

Constructors will typically add some per-instance properties to this. It

is also possible to declare properties directly in the class declaration. Unlike methods, such properties are added to instance objects and not the prototype.

```
class Particle {
  speed = 0;
  constructor(position) {
    this.position = position;
  }
}
```

Like `function`, `class` can be used both in statements and in expressions. When used as an expression, it doesn't define a binding but just produces the constructor as a value. You are allowed to omit the class name in a class expression.

```
let object = new class { getWord() { return "hello"; } };
console.log(object.getWord());
// → hello
```

PRIVATE PROPERTIES

It is common for classes to define some properties and methods for internal use that are not part of their interface. These are called *private* properties, as opposed to *public* ones, which are part of the object's external interface.

To declare a private method, put a `#` sign in front of its name. Such methods can be called only from inside the `class` declaration that de-

finishes them.

```
class SecretiveObject {
  #getSecret() {
    return "I ate all the plums";
  }
  interrogate() {
    let shallISayIt = this.#getSecret();
    return "never";
  }
}
```

When a class does not declare a constructor, it will automatically get an empty one.

If you try to call `#getSecret` from outside the class, you get an error. Its existence is entirely hidden inside the class declaration.

To use private instance properties, you must declare them. Regular properties can be created by just assigning to them, but private properties *must* be declared in the class declaration to be available at all.

This class implements an appliance for getting a random whole number below a given maximum number. It has only one public property: `getNumber`.

```
class RandomSource {
  #max;
  constructor(max) {
    this.#max = max;
  }
  getNumber() {
```

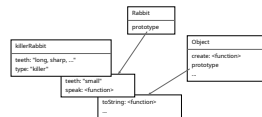
```
    return Math.floor(Math.random() * this.#max);
  }
}
```

OVERRIDING DERIVED PROPERTIES

When you add a property to an object, whether it is present in the prototype or not, the property is added to the object *itself*. If there was already a property with the same name in the prototype, this property will no longer affect the object, as it is now hidden behind the object's own property.

```
Rabbit.prototype.teeth = "small";
console.log(killerRabbit.teeth);
// → small
killerRabbit.teeth = "long, sharp, and bloody";
console.log(killerRabbit.teeth);
// → long, sharp, and bloody
console.log((new Rabbit("basic")).teeth);
// → small
console.log(Rabbit.prototype.teeth);
// → small
```

The following diagram sketches the situation after this code has run. The `Rabbit` and `Object` prototypes lie behind `killerRabbit` as a kind of backdrop, where properties that are not found in the object itself can be looked up.



Overriding properties that exist in a prototype can be a useful thing to do. As the rabbit teeth example shows, overriding can be used to express exceptional properties in instances of a more generic class of objects while letting the nonexceptional objects take a standard value from their prototype.

Overriding is also used to give the standard function and array prototypes a different `toString` method than the basic object prototype.

```

console.log(Array.prototype.toString ==
  Object.prototype.toString);
// → false
console.log([1, 2].toString());
// → 1,2

```

Calling `toString` on an array gives a result similar to calling `.join(",")` on it—it puts commas between the values in the array. Directly calling `Object.prototype.toString` with an array produces a different string. That function doesn't know about arrays, so it simply puts the word *object* and the name of the type between square brackets.

```

console.log(Object.prototype.toString.call([1, 2]));

```

```

// → [Object Array]

```

MAPS

We saw the word *map* used in the [previous chapter](#) for an operation that transforms a data structure by applying a function to its elements. Confusing as it is, in programming the same word is used for a related but rather different thing.

A *map* (noun) is a data structure that associates values (the keys) with other values. For example, you might want to map names to ages. It is possible to use objects for this.

```

let ages = {
  Boris: 39,
  Liang: 22,
  Júlia: 62
};

console.log(`Júlia is ${ages["Júlia"]}`);
// → Júlia is 62
console.log("Is Jack's age known?", "Jack" in ages);
// → Is Jack's age known? false
console.log("Is toString's age known?", "toString" in ages);
// → Is toString's age known? true

```

Here, the object's property names are the people's names and the property values are their ages. But we certainly didn't list anybody named `toString` in our map. Yet because plain objects derive from `Object`.

prototype, it looks like the property is there.

For this reason, using plain objects as maps is dangerous. There are several possible ways to avoid this problem. First, you can create objects with *no* prototype. If you pass `null` to `Object.create`, the resulting object will not derive from `Object.prototype` and can be safely used as a map.

```
console.log("toString" in Object.create(null));  
// → false
```

Object property names must be strings. If you need a map whose keys can't easily be converted to strings—such as objects—you cannot use an object as your map.

Fortunately, JavaScript comes with a class called `Map` that is written for this exact purpose. It stores a mapping and allows any type of keys.

```
let ages = new Map();  
ages.set("Boris", 39);  
ages.set("Liang", 22);  
ages.set("Júlia", 62);
```

```
console.log(`Júlia is ${ages.get("Júlia")}`);  
// → Júlia is 62  
console.log("Is Jack's age known?", ages.has("Jack"));  
// → Is Jack's age known? false  
console.log(ages.has("toString"));  
// → false
```

The methods `set`, `get`, and `has` are part of the interface of the `Map` object. Writing a data structure that can quickly update and search a large set

of values isn't easy, but we don't have to worry about that. Someone else did it for us, and we can go through this simple interface to use their work.

If you do have a plain object that you need to treat as a map for some reason, it is useful to know that `Object.keys` returns only an object's *own* keys, not those in the prototype. As an alternative to the `in` operator, you can use the `Object.hasOwn` function, which ignores the object's prototype.

```
console.log(Object.hasOwn({x: 1}, "x"));  
// → true  
console.log(Object.hasOwn({x: 1}, "toString"));  
// → false
```

POLYMORPHISM

When you call the `String` function (which converts a value to a string) on an object, it will call the `toString` method on that object to try to create a meaningful string from it. I mentioned that some of the standard prototypes define their own version of `toString` so they can create a string that contains more useful information than `"[object Object]"`. You can also do that yourself.

```
Rabbit.prototype.toString = function() {  
  return `a ${this.type} rabbit`;  
};
```

```
console.log(String(killerRabbit));
```

```
// → a killer rabbit
```

This is a simple instance of a powerful idea. When a piece of code is written to work with objects that have a certain interface—in this case, a `toString` method—any kind of object that happens to support this interface can be plugged into the code and will be able to work with it.

This technique is called *polymorphism*. Polymorphic code can work with values of different shapes, as long as they support the interface it expects.

An example of a widely used interface is that of array-like objects that have a `length` property holding a number and numbered properties for each of their elements. Both arrays and strings support this interface, as do various other objects, some of which we'll see later in the chapters about the browser. You can call most array methods—such as `Array.prototype.forEach`—on anything that provides this interface.

```
Array.prototype.forEach.call({
  length: 2,
  0: "A",
  1: "B"
}, elt => console.log(elt));
// → A
// → B
```

GETTERS, SETTERS, AND STATICS

Interfaces often contain plain properties, not just methods. For example, `Map` objects have a `size` property that tells you how many keys are stored in them.

It is not necessary for such an object to compute and store such a property directly in the instance. Even properties that are accessed directly may hide a method call. Such methods are called *getters* and are defined by writing `get` in front of the method name in an object expression or class declaration.

```
let varyingSize = {
  get size() {
    return Math.floor(Math.random() * 100);
  }
};

console.log(varyingSize.size);
// → 73
console.log(varyingSize.size);
// → 49
```

Whenever someone reads from this object's `size` property, the associated method is called. You can do a similar thing when a property is written to, using a *setter*.

```
class Temperature {
  constructor(celsius) {
    this.celsius = celsius;
  }
}
```

```

    get fahrenheit() {
      return this.celsius * 1.8 + 32;
    }
    set fahrenheit(value) {
      this.celsius = (value - 32) / 1.8;
    }

    static fromFahrenheit(value) {
      return new Temperature((value - 32) / 1.8);
    }
  }

  let temp = new Temperature(22);
  console.log(temp.fahrenheit);
  // → 71.6
  temp.fahrenheit = 86;
  console.log(temp.celsius);
  // → 30

```

The `Temperature` class allows you to read and write the temperature in either degrees Celsius or degrees Fahrenheit, but internally it stores only Celsius and automatically converts to and from Celsius in the `fahrenheit` getter and setter.

Sometimes you want to attach some properties directly to your constructor function rather than to the prototype. Such methods won't have access to a class instance but can, for example, be used to provide additional ways to create instances.

Inside a class declaration, methods or properties that have `static` written before their name are stored on the constructor. For example,

the `Temperature` class allows you to write `Temperature.fromFahrenheit(100)` to create a temperature using degrees Fahrenheit.

```

let boil = Temperature.fromFahrenheit(212);
console.log(boil.celsius);
// → 100

```

SYMBOLS

I mentioned in Chapter 4 that a `for/of` loop can loop over several kinds of data structures. This is another case of polymorphism—such loops expect the data structure to expose a specific interface, which arrays and strings do. And we can also add this interface to our own objects! But before we can do that, we need to briefly take a look at the symbol type.

It is possible for multiple interfaces to use the same property name for different things. For example, on array-like objects, `length` refers to the number of elements in the collection. But an object interface describing a hiking route could use `length` to provide the length of the route in meters. It would not be possible for an object to conform to both these interfaces.

An object trying to be a route and array-like (maybe to enumerate its waypoints) is somewhat far-fetched, and this kind of problem isn't that common in practice. For things like the iteration protocol, though, the language designers needed a type of property that *really* doesn't conflict with any others. So in 2015, *symbols* were added to the language.

Most properties, including all those we have seen so far, are named with strings. But it is also possible to use symbols as property names. Symbols are values created with the `Symbol` function. Unlike strings, newly created symbols are unique—you cannot create the same symbol twice.

```
let sym = Symbol("name");
console.log(sym == Symbol("name"));
// → false
Rabbit.prototype[sym] = 55;
console.log(killerRabbit[sym]);
// → 55
```

The string you pass to `Symbol` is included when you convert it to a string and can make it easier to recognize a symbol when, for example, showing it in the console. But it has no meaning beyond that—multiple symbols may have the same name.

Being both unique and usable as property names makes symbols suitable for defining interfaces that can peacefully live alongside other properties, no matter what their names are.

```
const length = Symbol("length");
Array.prototype[length] = 0;

console.log([1, 2].length);
// → 2
console.log([1, 2][length]);
// → 0
```

It is possible to include symbol properties in object expressions and

classes by using square brackets around the property name. That causes the expression between the brackets to be evaluated to produce the property name, analogous to the square bracket property access notation.

```
let myTrip = {
  length: 2,
  0: "Lankwitz",
  1: "Babelsberg",
  [length]: 21500
};
console.log(myTrip[length], myTrip.length);
// → 21500 2
```

THE ITERATOR INTERFACE

The object given to a *for/of* loop is expected to be *iterable*. This means it has a method named with the `Symbol.iterator` symbol (a symbol value defined by the language, stored as a property of the `Symbol` function).

When called, that method should return an object that provides a second interface, *iterator*. This is the actual thing that iterates. It has a `next` method that returns the next result. That result should be an object with a `value` property that provides the next value, if there is one, and a `done` property, which should be true when there are no more results and false otherwise.

Note that the `next`, `value`, and `done` property names are plain strings, not symbols. Only `Symbol.iterator`, which is likely to be added to a *lot*

of different objects, is an actual symbol.

We can directly use this interface ourselves.

```
let okIterator = "OK"[Symbol.iterator]();
console.log(okIterator.next());
// → {value: "0", done: false}
console.log(okIterator.next());
// → {value: "K", done: false}
console.log(okIterator.next());
// → {value: undefined, done: true}
```

Let's implement an iterable data structure similar to the linked list from the exercise in [Chapter 4](#). We'll write the list as a class this time.

```
class List {
  constructor(value, rest) {
    this.value = value;
    this.rest = rest;
  }

  get length() {
    return 1 + (this.rest ? this.rest.length : 0);
  }

  static fromArray(array) {
    let result = null;
    for (let i = array.length - 1; i >= 0; i--) {
      result = new this(array[i], result);
    }
    return result;
  }
}
```

```
}
```

Note that `this`, in a static method, points at the constructor of the class, not an instance—there is no instance around when a static method is called.

Iterating over a list should return all the list's elements from start to end. We'll write a separate class for the iterator.

```
class ListIterator {
  constructor(list) {
    this.list = list;
  }

  next() {
    if (this.list == null) {
      return {done: true};
    }
    let value = this.list.value;
    this.list = this.list.rest;
    return {value, done: false};
  }
}
```

The class tracks the progress of iterating through the list by updating its `list` property to move to the next list object whenever a value is returned and reports that it is done when that list is empty (`null`).

Let's set up the `List` class to be iterable. Throughout this book, I'll occasionally use after-the-fact prototype manipulation to add methods to classes so that the individual pieces of code remain small and self contained. In a regular program, where there is no need to split the

code into small pieces, you'd declare these methods directly in the class instead.

```
List.prototype[Symbol.iterator] = function() {  
  return new ListIterator(this);  
};
```

We can now loop over a list with `for/of`.

```
let list = List.fromArray([1, 2, 3]);  
for (let element of list) {  
  console.log(element);  
}  
// → 1  
// → 2  
// → 3
```

The `...` syntax in array notation and function calls similarly works with any iterable object. For example, you can use `[...value]` to create an array containing the elements in an arbitrary iterable object.

```
console.log([... "PCI"]);  
// → ["P", "C", "I"]
```

INHERITANCE

Imagine we need a list type much like the `List` class we saw before, but because we will be asking for its length all the time, we don't want it to have to scan through its `rest` every time. Instead, we want to store

the length in every instance for efficient access.

JavaScript's prototype system makes it possible to create a *new* class, much like the old class, but with new definitions for some of its properties. The prototype for the new class derives from the old prototype but adds a new definition for, say, the `length` getter.

In object-oriented programming terms, this is called *inheritance*. The new class inherits properties and behavior from the old class.

```
class LengthList extends List {  
  #length;  
  
  constructor(value, rest) {  
    super(value, rest);  
    this.#length = super.length;  
  }  
  
  get length() {  
    return this.#length;  
  }  
}  
  
console.log(LengthList.fromArray([1, 2, 3]).length);  
// → 3
```

The use of the word `extends` indicates that this class shouldn't be directly based on the default `Object` prototype but on some other class. This is called the *superclass*. The derived class is the *subclass*.

To initialize a `LengthList` instance, the constructor calls the constructor of its superclass through the `super` keyword. This is necessary be-

cause if this new object is to behave (roughly) like a `List`, it is going to need the instance properties that lists have.

The constructor then stores the list's length in a private property. If we had written `this.length` there, the class's own getter would have been called, which doesn't work yet since `#length` hasn't been filled in yet. We can use `super.something` to call methods and getters on the superclass's prototype, which is often useful.

Inheritance allows us to build slightly different data types from existing data types with relatively little work. It is a fundamental part of the object-oriented tradition, alongside encapsulation and polymorphism. But while the latter two are now generally regarded as wonderful ideas, inheritance is more controversial.

Whereas encapsulation and polymorphism can be used to *separate* pieces of code from one another, reducing the tangledness of the overall program, inheritance fundamentally ties classes together, creating *more* tangle. When inheriting from a class, you usually have to know more about how it works than when simply using it. Inheritance can be a useful tool to make some types of programs more succinct, but it shouldn't be the first tool you reach for, and you probably shouldn't actively go looking for opportunities to construct class hierarchies (family trees of classes).

THE INSTANCEOF OPERATOR

It is occasionally useful to know whether an object was derived from a specific class. For this, JavaScript provides a binary operator called

```
instanceof.  
  
console.log(  
  new LengthList(1, null) instanceof LengthList);  
// → true  
console.log(new LengthList(2, null) instanceof List);  
// → true  
console.log(new List(3, null) instanceof LengthList);  
// → false  
console.log([1] instanceof Array);  
// → true
```

The operator will see through inherited types, so a `LengthList` is an instance of `List`. The operator can also be applied to standard constructors like `Array`. Almost every object is an instance of `Object`.

SUMMARY

Objects do more than just hold their own properties. They have prototypes, which are other objects. They'll act as if they have properties they don't have as long as their prototype has that property. Simple objects have `Object.prototype` as their prototype.

Constructors, which are functions whose names usually start with a capital letter, can be used with the `new` operator to create new objects. The new object's prototype will be the object found in the `prototype` property of the constructor. You can make good use of this by putting the properties that all values of a given type share into their prototype. There's a `class` notation that provides a clear way to define a

constructor and its prototype.

You can define getters and setters to secretly call methods every time an object's property is accessed. Static methods are methods stored in a class's constructor rather than its prototype.

The `instanceof` operator can, given an object and a constructor, tell you whether that object is an instance of that constructor.

One useful thing to do with objects is to specify an interface for them and tell everybody that they are supposed to talk to your object only through that interface. The rest of the details that make up your object are now *encapsulated*, hidden behind the interface. You can use private properties to hide a part of your object from the outside world.

More than one type may implement the same interface. Code written to use an interface automatically knows how to work with any number of different objects that provide the interface. This is called *polymorphism*.

When implementing multiple classes that differ in only some details, it can be helpful to write the new classes as *subclasses* of an existing class, *inheriting* part of its behavior.

EXERCISES

A VECTOR TYPE

Write a class `Vec` that represents a vector in two-dimensional space. It takes `x` and `y` parameters (numbers), that it saves to properties of the same name.

Give the `Vec` prototype two methods, `plus` and `minus`, that take another vector as a parameter and return a new vector that has the sum or

difference of the two vectors' (this and the parameter) `x` and `y` values.

Add a getter property `length` to the prototype that computes the length of the vector—that is, the distance of the point (x, y) from the origin $(0, 0)$.

GROUPS

The standard JavaScript environment provides another data structure called `Set`. Like an instance of `Map`, a set holds a collection of values. Unlike `Map`, it does not associate other values with those—it just tracks which values are part of the set. A value can be part of a set only once—adding it again doesn't have any effect.

Write a class called `Group` (since `Set` is already taken). Like `Set`, it has `add`, `delete`, and `has` methods. Its constructor creates an empty group, `add` adds a value to the group (but only if it isn't already a member), `delete` removes its argument from the group (if it was a member), and `has` returns a Boolean value indicating whether its argument is a member of the group.

Use the `===` operator, or something equivalent such as `indexOf`, to determine whether two values are the same.

Give the class a static `from` method that takes an iterable object as its argument and creates a group that contains all the values produced by iterating over it.

ITERABLE GROUPS

Make the `Group` class from the previous exercise iterable. Refer to the section about the iterator interface earlier in the chapter if you aren't clear on the exact form of the interface anymore.

If you used an array to represent the group's members, don't just return the iterator created by calling the `Symbol.iterator` method on the array. That would work, but it defeats the purpose of this exercise.

It is okay if your iterator behaves strangely when the group is modified during iteration.

"The question of whether Machines Can Think [...] is about as relevant as the question of whether Submarines Can Swim."
—Edsger Dijkstra, The Threats to Computing Science

CHAPTER 7

PROJECT: A ROBOT

In “project” chapters, I'll stop pummeling you with new theory for a brief moment, and instead we'll work through a program together. Theory is necessary to learn to program, but reading and understanding actual programs is just as important.

Our project in this chapter is to build an automaton, a little program that performs a task in a virtual world. Our automaton will be a mail-delivery robot picking up and dropping off parcels.

MEADOWFIELD

The village of Meadowfield isn't very big. It consists of 11 places with 14 roads between them. It can be described with this array of roads:

```
const roads = [
  "Alice's House-Bob's House",  "Alice's House-Cabin",
  "Alice's House-Post Office",   "Bob's House-Town Hall",
  "Daria's House-Ernie's House", "Daria's House-Town Hall",
  "Ernie's House-Grete's House", "Grete's House-Farm",
```

```

"Grete's House-Shop",      "Marketplace-Farm",
"Marketplace-Post Office", "Marketplace-Shop",
"Marketplace-Town Hall",  "Shop-Town Hall"
];

```



The network of roads in the village forms a *graph*. A graph is a collection of points (places in the village) with lines between them (roads). This graph will be the world that our robot moves through.

The array of strings isn't very easy to work with. What we're interested in is the destinations that we can reach from a given place. Let's convert the list of roads to a data structure that, for each place, tells us what can be reached from there.

```

function buildGraph(edges) {
  let graph = Object.create(null);
  function addEdge(from, to) {
    if (from in graph) {
      graph[from].push(to);
    } else {
      graph[from] = [to];
    }
  }
  for (let [from, to] of edges.map(r => r.split("-"))) {
    addEdge(from, to);
    addEdge(to, from);
  }
  return graph;
}

```

```
const roadGraph = buildGraph(roads);
```

Given an array of edges, `buildGraph` creates a map object that, for each node, stores an array of connected nodes. It uses the `split` method to go from the road strings—which have the form "Start-End"—to two-element arrays containing the start and end as separate strings.

THE TASK

Our robot will be moving around the village. There are parcels in various places, each addressed to some other place. The robot picks up parcels when it comes across them and delivers them when it arrives at their destinations.

The automaton must decide, at each point, where to go next. It has finished its task when all parcels have been delivered.

To be able to simulate this process, we must define a virtual world that can describe it. This model tells us where the robot is and where the parcels are. When the robot has decided to move somewhere, we need to update the model to reflect the new situation.

If you're thinking in terms of object-oriented programming, your first impulse might be to start defining objects for the various elements in the world: a class for the robot, one for a parcel, maybe one for places. These could then hold properties that describe their current state, such as the pile of parcels at a location, which we could change when updating the world.

This is wrong. At least, it usually is. The fact that something sounds like an object does not automatically mean that it should be an object in your program. Reflexively writing classes for every concept in your application tends to leave you with a collection of interconnected objects that each have their own internal, changing state. Such programs are often hard to understand and thus easy to break.

Instead, let's condense the village's state down to the minimal set of values that define it. There's the robot's current location and the collection of undelivered parcels, each of which has a current location and a destination address. That's it.

While we're at it, let's make it so that we don't *change* this state when the robot moves but rather compute a *new* state for the situation after the move.

```
class VillageState {
```

```
    constructor(place, parcels) {
      this.place = place;
      this.parcels = parcels;
    }

    move(destination) {
      if (!roadGraph[this.place].includes(destination)) {
        return this;
      } else {
        let parcels = this.parcels.map(p => {
          if (p.place !== this.place) return p;
          return {place: destination, address: p.address};
        }).filter(p => p.place !== p.address);
        return new VillageState(destination, parcels);
      }
    }
  }
```

The `move` method is where the action happens. It first checks whether there is a road going from the current place to the destination, and if not, it returns the old state, since this is not a valid move.

Next, the method creates a new state with the destination as the robot's new place. It also needs to create a new set of parcels—parcels that the robot is carrying (that are at the robot's current place) need to be moved along to the new place. And parcels that are addressed to the new place need to be delivered—that is, they need to be removed from the set of undelivered parcels. The call to `map` takes care of the moving, and the call to `filter` does the delivering.

Parcel objects aren't changed when they are moved but re-created.

The `move` method gives us a new village state but leaves the old one entirely intact.

```
let first = new VillageState(
  "Post Office",
  [{place: "Post Office", address: "Alice's House"}]
);
let next = first.move("Alice's House");

console.log(next.place);
// → Alice's House
console.log(next.parcels);
// → []
console.log(first.place);
// → Post Office
```

The move causes the parcel to be delivered, which is reflected in the next state. But the initial state still describes the situation where the robot is at the post office and the parcel is undelivered.

PERSISTENT DATA

Data structures that don't change are called *immutable* or *persistent*. They behave a lot like strings and numbers in that they are who they are and stay that way, rather than containing different things at different times.

In JavaScript, just about everything *can* be changed, so working with values that are supposed to be persistent requires some restraint. There is a function called `Object.freeze` that changes an object so that writing

to its properties is ignored. You could use that to make sure your objects aren't changed, if you want to be careful. Freezing does require the computer to do some extra work, and having updates ignored is just about as likely to confuse someone as having them do the wrong thing. I usually prefer to just tell people that a given object shouldn't be messed with and hope they remember it.

```
let object = Object.freeze({value: 5});
object.value = 10;
console.log(object.value);
// → 5
```

Why am I going out of my way to not change objects when the language is obviously expecting me to? Because it helps me understand my programs. This is about complexity management again. When the objects in my system are fixed, stable things, I can consider operations on them in isolation—moving to Alice's house from a given start state always produces the same new state. When objects change over time, that adds a whole new dimension of complexity to this kind of reasoning.

For a small system like the one we are building in this chapter, we could handle that bit of extra complexity. But the most important limit on what kind of systems we can build is how much we can understand. Anything that makes your code easier to understand makes it possible to build a more ambitious system.

Unfortunately, although understanding a system built on persistent data structures is easier, *designing* one, especially when your programming language isn't helping, can be a little harder. We'll look for opportunities to use persistent data structures in this book, but we'll also

be using changeable ones.

SIMULATION

A delivery robot looks at the world and decides in which direction it wants to move. So we could say that a robot is a function that takes a `VillageState` object and returns the name of a nearby place.

Because we want robots to be able to remember things so they can make and execute plans, we also pass them their memory and allow them to return a new memory. Thus, the thing a robot returns is an object containing both the direction it wants to move in and a memory value that will be given back to it the next time it is called.

```
function runRobot(state, robot, memory) {
  for (let turn = 0;; turn++) {
    if (state.parcels.length == 0) {
      console.log(`Done in ${turn} turns`);
      break;
    }
    let action = robot(state, memory);
    state = state.move(action.direction);
    memory = action.memory;
    console.log(`Moved to ${action.direction}`);
  }
}
```

Consider what a robot has to do to “solve” a given state. It must pick up all parcels by visiting every location that has a parcel and deliver them by visiting every location to which a parcel is addressed, but only

after picking up the parcel.

What is the dumbest strategy that could possibly work? The robot could just walk in a random direction every turn. That means, with great likelihood, it will eventually run into all parcels and then also at some point reach the place where they should be delivered.

Here's what that could look like:

```
function randomPick(array) {
  let choice = Math.floor(Math.random() * array.length);
  return array[choice];
}

function randomRobot(state) {
  return {direction: randomPick(roadGraph[state.place])};
}
```

Remember that `Math.random()` returns a number between 0 and 1—but always below 1. Multiplying such a number by the length of an array and then applying `Math.floor` to it gives us a random index for the array.

Since this robot does not need to remember anything, it ignores its second argument (remember that JavaScript functions can be called with extra arguments without ill effects) and omits the `memory` property in its returned object.

To put this sophisticated robot to work, we'll first need a way to create a new state with some parcels. A static method (written here by directly adding a property to the constructor) is a good place to put that functionality.

```

VillageState.random = function(parcelCount = 5) {
  let parcels = [];
  for (let i = 0; i < parcelCount; i++) {
    let address = randomPick(Object.keys(roadGraph));
    let place;
    do {
      place = randomPick(Object.keys(roadGraph));
    } while (place == address);
    parcels.push({place, address});
  }
  return new VillageState("Post Office", parcels);
};

```

We don't want any parcels to be sent from the same place that they are addressed to. For this reason, the `do` loop keeps picking new places when it gets one that's equal to the address.

Let's start up a virtual world.

```

runRobot(VillageState.random(), randomRobot);
// → Moved to Marketplace
// → Moved to Town Hall
// → _
// → Done in 63 turns

```

It takes the robot a lot of turns to deliver the parcels because it isn't planning ahead very well. We'll address that soon.

THE MAIL TRUCK'S ROUTE

We should be able to do a lot better than the random robot. An easy improvement would be to take a hint from the way real-world mail delivery works. If we find a route that passes all places in the village, the robot could run that route twice, at which point it is guaranteed to be done. Here is one such route (starting from the post office):

```

const mailRoute = [
  "Alice's House", "Cabin", "Alice's House", "Bob's House",
  "Town Hall", "Daria's House", "Ernie's House",
  "Grete's House", "Shop", "Grete's House", "Farm",
  "Marketplace", "Post Office"
];

```

To implement the route-following robot, we'll need to make use of robot memory. The robot keeps the rest of its route in its memory and drops the first element every turn.

```

function routeRobot(state, memory) {
  if (memory.length == 0) {
    memory = mailRoute;
  }
  return {direction: memory[0], memory: memory.slice(1)};
}

```

This robot is a lot faster already. It'll take a maximum of 26 turns (twice the 13-step route) but usually less.

PATHFINDING

Still, I wouldn't really call blindly following a fixed route intelligent behavior. The robot could work more efficiently if it adjusted its behavior to the actual work that needs to be done.

To do that, it has to be able to deliberately move toward a given parcel or toward the location where a parcel has to be delivered. Doing that, even when the goal is more than one move away, will require some kind of route-finding function.

The problem of finding a route through a graph is a typical *search problem*. We can tell whether a given solution (a route) is valid, but we can't directly compute the solution the way we could for $2 + 2$. Instead, we have to keep creating potential solutions until we find one that works.

The number of possible routes through a graph is infinite. But when searching for a route from *A* to *B*, we are interested only in the ones that start at *A*. We also don't care about routes that visit the same place twice—those are definitely not the most efficient route anywhere. So that cuts down on the number of routes that the route finder has to consider.

In fact, since we are mostly interested in the *shortest* route, we want to make sure we look at short routes before we look at longer ones. A good approach would be to “grow” routes from the starting point, exploring every reachable place that hasn't been visited yet until a route reaches the goal. That way, we'll explore only routes that are potentially interesting, and we know that the first route we find is the shortest route (or one of the shortest routes, if there are more than

one).

Here is a function that does this:

```
function findRoute(graph, from, to) {
  let work = [{at: from, route: []}];
  for (let i = 0; i < work.length; i++) {
    let {at, route} = work[i];
    for (let place of graph[at]) {
      if (place == to) return route.concat(place);
      if (!work.some(w => w.at == place)) {
        work.push({at: place, route: route.concat(place)});
      }
    }
  }
}
```

The exploring has to be done in the right order—the places that were reached first have to be explored first. We can't immediately explore a place as soon as we reach it because that would mean places reached *from there* would also be explored immediately, and so on, even though there may be other, shorter paths that haven't yet been explored.

Therefore, the function keeps a *work list*. This is an array of places that should be explored next, along with the route that got us there. It starts with just the start position and an empty route.

The search then operates by taking the next item in the list and exploring that, which means it looks at all roads going from that place. If one of them is the goal, a finished route can be returned. Otherwise, if we haven't looked at this place before, a new item is added to the list. If we have looked at it before, since we are looking at short routes

first, we've found either a longer route to that place or one precisely as long as the existing one, and we don't need to explore it.

You can visualize this as a web of known routes crawling out from the start location, growing evenly on all sides (but never tangling back into itself). As soon as the first thread reaches the goal location, that thread is traced back to the start, giving us our route.

Our code doesn't handle the situation where there are no more work items on the work list because we know that our graph is *connected*, meaning that every location can be reached from all other locations. We'll always be able to find a route between two points, and the search can't fail.

```
function goalOrientedRobot((place, parcels), route) {
  if (route.length == 0) {
    let parcel = parcels[0];
    if (parcel.place != place) {
      route = findRoute(roadGraph, place, parcel.place);
    } else {
      route = findRoute(roadGraph, place, parcel.address);
    }
  }
  return {direction: route[0], memory: route.slice(1)};
}
```

This robot uses its memory value as a list of directions to move in, just like the route-following robot. Whenever that list is empty, it has to figure out what to do next. It takes the first undelivered parcel in the set and, if that parcel hasn't been picked up yet, plots a route toward it. If the parcel *has* been picked up, it still needs to be delivered, so the

robot creates a route toward the delivery address instead.

This robot usually finishes the task of delivering 5 parcels in about 16 turns. That's slightly better than `routeRobot` but still definitely not optimal. We'll continue refining it in the exercises.

EXERCISES

MEASURING A ROBOT

It's hard to objectively compare robots by just letting them solve a few scenarios. Maybe one robot just happened to get easier tasks or the kind of tasks that it is good at, whereas the other didn't.

Write a function `compareRobots` that takes two robots (and their starting memory). It should generate 100 tasks and let both of the robots solve each of these tasks. When done, it should output the average number of steps each robot took per task.

For the sake of fairness, make sure you give each task to both robots, rather than generating different tasks per robot.

ROBOT EFFICIENCY

Can you write a robot that finishes the delivery task faster than `goalOrientedRobot`? If you observe that robot's behavior, what obviously stupid things does it do? How could those be improved?

If you solved the previous exercise, you might want to use your `compareRobots` function to verify whether you improved the robot.

PERSISTENT GROUP

Most data structures provided in a standard JavaScript environment aren't very well suited for persistent use. Arrays have `slice` and `concat` methods, which allow us to easily create new arrays without damaging the old one. But `Set`, for example, has no methods for creating a new set with an item added or removed.

Write a new class `PGroup`, similar to the `Group` class from Chapter 6, which stores a set of values. Like `Group`, it has `add`, `delete`, and `has` methods. Its `add` method, however, should return a *new* `PGroup` instance with the given member added and leave the old one unchanged. Similarly, `delete` should create a new instance without a given member.

The class should work for values of any type, not just strings. It does *not* have to be efficient when used with large numbers of values.

The constructor shouldn't be part of the class's interface (though you'll definitely want to use it internally). Instead, there is an empty instance, `PGroup.empty`, that can be used as a starting value.

Why do you need only one `PGroup.empty` value rather than having a function that creates a new, empty map every time?

"Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."

—Brian Kernighan and P.J. Plauger, *The Elements of Programming Style*

CHAPTER 8

BUGS AND ERRORS

Flaws in computer programs are usually called *bugs*. It makes programmers feel good to imagine them as little things that just happen to crawl into our work. In reality, of course, we put them there ourselves.

If a program is crystallized thought, we can roughly categorize bugs into those caused by the thoughts being confused and those caused by mistakes introduced while converting a thought to code. The former type is generally harder to diagnose and fix than the latter.

LANGUAGE

Many mistakes could be pointed out to us automatically by the computer if it knew enough about what we're trying to do. But here, JavaScript's looseness is a hindrance. Its concept of bindings and properties is vague enough that it will rarely catch typos before actually running the program. Even then, it allows you to do some clearly nonsensical things without complaint, such as computing `true * "monkey"`.

There are some things that JavaScript does complain about. Writing

a program that does not follow the language's grammar will immediately make the computer complain. Other things, such as calling something that's not a function or looking up a property on an undefined value, will cause an error to be reported when the program tries to perform the action.

Often, however, your nonsense computation will merely produce `NaN` (not a number) or an undefined value, while the program happily continues, convinced that it's doing something meaningful. The mistake will manifest itself only later, after the bogus value has traveled through several functions. It might not trigger an error at all, but silently cause the program's output to be wrong. Finding the source of such problems can be difficult.

The process of finding mistakes—bugs—in programs is called *debugging*.

STRICT MODE

JavaScript can be made a *little* stricter by enabling *strict mode*. This can be done by putting the string "use strict" at the top of a file or a function body. Here's an example:

```
function canYouSpotTheProblem() {
  "use strict";
  for (counter = 0; counter < 10; counter++) {
    console.log("Happy happy");
  }
}
```

```
canYouSpotTheProblem();
// → ReferenceError: counter is not defined
```

Code inside classes and modules (which we will discuss in [Chapter 10](#)) is automatically strict. The old nonstrict behavior still exists only because some old code might depend on it, and the language designers work hard to avoid breaking any existing programs.

Normally, when you forget to put `let` in front of your binding, as with `counter` in the example, JavaScript quietly creates a global binding and uses that. In strict mode, an error is reported instead. This is very helpful. It should be noted, though, that this doesn't work when the binding in question already exists somewhere in scope. In that case, the loop will still quietly overwrite the value of the binding.

Another change in strict mode is that the `this` binding holds the value `undefined` in functions that are not called as methods. When making such a call outside of strict mode, `this` refers to the global scope object, which is an object whose properties are the global bindings. So if you accidentally call a method or constructor incorrectly in strict mode, JavaScript will produce an error as soon as it tries to read something from `this`, rather than happily writing to the global scope.

For example, consider the following code, which calls a constructor function without the `new` keyword so that its `this` will *not* refer to a newly constructed object:

```
function Person(name) { this.name = name; }
let ferdinand = Person("Ferdinand"); // oops
console.log(name);
```

```
// → Ferdinand
```

The bogus call to `Person` succeeded, but returned an undefined value and created the global binding `name`. In strict mode, the result is different.

```
"use strict";
function Person(name) { this.name = name; }
let ferdinand = Person("Ferdinand"); // forgot new
// → TypeError: Cannot set property 'name' of undefined
```

We are immediately told that something is wrong. This is helpful.

Fortunately, constructors created with the `class` notation will always complain if they are called without `new`, making this less of a problem even in nonstrict mode.

Strict mode does a few more things. It disallows giving a function multiple parameters with the same name and removes certain problematic language features entirely (such as the `with` statement, which is so wrong it is not further discussed in this book).

In short, putting `"use strict"` at the top of your program rarely hurts and might help you spot a problem.

TYPES

Some languages want to know the types of all your bindings and expressions before even running a program. They will tell you right away when a type is used in an inconsistent way. JavaScript considers types only when actually running the program, and even there often tries to implicitly convert values to the type it expects, so it's not much help.

Still, types provide a useful framework for talking about programs. A lot of mistakes come from being confused about the kind of value that goes into or comes out of a function. If you have that information written down, you're less likely to get confused.

You could add a comment like the following before the `findRoute` function from the previous chapter to describe its type:

```
// (graph: Object, from: string, to: string) => string[]
function findRoute(graph, from, to) {
  // ...
}
```

There are a number of different conventions for annotating JavaScript programs with types.

One thing about types is that they need to introduce their own complexity to be able to describe enough code to be useful. What do you think would be the type of the `randomPick` function that returns a random element from an array? You'd need to introduce a *type variable*, *T*, which can stand in for any type, so that you can give `randomPick` a type like `(T[])→T` (function from an array of *T*s to a *T*).

When the types of a program are known, it is possible for the computer to *check* them for you, pointing out mistakes before the program is run. There are several JavaScript dialects that add types to the language and check them. The most popular one is called TypeScript. If you are interested in adding more rigor to your programs, I recommend you give it a try.

In this book, we will continue using raw, dangerous, untyped JavaScript code.

TESTING

If the language is not going to do much to help us find mistakes, we'll have to find them the hard way: by running the program and seeing whether it does the right thing.

Doing this by hand, again and again, is a really bad idea. Not only is it annoying but it also tends to be ineffective, since it takes too much time to exhaustively test everything every time you make a change.

Computers are good at repetitive tasks, and testing is the ideal repetitive task. Automated testing is the process of writing a program that tests another program. Writing tests is a bit more work than testing manually, but once you've done it, you gain a kind of superpower: it takes you only a few seconds to verify that your program still behaves properly in all the situations you wrote tests for. When you break something, you'll immediately notice rather than randomly running into it at some later time.

Tests usually take the form of little labeled programs that verify some aspect of your code. For example, a set of tests for the (standard, probably already tested by someone else) `toUpperCase` method might look like this:

```
function test(label, body) {
  if (!body()) console.log(`Failed: ${label}`);
}

test("convert Latin text to uppercase", () => {
  return "hello".toUpperCase() == "HELLO";
});
```

```
test("convert Greek text to uppercase", () => {
  return "Χαίπετε".toUpperCase() == "ΧΑΙΠΕΤΕ";
});
test("don't convert case-less characters", () => {
  return "مرحبا".toUpperCase() == "مرحبا";
});
```

Writing tests like this tends to produce rather repetitive, awkward code. Fortunately, there exist pieces of software that help you build and run collections of tests (*test suites*) by providing a language (in the form of functions and methods) suited to expressing tests and by outputting informative information when a test fails. These are usually called *test runners*.

Some code is easier to test than other code. Generally, the more external objects that the code interacts with, the harder it is to set up the context in which to test it. The style of programming shown in the [previous chapter](#), which uses self-contained persistent values rather than changing objects, tends to be easy to test.

DEBUGGING

Once you notice there is something wrong with your program because it misbehaves or produces errors, the next step is to figure out *what* the problem is.

Sometimes it is obvious. The error message will point at a specific line of your program, and if you look at the error description and that line of code, you can often see the problem.

But not always. Sometimes the line that triggered the problem is simply the first place where a flaky value produced elsewhere gets used in an invalid way. If you have been solving the exercises in earlier chapters, you will probably have already experienced such situations.

The following example program tries to convert a whole number to a string in a given base (decimal, binary, and so on) by repeatedly picking out the last digit and then dividing the number to get rid of this digit. But the strange output that it currently produces suggests that it has a bug.

```
function numberToString(n, base = 10) {
  let result = "", sign = "";
  if (n < 0) {
    sign = "-";
    n = -n;
  }
  do {
    result = String(n % base) + result;
    n /= base;
  } while (n > 0);
  return sign + result;
}
console.log(numberToString(13, 10));
// → 1.5e-3231.3e-3221.3e-3211.3e-3201.3e-3191.3e-3181.3
```

Even if you see the problem already, pretend for a moment that you don't. We know that our program is malfunctioning, and we want to find out why.

This is where you must resist the urge to start making random

changes to the code to see whether that makes it better. Instead, *think*. Analyze what is happening and come up with a theory of why it might be happening. Then make additional observations to test this theory—or, if you don't yet have a theory, make additional observations to help you come up with one.

Putting a few strategic `console.log` calls into the program is a good way to get additional information about what the program is doing. In this case, we want `n` to take the values 13, 1, and then 0. Let's write out its value at the start of the loop.

```
13
1.3
0.13
0.013...

1.5e-323
```

Right. Dividing 13 by 10 does not produce a whole number. Instead of `n /= base`, what we actually want is `n = Math.floor(n / base)` so that the number is properly “shifted” to the right.

An alternative to using `console.log` to peek into the program's behavior is to use the *debugger* capabilities of your browser. Browsers come with the ability to set a *breakpoint* on a specific line of your code. When the execution of the program reaches a line with a breakpoint, it is paused, and you can inspect the values of bindings at that point. I won't go into details, as debuggers differ from browser to browser, but look in your browser's developer tools or search the web for instructions.

Another way to set a breakpoint is to include a `debugger` statement

(consisting simply of that keyword) in your program. If the developer tools of your browser are active, the program will pause whenever it reaches such a statement.

ERROR PROPAGATION

Not all problems can be prevented by the programmer, unfortunately. If your program communicates with the outside world in any way, it is possible to get malformed input, to become overloaded with work, or to have the network fail.

If you're programming only for yourself, you can afford to just ignore such problems until they occur. But if you build something that is going to be used by anybody else, you usually want the program to do better than just crash. Sometimes the right thing to do is take the bad input in stride and continue running. In other cases, it is better to report to the user what went wrong and then give up. In either situation the program has to actively do something in response to the problem.

Say you have a function `promptNumber` that asks the user for a number and returns it. What should it return if the user inputs "orange"?

One option is to make it return a special value. Common choices for such values are `null`, `undefined`, or `-1`.

```
function promptNumber(question) {
  let result = Number(prompt(question));
  if (Number.isNaN(result)) return null;
  else return result;
}
```

```
console.log(promptNumber("How many trees do you see?"));
```

Now any code that calls `promptNumber` must check whether an actual number was read and, failing that, must somehow recover—maybe by asking again or by filling in a default value. Or it could again return a special value to *its* caller to indicate that it failed to do what it was asked.

In many situations, mostly when errors are common and the caller should be explicitly taking them into account, returning a special value is a good way to indicate an error. It does, however, have its downsides. First, what if the function can already return every possible kind of value? In such a function, you'll have to do something like wrap the result in an object to be able to distinguish success from failure, the way the `next` method on the iterator interface does.

```
function lastElement(array) {
  if (array.length == 0) {
    return {failed: true};
  } else {
    return {value: array[array.length - 1]};
  }
}
```

The second issue with returning special values is that it can lead to awkward code. If a piece of code calls `promptNumber` 10 times, it has to check 10 times whether `null` was returned. If its response to finding `null` is to simply return `null` itself, callers of the function will in turn have to check for it, and so on.

EXCEPTIONS

When a function cannot proceed normally, what we would often *like* to do is just stop what we are doing and immediately jump to a place that knows how to handle the problem. This is what *exception handling* does.

Exceptions are a mechanism that makes it possible for code that runs into a problem to *raise* (or *throw*) an exception. An exception can be any value. Raising one somewhat resembles a super-charged return from a function: it jumps out of not just the current function but also its callers, all the way down to the first call that started the current execution. This is called *unwinding the stack*. You may remember the stack of function calls mentioned in Chapter 3. An exception zooms down this stack, throwing away all the call contexts it encounters.

If exceptions always zoomed right down to the bottom of the stack, they would not be of much use. They'd just provide a novel way to blow up your program. Their power lies in the fact that you can set “obstacles” along the stack to *catch* the exception as it is zooming down. Once you’ve caught an exception, you can do something with it to address the problem and then continue to run the program.

Here's an example:

```
function promptDirection(question) {
  let result = prompt(question);
  if (result.toLowerCase() == "left") return "L";
  if (result.toLowerCase() == "right") return "R";
  throw new Error("Invalid direction: " + result);
}
```

```
function look() {
  if (promptDirection("Which way?") == "L") {
    return "a house";
  } else {
    return "two angry bears";
  }
}

try {
  console.log("You see", look());
} catch (error) {
  console.log("Something went wrong: " + error);
}
```

The `throw` keyword is used to raise an exception. Catching one is done by wrapping a piece of code in a `try` block, followed by the keyword `catch`. When the code in the `try` block causes an exception to be raised, the `catch` block is evaluated, with the name in parentheses bound to the exception value. After the `catch` block finishes—or if the `try` block finishes without problems—the program proceeds beneath the entire `try/catch` statement.

In this case, we used the `Error` constructor to create our exception value. This is a standard JavaScript constructor that creates an object with a `message` property. Instances of `Error` also gather information about the call stack that existed when the exception was created, a so-called *stack trace*. This information is stored in the `stack` property and can be helpful when trying to debug a problem: it tells us the function where the problem occurred and which functions made the failing call.

Note that the `look` function completely ignores the possibility that `promptDirection` might go wrong. This is the big advantage of exceptions: error-handling code is necessary only at the point where the error occurs and at the point where it is handled. The functions in between can forget all about it.

Well, almost...

CLEANING UP AFTER EXCEPTIONS

The effect of an exception is another kind of control flow. Every action that might cause an exception, which is pretty much every function call and property access, might cause control to suddenly leave your code.

This means when code has several side effects, even if its “regular” control flow looks like they’ll always all happen, an exception might prevent some of them from taking place.

Here is some really bad banking code:

```
const accounts = {
  a: 100,
  b: 0,
  c: 20
};

function getAccount() {
  let accountName = prompt("Enter an account name");
  if (!Object.hasOwn(accounts, accountName)) {
    throw new Error(`No such account: ${accountName}`);
  }
}
```

```
    return accountName;
  }

  function transfer(from, amount) {
    if (accounts[from] < amount) return;
    accounts[from] -= amount;
    accounts[getAccount()] += amount;
  }
}
```

The `transfer` function transfers a sum of money from a given account to another, asking for the name of the other account in the process. If given an invalid account name, `getAccount` throws an exception.

But `transfer` *first* removes the money from the account and *then* calls `getAccount` before it adds it to another account. If it is broken off by an exception at that point, it’ll just make the money disappear.

That code could have been written a little more intelligently, for example by calling `getAccount` before it starts moving money around. But often problems like this occur in more subtle ways. Even functions that don’t look like they will throw an exception might do so in exceptional circumstances or when they contain a programmer mistake.

One way to address this is to use fewer side effects. Again, a programming style that computes new values instead of changing existing data helps. If a piece of code stops running in the middle of creating a new value, no existing data structures were damaged, making it easier to recover.

Since that isn’t always practical, try statements have another feature: they may be followed by a `finally` block either instead of or in addition to a `catch` block. A `finally` block says “no matter *what* happens, run

this code after trying to run the code in the try block.”

```
function transfer(from, amount) {
  if (accounts[from] < amount) return;
  let progress = 0;
  try {
    accounts[from] -= amount;
    progress = 1;
    accounts[getAccount()] += amount;
    progress = 2;
  } finally {
    if (progress == 1) {
      accounts[from] += amount;
    }
  }
}
```

This version of the function tracks its progress, and if, when leaving, it notices that it was aborted at a point where it had created an inconsistent program state, it repairs the damage it did.

Note that even though the `finally` code is run when an exception is thrown in the try block, it does not interfere with the exception. After the `finally` block runs, the stack continues unwinding.

Writing programs that operate reliably even when exceptions pop up in unexpected places is hard. Many people simply don’t bother, and because exceptions are typically reserved for exceptional circumstances, the problem may occur so rarely that it is never even noticed. Whether that is a good thing or a really bad thing depends on how much damage the software will do when it fails.

SELECTIVE CATCHING

When an exception makes it all the way to the bottom of the stack without being caught, it gets handled by the environment. What this means differs between environments. In browsers, a description of the error typically gets written to the JavaScript console (reachable through the browser’s Tools or Developer menu). Node.js, the browserless JavaScript environment we will discuss in [Chapter 20](#), is more careful about data corruption. It aborts the whole process when an unhandled exception occurs.

For programmer mistakes, just letting the error go through is often the best you can do. An unhandled exception is a reasonable way to signal a broken program, and the JavaScript console will, on modern browsers, provide you with some information about which function calls were on the stack when the problem occurred.

For problems that are *expected* to happen during routine use, crashing with an unhandled exception is a terrible strategy.

Invalid uses of the language, such as referencing a nonexistent binding, looking up a property on `null`, or calling something that’s not a function, will also result in exceptions being raised. Such exceptions can also be caught.

When a `catch` body is entered, all we know is that *something* in our `try` body caused an exception. But we don’t know *what* did or *which* exception it caused.

JavaScript (in a rather glaring omission) doesn’t provide direct support for selectively catching exceptions: either you catch them all or you don’t catch any. This makes it tempting to *assume* that the ex-

ception you get is the one you were thinking about when you wrote the catch block.

But it might not be. Some other assumption might be violated, or you might have introduced a bug that is causing an exception. Here is an example that *attempts* to keep on calling `promptDirection` until it gets a valid answer:

```
for (;;) {
  try {
    let dir = promptDirection("Where?"); // ← typo!
    console.log("You chose ", dir);
    break;
  } catch (e) {
    console.log("Not a valid direction. Try again.");
  }
}
```

The `for (;;)` construct is a way to intentionally create a loop that doesn't terminate on its own. We break out of the loop only when a valid direction is given. Unfortunately, we misspelled `promptDirection`, which will result in an "undefined variable" error. Because the catch block completely ignores its exception value (`e`), assuming it knows what the problem is, it wrongly treats the binding error as indicating bad input. Not only does this cause an infinite loop but it also "buries" the useful error message about the misspelled binding.

As a general rule, don't blanket-catch exceptions unless it is for the purpose of "routing" them somewhere—for example, over the network to tell another system that our program crashed. And even then, think

carefully about how you might be hiding information.

We want to catch a *specific* kind of exception. We can do this by checking in the catch block whether the exception we got is the one we are interested in, and if not, rethrow it. But how do we recognize an exception?

We could compare its `message` property against the error message we happen to expect. But that's a shaky way to write code—we'd be using information that's intended for human consumption (the message) to make a programmatic decision. As soon as someone changes (or translates) the message, the code will stop working.

Rather, let's define a new type of error and use `instanceof` to identify it.

```
class InputError extends Error {}

function promptDirection(question) {
  let result = prompt(question);
  if (result.toLowerCase() == "left") return "L";
  if (result.toLowerCase() == "right") return "R";
  throw new InputError("Invalid direction: " + result);
}
```

The new error class extends `Error`. It doesn't define its own constructor, which means that it inherits the `Error` constructor, which expects a string message as argument. In fact, it doesn't define anything at all—the class is empty. `InputError` objects behave like `Error` objects, except that they have a different class by which we can recognize them.

Now the loop can catch these more carefully.

```

for (;;) {
  try {
    let dir = promptDirection("Where?");
    console.log("You chose ", dir);
    break;
  } catch (e) {
    if (e instanceof InputError) {
      console.log("Not a valid direction. Try again.");
    } else {
      throw e;
    }
  }
}

```

This will catch only instances of `InputError` and let unrelated exceptions through. If you reintroduce the typo, the undefined binding error will be properly reported.

ASSERTIONS

Assertions are checks inside a program that verify that something is the way it is supposed to be. They are used not to handle situations that can come up in normal operation but to find programmer mistakes.

If, for example, `firstElement` is described as a function that should never be called on empty arrays, we might write it like this:

```

function firstElement(array) {
  if (array.length == 0) {
    throw new Error("firstElement called with []");
  }
}

```

```

}
return array[0];
}

```

Now, instead of silently returning undefined (which you get when reading an array property that does not exist), this will loudly blow up your program as soon as you misuse it. This makes it less likely for such mistakes to go unnoticed and easier to find their cause when they occur.

I do not recommend trying to write assertions for every possible kind of bad input. That'd be a lot of work and would lead to very noisy code. You'll want to reserve them for mistakes that are easy to make (or that you find yourself making).

SUMMARY

An important part of programming is finding, diagnosing, and fixing bugs. Problems can become easier to notice if you have an automated test suite or add assertions to your programs.

Problems caused by factors outside the program's control should usually be actively planned for. Sometimes, when the problem can be handled locally, special return values are a good way to track them. Otherwise, exceptions may be preferable.

Throwing an exception causes the call stack to be unwound until the next enclosing `try/catch` block or until the bottom of the stack. The exception value will be given to the `catch` block that catches it, which should verify that it is actually the expected kind of exception and then

do something with it. To help address the unpredictable control flow caused by exceptions, *finally* blocks can be used to ensure that a piece of code *always* runs when a block finishes.

EXERCISES

RETRY

Say you have a function `primitiveMultiply` that in 20 percent of cases multiplies two numbers and in the other 80 percent of cases raises an exception of type `MultiplicatorUnitFailure`. Write a function that wraps this clunky function and just keeps trying until a call succeeds, after which it returns the result.

Make sure you handle only the exceptions you are trying to handle.

THE LOCKED BOX

Consider the following (rather contrived) object:

```
const box = new class {
  locked = true;
  #content = [];

  unlock() { this.locked = false; }
  lock() { this.locked = true; }
  get content() {
    if (this.locked) throw new Error("Locked!");
    return this.#content;
  }
}
```

```
};
```

It is a box with a lock. There is an array in the box, but you can get at it only when the box is unlocked.

Write a function called `withBoxUnlocked` that takes a function value as argument, unlocks the box, runs the function, and then ensures that the box is locked again before returning, regardless of whether the argument function returned normally or threw an exception.

For extra points, make sure that if you call `withBoxUnlocked` when the box is already unlocked, the box stays unlocked.

“Some people, when confronted with a problem, think ‘I know, I’ll use regular expressions.’ Now they have two problems.”
—Jamie Zawinski

CHAPTER 9

REGULAR EXPRESSIONS

Programming tools and techniques survive and spread in a chaotic, evolutionary way. It’s not always the best or most brilliant ones that win, but rather the ones that function well enough within the right niche or that happen to be integrated with another successful piece of technology.

In this chapter, I will discuss one such tool, *regular expressions*. Regular expressions are a way to describe patterns in string data. They form a small, separate language that is part of JavaScript and many other languages and systems.

Regular expressions are both terribly awkward and extremely useful. Their syntax is cryptic and the programming interface JavaScript provides for them is clumsy. But they are a powerful tool for inspecting and processing strings. Properly understanding regular expressions will make you a more effective programmer.

CREATING A REGULAR EXPRESSION

A regular expression is a type of object. It can be either constructed with the `RegExp` constructor or written as a literal value by enclosing a pattern in forward slash (`/`) characters.

```
let re1 = new RegExp("abc");  
let re2 = /abc/;
```

Both of those regular expression objects represent the same pattern: an *a* character followed by a *b* followed by a *c*.

When using the `RegExp` constructor, the pattern is written as a normal string, so the usual rules apply for backslashes.

The second notation, where the pattern appears between slash characters, treats backslashes somewhat differently. First, since a forward slash ends the pattern, we need to put a backslash before any forward slash that we want to be *part* of the pattern. In addition, backslashes that aren’t part of special character codes (like `\n`) will be *preserved*, rather than ignored as they are in strings, and change the meaning of the pattern. Some characters, such as question marks and plus signs, have special meanings in regular expressions and must be preceded by a backslash if they are meant to represent the character itself.

```
let aPlus = /\+/;
```

TESTING FOR MATCHES

Regular expression objects have a number of methods. The simplest one is `test`. If you pass it a string, it will return a Boolean telling you whether the string contains a match of the pattern in the expression.

```
console.log(/abc/.test("abcde"));
// → true
console.log(/abc/.test("abxde"));
// → false
```

A regular expression consisting of only nonspecial characters simply represents that sequence of characters. If *abc* occurs anywhere in the string we are testing against (not just at the start), `test` will return `true`.

SETS OF CHARACTERS

Finding out whether a string contains *abc* could just as well be done with a call to `indexOf`. Regular expressions are useful because they allow us to describe more complicated patterns.

Say we want to match any number. In a regular expression, putting a set of characters between square brackets makes that part of the expression match any of the characters between the brackets.

Both of the following expressions match all strings that contain a digit:

```
console.log(/[0123456789]/.test("in 1992"));
```

```
// → true
console.log(/[0-9]/.test("in 1992"));
// → true
```

Within square brackets, a hyphen (-) between two characters can be used to indicate a range of characters, where the ordering is determined by the character's Unicode number. Characters 0 to 9 sit right next to each other in this ordering (codes 48 to 57), so `[0-9]` covers all of them and matches any digit.

A number of common character groups have their own built-in shortcuts. Digits are one of them: `\d` means the same thing as `[0-9]`.

- `\d` Any digit character
- `\w` An alphanumeric character ("word character")
- `\s` Any whitespace character (space, tab, newline, and similar)
- `\D` A character that is *not* a digit
- `\W` A nonalphanumeric character
- `\S` A nonwhitespace character
- `.` Any character except for newline

You could match a date and time format like 01-30-2003 15:20 with the following expression:

```
let dateTime = /\d\d-\d\d-\d\d\d\d \d\d:\d\d/;
console.log(dateTime.test("01-30-2003 15:20"));
// → true
console.log(dateTime.test("30-jan-2003 15:20"));
// → false
```

That regular expression looks completely awful, doesn't it? Half of it is backslashes, producing a background noise that makes it hard to spot

the actual pattern expressed. We'll see a slightly improved version of this expression later.

These backslash codes can also be used inside square brackets. For example, `[\d.]` means any digit or a period character. The period itself, between square brackets, loses its special meaning. The same goes for other special characters, such as the plus sign (`+`).

To *invert* a set of characters—that is, to express that you want to match any character *except* the ones in the set—you can write a caret (`^`) character after the opening bracket.

```
let nonBinary = /^[^01]/;
console.log(nonBinary.test("1100100010100110"));
// → false
console.log(nonBinary.test("0111010112101001"));
// → true
```

INTERNATIONAL CHARACTERS

Because of JavaScript's initial simplistic implementation and the fact that this simplistic approach was later set in stone as standard behavior, JavaScript's regular expressions are rather dumb about characters that do not appear in the English language. For example, as far as JavaScript's regular expressions are concerned, a “word character” is only one of the 26 characters in the Latin alphabet (uppercase or lowercase), decimal digits, and, for some reason, the underscore character. Things like *é* or *ß*, which most definitely are word characters, will not match `\w` (and *will* match uppercase `\W`, the nonword category).

By a strange historical accident, `\s` (whitespace) does not have this problem and matches all characters that the Unicode standard considers whitespace, including things like the nonbreaking space and the Mongolian vowel separator.

It is possible to use `\p` in a regular expression to match all characters to which the Unicode standard assigns a given property. This allows us to match things like letters in a more cosmopolitan way. However, again due to compatibility with the original language standards, those are recognized only when you put a `u` character (for Unicode) after the regular expression.

<code>\p{L}</code>	Any letter
<code>\p{N}</code>	Any numeric character
<code>\p{P}</code>	Any punctuation character
<code>\p{L}</code>	Any nonletter (uppercase <code>P</code> inverts)
<code>\p{Script=Hangul}</code>	Any character from the given script (see Chapter 5)

Using `\w` for text processing that may need to handle non-English text (or even English text with borrowed words like “*cliché*”) is a liability, since it won't treat characters like “*é*” as letters. Though they tend to be a bit more verbose, `\p` property groups are more robust.

```
console.log(/\p{L}/u.test("a"));
// → true
console.log(/\p{L}/u.test("!"));
// → false
console.log(/\p{Script=Greek}/u.test("a"));
// → true
console.log(/\p{Script=Arabic}/u.test("a"));
// → false
```

On the other hand, if you are matching numbers in order to do something with them, you often do want `\d` for digits, since converting arbitrary numeric characters into a JavaScript number is not something that a function like `Number` can do for you.

REPEATING PARTS OF A PATTERN

We now know how to match a single digit. What if we want to match a whole number—a sequence of one or more digits?

When you put a plus sign (+) after something in a regular expression, it indicates that the element may be repeated more than once. Thus, `/\d+/,` matches one or more digit characters.

```
console.log(/\d+/.test("123"));
// → true
console.log(/\d+/.test(""));
// → false
console.log(/\d+/.test("123"));
// → true
console.log(/\d+/.test(""));
// → true
```

The star (*) has a similar meaning but also allows the pattern to match zero times. Something with a star after it never prevents a pattern from matching—it'll just match zero instances if it can't find any suitable text to match.

A question mark (?) makes a part of a pattern *optional*, meaning it may occur zero times or one time. In the following example, the *u*

character is allowed to occur, but the pattern also matches when it is missing:

```
let neighbor = /neighbou?r/;
console.log(neighbor.test("neighbour"));
// → true
console.log(neighbor.test("neighbor"));
// → true
```

To indicate that a pattern should occur a precise number of times, use braces. Putting {4} after an element, for example, requires it to occur exactly four times. It is also possible to specify a range this way: {2,4} means the element must occur at least twice and at most four times.

Here is another version of the date and time pattern that allows both single- and double-digit days, months, and hours. It is also slightly easier to decipher.

```
let dateTime = /\d{1,2}-\d{1,2}-\d{4} \d{1,2}:\d{2}/;
console.log(dateTime.test("1-30-2003 8:45"));
// → true
```

You can also specify open-ended ranges when using braces by omitting the number after the comma. For example, {5,} means five or more times.

GROUPING SUBEXPRESSIONS

To use an operator like * or + on more than one element at a time, you must use parentheses. A part of a regular expression that is enclosed in

parentheses counts as a single element as far as the operators following it are concerned.

```
let cartoonCrying = /boo+(hoo+)+/i;
console.log(cartoonCrying.test("Boohooooohoooo"));
// → true
```

The first and second `+` characters apply only to the second `o` in `boo` and `hoo`, respectively. The third `+` applies to the whole group `(hoo+)`, matching one or more sequences like that.

The `i` at the end of the expression in the example makes this regular expression case insensitive, allowing it to match the uppercase *B* in the input string, even though the pattern is itself all lowercase.

MATCHES AND GROUPS

The `test` method is the absolute simplest way to match a regular expression. It tells you only whether it matched and nothing else. Regular expressions also have an `exec` (execute) method that will return `null` if no match was found and return an object with information about the match otherwise.

```
let match = /\d+/.exec("one two 100");
console.log(match);
// → ["100"]
console.log(match.index);
// → 8
```

An object returned from `exec` has an `index` property that tells us *where*

in the string the successful match begins. Other than that, the object looks like (and in fact is) an array of strings, whose first element is the string that was matched. In the previous example, this is the sequence of digits that we were looking for.

String values have a `match` method that behaves similarly.

```
console.log("one two 100".match(/\d+/));
// → ["100"]
```

When the regular expression contains subexpressions grouped with parentheses, the text that matched those groups will also show up in the array. The whole match is always the first element. The next element is the part matched by the first group (the one whose opening parenthesis comes first in the expression), then the second group, and so on.

```
let quotedText = /'([^']*)*'/;
console.log(quotedText.exec("she said 'hello'"));
// → ["'hello'", "hello"]
```

When a group does not end up being matched at all (for example, when followed by a question mark), its position in the output array will hold `undefined`. When a group is matched multiple times (for example, when followed by a `+`), only the last match ends up in the array.

```
console.log(/bad{1y}?/.exec("bad"));
// → ["bad", undefined]
console.log(/(\d)+/.exec("123"));
// → ["123", "3"]
```

If you want to use parentheses purely for grouping, without having

them show up in the array of matches, you can put `?:` after the opening parenthesis.

```
console.log(/(?:na)+/.exec("banana"));
// → ["nana"]
```

Groups can be useful for extracting parts of a string. If we don't just want to verify whether a string contains a date but also extract it and construct an object that represents it, we can wrap parentheses around the digit patterns and directly pick the date out of the result of `exec`.

But first we'll take a brief detour to discuss the built-in way to represent date and time values in JavaScript.

THE DATE CLASS

JavaScript has a standard `Date` class for representing dates, or rather, points in time. If you simply create a date object using `new`, you get the current date and time.

```
console.log(new Date());
// → Fri Feb 02 2024 18:03:06 GMT+0100 (CET)
```

You can also create an object for a specific time.

```
console.log(new Date(2009, 11, 9));
// → Wed Dec 09 2009 00:00:00 GMT+0100 (CET)
console.log(new Date(2009, 11, 9, 12, 59, 59, 999));
// → Wed Dec 09 2009 12:59:59 GMT+0100 (CET)
```

JavaScript uses a convention where month numbers start at zero (so

December is 11), yet day numbers start at one. This is confusing and silly. Be careful.

The last four arguments (hours, minutes, seconds, and milliseconds) are optional and taken to be zero when not given.

Timestamps are stored as the number of milliseconds since the start of 1970, in the UTC time zone. This follows a convention set by “Unix time”, which was invented around that time. You can use negative numbers for times before 1970. The `getTime` method on a date object returns this number. It is big, as you can imagine.

```
console.log(new Date(2013, 11, 19).getTime());
// → 1387407600000
console.log(new Date(1387407600000));
// → Thu Dec 19 2013 00:00:00 GMT+0100 (CET)
```

If you give the `Date` constructor a single argument, that argument is treated as such a millisecond count. You can get the current millisecond count by creating a new `Date` object and calling `getTime` on it or by calling the `Date.now` function.

Date objects provide methods such as `getFullYear`, `getMonth`, `getDate`, `getHours`, `getMinutes`, and `getSeconds` to extract their components. Besides `getFullYear` there's also `getYear`, which gives you the year minus 1900 (such as 98 or 125) and is mostly useless.

Putting parentheses around the parts of the expression that we are interested in, we can now create a date object from a string.

```
function getDate(string) {
  let [, month, day, year] =
    /(\d{1,2})-(\d{1,2})-(\d{4})/.exec(string);
```

```

    return new Date(year, month - 1, day);
}
console.log(getDate("1-30-2003"));
// → Thu Jan 30 2003 00:00:00 GMT+0100 (CET)

```

The underscore (`_`) binding is ignored and used only to skip the full match element in the array returned by `exec`.

BOUNDARIES AND LOOK-AHEAD

Unfortunately, `getDate` will also happily extract a date from the string `"100-1-30000"`. A match may happen anywhere in the string, so in this case, it'll just start at the second character and end at the second-to-last character.

If we want to enforce that the match must span the whole string, we can add the markers `^` and `$`. The caret matches the start of the input string, whereas the dollar sign matches the end. Thus `/^d+$/` matches a string consisting entirely of one or more digits, `/x!/?` matches any string that starts with an exclamation mark, and `/x*/` does not match any string (there cannot be an `x` before the start of the string).

There is also a `\b` marker that matches *word boundaries*, positions that have a word character on one side, and a nonword character on the other. Unfortunately, these use the same simplistic concept of word characters as `\w` and are therefore not very reliable.

Note that these boundary markers don't match any actual characters. They just enforce that a given condition holds at the place where it appears in the pattern.

Look-ahead tests do something similar. They provide a pattern and will make the match fail if the input doesn't match that pattern, but don't actually move the match position forward. They are written between `(?=` and `)`.

```

console.log(/a(?:e)/.exec("braeburn"));
// → ["a"]
console.log(/a(?:! )/.exec("a b"));
// → null

```

The `e` in the first example is necessary to match, but is not part of the matched string. The `(?! )` notation expresses a *negative* look-ahead. This matches only if the pattern in the parentheses *doesn't* match, causing the second example to match only a characters that don't have a space after them.

CHOICE PATTERNS

Say we want to know whether a piece of text contains not only a number but a number followed by one of the words *pig*, *cow*, or *chicken*, or any of their plural forms.

We could write three regular expressions and test them in turn, but there is a nicer way. The pipe character `(|)` denotes a choice between the pattern to its left and the pattern to its right. We can use it in expressions like this:

```

let animalCount = /\d+ (pig|cow|chicken)s?/;
console.log(animalCount.test("15 pigs"));

```



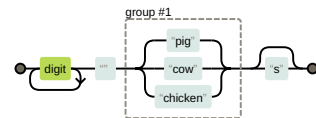
```
// → true
console.log(animalsCount.test("15 pugs"));
// → false
```

Parentheses can be used to limit the part of the pattern to which the pipe operator applies, and you can put multiple such operators next to each other to express a choice between more than two alternatives.

THE MECHANICS OF MATCHING

Conceptually, when you use `exec` or `test`, the regular expression engine looks for a match in your string by trying to match the expression first from the start of the string, then from the second character, and so on until it finds a match or reaches the end of the string. It'll either return the first match that can be found or fail to find any match at all.

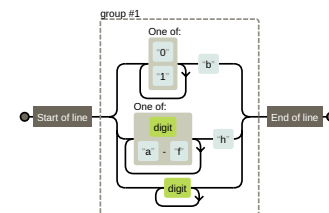
To do the actual matching, the engine treats a regular expression something like a flow diagram. This is the diagram for the livestock expression in the previous example:



If we can find a path from the left side of the diagram to the right side, our expression matches. We keep a current position in the string, and every time we move through a box, we verify that the part of the string after our current position matches that box.

BACKTRACKING

The regular expression `/^(?!(01)+b|(?da-f)+h|d+)$/` matches either a binary number followed by a b, a hexadecimal number (that is, base 16, with the letters a to f standing for the digits 10 to 15) followed by an h, or a regular decimal number with no suffix character. This is the corresponding diagram:



When matching this expression, the top (binary) branch will often be entered even though the input does not actually contain a binary number. When matching the string "103", for example, it becomes clear only at the 3 that we are in the wrong branch. The string *does* match the expression, just not the branch we are currently in.

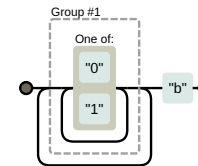
So the matcher *backtracks*. When entering a branch, it remembers its current position (in this case, at the start of the string, just past the first boundary box in the diagram) so that it can go back and try another branch if the current one does not work out. For the string "103", after encountering the 3 character, the matcher starts trying the branch for hexadecimal numbers, which fails again because there is no h after the number. It then tries the decimal number branch. This one fits, and a match is reported after all.

The matcher stops as soon as it finds a full match. This means that if multiple branches could potentially match a string, only the first one (ordered by where the branches appear in the regular expression) is used.

Backtracking also happens for repetition operators like + and *. If you match `/^.*x/` against "abcxe", the `.*` part will first try to consume the whole string. The engine will then realize that it needs an x to match the pattern. Since there is no x past the end of the string, the star operator tries to match one character less. But the matcher doesn't find an x after abcx either, so it backtracks again, matching the star operator to just abc. *Now* it finds an x where it needs it and reports a successful match from positions 0 to 4.

It is possible to write regular expressions that will do a *lot* of back-

tracking. This problem occurs when a pattern can match a piece of input in many different ways. For example, if we get confused while writing a binary-number regular expression, we might accidentally write something like `/([01]+)+b/`.



If that tries to match some long series of zeros and ones with no trailing *b* character, the matcher first goes through the inner loop until it runs out of digits. Then it notices there is no *b*, so it backtracks one position, goes through the outer loop once, and gives up again, trying to backtrack out of the inner loop once more. It will continue to try every possible route through these two loops. This means the amount of work *doubles* with each additional character. For even just a few dozen characters, the resulting match will take practically forever.

THE REPLACE METHOD

String values have a `replace` method that can be used to replace part of the string with another string.

```
console.log("papa".replace("p", "m"));
// → mapa
```

The first argument can also be a regular expression, in which case the first match of the regular expression is replaced. When a `g` option (for *global*) is added after the regular expression, *all* matches in the string will be replaced, not just the first.

```
console.log("Borobudur".replace(/ou/, "a"));
// → Barobudur
console.log("Borobudur".replace(/ou/g, "a"));
// → Barabadar
```

The real power of using regular expressions with `replace` comes from the fact that we can refer to matched groups in the replacement string. For example, say we have a big string containing the names of people, one name per line, in the format `Lastname, Firstname`. If we want to swap these names and remove the comma to get a `Firstname Lastname` format, we can use the following code:

```
console.log(
  "Liskov, Barbara\nMcCarthy, John\nMilner, Robin"
  .replace(/(\p{L}+), (\p{L}+)/gu, "$2 $1"));
// → Barbara Liskov
// → John McCarthy
```

```
// Robin Milner
```

The `$1` and `$2` in the replacement string refer to the parenthesized groups in the pattern. `$1` is replaced by the text that matched against the first group, `$2` by the second, and so on, up to `$9`. The whole match can be referred to with `$&`.

It is possible to pass a function—rather than a string—as the second argument to `replace`. For each replacement, the function will be called with the matched groups (as well as the whole match) as arguments, and its return value will be inserted into the new string.

Here's an example:

```
let stock = "1 lemon, 2 cabbages, and 101 eggs";
function minusOne(match, amount, unit) {
  amount = Number(amount) - 1;
  if (amount == 1) { // only one left, remove the 's'
    unit = unit.slice(0, unit.length - 1);
  } else if (amount == 0) {
    amount = "no";
  }
  return amount + " " + unit;
}
console.log(stock.replace(/(\d+) (\p{L}+)/gu, minusOne));
// → no lemon, 1 cabbage, and 100 eggs
```

This code takes a string, finds all occurrences of a number followed by an alphanumeric word, and returns a string that has one less of every such quantity.

The `(\d+)` group ends up as the `amount` argument to the function, and

the `(\p{L}+)` group gets bound to `unit`. The function converts `amount` to a number—which always works, since it matched `\d+` earlier—and makes some adjustments in case there is only one or zero left.

GREED

We can use `replace` to write a function that removes all comments from a piece of JavaScript code. Here is a first attempt:

```
function stripComments(code) {
  return code.replace(/\/\/*.*?\/\/*.*?\/g, "");
}
console.log(stripComments("1 + /* 2 */3"));
// → 1 + 3
console.log(stripComments("x = 10; // ten!"));
// → x = 10;
console.log(stripComments("1 /* a */+/* b */ 1"));
// → 1 + 1
```

The part before the `|` operator matches two slash characters followed by any number of non-newline characters. The part for multiline comments is more involved. We use `[^]` (any character that is not in the empty set of characters) as a way to match any character. We cannot just use a period here because block comments can continue on a new line, and the period character does not match newline characters.

But the output for the last line appears to have gone wrong. Why?

The `[^]*` part of the expression, as I described in the section on backtracking, will first match as much as it can. If that causes the next

part of the pattern to fail, the matcher moves back one character and tries again from there. In the example, the matcher first tries to match the whole rest of the string and then moves back from there. It will find an occurrence of `*/` after going back four characters and match that. This is not what we wanted—the intention was to match a single comment, not to go all the way to the end of the code and find the end of the last block comment.

Because of this behavior, we say the repetition operators `(*, *, ?, and {})` are *greedy*, meaning they match as much as they can and backtrack from there. If you put a question mark after them (`*, *, ?, {}, {}?`), they become nongreedy and start by matching as little as possible, matching more only when the remaining pattern does not fit the smaller match.

And that is exactly what we want in this case. By having the star match the smallest stretch of characters that brings us to a `*/`, we consume one block comment and nothing more.

```
function stripComments(code) {
  return code.replace(/\/\/*.*?\/\/*.*?\/g, "");
}
console.log(stripComments("1 /* a */+/* b */ 1"));
// → 1 + 1
```

A lot of bugs in regular expression programs can be traced to unintentionally using a greedy operator where a nongreedy one would work better. When using a repetition operator, prefer the nongreedy variant.

DYNAMICALLY CREATING REGEXP OBJECTS

In some cases you may not know the exact pattern you need to match against when you are writing your code. Say you want to test for the user's name in a piece of text. You can build up a string and use the `RegExp` constructor on that.

```
let name = "harry";
let regexp = new RegExp("(^|\\s)" + name + "(\\s|$)", "gi");
console.log(regexp.test("Harry is a dodgy character."));
// → true
```

When creating the `\\s` part of the string, we have to use two backslashes because we are writing them in a normal string, not a slash-enclosed regular expression. The second argument to the `RegExp` constructor contains the options for the regular expression—in this case, `"gi"` for global and case insensitive.

But what if the name is `"deah1[]rd"` because our user is a nerdy teenager? That would result in a nonsensical regular expression that won't actually match the user's name.

To work around this, we can add backslashes before any character that has a special meaning.

```
let name = "dea+h1[]rd";
let escaped = name.replace(/[\\.[+*?(){}|^$]/g, "\\$&");
let regexp = new RegExp("(^|\\s)" + escaped + "(\\s|$)",
    "gi");
let text = "This dea+h1[]rd guy is super annoying.";
console.log(regexp.test(text));
```

```
// → true
```

THE SEARCH METHOD

While the `indexOf` method on strings cannot be called with a regular expression, there is another method, `search`, that does expect a regular expression. Like `indexOf`, it returns the first index on which the expression was found, or `-1` when it wasn't found.

```
console.log(" word".search(/\\s/));
// → 2
console.log("  ".search(/\\s/));
// → -1
```

Unfortunately, there is no way to indicate that the match should start at a given offset (like we can with the second argument to `indexOf`), which would often be useful.

THE LASTINDEX PROPERTY

The `exec` method similarly does not provide a convenient way to start searching from a given position in the string. But it does provide an *inconvenient* way.

Regular expression objects have properties. One such property is `source`, which contains the string that expression was created from. Another property is `lastIndex`, which controls, in some limited circumstances, where the next match will start.

Those circumstances are that the regular expression must have the global (`g`) or sticky (`y`) option enabled, and the match must happen through the `exec` method. Again, a less confusing solution would have been to just allow an extra argument to be passed to `exec`, but confusion is an essential feature of JavaScript's regular expression interface.

```
let pattern = /y/g;
pattern.lastIndex = 3;
let match = pattern.exec("xyzy");
console.log(match.index);
// → 4
console.log(pattern.lastIndex);
// → 5
```

If the match was successful, the call to `exec` automatically updates the `lastIndex` property to point after the match. If no match was found, `lastIndex` is set back to 0, which is also the value it has in a newly constructed regular expression object.

The difference between the global and the sticky options is that when sticky is enabled, the match will succeed only if it starts directly at `lastIndex`, whereas with global, it will search ahead for a position where a match can start.

```
let global = /abc/g;
console.log(global.exec("xyz abc"));
// → ["abc"]
let sticky = /abc/y;
console.log(sticky.exec("xyz abc"));
// → null
```

When using a shared regular expression value for multiple `exec` calls, these automatic updates to the `lastIndex` property can cause problems. Your regular expression might be accidentally starting at an index left over from a previous call.

```
let digit = /\d/g;
console.log(digit.exec("here it is: 1"));
// → ["1"]
console.log(digit.exec("and now: 1"));
// → null
```

Another interesting effect of the global option is that it changes the way the `match` method on strings works. When called with a global expression, instead of returning an array similar to that returned by `exec`, `match` will find *all* matches of the pattern in the string and return an array containing the matched strings.

```
console.log("Banana".match(/an/g));
// → ["an", "an"]
```

So be cautious with global regular expressions. The cases where they are necessary—calls to `replace` and places where you want to explicitly use `lastIndex`—are typically the situations where you want to use them.

A common thing to do is to find all the matches of a regular expression in a string. We can do this by using the `matchAll` method.

```
let input = "A string with 3 numbers in it... 42 and 88.";
let matches = input.matchAll(/\d+/g);
for (let match of matches) {
  console.log("Found", match[0], "at", match.index);
}
```

```
}
// → Found 3 at 14
//   Found 42 at 33
//   Found 88 at 40
```

This method returns an array of match arrays. The regular expression given to `matchAll` *must* have `g` enabled.

PARSING AN INI FILE

To conclude the chapter, we'll look at a problem that calls for regular expressions. Imagine we are writing a program to automatically collect information about our enemies from the internet. (We will not actually write that program here, just the part that reads the configuration file. Sorry.) The configuration file looks like this:

```
searchengine=https://duckduckgo.com/?q=$1
spitefulness=9.7

; comments are preceded by a semicolon...
; each section concerns an individual enemy
[larry]
fullname=Larry Doe
type=kindergarten bully
website=http://www.geocities.com/CapeCanaveral/11451

[davaeorn]
fullname=Davaeorn
type=evil wizard
```

```
outputdir=/home/marijn/enemies/davaeorn
```

The exact rules for this format—which is a widely used file format, usually called an *INI* file—are as follows:

- Blank lines and lines starting with semicolons are ignored.
- Lines wrapped in [and] start a new section.
- Lines containing an alphanumeric identifier followed by an = character add a setting to the current section.
- Anything else is invalid.

Our task is to convert a string like this into an object whose properties hold strings for settings written before the first section header and subobjects for sections, with those subobjects holding the section's settings.

Since the format has to be processed line by line, splitting up the file into separate lines is a good start. We saw the `split` method in Chapter 4. Some operating systems, however, use not just a newline character to separate lines but a carriage return character followed by a newline ("`\r\n`"). Given that the `split` method also allows a regular expression as its argument, we can use a regular expression like `/\r?\n/` to split in a way that allows both "`\n`" and "`\r\n`" between lines.

```
function parseINI(string) {
  // Start with an object to hold the top-level fields
  let result = {};
```

```

let section = result;
for (let line of string.split(/\r?\n/)) {
  let match;
  if (match = line.match(/^(w+)=(.*)$/)) {
    section[match[1]] = match[2];
  } else if (match = line.match(/^(.*)\$/)) {
    section = result[match[1]] = {};
  } else if (!/^\s*(;|$/).test(line)) {
    throw new Error("Line '" + line + "' is not valid.");
  }
};
return result;
}

console.log(parseINI(`
name=Vasilis
[address]
city=Tessaloniki`));
// → {name: "Vasilis", address: {city: "Tessaloniki"}}

```

The code goes over the file's lines and builds up an object. Properties at the top are stored directly into that object, whereas properties found in sections are stored in a separate section object. The `section` binding points at the object for the current section.

There are two kinds of significant lines—section headers or property lines. When a line is a regular property, it is stored in the current section. When it is a section header, a new section object is created, and `section` is set to point at it.

Note the recurring use of `*` and `$` to make sure the expression matches

the whole line, not just part of it. Leaving these out results in code that mostly works but behaves strangely for some input, which can be a difficult bug to track down.

The pattern `if (match = string.match(...))` makes use of the fact that the value of an assignment expression (`=`) is the assigned value. You often aren't sure that your call to `match` will succeed, so you can access the resulting object only inside an `if` statement that tests for this. To not break the pleasant chain of `else if` forms, we assign the result of the match to a binding and immediately use that assignment as the test for the `if` statement.

If a line is not a section header or a property, the function checks whether it is a comment or an empty line using the expression `/^\s*(;|$/)` to match lines that either contain only whitespace, or whitespace followed by a semicolon (making the rest of the line a comment). When a line doesn't match any of the expected forms, the function throws an exception.

CODE UNITS AND CHARACTERS

Another design mistake that's been standardized in JavaScript regular expressions is that by default, operators like `.` or `?` work on code units (as discussed in [Chapter 5](#)), not actual characters. This means characters that are composed of two code units behave strangely.

```

console.log(/[C]{3}/.test("CCC"));
// → false
console.log(/<.>/.test("<C>"));

```



```
// → false
console.log(/<.>/u.test("<🌹>"));
// → true
```

The problem is that the 🌹 in the first line is treated as two code units, and {3} is applied only to the second unit. Similarly, the dot matches a single code unit, not the two that make up the rose emoji.

You must add the `u` (Unicode) option to your regular expression to make it treat such characters properly.

```
console.log(/🌹{3}/u.test("🌹🌹🌹"));
// → true
```

SUMMARY

Regular expressions are objects that represent patterns in strings. They use their own language to express these patterns.

<code>/abc/</code>	A sequence of characters
<code>/[abc]/</code>	Any character from a set of characters
<code>/[^abc]/</code>	Any character <i>not</i> in a set of characters
<code>/[0-9]/</code>	Any character in a range of characters
<code>/x*/</code>	One or more occurrences of the pattern <code>x</code>
<code>/x+?/</code>	One or more occurrences, nongreedy
<code>/x*/</code>	Zero or more occurrences
<code>/x?/</code>	Zero or one occurrence
<code>/x{2,4}/</code>	Two to four occurrences
<code>/(abc)/</code>	A group
<code>/a b c/</code>	Any one of several patterns
<code>/\d/</code>	Any digit character
<code>/\w/</code>	An alphanumeric character ("word character")
<code>/\s/</code>	Any whitespace character
<code>/./</code>	Any character except newlines
<code>/\p{L}/u</code>	Any letter character
<code>/^/</code>	Start of input
<code>/\$/</code>	End of input
<code>/(?=a)/</code>	A look-ahead test

A regular expression has a method `test` to test whether a given string matches it. It also has a method `exec` that, when a match is found, returns an array containing all matched groups. Such an array has an `index` property that indicates where the match started.

Strings have a `match` method to match them against a regular expression and a `search` method to search for one, returning only the starting position of the match. Their `replace` method can replace matches of a pattern with a replacement string or function.

Regular expressions can have options, which are written after the closing slash. The `i` option makes the match case insensitive. The `g` option makes the expression *global*, which, among other things, causes the `replace` method to replace all instances instead of just the first. The `y` option makes an expression sticky, which means that it will not search ahead and skip part of the string when looking for a match. The `u` option turns on Unicode mode, which enables `\p` syntax and fixes a number of problems around the handling of characters that take up two code units.

Regular expressions are a sharp tool with an awkward handle. They simplify some tasks tremendously but can quickly become unmanageable when applied to complex problems. Part of knowing how to use them is resisting the urge to try to shoehorn things into them that they cannot cleanly express.

EXERCISES

It is almost unavoidable that, in the course of working on these exercises, you will get confused and frustrated by some regular expression's inexplicable behavior. Sometimes it helps to enter your expression into an online tool like *debuggex.com* to see whether its visualization corresponds to what you intended and to experiment with the way it responds to various input strings.

REGEXP GOLF

Code golf is a term used for the game of trying to express a particular program in as few characters as possible. Similarly, *regexp golf* is the practice of writing as tiny a regular expression as possible to match a given pattern and *only* that pattern.

For each of the following items, write a regular expression to test whether the given pattern occurs in a string. The regular expression should match only strings containing the pattern. When your expression works, see whether you can make it any smaller.

1. *car* and *cat*
2. *pop* and *prop*
3. *ferret*, *ferry*, and *ferrari*
4. Any word ending in *ious*
5. A whitespace character followed by a period, comma, colon, or semicolon
6. A word longer than six letters
7. A word without the letter *e* (or *E*)

Refer to the table in the chapter summary for help. Test each solution with a few test strings.

QUOTING STYLE

Imagine you have written a story and used single quotation marks throughout to mark pieces of dialogue. Now you want to replace all the dialogue quotes with double quotes, while keeping the single quotes used in contractions like *aren't*.

Think of a pattern that distinguishes these two kinds of quote usage and craft a call to the `replace` method that does the proper replacement.

NUMBERS AGAIN

Write an expression that matches only JavaScript-style numbers. It must support an optional minus *or* plus sign in front of the number, the decimal dot, and exponent notation—`5e-3` or `1E10`—again with an optional sign in front of the exponent. Also note that it is not necessary for there to be digits in front of or after the dot, but the number cannot be a dot alone. That is, `.5` and `5.` are valid JavaScript numbers, but a lone dot isn't.

“Write code that is easy to delete, not easy to extend.”

—Tef, programming is terrible

CHAPTER 10

MODULES

Ideally, a program has a clear, straightforward structure. The way it works is easy to explain, and each part plays a well-defined role.

In practice, programs grow organically. Pieces of functionality are added as the programmer identifies new needs. Keeping such a program well structured requires constant attention and work. This is work that will pay off only in the future, the *next* time someone works on the program, so it's tempting to neglect it and allow the various parts of the program to become deeply entangled.

This causes two practical issues. First, understanding an entangled system is hard. If everything can touch everything else, it is difficult to look at any given piece in isolation. You are forced to build up a holistic understanding of the entire thing. Second, if you want to use any of the functionality from such a program in another situation, rewriting it may be easier than trying to disentangle it from its context.

The phrase “big ball of mud” is often used for such large, structureless programs. Everything sticks together, and when you try to pick out a piece, the whole thing comes apart, and you succeed only in making a

mess.

MODULAR PROGRAMS

Modules are an attempt to avoid these problems. A module is a piece of program that specifies which other pieces it relies on and which functionality it provides for other modules to use (its *interface*).

Module interfaces have a lot in common with object interfaces, as we saw them in [Chapter 6](#). They make part of the module available to the outside world and keep the rest private.

But the interface that a module provides for others to use is only half the story. A good module system also requires modules to specify which code *they* use from other modules. These relations are called *dependencies*. If module A uses functionality from module B, it is said to *depend* on that module. When these are clearly specified in the module itself, they can be used to figure out which other modules need to be present to be able to use a given module and to automatically load dependencies.

When the ways in which modules interact with each other are explicit, a system becomes more like LEGO, where pieces interact through well-defined connectors, and less like mud, where everything mixes with everything else.

ES MODULES

The original JavaScript language did not have any concept of a module. All scripts ran in the same scope, and accessing a function defined in another script was done by referencing the global bindings created by that script. This actively encouraged accidental, hard-to-see entanglement of code and invited problems like unrelated scripts trying to use the same binding name.

Since ECMAScript 2015, JavaScript supports two different types of programs. *Scripts* behave in the old way: their bindings are defined in the global scope, and they have no way to directly reference other scripts. *Modules* get their own separate scope and support the `import` and `export` keywords, which aren't available in scripts, to declare their dependencies and interface. This module system is usually called *ES modules* (where *ES* stands for ECMAScript).

A modular program is composed of a number of such modules, wired together via their imports and exports.

The following example module converts between day names and numbers (as returned by `Date`'s `getDay` method). It defines a constant that is not part of its interface, and two functions that are. It has no dependencies.

```
const names = ["Sunday", "Monday", "Tuesday", "Wednesday",
               "Thursday", "Friday", "Saturday"];

export function dayName(number) {
  return names[number];
}
```

```
export function dayNumber(name) {
  return names.indexOf(name);
}
```

The `export` keyword can be put in front of a function, class, or binding definition to indicate that that binding is part of the module's interface. This makes it possible for other modules to use that binding by importing it.

```
import {dayName} from "./dayname.js";
let now = new Date();
console.log('Today is ${dayName(now.getDay())}');
// → Today is Monday
```

The `import` keyword, followed by a list of binding names in braces, makes bindings from another module available in the current module. Modules are identified by quoted strings.

How such a module name is resolved to an actual program differs by platform. The browser treats them as web addresses, whereas Node.js resolves them to files. When you run a module, all the other modules it depends on—and the modules *those* depend on—are loaded, and the exported bindings are made available to the modules that import them.

Import and export declarations cannot appear inside of functions, loops, or other blocks. They are immediately resolved when the module is loaded, regardless of how the code in the module executes. To reflect this, they must appear only in the outer module body.

A module's interface thus consists of a collection of named bindings, which other modules that depend on the module can access. Imported bindings can be renamed to give them a new local name using `as` after

their name.

```
import {dayName as nomDeJour} from "./dayname.js";
console.log(nomDeJour(3));
// → Wednesday
```

A module may also have a special export named `default`, which is often used for modules that only export a single binding. To define a default export, you write `export default` before an expression, a function declaration, or a class declaration.

```
export default ["Winter", "Spring", "Summer", "Autumn"];
```

Such a binding is imported by omitting the braces around the name of the import.

```
import seasonNames from "./seasonname.js";
```

To import all bindings from a module at the same time, you can use `import *`. You provide a name, and that name will be bound to an object holding all the module's exports. This can be useful when you are using a lot of different exports.

```
import * as dayName from "./dayname.js";
console.log(dayName.dayName(3));
// → Wednesday
```

PACKAGES

One of the advantages of building a program out of separate pieces and being able to run some of those pieces on their own is that you might be able to use the same piece in different programs.

But how do you set this up? Say I want to use the `parseINI` function from Chapter 9 in another program. If it is clear what the function depends on (in this case, nothing), I can just copy that module into my new project and use it. But then, if I find a mistake in the code, I'll probably fix it in whichever program I'm working with at the time and forget to also fix it in the other program.

Once you start duplicating code, you'll quickly find yourself wasting time and energy moving copies around and keeping them up to date. That's where *packages* come in. A package is a chunk of code that can be distributed (copied and installed). It may contain one or more modules and has information about which other packages it depends on. A package also usually comes with documentation explaining what it does so that people who didn't write it might still be able to use it.

When a problem is found in a package or a new feature is added, the package is updated. Now the programs that depend on it (which may also be packages) can copy the new version to get the improvements that were made to the code.

Working in this way requires infrastructure. We need a place to store and find packages and a convenient way to install and upgrade them. In the JavaScript world, this infrastructure is provided by NPM (<https://npmjs.com>).

NPM is two things: an online service where you can download (and

upload) packages, and a program (bundled with Node.js) that helps you install and manage them.

At the time of writing, there are more than three million different packages available on NPM. A large portion of those are rubbish, to be fair. But almost every useful, publicly available JavaScript package can be found on NPM. For example, an INI file parser, similar to the one we built in Chapter 9, is available under the package name `ini`.

Chapter 20 will show how to install such packages locally using the `npm` command line program.

Having quality packages available for download is extremely valuable. It means that we can often avoid reinventing a program that 100 people have written before and get a solid, well-tested implementation at the press of a few keys.

Software is cheap to copy, so once someone has written it, distributing it to other people is an efficient process. Writing it in the first place *is* work, though, and responding to people who have found problems in the code or who want to propose new features is even more work.

By default, you own the copyright to the code you write, and other people may use it only with your permission. But because some people are just nice and because publishing good software can help make you a little bit famous among programmers, many packages are published under a license that explicitly allows other people to use it.

Most code on NPM is licensed this way. Some licenses require you to also publish code that you build on top of the package under the same license. Others are less demanding, requiring only that you keep the license with the code as you distribute it. The JavaScript commu-

nity mostly uses the latter type of license. When using other people's packages, make sure you are aware of their licenses.

Now, instead of writing our own INI file parser, we can use one from NPM.

```
import {parse} from "ini";

console.log(parse("x = 10\ny = 20"));
// → {x: "10", y: "20"}
```

COMMONJS MODULES

Before 2015, when the JavaScript language had no built-in module system, people were already building large systems in JavaScript. To make that workable, they *needed* modules.

The community designed its own improvised module systems on top of the language. These use functions to create a local scope for the modules and regular objects to represent module interfaces.

Initially, people just manually wrapped their entire module in an “immediately invoked function expression” to create the module's scope and assigned their interface objects to a single global variable.

```
const weekDay = function() {
  const names = ["Sunday", "Monday", "Tuesday", "Wednesday",
    "Thursday", "Friday", "Saturday"];

  return {
    name(number) { return names[number]; },
    number(name) { return names.indexOf(name); }
  };
};
```

```
};
}();

console.log(weekDay.name(weekDay.number("Sunday")));
// → Sunday
```

This style of modules provides isolation, to a certain degree, but it does not declare dependencies. Instead, it just puts its interface into the global scope and expects its dependencies, if any, to do the same. This is not ideal.

If we implement our own module loader, we can do better. The most widely used approach to bolted-on JavaScript modules is called *CommonJS modules*. Node.js used this module system from the start (though it now also knows how to load ES modules), and it is the module system used by many packages on NPM.

A CommonJS module looks like a regular script, but it has access to two bindings that it uses to interact with other modules. The first is a function called `require`. When you call this with the module name of your dependency, it makes sure the module is loaded and returns its interface. The second is an object named `exports`, which is the interface object for the module. It starts out empty and you add properties to it to define exported values.

This CommonJS example module provides a date-formatting function. It uses two packages from NPM—`ordinal` to convert numbers to strings like “1st” and “2nd”, and `date-names` to get the English names for weekdays and months. It exports a single function, `formatDate`, which takes a `Date` object and a template string.

The template string may contain codes that direct the format, such as YYYY for the full year and Do for the ordinal day of the month. You could give it a string like "MMM Do YYYY" to get output like November 22nd 2017.

```
const ordinal = require("ordinal");
const {days, months} = require("date-names");

exports.formatDate = function(date, format) {
  return format.replace(/YYYY|M(MMM)?|Do?|ddd/g, tag => {
    if (tag == "YYYY") return date.getFullYear();
    if (tag == "M") return date.getMonth();
    if (tag == "MMM") return months[date.getMonth()];
    if (tag == "D") return date.getDate();
    if (tag == "Do") return ordinal(date.getDate());
    if (tag == "ddd") return days[date.getDay()];
  });
};
```

The interface of ordinal is a single function, whereas date-names exports an object containing multiple things—days and months are arrays of names. Destructuring is very convenient when creating bindings for imported interfaces.

The module adds its interface function to exports so that modules that depend on it get access to it. We could use the module like this:

```
const {formatDate} = require("../format-date.js");

console.log(formatDate(new Date(2017, 9, 13),
  "ddd the Do"));
```

```
// → Friday the 13th
```

CommonJS is implemented with a module loader that, when loading a module, wraps its code in a function (giving it its own local scope) and passes the require and exports bindings to that function as arguments.

If we assume we have access to a readFile function that reads a file by name and gives us its content, we can define a simplified form of require like this:

```
function require(name) {
  if (!(name in require.cache)) {
    let code = readFile(name);
    let exports = require.cache[name] = {};
    let wrapper = Function("require, exports", code);
    wrapper(require, exports);
  }
  return require.cache[name];
}
require.cache = Object.create(null);
```

Function is a built-in JavaScript function that takes a list of arguments (as a comma-separated string) and a string containing the function body and returns a function value with those arguments and that body. This is an interesting concept—it allows a program to create new pieces of program from string data—but also a dangerous one, since if someone can trick your program into putting a string they provide into Function, they can make the program do anything they want.

Standard JavaScript provides no such function as readFile, but different JavaScript environments, such as the browser and Node.js, pro-

vide their own ways of accessing files. The example just pretends that `readFile` exists.

To avoid loading the same module multiple times, `require` keeps a store (cache) of already loaded modules. When called, it first checks whether the requested module has been loaded and, if not, loads it. This involves reading the module's code, wrapping it in a function, and calling it.

By defining `require` and `exports` as parameters for the generated wrapper function (and passing the appropriate values when calling it), the loader makes sure that these bindings are available in the module's scope.

An important difference between this system and ES modules is that ES module imports happen before a module's script starts running, whereas `require` is a normal function, invoked when the module is already running. Unlike `import` declarations, `require` calls *can* appear inside functions, and the name of the dependency can be any expression that evaluates to a string, whereas `import` allows only plain quoted strings.

The transition of the JavaScript community from CommonJS style to ES modules has been a slow and somewhat rough one. Fortunately we are now at a point where most of the popular packages on NPM provide their code as ES modules, and Node.js allows ES modules to import from CommonJS modules. While CommonJS code is still something you will run across, there is no real reason to write new programs in this style anymore.

BUILDING AND BUNDLING

Many JavaScript packages aren't technically written in JavaScript. Language extensions such as TypeScript, the type checking dialect mentioned in Chapter 8, are widely used. People also often start using planned new language features long before they have been added to the platforms that actually run JavaScript. To make this possible, they *compile* their code, translating it from their chosen JavaScript dialect to plain old JavaScript—or even to a past version of JavaScript—so that browsers can run it.

Including a modular program that consists of 200 different files in a web page produces its own problems. If fetching a single file over the network takes 50 milliseconds, loading the whole program takes 10 seconds, or maybe half that if you can load several files simultaneously. That's a lot of wasted time. Because fetching a single big file tends to be faster than fetching a lot of tiny ones, web programmers have started using tools that combine their programs (which they painstakingly split into modules) into a single big file before they publish it to the web. Such tools are called *bundlers*.

And we can go further. Apart from the number of files, the *size* of the files also determines how fast they can be transferred over the network. Thus, the JavaScript community has invented *minifiers*. These are tools that take a JavaScript program and make it smaller by automatically removing comments and whitespace, renaming bindings, and replacing pieces of code with equivalent code that take up less space.

It is not uncommon for the code that you find in an NPM package or that runs on a web page to have gone through *multiple* stages of

transformation—converting from modern JavaScript to historic JavaScript—combining the modules into a single file, and minifying the code. We won't go into the details of these tools in this book, since there are many of them, and which one is popular changes regularly. Just be aware that such things exist, and look them up when you need them.

MODULE DESIGN

Structuring programs is one of the subtler aspects of programming. Any nontrivial piece of functionality can be organized in various ways.

Good program design is subjective—there are trade-offs involved, and matters of taste. The best way to learn the value of well-structured design is to read or work on a lot of programs and notice what works and what doesn't. Don't assume that a painful mess is “just the way it is”. You can improve the structure of almost everything by putting more thought into it.

One aspect of module design is ease of use. If you are designing something that is intended to be used by multiple people—or even by yourself, in three months when you no longer remember the specifics of what you did—it is helpful if your interface is simple and predictable.

That may mean following existing conventions. A good example is the `ini` package. This module imitates the standard `JSON` object by providing `parse` and `stringify` (to write an INI file) functions, and, like `JSON`, converts between strings and plain objects. The interface is small and familiar, and after you've worked with it once, you're likely to remember how to use it.

Even if there's no standard function or widely used package to imitate, you can keep your modules predictable by using simple data structures and doing a single, focused thing. Many of the INI-file parsing modules on NPM provide a function that directly reads such a file from the hard disk and parses it, for example. This makes it impossible to use such modules in the browser, where we don't have direct filesystem access, and adds complexity that would have been better addressed by *composing* the module with some file-reading function.

This points to another helpful aspect of module design—the ease with which something can be composed with other code. Focused modules that compute values are applicable in a wider range of programs than bigger modules that perform complicated actions with side effects. An INI file reader that insists on reading the file from disk is useless in a scenario where the file's content comes from some other source.

Relatedly, stateful objects are sometimes useful or even necessary, but if something can be done with a function, use a function. Several of the INI file readers on NPM provide an interface style that requires you to first create an object, then load the file into your object, and finally use specialized methods to get at the results. This type of thing is common in the object-oriented tradition, and it's terrible. Instead of making a single function call and moving on, you have to perform the ritual of moving your object through its various states. And because the data is now wrapped in a specialized object type, all code that interacts with it has to know about that type, creating unnecessary interdependencies.

Often, defining new data structures can't be avoided—only a few basic ones are provided by the language standard, and many types of

data have to be more complex than an array or a map. But when an array suffices, use an array.

An example of a slightly more complex data structure is the graph from Chapter 7. There is no single obvious way to represent a graph in JavaScript. In that chapter, we used an object whose properties hold arrays of strings—the other nodes reachable from that node.

There are several different pathfinding packages on NPM, but none of them uses this graph format. They usually allow the graph's edges to have a weight, which is the cost or distance associated with it. That isn't possible in our representation.

For example, there's the `dijkstrajs` package. A well-known approach to pathfinding, quite similar to our `findRoute` function, is called *Dijkstra's algorithm*, after Edsger Dijkstra, who first wrote it down. The `js` suffix is often added to package names to indicate the fact that they are written in JavaScript. This `dijkstrajs` package uses a graph format similar to ours, but instead of arrays, it uses objects whose property values are numbers—the weights of the edges.

If we wanted to use that package, we'd have to make sure that our graph was stored in the format it expects. All edges get the same weight, since our simplified model treats each road as having the same cost (one turn).

```
const {find_path} = require("dijkstrajs");

let graph = {};
for (let node of Object.keys(roadGraph)) {
  let edges = graph[node] = {};
  for (let dest of roadGraph[node]) {
```

```
    edges[dest] = 1;
  }
}

console.log(find_path(graph, "Post Office", "Cabin"));
// → ["Post Office", "Alice's House", "Cabin"]
```

This can be a barrier to composition—when various packages are using different data structures to describe similar things, combining them is difficult. Therefore, if you want to design for composability, find out what data structures other people are using and, when possible, follow their example.

Designing a fitting module structure for a program can be difficult. In the phase where you are still exploring the problem, trying different things to see what works, you might want to not worry about it too much, since keeping everything organized can be a big distraction. Once you have something that feels solid, that's a good time to take a step back and organize it.

SUMMARY

Modules provide structure to bigger programs by separating the code into pieces with clear interfaces and dependencies. The interface is the part of the module that's visible to other modules, and the dependencies are the other modules it makes use of.

Because JavaScript historically did not provide a module system, the CommonJS system was built on top of it. Then at some point it *did*

get a built-in system, which now coexists uneasily with the CommonJS system.

A package is a chunk of code that can be distributed on its own. NPM is a repository of JavaScript packages. You can download all kinds of useful (and useless) packages from it.

EXERCISES

A MODULAR ROBOT

These are the bindings that the project from Chapter 7 creates:

```
roads
buildGraph
roadGraph
VillageState
runRobot
randomPick
randomRobot
mailRoute
routeRobot
findRoute
goalOrientedRobot
```

If you were to write that project as a modular program, what modules would you create? Which module would depend on which other module, and what would their interfaces look like?

Which pieces are likely to be available prewritten on NPM? Would you prefer to use an NPM package or write them yourself?

ROADS MODULE

Write an ES module based on the example from Chapter 7 that contains the array of roads and exports the graph data structure representing them as `roadGraph`. It depends on a module `./graph.js` that exports a function `buildGraph`, used to build the graph. This function expects an array of two-element arrays (the start and end points of the roads).

CIRCULAR DEPENDENCIES

A circular dependency is a situation where module A depends on B, and B also, directly or indirectly, depends on A. Many module systems simply forbid this because whichever order you choose for loading such modules, you cannot make sure that each module's dependencies have been loaded before it runs.

CommonJS modules allow a limited form of cyclic dependencies. As long as the modules don't access each other's interface until after they finish loading, cyclic dependencies are okay.

The `require` function given earlier in this chapter supports this type of dependency cycle. Can you see how it handles cycles?

“Who can wait quietly while the mud settles?
Who can remain still until the moment of action?”
—Laozi, Tao Te Ching

CHAPTER II

ASYNCHRONOUS PROGRAMMING

The central part of a computer, the part that carries out the individual steps that make up our programs, is called the *processor*. The programs we have seen so far will keep the processor busy until they have finished their work. The speed at which something like a loop that manipulates numbers can be executed depends pretty much entirely on the speed of the computer’s processor and memory.

But many programs interact with things outside of the processor. For example, they may communicate over a computer network or request data from the hard disk—which is a lot slower than getting it from memory.

When such a thing is happening, it would be a shame to let the processor sit idle—there might be some other work it could do in the meantime. In part, this is handled by your operating system, which will switch the processor between multiple running programs. But that doesn’t help when we want a *single* program to be able to make progress while it is waiting for a network request.

ASYNCHRONICITY

In a *synchronous* programming model, things happen one at a time. When you call a function that performs a long-running action, it returns only when the action has finished and it can return the result. This stops your program for the time the action takes.

An *asynchronous* model allows multiple things to happen at the same time. When you start an action, your program continues to run. When the action finishes, the program is informed and gets access to the result (for example, the data read from disk).

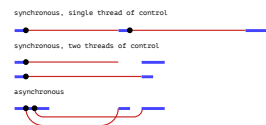
We can compare synchronous and asynchronous programming using a small example: a program that makes two requests over the network and then combines the results.

In a synchronous environment, where the request function returns only after it has done its work, the easiest way to perform this task is to make the requests one after the other. This has the drawback that the second request will be started only when the first has finished. The total time taken will be at least the sum of the two response times.

The solution to this problem, in a synchronous system, is to start additional threads of control. A *thread* is another running program whose execution may be interleaved with other programs by the operating system—since most modern computers contain multiple processors, multiple threads may even run at the same time, on different processors. A second thread could start the second request, and then both threads wait for their results to come back, after which they resynchronize to combine their results.

In the following diagram, the thick lines represent time the program

spends running normally, and the thin lines represent time spent waiting for the network. In the synchronous model, the time taken by the network is *part of* the timeline for a given thread of control. In the asynchronous model, starting a network action allows the program to continue running while the network communication happens alongside it, notifying the program when it is finished.



Another way to describe the difference is that waiting for actions to finish is *implicit* in the synchronous model, while it is *explicit*—under our control—in the asynchronous one.

Asynchronicity cuts both ways. It makes expressing programs that do not fit the straight-line model of control easier, but it can also make expressing programs that do follow a straight line more awkward. We'll see some ways to reduce this awkwardness later in the chapter.

Both prominent JavaScript programming platforms—browsers and Node.js—make operations that might take a while asynchronous, rather than relying on threads. Since programming with threads is notoriously hard (understanding what a program does is much more difficult when it's doing multiple things at once), this is generally considered a good

thing.

CALLBACKS

One approach to asynchronous programming is to make functions that need to wait for something take an extra argument, a *callback function*. The asynchronous function starts a process, sets things up so that the callback function is called when the process finishes, and then returns.

As an example, the `setTimeout` function, available both in Node.js and in browsers, waits a given number of milliseconds and then calls a function.

```
setTimeout(() => console.log("Tick"), 500);
```

Waiting is not generally important work, but it can be very useful when you need to arrange for something to happen at a certain time or check whether some action is taking longer than expected.

Another example of a common asynchronous operation is reading a file from a device's storage. Imagine you have a function `readTextFile` that reads a file's content as a string and passes it to a callback function.

```
readTextFile("shopping_list.txt", content => {  
  console.log(`Shopping List:\n${content}`);  
});  
// → Shopping List:  
// → Peanut butter  
// → Bananas
```

The `readTextFile` function is not part of standard JavaScript. We will see how to read files in the browser and in Node.js in later chapters.

Performing multiple asynchronous actions in a row using callbacks means that you have to keep passing new functions to handle the continuation of the computation after the actions. An asynchronous function that compares two files and produces a boolean indicating whether their content is the same might look like this:

```
function compareFiles(fileA, fileB, callback) {
  readTextFile(fileA, contentA => {
    readTextFile(fileB, contentB => {
      callback(contentA == contentB);
    });
  });
}
```

This style of programming is workable, but the indentation level increases with each asynchronous action because you end up in another function. Doing more complicated things, such as wrapping asynchronous actions in a loop, can get awkward.

In a way, asynchronicity is *contagious*. Any function that calls a function that works asynchronously must itself be asynchronous, using a callback or similar mechanism to deliver its result. Calling a callback is somewhat more involved and error prone than simply returning a value, so needing to structure large parts of your program that way is not great.

PROMISES

A slightly different way to build an asynchronous program is to have asynchronous functions return an object that represents its (future) result instead of passing around callback functions. This way, such functions actually return something meaningful, and the shape of the program more closely resembles that of synchronous programs.

This is what the standard class `Promise` is for. A *promise* is a receipt representing a value that may not be available yet. It provides a `then` method that allows you to register a function that should be called when the action for which it is waiting finishes. When the promise is *resolved*, meaning its value becomes available, such functions (there can be multiple) are called with the result value. It is possible to call then on a promise that has already resolved—your function will still be called.

The easiest way to create a promise is by calling `Promise.resolve`. This function ensures that the value you give it is wrapped in a promise. If it's already a promise, it is simply returned. Otherwise, you get a new promise that immediately resolves with your value as its result.

```
let fifteen = Promise.resolve(15);
fifteen.then(value => console.log(`Got ${value}`));
// → Got 15
```

To create a promise that does not immediately resolve, you can use `Promise` as a constructor. It has a somewhat odd interface: the constructor expects a function as its argument, which it immediately calls, passing it a function that it can use to resolve the promise.

For example, this is how you could create a promise-based interface for the `readTextFile` function:

```
function textFile(filename) {
  return new Promise(resolve => {
    readTextFile(filename, text => resolve(text));
  });
}
```

```
textFile("plans.txt").then(console.log);
```

Note how, in contrast to callback-style functions, this asynchronous function returns a meaningful value—a promise to give you the contents of the file at some point in the future.

A useful thing about the `then` method is that it itself returns another promise. This one resolves to the value returned by the callback function or, if that returned value is a promise, to the value that promise resolves to. Thus, you can “chain” multiple calls to `then` together to set up a sequence of asynchronous actions.

This function, which reads a file full of filenames and returns the content of a random file in that list, shows this kind of asynchronous promise pipeline:

```
function randomFile(listFile) {
  return textFile(listFile)
    .then(content => content.trim().split("\n"))
    .then(ls => ls[Math.floor(Math.random() * ls.length)])
    .then(filename => textFile(filename));
}
```

The function returns the result of this chain of `then` calls. The initial promise fetches the list of files as a string. The first `then` call transforms that string into an array of lines, producing a new promise. The second `then` call picks a random line from that, producing a third promise that yields a single filename. The final `then` call reads this file, so the result of the function as a whole is a promise that returns the content of a random file.

In this code, the functions used in the first two `then` calls return a regular value that will immediately be passed into the promise returned by `then` when the function returns. The last `then` call returns a promise (`textFile(filename)`), making it an actual asynchronous step.

It would also have been possible to perform all these steps inside a single `then` callback, since only the last step is actually asynchronous. But the kind of `then` wrappers that only do some synchronous data transformation are often useful, such as when you want to return a promise that produces a processed version of some asynchronous result.

```
function jsonFile(filename) {
  return textFile(filename).then(JSON.parse);
}
```

```
jsonFile("package.json").then(console.log);
```

Generally, it is useful to think of a promise as a device that lets code ignore the question of when a value is going to arrive. A normal value has to actually exist before we can reference it. A promised value is a value that *might* already be there or might appear at some point in the future. Computations defined in terms of promises, by wiring them

together with `then` calls, are executed asynchronously as their inputs become available.

FAILURE

Regular JavaScript computations can fail by throwing an exception. Asynchronous computations often need something like that. A network request may fail, a file may not exist, or some code that is part of the asynchronous computation may throw an exception.

One of the most pressing problems with the callback style of asynchronous programming is that it makes it extremely difficult to ensure failures are properly reported to the callbacks.

A common convention is to use the first argument to the callback to indicate that the action failed, and the second to pass the value produced by the action when it was successful.

```
someAsyncFunction(error, value) => {  
  if (error) handleError(error);  
  else processValue(value);  
});
```

Such callback functions must always check whether they received an exception and make sure that any problems they cause, including exceptions thrown by functions they call, are caught and given to the right function.

Promises make this easier. They can be either resolved (the action finished successfully) or rejected (it failed). Resolve handlers (as registered with `then`) are called only when the action is successful, and

rejections are propagated to the new promise returned by `then`. When a handler throws an exception, this automatically causes the promise produced by its `then` call to be rejected. If any element in a chain of asynchronous actions fails, the outcome of the whole chain is marked as rejected, and no success handlers are called beyond the point where it failed.

Much like resolving a promise provides a value, rejecting one also provides a value, usually called the *reason* of the rejection. When an exception in a handler function causes the rejection, the exception value is used as the reason. Similarly, when a handler returns a promise that is rejected, that rejection flows into the next promise. There's a `Promise.reject` function that creates a new, immediately rejected promise.

To explicitly handle such rejections, promises have a `catch` method that registers a handler to be called when the promise is rejected, similar to how `then` handlers handle normal resolution. It's also very much like `then` in that it returns a new promise, which resolves to the original promise's value when that resolves normally and to the result of the `catch` handler otherwise. If a `catch` handler throws an error, the new promise is also rejected.

As a shorthand, `then` also accepts a rejection handler as a second argument, so you can install both types of handlers in a single method call: `.then(acceptHandler, rejectHandler)`.

A function passed to the `Promise` constructor receives a second argument, alongside the resolve function, which it can use to reject the new promise.

When our `readTextFile` function encounters a problem, it passes the

error to its callback function as a second argument. Our `textFile` wrapper should actually check that argument so that a failure causes the promise it returns to be rejected.

```
function textFile(filename) {
  return new Promise((resolve, reject) => {
    readTextFile(filename, (text, error) => {
      if (error) reject(error);
      else resolve(text);
    });
  });
}
```

The chains of promise values created by calls to `then` and `catch` thus form a pipeline through which asynchronous values or failures move. Since such chains are created by registering handlers, each link has a success handler or a rejection handler (or both) associated with it. Handlers that don't match the type of outcome (success or failure) are ignored. Handlers that do match are called, and their outcome determines what kind of value comes next—success when they return a non-promise value, rejection when they throw an exception, and the outcome of the promise when they return a promise.

```
new Promise((_, reject) => reject(new Error("Fail")))
  .then(value => console.log("Handler 1:", value))
  .catch(reason => {
    console.log("Caught failure " + reason);
    return "nothing";
  })
  .then(value => console.log("Handler 2:", value));
```

```
// → Caught failure Error: Fail
// → Handler 2: nothing
```

The first `then` handler function isn't called because at that point of the pipeline the promise holds a rejection. The `catch` handler handles that rejection and returns a value, which is given to the second `then` handler function.

Much like an uncaught exception is handled by the environment, JavaScript environments can detect when a promise rejection isn't handled and will report this as an error.

CARLA

It's a sunny day in Berlin. The runway of the old, decommissioned airport is teeming with cyclists and inline skaters. In the grass near a garbage container, a flock of crows noisily mills about, trying to convince a group of tourists to part with their sandwiches.

One of the crows stands out—a large scruffy female with a few white feathers in her right wing. She is baiting people with a skill and confidence that suggest she's been doing this for a long time. When an elderly man is distracted by the antics of another crow, she casually swoops in, snatches his half-eaten bun from his hand, and sails away.

Contrary to the rest of the group, who look like they are happy to spend the day goofing around here, the large crow looks purposeful. Carrying her loot, she flies straight toward the roof of the hangar building, disappearing into an air vent.

Inside the building, you can hear an odd tapping sound—soft, but

persistent. It comes from a narrow space under the roof of an unfinished stairwell. The crow is sitting there, surrounded by her stolen snacks, half a dozen smartphones (several of which are turned on), and a mess of cables. She rapidly taps the screen of one of the phones with her beak. Words are appearing on it. If you didn't know better, you'd think she was typing.

This crow is known to her peers as “cāāw-krō”. But since those sounds are poorly suited for human vocal chords, we'll refer to her as Carla.

Carla is a somewhat peculiar crow. In her youth, she was fascinated by human language, eavesdropping on people until she had a good grasp of what they were saying. Later in life, her interest shifted to human technology, and she started stealing phones to study them. Her current project is learning to program. The text she is typing in her hidden lab is, in fact, a piece of asynchronous JavaScript code.

BREAKING IN

Carla loves the internet. Annoyingly, the phone she is working on is about to run out of prepaid data. The building has a wireless network, but it requires a code to access.

Fortunately, the wireless routers in the building are 20 years old and poorly secured. Doing some research, Carla finds out that the network authentication mechanism has a flaw she can use. When joining the network, a device must send along the correct six-digit passcode. The access point will reply with a success or failure message depending on whether the right code is provided. However, when sending a partial

code (say, only three digits), the response is different based on whether those digits are the correct start of the code or not. Sending incorrect numbers immediately returns a failure message. When sending the correct ones, the access point waits for more digits.

This makes it possible to greatly speed up the guessing of the number. Carla can find the first digit by trying each number in turn, until she finds one that doesn't immediately return failure. Having one digit, she can find the second digit in the same way, and so on, until she knows the entire passcode.

Assume Carla has a `joinWifi` function. Given the network name and the passcode (as a string), the function tries to join the network, returning a promise that resolves if successful and rejects if the authentication failed. The first thing she needs is a way to wrap a promise so that it automatically rejects after it takes too much time, to allow the program to quickly move on if the access point doesn't respond.

```
function withTimeout(promise, time) {
  return new Promise((resolve, reject) => {
    promise.then(resolve, reject);
    setTimeout(() => reject("Timed out"), time);
  });
}
```

This makes use of the fact that a promise can be resolved or rejected only once. If the promise given as its argument resolves or rejects first, that result will be the result of the promise returned by `withTimeout`. If, on the other hand, the `setTimeout` fires first, rejecting the promise, any further resolve or reject calls are ignored.

To find the whole passcode, the program needs to repeatedly look for the next digit by trying each digit. If authentication succeeds, we know we have found what we are looking for. If it immediately fails, we know that digit was wrong and must try the next digit. If the request times out, we have found another correct digit and must continue by adding another digit.

Because you cannot wait for a promise inside a `for` loop, Carla uses a recursive function to drive this process. On each call, this function gets the code as we know it so far, as well as the next digit to try. Depending on what happens, it may return a finished code or call through to itself, to either start cracking the next position in the code or to try again with another digit.

```
function crackPasscode(networkID) {
  function nextDigit(code, digit) {
    let newCode = code + digit;
    return withTimeout(joinWifi(networkID, newCode), 50)
      .then(() => newCode)
      .catch(failure => {
        if (failure == "Timed out") {
          return nextDigit(newCode, 0);
        } else if (digit < 9) {
          return nextDigit(code, digit + 1);
        } else {
          throw failure;
        }
      });
  }
  return nextDigit("", 0);
}
```

```
}
```

The access point tends to respond to bad authentication requests in about 20 milliseconds, so to be safe, this function waits for 50 milliseconds before timing out a request.

```
crackPasscode("HANGAR 2").then(console.log);
// → 555555
```

Carla tilts her head and sighs. This would have been more satisfying if the code had been a bit harder to guess.

ASYNC FUNCTIONS

Even with promises, this kind of asynchronous code is annoying to write. Promises often need to be tied together in verbose, arbitrary-looking ways. To create an asynchronous loop, Carla was forced to introduce a recursive function.

The thing the cracking function actually does is completely linear—it always waits for the previous action to complete before starting the next one. In a synchronous programming model, it'd be more straightforward to express.

The good news is that JavaScript allows you to write pseudosynchronous code to describe asynchronous computation. An `async` function implicitly returns a promise and can, in its body, `await` other promises in a way that *looks* synchronous.

We can rewrite `crackPasscode` like this:

```

async function crackPasscode(networkID) {
  for (let code = "";;) {
    for (let digit = 0;; digit++) {
      let newCode = code + digit;
      try {
        await withTimeout(joinWifi(networkID, newCode), 50);
        return newCode;
      } catch (failure) {
        if (failure == "Timed out") {
          code = newCode;
          break;
        } else if (digit == 9) {
          throw failure;
        }
      }
    }
  }
}

```

This version more clearly shows the double loop structure of the function (the inner loop tries digit 0 to 9 and the outer loop adds digits to the passcode).

An `async` function is marked by the word `async` before the function keyword. Methods can also be made `async` by writing `async` before their name. When such a function or method is called, it returns a promise. As soon as the function returns something, that promise is resolved. If the body throws an exception, the promise is rejected.

Inside an `async` function, the word `await` can be put in front of an expression to wait for a promise to resolve and only then continue the

execution of the function. If the promise rejects, an exception is raised at the point of the `await`.

Such a function no longer runs from start to completion in one go like a regular JavaScript function. Instead, it can be *frozen* at any point that has an `await` and can be resumed at a later time.

For most asynchronous code, this notation is more convenient than directly using promises. You do still need an understanding of promises, since in many cases you'll still interact with them directly. But when wiring them together, `async` functions are generally more pleasant to write than chains of `then` calls.

GENERATORS

This ability of functions to be paused and then resumed again is not exclusive to `async` functions. JavaScript also has a feature called *generator* functions. These are similar, but without the promises.

When you define a function with `function*` (placing an asterisk after the word `function`), it becomes a generator. When you call a generator, it returns an iterator, which we already saw in Chapter 6.

```

function* powers(n) {
  for (let current = n;; current *= n) {
    yield current;
  }
}

for (let power of powers(3)) {
  if (power > 50) break;
}

```

```

    console.log(power);
  }
  // → 3
  // → 9
  // → 27

```

Initially, when you call `powers`, the function is frozen at its start. Every time you call `next` on the iterator, the function runs until it hits a `yield` expression, which pauses it and causes the yielded value to become the next value produced by the iterator. When the function returns (the one in the example never does), the iterator is done.

Writing iterators is often much easier when you use generator functions. The iterator for the `Group` class (from the exercise in Chapter 6) can be written with this generator:

```

Group.prototype[Symbol.iterator] = function*() {
  for (let i = 0; i < this.members.length; i++) {
    yield this.members[i];
  }
};

```

There's no longer a need to create an object to hold the iteration state—generators automatically save their local state every time they yield.

Such `yield` expressions may occur only directly in the generator function itself and not in an inner function you define inside of it. The state a generator saves, when yielding, is only its *local* environment and the position where it yielded.

An `async` function is a special type of generator. It produces a promise when called, which is resolved when it returns (finishes) and rejected

when it throws an exception. Whenever it yields (awaits) a promise, the result of that promise (value or thrown exception) is the result of the `await` expression.

A CORVID ART PROJECT

One morning, Carla wakes up to unfamiliar noise from the tarmac outside of her hangar. Hopping onto the edge of the roof, she sees the humans are setting up for something. There's a lot of electric cabling, a stage, and some kind of big black wall being built up.

Being a curious crow, Carla takes a closer look at the wall. It appears to consist of a number of large glass-fronted devices wired up to cables. On the back, the devices say "LedTec SIG-5030".

A quick internet search turns up a user manual for these devices. They appear to be traffic signs, with a programmable matrix of amber LED lights. The intent of the humans is probably to display some kind of information on them during their event. Interestingly, the screens can be programmed over a wireless network. Could it be they are connected to the building's local network?

Each device on a network gets an *IP address*, which other devices can use to send it messages. We talk more about that in Chapter 13. Carla notices that her own phones all get addresses like `10.0.0.20` or `10.0.0.33`. It might be worth trying to send messages to all such addresses and see if any one of them responds to the interface described in the manual for the signs.

Chapter 18 shows how to make real requests on real networks. In

this chapter, we'll use a simplified dummy function called `request` for network communication. This function takes two arguments—a network address and a message, which may be anything that can be sent as JSON—and returns a promise that either resolves to a response from the machine at the given address, or rejects if there was a problem.

According to the manual, you can change what is displayed on a SIG-5030 sign by sending it a message with content like `{"command": "display", "data": [0, 0, 3, ...]}`, where `data` holds one number per LED dot, providing its brightness—0 means off, 3 means maximum brightness. Each sign is 50 lights wide and 30 lights high, so an update command should send 1,500 numbers.

This code sends a display update message to all addresses on the local network, to see what sticks. Each of the numbers in an IP address can go from 0 to 255. In the data it sends, it activates a number of lights corresponding to the network address's last number.

```
for (let addr = 1; addr < 256; addr++) {
  let data = [];
  for (let n = 0; n < 1500; n++) {
    data.push(n < addr ? 3 : 0);
  }
  let ip = `10.0.0.${addr}`;
  request(ip, {command: "display", data})
    .then(() => console.log(`Request to ${ip} accepted`))
    .catch(() => {});
}
```

Since most of these addresses won't exist or will not accept such messages, the `catch` call makes sure network errors don't crash the program.

The requests are all sent out immediately, without waiting for other requests to finish, in order to not waste time when some of the machines don't answer.

Having fired off her network scan, Carla heads back outside to see the result. To her delight, all of the screens are now showing a stripe of light in their upper-left corners. They *are* on the local network, and they *do* accept commands. She quickly notes the numbers shown on each screen. There are nine screens, arranged three high and three wide. They have the following network addresses:

```
const screenAddresses = [
  "10.0.0.44", "10.0.0.45", "10.0.0.41",
  "10.0.0.31", "10.0.0.40", "10.0.0.42",
  "10.0.0.48", "10.0.0.47", "10.0.0.46"
];
```

Now this opens up possibilities for all kinds of shenanigans. She could show “crows rule, humans drool” on the wall in giant letters. But that feels a bit crude. Instead, she plans to show a video of a flying crow covering all of the screens at night.

Carla finds a fitting video clip, in which a second and a half of footage can be repeated to create a looping video showing a crow's wingbeat. To fit the nine screens (each of which can show 50×30 pixels), Carla cuts and resizes the videos to get a series of 150×90 images, 10 per second. Those are then each cut into nine rectangles, and processed so that the dark spots on the video (where the crow is) show a bright light, and the light spots (no crow) are left dark, which should create the effect of an amber crow flying against a black background.

She has set up the `clipImages` variable to hold an array of frames, where each frame is represented with an array of nine sets of pixels—one for each screen—in the format that the signs expect.

To display a single frame of the video, Carla needs to send a request to all the screens at once. But she also needs to wait for the result of these requests, both in order to not start sending the next frame before the current one has been properly sent and in order to notice when requests are failing.

`Promise` has a static method `all` that can be used to convert an array of promises into a single promise that resolves to an array of results. This provides a convenient way to have some asynchronous actions happen alongside each other, wait for them all to finish, and then do something with their results (or at least wait for them to make sure they don't fail).

```
function displayFrame(frame) {
  return Promise.all(frame.map((data, i) => {
    return request(screenAddresses[i], {
      command: "display",
      data
    });
  }));
}
```

This maps over the images in `frame` (which is an array of display data arrays) to create an array of request promises. It then returns a promise that combines all of those.

In order to be able to stop a playing video, the process is wrapped

in a class. This class has an asynchronous `play` method that returns a promise that resolves only when the playback is stopped again via the `stop` method.

```
function wait(time) {
  return new Promise(accept => setTimeout(accept, time));
}

class VideoPlayer {
  constructor(frames, frameTime) {
    this.frames = frames;
    this.frameTime = frameTime;
    this.stopped = true;
  }

  async play() {
    this.stopped = false;
    for (let i = 0; !this.stopped; i++) {
      let nextFrame = wait(this.frameTime);
      await displayFrame(this.frames[i % this.frames.length]);
      await nextFrame;
    }
  }

  stop() {
    this.stopped = true;
  }
}
```

The `wait` function wraps `setTimeout` in a promise that resolves after the

given number of milliseconds. This is useful for controlling the speed of the playback.

```
let video = new VideoPlayer(clipImages, 100);
video.play().catch(e => {
  console.log("Playback failed: " + e);
});
setTimeout(() => video.stop(), 15000);
```

For the entire week that the screen wall stands, every evening, when it is dark, a huge glowing orange bird mysteriously appears on it.

THE EVENT LOOP

An asynchronous program starts by running its main script, which will often set up callbacks to be called later. That main script, as well as the callbacks, run to completion in one piece, uninterrupted. But between them, the program may sit idle, waiting for something to happen.

So callbacks are not directly called by the code that scheduled them. If I call `setTimeout` from within a function, that function will have returned by the time the callback function is called. And when the callback returns, control does not go back to the function that scheduled it.

Asynchronous behavior happens on its own empty function call stack. This is one of the reasons that, without promises, managing exceptions across asynchronous code is so hard. Since each callback starts with a mostly empty stack, your catch handlers won't be on the stack when they throw an exception.

```
try {
  setTimeout(() => {
    throw new Error("Woosh");
  }, 20);
} catch (e) {
  // This will not run
  console.log("Caught", e);
}
```

No matter how closely together events—such as timeouts or incoming requests—happen, a JavaScript environment will run only one program at a time. You can think of this as it running a big loop *around* your program, called the *event loop*. When there's nothing to be done, that loop is paused. But as events come in, they are added to a queue, and their code is executed one after the other. Because no two things run at the same time, slow-running code can delay the handling of other events.

This example sets a timeout but then dallies until after the timeout's intended point of time, causing the timeout to be late.

```
let start = Date.now();
setTimeout(() => {
  console.log("Timeout ran at", Date.now() - start);
}, 20);
while (Date.now() < start + 50) {}
console.log("Wasted time until", Date.now() - start);
// → Wasted time until 50
// → Timeout ran at 55
```

Promises always resolve or reject as a new event. Even if a promise is

already resolved, waiting for it will cause your callback to run after the current script finishes, rather than right away.

```
Promise.resolve("Done").then(console.log);
console.log("Me first!");
// → Me first!
// → Done
```

In later chapters we'll see various other types of events that run on the event loop.

ASYNCHRONOUS BUGS

When your program runs synchronously, in a single go, there are no state changes happening except those that the program itself makes. For asynchronous programs this is different—they may have *gaps* in their execution during which other code can run.

Let's look at an example. This is a function that tries to report the size of each file in an array of files, making sure to read them all at the same time rather than in sequence.

```
async function fileSizes(files) {
  let list = "";
  await Promise.all(files.map(async fileName => {
    list += fileName + ": " +
      (await textFile(fileName)).length + "\n";
  }));
  return list;
}
```

The `async fileName =>` part shows how arrow functions can also be made `async` by putting the word `async` in front of them.

The code doesn't immediately look suspicious... it maps the `async` arrow function over the array of names, creating an array of promises, and then uses `Promise.all` to wait for all of these before returning the list they build up.

But this program is entirely broken. It'll always return only a single line of output, listing the file that took the longest to read.

Can you work out why?

The problem lies in the `+=` operator, which takes the *current* value of `list` at the time the statement starts executing and then, when the `await` finishes, sets the `list` binding to be that value plus the added string.

But between the time the statement starts executing and the time it finishes, there's an asynchronous gap. The `map` expression runs before anything has been added to the list, so each of the `+=` operators starts from an empty string and ends up, when its storage retrieval finishes, setting `list` to the result of adding its line to the empty string.

This could have easily been avoided by returning the lines from the mapped promises and calling `join` on the result of `Promise.all`, instead of building up the list by changing a binding. As usual, computing new values is less error prone than changing existing values.

```
async function fileSizes(files) {
  let lines = files.map(async fileName => {
    return fileName + ": " +
      (await textFile(fileName)).length;
  });
  return lines.join("\n");
}
```

```
});  
return (await Promise.all(lines)).join("\n");  
}
```

Mistakes like this are easy to make, especially when using `await`, and you should be aware of where the gaps in your code occur. An advantage of JavaScript's *explicit* asynchronicity (whether through callbacks, promises, or `await`) is that spotting these gaps is relatively easy.

SUMMARY

Asynchronous programming makes it possible to express waiting for long-running actions without freezing the whole program. JavaScript environments typically implement this style of programming using callbacks, functions that are called when the actions complete. An event loop schedules such callbacks to be called when appropriate, one after the other, so that their execution does not overlap.

Programming asynchronously is made easier by promises, objects that represent actions that might complete in the future, and `async` functions, which allow you to write an asynchronous program as if it were synchronous.

EXERCISES

QUIET TIMES

There's a security camera near Carla's lab that's activated by a motion sensor. It is connected to the network and starts sending out a video stream when it is active. Because she'd rather not be discovered, Carla has set up a system that notices this kind of wireless network traffic and turns on a light in her lair whenever there is activity outside, so she knows when to keep quiet.

She's also been logging the times at which the camera is tripped for a while and wants to use this information to visualize which times, in an average week, tend to be quiet and which tend to be busy. The log is stored in files holding one time stamp number (as returned by `Date.now()`) per line.

```
1695709940692  
1695701068331  
1695701189163
```

The "camera_logs.txt" file holds a list of logfiles. Write an asynchronous function `activityTable(day)` that for a given day of the week returns an array of 24 numbers, one for each hour of the day, that hold the number of camera network traffic observations seen in that hour of the day. Days are identified by number using the system used by `Date.getDay`, where Sunday is 0 and Saturday is 6.

The `activityGraph` function, provided by the sandbox, summarizes such a table into a string.

To read the files, use the `textFile` function defined earlier—given a filename, it returns a promise that resolves to the file's content. Remember that `new Date(timestamp)` creates a `Date` object for that time, which has `getDay` and `getHours` methods returning the day of the week and the hour of the day.

Both types of files—the list of logfiles and the logfiles themselves—have each piece of data on its own line, separated by newline (`"\n"`) characters.

REAL PROMISES

Rewrite the function from the previous exercise without `async/await`, using plain `Promise` methods.

In this style, using `Promise.all` will be more convenient than trying to model a loop over the logfiles. In the `async` function, just using `await` in a loop is simpler. If reading a file takes some time, which of these two approaches will take the least time to run?

If one of the files listed in the file list has a typo, and reading it fails, how does that failure end up in the `Promise` object that your function returns?

BUILDING PROMISE.ALL

As we saw, given an array of promises, `Promise.all` returns a promise that waits for all of the promises in the array to finish. It then succeeds, yielding an array of result values. If a promise in the array fails, the promise returned by `all` fails too, passing on the failure reason from

the failing promise.

Implement something like this yourself as a regular function called `Promise.all`.

Remember that after a promise has succeeded or failed, it can't succeed or fail again, and further calls to the functions that resolve it are ignored. This can simplify the way you handle a failure of your promise.

“The evaluator, which determines the meaning of expressions in a programming language, is just another program.”

—Hal Abelson and Gerald Sussman, Structure and Interpretation of Computer Programs

CHAPTER 12

PROJECT: A PROGRAMMING LANGUAGE

Building your own programming language is surprisingly easy (as long as you do not aim too high) and very enlightening.

The main thing I want to show in this chapter is that there’s no magic involved in building a programming language. I’ve often felt that some human inventions were so immensely clever and complicated that I’d never be able to understand them. But with a little reading and experimenting, they often turn out to be quite mundane.

We will build a programming language called Egg. It will be a tiny, simple language—but one that is powerful enough to express any computation you can think of. It will allow simple abstraction based on functions.

PARSING

The most immediately visible part of a programming language is its *syntax*, or notation. A *parser* is a program that reads a piece of text and produces a data structure that reflects the structure of the program

contained in that text. If the text does not form a valid program, the parser should point out the error.

Our language will have a simple and uniform syntax. Everything in Egg is an expression. An expression can be the name of a binding, a number, a string, or an *application*. Applications are used for function calls but also for constructs such as `if` or `while`.

To keep the parser simple, strings in Egg do not support anything like backslash escapes. A string is simply a sequence of characters that are not double quotes, wrapped in double quotes. A number is a sequence of digits. Binding names can consist of any character that is not whitespace and that does not have a special meaning in the syntax.

Applications are written the way they are in JavaScript, by putting parentheses after an expression and having any number of arguments between those parentheses, separated by commas.

```
do(define(x, 10),
  if(>(x, 5),
    print("large"),
    print("small")))
```

The uniformity of the Egg language means that things that are operators in JavaScript (such as `>`) are normal bindings in this language, applied just like other functions. Since the syntax has no concept of a block, we need a `do` construct to represent doing multiple things in sequence.

The data structure that the parser will use to describe a program consists of expression objects, each of which has a `type` property indicating the kind of expression it is and other properties to describe its

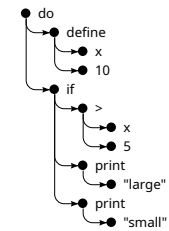
content.

Expressions of type "value" represent literal strings or numbers. Their `value` property contains the string or number value that they represent. Expressions of type "word" are used for identifiers (names). Such objects have a `name` property that holds the identifier's name as a string. Finally, "apply" expressions represent applications. They have an `operator` property that refers to the expression that is being applied, as well as an `args` property that holds an array of argument expressions.

The `>(x, 5)` part of the previous program would be represented like this:

```
{
  type: "apply",
  operator: {type: "word", name: ">"},
  args: [
    {type: "word", name: "x"},
    {type: "value", value: 5}
  ]
}
```

Such a data structure is called a *syntax tree*. If you imagine the objects as dots and the links between them as lines between those dots, as shown in the following diagram, the structure has a treelike shape. The fact that expressions contain other expressions, which in turn might contain more expressions, is similar to the way tree branches split and split again.



Contrast this to the parser we wrote for the configuration file format in Chapter 9, which had a simple structure: it split the input into lines and handled those lines one at a time. There were only a few simple forms that a line was allowed to have.

Here we must find a different approach. Expressions are not separated into lines, and they have a recursive structure. Application expressions *contain* other expressions.

Fortunately, this problem can be solved very well by writing a parser function that is recursive in a way that reflects the recursive nature of the language.

We define a function `parseExpression` that takes a string as input. It returns an object containing the data structure for the expression at the start of the string, along with the part of the string left after

parsing this expression. When parsing subexpressions (the argument to an application, for example), this function can be called again, yielding the argument expression as well as the text that remains. This text may in turn contain more arguments or may be the closing parenthesis that ends the list of arguments.

This is the first part of the parser:

```
function parseExpression(program) {
  program = skipSpace(program);
  let match, expr;
  if (match = /^"([^"]*)"/.exec(program)) {
    expr = {type: "value", value: match[1]};
  } else if (match = /^(\d+)/.exec(program)) {
    expr = {type: "value", value: Number(match[0])};
  } else if (match = /^[^\s()#]*/.exec(program)) {
    expr = {type: "word", name: match[0]};
  } else {
    throw new SyntaxError("Unexpected syntax: " + program);
  }
  return parseApply(expr, program.slice(match[0].length));
}

function skipSpace(string) {
  let first = string.search(/\S/);
  if (first == -1) return "";
  return string.slice(first);
}
```

Because Egg, like JavaScript, allows any amount of whitespace between

its elements, we have to repeatedly cut the whitespace off the start of the program string. The `skipSpace` function helps with this.

After skipping any leading space, `parseExpression` uses three regular expressions to spot the three atomic elements that Egg supports: strings, numbers, and words. The parser constructs a different kind of data structure depending on which expression matches. If the input does not match one of these three forms, it is not a valid expression, and the parser throws an error. We use the `SyntaxError` constructor here. This is an exception class defined by the standard, like `Error`, but more specific.

We then cut off the part that was matched from the program string and pass that, along with the object for the expression, to `parseApply`, which checks whether the expression is an application. If so, it parses a parenthesized list of arguments.

```
function parseApply(expr, program) {
  program = skipSpace(program);
  if (program[0] != "(") {
    return {expr: expr, rest: program};
  }

  program = skipSpace(program.slice(1));
  expr = {type: "apply", operator: expr, args: []};
  while (program[0] != ")") {
    let arg = parseExpression(program);
    expr.args.push(arg.expr);
    program = skipSpace(arg.rest);
    if (program[0] == ",") {
      program = skipSpace(program.slice(1));
    }
  }
}
```

```

    } else if (program[0] != ")") {
      throw new SyntaxError("Expected ',', ' ' or ')'");
    }
  }
  return parseApply(expr, program.slice(1));
}

```

If the next character in the program is not an opening parenthesis, this is not an application, and `parseApply` returns the expression it was given. Otherwise, it skips the opening parenthesis and creates the syntax tree object for this application expression. It then recursively calls `parseExpression` to parse each argument until a closing parenthesis is found. The recursion is indirect, through `parseApply` and `parseExpression` calling each other.

Because an application expression can itself be applied (such as in `multiplier(2)(1)`), `parseApply` must, after it has parsed an application, call itself again to check whether another pair of parentheses follows.

This is all we need to parse Egg. We wrap it in a convenient `parse` function that verifies that it has reached the end of the input string after parsing the expression (an Egg program is a single expression), and that gives us the program's data structure.

```

function parse(program) {
  let {expr, rest} = parseExpression(program);
  if (skipSpace(rest).length > 0) {
    throw new SyntaxError("Unexpected text after program");
  }
  return expr;
}

```

```

console.log(parse("+(a, 10)"));
// + {type: "apply",
//   operator: {type: "word", name: "+"},
//   args: [{type: "word", name: "a"},
//          {type: "value", value: 10}]}

```

It works! It doesn't give us very helpful information when it fails and doesn't store the line and column on which each expression starts, which might be helpful when reporting errors later, but it's good enough for our purposes.

THE EVALUATOR

What can we do with the syntax tree for a program? Run it, of course! And that is what the evaluator does. You give it a syntax tree and a scope object that associates names with values, and it will evaluate the expression that the tree represents and return the value that this produces.

```

const specialForms = Object.create(null);

function evaluate(expr, scope) {
  if (expr.type == "value") {
    return expr.value;
  } else if (expr.type == "word") {
    if (expr.name in scope) {
      return scope[expr.name];
    } else {

```



```

        throw new ReferenceError(
          `Undefined binding: ${expr.name}`);
      }
    } else if (expr.type == "apply") {
      let {operator, args} = expr;
      if (operator.type == "word" &&
          operator.name in specialForms) {
        return specialForms[operator.name](expr.args, scope);
      } else {
        let op = evaluate(operator, scope);
        if (typeof op == "function") {
          return op(...args.map(arg => evaluate(arg, scope)));
        } else {
          throw new TypeError("Applying a non-function.");
        }
      }
    }
  }
}

```

The evaluator has code for each of the expression types. A literal value expression produces its value. (For example, the expression `100` evaluates to the number 100.) For a binding, we must check whether it is actually defined in the scope and, if it is, fetch the binding's value.

Applications are more involved. If they are a special form, like `if`, we do not evaluate anything—we just pass the argument expressions, along with the scope, to the function that handles this form. If it is a normal call, we evaluate the operator, verify that it is a function, and call it with the evaluated arguments.

We use plain JavaScript function values to represent Egg's function

values. We will come back to this later, when the special form `fun` is defined.

The recursive structure of `evaluate` resembles the structure of the parser, and both mirror the structure of the language itself. It would also be possible to combine the parser and the evaluator into one function and evaluate during parsing, but splitting them up this way makes the program clearer and more flexible.

This is really all that's needed to interpret Egg. It's that simple. But without defining a few special forms and adding some useful values to the environment, you can't do much with this language yet.

SPECIAL FORMS

The `specialForms` object is used to define special syntax in Egg. It associates words with functions that evaluate such forms. It is currently empty. Let's add `if`.

```

specialForms.if = (args, scope) => {
  if (args.length != 3) {
    throw new SyntaxError("Wrong number of args to if");
  } else if (evaluate(args[0], scope) != false) {
    return evaluate(args[1], scope);
  } else {
    return evaluate(args[2], scope);
  }
};

```

Egg's `if` construct expects exactly three arguments. It will evaluate

the first, and if the result isn't the value `false`, it will evaluate the second. Otherwise, the third gets evaluated. This `if` form is more similar to JavaScript's ternary `?:` operator than to JavaScript's `if`. It is an expression, not a statement, and it produces a value—namely, the result of the second or third argument.

Egg also differs from JavaScript in how it handles the condition value to `if`. It will treat only the value `false` as false, not things like zero or the empty string.

The reason we need to represent `if` as a special form rather than a regular function is that all arguments to functions are evaluated before the function is called, whereas `if` should evaluate only *either* its second or its third argument, depending on the value of the first.

The `while` form is similar.

```
specialForms.while = (args, scope) => {
  if (args.length !== 2) {
    throw new SyntaxError("Wrong number of args to while");
  }
  while (evaluate(args[0], scope) !== false) {
    evaluate(args[1], scope);
  }

  // Since undefined does not exist in Egg, we return false,
  // for lack of a meaningful result
  return false;
};
```

Another basic building block is `do`, which executes all its arguments from top to bottom. Its value is the value produced by the last argument.

```
specialForms.do = (args, scope) => {
  let value = false;
  for (let arg of args) {
    value = evaluate(arg, scope);
  }
  return value;
};
```

To be able to create bindings and give them new values, we also create a form called `define`. It expects a word as its first argument and an expression producing the value to assign to that word as its second argument. Since `define`, like everything, is an expression, it must return a value. We'll make it return the value that was assigned (just like JavaScript's `=` operator).

```
specialForms.define = (args, scope) => {
  if (args.length !== 2 || args[0].type !== "word") {
    throw new SyntaxError("Incorrect use of define");
  }
  let value = evaluate(args[1], scope);
  scope[args[0].name] = value;
  return value;
};
```

THE ENVIRONMENT

The scope accepted by `evaluate` is an object with properties whose names correspond to binding names and whose values correspond to the

values those bindings are bound to. Let's define an object to represent the global scope.

To be able to use the `if` construct we just defined, we must have access to Boolean values. Since there are only two Boolean values, we do not need special syntax for them. We simply bind two names to the values `true` and `false` and use them.

```
const topScope = Object.create(null);

topScope.true = true;
topScope.false = false;
```

We can now evaluate a simple expression that negates a Boolean value.

```
let prog = parse(`if(true, false, true)`);
console.log(evaluate(prog, topScope));
// → false
```

To supply basic arithmetic and comparison operators, we will also add some function values to the scope. In the interest of keeping the code short, we'll use `Function` to synthesize a bunch of operator functions in a loop instead of defining them individually.

```
for (let op of ["+", "-", "*", "/", "=", "<", ">"]) {
  topScope[op] = Function("a, b", `return a ${op} b;`);
}
```

It is also useful to have a way to output values, so we'll wrap `console.log` in a function and call it `print`.

```
topScope.print = value => {
```

```
  console.log(value);
  return value;
};
```

That gives us enough elementary tools to write simple programs. The following function provides a convenient way to parse a program and run it in a fresh scope:

```
function run(program) {
  return evaluate(parse(program), Object.create(topScope));
}
```

We'll use object prototype chains to represent nested scopes so that the program can add bindings to its local scope without changing the top-level scope.

```
run(`
do(define(total, 0),
  define(count, 1),
  while(<(count, 11),
    do(define(total, +(total, count)),
      define(count, +(count, 1))))),
  print(total))
`);
// → 55
```

This is the program we've seen several times before that computes the sum of the numbers 1 to 10, expressed in Egg. It is clearly uglier than the equivalent JavaScript program—but not bad for a language implemented in fewer than 150 lines of code.

FUNCTIONS

A programming language without functions is a poor programming language indeed. Fortunately, it isn't hard to add a `fun` construct, which treats its last argument as the function's body and uses all arguments before that as the names of the function's parameters.

```
specialForms.fun = (args, scope) => {
  if (!args.length) {
    throw new SyntaxError("Functions need a body");
  }
  let body = args[args.length - 1];
  let params = args.slice(0, args.length - 1).map(expr => {
    if (expr.type !== "word") {
      throw new SyntaxError("Parameter names must be words");
    }
    return expr.name;
  });
  return function(...args) {
    if (args.length !== params.length) {
      throw new TypeError("Wrong number of arguments");
    }
    let localScope = Object.create(scope);
    for (let i = 0; i < args.length; i++) {
      localScope[params[i]] = args[i];
    }
    return evaluate(body, localScope);
  };
};
```

Functions in Egg get their own local scope. The function produced by the `fun` form creates this local scope and adds the argument bindings to it. It then evaluates the function body in this scope and returns the result.

```
run(`
do(define(plusOne, fun(a, +(a, 1))),
  print(plusOne(10)))
`);
// → 11

run(`
do(define(pow, fun(base, exp,
  if(==(exp, 0),
    1,
    *(base, pow(base, -(exp, 1)))))),
  print(pow(2, 10)))
`);
// → 1024
```

COMPILATION

What we have built is an interpreter. During evaluation, it acts directly on the representation of the program produced by the parser.

Compilation is the process of adding another step between the parsing and the running of a program, which transforms the program into something that can be evaluated more efficiently by doing as much work as possible in advance. For example, in well-designed languages it

is obvious, for each use of a binding, which binding is being referred to, without actually running the program. This can be used to avoid looking up the binding by name every time it is accessed, instead directly fetching it from some predetermined memory location.

Traditionally, compilation involves converting the program to machine code, the raw format that a computer's processor can execute. But any process that converts a program to a different representation can be thought of as compilation.

It would be possible to write an alternative evaluation strategy for Egg, one that first converts the program to a JavaScript program, uses `Function` to invoke the JavaScript compiler on it, and runs the result. When done right, this would make Egg run very fast while still being quite simple to implement.

If you are interested in this topic and willing to spend some time on it, I encourage you to try to implement such a compiler as an exercise.

CHEATING

When we defined `if` and `while`, you probably noticed that they were more or less trivial wrappers around JavaScript's own `if` and `while`. Similarly, the values in Egg are just regular old JavaScript values. Bridging the gap to a more primitive system, such as the machine code the processor understands, takes more effort—but the way it works resembles what we are doing here.

Though the toy language in this chapter doesn't do anything that couldn't be done better in JavaScript, there *are* situations where writing

small languages helps get real work done.

Such a language does not have to resemble a typical programming language. If JavaScript didn't come equipped with regular expressions, for example, you could write your own parser and evaluator for regular expressions.

Or imagine you are building a program that makes it possible to quickly create parsers by providing a logical description of the language they need to parse. You could define a specific notation for that, and a compiler that compiles it to a parser program.

```
expr = number | string | name | application

number = digit+

name = letter+

string = '"' (! '"' ) * '"'

application = expr '(' (expr ',' expr)* ')' '
```

This is what is usually called a *domain-specific language*, a language tailored to express a narrow domain of knowledge. Such a language can be more expressive than a general-purpose language because it is designed to describe exactly the things that need to be described in its domain and nothing else.

EXERCISES

ARRAYS

Add support for arrays to Egg by adding the following three functions to the top scope: `array(...values)` to construct an array containing the argument values, `length(array)` to get an array's length, and `element(array, n)` to fetch the n th element from an array.

CLOSURE

The way we have defined `fun` allows functions in Egg to reference the surrounding scope, allowing the function's body to use local values that were visible at the time the function was defined, just like JavaScript functions do.

The following program illustrates this: function `f` returns a function that adds its argument to `f`'s argument, meaning that it needs access to the local scope inside `f` to be able to use binding `a`.

```
run(`
do(define(f, fun(a, fun(b, +(a, b)))),
  print(f(4)(5)))
`);
// → 9
```

Go back to the definition of the `fun` form and explain which mechanism causes this to work.

COMMENTS

It would be nice if we could write comments in Egg. For example, whenever we find a hash sign (`#`), we could treat the rest of the line as a comment and ignore it, similar to `//` in JavaScript.

We do not have to make any big changes to the parser to support this. We can simply change `skipSpace` to skip comments as if they are whitespace so that all the points where `skipSpace` is called will now also skip comments. Make this change.

FIXING SCOPE

Currently, the only way to assign a binding a value is `define`. This construct acts as a way both to define new bindings and to give existing ones a new value.

This ambiguity causes a problem. When you try to give a nonlocal binding a new value, you will end up defining a local one with the same name instead. Some languages work like this by design, but I've always found it an awkward way to handle scope.

Add a special form `set`, similar to `define`, which gives a binding a new value, updating the binding in an outer scope if it doesn't already exist in the inner scope. If the binding is not defined at all, throw a `ReferenceError` (another standard error type).

The technique of representing scopes as simple objects, which has made things convenient so far, will get in your way a little at this point. You might want to use the `Object.getPrototypeOf` function, which returns the prototype of an object. Also remember that you can use

`Object.hasOwnProperty` to find out if a given object has a property.

“The dream behind the web is of a common information space in which we communicate by sharing information. Its universality is essential: the fact that a hypertext link can point to anything, be it personal, local or global, be it draft or highly polished.”

—Tim Berners-Lee, *The World Wide Web: A Very Short Personal History*

CHAPTER 13

JAVASCRIPT AND THE BROWSER

The next chapters of this book will discuss web browsers. Without browsers, there would be no JavaScript—or if there were, no one would ever have paid any attention to it.

Web technology has been decentralized from the start, not just technically but also in terms of the way it evolved. Various browser vendors have added new functionality in ad hoc and sometimes poorly thought-out ways, which were then—sometimes—adopted by others, and finally set down in standards.

This is both a blessing and a curse. On the one hand, it is empowering to not have a central party control a system but have it be improved by various parties working in loose collaboration (or occasionally, open hostility). On the other hand, the haphazard way in which the web was developed means that the resulting system is not exactly a shining example of internal consistency. Some parts of it are downright confusing and badly designed.

NETWORKS AND THE INTERNET

Computer networks have been around since the 1950s. If you put cables between two or more computers and allow them to send data back and forth through these cables, you can do all kinds of wonderful things.

If connecting two machines in the same building allows us to do wonderful things, connecting machines all over the planet should be even better. The technology to start implementing this vision was developed in the 1980s, and the resulting network is called the *internet*. It has lived up to its promise.

A computer can use this network to shoot bits at another computer. For any effective communication to arise out of this bit-shooting, the computers on both ends must know what the bits are supposed to represent. The meaning of any given sequence of bits depends entirely on the kind of thing that it is trying to express and on the encoding mechanism used.

A *network protocol* describes a style of communication over a network. There are protocols for sending email, for fetching email, for sharing files, and even for controlling computers that happen to be infected by malicious software.

The *HyperText Transfer Protocol* (HTTP) is a protocol for retrieving named resources (chunks of information, such as web pages or pictures). It specifies that the side making the request should start with a line like this, naming the resource and the version of the protocol that it is trying to use:

```
GET /index.html HTTP/1.1
```

There are many more rules about the way the requester can include more information in the request and the way the other side, which returns the resource, packages up its content. We'll look at HTTP in a little more detail in Chapter 18.

Most protocols are built on top of other protocols. HTTP treats the network as a streamlike device into which you can put bits and have them arrive at the correct destination in the correct order. Providing those guarantees on top of the primitive data-sending that the network gives you is already a rather tricky problem.

The *Transmission Control Protocol* (TCP) is a protocol that addresses this problem. All internet-connected devices “speak” it, and most communication on the internet is built on top of it.

A TCP connection works as follows: one computer must be waiting, or *listening*, for other computers to start talking to it. To be able to listen for different kinds of communication at the same time on a single machine, each listener has a number (called a *port*) associated with it. Most protocols specify which port should be used by default. For example, when we want to send an email using the SMTP protocol, the machine through which we send it is expected to be listening on port 25.

Another computer can then establish a connection by connecting to the target machine using the correct port number. If the target machine can be reached and is listening on that port, the connection is successfully created. The listening computer is called the *server*, and the connecting computer is called the *client*.

Such a connection acts as a two-way pipe through which bits can

flow—the machines on both ends can put data into it. Once the bits are successfully transmitted, they can be read out again by the machine on the other side. This is a convenient model. You could say that TCP provides an abstraction of the network.

THE WEB

The *World Wide Web* (not to be confused with the internet as a whole) is a set of protocols and formats that allow us to visit web pages in a browser. The word *Web* refers to the fact that such pages can easily link to each other, thus connecting into a huge mesh that users can move through.

To become part of the web, all you need to do is connect a machine to the internet and have it listen on port 80 with the HTTP protocol so that other computers can ask it for documents.

Each document on the web is named by a *uniform resource locator* (URL), which looks something like this:

```
http://eloquentjavascript.net/13_browser.html
|      |              |           |           |
protocol  server      path
```

The first part tells us that this URL uses the HTTP protocol (as opposed to, for example, encrypted HTTP, which would be *https://*). Then comes the part that identifies which server we are requesting the document from. Last is a path string that identifies the document (or *resource*) we are interested in.

Machines connected to the internet get an *IP address*, a number that can be used to send messages to that machine, and looks something like 149.210.142.219 or 2001:4860:4860::8888. Since lists of more or less random numbers are hard to remember and awkward to type, you can instead register a *domain name* for an address or set of addresses. I registered *eloquentjavascript.net* to point at the IP address of a machine I control and can thus use that domain name to serve web pages.

If you type this URL into your browser's address bar, the browser will try to retrieve and display the document at that URL. First, your browser has to find out what address *eloquentjavascript.net* refers to. Then, using the HTTP protocol, it will make a connection to the server at that address and ask for the resource */13_browser.html*. If all goes well, the server sends back a document, which your browser then displays on your screen.

HTML

HTML, which stands for HyperText Markup Language, is the document format used for web pages. An HTML document contains text, as well as *tags* that give structure to the text, describing things such as links, paragraphs, and headings.

A short HTML document might look like this:

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
```

```

<title>My home page</title>
</head>
<body>
<h1>My home page</h1>
<p>Hello, I am Marijn and this is my home page.</p>
<p>I also wrote a book! Read it
  <a href="http://eloquentjavascript.net">here</a>.</p>
</body>
</html>

```

This is what such a document would look like in the browser:

My home page

Hello, I am Marijn and this is my home page.

I also wrote a book! Read it [here](#).

The tags, wrapped in angle brackets (< and >, the symbols for *less than* and *greater than*), provide information about the structure of the document. The other text is just plain text.

The document starts with `<!doctype html>`, which tells the browser to interpret the page as *modern* HTML, as opposed to obsolete styles used in the past.

HTML documents have a head and a body. The head contains information *about* the document, and the body contains the document itself. In this case, the head declares that the title of this document is “My home page” and that it uses the UTF-8 encoding, which is a way to encode Unicode text as binary data. The document’s body

contains a heading (<h1>, meaning “heading 1”—<h2> to <h6> produce subheadings) and two paragraphs (<p>).

Tags come in several forms. An element, such as the body, a paragraph, or a link, is started by an *opening tag* like <p> and ended by a *closing tag* like </p>. Some opening tags, such as the one for the link (<a>), contain extra information in the form of *name*=*value* pairs. These are called *attributes*. In this case, the destination of the link is indicated with `href="http://eloquentjavascript.net"`, where `href` stands for “hypertext reference”.

Some kinds of tags do not enclose anything and thus do not need to be closed. The metadata tag `<meta charset="utf-8">` is an example of this.

To be able to include angle brackets in the text of a document even though they have a special meaning in HTML, yet another form of special notation has to be introduced. A plain opening angle bracket is written as `<`; (“less than”), and a closing bracket is written as `>`; (“greater than”). In HTML, an ampersand (&) character followed by a name or character code and a semicolon (;) is called an *entity* and will be replaced by the character it encodes.

This is analogous to the way backslashes are used in JavaScript strings. Since this mechanism gives ampersand characters a special meaning too, they need to be escaped as `&`. Inside attribute values, which are wrapped in double quotes, `"`; can be used to insert a literal quote character.

HTML is parsed in a remarkably error-tolerant way. When tags that should be there are missing, the browser automatically adds them. The

way this is done has been standardized, and you can rely on all modern browsers to do it in the same way.

The following document will be treated just like the one shown previously:

```
<!doctype html>

<meta charset=utf-8>
<title>My home page</title>

<h1>My home page</h1>
<p>Hello, I am Marijn and this is my home page.
<p>I also wrote a book! Read it
  <a href=http://eloquentjavascript.net>here</a>.
```

The `<html>`, `<head>`, and `<body>` tags are completely gone. The browser knows that `<meta>` and `<title>` belong in the head and that `<h1>` means the body has started. Furthermore, I am no longer explicitly closing the paragraphs, since opening a new paragraph or ending the document will close them implicitly. The quotes around the attribute values are also gone.

This book will usually omit the `<html>`, `<head>`, and `<body>` tags from examples to keep them short and free of clutter. I *will* close tags and include quotes around attributes, though.

I will also usually omit the `doctype` and `charset` declaration. Don't take this as encouragement to drop these from HTML documents. Browsers will often do ridiculous things when you forget them. Consider the `doctype` and the `charset` metadata to be implicitly present in

examples, even when they are not actually shown in the text.

HTML AND JAVASCRIPT

In the context of this book, the most important HTML tag is `<script>`, which allows us to include a piece of JavaScript in a document.

```
<h1>Testing alert</h1>
<script>alert("hello!");</script>
```

Such a script will run as soon as its `<script>` tag is encountered while the browser reads the HTML. This page will pop up a dialog when opened—the `alert` function resembles `prompt`, in that it pops up a little window, but only shows a message without asking for input.

Including large programs directly in HTML documents is often impractical. The `<script>` tag can be given an `src` attribute to fetch a script file (a text file containing a JavaScript program) from a URL.

```
<h1>Testing alert</h1>
<script src="code/hello.js"></script>
```

The `code/hello.js` file included here contains the same program—`alert("hello!")`. When an HTML page references other URLs as part of itself, such as an image file or a script, web browsers will retrieve them immediately and include them in the page.

A script tag must always be closed with `</script>`, even if it refers to a script file and doesn't contain any code. If you forget this, the rest of the page will be interpreted as part of the script.

You can load ES modules (see Chapter 10) in the browser by giving your script tag a `type="module"` attribute. Such modules can depend on other modules by using URLs relative to themselves as module names in `import` declarations.

Some attributes can also contain a JavaScript program. The `<button>` tag (which shows up as a button) supports an `onclick` attribute. The attribute's value will be run whenever the button is clicked.

```
<button onclick="alert('Boom!');">DO NOT PRESS</button>
```

Note that I had to use single quotes for the string in the `onclick` attribute because double quotes are already used to quote the whole attribute. I could also have used `"`; to escape the inner quotes.

IN THE SANDBOX

Running programs downloaded from the internet is potentially dangerous. You don't know much about the people behind most sites you visit, and they do not necessarily mean well. Running programs by malicious actors is how you get your computer infected by viruses, your data stolen, and your accounts hacked.

Yet the attraction of the web is that you can browse it without necessarily trusting all the pages you visit. This is why browsers severely limit the things a JavaScript program may do: it can't look at the files on your computer or modify anything not related to the web page it was embedded in.

Isolating a programming environment in this way is called *sandboxing*,

the idea being that the program is harmlessly playing in a sandbox. But you should imagine this particular kind of sandbox as having a cage of thick steel bars over it so that the programs playing in it can't actually get out.

The hard part of sandboxing is allowing programs enough room to be useful while restricting them from doing anything dangerous. Lots of useful functionality, such as communicating with other servers or reading the content of the copy-paste clipboard, can also be used for problematic, privacy-invading purposes.

Every now and then, someone comes up with a new way to circumvent the limitations of a browser and do something harmful, ranging from leaking minor private information to taking over the whole machine on which the browser is running. The browser developers respond by fixing the hole, and all is well again—until the next problem is discovered, and hopefully publicized rather than secretly exploited by some government agency or criminal organization.

COMPATIBILITY AND THE BROWSER WARS

In the early stages of the web, a browser called Mosaic dominated the market. After a few years, the balance shifted to Netscape, which was, in turn, largely supplanted by Microsoft's Internet Explorer. At any point where a single browser was dominant, that browser's vendor would feel entitled to unilaterally invent new features for the web. Since most users used the most popular browser, websites would simply start using those features—never mind the other browsers.

This was the dark age of compatibility, often called the *browser wars*. Web developers were left with not one unified web but two or three incompatible platforms. To make things worse, the browsers in use around 2003 were all full of bugs, and of course the bugs were different for each browser. Life was hard for people writing web pages.

Mozilla Firefox, a not-for-profit offshoot of Netscape, challenged Internet Explorer's position in the late 2000s. Because Microsoft was not particularly interested in staying competitive at the time, Firefox took a lot of market share away from it. Around the same time, Google introduced its Chrome browser and Apple's Safari browser gained popularity, leading to a situation where there were four major players, rather than one.

The new players had a more serious attitude toward standards and better engineering practices, giving us less incompatibility and fewer bugs. Microsoft, seeing its market share crumble, came around and adopted these attitudes in its Edge browser, which replaced Internet Explorer. If you are starting to learn web development today, consider yourself lucky. The latest versions of the major browsers behave quite uniformly and have relatively few bugs.

Unfortunately, with Firefox's market share getting ever smaller, and Edge becoming just a wrapper around Chrome's core in 2018, this uniformity might once again take the form of a single vendor—Google, this time—having enough control over the browser market to push its idea of what the web should look like onto the rest of the world.

For what it is worth, this long chain of historical events and accidents has produced the web platform that we have today. In the next

chapters, we are going to write programs for it.

“Too bad! Same old story! Once you’ve finished building your house you notice you’ve accidentally learned something that you really should have known—before you started.”

—Friedrich Nietzsche, Beyond Good and Evil

CHAPTER 14

THE DOCUMENT OBJECT MODEL

When you open a web page, your browser retrieves the page’s HTML text and parses it, much like our parser from Chapter 12 parsed programs. The browser builds up a model of the document’s structure and uses this model to draw the page on the screen.

This representation of the document is one of the toys that a JavaScript program has available in its sandbox. It is a data structure that you can read or modify. It acts as a *live* data structure: when it’s modified, the page on the screen is updated to reflect the changes.

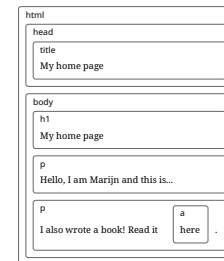
DOCUMENT STRUCTURE

You can imagine an HTML document as a nested set of boxes. Tags such as `<body>` and `</body>` enclose other tags, which in turn contain other tags or text. Here’s the example document from the previous chapter:

```
<!doctype html>
<html>
```

```
<head>
  <title>My home page</title>
</head>
<body>
  <h1>My home page</h1>
  <p>Hello, I am Marijn and this is my home page.</p>
  <p>I also wrote a book! Read it
    <a href="http://eloquentjavascript.net">here</a>.</p>
</body>
</html>
```

This page has the following structure:



The data structure the browser uses to represent the document follows this shape. For each box, there is an object, which we can interact with to find out things such as what HTML tag it represents and which boxes and text it contains. This representation is called the *Document Object Model*, or *DOM* for short.

The global binding document gives us access to these objects. Its `documentElement` property refers to the object representing the `<html>` tag. Since every HTML document has a head and a body, it also has `head` and `body` properties pointing at those elements.

TREES

Think back to the syntax trees from Chapter 12 for a moment. Their structures are strikingly similar to the structure of a browser's document. Each *node* may refer to other nodes, *children*, which in turn may have their own children. This shape is typical of nested structures, where elements can contain subelements that are similar to themselves.

We call a data structure a *tree* when it has a branching structure, no cycles (a node may not contain itself, directly or indirectly), and a single, well-defined *root*. In the case of the DOM, `document.documentElement` serves as the root.

Trees come up a lot in computer science. In addition to representing recursive structures such as HTML documents or programs, they are often used to maintain sorted sets of data because elements can usually be found or inserted more efficiently in a tree than in a flat array.

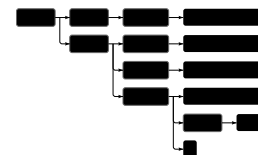
A typical tree has different kinds of nodes. The syntax tree for *the*

Egg language had identifiers, values, and application nodes. Application nodes may have children, whereas identifiers and values are *leaves*, or nodes without children.

The same goes for the DOM. Nodes for *elements*, which represent HTML tags, determine the structure of the document. These can have child nodes. An example of such a node is `document.body`. Some of these children can be leaf nodes, such as pieces of text or comment nodes.

Each DOM node object has a `nodetype` property, which contains a code (number) that identifies the type of node. Elements have code 1, which is also defined as the constant property `Node.ELEMENT_NODE`. Text nodes, representing a section of text in the document, get code 3 (`Node.TEXT_NODE`). Comments have code 8 (`Node.COMMENT_NODE`).

Another way to visualize our document tree is as follows:



The leaves are text nodes, and the arrows indicate parent-child relationships between nodes.

THE STANDARD

Using cryptic numeric codes to represent node types is not a very JavaScript-like thing to do. Later in this chapter, we'll see that other parts of the DOM interface also feel cumbersome and alien. This is because the DOM interface wasn't designed for JavaScript alone. Rather, it tries to be a language-neutral interface that can be used in other systems as well—not just for HTML but also for XML, which is a generic data format with an HTML-like syntax.

This is unfortunate. Standards are often useful. But in this case, the advantage (cross-language consistency) isn't all that compelling. Having an interface that is properly integrated with the language you're using will save you more time than having a familiar interface across languages.

As an example of this poor integration, consider the `childNodes` property that element nodes in the DOM have. This property holds an array-like object with a `length` property and properties labeled by numbers to access the child nodes. But it is an instance of the `NodeList` type, not a real array, so it does not have methods such as `slice` and `map`.

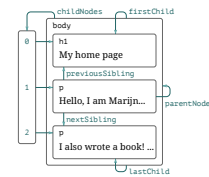
Then there are issues that are simply caused by poor design. For example, there is no way to create a new node and immediately add children or attributes to it. Instead, you have to first create it and then add the children and attributes one by one, using side effects. Code that interacts heavily with the DOM tends to get long, repetitive, and ugly.

But these flaws aren't fatal. Since JavaScript allows us to create

our own abstractions, it is possible to design improved ways to express the operations we are performing. Many libraries intended for browser programming come with such tools.

MOVING THROUGH THE TREE

DOM nodes contain a wealth of links to other nearby nodes. The following diagram illustrates these:



Although the diagram shows only one link of each type, every node has a `parentNode` property that points to the node it is part of, if any. Likewise, every element node (node type 1) has a `childNodes` property that points to an array-like object holding its children.

In theory, you could move anywhere in the tree using just these parent and child links. But JavaScript also gives you access to a number of additional convenience links. The `firstChild` and `lastChild` prop-

erties point to the first and last child elements or have the value `null` for nodes without children. Similarly, `previousSibling` and `nextSibling` point to adjacent nodes, which are nodes with the same parent that appear immediately before or after the node itself. For a first child, `previousSibling` will be `null`, and for a last child, `nextSibling` will be `null`.

There's also the `children` property, which is like `childNodes` but contains only element (type 1) children, not other types of child nodes. This can be useful when you aren't interested in text nodes.

When dealing with a nested data structure like this one, recursive functions are often useful. The following function scans a document for text nodes containing a given string and returns `true` when it has found one:

```
function talksAbout(node, string) {
  if (node.nodeType == Node.ELEMENT_NODE) {
    for (let child of node.childNodes) {
      if (talksAbout(child, string)) {
        return true;
      }
    }
    return false;
  } else if (node.nodeType == Node.TEXT_NODE) {
    return node.nodeValue.indexOf(string) > -1;
  }
}

console.log(talksAbout(document.body, "book"));
// → true
```

The `nodeValue` property of a text node holds the string of text that it represents.

FINDING ELEMENTS

Navigating these links among parents, children, and siblings is often useful. But if we want to find a specific node in the document, reaching it by starting at `document.body` and following a fixed path of properties is a bad idea. Doing so bakes assumptions into our program about the precise structure of the document—a structure you might want to change later. Another complicating factor is that text nodes are created even for the whitespace between nodes. The example document's `<body>` tag has not just three children (`<h1>` and two `<p>` elements), but seven: those three, plus the spaces before, after, and between them.

If we want to get the `href` attribute of the link in that document, we don't want to say something like “Get the second child of the sixth child of the document body”. It'd be better if we could say “Get the first link in the document”. And we can.

```
let link = document.body.getElementsByTagName("a")[0];
console.log(link.href);
```

All element nodes have a `getElementsByTagName` method, which collects all elements with the given tag name that are descendants (direct or indirect children) of that node and returns them as an array-like object.

To find a specific *single* node, you can give it an `id` attribute and use `document.getElementById` instead.

```
<p>My ostrich Gertrude:</p>
<p></p>

<script>
  let ostrich = document.getElementById("gertrude");
  console.log(ostrich.src);
</script>
```

A third, similar method is `getElementsByClassName`, which, like `getElements`, searches through the contents of an element node and retrieves all elements that have the given string in their `class` attribute.

CHANGING THE DOCUMENT

Almost everything about the DOM data structure can be changed. The shape of the document tree can be modified by changing parent-child relationships. Nodes have a `remove` method to remove them from their current parent node. To add a child node to an element node, we can use `appendChild`, which puts it at the end of the list of children, or `insertBefore`, which inserts the node given as the first argument before the node given as the second argument.

```
<p>One</p>
<p>Two</p>
<p>Three</p>

<script>
  let paragraphs = document.body.getElementsByTagName("p");
  document.body.insertBefore(paragraphs[2], paragraphs[0]);
```

```
</script>
```

A node can exist in the document in only one place. Thus, inserting paragraph *Three* in front of paragraph *One* will first remove it from the end of the document and then insert it at the front, resulting in *Three/One/Two*. All operations that insert a node somewhere will, as a side effect, cause it to be removed from its current position (if it has one).

The `replaceChild` method is used to replace a child node with another one. It takes as arguments two nodes: a new node and the node to be replaced. The replaced node must be a child of the element the method is called on. Note that both `replaceChild` and `insertBefore` expect the *new* node as their first argument.

CREATING NODES

Say we want to write a script that replaces all images (`<img>` tags) in the document with the text held in their `alt` attributes, which specifies an alternative textual representation of the image. This involves not only removing the images but also adding a new text node to replace them.

```
<p>The  in the
.</p>

<p><button onclick="replaceImages()">Replace</button></p>

<script>
```

```
function replaceImages() {
  let images = document.body.getElementsByTagName("img");
  for (let i = images.length - 1; i >= 0; i--) {
    let image = images[i];
    if (image.alt) {
      let text = document.createTextNode(image.alt);
      image.parentNode.replaceChild(text, image);
    }
  }
}
</script>
```

Given a string, `createTextNode` gives us a text node that we can insert into the document to make it show up on the screen.

The loop that goes over the images starts at the end of the list. This is necessary because the node list returned by a method like `getElementsByTagName` (or a property like `childNodes`) is *live*. That is, it is updated as the document changes. If we started from the front, removing the first image would cause the list to lose its first element so that the second time the loop repeats, where `i` is 1, it would stop because the length of the collection is now also 1.

If you want a *solid* collection of nodes, as opposed to a live one, you can convert the collection to a real array by calling `Array.from`.

```
let arrayish = {0: "one", 1: "two", length: 2};
let array = Array.from(arrayish);
console.log(array.map(s => s.toUpperCase()));
// → ["ONE", "TWO"]
```

To create element nodes, you can use the `document.createElement` method.

This method takes a tag name and returns a new empty node of the given type.

The following example defines a utility `elt`, which creates an element node and treats the rest of its arguments as children to that node. This function is then used to add an attribution to a quote.

```
<blockquote id="quote">
  No book can ever be finished. While working on it we learn
  just enough to find it immature the moment we turn away
  from it.
</blockquote>

<script>
function elt(type, ...children) {
  let node = document.createElement(type);
  for (let child of children) {
    if (typeof child !== "string") node.appendChild(child);
    else node.appendChild(document.createTextNode(child));
  }
  return node;
}

document.getElementById("quote").appendChild(
  elt("footer", "-",
    elt("strong", "Karl Popper"),
    ", preface to the second edition of ",
    elt("em", "The Open Society and Its Enemies"),
    ", 1950"));
</script>
```

This is what the resulting document looks like:

No book can ever be finished. While working on it we learn
just enough to find it immature the moment we turn away
from it.
—Karl Popper, preface to the second edition of *The Open
Society and Its Enemies*, 1950

ATTRIBUTES

Some element attributes, such as `href` for links, can be accessed through a property of the same name on the element’s DOM object. This is the case for most commonly used standard attributes.

HTML allows you to set any attribute you want on nodes. This can be useful because it allows you to store extra information in a document. To read or change custom attributes, which aren’t available as regular object properties, you have to use the `getAttribute` and `setAttribute` methods.

```
<p data-classified="secret">The launch code is 00000000.</p>
<p data-classified="unclassified">I have two feet.</p>

<script>
  let paras = document.body.getElementsByTagName("p");
  for (let para of Array.from(paras)) {
    if (para.getAttribute("data-classified") == "secret") {
      para.remove();
    }
  }
}
```

</script>

It is recommended to prefix the names of such made-up attributes with `data-` to ensure they do not conflict with any other attributes.

There is a commonly used attribute, `class`, which is a keyword in the JavaScript language. For historical reasons—some old JavaScript implementations could not handle property names that matched keywords—the property used to access this attribute is called `className`. You can also access it under its real name, `"class"`, with the `getAttribute` and `setAttribute` methods.

LAYOUT

You may have noticed that different types of elements are laid out differently. Some, such as paragraphs (`<p>`) or headings (`<h1>`), take up the whole width of the document and are rendered on separate lines. These are called *block* elements. Others, such as links (`<a>`) or the `<strong>` element, are rendered on the same line with their surrounding text. Such elements are called *inline* elements.

For any given document, browsers are able to compute a layout, which gives each element a size and position based on its type and content. This layout is then used to actually draw the document.

The size and position of an element can be accessed from JavaScript. The `offsetWidth` and `offsetHeight` properties give you the space the element takes up in *pixels*. A pixel is the basic unit of measurement in the browser. It traditionally corresponds to the smallest dot that the screen can draw, but on modern displays, which can draw *very* small

dots, that may no longer be the case, and a browser pixel may span multiple display dots.

Similarly, `clientWidth` and `clientHeight` give you the size of the space *inside* the element, ignoring border width.

```
<p style="border: 3px solid red">
  I'm boxed in
</p>

<script>
  let para = document.body.getElementsByTagName("p")[0];
  console.log("clientHeight:", para.clientHeight);
  // → 19
  console.log("offsetHeight:", para.offsetHeight);
  // → 25
</script>
```

Giving a paragraph a border causes a rectangle to be drawn around it.

I'm boxed in

The most effective way to find the precise position of an element on the screen is the `getBoundingClientRect` method. It returns an object with `top`, `bottom`, `left`, and `right` properties, indicating the pixel positions of the sides of the element relative to the upper left of the screen. If you want pixel positions relative to the whole document, you must add the current scroll position, which you can find in the `pageXOffset` and `pageYOffset` bindings.

Laying out a document can be quite a lot of work. In the interest

of speed, browser engines do not immediately re-layout a document every time you change it but wait as long as they can before doing so. When a JavaScript program that changed the document finishes running, the browser will have to compute a new layout to draw the changed document to the screen. When a program *asks* for the position or size of something by reading properties such as `offsetHeight` or calling `getBoundingClientRect`, providing that information also requires computing a layout.

A program that repeatedly alternates between reading DOM layout information and changing the DOM forces a lot of layout computations to happen and will consequently run very slowly. The following code is an example of this. It contains two different programs that build up a line of *X* characters 2,000 pixels wide and measures the time each one takes.

```
<p><span id="one"></span></p>
<p><span id="two"></span></p>

<script>
  function time(name, action) {
    let start = Date.now(); // Current time in milliseconds
    action();
    console.log(name, "took", Date.now() - start, "ms");
  }

  time("naive", () => {
    let target = document.getElementById("one");
    while (target.offsetWidth < 2000) {
      target.appendChild(document.createTextNode("X"));
    }
  });
}
```

```
    }
  });
  // → naive took 32 ms

  time("clever", function() {
    let target = document.getElementById("two");
    target.appendChild(document.createTextNode("XXXXX"));
    let total = Math.ceil(2000 / (target.offsetWidth / 5));
    target.firstChild.nodeValue = "X".repeat(total);
  });
  // → clever took 1 ms
</script>
```

STYLING

We have seen that different HTML elements are drawn differently. Some are displayed as blocks, others inline. Some add styling—`<strong>` makes its content bold, and `<a>` makes it blue and underlines it.

The way an `<img>` tag shows an image or an `<a>` tag causes a link to be followed when it is clicked is strongly tied to the element type. But we can change the styling associated with an element, such as the text color or underline. Here is an example that uses the `style` property:

```
<p><a href="#">Normal link</a></p>
<p><a href="#" style="color: green">Green link</a></p>
```

The second link will be green instead of the default link color:

Normal link
Green link

A style attribute may contain one or more *declarations*, which are a property (such as `color`) followed by a colon and a value (such as `green`). When there is more than one declaration, they must be separated by semicolons, as in `"color: red; border: none"`.

A lot of aspects of the document can be influenced by styling. For example, the `display` property controls whether an element is displayed as a block or an inline element.

```
This text is displayed <strong>inline</strong>,
<strong style="display: block">as a block</strong>, and
<strong style="display: none">not at all</strong>.
```

The `block` tag will end up on its own line, since block elements are not displayed inline with the text around them. The last tag is not displayed at all—`display: none` prevents an element from showing up on the screen. This is a way to hide elements. It is often preferable to removing them from the document entirely because it makes it easy to reveal them again later.

This text is displayed **inline**,
as a block
, and .

JavaScript code can directly manipulate the style of an element through the element's `style` property. This property holds an object that has properties for all possible style properties. The values of these proper-

ties are strings, which we can write to in order to change a particular aspect of the element's style.

```
<p id="para" style="color: purple">
  Nice text
</p>
```

```
<script>
  let para = document.getElementById("para");
  console.log(para.style.color);
  para.style.color = "magenta";
</script>
```

Some style property names contain hyphens, such as `font-family`. Because such property names are awkward to work with in JavaScript (you'd have to say `style["font-family"]`), the property names in the `style` object for such properties have their hyphens removed and the letters after them capitalized (`style.fontFamily`).

CASCADING STYLES

The styling system for HTML is called *CSS*, for *Cascading Style Sheets*. A *style sheet* is a set of rules for how to style elements in a document. It can be given inside a `<style>` tag.

```
<style>
  strong {
    font-style: italic;
    color: gray;
  }
```

```
  }
</style>
<p>Now <strong>strong text</strong> is italic and gray.</p>
```

The *cascading* in the name refers to the fact that multiple such rules are combined to produce the final style for an element. In the example, the default styling for `<strong>` tags, which gives them `font-weight: bold`, is overlaid by the rule in the `<style>` tag, which adds `font-style` and `color`.

When multiple rules define a value for the same property, the most recently read rule gets a higher precedence and wins. For example, if the rule in the `<style>` tag included `font-weight: normal`, contradicting the default `font-weight` rule, the text would be normal, *not* bold. Styles in a `style` attribute applied directly to the node have the highest precedence and always win.

It is possible to target things other than tag names in CSS rules. A rule for `.abc` applies to all elements with "abc" in their `class` attribute. A rule for `#xyz` applies to the element with an `id` attribute of "xyz" (which should be unique within the document).

```
.subtle {
  color: gray;
  font-size: 80%;
}
#header {
  background: blue;
  color: white;
}
/* p elements with id main and with classes a and b */
```

```
p#main.a.b {
  margin-bottom: 20px;
}
```

The precedence rule favoring the most recently defined rule applies only when the rules have the same *specificity*. A rule's specificity is a measure of how precisely it describes matching elements, determined by the number and kind (tag, class, or ID) of element aspects it requires. For example, a rule that targets `p.a` is more specific than rules that target `p` or just `.a` and would thus take precedence over them.

The notation `p > a .{ }` applies the given styles to all `<a>` tags that are direct children of `<p>` tags. Similarly, `p a .{ }` applies to all `<a>` tags inside `<p>` tags, whether they are direct or indirect children.

QUERY SELECTORS

We won't be using style sheets very much in this book. Understanding them is helpful when programming in the browser, but they are complicated enough to warrant a separate book. The main reason I introduced *selector* syntax—the notation used in style sheets to determine which elements a set of styles apply to—is that we can use this same mini-language as an effective way to find DOM elements.

The `querySelectorAll` method, which is defined both on the `document` object and on element nodes, takes a selector string and returns a `NodeList` containing all the elements that it matches.

```
<p>And if you go chasing
  <span class="animal">rabbits</span></p>
```

```
<p>And you know you're going to fall</p>
<p>Tell 'em a <span class="character">hookah smoking
  <span class="animal">caterpillar</span></span></p>
<p>Has given you the call</p>
```

```
<script>
  function count(selector) {
    return document.querySelectorAll(selector).length;
  }
  console.log(count("p"));           // All <p> elements
  // + 4
  console.log(count(".animal"));     // Class animal
  // + 2
  console.log(count("p .animal"));   // Animal inside of <p>
  // + 2
  console.log(count("p > .animal")); // Direct child of <p>
  // + 1
</script>
```

Unlike methods such as `getElementsByTagName`, the object returned by `querySelectorAll` is *not* live. It won't change when you change the document. It is still not a real array, though, so you need to call `Array.from` if you want to treat it like one.

The `querySelector` method (without the `All` part) works in a similar way. This one is useful if you want a specific single element. It will return only the first matching element, or `null` when no element matches.

POSITIONING AND ANIMATING

The `position` style property influences layout in a powerful way. It has a default value of `static`, meaning the element sits in its normal place in the document. When it is set to `relative`, the element still takes up space in the document, but now the `top` and `left` style properties can be used to move it relative to that normal place. When `position` is set to `absolute`, the element is removed from the normal document flow—that is, it no longer takes up space and may overlap with other elements. Its `top` and `left` properties can be used to absolutely position it relative to the upper-left corner of the nearest enclosing element whose `position` property isn't `static`, or relative to the document if no such enclosing element exists.

We can use this to create an animation. The following document displays a picture of a cat that moves around in an ellipse:

```
<p style="text-align: center">
  
</p>
<script>
  let cat = document.querySelector("img");
  let angle = Math.PI / 2;
  function animate(time, lastTime) {
    if (lastTime != null) {
      angle += (time - lastTime) * 0.001;
    }
    cat.style.top = (Math.sin(angle) * 20) + "px";
    cat.style.left = (Math.cos(angle) * 200) + "px";
    requestAnimationFrame(newTime => animate(newTime, time));
  }
```

```
  }
  requestAnimationFrame(animate);
</script>
```

The gray arrow shows the path along which the image moves.



Our picture is centered on the page and given a `position` of `relative`. We'll repeatedly update that picture's `top` and `left` styles to move it.

The script uses `requestAnimationFrame` to schedule the `animate` function to run whenever the browser is ready to repaint the screen. The `animate` function itself again calls `requestAnimationFrame` to schedule the next update. When the browser window (or tab) is active, this will cause updates to happen at a rate of about 60 per second, which tends to produce a good-looking animation.

If we just updated the DOM in a loop, the page would freeze, and nothing would show up on the screen. Browsers do not update their display while a JavaScript program is running, nor do they allow any interaction with the page. This is why we need `requestAnimationFrame`—it lets the browser know that we are done for now, and it can go ahead and do the things that browsers do, such as updating the screen and responding to user actions.

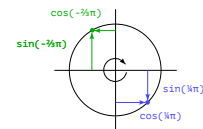
The animation function is passed the current time as an argument.

To ensure that the motion of the cat per millisecond is stable, it bases the speed at which the angle changes on the difference between the current time and the last time the function ran. If it just moved the angle by a fixed amount per step, the motion would stutter when, for example, another heavy task running on the same computer prevented the function from running for a fraction of a second.

Moving in circles is done using the trigonometry functions `Math.cos` and `Math.sin`. For those who aren't familiar with these, I'll briefly introduce them, since we will occasionally use them in this book.

`Math.cos` and `Math.sin` are useful for finding points that lie on a circle around point $(0, 0)$ with a radius of 1. Both functions interpret their argument as the position on this circle, with 0 denoting the point on the far right of the circle, going clockwise until 2π (about 6.28) has taken us around the whole circle. `Math.cos` tells you the x-coordinate of the point that corresponds to the given position, and `Math.sin` yields the y-coordinate. Positions (or angles) greater than 2π or less than 0 are valid—the rotation repeats so that $a+2\pi$ refers to the same angle as a .

This unit for measuring angles is called radians—a full circle is 2π radians, similar to how it is 360 degrees when measuring in degrees. The constant π is available as `Math.PI` in JavaScript.



The cat animation code keeps a counter, `angle`, for the current angle of the animation and increments it every time the `animate` function is called. It can then use this angle to compute the current position of the image element. The `top` style is computed with `Math.sin` and multiplied by 20, which is the vertical radius of our ellipse. The `left` style is based on `Math.cos` and multiplied by 200 so that the ellipse is much wider than it is high.

Note that styles usually need *units*. In this case, we have to append "px" to the number to tell the browser that we are counting in pixels (as opposed to centimeters, "ems", or other units). This is easy to forget. Using numbers without units will result in your style being ignored—unless the number is 0, which always means the same thing, regardless of its unit.

SUMMARY

JavaScript programs may inspect and interfere with the document that the browser is displaying through a data structure called the DOM. This

data structure represents the browser's model of the document, and a JavaScript program can modify it to change the visible document.

The DOM is organized like a tree, where elements are arranged hierarchically according to the structure of the document. The objects representing elements have properties such as `parentNode` and `childNodes`, which can be used to navigate through this tree.

The way a document is displayed can be influenced by *styling*, both by attaching styles to nodes directly and by defining rules that match certain nodes. There are many different style properties, such as `color` or `display`. JavaScript code can manipulate an element's style directly through its `style` property.

EXERCISES

BUILD A TABLE

An HTML table is built with the following tag structure:

```
<table>
<tr>
  <th>name</th>
  <th>height</th>
  <th>place</th>
</tr>
<tr>
  <td>Kilimanjaro</td>
  <td>5895</td>
  <td>Tanzania</td>
```

```
</tr>
</table>
```

For each *row*, the `<table>` tag contains a `<tr>` tag. Inside of these `<tr>` tags, we can put cell elements: either heading cells (`<th>`) or regular cells (`<td>`).

Given a dataset of mountains, an array of objects with `name`, `height`, and `place` properties, generate the DOM structure for a table that enumerates the objects. It has one column per key and one row per object, plus a header row with `<th>` elements at the top, listing the column names.

Write this so that the columns are automatically derived from the objects, by taking the property names of the first object in the data.

Show the resulting table in the document by appending it to the element that has an `id` attribute of "mountains".

Once you have this working, right-align cells that contain number values by setting their `style.textAlign` property to "right".

ELEMENTS BY TAG NAME

The `document.getElementsByTagName` method returns all child elements with a given tag name. Implement your own version of this as a function that takes a node and a string (the tag name) as arguments and returns an array containing all descendant element nodes with the given tag name. Your function should go through the document itself. It may not use a method like `querySelectorAll` to do the work.

To find the tag name of an element, use its `nodeName` property. But

note that this will return the tag name in all uppercase. Use the `toLowerCase` or `toUpperCase` string methods to compensate for this.

THE CAT’S HAT

Extend the cat animation defined earlier so that both the cat and his hat (``) orbit at opposite sides of the ellipse.

Or make the hat circle around the cat. Or alter the animation in some other interesting way.

To make positioning multiple objects easier, you’ll probably want to switch to absolute positioning. This means that `top` and `left` are counted relative to the upper left of the document. To avoid using negative coordinates, which would cause the image to move outside of the visible page, you can add a fixed number of pixels to the position values.

“You have power over your mind—not outside events. Realize this, and you will find strength.”

—Marcus Aurelius, *Meditations*

CHAPTER 15

HANDLING EVENTS

Some programs work with direct user input, such as mouse and keyboard actions. That kind of input isn’t available ahead of time, as a well-organized data structure—it comes in piece by piece, in real time, and the program must respond to it as it happens.

EVENT HANDLERS

Imagine an interface where the only way to find out whether a key on the keyboard is being pressed is to read the current state of that key. To be able to react to keypresses, you would have to constantly read the key’s state to catch it before it is released again. It would be dangerous to perform other time-intensive computations, since you might miss a keypress.

Some primitive machines handle input like this. A step up from this is for the hardware or operating system to notice the keypress and put it in a queue. A program can then periodically check the queue for new events and react to what it finds there.

Of course, the program has to remember to look at the queue, and to do it often because any time between the key being pressed and the program noticing the event will cause the software to feel unresponsive. This approach is called *polling*. Most programmers prefer to avoid it.

A better mechanism is for the system to actively notify the code when an event occurs. Browsers do this by allowing us to register functions as *handlers* for specific events.

```
<p>Click this document to activate the handler.</p>
<script>
  window.addEventListener("click", () => {
    console.log("You knocked?");
  });
</script>
```

The window binding refers to a built-in object provided by the browser. It represents the browser window that contains the document. Calling its `addEventListener` method registers the second argument to be called whenever the event described by its first argument occurs.

EVENTS AND DOM NODES

Each browser event handler is registered in a context. In the previous example, we called `addEventListener` on the `window` object to register a handler for the whole window. Such a method can also be found on DOM elements and some other types of objects. Event listeners are called only when the event happens in the context of the object on which they are registered.

```
<button>Click me</button>
<p>No handler here.</p>
<script>
  let button = document.querySelector("button");
  button.addEventListener("click", () => {
    console.log("Button clicked.");
  });
</script>
```

That example attaches a handler to the button node. Clicks on the button cause that handler to run, but clicks on the rest of the document do not.

Giving a node an `onclick` attribute has a similar effect. This works for most types of events—you can attach a handler through the attribute whose name is the event name with `on` in front of it.

But a node can have only one `onclick` attribute, so you can register only one handler per node that way. The `addEventListener` method allows you to add any number of handlers meaning it's safe to add handlers even if there is already another handler on the element.

The `removeEventListener` method, called with arguments similar to `addEventListener`, removes a handler.

```
<button>Act-once button</button>
<script>
  let button = document.querySelector("button");
  function once() {
    console.log("Done.");
    button.removeEventListener("click", once);
  }
}
```

```
button.addEventListener("click", once);
</script>
```

The function given to `removeEventListener` has to be the same function value given to `addEventListener`. When you need to unregister a handler, you'll want to give the handler function a name (once, in the example) to be able to pass the same function value to both methods.

EVENT OBJECTS

Though we have ignored it so far, event handler functions are passed an argument: the *event object*. This object holds additional information about the event. For example, if we want to know *which* mouse button was pressed, we can look at the event object's `button` property.

```
<button>Click me any way you want</button>
<script>
  let button = document.querySelector("button");
  button.addEventListener("mousedown", event => {
    if (event.button == 0) {
      console.log("Left button");
    } else if (event.button == 1) {
      console.log("Middle button");
    } else if (event.button == 2) {
      console.log("Right button");
    }
  });
</script>
```

The information stored in an event object differs per type of event. (We'll discuss different types later in the chapter.) The object's `type` property always holds a string identifying the event (such as `"click"` or `"mousedown"`).

PROPAGATION

For most event types, handlers registered on nodes with children will also receive events that happen in the children. If a button inside a paragraph is clicked, event handlers on the paragraph will also see the click event.

But if both the paragraph and the button have a handler, the more specific handler—the one on the button—gets to go first. The event is said to *propagate* outward from the node where it happened to that node's parent node and on to the root of the document. Finally, after all handlers registered on a specific node have had their turn, handlers registered on the whole window get a chance to respond to the event.

At any point, an event handler can call the `stopPropagation` method on the event object to prevent handlers further up from receiving the event. This can be useful when, for example, you have a button inside another clickable element and you don't want clicks on the button to activate the outer element's click behavior.

The following example registers `"mousedown"` handlers on both a button and the paragraph around it. When clicked with the right mouse button, the handler for the button calls `stopPropagation`, which will prevent the handler on the paragraph from running. When the button

is clicked with another mouse button, both handlers will run.

```
<p>A paragraph with a <button>button</button>.</p>
<script>
  let para = document.querySelector("p");
  let button = document.querySelector("button");
  para.addEventListener("mousedown", () => {
    console.log("Handler for paragraph.");
  });
  button.addEventListener("mousedown", event => {
    console.log("Handler for button.");
    if (event.button == 2) event.stopPropagation();
  });
</script>
```

Most event objects have a `target` property that refers to the node where they originated. You can use this property to ensure that you're not accidentally handling something that propagated up from a node you do not want to handle.

It is also possible to use the `target` property to cast a wide net for a specific type of event. For example, if you have a node containing a long list of buttons, it may be more convenient to register a single click handler on the outer node and have it use the `target` property to figure out whether a button was clicked, rather than registering individual handlers on all of the buttons.

```
<button>A</button>
<button>B</button>
<button>C</button>
<script>
```

```
document.body.addEventListener("click", event => {
  if (event.target.nodeName == "BUTTON") {
    console.log("Clicked", event.target.textContent);
  }
});
</script>
```

DEFAULT ACTIONS

Many events have a default action. If you click a link, you will be taken to the link's target. If you press the down arrow, the browser will scroll the page down. If you right-click, you'll get a context menu. And so on.

For most types of events, the JavaScript event handlers are called *before* the default behavior takes place. If the handler doesn't want this normal behavior to happen, typically because it has already taken care of handling the event, it can call the `preventDefault` method on the event object.

This can be used to implement your own keyboard shortcuts or context menu. It can also be used to obnoxiously interfere with the behavior that users expect. For example, here is a link that cannot be followed:

```
<a href="https://developer.mozilla.org/">MDN</a>
<script>
  let link = document.querySelector("a");
  link.addEventListener("click", event => {
```

```
        console.log("Nope.");
        event.preventDefault();
    });
</script>
```

Try not to do such things without a really good reason. It'll be unpleasant for people who use your page when expected behavior is broken.

Depending on the browser, some events can't be intercepted at all. On Chrome, for example, the keyboard shortcut to close the current tab (CTRL-W or COMMAND-W) cannot be handled by JavaScript.

KEY EVENTS

When a key on the keyboard is pressed, your browser fires a "keydown" event. When it is released, you get a "keyup" event.

```
<p>This page turns violet when you hold the V key.</p>
<script>
    window.addEventListener("keydown", event => {
        if (event.key == "v") {
            document.body.style.background = "violet";
        }
    });
    window.addEventListener("keyup", event => {
        if (event.key == "v") {
            document.body.style.background = "";
        }
    });
</script>
```

Despite its name, "keydown" fires not only when the key is physically pushed down. When a key is pressed and held, the event fires again every time the key *repeats*. Sometimes you have to be careful about this. For example, if you add a button to the DOM when a key is pressed and remove it again when the key is released, you might accidentally add hundreds of buttons when the key is held down longer.

The previous example looks at the `key` property of the event object to see which key the event is about. This property holds a string that, for most keys, corresponds to the thing that pressing that key would type. For special keys such as ENTER, it holds a string that names the key ("Enter", in this case). If you hold SHIFT while pressing a key, that might also influence the name of the key—"v" becomes "V", and "1" may become "!", if that is what pressing SHIFT-1 produces on your keyboard.

Modifier keys such as SHIFT, CTRL, ALT, and META (COMMAND on Mac) generate key events just like normal keys. When looking for key combinations, you can also find out whether these keys are held down by looking at the `shiftKey`, `ctrlKey`, `altKey`, and `metaKey` properties of keyboard and mouse events.

```
<p>Press Control-Space to continue.</p>
<script>
    window.addEventListener("keydown", event => {
        if (event.key == " " && event.ctrlKey) {
            console.log("Continuing!");
        }
    });
</script>
```


The DOM node where a key event originates depends on the element that has focus when the key is pressed. Most nodes cannot have focus unless you give them a `tabindex` attribute, but things like links, buttons, and form fields can. We'll come back to form fields in Chapter 18. When nothing in particular has focus, `document.body` acts as the target node of key events.

When the user is typing text, using key events to figure out what is being typed is problematic. Some platforms, most notably the virtual keyboard on Android phones, don't fire key events. But even when you have an old-fashioned keyboard, some types of text input don't match keypresses in a straightforward way, such as *input method editor (IME)* software used by people whose scripts don't fit on a keyboard, where multiple keystrokes are combined to create characters.

To notice when something was typed, elements that you can type into, such as the `<input>` and `<textarea>` tags, fire "input" events whenever the user changes their content. To get the actual content that was typed, it is best to directly read it from the focused field, which we discuss in Chapter 18.

POINTER EVENTS

There are currently two widely used ways to point at things on a screen: mice (including devices that act like mice, such as touchpads and trackballs) and touchscreens. These produce different kinds of events.

MOUSE CLICKS

Pressing a mouse button causes a number of events to fire. The "mousedown" and "mouseup" events are similar to "keydown" and "keyup" and fire when the button is pressed and released. These happen on the DOM nodes that are immediately below the mouse pointer when the event occurs.

After the "mouseup" event, a "click" event fires on the most specific node that contained both the press and the release of the button. For example, if I press down the mouse button on one paragraph and then move the pointer to another paragraph and release the button, the "click" event will happen on the element that contains both those paragraphs.

If two clicks happen close together, a "dblclick" (double-click) event also fires, after the second click event.

To get precise information about the place where a mouse event happened, you can look at its `clientX` and `clientY` properties, which contain the event's coordinates (in pixels) relative to the upper-left corner of the window, or `pageX` and `pageY`, which are relative to the upper-left corner of the whole document (which may be different when the window has been scrolled).

The following program implements a primitive drawing application. Every time you click the document, it adds a dot under your mouse pointer.

```
<style>
body {
  height: 200px;
```

```

        background: beige;
    }
    .dot {
        height: 8px; width: 8px;
        border-radius: 4px; /* rounds corners */
        background: teal;
        position: absolute;
    }
</style>
<script>
    window.addEventListener("click", event => {
        let dot = document.createElement("div");
        dot.className = "dot";
        dot.style.left = (event.pageX - 4) + "px";
        dot.style.top = (event.pageY - 4) + "px";
        document.body.appendChild(dot);
    });
</script>

```

We'll create a less primitive drawing application in [Chapter 19](#).

MOUSE MOTION

Every time the mouse pointer moves, a "mousemove" event fires. This event can be used to track the position of the mouse. A common situation in which this is useful is when implementing some form of mouse-dragging functionality.

As an example, the following program displays a bar and sets up event handlers so that dragging to the left or right on this bar makes it

narrower or wider:

```

<p>Drag the bar to change its width:</p>
<div style="background: orange; width: 60px; height: 20px">
</div>
<script>
    let lastX; // Tracks the last observed mouse X position
    let bar = document.querySelector("div");
    bar.addEventListener("mousedown", event => {
        if (event.button == 0) {
            lastX = event.clientX;
            window.addEventListener("mousemove", moved);
            event.preventDefault(); // Prevent selection
        }
    });

    function moved(event) {
        if (event.buttons == 0) {
            window.removeEventListener("mousemove", moved);
        } else {
            let dist = event.clientX - lastX;
            let newWidth = Math.max(10, bar.offsetWidth + dist);
            bar.style.width = newWidth + "px";
            lastX = event.clientX;
        }
    }
</script>

```

The resulting page looks like this:

Drag the bar to change its width:



Note that the "mousemove" handler is registered on the whole window. Even if the mouse goes outside of the bar during resizing, as long as the button is held, we still want to update its size.

We must stop resizing the bar when the mouse button is released. For that, we can use the `buttons` property (note the plural), which tells us about the buttons that are currently held down. When it is 0, no buttons are down. When buttons are held, the value of the `buttons` property is the sum of the codes for those buttons—the left button has code 1, the right button 2, and the middle one 4. With the left and right buttons held, for example, the value of `buttons` will be 3.

Note that the order of these codes is different from the one used by `button`, where the middle button came before the right one. As mentioned, consistency isn't a strong point of the browser's programming interface.

TOUCH EVENTS

The style of graphical browser that we use was designed with mouse interfaces in mind, at a time where touchscreens were rare. To make the web "work" on early touchscreen phones, browsers for those devices pretended, to a certain extent, that touch events were mouse events. If you tap your screen, you'll get "mousedown", "mouseup", and "click" events.

But this illusion isn't very robust. A touchscreen doesn't work like a mouse: it doesn't have multiple buttons, you can't track the finger when it isn't on the screen (to simulate "mousemove"), and it allows multiple fingers to be on the screen at the same time.

Mouse events cover touch interaction only in straightforward cases—if you add a "click" handler to a button, touch users will still be able to use it. But something like the resizable bar in the previous example does not work on a touchscreen.

There are specific event types fired by touch interaction. When a finger starts touching the screen, you get a "touchstart" event. When it is moved while touching, "touchmove" events fire. Finally, when it stops touching the screen, you'll see a "touchend" event.

Because many touchscreens can detect multiple fingers at the same time, these events don't have a single set of coordinates associated with them. Rather, their event objects have a `touches` property, which holds an array-like object of points, each of which has its own `clientX`, `clientY`, `pageX`, and `pageY` properties.

You could do something like this to show red circles around every touching finger:

```
<style>
  dot { position: absolute; display: block;
        border: 2px solid red; border-radius: 50px;
        height: 100px; width: 100px; }
</style>
<p>Touch this page</p>
<script>
  function update(event) {
```

```

    for (let dot; dot = document.querySelector("dot");) {
      dot.remove();
    }
    for (let i = 0; i < event.touches.length; i++) {
      let {pageX, pageY} = event.touches[i];
      let dot = document.createElement("dot");
      dot.style.left = (pageX - 50) + "px";
      dot.style.top = (pageY - 50) + "px";
      document.body.appendChild(dot);
    }
  }
  window.addEventListener("touchstart", update);
  window.addEventListener("touchmove", update);
  window.addEventListener("touchend", update);
</script>

```

You'll often want to call `preventDefault` in touch event handlers to override the browser's default behavior (which may include scrolling the page on swiping) and to prevent the mouse events from being fired, for which you may *also* have a handler.

SCROLL EVENTS

Whenever an element is scrolled, a "scroll" event is fired on it. This has various uses, such as knowing what the user is currently looking at (for disabling off-screen animations or sending spy reports to your evil headquarters) or showing some indication of progress (by highlighting part of a table of contents or showing a page number).

The following example draws a progress bar above the document and

updates it to fill up as you scroll down:

```

<style>
  #progress {
    border-bottom: 2px solid blue;
    width: 0;
    position: fixed;
    top: 0; left: 0;
  }
</style>
<div id="progress"></div>
<script>
  // Create some content
  document.body.appendChild(document.createTextNode(
    "supercalifragilisticexpialidocious ".repeat(1000)));

  let bar = document.querySelector("#progress");
  window.addEventListener("scroll", () => {
    let max = document.body.scrollHeight - innerHeight;
    bar.style.width = `${(pageYOffset / max) * 100}%`;
  });
</script>

```

Giving an element a position of `fixed` acts much like an `absolute` position but also prevents it from scrolling along with the rest of the document. The effect is to make our progress bar stay at the top. Its width is changed to indicate the current progress. We use `%`, rather than `px`, as a unit when setting the width so that the element is sized relative to the page width.

The global `innerHeight` binding gives us the height of the window,

which we must subtract from the total scrollable height—you can’t keep scrolling when you hit the bottom of the document. There’s also an `innerWidth` for the window width. By dividing `pageYOffset`, the current scroll position, by the maximum scroll position and multiplying by 100, we get the percentage for the progress bar.

Calling `preventDefault` on a scroll event does not prevent the scrolling from happening. In fact, the event handler is called only *after* the scrolling takes place.

FOCUS EVENTS

When an element gains focus, the browser fires a “focus” event on it. When it loses focus, the element gets a “blur” event.

Unlike the events discussed earlier, these two events do not propagate. A handler on a parent element is not notified when a child element gains or loses focus.

The following example displays help text for the text field that currently has focus:

```
<p>Name: <input type="text" data-help="Your full name"></p>
<p>Age: <input type="text" data-help="Your age in years"></p>
<p id="help"></p>

<script>
  let help = document.querySelector("#help");
  let fields = document.querySelectorAll("input");
  for (let field of Array.from(fields)) {
    field.addEventListener("focus", event => {
```

```
    let text = event.target.getAttribute("data-help");
    help.textContent = text;
  });
  field.addEventListener("blur", event => {
    help.textContent = "";
  });
}
</script>
```

This screenshot shows the help text for the age field:

Name:

Age:

Age in years

The window object will receive “focus” and “blur” events when the user moves from or to the browser tab or window in which the document is shown.

LOAD EVENT

When a page finishes loading, the “load” event fires on the window and the document body objects. This is often used to schedule initialization actions that require the whole document to have been built. Remember that the content of `<script>` tags is run immediately when the tag is encountered. This may be too soon, for example when the script needs to do something with parts of the document that appear after the `<`

`script`> tag.

Elements such as images and script tags that load an external file also have a "load" event that indicates the files they reference were loaded. Like the focus-related events, loading events do not propagate.

When you close page or navigate away from it (for example, by following a link), a "beforeunload" event fires. The main use of this event is to prevent the user from accidentally losing work by closing a document. If you prevent the default behavior on this event *and* set the `returnValue` property on the event object to a string, the browser will show the user a dialog asking if they really want to leave the page. That dialog might include your string, but because some malicious sites try to use these dialogs to confuse people into staying on their page to look at dodgy weight-loss ads, most browsers no longer display them.

EVENTS AND THE EVENT LOOP

In the context of the event loop, as discussed in Chapter 11, browser event handlers behave like other asynchronous notifications. They are scheduled when the event occurs but must wait for other scripts that are running to finish before they get a chance to run.

The fact that events can be processed only when nothing else is running means that if the event loop is tied up with other work, any interaction with the page (which happens through events) will be delayed until there's time to process it. So if you schedule too much work, either with long-running event handlers or with lots of short-running ones, the page will become slow and cumbersome to use.

For cases where you *really* do want to do some time-consuming thing in the background without freezing the page, browsers provide something called *web workers*. A worker is a JavaScript process that runs alongside the main script, on its own timeline.

Imagine that squaring a number is a heavy, long-running computation that we want to perform in a separate thread. We could write a file called `code/squareworker.js` that responds to messages by computing a square and sending a message back.

```
addEventListener("message", event => {  
  postMessage(event.data * event.data);  
});
```

To avoid the problems of having multiple threads touching the same data, workers do not share their global scope or any other data with the main script's environment. Instead, you have to communicate with them by sending messages back and forth.

This code spawns a worker running that script, sends it a few messages, and outputs the responses.

```
let squareWorker = new Worker("code/squareworker.js");  
squareWorker.addEventListener("message", event => {  
  console.log("The worker responded:", event.data);  
});  
squareWorker.postMessage(10);  
squareWorker.postMessage(24);
```

The `postMessage` function sends a message, which will cause a "message" event to fire in the receiver. The script that created the worker sends

and receives messages through the `Worker` object, whereas the worker talks to the script that created it by sending and listening directly on its global scope. Only values that can be represented as JSON can be sent as messages—the other side will receive a *copy* of them, rather than the value itself.

TIMERS

The `setTimeout` function we saw in Chapter 11 schedules another function to be called later, after a given number of milliseconds. Sometimes you need to cancel a function you have scheduled. You can do this by storing the value returned by `setTimeout` and calling `clearTimeout` on it.

```
let bombTimer = setTimeout(() => {
  console.log("BOOM!");
}, 500);

if (Math.random() < 0.5) { // 50% chance
  console.log("Defused.");
  clearTimeout(bombTimer);
}
```

The `cancelAnimationFrame` function works in the same way as `clearTimeout`. Calling it on a value returned by `requestAnimationFrame` will cancel that frame (assuming it hasn't already been called).

A similar set of functions, `setInterval` and `clearInterval`, are used to set timers that should repeat every *X* milliseconds.

```
let ticks = 0;
```

```
let clock = setInterval(() => {
  console.log("tick", ticks++);
  if (ticks == 10) {
    clearInterval(clock);
    console.log("stop.");
  }
}, 200);
```

DEBOUNCING

Some types of events have the potential to fire rapidly many times in a row, such as the "mousemove" and "scroll" events. When handling such events, you must be careful not to do anything too time-consuming or your handler will take up so much time that interaction with the document starts to feel slow.

If you do need to do something nontrivial in such a handler, you can use `setTimeout` to make sure you are not doing it too often. This is usually called *debouncing* the event. There are several slightly different approaches to this.

For example, suppose we want to react when the user has typed something, but we don't want to do it immediately for every input event. When they are typing quickly, we just want to wait until a pause occurs. Instead of immediately performing an action in the event handler, we set a timeout. We also clear the previous timeout (if any) so that when events occur close together (closer than our timeout delay), the timeout from the previous event will be canceled.

```

<textarea>Type something here...</textarea>
<script>
  let textarea = document.querySelector("textarea");
  let timeout;
  textarea.addEventListener("input", () => {
    clearTimeout(timeout);
    timeout = setTimeout(() => console.log("Typed!"), 500);
  });
</script>

```

Giving an undefined value to `clearTimeout` or calling it on a timeout that has already fired has no effect. Thus, we don't have to be careful about when to call it, and we simply do so for every event.

We can use a slightly different pattern if we want to space responses so that they're separated by at least a certain length of time but want to fire them *during* a series of events, not just afterward. For example, we might want to respond to "mousemove" events by showing the current coordinates of the mouse, but only every 250 milliseconds.

```

<script>
  let scheduled = null;
  window.addEventListener("mousemove", event => {
    if (!scheduled) {
      setTimeout(() => {
        document.body.textContent =
          `Mouse at ${scheduled.pageX}, ${scheduled.pageY}`;
        scheduled = null;
      }, 250);
    }
    scheduled = event;
  });

```

```

  });
</script>

```

SUMMARY

Event handlers make it possible to detect and react to events happening in our web page. The `addEventListener` method is used to register such a handler.

Each event has a type ("keydown", "focus", and so on) that identifies it. Most events are called on a specific DOM element and then propagate to that element's ancestors, allowing handlers associated with those elements to handle them.

When an event handler is called, it's passed an event object with additional information about the event. This object also has methods that allow us to stop further propagation (`stopPropagation`) and prevent the browser's default handling of the event (`preventDefault`).

Pressing a key fires "keydown" and "keyup" events. Pressing a mouse button fires "mousedown", "mouseup", and "click" events. Moving the mouse fires "mousemove" events. Touchscreen interaction will result in "touchstart", "touchmove", and "touchend" events.

Scrolling can be detected with the "scroll" event, and focus changes can be detected with the "focus" and "blur" events. When the document finishes loading, a "load" event fires on the window.

EXERCISES

BALLOON

Write a page that displays a balloon (using the balloon emoji, 🎈). When you press the up arrow, it should inflate (grow) 10 percent. When you press the down arrow, it should deflate (shrink) 10 percent.

You can control the size of text (emoji are text) by setting the `font-size` CSS property (`style.fontSize`) on its parent element. Remember to include a unit in the value—for example, pixels (10px).

The key names of the arrow keys are `"ArrowUp"` and `"ArrowDown"`. Make sure the keys change only the balloon, without scrolling the page.

Once you have that working, add a feature where if you blow up the balloon past a certain size, it “explodes”. In this case, exploding means that it is replaced with an 💣 emoji, and the event handler is removed (so that you can’t inflate or deflate the explosion).

MOUSE TRAIL

In JavaScript’s early days, which was the high time of gaudy home pages with lots of animated images, people came up with some truly inspiring ways to use the language. One of these was the *mouse trail*—a series of elements that would follow the mouse pointer as you moved it across the page.

In this exercise, I want you to implement a mouse trail. Use absolutely positioned `<div>` elements with a fixed size and background color (refer to the code in the “Mouse Clicks” section for an example). Create

a bunch of these elements and, when the mouse moves, display them in the wake of the mouse pointer.

There are various possible approaches here. You can make your trail as simple or as complex as you want. A simple solution to start with is to keep a fixed number of trail elements and cycle through them, moving the next one to the mouse’s current position every time a `"mousemove"` event occurs.

TABS

Tabbed panels are common in user interfaces. They allow you to select an interface panel by choosing from a number of tabs “sticking out” above an element.

Implement a simple tabbed interface. Write a function, `asTabs`, that takes a DOM node and creates a tabbed interface showing the child elements of that node. It should insert a list of `<button>` elements at the top of the node, one for each child element, containing text retrieved from the `data-tabname` attribute of the child. All but one of the original children should be hidden (given a `display` style of `none`). The currently visible node can be selected by clicking the buttons.

When that works, extend it to style the button for the currently selected tab differently so that it is obvious which tab is selected.

“All reality is a game.”

—Iain Banks, *The Player of Games*

CHAPTER 16

PROJECT: A PLATFORM GAME

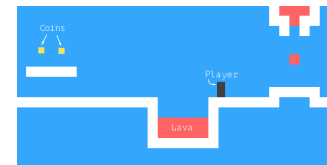
Much of my initial fascination with computers, like that of many nerdy kids, had to do with computer games. I was drawn into the tiny simulated worlds that I could manipulate and in which stories (sort of) unfolded—more, I suppose, because of the way I projected my imagination into them than because of the possibilities they actually offered.

I don’t wish a career in game programming on anyone. As with the music industry, the discrepancy between the number of eager young people wanting to work in it and the actual demand for such people creates a rather unhealthy environment. But writing games for fun is amusing.

This chapter will walk through the implementation of a small platform game. Platform games (or “jump and run” games) are games that expect the player to move a figure through a world, which is usually two-dimensional and viewed from the side, while jumping over and onto things.

THE GAME

Our game will be roughly based on Dark Blue (www.lessmilk.com/games/10) by Thomas Palef. I chose that game because it is both entertaining and minimalist and because it can be built without too much code. It looks like this:



The dark box represents the player, whose task is to collect the yellow boxes (coins) while avoiding the red stuff (lava). A level is completed when all coins have been collected.

The player can walk around with the left and right arrow keys and can jump with the up arrow. Jumping is this game character’s specialty. It can reach several times its own height and can change direction in midair. This may not be entirely realistic, but it helps give the player the feeling of being in direct control of the on-screen avatar.

The game consists of a static background, laid out like a grid, with the moving elements overlaid on that background. Each field on the grid is either empty, solid, or lava. The moving elements are the player,

coins, and certain pieces of lava. The positions of these elements are not constrained to the grid—their coordinates may be fractional, allowing smooth motion.

THE TECHNOLOGY

We will use the browser DOM to display the game, and we'll read user input by handling key events.

The screen- and keyboard-related code is only a small part of the work we need to do to build this game. Since everything looks like colored boxes, drawing is uncomplicated: we create DOM elements and use styling to give them a background color, size, and position.

We can represent the background as a table, since it is an unchanging grid of squares. The free-moving elements can be overlaid using absolutely positioned elements.

In games and other programs that should animate graphics and respond to user input without noticeable delay, efficiency is important. Although the DOM was not originally designed for high-performance graphics, it is actually better at this than you would expect. You saw some animations in Chapter 14. On a modern machine, a simple game like this performs well, even if we don't worry about optimization very much.

In the next chapter, we will explore another browser technology, the `<canvas>` tag, which provides a more traditional way to draw graphics, working in terms of shapes and pixels rather than DOM elements.

LEVELS

We'll want a human-readable, human-editable way to specify levels. Since it is okay for everything to start out on a grid, we could use big strings in which each character represents an element—either a part of the background grid or a moving element.

The plan for a small level might look like this:

```
let simpleLevelPlan = `
.....
..#.....#..
..#.....=..#..
..#.....O.O...#..
..#.@.....####...#..
..#####...#..
.....#+++++++#+#..
.....#####...#..
.....`;
```

Periods are empty space, hash (#) characters are walls, and plus signs are lava. The player's starting position is the at sign (@). Every O character is a coin, and the equal sign (=) at the top is a block of lava that moves back and forth horizontally.

We'll support two additional kinds of moving lava: the pipe character (|) creates vertically moving blobs, and v indicates *dripping* lava—vertically moving lava that doesn't bounce back and forth but only moves down, jumping back to its start position when it hits the floor.

A whole game consists of multiple levels that the player must complete. A level is completed when all coins have been collected. If the

player touches lava, the current level is restored to its starting position, and the player may try again.

READING A LEVEL

The following class stores a level object. Its argument should be the string that defines the level.

```
class Level {
  constructor(plan) {
    let rows = plan.trim().split("\n").map(l => [...l]);
    this.height = rows.length;
    this.width = rows[0].length;
    this.startActors = [];

    this.rows = rows.map((row, y) => {
      return row.map((ch, x) => {
        let type = levelChars[ch];
        if (typeof type != "string") {
          let pos = new Vec(x, y);
          this.startActors.push(type.create(pos, ch));
          type = "empty";
        }
        return type;
      });
    });
  }
}
```

The `trim` method is used to remove whitespace at the start and end of

the plan string. This allows our example plan to start with a newline so that all lines are directly below each other. The remaining string is split on newline characters, and each line is spread into an array, producing arrays of characters.

So `rows` holds an array of arrays of characters, the rows of the plan. We can derive the level's width and height from these. But we must still separate the moving elements from the background grid. We'll call moving elements *actors*. They'll be stored in an array of objects. The background will be an array of arrays of strings, holding field types such as "empty", "wall", or "lava".

To create these arrays, we map over the rows and then over their content. Remember that `map` passes the array index as a second argument to the mapping function, which tells us the x- and y-coordinates of a given character. Positions in the game will be stored as pairs of coordinates, with the upper left being 0,0 and each background square being 1 unit high and wide.

To interpret the characters in the plan, the `Level` constructor uses the `levelChars` object, which, for each character used in the level descriptions, holds a string if it is a background type, and a class if it produces an actor. When `type` is an actor class, its static `create` method is used to create an object, which is added to `startActors`, and the mapping function returns "empty" for this background square.

The position of the actor is stored as a `Vec` object. This is a two-dimensional vector, an object with `x` and `y` properties, as seen in the exercises of Chapter 6.

As the game runs, actors will end up in different places or even dis-

appear entirely (as coins do when collected). We'll use a `State` class to track the state of a running game.

```
class State {
  constructor(level, actors, status) {
    this.level = level;
    this.actors = actors;
    this.status = status;
  }

  static start(level) {
    return new State(level, level.startActors, "playing");
  }

  get player() {
    return this.actors.find(a => a.type == "player");
  }
}
```

The `status` property will switch to "lost" or "won" when the game has ended.

This is again a persistent data structure—updating the game state creates a new state and leaves the old one intact.

ACTORS

Actor objects represent the current position and state of a given moving element (player, coin, or mobile lava) in our game. All actor objects conform to the same interface. They have `size` and `pos` properties hold-

ing the size and the coordinates of the upper-left corner of the rectangle representing this actor, and an `update` method.

This `update` method is used to compute their new state and position after a given time step. It simulates the thing the actor does—moving in response to the arrow keys for the player and bouncing back and forth for the lava—and returns a new, updated actor object.

A `type` property contains a string that identifies the type of the actor—"player", "coin", or "lava". This is useful when drawing the game—the look of the rectangle drawn for an actor is based on its type.

Actor classes have a static `create` method that is used by the `Level` constructor to create an actor from a character in the level plan. It is given the coordinates of the character and the character itself, which is necessary because the `Lava` class handles several different characters.

This is the `Vec` class that we'll use for our two-dimensional values, such as the position and size of actors.

```
class Vec {
  constructor(x, y) {
    this.x = x; this.y = y;
  }
  plus(other) {
    return new Vec(this.x + other.x, this.y + other.y);
  }
  times(factor) {
    return new Vec(this.x * factor, this.y * factor);
  }
}
```

The `times` method scales a vector by a given number. It will be useful when we need to multiply a speed vector by a time interval to get the distance traveled during that time.

The different types of actors get their own classes, since their behavior is very different. Let's define these classes. We'll get to their `update` methods later.

The player class has a `speed` property that stores its current speed to simulate momentum and gravity.

```
class Player {
  constructor(pos, speed) {
    this.pos = pos;
    this.speed = speed;
  }

  get type() { return "player"; }

  static create(pos) {
    return new Player(pos.plus(new Vec(0, -0.5)),
                      new Vec(0, 0));
  }
}

Player.prototype.size = new Vec(0.8, 1.5);
```

Because a player is one-and-a-half squares high, its initial position is set to be half a square above the position where the @ character appeared. This way, its bottom aligns with the bottom of the square where it appeared.

The `size` property is the same for all instances of `Player`, so we store it on the prototype rather than on the instances themselves. We could have used a getter like `type`, but that would create and return a new `Vec` object every time the property is read, which would be wasteful. (Strings, being immutable, don't have to be re-created every time they are evaluated.)

When constructing a `Lava` actor, we need to initialize the object differently depending on the character it is based on. Dynamic lava moves along at its current speed until it hits an obstacle. At that point, if it has a `reset` property, it will jump back to its start position (dripping). If it does not, it will invert its speed and continue in the other direction (bouncing).

The `create` method looks at the character that the `Level` constructor passes and creates the appropriate lava actor.

```
class Lava {
  constructor(pos, speed, reset) {
    this.pos = pos;
    this.speed = speed;
    this.reset = reset;
  }

  get type() { return "lava"; }

  static create(pos, ch) {
    if (ch == "=") {
      return new Lava(pos, new Vec(2, 0));
    } else if (ch == "|") {
      return new Lava(pos, new Vec(0, 2));
    }
  }
}
```

```

    } else if (ch == "v") {
        return new Lava(pos, new Vec(0, 3), pos);
    }
}

```

```
Lava.prototype.size = new Vec(1, 1);
```

Coin actors are relatively simple. They mostly just sit in their place. But to liven up the game a little, they are given a “wobble”, a slight vertical back-and-forth motion. To track this, a coin object stores a base position as well as a wobble property that tracks the phase of the bouncing motion. Together, these determine the coin’s actual position (stored in the `pos` property).

```

class Coin {
  constructor(pos, basePos, wobble) {
    this.pos = pos;
    this.basePos = basePos;
    this.wobble = wobble;
  }

  get type() { return "coin"; }

  static create(pos) {
    let basePos = pos.plus(new Vec(0.2, 0.1));
    return new Coin(basePos, basePos,
                    Math.random() * Math.PI * 2);
  }
}

```

```
Coin.prototype.size = new Vec(0.6, 0.6);
```

In [Chapter 14](#), we saw that `Math.sin` gives us the y-coordinate of a point on a circle. That coordinate goes back and forth in a smooth waveform as we move along the circle, which makes the sine function useful for modeling a wavy motion.

To avoid a situation where all coins move up and down synchronously, the starting phase of each coin is randomized. The period of `Math.sin`’s wave, the width of a wave it produces, is 2π . We multiply the value returned by `Math.random` by that number to give the coin a random starting position on the wave.

We can now define the `levelChars` object that maps plan characters to either background grid types or actor classes.

```

const levelChars = {
  ".": "empty", "#": "wall", "+": "lava",
  "@": Player, "o": Coin,
  "=": Lava, "|": Lava, "v": Lava
};

```

That gives us all the parts needed to create a `Level` instance.

```

let simpleLevel = new Level(simpleLevelPlan);
console.log(`${simpleLevel.width} by ${simpleLevel.height}`);
// → 22 by 9

```

The task ahead is to display such levels on the screen and to model time and motion inside them.

DRAWING

In the next chapter, we'll display the same game in a different way. To make that possible, we put the drawing logic behind an interface and pass it to the game as an argument. That way, we can use the same game program with different new display modules.

A game display object draws a given level and state. We pass its constructor to the game to allow it to be replaced. The display class we define in this chapter is called `DOMDisplay` because it uses DOM elements to show the level.

We'll be using a style sheet to set the actual colors and other fixed properties of the elements that make up the game. It would also be possible to directly assign to the elements' `style` property when we create them, but that would produce more verbose programs.

The following helper function provides a succinct way to create an element and give it some attributes and child nodes:

```
function elt(name, attrs, ...children) {
  let dom = document.createElement(name);
  for (let attr of Object.keys(attrs)) {
    dom.setAttribute(attr, attrs[attr]);
  }
  for (let child of children) {
    dom.appendChild(child);
  }
  return dom;
}
```

A display is created by giving it a parent element to which it should

append itself and a level object.

```
class DOMDisplay {
  constructor(parent, level) {
    this.dom = elt("div", {class: "game"}, drawGrid(level));
    this.actorLayer = null;
    parent.appendChild(this.dom);
  }

  clear() { this.dom.remove(); }
}
```

The level's background grid, which never changes, is drawn once. Actors are redrawn every time the display is updated with a given state. The `actorLayer` property will be used to track the element that holds the actors so that they can be easily removed and replaced.

Our coordinates and sizes are tracked in grid units, where a size or distance of 1 means one grid block. When setting pixel sizes, we will have to scale these coordinates up—everything in the game would be ridiculously small at a single pixel per square. The `scale` constant gives the number of pixels that a single unit takes up on the screen.

```
const scale = 20;

function drawGrid(level) {
  return elt("table", {
    class: "background",
    style: `width: ${level.width * scale}px`
  }, ...level.rows.map(row =>
    elt("tr", {style: `height: ${scale}px`},
```



```

    ...row.map(type => elt("td", {class: type})))
  });
}

```

The `<table>` element's form nicely corresponds to the structure of the `rows` property of the level—each row of the grid is turned into a table row (`<tr>` element). The strings in the grid are used as class names for the table cell (`<td>`) elements. The code uses the spread (triple dot) operator to pass arrays of child nodes to `elt` as separate arguments.

The following CSS makes the table look like the background we want:

```

.background { background: rgb(52, 166, 251);
               table-layout: fixed;
               border-spacing: 0;
               }
.background td { padding: 0;
                 }
.lava { background: rgb(255, 100, 100); }
.wall { background: white;
        }

```

Some of these (`table-layout`, `border-spacing`, and `padding`) are used to suppress unwanted default behavior. We don't want the layout of the table to depend upon the contents of its cells, and we don't want space between the table cells or padding inside them.

The `background` rule sets the background color. CSS allows colors to be specified both as words (`white`) or with a format such as `rgb(R, G, B)`, where the red, green, and blue components of the color are separated into three numbers from 0 to 255. In `rgb(52, 166, 251)`, the red component is 52, green is 166, and blue is 251. Since the blue component is the largest, the resulting color will be bluish. In the `.lava` rule, the first number (red) is the largest.

We draw each actor by creating a DOM element for it and setting that element's position and size based on the actor's properties. The values must be multiplied by `scale` to go from game units to pixels.

```

function drawActors(actors) {
  return elt("div", {}, ...actors.map(actor => {
    let rect = elt("div", {class: 'actor ${actor.type}'});
    rect.style.width = `${actor.size.x * scale}px`;
    rect.style.height = `${actor.size.y * scale}px`;
    rect.style.left = `${actor.pos.x * scale}px`;
    rect.style.top = `${actor.pos.y * scale}px`;
    return rect;
  }));
}

```

To give an element more than one class, we separate the class names by spaces. In the following CSS code, the `actor` class gives the actors their absolute position. Their type name is used as an extra class to give them a color. We don't have to define the `lava` class again because we're reusing the class for the lava grid squares we defined earlier.

```

.actor { position: absolute;
          }
.coin { background: rgb(241, 229, 89); }
.player { background: rgb(64, 64, 64);
          }

```

The `syncState` method is used to make the display show a given state. It first removes the old actor graphics, if any, and then redraws the actors in their new positions. It may be tempting to try to reuse the DOM elements for actors, but to make that work, we would need a lot of additional bookkeeping to associate actors with DOM elements and

to make sure we remove elements when their actors vanish. Since there will typically be only a handful of actors in the game, redrawing all of them is not expensive.

```
DOMDisplay.prototype.syncState = function(state) {
  if (this.actorLayer) this.actorLayer.remove();
  this.actorLayer = drawActors(state.actors);
  this.dom.appendChild(this.actorLayer);
  this.dom.className = 'game ${state.status}';
  this.scrollPlayerIntoView(state);
};
```

By adding the level's current status as a class name to the wrapper, we can style the player actor slightly differently when the game is won or lost by adding a CSS rule that takes effect only when the player has an ancestor element with a given class.

```
.lost .player {
  background: rgb(160, 64, 64);
}
.won .player {
  box-shadow: -4px -7px 8px white, 4px -7px 8px white;
}
```

After touching lava, the player turns dark red, suggesting scorching. When the last coin has been collected, we add two blurred white shadows—one to the upper left and one to the upper right—to create a white halo effect.

We can't assume that the level always fits in the *viewport*, the element into which we draw the game. That is why we need the `scrollPlayerIntoView`

call: it ensures that if the level is protruding outside the viewport, we scroll that viewport to make sure the player is near its center. The following CSS gives the game's wrapping DOM element a maximum size and ensures that anything that sticks out of the element's box is not visible. We also give it a relative position so that the actors inside it are positioned relative to the level's upper-left corner.

```
.game {
  overflow: hidden;
  max-width: 600px;
  max-height: 450px;
  position: relative;
}
```

In the `scrollPlayerIntoView` method, we find the player's position and update the wrapping element's scroll position. We change the scroll position by manipulating that element's `scrollLeft` and `scrollTop` properties when the player is too close to the edge.

```
DOMDisplay.prototype.scrollPlayerIntoView = function(state) {
  let width = this.dom.clientWidth;
  let height = this.dom.clientHeight;
  let margin = width / 3;

  // The viewport
  let left = this.dom.scrollLeft, right = left + width;
  let top = this.dom.scrollTop, bottom = top + height;

  let player = state.player;
  let center = player.pos.plus(player.size.times(0.5))
```

```

        .times(scale);

    if (center.x < left + margin) {
        this.dom.scrollLeft = center.x - margin;
    } else if (center.x > right - margin) {
        this.dom.scrollLeft = center.x + margin - width;
    }
    if (center.y < top + margin) {
        this.dom.scrollTop = center.y - margin;
    } else if (center.y > bottom - margin) {
        this.dom.scrollTop = center.y + margin - height;
    }
}
};

```

The way the player's center is found shows how the methods on our `Vec` type allow computations with objects to be written in a relatively readable way. To find the actor's center, we add its position (its upper-left corner) and half its size. That is the center in level coordinates, but we need it in pixel coordinates, so we then multiply the resulting vector by our display scale.

Next, a series of checks verifies that the player position isn't outside of the allowed range. Note that sometimes this will set nonsense scroll coordinates that are below zero or beyond the element's scrollable area. This is okay—the DOM will constrain them to acceptable values. Setting `scrollLeft` to `-10` will cause it to become `0`.

While it would have been slightly simpler to always try to scroll the player to the center of the viewport, this creates a rather jarring effect. As you are jumping, the view will constantly shift up and down. It's

more pleasant to have a “neutral” area in the middle of the screen where you can move around without causing any scrolling.

We are now able to display our tiny level.

```

<link rel="stylesheet" href="css/game.css">

<script>
    let simpleLevel = new Level(simpleLevelPlan);
    let display = new DOMDisplay(document.body, simpleLevel);
    display.syncState(State.start(simpleLevel));
</script>

```



The `<link>` tag, when used with `rel="stylesheet"`, is a way to load a CSS file into a page. The file `game.css` contains the styles necessary for our game.

MOTION AND COLLISION

Now we're at the point where we can start adding motion. The basic approach taken by most games like this is to split time into small steps and, for each step, move the actors by a distance corresponding to their

speed multiplied by the size of the time step. We'll measure time in seconds, so speeds are expressed in units per second.

Moving things is easy. The difficult part is dealing with the interactions between the elements. When the player hits a wall or floor, they should not simply move through it. The game must notice when a given motion causes an object to hit another object and respond accordingly. For walls, the motion must be stopped. When hitting a coin, that coin must be collected. When touching lava, the game should be lost.

Solving this for the general case is a major task. You can find libraries, usually called *physics engines*, that simulate interaction between physical objects in two or three dimensions. We'll take a more modest approach in this chapter, handling only collisions between rectangular objects and handling them in a rather simplistic way.

Before moving the player or a block of lava, we test whether the motion would take it inside of a wall. If it does, we simply cancel the motion altogether. The response to such a collision depends on the type of actor—the player will stop, whereas a lava block will bounce back.

This approach requires our time steps to be rather small, since it will cause motion to stop before the objects actually touch. If the time steps (and thus the motion steps) are too big, the player would end up hovering a noticeable distance above the ground. Another approach, arguably better but more complicated, would be to find the exact collision spot and move there. We will take the simple approach and hide its problems by ensuring the animation proceeds in small steps.

This method tells us whether a rectangle (specified by a position and a size) touches a grid element of the given type.

```
Level.prototype.touches = function(pos, size, type) {
  let xStart = Math.floor(pos.x);
  let xEnd = Math.ceil(pos.x + size.x);
  let yStart = Math.floor(pos.y);
  let yEnd = Math.ceil(pos.y + size.y);

  for (let y = yStart; y < yEnd; y++) {
    for (let x = xStart; x < xEnd; x++) {
      let isOutside = x < 0 || x >= this.width ||
                     y < 0 || y >= this.height;
      let here = isOutside ? "wall" : this.rows[y][x];
      if (here == type) return true;
    }
  }
  return false;
};
```

The method computes the set of grid squares that the body overlaps with by using `Math.floor` and `Math.ceil` on its coordinates. Remember that grid squares are 1 by 1 units in size. By rounding the sides of a box up and down, we get the range of background squares that the box touches.



We loop over the block of grid squares found by rounding the coordi-

nates and return true when a matching square is found. Squares outside of the level are always treated as "wall" to ensure that the player can't leave the world and that we won't accidentally try to read outside of the bounds of our `rows` array.

The state `update` method uses `touches` to figure out whether the player is touching lava.

```
State.prototype.update = function(time, keys) {
  let actors = this.actors
  .map(actor => actor.update(time, this, keys));
  let newState = new State(this.level, actors, this.status);

  if (newState.status !== "playing") return newState;

  let player = newState.player;
  if (this.level.touches(player.pos, player.size, "lava")) {
    return new State(this.level, actors, "lost");
  }

  for (let actor of actors) {
    if (actor !== player && overlap(actor, player)) {
      newState = actor.collide(newState);
    }
  }
  return newState;
};
```

The method is passed a time step and a data structure that tells it which keys are being held down. The first thing it does is call the `update` method on all actors, producing an array of updated actors.

The actors also get the time step, the keys, and the state so that they can base their update on those. Only the player will actually read keys, since that's the only actor that's controlled by the keyboard.

If the game is already over, no further processing has to be done (the game can't be won after being lost, or vice versa). Otherwise, the method tests whether the player is touching background lava. If so, the game is lost and we're done. Finally, if the game really is still going on, it sees whether any other actors overlap the player.

Overlap between actors is detected with the `overlap` function. It takes two actor objects and returns true when they touch—which is the case when they overlap both along the x-axis and along the y-axis.

```
function overlap(actor1, actor2) {
  return actor1.pos.x + actor1.size.x > actor2.pos.x &&
    actor1.pos.x < actor2.pos.x + actor2.size.x &&
    actor1.pos.y + actor1.size.y > actor2.pos.y &&
    actor1.pos.y < actor2.pos.y + actor2.size.y;
}
```

If any actor does overlap, its `collide` method gets a chance to update the state. Touching a lava actor sets the game status to "lost". Coins vanish when you touch them and set the status to "won" when they are the last coin of the level.

```
Lava.prototype.collide = function(state) {
  return new State(state.level, state.actors, "lost");
};
```

```
Coin.prototype.collide = function(state) {
```

```

    let filtered = state.actors.filter(a => a !== this);
    let status = state.status;
    if (!filtered.some(a => a.type == "coin")) status = "won";
    return new State(state.level, filtered, status);
};

```

ACTOR UPDATES

Actor objects' update methods take as arguments the time step, the state object, and a keys object. The one for the Lava actor type ignores the keys object.

```

Lava.prototype.update = function(time, state) {
    let newPos = this.pos.plus(this.speed.times(time));
    if (!state.level.touches(newPos, this.size, "wall")) {
        return new Lava(newPos, this.speed, this.reset);
    } else if (this.reset) {
        return new Lava(this.reset, this.speed, this.reset);
    } else {
        return new Lava(this.pos, this.speed.times(-1));
    }
};

```

This `update` method computes a new position by adding the product of the time step and the current speed to its old position. If no obstacle blocks that new position, it moves there. If there is an obstacle, the behavior depends on the type of the lava block—dripping lava has a `reset` position, to which it jumps back when it hits something. Bouncing

lava inverts its speed by multiplying it by `-1` so that it starts moving in the opposite direction.

Coins use their `update` method to wobble. They ignore collisions with the grid, since they are simply wobbling around inside of their own square.

```

const wobbleSpeed = 8, wobbleDist = 0.07;

Coin.prototype.update = function(time) {
    let wobble = this.wobble + time * wobbleSpeed;
    let wobblePos = Math.sin(wobble) * wobbleDist;
    return new Coin(this.basePos.plus(new Vec(0, wobblePos)),
        this.basePos, wobble);
};

```

The `wobble` property is incremented to track time and then used as an argument to `Math.sin` to find the new position on the wave. The coin's current position is then computed from its base position and an offset based on this wave.

That leaves the player itself. Player motion is handled separately per axis because hitting the floor should not prevent horizontal motion, and hitting a wall should not stop falling or jumping motion.

```

const playerXSpeed = 7;
const gravity = 30;
const jumpSpeed = 17;

Player.prototype.update = function(time, state, keys) {
    let xSpeed = 0;
    if (keys.ArrowLeft) xSpeed -= playerXSpeed;

```

```

if (keys.ArrowRight) xSpeed += playerXSpeed;
let pos = this.pos;
let movedX = pos.plus(new Vec(xSpeed * time, 0));
if (!state.level.touches(movedX, this.size, "wall")) {
  pos = movedX;
}

let ySpeed = this.speed.y + time * gravity;
let movedY = pos.plus(new Vec(0, ySpeed * time));
if (!state.level.touches(movedY, this.size, "wall")) {
  pos = movedY;
} else if (keys.ArrowUp && ySpeed > 0) {
  ySpeed = -jumpSpeed;
} else {
  ySpeed = 0;
}
return new Player(pos, new Vec(xSpeed, ySpeed));
};

```

The horizontal motion is computed based on the state of the left and right arrow keys. When there's no wall blocking the new position created by this motion, it is used. Otherwise, the old position is kept.

Vertical motion works in a similar way but has to simulate jumping and gravity. The player's vertical speed (*ySpeed*) is first accelerated to account for gravity.

We check for walls again. If we don't hit any, the new position is used. If there *is* a wall, there are two possible outcomes. When the up arrow is pressed *and* we are moving down (meaning the thing we hit is below us), the speed is set to a relatively large, negative value. This

causes the player to jump. If that is not the case, the player simply bumped into something, and the speed is set to zero.

The gravity strength, jumping speed, and other constants in the game were determined by simply trying out some numbers and seeing which ones felt right. You can try experimenting with them.

TRACKING KEYS

For a game like this, we do not want keys to take effect once per key-press. Rather, we want their effect (moving the player figure) to stay active as long as they are held.

We need to set up a key handler that stores the current state of the left, right, and up arrow keys. We will also want to call `preventDefault` for those keys so that they don't end up scrolling the page.

The following function, when given an array of key names, will return an object that tracks the current position of those keys. It registers event handlers for "keydown" and "keyup" events and, when the key code in the event is present in the set of codes that it is tracking, updates the object.

```

function trackKeys(keys) {
  let down = Object.create(null);
  function track(event) {
    if (keys.includes(event.key)) {
      down[event.key] = event.type == "keydown";
      event.preventDefault();
    }
  }
}

```

```

window.addEventListener("keydown", track);
window.addEventListener("keyup", track);
return down;
}

const arrowKeys =
  trackKeys(["ArrowLeft", "ArrowRight", "ArrowUp"]);

```

The same handler function is used for both event types. It looks at the event object's `type` property to determine whether the key state should be updated to `true` ("keydown") or `false` ("keyup").

RUNNING THE GAME

The `requestAnimationFrame` function, which we saw in Chapter 14, provides a good way to animate a game. But its interface is quite primitive—using it requires us to track the time at which our function was called the last time around and call `requestAnimationFrame` again after every frame.

Let's define a helper function that wraps all that in a convenient interface and allows us to simply call `runAnimation`, giving it a function that expects a time difference as an argument and draws a single frame. When the frame function returns the value `false`, the animation stops.

```

function runAnimation(frameFunc) {
  let lastTime = null;
  function frame(time) {
    if (lastTime != null) {

```

```

    let timeStep = Math.min(time - lastTime, 100) / 1000;
    if (frameFunc(timeStep) === false) return;
  }
  lastTime = time;
  requestAnimationFrame(frame);
}
requestAnimationFrame(frame);
}

```

I have set a maximum frame step of 100 milliseconds (one-tenth of a second). When the browser tab or window with our page is hidden, `requestAnimationFrame` calls will be suspended until the tab or window is shown again. In this case, the difference between `lastTime` and `time` will be the entire time in which the page was hidden. Advancing the game by that much in a single step would look silly and might cause weird side effects, such as the player falling through the floor.

The function also converts the time steps to seconds, which are an easier quantity to think about than milliseconds.

The `runLevel` function takes a `Level` object and a display constructor and returns a promise. It displays the level (in `document.body`) and lets the user play through it. When the level is finished (lost or won), `runLevel` waits one more second (to let the user see what happens) and then clears the display, stops the animation, and resolves the promise to the game's end status.

```

function runLevel(level, Display) {
  let display = new Display(document.body, level);
  let state = State.start(level);
  let ending = 1;

```



```

return new Promise(resolve => {
  runAnimation(time => {
    state = state.update(time, arrowKeys);
    display.syncState(state);
    if (state.status == "playing") {
      return true;
    } else if (ending > 0) {
      ending -= time;
      return true;
    } else {
      display.clear();
      resolve(state.status);
      return false;
    }
  });
});
}

```

A game is a sequence of levels. Whenever the player dies, the current level is restarted. When a level is completed, we move on to the next level. This can be expressed by the following function, which takes an array of level plans (strings) and a display constructor:

```

async function runGame(plans, Display) {
  for (let level = 0; level < plans.length; ) {
    let status = await runLevel(new Level(plans[level]),
      Display);
    if (status == "won") level++;
  }
  console.log("You've won!");
}

```

```

}

```

Because we made `runLevel` return a promise, `runGame` can be written using an `async` function, as shown in Chapter 11. It returns another promise, which resolves when the player finishes the game.

There is a set of level plans available in the `GAME_LEVELS` binding in this chapter's sandbox (<https://eloquentjavascript.net/code#16>). This page feeds them to `runGame`, starting an actual game.

```

<link rel="stylesheet" href="css/game.css">

<body>
  <script>
    runGame(GAME_LEVELS, DOMDisplay);
  </script>
</body>

```

EXERCISES

GAME OVER

It's traditional for platform games to have the player start with a limited number of *lives* and subtract one life each time they die. When the player is out of lives, the game restarts from the beginning.

Adjust `runGame` to implement lives. Have the player start with three. Output the current number of lives (using `console.log`) every time a level starts.

PAUSING THE GAME

Make it possible to pause (suspend) and unpause the game by pressing ESC. You can do this by changing the `runLevel` function to set up a keyboard event handler that interrupts or resumes the animation whenever ESC is hit.

The `runAnimation` interface may not look like it is suitable for this at first glance, but it is if you rearrange the way `runLevel` calls it.

When you have that working, there's something else you can try. The way we've been registering keyboard event handlers is somewhat problematic. The `arrowKeys` object is currently a global binding, and its event handlers are kept around even when no game is running. You could say they *leak* out of our system. Extend `trackKeys` to provide a way to unregister its handlers, then change `runLevel` to register its handlers when it starts and unregister them again when it is finished.

A MONSTER

It is traditional for platform games to have enemies that you can defeat by jumping on top of them. This exercise asks you to add such an actor type to the game.

We'll call this actor a monster. Monsters move only horizontally. You can make them move in the direction of the player, bounce back and forth like horizontal lava, or have any other movement pattern you want. The class doesn't have to handle falling, but it should make sure the monster doesn't walk through walls.

When a monster touches the player, the effect depends on whether

the player is jumping on top of them or not. You can approximate this by checking whether the player's bottom is near the monster's top. If this is the case, the monster disappears. If not, the game is lost.

“Drawing is deception.”

—M.C. Escher, cited by Bruno Ernst in *The Magic Mirror* of
M.C. Escher

CHAPTER 17

DRAWING ON CANVAS

Browsers give us several ways to display graphics. The simplest way is to use styles to position and color regular DOM elements. This can get us quite far, as the game in the previous chapter showed. By adding partially transparent background images to the nodes, we can make them look exactly the way we want. It is even possible to rotate or skew nodes with the `transform` style.

But we’d be using the DOM for something that it wasn’t originally designed for. Some tasks, such as drawing a line between arbitrary points, are extremely awkward to do with regular HTML elements.

There are two alternatives. The first is DOM based but utilizes *Scalable Vector Graphics* (SVG) rather than HTML. Think of SVG as a document-markup dialect that focuses on shapes rather than text. You can embed an SVG document directly in an HTML document or include it with an `<img>` tag.

The second alternative is called a *canvas*. A canvas is a single DOM element that encapsulates a picture. It provides a programming interface for drawing shapes onto the space taken up by the node. The

main difference between a canvas and an SVG picture is that in SVG the original description of the shapes is preserved so that they can be moved or resized at any time. A canvas, on the other hand, converts the shapes to pixels (colored dots on a raster) as soon as they are drawn and does not remember what these pixels represent. The only way to move a shape on a canvas is to clear the canvas (or the part of the canvas around the shape) and redraw it with the shape in a new position.

SVG

This book won’t go into SVG in detail, but I’ll briefly explain how it works. At the end of the chapter, I’ll come back to the trade-offs that you must consider when deciding which drawing mechanism is appropriate for a given application.

This is an HTML document with a simple SVG picture in it:

```
<p>Normal HTML here.</p>
<svg xmlns="http://www.w3.org/2000/svg">
  <circle r="50" cx="50" cy="50" fill="red"/>
  <rect x="120" y="5" width="90" height="90"
        stroke="blue" fill="none"/>
</svg>
```

The `xmlns` attribute changes an element (and its children) to a different *XML namespace*. This namespace, identified by a URL, specifies the dialect that we are currently speaking. The `<circle>` and `<rect>` tags, which do not exist in HTML, do have a meaning in SVG—they draw shapes using the style and position specified by their attributes.

The document is displayed like this:

Normal HTML here.



These tags create DOM elements, just like HTML tags, that scripts can interact with. For example, this changes the `<circle>` element to be colored cyan instead:

```
let circle = document.querySelector("circle");
circle.setAttribute("fill", "cyan");
```

THE CANVAS ELEMENT

Canvas graphics can be drawn onto a `<canvas>` element. You can give such an element `width` and `height` attributes to determine its size in pixels.

A new canvas is empty, meaning it is entirely transparent and thus shows up as empty space in the document.

The `<canvas>` tag is intended to allow different styles of drawing. To get access to an actual drawing interface, we first need to create a *context*, an object whose methods provide the drawing interface. There are currently three widely supported drawing styles: "2d" for two-dimensional graphics, "webgl" for three-dimensional graphics through

the OpenGL interface, and "webgpu", a more modern and flexible alternative to WebGL.

This book won't discuss WebGL or WebGPU—we'll stick to two dimensions. But if you are interested in three-dimensional graphics, I do encourage you to look into WebGPU. It provides a direct interface to graphics hardware and allows you to render even complicated scenes efficiently, using JavaScript.

You create a context with the `getContext` method on the `<canvas>` DOM element.

```
<p>Before canvas.</p>
<canvas width="120" height="60"></canvas>
<p>After canvas.</p>
<script>
  let canvas = document.querySelector("canvas");
  let context = canvas.getContext("2d");
  context.fillStyle = "red";
  context.fillRect(10, 10, 100, 50);
</script>
```

After creating the context object, the example draws a red rectangle that is 100 pixels wide and 50 pixels high, with its upper-left corner at coordinates (10, 10).

Before canvas.



After canvas.

Just like in HTML (and SVG), the coordinate system that the canvas uses puts (0, 0) at the upper-left corner, and the positive y-axis goes down from there. This means (10, 10) is 10 pixels below and to the right of the upper-left corner.

LINES AND SURFACES

In the canvas interface, a shape can be *filled*, meaning its area is given a certain color or pattern, or it can be *stroked*, which means a line is drawn along its edge. SVG uses the same terminology.

The `fillRect` method fills a rectangle. It takes first the x- and y-coordinates of the rectangle's upper-left corner, then its width, and then its height. A similar method called `strokeRect` draws the outline of a rectangle.

Neither method takes any further parameters. The color of the fill, thickness of the stroke, and so on, are not determined by an argument to the method, as you might reasonably expect, but rather by properties of the context object.

The `fillStyle` property controls the way shapes are filled. It can be set to a string that specifies a color, using the color notation used by

CSS.

The `strokeStyle` property works similarly but determines the color used for a stroked line. The width of that line is determined by the `lineWidth` property, which may contain any positive number.

```
<canvas></canvas>
<script>
  let cx = document.querySelector("canvas").getContext("2d");
  cx.strokeStyle = "blue";
  cx.strokeRect(5, 5, 50, 50);
  cx.lineWidth = 5;
  cx.strokeRect(135, 5, 50, 50);
</script>
```

This code draws two blue squares, using a thicker line for the second one.



When no `width` or `height` attribute is specified, as in the example, a canvas element gets a default width of 300 pixels and height of 150 pixels.

PATHS

A path is a sequence of lines. The 2D canvas interface takes a peculiar approach to describing such a path. It is done entirely through side

effects. Paths are not values that can be stored and passed around. Instead, if you want to do something with a path, you make a sequence of method calls to describe its shape.

```
<canvas></canvas>
<script>
  let cx = document.querySelector("canvas").getContext("2d");
  cx.beginPath();
  for (let y = 10; y < 100; y += 10) {
    cx.moveTo(10, y);
    cx.lineTo(90, y);
  }
  cx.stroke();
</script>
```

This example creates a path with a number of horizontal line segments and then strokes it using the `stroke` method. Each segment created with `lineTo` starts at the path's *current* position. That position is usually the end of the last segment, unless `moveTo` was called. In that case, the next segment would start at the position passed to `moveTo`.

The path described by the previous program looks like this:



When filling a path (using the `fill` method), each shape is filled separately. A path can contain multiple shapes—each `moveTo` motion

starts a new one. But the path needs to be *closed* (meaning its start and end are in the same position) before it can be filled. If the path is not already closed, a line is added from its end to its start, and the shape enclosed by the completed path is filled.

```
<canvas></canvas>
<script>
  let cx = document.querySelector("canvas").getContext("2d");
  cx.beginPath();
  cx.moveTo(50, 10);
  cx.lineTo(10, 70);
  cx.lineTo(90, 70);
  cx.fill();
</script>
```

This example draws a filled triangle. Note that only two of the triangle's sides are explicitly drawn. The third, from the lower-right corner back to the top, is implied and wouldn't be there if you stroked the path.



You could also use the `closePath` method to explicitly close a path by adding an actual line segment back to the path's start. This segment *is* drawn when stroking the path.

CURVES

A path may also contain curved lines. These are unfortunately a bit more involved to draw.

The `quadraticCurveTo` method draws a curve to a given point. To determine the curvature of the line, the method is given a control point as well as a destination point. Imagine this control point as *attracting* the line, giving it its curve. The line won't go through the control point, but its direction at the start and end points will be such that a straight line in that direction would point toward the control point. The following example illustrates this:

```
<canvas></canvas>
<script>
  let cx = document.querySelector("canvas").getContext("2d");
  cx.beginPath();
  cx.moveTo(10, 90);
  // control=(50, 10) goal=(90, 90)
  cx.quadraticCurveTo(50, 10, 90, 90);
  cx.lineTo(60, 10);
  cx.closePath();
  cx.stroke();
</script>
```

It produces a path that looks like this:



We draw a quadratic curve from the left to the right, with (60, 10) as the control point, and then draw two line segments going through that control point and back to the start of the line. The result somewhat resembles a *Star Trek* insignia. You can see the effect of the control point: the lines leaving the lower corners start off in the direction of the control point and then curve toward their target.

The `bezierCurveTo` method draws a similar kind of curve. Instead of a single control point, this method has two—one for each of the line's end points. Here is a similar sketch to illustrate the behavior of such a curve:

```
<canvas></canvas>
<script>
  let cx = document.querySelector("canvas").getContext("2d");
  cx.beginPath();
  cx.moveTo(10, 90);
  // control1=(10, 10) control2=(90, 10) goal=(50, 90)
  cx.bezierCurveTo(10, 10, 90, 10, 50, 90);
  cx.lineTo(90, 10);
  cx.lineTo(10, 10);
  cx.closePath();
  cx.stroke();
</script>
```

The two control points specify the direction at both ends of the curve. The farther they are away from their corresponding point, the more the curve will “bulge” in that direction.



Such curves can be hard to work with—it’s not always clear how to find the control points that provide the shape you are looking for. Sometimes you can compute them, and sometimes you’ll just have to find a suitable value by trial and error.

The `arc` method is a way to draw a line that curves along the edge of a circle. It takes a pair of coordinates for the arc’s center, a radius, and then a start angle and end angle.

Those last two parameters make it possible to draw only part of the circle. The angles are measured in radians, not degrees. This means a full circle has an angle of 2π , or $2 * \text{Math.PI}$, which is about 6.28. The angle starts counting at the point to the right of the circle’s center and goes clockwise from there. You can use a start of 0 and an end bigger than 2π (say, 7) to draw a full circle.

```
<canvas></canvas>
<script>
  let cx = document.querySelector("canvas").getContext("2d");
  cx.beginPath();
  // center=(50, 50) radius=40 angle=0 to 7
```

```
cx.arc(50, 50, 40, 0, 7);
// center=(150, 50) radius=40 angle=0 to π/2
cx.arc(150, 50, 40, 0, 0.5 * Math.PI);
cx.stroke();
</script>
```

The resulting picture contains a line from the right of the full circle (first call to `arc`) to the right of the quarter-circle (second call).



Like other path-drawing methods, a line drawn with `arc` is connected to the previous path segment. You can call `moveTo` or start a new path to avoid this.

DRAWING A PIE CHART

Imagine you’ve just taken a job at EconomiCorp, Inc. Your first assignment is to draw a pie chart of its customer satisfaction survey results.

The `results` binding contains an array of objects that represent the survey responses.

```
const results = [
  {name: "Satisfied", count: 1043, color: "lightblue"},
  {name: "Neutral", count: 563, color: "lightgreen"},
```



```

    {name: "Unsatisfied", count: 510, color: "pink"},
    {name: "No comment", count: 175, color: "silver"}
  ];

```

To draw a pie chart, we draw a number of pie slices, each made up of an arc and a pair of lines to the center of that arc. We can compute the angle taken up by each arc by dividing a full circle (2π) by the total number of responses and then multiplying that number (the angle per response) by the number of people who picked a given choice.

```

<canvas width="200" height="200"></canvas>
<script>
  let cx = document.querySelector("canvas").getContext("2d");
  let total = results
    .reduce((sum, {count}) => sum + count, 0);
  // Start at the top
  let currentAngle = -0.5 * Math.PI;
  for (let result of results) {
    let sliceAngle = (result.count / total) * 2 * Math.PI;
    cx.beginPath();
    // center=100,100, radius=100
    // from current angle, clockwise by slice's angle
    cx.arc(100, 100, 100,
      currentAngle, currentAngle + sliceAngle);
    currentAngle += sliceAngle;
    cx.lineTo(100, 100);
    cx.fillStyle = result.color;
    cx.fill();
  }
</script>

```

436

This draws the following chart:



But a chart that doesn't tell us what the slices mean isn't very helpful. We need a way to draw text to the canvas.

TEXT

A 2D canvas drawing context provides the methods `fillText` and `strokeText`. The latter can be useful for outlining letters, but usually `fillText` is what you need. It will fill the outline of the given text with the current `fillStyle`.

```

<canvas></canvas>
<script>
  let cx = document.querySelector("canvas").getContext("2d");
  cx.font = "28px Georgia";
  cx.fillStyle = "fuchsia";
  cx.fillText("I can draw text, too!", 10, 50);

```

437

```
</script>
```

You can specify the size, style, and font of the text with the `font` property. This example just gives a font size and family name. It is also possible to add *italic* or **bold** to the start of the string to select a style.

The last two arguments to `fillText` and `strokeText` provide the position at which the font is drawn. By default, they indicate the position of the start of the text's alphabetic baseline, which is the line that letters "stand" on, not counting hanging parts in letters such as *j* or *p*. You can change the horizontal position by setting the `textAlign` property to "end" or "center" and the vertical position by setting `textBaseline` to "top", "middle", or "bottom".

We'll come back to our pie chart, and the problem of labeling the slices, in the [exercises](#) at the end of the chapter.

IMAGES

In computer graphics, a distinction is often made between *vector* graphics and *bitmap* graphics. The first is what we have been doing so far in this chapter—specifying a picture by giving a logical description of shapes. Bitmap graphics, on the other hand, don't specify actual shapes but rather work with pixel data (rasters of colored dots).

The `drawImage` method allows us to draw pixel data onto a canvas. This pixel data can originate from an `<img>` element or from another canvas. The following example creates a detached `<img>` element and loads an image file into it. But the method cannot immediately start drawing from this picture because the browser may not have loaded it

yet. To deal with this, we register a "load" event handler and do the drawing after the image has loaded.

```
<canvas></canvas>
<script>
  let cx = document.querySelector("canvas").getContext("2d");
  let img = document.createElement("img");
  img.src = "img/hat.png";
  img.addEventListener("load", () => {
    for (let x = 10; x < 200; x += 30) {
      cx.drawImage(img, x, 10);
    }
  });
</script>
```

By default, `drawImage` will draw the image at its original size. You can also give it two additional arguments to specify the width and height of the drawn image, when those aren't the same as the origin image.

When `drawImage` is given *nine* arguments, it can be used to draw only a fragment of an image. The second through fifth arguments indicate the rectangle (x, y, width, and height) in the source image that should be copied, and the sixth to ninth arguments give the rectangle (on the canvas) into which it should be copied.

This can be used to pack multiple *sprites* (image elements) into a single image file and then draw only the part you need. For example, this picture contains a game character in multiple poses:



By alternating which pose we draw, we can show an animation that looks like a walking character.

To animate a picture on a canvas, the `clearRect` method is useful. It resembles `fillRect`, but instead of coloring the rectangle, it makes it transparent, removing the previously drawn pixels.

We know that each *sprite*, each subpicture, is 24 pixels wide and 30 pixels high. The following code loads the image and then sets up an interval (repeated timer) to draw the next frame:

```
<canvas></canvas>
<script>
  let cx = document.querySelector("canvas").getContext("2d");
  let img = document.createElement("img");
  img.src = "img/player.png";
  let spriteW = 24, spriteH = 30;
  img.addEventListener("load", () => {
    let cycle = 0;
    setInterval(() => {
      cx.clearRect(0, 0, spriteW, spriteH);
      cx.drawImage(img,
        // source rectangle
        cycle * spriteW, 0, spriteW, spriteH,
        // destination rectangle
        0, 0, spriteW, spriteH);
      cycle = (cycle + 1) % 8;
    }, 120);
  });
</script>
```

The `cycle` binding tracks our position in the animation. For each frame,

it is incremented and then clipped back to the 0 to 7 range by using the remainder operator. This binding is then used to compute the x-coordinate that the sprite for the current pose has in the picture.

TRANSFORMATION

What if we want our character to walk to the left instead of to the right? We could draw another set of sprites, of course. But we could also instruct the canvas to draw the picture the other way round.

Calling the `scale` method will cause anything drawn after it to be scaled. This method takes two parameters, one to set a horizontal scale and one to set a vertical scale.

```
<canvas></canvas>
<script>
  let cx = document.querySelector("canvas").getContext("2d");
  cx.scale(3, .5);
  cx.beginPath();
  cx.arc(50, 50, 40, 0, 7);
  cx.lineWidth = 3;
  cx.stroke();
</script>
```

Because of the call to `scale`, the circle is drawn three times as wide and half as high.

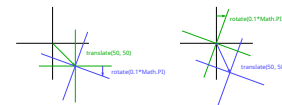


Scaling will cause everything about the drawn image, including the line width, to be stretched out or squeezed together as specified. Scaling by a negative amount will flip the picture around. The flipping happens around point (0, 0), which means it will also flip the direction of the coordinate system. When a horizontal scaling of -1 is applied, a shape drawn at x position 100 will end up at what used to be position -100.

To turn a picture around, we can't simply add `cx.scale(-1, 1)` before the call to `drawImage`. That would move our picture outside of the canvas, where it won't be visible. We could adjust the coordinates given to `drawImage` to compensate for this by drawing the image at x position -50 instead of 0. Another solution, which doesn't require the code doing the drawing to know about the scale change, is to adjust the axis around which the scaling happens.

There are several other methods besides `scale` that influence the coordinate system for a canvas. You can rotate subsequently drawn shapes with the `rotate` method and move them with the `translate` method. The interesting—and confusing—thing is that these transformations *stack*, meaning that each one happens relative to the previous transformations.

If we translate by 10 horizontal pixels twice, everything will be drawn 20 pixels to the right. If we first move the center of the coordinate system to (50, 50) and then rotate by 20 degrees (about 0.1π radians), that rotation will happen *around* point (50, 50).

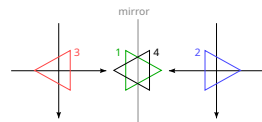


But if we *first* rotate by 20 degrees and *then* translate by (50, 50), the translation will happen in the rotated coordinate system and thus produce a different orientation. The order in which transformations are applied matters.

To flip a picture around the vertical line at a given x position, we can do the following:

```
function flipHorizontally(context, around) {
  context.translate(around, 0);
  context.scale(-1, 1);
  context.translate(-around, 0);
}
```

We move the y -axis to where we want our mirror to be, apply the mirroring, and finally move the y -axis back to its proper place in the mirrored universe. The following picture explains why this works:



This shows the coordinate systems before and after mirroring across the central line. The triangles are numbered to illustrate each step. If we draw a triangle at a positive x position, it would, by default, be in the place where triangle 1 is. A call to `flipHorizontally` first does a translation to the right, which gets us to triangle 2. It then scales, flipping the triangle over to position 3. This is not where it should be, if it were mirrored in the given line. The second `translate` call fixes this—it “cancels” the initial translation and makes triangle 4 appear exactly where it should.

We can now draw a mirrored character at position (100, 0) by flipping the world around the character's vertical center.

```
<canvas></canvas>
<script>
let cx = document.querySelector("canvas").getContext("2d");
let img = document.createElement("img");
img.src = "img/player.png";
let spriteW = 24, spriteH = 30;
img.addEventListener("load", () => {
  flipHorizontally(cx, 100 + spriteW / 2);
});
</script>
```

```
cx.drawImage(img, 0, 0, spriteW, spriteH,
             100, 0, spriteW, spriteH);
});
</script>
```

STORING AND CLEARING TRANSFORMATIONS

Transformations stick around. Everything else we draw after drawing that mirrored character would also be mirrored. That might be inconvenient.

It is possible to save the current transformation, do some drawing and transforming, and then restore the old transformation. This is usually the proper thing to do for a function that needs to temporarily transform the coordinate system. First, we save whatever transformation the code that called the function was using. Then the function does its thing, adding more transformations on top of the current transformation. Finally, we revert to the transformation we started with.

The `save` and `restore` methods on the 2D canvas context do this transformation management. They conceptually keep a stack of transformation states. When you call `save`, the current state is pushed onto the stack, and when you call `restore`, the state on top of the stack is taken off and used as the context's current transformation. You can also call `resetTransform` to fully reset the transformation.

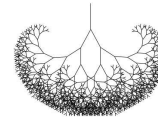
The branch function in the following example illustrates what you can do with a function that changes the transformation and then calls a function (in this case itself), which continues drawing with the given

transformation.

This function draws a treelike shape by drawing a line, moving the center of the coordinate system to the end of the line, and calling itself twice—first rotated to the left and then rotated to the right. Every call reduces the length of the branch drawn, and the recursion stops when the length drops below 8.

```
<canvas width="600" height="300"></canvas>
<script>
  let cx = document.querySelector("canvas").getContext("2d");
  function branch(length, angle, scale) {
    cx.fillRect(0, 0, 1, length);
    if (length < 8) return;
    cx.save();
    cx.translate(0, length);
    cx.rotate(-angle);
    branch(length * scale, angle, scale);
    cx.rotate(2 * angle);
    branch(length * scale, angle, scale);
    cx.restore();
  }
  cx.translate(300, 0);
  branch(60, 0.5, 0.8);
</script>
```

The result is a simple fractal.



If the calls to `save` and `restore` were not there, the second recursive call to `branch` would end up with the position and rotation created by the first call. It would be connected not to the current branch but rather to the innermost, rightmost branch drawn by the first call. The resulting shape might also be interesting, but it is definitely not a tree.

BACK TO THE GAME

We now know enough about canvas drawing to start working on a canvas-based display system for the game from the [previous chapter](#). The new display will no longer be showing just colored boxes. Instead, we'll use `drawImage` to draw pictures that represent the game's elements.

We define another display object type called `CanvasDisplay`, supporting the same interface as `DOMDisplay` from [Chapter 16](#)—namely, the methods `syncState` and `clear`.

This object keeps a little more information than `DOMDisplay`. Rather than using the scroll position of its DOM element, it tracks its own viewport, which tells us which part of the level we are currently looking at. Finally, it keeps a `flipPlayer` property so that even when the player

is standing still, it keeps facing the direction in which it last moved.

```
class CanvasDisplay {
  constructor(parent, level) {
    this.canvas = document.createElement("canvas");
    this.canvas.width = Math.min(600, level.width * scale);
    this.canvas.height = Math.min(450, level.height * scale);
    parent.appendChild(this.canvas);
    this.cx = this.canvas.getContext("2d");

    this.flipPlayer = false;

    this.viewport = {
      left: 0,
      top: 0,
      width: this.canvas.width / scale,
      height: this.canvas.height / scale
    };
  }

  clear() {
    this.canvas.remove();
  }
}
```

The `syncState` method first computes a new viewport and then draws the game scene at the appropriate position.

```
CanvasDisplay.prototype.syncState = function(state) {
  this.updateViewport(state);
  this.clearDisplay(state.status);
}
```

```
this.drawBackground(state.level);
this.drawActors(state.actors);
};
```

Contrary to `DOMDisplay`, this display style *does* have to redraw the background on every update. Because shapes on a canvas are just pixels, after we draw them there is no good way to move them (or remove them). The only way to update the canvas display is to clear it and redraw the scene. We may also have scrolled, which requires the background to be in a different position.

The `updateViewport` method is similar to `DOMDisplay`'s `scrollPlayerIntoView` method. It checks whether the player is too close to the edge of the screen and moves the viewport when this is the case.

```
CanvasDisplay.prototype.updateViewport = function(state) {
  let view = this.viewport, margin = view.width / 3;
  let player = state.player;
  let center = player.pos.plus(player.size.times(0.5));

  if (center.x < view.left + margin) {
    view.left = Math.max(center.x - margin, 0);
  } else if (center.x > view.left + view.width - margin) {
    view.left = Math.min(center.x + margin - view.width,
      state.level.width - view.width);
  }

  if (center.y < view.top + margin) {
    view.top = Math.max(center.y - margin, 0);
  } else if (center.y > view.top + view.height - margin) {
    view.top = Math.min(center.y + margin - view.height,
      state.level.height - view.height);
  }
}
```

```

    }
  };

```

The calls to `Math.max` and `Math.min` ensure that the viewport does not end up showing space outside of the level. `Math.max(x, 0)` makes sure the resulting number is not less than zero. `Math.min` similarly guarantees that a value stays below a given bound.

When clearing the display, we'll use a slightly different color depending on whether the game is won (brighter) or lost (darker).

```

CanvasDisplay.prototype.clearDisplay = function(status) {
  if (status == "won") {
    this.cx.fillStyle = "rgb(68, 191, 255)";
  } else if (status == "lost") {
    this.cx.fillStyle = "rgb(44, 136, 214)";
  } else {
    this.cx.fillStyle = "rgb(52, 166, 251)";
  }
  this.cx.fillRect(0, 0,
    this.canvas.width, this.canvas.height);
};

```

To draw the background, we run through the tiles that are visible in the current viewport, using the same trick used in the `touches` method from the previous chapter.

```

let otherSprites = document.createElement("img");
otherSprites.src = "img/sprites.png";

```

```

CanvasDisplay.prototype.drawBackground = function(level) {

```

```

let {left, top, width, height} = this.viewport;
let xStart = Math.floor(left);
let xEnd = Math.ceil(left + width);
let yStart = Math.floor(top);
let yEnd = Math.ceil(top + height);

for (let y = yStart; y < yEnd; y++) {
  for (let x = xStart; x < xEnd; x++) {
    let tile = level.rows[y][x];
    if (tile == "empty") continue;
    let screenX = (x - left) * scale;
    let screenY = (y - top) * scale;
    let tileX = tile == "lava" ? scale : 0;
    this.cx.drawImage(otherSprites,
      tileX, 0, scale, scale,
      screenX, screenY, scale, scale);
  }
};

```

Tiles that are not empty are drawn with `drawImage`. The `otherSprites` image contains the pictures used for elements other than the player. It contains, from left to right, the wall tile, the lava tile, and the sprite for a coin.



Background tiles are 20 by 20 pixels, since we'll use the same scale as in `DOMDisplay`. Thus, the offset for lava tiles is 20 (the value of the `scale` binding), and the offset for walls is 0.

We don't bother waiting for the sprite image to load. Calling `drawImage` with an image that hasn't been loaded yet will simply do nothing. Thus, we might fail to draw the game properly for the first few frames while the image is still loading, but that isn't a serious problem. Since we keep updating the screen, the correct scene will appear as soon as the loading finishes.

The walking character shown earlier will be used to represent the player. The code that draws it needs to pick the right sprite and direction based on the player's current motion. The first eight sprites contain a walking animation. When the player is moving along a floor, we cycle through them based on the current time. We want to switch frames every 60 milliseconds, so the time is divided by 60 first. When the player is standing still, we draw the ninth sprite. During jumps, which are recognized by the fact that the vertical speed is not zero, we use the tenth, rightmost sprite.

Because the sprites are slightly wider than the player object—24 instead of 16 pixels to allow some space for feet and arms—the method has to adjust the x-coordinate and width by a given amount (`playerXOverlap`).

```
let playerSprites = document.createElement("img");
playerSprites.src = "img/player.png";
const playerXOverlap = 4;
```

```
CanvasDisplay.prototype.drawPlayer = function(player, x, y,
                                              width, height){
    width += playerXOverlap * 2;
    x -= playerXOverlap;
```

```
if (player.speed.x != 0) {
    this.flipPlayer = player.speed.x < 0;
}

let tile = 8;
if (player.speed.y != 0) {
    tile = 9;
} else if (player.speed.x != 0) {
    tile = Math.floor(Date.now() / 60) % 8;
}

this.cx.save();
if (this.flipPlayer) {
    flipHorizontally(this.cx, x + width / 2);
}
let tileX = tile * width;
this.cx.drawImage(playerSprites, tileX, 0, width, height,
                  x, y, width, height);
this.cx.restore();
};
```

The `drawPlayer` method is called by `drawActors`, which is responsible for drawing all the actors in the game.

```
CanvasDisplay.prototype.drawActors = function(actors) {
    for (let actor of actors) {
        let width = actor.size.x * scale;
        let height = actor.size.y * scale;
        let x = (actor.pos.x - this.viewport.left) * scale;
        let y = (actor.pos.y - this.viewport.top) * scale;
        if (actor.type == "player") {
```

```

        this.drawPlayer(actor, x, y, width, height);
    } else {
        let tileX = (actor.type == "coin" ? 2 : 1) * scale;
        this.cx.drawImage(otherSprites,
                           tileX, 0, width, height,
                           x, y, width, height);
    }
};

```

When drawing something that is not the player, we look at its type to find the offset of the correct sprite. The lava tile is found at offset 20, and the coin sprite is found at 40 (two times `scale`).

We have to subtract the viewport's position when computing the actor's position, since (0, 0) on our canvas corresponds to the top left of the viewport, not the top left of the level. We could also have used `translate` for this. Either way works.

That concludes the new display system. The resulting game looks something like this:



CHOOSING A GRAPHICS INTERFACE

When you need to generate graphics in the browser, you can choose between plain HTML, SVG, and canvas. There is no single *best* approach that works in all situations. Each option has strengths and weaknesses.

Plain HTML has the advantage of being simple. It also integrates well with text. Both SVG and canvas allow you to draw text, but they won't help you position that text or wrap it when it takes up more than one line. In an HTML-based picture, it is much easier to include blocks of text.

SVG can be used to produce crisp graphics that look good at any zoom level. Unlike HTML, it is designed for drawing and is thus more suitable for that purpose.

Both SVG and HTML build up a data structure (the DOM) that represents your picture. This makes it possible to modify elements

after they are drawn. If you need to repeatedly change a small part of a big picture in response to what the user is doing or as part of an animation, doing it in a canvas can be needlessly expensive. The DOM also allows us to register mouse event handlers on every element in the picture (even on shapes drawn with SVG). You can't do that with canvas.

But canvas's pixel-oriented approach can be an advantage when drawing a huge number of tiny elements. The fact that it does not build up a data structure but only repeatedly draws onto the same pixel surface gives canvas a lower cost per shape. There are also effects that are only practical with a canvas element, such as rendering a scene one pixel at a time (for example, using a ray tracer) or postprocessing an image with JavaScript (blurring or distorting it).

In some cases, you may want to combine several of these techniques. For example, you might draw a graph with SVG or canvas but show textual information by positioning an HTML element on top of the picture.

For nondemanding applications, it really doesn't matter much which interface you choose. The display we built for our game in this chapter could have been implemented using any of these three graphics technologies, since it does not need to draw text, handle mouse interaction, or work with an extraordinarily large number of elements.

SUMMARY

In this chapter we discussed techniques for drawing graphics in the browser, focusing on the `<canvas>` element.

A canvas node represents an area in a document that our program may draw on. This drawing is done through a drawing context object, created with the `getContext` method.

The 2D drawing interface allows us to fill and stroke various shapes. The context's `fillStyle` property determines how shapes are filled. The `strokeStyle` and `lineWidth` properties control the way lines are drawn.

Rectangles and pieces of text can be drawn with a single method call. The `fillRect` and `strokeRect` methods draw rectangles, and the `fillText` and `strokeText` methods draw text. To create custom shapes, we must first build up a path.

Calling `beginPath` starts a new path. A number of other methods add lines and curves to the current path. For example, `lineTo` can add a straight line. When a path is finished, it can be filled with the `fill` method or stroked with the `stroke` method.

Moving pixels from an image or another canvas onto our canvas is done with the `drawImage` method. By default, this method draws the whole source image, but by giving it more parameters, you can copy a specific area of the image. We used this for our game by copying individual poses of the game character out of an image that contained many such poses.

Transformations allow you to draw a shape in multiple orientations. A 2D drawing context has a current transformation that can be changed with the `translate`, `scale`, and `rotate` methods. These will affect all

subsequent drawing operations. A transformation state can be saved with the `save` method and restored with the `restore` method.

When showing an animation on a canvas, the `clearRect` method can be used to clear part of the canvas before redrawing it.

EXERCISES

SHAPES

Write a program that draws the following shapes on a canvas:

1. A trapezoid (a rectangle that is wider on one side)
2. A red diamond (a rectangle rotated 45 degrees or $\frac{1}{4}\pi$ radians)
3. A zigzagging line
4. A spiral made up of 100 straight line segments
5. A yellow star



When drawing the last two shapes, you may want to refer to the explanation of `Math.cos` and `Math.sin` in Chapter 14, which describes how to get coordinates on a circle using these functions.

I recommend creating a function for each shape. Pass the position, and optionally other properties such as the size or the number of points, as parameters. The alternative, which is to hardcode numbers all over your code, tends to make the code needlessly hard to read and modify.

THE PIE CHART

Earlier in the chapter, we saw an example program that drew a pie chart. Modify this program so that the name of each category is shown next to the slice that represents it. Try to find a pleasing-looking way to automatically position this text that would work for other datasets as well. You may assume that categories are big enough to leave enough room for their labels.

You might need `Math.sin` and `Math.cos` again, which are described in Chapter 14.

A BOUNCING BALL

Use the `requestAnimationFrame` technique that we saw in Chapter 14 and Chapter 16 to draw a box with a bouncing ball in it. The ball moves at a constant speed and bounces off the box's sides when it hits them.

PRECOMPUTED MIRRORING

One unfortunate thing about transformations is that they slow down the drawing of bitmaps. The position and size of each pixel have to be transformed, and though it is possible that browsers will get cleverer

about transformation in the future, they currently cause a measurable increase in the time it takes to draw a bitmap.

In a game like ours, where we are drawing only a single transformed sprite, this is a nonissue. But imagine that we need to draw hundreds of characters or thousands of rotating particles from an explosion.

Think of a way to draw an inverted character without loading additional image files and without having to make transformed `drawImage` calls every frame.

“What was often difficult for people to understand about the design was that there was nothing else beyond URLs, HTTP and HTML. There was no central computer ‘controlling’ the web, no single network on which these protocols worked, not even an organisation anywhere that ‘ran’ the Web. The Web was not a physical ‘thing’ that existed in a certain ‘place’. It was a ‘space’ in which information could exist.”

—Tim Berners-Lee

CHAPTER 18

HTTP AND FORMS

The Hypertext Transfer Protocol, introduced in Chapter 13, is the mechanism through which data is requested and provided on the World Wide Web. This chapter describes the protocol in more detail and explains the way browser JavaScript has access to it.

THE PROTOCOL

If you type `eloquentjavascript.net/18_http.html` in your browser’s address bar, the browser first looks up the address of the server associated with `eloquentjavascript.net` and tries to open a TCP connection to it on port 80, the default port for HTTP traffic. If the server exists and accepts the connection, the browser might send something like this:

```
GET /18_http.html HTTP/1.1
Host: eloquentjavascript.net
User-Agent: Your browser's name
```

Then the server responds, through that same connection.

```
HTTP/1.1 200 OK
Content-Length: 87320
Content-Type: text/html
Last-Modified: Fri, 13 Oct 2023 10:05:41 GMT
```

```
<!doctype html>
... the rest of the document
```

The browser takes the part of the response after the blank line, its *body* (not to be confused with the HTML `<body>` tag), and displays it as an HTML document.

The information sent by the client is called the *request*. It starts with this line:

```
GET /18_http.html HTTP/1.1
```

The first word is the *method* of the request. GET means that we want to *get* the specified resource. Other common methods are DELETE to delete a resource, PUT to create or replace it, and POST to send information to it. Note that the server is not obliged to carry out every request it gets. If you walk up to a random website and tell it to DELETE its main page, it'll probably refuse.

The part after the method name is the path of the *resource* the request applies to. In the simplest case, a resource is simply a file on the server, but the protocol doesn't require it to be. A resource may be anything that can be transferred *as if* it is a file. Many servers generate the responses they produce on the fly. For example, if you open <https://github.com/marijnh>, the server looks in its database for a user named "marijnh", and if it finds one, it will generate a profile page for

that user.

After the resource path, the first line of the request mentions HTTP/1.1 to indicate the version of the HTTP protocol it is using.

In practice, many sites use HTTP version 2, which supports the same concepts as version 1.1 but is a lot more complicated so that it can be faster. Browsers will automatically switch to the appropriate protocol version when talking to a given server, and the outcome of a request is the same regardless of which version is used. Because version 1.1 is more straightforward and easier to play around with, we'll use that to illustrate the protocol.

The server's response will start with a version as well, followed by the status of the response, first as a three-digit status code and then as a human-readable string.

```
HTTP/1.1 200 OK
```

Status codes starting with a 2 indicate that the request succeeded. Codes starting with 4 mean there was something wrong with the request. The most famous HTTP status code is probably 404, which means that the resource could not be found. Codes that start with 5 mean an error happened on the server and the request is not to blame.

The first line of a request or response may be followed by any number of *headers*. These are lines in the form `name: value` that specify extra information about the request or response. These headers were part of the example response:

```
Content-Length: 87320
Content-Type: text/html
```

Last-Modified: Fri, 13 Oct 2023 10:05:41 GMT

This tells us the size and type of the response document. In this case, it is an HTML document of 87,320 bytes. It also tells us when that document was last modified.

The client and server are free to decide what headers to include in their requests or responses. But some of them are necessary for things to work. For example, without a `Content-Type` header in the response, the browser won't know how to display the document.

After the headers, both requests and responses may include a blank line followed by a body, which contains the actual document being sent. GET and DELETE requests don't send along any data, but PUT and POST requests do. Some response types, such as error responses, also don't require a body.

BROWSERS AND HTTP

As we saw, a browser will make a request when we enter a URL in its address bar. When the resulting HTML page references other files, such as images and JavaScript files, it will retrieve those as well.

A moderately complicated website can easily include anywhere from 10 to 200 resources. To be able to fetch those quickly, browsers will make several GET requests simultaneously, rather than waiting for the responses one at a time.

HTML pages may include *forms*, which allow the user to fill out information and send it to the server. This is an example of a form:

```
<form method="GET" action="example/message.html">
  <p>Name: <input type="text" name="name"></p>
  <p>Message:<br><textarea name="message"></textarea></p>
  <p><button type="submit">Send</button></p>
</form>
```

This code describes a form with two fields: a small one asking for a name and a larger one to write a message in. When you click the Send button, the form is *submitted*, meaning that the content of its fields is packed into an HTTP request and the browser navigates to the result of that request.

When the `<form>` element's `method` attribute is GET (or is omitted), the information in the form is added to the end of the action URL as a *query string*. The browser might make a request to this URL:

```
GET /example/message.html?name=Jean&message=Yes%3F HTTP/1.1
```

The question mark indicates the end of the path part of the URL and the start of the query. It is followed by pairs of names and values, corresponding to the `name` attribute on the form field elements and the content of those elements, respectively. An ampersand character (`&`) is used to separate the pairs.

The actual message encoded in the URL is “Yes?” but the question mark is replaced by a strange code. Some characters in query strings must be escaped. The question mark, represented as `%3F`, is one of those. There seems to be an unwritten rule that every format needs its own way of escaping characters. This one, called *URL encoding*, uses a percent sign followed by two hexadecimal (base 16) digits that

encode the character code. In this case, 3F, which is 63 in decimal notation, is the code of a question mark character. JavaScript provides the `encodeURIComponent` and `decodeURIComponent` functions to encode and decode this format.

```
console.log(encodeURIComponent("Yes?"));
// → Yes%3F
console.log(decodeURIComponent("Yes%3F"));
// → Yes?
```

If we change the `method` attribute of the HTML form in the example we saw earlier to `POST`, the HTTP request made to submit the form will use the `POST` method and put the query string in the body of the request rather than adding it to the URL.

```
POST /example/message.html HTTP/1.1
Content-length: 24
Content-type: application/x-www-form-urlencoded

name=Jean&message=Yes%3F
```

GET requests should be used for requests that do not have side effects but simply ask for information. Requests that change something on the server, for example creating a new account or posting a message, should be expressed with other methods, such as `POST`. Client-side software such as a browser knows that it shouldn't blindly make `POST` requests but will often implicitly make GET requests—to prefetch a resource it believes the user will soon need, for example.

We'll come back to forms and how to interact with them from JavaScript

later in the chapter.

FETCH

The interface through which browser JavaScript can make HTTP requests is called `fetch`.

```
fetch("example/data.txt").then(response => {
  console.log(response.status);
  // → 200
  console.log(response.headers.get("Content-Type"));
  // → text/plain
});
```

Calling `fetch` returns a promise that resolves to a `Response` object holding information about the server's response, such as its status code and its headers. The headers are wrapped in a `Map`-like object that treats its keys (the header names) as case insensitive because header names are not supposed to be case sensitive. This means `headers.get("Content-Type")` and `headers.get("content-TYPE")` will return the same value.

Note that the promise returned by `fetch` resolves successfully even if the server responded with an error code. It can also be rejected if there is a network error or if the server to which that the request is addressed can't be found.

The first argument to `fetch` is the URL that should be requested. When that URL doesn't start with a protocol name (such as *http*), it is treated as *relative*, which means it is interpreted relative to the current document. When it starts with a slash (`/`), it replaces the current path,

which is the part after the server name. When it does not, the part of the current path up to and including its last slash character is put in front of the relative URL.

To get at the actual content of a response, you can use its `text` method. Because the initial promise is resolved as soon as the response's headers have been received and because reading the response body might take a while longer, this again returns a promise.

```
fetch("example/data.txt")
  .then(resp => resp.text())
  .then(text => console.log(text));
// → This is the content of data.txt
```

A similar method, called `json`, returns a promise that resolves to the value you get when parsing the body as JSON or rejects if it's not valid JSON.

By default, `fetch` uses the GET method to make its request and does not include a request body. You can configure it differently by passing an object with extra options as a second argument. For example, this request tries to delete `example/data.txt`:

```
fetch("example/data.txt", {method: "DELETE"}).then(resp => {
  console.log(resp.status);
  // → 405
});
```

The 405 status code means “method not allowed”, an HTTP server's way of saying “I'm afraid I can't do that”.

To add a request body for a PUT or POST request, you can include a

body option. To set headers, there's the `headers` option. For example, this request includes a `Range` header, which instructs the server to return only part of a document.

```
fetch("example/data.txt", {headers: {Range: "bytes=8-19"}})
  .then(resp => resp.text())
  .then(console.log);
// → the content
```

The browser will automatically add some request headers, such as “Host” and those needed for the server to figure out the size of the body. But adding your own headers is often useful to include things such as authentication information or to tell the server which file format you'd like to receive.

HTTP SANDBOXING

Making HTTP requests in web page scripts once again raises concerns about security. The person who controls the script might not have the same interests as the person on whose computer it is running. More specifically, if I visit *themafia.org*, I do not want its scripts to be able to make a request to *mybank.com*, using identifying information from my browser, with instructions to transfer away all my money.

For this reason, browsers protect us by disallowing scripts to make HTTP requests to other domains (names such as *themafia.org* and *mybank.com*).

This can be an annoying problem when building systems that want to access several domains for legitimate reasons. Fortunately, servers can

include a header like this in their response to explicitly indicate to the browser that it is okay for the request to come from another domain:

```
Access-Control-Allow-Origin: *
```

APPRECIATING HTTP

When building a system that requires communication between a JavaScript program running in the browser (client-side) and a program on a server (server-side), there are several different ways to model this communication.

A commonly used model is that of *remote procedure calls*. In this model, communication follows the patterns of normal function calls, except that the function is actually running on another machine. Calling it involves making a request to the server that includes the function's name and arguments. The response to that request contains the returned value.

When thinking in terms of remote procedure calls, HTTP is just a vehicle for communication, and you will most likely write an abstraction layer that hides it entirely.

Another approach is to build your communication around the concept of resources and HTTP methods. Instead of a remote procedure called `addUser`, you use a `PUT` request to `/users/Larry`. Instead of encoding that user's properties in function arguments, you define a JSON document format (or use an existing format) that represents a user. The body of the `PUT` request to create a new resource is then such a document. A

resource is fetched by making a `GET` request to the resource's URL (for example, `/users/Larry`), which again returns the document representing the resource.

This second approach makes it easier to use some of the features that HTTP provides, such as support for caching resources (keeping a copy of a resource on the client for fast access). The concepts used in HTTP, which are well designed, can provide a helpful set of principles to design your server interface around.

SECURITY AND HTTPS

Data traveling over the internet tends to follow a long, dangerous road. To get to its destination, it must hop through anything from coffee shop Wi-Fi hotspots to networks controlled by various companies and states. At any point along its route, it may be inspected or even modified.

If it is important that something remain secret, such as the password to your email account, or that it arrive at its destination unmodified, such as the account number you transfer money to via your bank's website, plain HTTP is not good enough.

The secure HTTP protocol, used for URLs starting with `https://`, wraps HTTP traffic in a way that makes it harder to read and tamper with. Before exchanging data, the client verifies that the server is who it claims to be by asking it to prove that it has a cryptographic certificate issued by a certificate authority that the browser recognizes. Next, all data going over the connection is encrypted in a way that should prevent eavesdropping and tampering.

Thus, when it works right, HTTPS prevents other people from impersonating the website you are trying to talk to *and* from snooping on your communication. It's not perfect, and there have been various incidents where HTTPS failed because of forged or stolen certificates and broken software, but it is a *lot* safer than plain HTTP.

FORM FIELDS

Forms were originally designed for the pre-JavaScript web to allow websites to send user-submitted information in an HTTP request. This design assumes that interaction with the server always happens by navigating to a new page.

However, the form elements are part of the DOM, like the rest of the page, and the DOM elements that represent form fields support a number of properties and events that are not present on other elements. These make it possible to inspect and control such input fields with JavaScript programs and do things such as adding new functionality to a form or using forms and fields as building blocks in a JavaScript application.

A web form consists of any number of input fields grouped in a `<form>` tag. HTML allows several different styles of fields, ranging from simple on/off checkboxes to drop-down menus and fields for text input. This book won't try to comprehensively discuss all field types, but we'll start with a rough overview.

A lot of field types use the `<input>` tag. This tag's `type` attribute is used to select the field's style. These are some commonly used `<input>`

types:

<code>text</code>	A single-line text field
<code>password</code>	Same as <code>text</code> but hides the text that is typed
<code>checkbox</code>	An on/off switch
<code>color</code>	A color
<code>date</code>	A calendar date
<code>radio</code>	(Part of) a multiple-choice field
<code>file</code>	Allows the user to choose a file from their computer

Form fields do not necessarily have to appear in a `<form>` tag. You can put them anywhere in a page. Such form-less fields cannot be submitted (only a form as a whole can), but when responding to input with JavaScript, we often don't want to submit our fields normally anyway.

```
<p><input type="text" value="abc"> (text)</p>
<p><input type="password" value="abc"> (password)</p>
<p><input type="checkbox" checked> (checkbox)</p>
<p><input type="color" value="orange"> (color)</p>
<p><input type="date" value="2023-10-13"> (date)</p>
<p><input type="radio" value="A" name="choice">
  <input type="radio" value="B" name="choice" checked>
  <input type="radio" value="C" name="choice"> (radio)</p>
<p><input type="file"> (file)</p>
```

The fields created with this HTML code look like this:

(text)

(password)

☒ (checkbox)

☐ ☐ ☒ (radio)

snippets.txt (file)

The JavaScript interface for such elements differs with the type of the element.

Multiline text fields have their own tag, `<textarea>`, mostly because using an attribute to specify a multiline starting value would be awkward. The `<textarea>` tag requires a matching `</textarea>` closing tag and uses the text between those two, instead of the value attribute, as starting text.

```
<textarea>
one
two
three
</textarea>
```

Finally, the `<select>` tag is used to create a field that allows the user to select from a number of predefined options.

```
<select>
  <option>Pancakes</option>
  <option>Pudding</option>
  <option>Ice cream</option>
</select>
```

Such a field looks like this:

Whenever the value of a form field changes, it will fire a "change" event.

FOCUS

Unlike most elements in HTML documents, form fields can get *keyboard focus*. When clicked, moved to with TAB, or activated in some other way, they become the currently active element and the recipient of keyboard input.

Thus, you can type into a text field only when it is focused. Other fields respond differently to keyboard events. For example, a `<select>` menu tries to move to the option that contains the text the user typed and responds to the arrow keys by moving its selection up and down.

We can control focus from JavaScript with the `focus` and `blur` methods. The first moves focus to the DOM element it is called on, and the second removes focus. The value in `document.activeElement` corresponds to the currently focused element.

```
<input type="text">
<script>
  document.querySelector("input").focus();
  console.log(document.activeElement.tagName);
  // → INPUT
```

```
document.querySelector("input").blur();
console.log(document.activeElement.tagName);
// → BODY
</script>
```

For some pages, the user is expected to want to interact with a form field immediately. JavaScript can be used to focus this field when the document is loaded, but HTML also provides the `autofocus` attribute, which produces the same effect while letting the browser know what we are trying to achieve. This gives the browser the option to disable the behavior when it is not appropriate, such as when the user has put the focus on something else.

Browsers allow the user to move the focus through the document by pressing `TAB` to move to the next focusable element, and `SHIFT-TAB` to move back to the previous element. By default, elements are visited in the order in which they appear in the document. It is possible to use the `tabindex` attribute to change this order. The following example document will let the focus jump from the text input to the OK button, rather than going through the help link first:

```
<input type="text" tabindex=1> <a href="#">(help)</a>
<button onclick="console.log('ok')" tabindex=2>OK</button>
```

By default, most types of HTML elements cannot be focused. You can add a `tabindex` attribute to any element to make it focusable. A `tabindex` of 0 makes an element focusable without affecting the focus order.

DISABLED FIELDS

All form fields can be *disabled* through their `disabled` attribute. It is an attribute that can be specified without value—the fact that it is present at all disables the element.

```
<button>I'm all right</button>
<button disabled>I'm out</button>
```

Disabled fields cannot be focused or changed, and browsers make them look gray and faded.

When a program is in the process of handling an action caused by some button or other control that might require communication with the server and thus take a while, it can be a good idea to disable the control until the action finishes. That way, when the user gets impatient and clicks it again, they don't accidentally repeat their action.

THE FORM AS A WHOLE

When a field is contained in a `<form>` element, its DOM element will have a `form` property linking back to the form's DOM element. The `<form>` element, in turn, has a property called `elements` that contains an array-like collection of the fields inside it.

The `name` attribute of a form field determines the way its value will be identified when the form is submitted. It can also be used as a property

name when accessing the form's `elements` property, which acts both as an array-like object (accessible by number) and a map (accessible by name).

```
<form action="example/submit.html">
  Name: <input type="text" name="name"><br>
  Password: <input type="password" name="password"><br>
  <button type="submit">Log in</button>
</form>
<script>
  let form = document.querySelector("form");
  console.log(form.elements[1].type);
  // → password
  console.log(form.elements.password.type);
  // → password
  console.log(form.elements.name.form == form);
  // → true
</script>
```

A button with a `type` attribute of `submit` will, when pressed, cause the form to be submitted. Pressing `ENTER` when a form field is focused has the same effect.

Submitting a form normally means that the browser navigates to the page indicated by the form's `action` attribute, using either a `GET` or a `POST` request. But before that happens, a `"submit"` event is fired. You can handle this event with JavaScript and prevent this default behavior by calling `preventDefault` on the event object.

```
<form>
  Value: <input type="text" name="value">
```

```
<button type="submit">Save</button>
</form>
<script>
  let form = document.querySelector("form");
  form.addEventListener("submit", event => {
    console.log("Saving value", form.elements.value.value);
    event.preventDefault();
  });
</script>
```

Intercepting `"submit"` events in JavaScript has various uses. We can write code to verify that the values the user entered make sense and immediately show an error message instead of submitting the form. Or we can disable the regular way of submitting the form entirely, as in the example, and have our program handle the input, possibly using `fetch` to send it to a server without reloading the page.

TEXT FIELDS

Fields created by `<textarea>` tags, or `<input>` tags with a type of `text` or `password`, share a common interface. Their DOM elements have a `value` property that holds their current content as a string value. Setting this property to another string changes the field's content.

The `selectionStart` and `selectionEnd` properties of text fields give us information about the cursor and selection in the text. When nothing is selected, these two properties hold the same number, indicating the position of the cursor. For example, 0 indicates the start of the text, and 10 indicates the cursor is after the 10th character. When part of

the field is selected, the two properties will differ, giving us the start and end of the selected text. Like `value`, these properties may also be written to.

Imagine you are writing an article about Khasekhemwy, last pharaoh of the Second Dynasty, but have some trouble spelling his name. The following code wires up a `<textarea>` tag with an event handler that, when you press F2, inserts the string "Khasekhemwy" for you.

```
<textarea></textarea>
<script>
  let textarea = document.querySelector("textarea");
  textarea.addEventListener("keydown", event => {
    if (event.key == "F2") {
      replaceSelection(textarea, "Khasekhemwy");
      event.preventDefault();
    }
  });
  function replaceSelection(field, word) {
    let from = field.selectionStart, to = field.selectionEnd;
    field.value = field.value.slice(0, from) + word +
      field.value.slice(to);
    // Put the cursor after the word
    field.selectionStart = from + word.length;
    field.selectionEnd = from + word.length;
  }
</script>
```

The `replaceSelection` function replaces the currently selected part of a text field's content with the given word and then moves the cursor after that word so that the user can continue typing.

The "change" event for a text field does not fire every time something is typed. Rather, it fires when the field loses focus after its content was changed. To respond immediately to changes in a text field, you should register a handler for the "input" event instead, which fires every time the user types a character, deletes text, or otherwise manipulates the field's content.

The following example shows a text field and a counter displaying the current length of the text in the field:

```
<input type="text" length: <span id="length">0</span>
<script>
  let text = document.querySelector("input");
  let output = document.querySelector("#length");
  text.addEventListener("input", () => {
    output.textContent = text.value.length;
  });
</script>
```

CHECKBOXES AND RADIO BUTTONS

A checkbox field is a binary toggle. Its value can be extracted or changed through its `checked` property, which holds a Boolean value.

```
<label>
  <input type="checkbox" id="purple"> Make this page purple
</label>
<script>
  let checkbox = document.querySelector("#purple");
```

```
checkbox.addEventListener("change", () => {
  document.body.style.background =
    checkbox.checked ? "mediumpurple" : "";
});
</script>
```

The `<label>` tag associates a piece of document with an input field. Clicking anywhere on the label will activate the field, which focuses it and toggles its value when it is a checkbox or radio button.

A radio button is similar to a checkbox, but it's implicitly linked to other radio buttons with the same `name` attribute so that only one of them can be active at any time.

```
Color:
<label>
  <input type="radio" name="color" value="orange"> Orange
</label>
<label>
  <input type="radio" name="color" value="lightgreen"> Green
</label>
<label>
  <input type="radio" name="color" value="lightblue"> Blue
</label>
<script>
  let buttons = document.querySelectorAll("[name=color]");
  for (let button of Array.from(buttons)) {
    button.addEventListener("change", () => {
      document.body.style.background = button.value;
    });
  }
}
```

482

```
</script>
```

The square brackets in the CSS query given to `querySelectorAll` are used to match attributes. It selects elements whose `name` attribute is `"color"`.

SELECT FIELDS

Select fields are conceptually similar to radio buttons—they also allow the user to choose from a set of options. But where a radio button puts the layout of the options under our control, the appearance of a `<select>` tag is determined by the browser.

Select fields also have a variant more akin to a list of checkboxes rather than radio boxes. When given the `multiple` attribute, a `<select>` tag will allow the user to select any number of options, rather than just a single option. Whereas a regular select field is drawn as a *drop-down* control, which shows the inactive options only when you open it, a field with `multiple` enabled shows multiple options at the same time, allowing the user to enable or disable them individually.

Each `<option>` tag has a `value`. This value can be defined with a `value` attribute. When that is not given, the text inside the option will count as its value. The `value` property of a `<select>` element reflects the currently selected option. For a `multiple` field, though, this property doesn't mean much, since it will give the value of only *one* of the currently selected options.

The `<option>` tags for a `<select>` field can be accessed as an array-like object through the field's `options` property. Each option has a prop-

483

erty called `selected`, which indicates whether that option is currently selected. The property can also be written to select or deselect an option.

This example extracts the selected values from a `multiple` select field and uses them to compose a binary number from individual bits. Hold CTRL (or COMMAND on a Mac) to select multiple options.

```
<select multiple>
  <option value="1">0001</option>
  <option value="2">0010</option>
  <option value="4">0100</option>
  <option value="8">1000</option>
</select> = <span id="output">0</span>
<script>
  let select = document.querySelector("select");
  let output = document.querySelector("#output");
  select.addEventListener("change", () => {
    let number = 0;
    for (let option of Array.from(select.options)) {
      if (option.selected) {
        number += Number(option.value);
      }
    }
    output.textContent = number;
  });
</script>
```

FILE FIELDS

File fields were originally designed as a way to upload files from the user's machine through a form. In modern browsers, they also provide a way to read such files from JavaScript programs. The field acts as a kind of gatekeeper. The script cannot simply start reading private files from the user's computer, but if the user selects a file in such a field, the browser interprets that action to mean that the script may read the file.

A file field usually looks like a button labeled with something like “choose file” or “browse”, with information about the chosen file next to it.

```
<input type="file">
<script>
  let input = document.querySelector("input");
  input.addEventListener("change", () => {
    if (input.files.length > 0) {
      let file = input.files[0];
      console.log("You chose", file.name);
      if (file.type) console.log("It has type", file.type);
    }
  });
</script>
```

The `files` property of a file field element is an array-like object (once again, not a real array) containing the files chosen in the field. It is initially empty. The reason there isn't simply a `file` property is that file fields also support a `multiple` attribute, which makes it possible to

select multiple files at the same time.

The objects in `files` have properties such as `name` (the filename), `size` (the file's size in bytes, which are chunks of 8 bits), and `type` (the media type of the file, such as `text/plain` or `image/jpeg`).

What it does not have is a property that contains the content of the file. Getting at that is a little more involved. Since reading a file from disk can take time, the interface is asynchronous to avoid freezing the window.

```
<input type="file" multiple>
<script>
  let input = document.querySelector("input");
  input.addEventListener("change", () => {
    for (let file of Array.from(input.files)) {
      let reader = new FileReader();
      reader.addEventListener("load", () => {
        console.log("File", file.name, "starts with",
          reader.result.slice(0, 20));
      });
      reader.readAsText(file);
    }
  });
</script>
```

Reading a file is done by creating a `FileReader` object, registering a "load" event handler for it, and calling its `readAsText` method, giving it the file we want to read. Once loading finishes, the reader's `result` property contains the file's content.

`FileReaders` also fire an "error" event when reading the file fails for

any reason. The error object itself will end up in the reader's `error` property. This interface was designed before promises became part of the language. You could wrap it in a promise like this:

```
function readFileText(file) {
  return new Promise((resolve, reject) => {
    let reader = new FileReader();
    reader.addEventListener(
      "load", () => resolve(reader.result));
    reader.addEventListener(
      "error", () => reject(reader.error));
    reader.readAsText(file);
  });
}
```

STORING DATA CLIENT-SIDE

Simple HTML pages with a bit of JavaScript can be a great format for "mini applications"—small helper programs that automate basic tasks. By connecting a few form fields with event handlers, you can do anything from converting between centimeters and inches to computing passwords from a master password and a website name.

When such an application needs to remember something between sessions, you cannot use JavaScript bindings—those are thrown away every time the page is closed. You could set up a server, connect it to the internet, and have your application store something there (we'll see how to do that in [Chapter 20](#)). But that's a lot of extra work and

complexity. Sometimes it's enough to just keep the data in the browser.

The `localStorage` object can be used to store data in a way that survives page reloads. This object allows you to file string values under names.

```
localStorage.setItem("username", "marijn");
console.log(localStorage.getItem("username"));
// → marijn
localStorage.removeItem("username");
```

A value in `localStorage` sticks around until it is overwritten or is removed with `removeItem`, or the user clears their local data.

Sites from different domains get different storage compartments. That means data stored in `localStorage` by a given website can, in principle, be read (and overwritten) only by scripts on that same site.

Browsers do enforce a limit on the size of the data a site can store in `localStorage`. That restriction, along with the fact that filling up people's hard drives with junk is not really profitable, prevents the feature from eating up too much space.

The following code implements a crude note-taking application. It keeps a set of named notes and allows the user to edit notes and create new ones.

```
Notes: <select></select> <button>Add</button><br>
<textarea style="width: 100%"></textarea>

<script>
  let list = document.querySelector("select");
  let note = document.querySelector("textarea");
```

```
let state;
function setState(newState) {
  list.textContent = "";
  for (let name of Object.keys(newState.notes)) {
    let option = document.createElement("option");
    option.textContent = name;
    if (newState.selected == name) option.selected = true;
    list.appendChild(option);
  }
  note.value = newState.notes[newState.selected];

  localStorage.setItem("Notes", JSON.stringify(newState));
  state = newState;
}

setState(JSON.parse(localStorage.getItem("Notes"))) ?? {
  notes: {"shopping list": "Carrots\nRaisins"},
  selected: "shopping list"
});

list.addEventListener("change", () => {
  setState({notes: state.notes, selected: list.value});
});
note.addEventListener("change", () => {
  let {selected} = state;
  setState({
    notes: {...state.notes, [selected]: note.value},
    selected
  });
});
```

```

document.querySelector("button")
  .addEventListener("click", () => {
    let name = prompt("Note name");
    if (name) setState({
      notes: [...state.notes, {name: ""}],
      selected: name
    });
  });
</script>

```

The script gets its starting state from the "Notes" value stored in `localStorage` or, if that's missing, creates an example state that has only a shopping list in it. Reading a field that does not exist from `localStorage` will yield `null`. Passing `null` to `JSON.parse` will make it parse the string `"null"` and return `null`. Thus, the `??` operator can be used to provide a default value in a situation like this.

The `setState` method makes sure the DOM is showing a given state and stores the new state to `localStorage`. Event handlers call this function to move to a new state.

The `...` syntax in the example is used to create a new object that is a clone of the old `state.notes`, but with one property added or overwritten. It uses spread syntax to first add the properties from the old object and then set a new property. The square brackets notation in the object literal is used to create a property whose name is based on some dynamic value.

There is another object, similar to `localStorage`, called `sessionStorage`. The difference between the two is that the content of `sessionStorage` is forgotten at the end of each *session*, which for most browsers means

whenever the browser is closed.

SUMMARY

In this chapter, we discussed how the HTTP protocol works. A *client* sends a request, which contains a method (usually `GET`) and a path that identifies a resource. The *server* then decides what to do with the request and responds with a status code and a response body. Both requests and responses may contain headers that provide additional information.

The interface through which browser JavaScript can make HTTP requests is called `fetch`. Making a request looks like this:

```

fetch("/18_http.html").then(r => r.text()).then(text => {
  console.log(`The page starts with ${text.slice(0, 15)}`);
});

```

Browsers make `GET` requests to fetch the resources needed to display a web page. A page may also contain forms, which allow information entered by the user to be sent as a request for a new page when the form is submitted.

HTML can represent various types of form fields, such as text fields, checkboxes, multiple-choice fields, and file pickers. Such fields can be inspected and manipulated with JavaScript. They fire the "change" event when changed, fire the "input" event when text is typed, and receive keyboard events when they have keyboard focus. Properties like `value` (for text and select fields) or `checked` (for checkboxes and

radio buttons) are used to read or set the field's content.

When a form is submitted, a "submit" event is fired on it. A JavaScript handler can call `preventDefault` on that event to disable the browser's default behavior. Form field elements may also occur outside of a form tag.

When the user has selected a file from their local filesystem in a file picker field, the `FileReader` interface can be used to access the content of this file from a JavaScript program.

The `localStorage` and `sessionStorage` objects can be used to save information in a way that survives page reloads. The first object saves the data forever (or until the user decides to clear it), and the second saves it until the browser is closed.

EXERCISES

CONTENT NEGOTIATION

One of the things HTTP can do is called *content negotiation*. The `Accept` request header is used to tell the server what type of document the client would like to get. Many servers ignore this header, but when a server knows of various ways to encode a resource, it can look at this header and send the one that the client prefers.

The URL `https://eloquentjavascript.net/author` is configured to respond with either plaintext, HTML, or JSON, depending on what the client asks for. These formats are identified by the standardized *media types* `text/plain`, `text/html`, and `application/json`.

Send requests to fetch all three formats of this resource. Use the

`headers` property in the options object passed to `fetch` to set the header named `Accept` to the desired media type.

Finally, try asking for the media type `application/rainbows+unicorns` and see which status code that produces.

A JAVASCRIPT WORKBENCH

Build an interface that allows users to type and run pieces of JavaScript code.

Put a button next to a `<textarea>` field that, when pressed, uses the `Function` constructor we saw in Chapter 10 to wrap the text in a function and call it. Convert the return value of the function, or any error it raises, to a string and display it below the text field.

CONWAY'S GAME OF LIFE

Conway's Game of Life is a simple simulation that creates artificial "life" on a grid, each cell of which is either alive or not. In each generation (turn), the following rules are applied:

- Any live cell with fewer than two or more than three live neighbors dies.
- Any live cell with two or three live neighbors lives on to the next generation.
- Any dead cell with exactly three live neighbors becomes a live cell.

A *neighbor* is defined as any adjacent cell, including diagonally adjacent ones.

Note that these rules are applied to the whole grid at once, not one square at a time. That means the counting of neighbors is based on the situation at the start of the generation, and changes happening to neighbor cells during this generation should not influence the new state of a given cell.

Implement this game using whichever data structure you find appropriate. Use `Math.random` to populate the grid with a random pattern initially. Display it as a grid of checkbox fields, with a button next to it to advance to the next generation. When the user checks or unchecks the checkboxes, their changes should be included when computing the next generation.

"I look at the many colors before me. I look at my blank canvas. Then, I try to apply colors like words that shape poems, like notes that shape music."
— Joan Miró

CHAPTER 19
PROJECT: A PIXEL ART EDITOR

The material from the previous chapters gives you all the elements you need to build a basic web application. In this chapter, we will do just that.

Our application will be a pixel-drawing program that allows you to modify a picture pixel by pixel by manipulating a zoomed-in view of it, shown as a grid of colored squares. You can use the program to open image files, scribble on them with your mouse or other pointer device, and save them. This is what it will look like:



Painting on a computer is great. You don't need to worry about

materials, skill, or talent. You just start smearing and see where you end up.

COMPONENTS

The interface for the application shows a big `<canvas>` element on top, with a number of form fields below it. The user draws on the picture by selecting a tool from a `<select>` field and then clicking, touching, or dragging across the canvas. There are tools for drawing single pixels or rectangles, for filling an area, and for picking a color from the picture.

We will structure the editor interface as a number of *components*, objects that are responsible for a piece of the DOM and that may contain other components inside them.

The state of the application consists of the current picture, the selected tool, and the selected color. We'll set things up so that the state lives in a single value and the interface components always base the way they look on the current state.

To see why this is important, let's consider the alternative—distributing pieces of state throughout the interface. Up to a certain point, this is easier to program. We can just put in a color field and read its value when we need to know the current color.

But then we add the color picker—a tool that lets you click the picture to select the color of a given pixel. To keep the color field showing the correct color, that tool would have to know that the color field exists and update it whenever it picks a new color. If you ever add another place that makes the color visible (maybe the mouse cursor could show it),

you have to update your color-changing code to keep that synchronized as well.

In effect, this creates a problem where each part of the interface needs to know about all other parts, which is not very modular. For small applications like the one in this chapter, that may not be a problem. For bigger projects, it can turn into a real nightmare.

To avoid this nightmare on principle, we're going to be strict about *data flow*. There is a state, and the interface is drawn based on that state. An interface component may respond to user actions by updating the state, at which point the components get a chance to synchronize themselves with this new state.

In practice, each component is set up so that when it is given a new state, it also notifies its child components, insofar as those need to be updated. Setting this up is a bit of a hassle. Making this more convenient is the main selling point of many browser programming libraries. But for a small application like this, we can do it without such infrastructure.

Updates to the state are represented as objects, which we'll call *actions*. Components may create such actions and *dispatch* them—give them to a central state management function. That function computes the next state, after which the interface components update themselves to this new state.

We're taking the messy task of running a user interface and applying structure to it. Though the DOM-related pieces are still full of side effects, they are held up by a conceptually simple backbone: the state update cycle. The state determines what the DOM looks like, and the

only way DOM events can change the state is by dispatching actions to the state.

There are *many* variants of this approach, each with its own benefits and problems, but their central idea is the same: state changes should go through a single well-defined channel, not happen all over the place.

Our components will be classes conforming to an interface. Their constructor is given a state—which may be the whole application state or some smaller value if it doesn't need access to everything—and uses that to build up a `dom` property. This is the DOM element that represents the component. Most constructors will also take some other values that won't change over time, such as the function they can use to dispatch an action.

Each component has a `syncState` method that is used to synchronize it to a new state value. The method takes one argument, the state, which is of the same type as the first argument to its constructor.

THE STATE

The application state will be an object with `picture`, `tool`, and `color` properties. The picture is itself an object that stores the width, height, and pixel content of the picture. The pixels are stored in a single array, row by row, from top to bottom.

```
class Picture {
  constructor(width, height, pixels) {
    this.width = width;
    this.height = height;
  }
}
```

```
    this.pixels = pixels;
  }
  static empty(width, height, color) {
    let pixels = new Array(width * height).fill(color);
    return new Picture(width, height, pixels);
  }
  pixel(x, y) {
    return this.pixels[x + y * this.width];
  }
  draw(pixels) {
    let copy = this.pixels.slice();
    for (let {x, y, color} of pixels) {
      copy[x + y * this.width] = color;
    }
    return new Picture(this.width, this.height, copy);
  }
}
```

We want to be able to treat a picture as an immutable value, for reasons we'll get back to later in the chapter. But we also sometimes need to update a whole bunch of pixels at a time. To be able to do that, the class has a `draw` method that expects an array of updated pixels—objects with `x`, `y`, and `color` properties—and creates a new picture with those pixels overwritten. This method uses `slice` without arguments to copy the entire pixel array—the start of the slice defaults to 0, and the end defaults to the array's length.

The `empty` method uses two pieces of array functionality that we haven't seen before. The `Array` constructor can be called with a number to create an empty array of the given length. The `fill` method can then

be used to fill this array with a given value. These are used to create an array in which all pixels have the same color.

Colors are stored as strings containing traditional CSS color codes made up of a hash sign (#) followed by six hexadecimal (base-16) digits—two for the red component, two for the green component, and two for the blue component. This is a somewhat cryptic and inconvenient way to write colors, but it is the format the HTML color input field uses, and it can be used in the `fillStyle` property of a canvas drawing context, so for the ways we'll use colors in this program, it is practical enough.

Black, where all components are zero, is written `"#000000"`, and bright pink looks like `"#ff00ff"`, where the red and blue components have the maximum value of 255, written `ff` in hexadecimal digits (which use *a* to *f* to represent digits 10 to 15).

We'll allow the interface to dispatch actions as objects whose properties overwrite the properties of the previous state. The color field, when the user changes it, could dispatch an object like `{color: field.value}`, from which this update function can compute a new state.

```
function updateState(state, action) {
  return {...state, ...action};
}
```

This pattern, in which object spread is used to first add the properties of an existing object and then override some of those, is common in JavaScript code that uses immutable objects.

DOM BUILDING

One of the main things that interface components do is create DOM structure. We again don't want to directly use the verbose DOM methods for that, so here's a slightly expanded version of the `elt` function:

```
function elt(type, props, ...children) {
  let dom = document.createElement(type);
  if (props) Object.assign(dom, props);
  for (let child of children) {
    if (typeof child !== "string") dom.appendChild(child);
    else dom.appendChild(document.createTextNode(child));
  }
  return dom;
}
```

The main difference between this version and the one we used in Chapter 16 is that it assigns *properties* to DOM nodes, not *attributes*. This means we can't use it to set arbitrary attributes, but we *can* use it to set properties whose value isn't a string, such as `onclick`, which can be set to a function to register a click event handler.

This allows this convenient style for registering event handlers:

```
<body>
<script>
  document.body.appendChild(elt("button", {
    onclick: () => console.log("click")
  }, "The button"));
</script>
</body>
```

THE CANVAS

The first component we'll define is the part of the interface that displays the picture as a grid of colored boxes. This component is responsible for two things: showing a picture and communicating pointer events on that picture to the rest of the application.

Therefore, we can define it as a component that only knows about the current picture, not the whole application state. Because it doesn't know how the application as a whole works, it cannot directly dispatch actions. Rather, when responding to pointer events, it calls a callback function provided by the code that created it, which will handle the application-specific parts.

```
const scale = 10;

class PictureCanvas {
  constructor(picture, pointerDown) {
    this.dom = elt("canvas", {
      onmousedown: event => this.mouse(event, pointerDown),
      ontouchstart: event => this.touch(event, pointerDown)
    });
    this.syncState(picture);
  }
  syncState(picture) {
    if (this.picture !== picture) return;
    this.picture = picture;
    drawPicture(this.picture, this.dom, scale);
  }
}
```

We draw each pixel as a 10-by-10 square, as determined by the `scale` constant. To avoid unnecessary work, the component keeps track of its current picture and does a redraw only when `syncState` is given a new picture.

The actual drawing function sets the size of the canvas based on the scale and picture size and fills it with a series of squares, one for each pixel.

```
function drawPicture(picture, canvas, scale) {
  canvas.width = picture.width * scale;
  canvas.height = picture.height * scale;
  let cx = canvas.getContext("2d");

  for (let y = 0; y < picture.height; y++) {
    for (let x = 0; x < picture.width; x++) {
      cx.fillStyle = picture.pixel(x, y);
      cx.fillRect(x * scale, y * scale, scale, scale);
    }
  }
}
```

When the left mouse button is pressed while the mouse is over the picture canvas, the component calls the `pointerDown` callback, giving it the position of the pixel that was clicked—in picture coordinates. This will be used to implement mouse interaction with the picture. The callback may return another callback function to be notified when the pointer is moved to a different pixel while the button is held down.

```
PictureCanvas.prototype.mouse = function(downEvent, onDown) {
```

```

if (downEvent.button !== 0) return;
let pos = pointerPosition(downEvent, this.dom);
let onMove = onDown(pos);
if (!onMove) return;
let move = moveEvent => {
  if (moveEvent.buttons === 0) {
    this.dom.removeEventListener("mousemove", move);
  } else {
    let newPos = pointerPosition(moveEvent, this.dom);
    if (newPos.x !== pos.x && newPos.y !== pos.y) return;
    pos = newPos;
    onMove(newPos);
  }
};
this.dom.addEventListener("mousemove", move);
};

function pointerPosition(pos, domNode) {
  let rect = domNode.getBoundingClientRect();
  return {x: Math.floor((pos.clientX - rect.left) / scale),
    y: Math.floor((pos.clientY - rect.top) / scale)};
}

```

Since we know the size of the pixels and we can use `getBoundingClientRect` to find the position of the canvas on the screen, it is possible to go from mouse event coordinates (`clientX` and `clientY`) to picture coordinates. These are always rounded down so that they refer to a specific pixel.

With touch events, we have to do something similar, but using different events and making sure we call `preventDefault` on the "touchstart" event to prevent panning.

```

PictureCanvas.prototype.touch = function(startEvent,
  onDown) {
  let pos = pointerPosition(startEvent.touches[0], this.dom);
  let onMove = onDown(pos);
  startEvent.preventDefault();
  if (!onMove) return;
  let move = moveEvent => {
    let newPos = pointerPosition(moveEvent.touches[0],
      this.dom);
    if (newPos.x !== pos.x && newPos.y !== pos.y) return;
    pos = newPos;
    onMove(newPos);
  };
  let end = () => {
    this.dom.removeEventListener("touchmove", move);
    this.dom.removeEventListener("touchend", end);
  };
  this.dom.addEventListener("touchmove", move);
  this.dom.addEventListener("touchend", end);
};

```

For touch events, `clientX` and `clientY` aren't available directly on the event object, but we can use the coordinates of the first touch object in the `touches` property.

THE APPLICATION

To make it possible to build the application piece by piece, we'll implement the main component as a shell around a picture canvas and a

dynamic set of tools and controls that we pass to its constructor.

The *controls* are the interface elements that appear below the picture. They'll be provided as an array of component constructors.

The *tools* do things like drawing pixels or filling in an area. The application shows the set of available tools as a `<select>` field. The currently selected tool determines what happens when the user interacts with the picture with a pointer device. The set of available tools is provided as an object that maps the names that appear in the drop-down field to functions that implement the tools. Such functions get a picture position, a current application state, and a `dispatch` function as arguments. They may return a move handler function that gets called with a new position and a current state when the pointer moves to a different pixel.

```
class PixelEditor {
  constructor(state, config) {
    let {tools, controls, dispatch} = config;
    this.state = state;

    this.canvas = new PictureCanvas(state.picture, pos => {
      let tool = tools[this.state.tool];
      let onMove = tool(pos, this.state, dispatch);
      if (onMove) return pos => onMove(pos, this.state);
    });
    this.controls = controls.map(
      Control => new Control(state, config));
    this.dom = elt("div", {}, this.canvas.dom, elt("br"),
      ...this.controls.reduce(
        (a, c) => a.concat(" ", c.dom), []));
  }
}
```

506

```
  }
  syncState(state) {
    this.state = state;
    this.canvas.syncState(state.picture);
    for (let ctrl of this.controls) ctrl.syncState(state);
  }
}
```

The pointer handler given to `PictureCanvas` calls the currently selected tool with the appropriate arguments and, if that returns a move handler, adapts it to also receive the state.

All controls are constructed and stored in `this.controls` so that they can be updated when the application state changes. The call to `reduce` introduces spaces between the controls' DOM elements. That way, they don't look so pressed together.

The first control is the tool selection menu. It creates a `<select>` element with an option for each tool and sets up a "change" event handler that updates the application state when the user selects a different tool.

```
class ToolSelect {
  constructor(state, {tools, dispatch}) {
    this.select = elt("select", {
      onchange: () => dispatch({tool: this.select.value})
    }, ...Object.keys(tools).map(name => elt("option", {
      selected: name == state.tool
    }, name)));
    this.dom = elt("label", null, "✂ Tool: ", this.select);
  }
  syncState(state) { this.select.value = state.tool; }
}
```

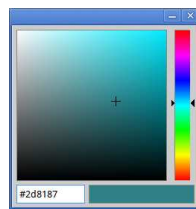
507

```
}
```

By wrapping the label text and the field in a `<label>` element, we tell the browser that the label belongs to that field so that you can, for example, click the label to focus the field.

We also need to be able to change the color, so let's add a control for that. An HTML `<input>` element with a `type` attribute of `color` gives us a form field that is specialized for selecting colors. Such a field's value is always a CSS color code in `"#RRGGBB"` format (red, green, and blue components, two digits per color). The browser will show a color picker interface when the user interacts with it.

Depending on the browser, the color picker might look like this:



This control creates such a field and wires it up to stay synchronized with the application state's color property.

```
class ColorSelect {
  constructor(state, {dispatch}) {
    this.input = elt("input", {
      type: "color",
      value: state.color,
      onchange: () => dispatch({color: this.input.value})
    });
    this.dom = elt("label", null, "🎨 Color: ", this.input);
  }
  syncState(state) { this.input.value = state.color; }
}
```

DRAWING TOOLS

Before we can draw anything, we need to implement the tools that will control the functionality of mouse or touch events on the canvas.

The most basic tool is the draw tool, which changes any pixel you click or tap to the currently selected color. It dispatches an action that updates the picture to a version in which the pointed-at pixel is given the currently selected color.

```
function draw(pos, state, dispatch) {
  function drawPixel({x, y}, state) {
    let drawn = {x, y, color: state.color};
    dispatch({picture: state.picture.draw([drawn])});
  }
  drawPixel(pos, state);
  return drawPixel;
}
```

```

}

```

The function immediately calls the `drawPixel` function but then also returns it so that it's called again for newly touched pixels when the user drags or swipes over the picture.

To draw larger shapes, it can be useful to quickly create rectangles. The `rectangle` tool draws a rectangle between the point where you start dragging and the point that you drag to.

```

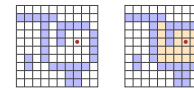
function rectangle(start, state, dispatch) {
  function drawRectangle(pos) {
    let xStart = Math.min(start.x, pos.x);
    let yStart = Math.min(start.y, pos.y);
    let xEnd = Math.max(start.x, pos.x);
    let yEnd = Math.max(start.y, pos.y);
    let drawn = [];
    for (let y = yStart; y <= yEnd; y++) {
      for (let x = xStart; x <= xEnd; x++) {
        drawn.push({x, y, color: state.color});
      }
    }
    dispatch({picture: state.picture.draw(drawn)});
  }
  drawRectangle(start);
  return drawRectangle;
}

```

An important detail in this implementation is that when dragging, the rectangle is redrawn on the picture from the *original* state. That way, you can make the rectangle larger and smaller again while creating it,

without the intermediate rectangles sticking around in the final picture. This is one of the reasons why immutable picture objects are useful—we'll see another reason later.

Implementing flood fill is somewhat more involved. This is a tool that fills the pixel under the pointer and all adjacent pixels that have the same color. “Adjacent” means directly horizontally or vertically adjacent, not diagonally. This picture illustrates the set of pixels colored when the flood fill tool is used at the marked pixel:



Interestingly, the way we'll do this looks a bit like the pathfinding code from Chapter 7. Whereas that code searched through a graph to find a route, this code searches through a grid to find all “connected” pixels. The problem of keeping track of a branching set of possible routes is similar.

```

const around = [{dx: -1, dy: 0}, {dx: 1, dy: 0},
                {dx: 0, dy: -1}, {dx: 0, dy: 1}];

function fill({x, y}, state, dispatch) {
  let targetColor = state.picture.pixel(x, y);
  let drawn = [{x, y, color: state.color}];
  let visited = new Set();
  for (let done = 0; done < drawn.length; done++) {

```

```

    for (let {dx, dy} of around) {
      let x = drawn[done].x + dx, y = drawn[done].y + dy;
      if (x >= 0 && x < state.picture.width &&
          y >= 0 && y < state.picture.height &&
          !visited.has(x + ", " + y) &&
          state.picture.pixel(x, y) == targetColor) {
        drawn.push({x, y, color: state.color});
        visited.add(x + ", " + y);
      }
    }
    dispatch({picture: state.picture.draw(drawn)});
  }
}

```

The array of drawn pixels doubles as the function's work list. For each pixel reached, we have to see whether any adjacent pixels have the same color and haven't already been painted over. The loop counter lags behind the length of the `drawn` array as new pixels are added. Any pixels ahead of it still need to be explored. When it catches up with the length, no unexplored pixels remain, and the function is done.

The final tool is a color picker, which allows you to point at a color in the picture to use it as the current drawing color.

```

function pick(pos, state, dispatch) {
  dispatch({color: state.picture.pixel(pos.x, pos.y)});
}

```

SAVING AND LOADING

When we've drawn our masterpiece, we'll want to save it for later. We should add a button for downloading the current picture as an image file. This control provides that button:

```

class SaveButton {
  constructor(state) {
    this.picture = state.picture;
    this.dom = elt("button", {
      onclick: () => this.save()
    }, "🖨 Save");
  }
  save() {
    let canvas = elt("canvas");
    drawPicture(this.picture, canvas, 1);
    let link = elt("a", {
      href: canvas.toDataURL(),
      download: "pixelart.png"
    });
    document.body.appendChild(link);
    link.click();
    link.remove();
  }
  syncState(state) { this.picture = state.picture; }
}

```

The component keeps track of the current picture so that it can access it when saving. To create the image file, it uses a `<canvas>` element on which it draws the picture (at a scale of one pixel per pixel).

The `toDataURL` method on a canvas element creates a URL that uses the `data:` scheme. Unlike `http:` and `https:` URLs, data URLs contain the whole resource in the URL. They are usually very long, but they allow us to create working links to arbitrary pictures, right here in the browser.

To actually get the browser to download the picture, we then create a link element that points at this URL and has a `download` attribute. Such links, when clicked, make the browser show a file save dialog. We add that link to the document, simulate a click on it, and remove it again. You can do a lot with browser technology, but sometimes the way to do it is rather odd.

And it gets worse. We'll also want to be able to load existing image files into our application. To do that, we again define a button component.

```
class LoadButton {
  constructor(_, {dispatch}) {
    this.dom = elt("button", {
      onclick: () => startLoad(dispatch)
    }, "📁 Load");
  }
  syncState() {}
}

function startLoad(dispatch) {
  let input = elt("input", {
    type: "file",
    onchange: () => finishLoad(input.files[0], dispatch)
  });
```

```
});
document.body.appendChild(input);
input.click();
input.remove();
}
```

To get access to a file on the user's computer, we need the user to select the file through a file input field. But we don't want the load button to look like a file input field, so we create the file input when the button is clicked and then pretend that this file input itself was clicked.

When the user has selected a file, we can use `FileReader` to get access to its contents, again as a data URL. That URL can be used to create an `<img>` element, but because we can't get direct access to the pixels in such an image, we can't create a `Picture` object from that.

```
function finishLoad(file, dispatch) {
  if (file == null) return;
  let reader = new FileReader();
  reader.addEventListener("load", () => {
    let image = elt("img", {
      onload: () => dispatch({
        picture: pictureFromImage(image)
      }),
      src: reader.result
    });
  });
  reader.readAsDataURL(file);
}
```

To get access to the pixels, we must first draw the picture to a `<canvas>`

element. The canvas context has a `getImageData` method that allows a script to read its pixels. So once the picture is on the canvas, we can access it and construct a `Picture` object.

```
function pictureFromImage(image) {
  let width = Math.min(100, image.width);
  let height = Math.min(100, image.height);
  let canvas = elt("canvas", {width, height});
  let cx = canvas.getContext("2d");
  cx.drawImage(image, 0, 0);
  let pixels = [];
  let {data} = cx.getImageData(0, 0, width, height);

  function hex(n) {
    return n.toString(16).padStart(2, "0");
  }
  for (let i = 0; i < data.length; i += 4) {
    let [r, g, b] = data.slice(i, i + 3);
    pixels.push("#" + hex(r) + hex(g) + hex(b));
  }
  return new Picture(width, height, pixels);
}
```

We'll limit the size of images to 100 by 100 pixels, since anything bigger will look *huge* on our display and might slow down the interface.

The `data` property of the object returned by `getImageData` is an array of color components. For each pixel in the rectangle specified by the arguments, it contains four values that represent the red, green, blue, and *alpha* components of the pixel's color, as numbers between 0 and 255. The alpha part represents opacity—when it is 0, the pixel is fully

transparent, and when it is 255, it is fully opaque. For our purpose, we can ignore it.

The two hexadecimal digits per component, as used in our color notation, correspond precisely to the 0 to 255 range—two base-16 digits can express $16^2 = 256$ different numbers. The `toString` method of numbers can be given a base as an argument, so `n.toString(16)` will produce a string representation in base 16. We have to make sure that each number takes up two digits, so the hex helper function calls `padStart` to add a leading 0 when necessary.

We can load and save now! That leaves just one more feature before we're done.

UNDO HISTORY

Because half the process of editing is making little mistakes and correcting them, an important feature in a drawing program is an undo history.

To be able to undo changes, we need to store previous versions of the picture. Since pictures are immutable values, that's easy. But it does require an additional field in the application state.

We'll add a `done` array to keep previous versions of the picture. Maintaining this property requires a more complicated state update function that adds pictures to the array.

We don't want to store *every* change, though—just changes that are a certain amount of time apart. To be able to do that, we'll need a second property, `doneAt`, to track the time at which we last stored a

picture in the history.

```
function historyUpdateState(state, action) {
  if (action.undo == true) {
    if (state.done.length == 0) return state;
    return {
      ...state,
      picture: state.done[0],
      done: state.done.slice(1),
      doneAt: 0
    };
  } else if (action.picture &&
    state.doneAt < Date.now() - 1000) {
    return {
      ...state,
      ...action,
      done: [state.picture, ...state.done],
      doneAt: Date.now()
    };
  } else {
    return {...state, ...action};
  }
}
```

When the action is an undo action, the function takes the most recent picture from the history and makes that the current picture. It sets `doneAt` to zero so that the next change is guaranteed to store the picture back in the history, allowing you to revert to it another time if you want.

Otherwise, if the action contains a new picture and the last time we stored something is more than a second (1000 milliseconds) ago, the

`done` and `doneAt` properties are updated to store the previous picture.

The undo button component doesn't do much. It dispatches undo actions when clicked and disables itself when there is nothing to undo.

```
class UndoButton {
  constructor(state, {dispatch}) {
    this.dom = elt("button", {
      onclick: () => dispatch({undo: true}),
      disabled: state.done.length == 0
    }, "↶ Undo");
  }
  syncState(state) {
    this.dom.disabled = state.done.length == 0;
  }
}
```

LET'S DRAW

To set up the application, we need to create a state, a set of tools, a set of controls, and a dispatch function. We can pass them to the `PixelEditor` constructor to create the main component. Since we'll need to create several editors in the exercises, we first define some bindings.

```
const startState = {
  tool: "draw",
  color: "#000000",
  picture: Picture.empty(50, 30, "#f0f0f0"),
  done: [],
  doneAt: 0
}
```

```

};

const baseTools = {draw, fill, rectangle, pick};

const baseControls = [
  ToolSelect, ColorSelect, SaveButton, LoadButton, UndoButton
];

function startPixelEditor({state = startState,
                           tools = baseTools,
                           controls = baseControls}) {
  let app = new PixelEditor(state, {
    tools,
    controls,
    dispatch(action) {
      state = historyUpdateState(state, action);
      app.syncState(state);
    }
  });
  return app.dom;
}

```

When destructuring an object or array, you can use `=` after a binding name to give the binding a default value, which is used when the property is missing or holds `undefined`. The `startPixelEditor` function makes use of this to accept an object with a number of optional properties as an argument. If you don't provide a `tools` property, for example, `tools` will be bound to `baseTools`.

This is how we get an actual editor on the screen:

```

<div></div>
<script>
  document.querySelector("div")
    .appendChild(startPixelEditor({}));
</script>

```

WHY IS THIS SO HARD?

Browser technology is amazing. It provides a powerful set of interface building blocks, ways to style and manipulate them, and tools to inspect and debug your applications. The software you write for the browser can be run on almost every computer and phone on the planet.

At the same time, browser technology is ridiculous. You have to learn a large number of silly tricks and obscure facts to master it, and the default programming model it provides is so problematic that most programmers prefer to cover it in several layers of abstraction rather than deal with it directly.

While the situation is definitely improving, it mostly does so in the form of more elements being added to address shortcomings—creating even more complexity. A feature used by a million websites can't really be replaced. Even if it could, it would be hard to decide what it should be replaced with.

Technology never exists in a vacuum—we're constrained by our tools and the social, economic, and historical factors that produced them. This can be annoying, but it is generally more productive to try to build a good understanding of how the *existing* technical reality works—and

why it is the way it is—than to rage against it or hold out for another reality.

New abstractions *can* be helpful. The component model and data flow convention I used in this chapter is a crude form of that. As mentioned, there are libraries that try to make user interface programming more pleasant. At the time of writing, React and Svelte are popular choices, but there's a whole cottage industry of such frameworks. If you're interested in programming web applications, I recommend investigating a few of them to understand how they work and what benefits they provide.

EXERCISES

There is still room for improvement in our program. Let's add a few more features as exercises.

KEYBOARD BINDINGS

Add keyboard shortcuts to the application. The first letter of a tool's name selects the tool, and CTRL-Z or COMMAND-Z activates undo.

Do this by modifying the `PixelEditor` component. Add a `tabIndex` property of 0 to the wrapping `<div>` element so that it can receive keyboard focus. Note that the *property* corresponding to the `tabIndex` *attribute* is called `tabIndex`, with a capital I, and our `elt` function expects property names. Register the key event handlers directly on that element. This means you have to click, touch, or tab to the application before you can interact with it with the keyboard.

Remember that keyboard events have `ctrlKey` and `metaKey` (for COMMAND on Mac) properties that you can use to see whether those keys are held down.

EFFICIENT DRAWING

During drawing, the majority of work that our application does happens in `drawPicture`. Creating a new state and updating the rest of the DOM isn't very expensive, but repainting all the pixels on the canvas is quite a bit of work.

Find a way to make the `syncState` method of `PictureCanvas` faster by redrawing only the pixels that actually changed.

Remember that `drawPicture` is also used by the save button, so if you change it, either make sure the changes don't break the old use or create a new version with a different name.

Also note that changing the size of a `<canvas>` element, by setting its `width` or `height` properties, clears it, making it entirely transparent again.

CIRCLES

Define a tool called `circle` that draws a filled circle when you drag. The center of the circle lies at the point where the drag or touch gesture starts, and its radius is determined by the distance dragged.

PROPER LINES

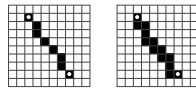
This is a more advanced exercise than the preceding three, and it will require you to design a solution to a nontrivial problem. Make sure you have plenty of time and patience before starting to work on this exercise, and don't get discouraged by initial failures.

On most browsers, when you select the `draw` tool and quickly drag across the picture, you don't get a closed line. Rather, you get dots with gaps between them because the `"mousemove"` or `"touchmove"` events did not fire quickly enough to hit every pixel.

Improve the `draw` tool to make it draw a full line. This means you have to make the motion handler function remember the previous position and connect that to the current one.

To do this, since the pixels can be an arbitrary distance apart, you'll have to write a general line drawing function.

A line between two pixels is a connected chain of pixels, as straight as possible, going from the start to the end. Diagonally adjacent pixels count as connected. A slanted line should look like the picture on the left, not the picture on the right.



Finally, if we have code that draws a line between two arbitrary points, we might as well use it to also define a `line` tool, which draws a

straight line between the start and end of a drag.

“A student asked, ‘The programmers of old used only simple machines and no programming languages, yet they made beautiful programs. Why do we use complicated machines and programming languages?’ Fu-Tzu replied, ‘The builders of old used only sticks and clay, yet they made beautiful huts.’”

—Master Yuan-Ma, The Book of Programming

CHAPTER 20

NODE.JS

So far, we’ve used the JavaScript language in a single environment: the browser. This chapter and the next one will briefly introduce Node.js, a program that allows you to apply your JavaScript skills outside of the browser. With it, you can build anything from small command line tools to HTTP servers that power dynamic websites.

These chapters aim to teach you the main concepts that Node.js uses and to give you enough information to write useful programs for it. They do not try to be a complete, or even a thorough, treatment of the platform.

If you want to follow along and run the code in this chapter, you’ll need to install Node.js version 18 or higher. To do so, go to <https://nodejs.org/en> and follow the installation instructions for your operating system. You can also find further documentation for Node.js there.

BACKGROUND

When building systems that communicate over the network, the way you manage input and output—that is, the reading and writing of data to and from the network and hard drive—can make a big difference in how quickly a system responds to the user or to network requests.

In such programs, asynchronous programming is often helpful. It allows the program to send and receive data from and to multiple devices at the same time without complicated thread management and synchronization.

Node was initially conceived for the purpose of making asynchronous programming easy and convenient. JavaScript lends itself well to a system like Node. It is one of the few programming languages that does not have a built-in way to do input and output. Thus, JavaScript could be fit onto Node’s rather eccentric approach to network and filesystem programming without ending up with two inconsistent interfaces. In 2009, when Node was being designed, people were already doing callback-based programming in the browser, so the community around the language was used to an asynchronous programming style.

THE NODE COMMAND

When Node.js is installed on a system, it provides a program called `node`, which is used to run JavaScript files. Say you have a file `hello.js`, containing this code:

```
let message = "Hello world";
```

```
console.log(message);
```

You can then run `node` from the command line like this to execute the program:

```
$ node hello.js
Hello world
```

The `console.log` method in Node does something similar to what it does in the browser. It prints out a piece of text. But in Node, the text will go to the process's standard output stream rather than to a browser's JavaScript console. When running `node` from the command line, that means you see the logged values in your terminal.

If you run `node` without giving it a file, it provides you with a prompt at which you can type JavaScript code and immediately see the result.

```
$ node
> 1 + 1
2
> [-1, -2, -3].map(Math.abs)
[1, 2, 3]
> process.exit(0)
$
```

The `process` binding, just like the `console` binding, is available globally in Node. It provides various ways to inspect and manipulate the current program. The `exit` method ends the process and can be given an exit status code, which tells the program that started `node` (in this case, the command line shell) whether the program completed successfully (code zero) or encountered an error (any other code).

To find the command line arguments given to your script, you can read `process.argv`, which is an array of strings. Note that it also includes the name of the `node` command and your script name, so the actual arguments start at index 2. If `showargv.js` contains the statement `console.log(process.argv)`, you could run it like this:

```
$ node showargv.js one --and two
["node", "/tmp/showargv.js", "one", "--and", "two"]
```

All the standard JavaScript global bindings, such as `Array`, `Math`, and `JSON`, are also present in Node's environment. Browser-related functionality, such as `document` or `prompt`, is not.

MODULES

Beyond the bindings I mentioned, such as `console` and `process`, Node puts few additional bindings in the global scope. If you want to access built-in functionality, you have to ask the module system for it.

Node started out using the CommonJS module system, based on the `require` function, which we saw in Chapter 10. It will still use this system by default when you load a `.js` file.

But today, Node also supports the more modern ES module system. When a script's filename ends in `.mjs`, it is considered to be such a module, and you can use `import` and `export` in it (but not `require`). We will use ES modules in this chapter.

When importing a module—whether with `require` or `import`—Node has to resolve the given string to an actual file that it can load. Names

that start with `/`, `./`, or `../` are resolved as files, relative to the current module's path. Here, `.` stands for the current directory, `../` for one directory up, and `/` for the root of the filesystem. If you ask for `../graph.mjs` from the file `/tmp/robot/robot.mjs`, Node will try to load the file `/tmp/robot/graph.mjs`.

When a string that does not look like a relative or absolute path is imported, it is assumed to refer to either a built-in module or a module installed in a `node_modules` directory. For example, importing from `"node:fs"` will give you Node's built-in `filesystem` module. Importing `"robot"` might try to load the library found in `node_modules/robot/`. It's common to install such libraries using NPM, which we'll return to in a moment.

Let's set up a small project consisting of two files. The first one, called `main.mjs`, defines a script that can be called from the command line to reverse a string.

```
import {reverse} from "../reverse.mjs";

// Index 2 holds the first actual command line argument
let argument = process.argv[2];

console.log(reverse(argument));
```

The file `reverse.mjs` defines a library for reversing strings, which can be used both by this command line tool and by other scripts that need direct access to a string-reversing function.

```
export function reverse(string) {
  return Array.from(string).reverse().join("");
}
```

```
}
```

Remember that `export` is used to declare that a binding is part of the module's interface. That allows `main.mjs` to import and use the function.

We can now call our tool like this:

```
$ node main.mjs JavaScript
tpircSavaJ
```

INSTALLING WITH NPM

NPM, introduced in Chapter 10, is an online repository of JavaScript modules, many of which are specifically written for Node. When you install Node on your computer, you also get the `npm` command, which you can use to interact with this repository.

NPM's main use is downloading packages. We saw the `ini` package in Chapter 10. We can use NPM to fetch and install that package on our computer.

```
$ npm install ini
added 1 package in 723ms

$ node
> const {parse} = require("ini");
> parse("x = 1\ny = 2");
{ x: '1', y: '2' }
```


After running `npm install`, NPM will have created a directory called `node_modules`. Inside that directory will be an `ini` directory that contains the library. You can open it and look at the code. When we import `"ini"`, this library is loaded, and we can call its `parse` property to parse a configuration file.

By default, NPM installs packages under the current directory rather than in a central place. If you are used to other package managers, this may seem unusual, but it has advantages—it puts each application in full control of the packages it installs and makes it easier to manage versions and clean up when removing an application.

PACKAGE FILES

After running `npm install` to install some package, you will find not only a `node_modules` directory but also a file called `package.json` in your current directory. It is recommended to have such a file for each project. You can create it manually or run `npm init`. This file contains information about the project, such as its name and version, and lists its dependencies.

The robot simulation from Chapter 7, as modularized in the exercise in Chapter 10, might have a `package.json` file like this:

```
{
  "author": "Marijn Haverbeke",
  "name": "eloquent-javascript-robot",
  "description": "Simulation of a package-delivery robot",
  "version": "1.0.0",
  "main": "run.mjs",
```

```
  "dependencies": {
    "dijkstrajs": "^1.0.1",
    "random-item": "^1.0.0"
  },
  "license": "ISC"
}
```

When you run `npm install` without naming a package to install, NPM will install the dependencies listed in `package.json`. When you install a specific package that is not already listed as a dependency, NPM will add it to `package.json`.

VERSIONS

A `package.json` file lists both the program's own version and versions for its dependencies. Versions are a way to deal with the fact that packages evolve separately, and code written to work with a package as it existed at one point may not work with a later, modified version of the package.

NPM demands that its packages follow a schema called *semantic versioning*, which encodes some information about which versions are *compatible* (don't break the old interface) in the version number. A semantic version consists of three numbers separated by periods, such as `2.3.0`. Every time new functionality is added, the middle number has to be incremented. Every time compatibility is broken, so that existing code that uses the package might not work with the new version, the first number has to be incremented.

A caret character (`^`) in front of the version number for a dependency

in `package.json` indicates that any version compatible with the given number may be installed. For example, "`^2.3.0`" would mean that any version greater than or equal to 2.3.0 and less than 3.0.0 is allowed.

The `npm` command is also used to publish new packages or new versions of packages. If you run `npm publish` in a directory that has a `package.json` file, it will publish a package with the name and version listed in the JSON file to the registry. Anyone can publish packages to NPM—though only under a package name that isn't in use yet, since it wouldn't be good if random people could update existing packages.

This book won't delve further into the details of NPM usage. Refer to <https://npmjs.com> for further documentation and a way to search for packages.

THE FILESYSTEM MODULE

One of the most commonly used built-in modules in Node is the `node:fs` module, which stands for *filesystem*. It exports functions for working with files and directories.

For example, the function called `readFile` reads a file and then calls a callback with the file's contents.

```
import {readFile} from "node:fs";
readFile("file.txt", "utf8", (error, text) => {
  if (error) throw error;
  console.log("The file contains:", text);
});
```

The second argument to `readFile` indicates the *character encoding* used

to decode the file into a string. There are several ways in which text can be encoded to binary data, but most modern systems use UTF-8. Unless you have reasons to believe another encoding is used, pass "`utf8`" when reading a text file. If you do not pass an encoding, Node will assume you are interested in the binary data and will give you a `Buffer` object instead of a string. This is an array-like object that contains numbers representing the bytes (8-bit chunks of data) in the files.

```
import {readFile} from "node:fs";
readFile("file.txt", (error, buffer) => {
  if (error) throw error;
  console.log("The file contained", buffer.length, "bytes.",
    "The first byte is:", buffer[0]);
});
```

A similar function, `writeFile`, is used to write a file to disk.

```
import {writeFile} from "node:fs";
writeFile("graffiti.txt", "Node was here", err => {
  if (err) console.log('Failed to write file: ${err}');
  else console.log("File written.");
});
```

Here it was not necessary to specify the encoding—`writeFile` will assume that when it is given a string to write, rather than a `Buffer` object, it should write it out as text using its default character encoding, which is UTF-8.

The `node:fs` module contains many other useful functions: `readdir` will give you the files in a directory as an array of strings, `stat` will retrieve information about a file, `rename` will rename a file, `unlink` will

remove one, and so on. See the documentation at <https://nodejs.org> for specifics.

Most of these take a callback function as the last parameter, which they call either with an error (the first argument) or with a successful result (the second). As we saw in Chapter 11, there are downsides to this style of programming—the biggest one being that error handling becomes verbose and error prone.

The `node:fs/promises` module exports most of the same functions as the old `node:fs` module but uses promises rather than callback functions.

```
import {readFile} from "node:fs/promises";
readFile("file.txt", "utf8")
  .then(text => console.log("The file contains:", text));
```

Sometimes you don't need asynchronicity and it just gets in the way. Many of the functions in `node:fs` also have a synchronous variant, which has the same name with `Sync` added to the end. For example, the synchronous version of `readFile` is called `readFileSync`.

```
import {readFileSync} from "node:fs";
console.log("The file contains:",
  readFileSync("file.txt", "utf8"));
```

Note that while such a synchronous operation is being performed, your program is stopped entirely. If it should be responding to the user or to other machines on the network, being stuck on a synchronous action might produce annoying delays.

THE HTTP MODULE

Another central module is called `node:http`. It provides functionality for running an HTTP server.

This is all it takes to start an HTTP server:

```
import {createServer} from "node:http";
let server = createServer((request, response) => {
  response.writeHead(200, {"Content-Type": "text/html"});
  response.write(
    <h1>Hello!</h1>
    <p>You asked for <code>${request.url}</code></p>);
  response.end();
});
server.listen(8000);
console.log("Listening! (port 8000)");
```

If you run this script on your own machine, you can point your web browser at `http://localhost:8000/hello` to make a request to your server. It will respond with a small HTML page.

The function passed as the argument to `createServer` is called every time a client connects to the server. The `request` and `response` bindings are objects representing the incoming and outgoing data. The first contains information about the request, such as its `url` property, which tells us to what URL the request was made.

When you open that page in your browser, it sends a request to your own computer. This causes the server function to run and send back a response, which you can then see in the browser.

To send something to the client, you call methods on the `response`

object. The first, `writehead`, will write out the response headers (see Chapter 18). You give it the status code (200 for “OK” in this case) and an object that contains header values. The example sets the `Content-Type` header to inform the client that we’ll be sending back an HTML document.

Next, the actual response body (the document itself) is sent with `response.write`. You’re allowed to call this method multiple times if you want to send the response piece by piece—for example, to stream data to the client as it becomes available. Finally, `response.end` signals the end of the response.

The call to `server.listen` causes the server to start waiting for connections on port 8000. This is why you have to connect to *localhost:8000* to speak to this server, rather than just *localhost*, which would use the default port 80.

When you run this script, the process just sits there and waits. When a script is listening for events—in this case, network connections—`node` will not automatically exit when it reaches the end of the script. To close it, press CTRL-C.

A real web server usually does more than the one in the example—it looks at the request’s method (the `method` property) to see what action the client is trying to perform and looks at the request’s URL to find out on which resource this action is being performed. We’ll see a more advanced server later in this chapter.

The `node:http` module also provides a `request` function that can be used to make HTTP requests. However, it is a lot more cumbersome to use than `fetch`, which we saw in Chapter 18. Fortunately, `fetch` is also

available in Node as a global binding. Unless you want to do something very specific, such as processing the response document piece by piece as the data comes in over the network, I recommend sticking to `fetch`.

STREAMS

The response object that the HTTP server could write to is an example of a *writable stream* object, which is a widely used concept in Node. Such objects have a `write` method that can be passed a string or a `Buffer` object to write something to the stream. Their `end` method closes the stream and optionally takes a value to write to the stream before closing. Both of these methods can also be given a callback as an additional argument, which they will call when the writing or closing has finished.

It is possible to create a writable stream that points at a file with the `createWriteStream` function from the `node:fs` module. You can then use the `write` method on the resulting object to write the file one piece at a time rather than in one shot, as with `writeFile`.

Readable streams are a little more involved. The `request` argument to the HTTP server’s callback is a readable stream. Reading from a stream is done using event handlers rather than methods.

Objects that emit events in Node have a method called `on` that is similar to the `addEventListener` method in the browser. You give it an event name and then a function, and it will register that function to be called whenever the given event occurs.

Readable streams have “`data`” and “`end`” events. The first is fired ev-

ery time data comes in, and the second is called whenever the stream is at its end. This model is most suited for *streaming* data that can be immediately processed, even when the whole document isn't available yet. A file can be read as a readable stream by using the `createReadStream` function from `node:fs`.

This code creates a server that reads request bodies and streams them back to the client as all-uppercase text:

```
import {createServer} from "node:http";
createServer((request, response) => {
  response.writeHead(200, {"Content-Type": "text/plain"});
  request.on("data", chunk =>
    response.write(chunk.toString().toUpperCase()));
  request.on("end", () => response.end());
}).listen(8000);
```

The chunk value passed to the data handler will be a binary `Buffer`. We can convert this to a string by decoding it as UTF-8 encoded characters with its `toString` method.

The following piece of code, when run with the uppercasing server active, will send a request to that server and write out the response it gets:

```
fetch("http://localhost:8000/", {
  method: "POST",
  body: "Hello server"
}).then(resp => resp.text()).then(console.log);
// → HELLO SERVER
```

A FILE SERVER

Let's combine our newfound knowledge about HTTP servers and working with the filesystem to create a bridge between the two: an HTTP server that allows remote access to a filesystem. Such a server has all kinds of uses—it allows web applications to store and share data, or it can give a group of people shared access to a bunch of files.

When we treat files as HTTP resources, the HTTP methods `GET`, `PUT`, and `DELETE` can be used to read, write, and delete the files, respectively. We will interpret the path in the request as the path of the file that the request refers to.

We probably don't want to share our whole filesystem, so we'll interpret these paths as starting in the server's working directory, which is the directory in which it was started. If I ran the server from `/tmp/public/` (or `C:\tmp\public\` on Windows), then a request for `/file.txt` should refer to `/tmp/public/file.txt` (or `C:\tmp\public\file.txt`).

We'll build the program piece by piece, using an object called `methods` to store the functions that handle the various HTTP methods. Method handlers are `async` functions that get the request object as their argument and return a promise that resolves to an object that describes the response.

```
import {createServer} from "node:http";

const methods = Object.create(null);

createServer((request, response) => {
  let handler = methods[request.method] || notAllowed;
```

```

handler(request).catch(error => {
  if (error.status != null) return error;
  return {body: String(error), status: 500};
}).then(({body, status = 200, type = "text/plain"}) => {
  response.writeHead(status, {"Content-Type": type});
  if (body?.pipe) body.pipe(response);
  else response.end(body);
});
}).listen(8000);

async function notAllowed(request) {
  return {
    status: 405,
    body: `Method ${request.method} not allowed.`
  };
}

```

This starts a server that just returns 405 error responses, which is the code used to indicate that the server refuses to handle a given method.

When a request handler's promise is rejected, the `catch` call translates the error into a response object, if it isn't one already, so that the server can send back an error response to inform the client that it failed to handle the request.

The `status` field of the response description may be omitted, in which case it defaults to 200 (OK). The content type, in the `type` property, can also be left off, in which case the response is assumed to be plain text.

When the value of `body` is a readable stream, it will have a `pipe` method that we can use to forward all content from a readable stream

to a writable stream. If not, it is assumed to be either `null` (no body), a string, or a buffer, and it is passed directly to the response's `end` method.

To figure out which file path corresponds to a request URL, the `urlPath` function uses the built-in `URL` class (which also exists in the browser) to parse the URL. This constructor expects a full URL, not just the part starting with the slash that we get from `request.url`, so we give it a dummy domain name to fill in. It extracts its pathname, which will be something like `"/file.txt"`, decodes that to get rid of the `%`-style escape codes, and resolves it relative to the program's working directory.

```

import {resolve, sep} from "node:path";

const baseDirectory = process.cwd();

function urlPath(url) {
  let {pathname} = new URL(url, "http://d");
  let path = resolve(decodeURIComponent(pathname).slice(1));
  if (path != baseDirectory &&
      !path.startsWith(baseDirectory + sep)) {
    throw {status: 403, body: "Forbidden"};
  }
  return path;
}

```

As soon as you set up a program to accept network requests, you have to start worrying about security. In this case, if we aren't careful, it is likely that we'll accidentally expose our whole filesystem to the network.

File paths are strings in Node. To map such a string to an actual file, there's a nontrivial amount of interpretation going on. Paths may, for example, include `../` to refer to a parent directory. One obvious source of problems would be requests for paths like `../secret_file`.

To avoid such problems, `urlPath` uses the `resolve` function from the `node:path` module, which resolves relative paths. It then verifies that the result is *below* the working directory. The `process.cwd` function (where `cwd` stands for *current working directory*) can be used to find this working directory. The `sep` binding from the `node:path` package is the system's path separator—a backslash on Windows and a forward slash on most other systems. When the path doesn't start with the base directory, the function throws an error response object, using the HTTP status code indicating that access to the resource is forbidden.

We'll set up the `GET` method to return a list of files when reading a directory and to return the file's content when reading a regular file.

One tricky question is what kind of `Content-Type` header we should set when returning a file's content. Since these files could be anything, our server can't simply return the same content type for all of them. NPM can help us again here. The `mime-types` package (content type indicators like `text/plain` are also called *MIME types*) knows the correct type for a large number of file extensions.

The following `npm` command, in the directory where the server script lives, installs a specific version of `mime`:

```
$ npm install mime-types@2.1.0
```

When a requested file does not exist, the correct HTTP status code to

return is 404. We'll use the `stat` function, which looks up information about a file, to find out both whether the file exists and whether it is a directory.

```
import {createReadStream} from "node:fs";
import {stat, readdir} from "node:fs/promises";
import {lookup} from "mime-types";

methods.GET = async function(request) {
  let path = urlPath(request.url);
  let stats;
  try {
    stats = await stat(path);
  } catch (error) {
    if (error.code !== "ENOENT") throw error;
    else return {status: 404, body: "File not found"};
  }
  if (stats.isDirectory()) {
    return {body: (await readdir(path)).join("\n")};
  } else {
    return {body: createReadStream(path),
            type: lookup(path)};
  }
};
```

Because it has to touch the disk and thus might take a while, `stat` is asynchronous. Since we're using promises rather than callback style, it has to be imported from `node:fs/promises` instead of directly from `node:fs`.

When the file does not exist, `stat` will throw an error object with

a code property of "ENOENT". These somewhat obscure, Unix-inspired codes are how you recognize error types in Node.

The `stats` object returned by `stat` tells us a number of things about a file, such as its size (`size` property) and its modification date (`mtime` property). Here we are interested in the question of whether it is a directory or a regular file, which the `isDirectory` method tells us.

We use `readdir` to read the array of files in a directory and return it to the client. For normal files, we create a readable stream with `createReadStream` and return that as the body, along with the content type that the `mime` package gives us for the file's name.

The code to handle DELETE requests is slightly simpler.

```
import {rmdir, unlink} from "node:fs/promises";

methods.DELETE = async function(request) {
  let path = urlPath(request.url);
  let stats;
  try {
    stats = await stat(path);
  } catch (error) {
    if (error.code !== "ENOENT") throw error;
    else return {status: 204};
  }
  if (stats.isDirectory()) await rmdir(path);
  else await unlink(path);
  return {status: 204};
};
```

When an HTTP response does not contain any data, the status code

204 ("no content") can be used to indicate this. Since the response to deletion doesn't need to transmit any information beyond whether the operation succeeded, that is a sensible thing to return here.

You may be wondering why trying to delete a nonexistent file returns a success status code rather than an error. When the file being deleted is not there, you could say that the request's objective is already fulfilled. The HTTP standard encourages us to make requests *idempotent*, which means that making the same request multiple times produces the same result as making it once. In a way, if you try to delete something that's already gone, the effect you were trying to create has been achieved—the thing is no longer there.

This is the handler for PUT requests:

```
import {createWriteStream} from "node:fs";

function pipeStream(from, to) {
  return new Promise((resolve, reject) => {
    from.on("error", reject);
    to.on("error", reject);
    to.on("finish", resolve);
    from.pipe(to);
  });
}

methods.PUT = async function(request) {
  let path = urlPath(request.url);
  await pipeStream(request, createWriteStream(path));
  return {status: 204};
};
```


We don't need to check whether the file exists this time—if it does, we'll just overwrite it. We again use `pipe` to move data from a readable stream to a writable one, in this case from the request to the file. But since `pipe` isn't written to return a promise, we have to write a wrapper, `pipeStream`, that creates a promise around the outcome of calling `pipe`. When something goes wrong when opening the file, `createWriteStream` will still return a stream, but that stream will fire an `"error"` event. The stream from the request may also fail—for example, if the network goes down. So we wire up both streams' `"error"` events to reject the promise. When `pipe` is done, it will close the output stream, which causes it to fire a `"finish"` event. That's the point at which we can successfully resolve the promise (returning nothing).

The full script for the server is available at https://eloquentjavascript.net/code/file_server.mjs. You can download that and, after installing its dependencies, run it with Node to start your own file server. And, of course, you can modify and extend it to solve this chapter's exercises or to experiment.

The command line tool `curl`, widely available on Unix-like systems (such as macOS and Linux), can be used to make HTTP requests. The following session briefly tests our server. The `-X` option is used to set the request's method, and `-d` is used to include a request body.

```
$ curl http://localhost:8000/file.txt
File not found
$ curl -X PUT -d CONTENT http://localhost:8000/file.txt
$ curl http://localhost:8000/file.txt
CONTENT
$ curl -X DELETE http://localhost:8000/file.txt
```

```
$ curl http://localhost:8000/file.txt
File not found
```

The first request for `file.txt` fails since the file does not exist yet. The `PUT` request creates the file, and behold, the next request successfully retrieves it. After deleting it with a `DELETE` request, the file is again missing.

SUMMARY

Node is a nice, small system that lets us run JavaScript in a nonbrowser context. It was originally designed for network tasks to play the role of a node in a network, but it lends itself to all kinds of scripting tasks. If writing JavaScript is something you enjoy, automating tasks with Node may work well for you.

NPM provides packages for everything you can think of (and quite a few things you'd probably never think of), and it allows you to fetch and install those packages with the `npm` program. Node comes with a number of built-in modules, including the `node:fs` module for working with the filesystem and the `node:http` module for running HTTP servers.

All input and output in Node is done asynchronously, unless you explicitly use a synchronous variant of a function, such as `readFileSync`. Node originally used callbacks for asynchronous functionality, but the `node:fs/promises` package provides a promise-based interface to the filesystem.

EXERCISES

SEARCH TOOL

On Unix systems, there is a command line tool called `grep` that can be used to quickly search files for a regular expression.

Write a Node script that can be run from the command line and acts somewhat like `grep`. It treats its first command line argument as a regular expression and treats any further arguments as files to search. It outputs the names of any file whose content matches the regular expression.

When that works, extend it so that when one of the arguments is a directory, it searches through all files in that directory and its subdirectories.

Use asynchronous or synchronous filesystem functions as you see fit. Setting things up so that multiple asynchronous actions are requested at the same time might speed things up a little, but not a huge amount, since most filesystems can read only one thing at a time.

DIRECTORY CREATION

Though the `DELETE` method in our file server is able to delete directories (using `rmdir`), the server currently does not provide any way to *create* a directory.

Add support for the `MKCOL` method (“make collection”), which should create a directory by calling `mkdir` from the `node:fs` module. `MKCOL` is not a widely used HTTP method, but it does exist for this same purpose

in the *WebDAV* standard, which specifies a set of conventions on top of HTTP that make it suitable for creating documents.

A PUBLIC SPACE ON THE WEB

Since the file server serves up any kind of file and even includes the right `Content-Type` header, you can use it to serve a website. Given that this server allows everybody to delete and replace files, this would make for an interesting kind of website: one that can be modified, improved, and vandalized by everybody who takes the time to make the right HTTP request.

Write a basic HTML page that includes a simple JavaScript file. Put the files in a directory served by the file server and open them in your browser.

Next, as an advanced exercise or even a weekend project, combine all the knowledge you gained from this book to build a more user-friendly interface for modifying the website—from *inside* the website.

Use an HTML form to edit the content of the files that make up the website, allowing the user to update them on the server by using HTTP requests, as described in Chapter 18.

Start by making only a single file editable. Then make it so that the user can select which file to edit. Use the fact that our file server returns lists of files when reading a directory.

Don’t work directly in the code exposed by the file server, since if you make a mistake, you are likely to damage the files there. Instead, keep your work outside of the publicly accessible directory and copy it there when testing.

"If you have knowledge, let others light their candles at it."
—Margaret Fuller

CHAPTER 21

PROJECT: SKILL-SHARING WEBSITE

A *skill-sharing* meeting is an event where people with a shared interest come together and give small, informal presentations about things they know. At a gardening skill-sharing meeting, someone might explain how to cultivate celery. Or in a programming skill-sharing group, you could drop by and tell people about Node.js.

In this final project chapter, our goal is to set up a website for managing talks given at a skill-sharing meeting. Imagine a small group of people meeting up regularly in the office of one of the members to talk about unicycling. The previous organizer of the meetings moved to another town, and nobody stepped forward to take over this task. We want a system that will let the participants propose and discuss talks among themselves without an active organizer.

The full code for the project can be downloaded from <https://eloquentjavascript.org/code/skillsharing.zip>.

DESIGN

There is a *server* part to this project, written for Node.js, and a *client* part, written for the browser. The server stores the system's data and provides it to the client. It also serves the files that implement the client-side system.

The server keeps the list of talks proposed for the next meeting, and the client shows this list. Each talk has a presenter name, a title, a summary, and an array of comments associated with it. The client allows users to propose new talks (adding them to the list), delete talks, and comment on existing talks. Whenever the user makes such a change, the client makes an HTTP request to tell the server about it.

Skill Sharing

Your name:

Fatma

Unituning

Delete

by Jamal

Modifying your cycle for extra style

Iman: Will you talk about raising a cycle?

Jamal: Definitely

Iman: I'll be there

Add comment

Submit a talk

Title:

Summary:

Submit

The application will be set up to show a *live* view of the current proposed talks and their comments. Whenever someone, somewhere, submits a new talk or adds a comment, all people who have the page open in their browsers should immediately see the change. This poses a bit of a challenge—there is no way for a web server to open a connection to a client, nor is there a good way to know which clients are currently looking at a given website.

A common solution to this problem is called *long polling*, which hap-

pens to be one of the motivations for Node’s design.

LONG POLLING

To be able to immediately notify a client that something changed, we need a connection to that client. Since web browsers do not traditionally accept connections and clients are often behind routers that would block such connections anyway, having the server initiate this connection is not practical.

We can arrange for the client to open the connection and keep it around so that the server can use it to send information when it needs to do so. But an HTTP request allows only a simple flow of information: the client sends a request, the server comes back with a single response, and that’s it. A technology called *WebSockets* makes it possible to open connections for arbitrary data exchange, but using such sockets properly is somewhat tricky.

In this chapter, we use a simpler technique, long polling, where clients continuously ask the server for new information using regular HTTP requests, and the server stalls its answer when it has nothing new to report.

As long as the client makes sure it constantly has a polling request open, it will receive information from the server quickly after it becomes available. For example, if Fatma has our skill-sharing application open in her browser, that browser will have made a request for updates and will be waiting for a response to that request. When Iman submits a talk on Extreme Downhill Unicycling, the server will notice that Fatma

is waiting for updates and send a response containing the new talk to her pending request. Fatma's browser will receive the data and update the screen to show the talk.

To prevent connections from timing out (being aborted because of a lack of activity), long polling techniques usually set a maximum time for each request, after which the server will respond anyway, even though it has nothing to report. The client can then start a new request. Periodically restarting the request also makes the technique more robust, allowing clients to recover from temporary connection failures or server problems.

A busy server that is using long polling may have thousands of waiting requests, and thus TCP connections, open. Node, which makes it easy to manage many connections without creating a separate thread of control for each one, is a good fit for such a system.

HTTP INTERFACE

Before we start designing either the server or the client, let's think about the point where they touch: the HTTP interface over which they communicate.

We will use JSON as the format of our request and response body. Like in the file server from Chapter 20, we'll try to make good use of HTTP methods and headers. The interface is centered around the `/talks` path. Paths that do not start with `/talks` will be used for serving static files—the HTML and JavaScript code for the client-side system.

A GET request to `/talks` returns a JSON document like this:

```
[{"title": "Unituning",
  "presenter": "Jamal",
  "summary": "Modifying your cycle for extra style",
  "comments": []}]
```

Creating a new talk is done by making a PUT request to a URL like `/talks/unituning`, where the part after the second slash is the title of the talk. The PUT request's body should contain a JSON object that has `presenter` and `summary` properties.

Since talk titles may contain spaces and other characters that may not appear normally in a URL, title strings must be encoded with the `encodeURIComponent` function when building up such a URL.

```
console.log("/talks/" + encodeURIComponent("How to Idle"));
// → /talks/How%20to%20Idle
```

A request to create a talk about idling might look something like this:

```
PUT /talks/How%20to%20Idle HTTP/1.1
Content-Type: application/json
Content-Length: 92
```

```
{"presenter": "Maureen",
  "summary": "Standing still on a unicycle"}
```

Such URLs also support GET requests to retrieve the JSON representation of a talk and DELETE requests to delete a talk.

Adding a comment to a talk is done with a POST request to a URL like `/talks/unituning/comments`, with a JSON body that has `author` and `message` properties.

```
POST /talks/Unituning/comments HTTP/1.1
Content-Type: application/json
Content-Length: 72
```

```
{"author": "Iman",
 "message": "Will you talk about raising a cycle?"}
```

To support long polling, GET requests to `/talks` may include extra headers that inform the server to delay the response if no new information is available. We'll use a pair of headers normally intended to manage caching: `ETag` and `If-None-Match`.

Servers may include an `ETag` (“entity tag”) header in a response. Its value is a string that identifies the current version of the resource. Clients, when they later request that resource again, may make a *conditional request* by including an `If-None-Match` header whose value holds that same string. If the resource hasn't changed, the server will respond with status code 304, which means “not modified”, telling the client that its cached version is still current. When the tag does not match, the server responds as normal.

We need something like this, where the client can tell the server which version of the list of talks it has, and the server responds only when that list has changed. But instead of immediately returning a 304 response, the server should stall the response and return only when something new is available or a given amount of time has elapsed. To distinguish long polling requests from normal conditional requests, we give them another header, `Prefer: wait=90`, which tells the server that the client is willing to wait up to 90 seconds for the response.

The server will keep a version number that it updates every time

the talks change and will use that as the `ETag` value. Clients can make requests like this to be notified when the talks change:

```
GET /talks HTTP/1.1
If-None-Match: "4"
Prefer: wait=90
```

(time passes)

```
HTTP/1.1 200 OK
Content-Type: application/json
ETag: "5"
Content-Length: 295
```

[...]

The protocol described here doesn't do any access control. Everybody can comment, modify talks, and even delete them. (Since the internet is full of hooligans, putting such a system online without further protection probably wouldn't end well.)

THE SERVER

Let's start by building the server-side part of the program. The code in this section runs on Node.js.

ROUTING

Our server will use Node's `createServer` to start an HTTP server. In the function that handles a new request, we must distinguish between the various kinds of requests (as determined by the method and the path) that we support. This can be done with a long chain of `if` statements, but there's a nicer way.

A *router* is a component that helps dispatch a request to the function that can handle it. You can tell the router, for example, that `PUT` requests with a path that matches the regular expression `/^\/talks\/([^\/]*)$/` (`/talks/` followed by a talk title) can be handled by a given function. In addition, it can help extract the meaningful parts of the path (in this case the talk title), wrapped in parentheses in the regular expression, and pass them to the handler function.

There are a number of good router packages on NPM, but here we'll write one ourselves to illustrate the principle.

This is `router.mjs`, which we will later `import` from our server module:

```
export class Router {
  constructor() {
    this.routes = [];
  }
  add(method, url, handler) {
    this.routes.push({method, url, handler});
  }
  async resolve(request, context) {
    let {pathname} = new URL(request.url, "http://d");
    for (let {method, url, handler} of this.routes) {
      let match = url.exec(pathname);
```

```
      if (!match || request.method !== method) continue;
      let parts = match.slice(1).map(decodeURIComponent);
      return handler(context, ...parts, request);
    }
  }
}
```

The module exports the `Router` class. A router object allows you to register handlers for specific methods and URL patterns with its `add` method. When a request is resolved with the `resolve` method, the router calls the handler whose method and URL match the request and return its result.

Handler functions are called with the `context` value given to `resolve`. We will use this to give them access to our server state. Additionally, they receive the match strings for any groups they defined in their regular expression, and the request object. The strings have to be URL-decoded, since the raw URL may contain `%20`-style codes.

SERVING FILES

When a request matches none of the request types defined in our router, the server must interpret it as a request for a file in the `public` directory. It would be possible to use the file server defined in [Chapter 20](#) to serve such files, but we neither need nor want to support `PUT` and `DELETE` requests on files, and we would like to have advanced features such as support for caching. Let's use a solid, well-tested static file server from NPM instead.

I opted for `serve-static`. This isn't the only such server on NPM,

but it works well and fits our purposes. The `serve-static` package exports a function that can be called with a root directory to produce a request handler function. The handler function accepts the `request` and `response` arguments provided by the server from `"node:http"`, and a third argument, a function that it will call if no file matches the request. We want our server to first check for requests we should handle specially, as defined in the router, so we wrap it in another function.

```
import {createServer} from "node:http";
import serveStatic from "serve-static";

function notFound(request, response) {
  response.writeHead(404, "Not found");
  response.end("<h1>Not found</h1>");
}

class SkillShareServer {
  constructor(talks) {
    this.talks = talks;
    this.version = 0;
    this.waiting = [];

    let fileServer = serveStatic("./public");
    this.server = createServer((request, response) => {
      serveFromRouter(this, request, response, () => {
        fileServer(request, response,
          () => notFound(request, response));
      });
    });
  }
}
```

562

```
start(port) {
  this.server.listen(port);
}
stop() {
  this.server.close();
}
}
```

The `serveFromRouter` function has the same interface as `fileServer`, taking `(request, response, next)` arguments. We can use this to “chain” several request handlers, allowing each to either handle the request or pass responsibility for that on to the next handler. The final handler, `notFound`, simply responds with a “not found” error.

Our `serveFromRouter` function uses a similar convention to the `file server` from the [previous chapter](#) for responses—handlers in the router return promises that resolve to objects describing the response.

```
import {Router} from "../router.mjs";

const router = new Router();
const defaultHeaders = {"Content-Type": "text/plain"};

async function serveFromRouter(server, request,
                                response, next) {
  let resolved = await router.resolve(request, server)
    .catch(error => {
      if (error.status !== null) return error;
      return {body: String(err), status: 500};
    });
  if (!resolved) return next();
}
```

563


```

let {body, status = 200, headers = defaultHeaders} =
  await resolved;
response.writeHead(status, headers);
response.end(body);
}

```

TALKS AS RESOURCES

The talks that have been proposed are stored in the `talks` property of the server, an object whose property names are the talk titles. We'll add some handlers to our router that expose these as HTTP resources under `/talks/<title>`.

The handler for requests that GET a single talk must look up the talk and respond either with the talk's JSON data or with a 404 error response.

```

const talkPath = /^\/talks\/([^\/]*)$/;

router.add("GET", talkPath, async (server, title) => {
  if (Object.hasOwn(server.talks, title)) {
    return {body: JSON.stringify(server.talks[title]),
      headers: {"Content-Type": "application/json"}};
  } else {
    return {status: 404, body: `No talk '${title}' found`};
  }
});

```

Deleting a talk is done by removing it from the `talks` object.

```

router.add("DELETE", talkPath, async (server, title) => {
  if (Object.hasOwn(server.talks, title)) {
    delete server.talks[title];
    server.updated();
  }
  return {status: 204};
});

```

The `updated` method, which we will define later, notifies waiting long polling requests about the change.

One handler that needs to read request bodies is the `PUT` handler, which is used to create new talks. It has to check whether the data it was given has `presenter` and `summary` properties, which are strings. Any data coming from outside the system might be nonsense, and we don't want to corrupt our internal data model or crash when bad requests come in.

If the data looks valid, the handler stores an object that represents the new talk in the `talks` object, possibly overwriting an existing talk with this title, and again calls `updated`.

To read the body from the request stream, we will use the `json` function from `"node:stream/consumers"`, which collects the data in the stream and then parses it as JSON. There are similar exports called `text` (to read the content as a string) and `buffer` (to read it as binary data) in this package. Since `json` is a very generic name, the import renames it to `readJSON` to avoid confusion.

```

import {json as readJSON} from "node:stream/consumers";

```

```

router.add("PUT", talkPath,
  async (server, title, request) => {
    let talk = await readJSON(request);
    if (!talk ||
      typeof talk.presenter !== "string" ||
      typeof talk.summary !== "string") {
      return {status: 400, body: "Bad talk data"};
    }
    server.talks[title] = {
      title,
      presenter: talk.presenter,
      summary: talk.summary,
      comments: []
    };
    server.updated();
    return {status: 204};
  });

```

Adding a comment to a talk works similarly. We use `readJSON` to get the content of the request, validate the resulting data, and store it as a comment when it looks valid.

```

router.add("POST", /^\/talks\/([^\s]+)\/comments$/,
  async (server, title, request) => {
    let comment = await readJSON(request);
    if (!comment ||
      typeof comment.author !== "string" ||
      typeof comment.message !== "string") {
      return {status: 400, body: "Bad comment data"};
    } else if (Object.hasOwn(server.talks, title)) {

```

```

    server.talks[title].comments.push(comment);
    server.updated();
    return {status: 204};
  } else {
    return {status: 404, body: `No talk '${title}' found`};
  }
});

```

Trying to add a comment to a nonexistent talk returns a 404 error.

LONG POLLING SUPPORT

The most interesting aspect of the server is the part that handles long polling. When a GET request comes in for `/talks`, it may be either a regular request or a long polling request.

There will be multiple places in which we have to send an array of talks to the client, so we first define a helper method that builds up such an array and includes an ETag header in the response.

```

SkillShareServer.prototype.talkResponse = function() {
  let talks = Object.keys(this.talks)
    .map(title => this.talks[title]);
  return {
    body: JSON.stringify(talks),
    headers: {"Content-Type": "application/json",
      "ETag": `"${this.version}"`,
      "Cache-Control": "no-store"}
  };
};

```

The handler itself needs to look at the request headers to see whether `If-None-Match` and `Prefer` headers are present. Node stores headers, whose names are specified to be case insensitive, under their lowercase names.

```
router.add("GET", /^\/talks$/, async (server, request) => {
  let tag = /"(.*)"/.exec(request.headers["if-none-match"]);
  let wait = /\bwait=(\d+)/.exec(request.headers["prefer"]);
  if (!tag || tag[1] !== server.version) {
    return server.talkResponse();
  } else if (!wait) {
    return {status: 304};
  } else {
    return server.waitForChanges(Number(wait[1]));
  }
});
```

If no tag was given or a tag was given that doesn't match the server's current version, the handler responds with the list of talks. If the request is conditional and the talks did not change, we consult the `Prefer` header to see whether we should delay the response or respond right away.

Callback functions for delayed requests are stored in the server's `waiting` array so that they can be notified when something happens. The `waitForChanges` method also immediately sets a timer to respond with a 304 status when the request has waited long enough.

```
SkillShareServer.prototype.waitForChanges = function(time) {
  return new Promise(resolve => {
```

```
    this.waiting.push(resolve);
    setTimeout(() => {
      if (!this.waiting.includes(resolve)) return;
      this.waiting = this.waiting.filter(r => r !== resolve);
      resolve({status: 304});
    }, time * 1000);
  });
};
```

Registering a change with `updated` increases the version property and wakes up all waiting requests.

```
SkillShareServer.prototype.updated = function() {
  this.version++;
  let response = this.talkResponse();
  this.waiting.forEach(resolve => resolve(response));
  this.waiting = [];
};
```

That concludes the server code. If we create an instance of `SkillShareServer` and start it on port 8000, the resulting HTTP server serves files from the `public` subdirectory alongside a talk-managing interface under the `/talks` URL.

```
new SkillShareServer({}).start(8000);
```

THE CLIENT

The client-side part of the skill-sharing website consists of three files: a tiny HTML page, a style sheet, and a JavaScript file.

HTML

It is a widely used convention for web servers to try to serve a file named `index.html` when a request is made directly to a path that corresponds to a directory. The file server module we use, `serve-static`, supports this convention. When a request is made to the path `/`, the server looks for the file `./public/index.html` (`./public` being the root we gave it) and returns that file if found.

Thus, if we want a page to show up when a browser is pointed at our server, we should put it in `public/index.html`. This is our index file:

```
<!doctype html>
<meta charset="utf-8">
<title>Skill Sharing</title>
<link rel="stylesheet" href="skillsharing.css">

<h1>Skill Sharing</h1>

<script src="skillsharing_client.js"></script>
```

It defines the document title and includes a style sheet, which defines a few styles to, among other things, make sure there is some space between talks. It then adds a heading at the top of the page and loads the script that contains the client-side application.

ACTIONS

The application state consists of the list of talks and the name of the user, and we'll store it in a `{talks, user}` object. We don't allow the user interface to directly manipulate the state or send off HTTP requests. Rather, it may emit *actions* that describe what the user is trying to do.

The `handleAction` function takes such an action and makes it happen. Because our state updates are so simple, state changes are handled in the same function.

```
function handleAction(state, action) {
  if (action.type == "setUser") {
    localStorage.setItem("userName", action.user);
    return {...state, user: action.user};
  } else if (action.type == "setTalks") {
    return {...state, talks: action.talks};
  } else if (action.type == "newTalk") {
    fetchOK(talkURL(action.title), {
      method: "PUT",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({
        presenter: state.user,
        summary: action.summary
      })
    }).catch(reportError);
  } else if (action.type == "deleteTalk") {
    fetchOK(talkURL(action.talk), {method: "DELETE"})
      .catch(reportError);
  } else if (action.type == "newComment") {
```

```

    fetchOK(talkURL(action.talk) + "/comments", {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({
        author: state.user,
        message: action.message
      })
    }).catch(reportError);
  }
  return state;
}

```

We'll store the user's name in `localStorage` so that it can be restored when the page is loaded.

The actions that need to involve the server make network requests, using `fetch`, to the HTTP interface described earlier. We use a wrapper function, `fetchOK`, which makes sure the returned promise is rejected when the server returns an error code.

```

function fetchOK(url, options) {
  return fetch(url, options).then(response => {
    if (response.status < 400) return response;
    else throw new Error(response.statusText);
  });
}

```

This helper function is used to build up a URL for a talk with a given title.

```

function talkURL(title) {

```

```

  return "talks/" + encodeURIComponent(title);
}

```

When the request fails, we don't want our page to just sit there doing nothing without explanation. The function called `reportError`, which we used as the catch handler, shows the user a crude dialog to tell them something went wrong.

```

function reportError(error) {
  alert(String(error));
}

```

RENDERING COMPONENTS

We'll use an approach similar to the one we saw in [Chapter 19](#), splitting the application into components. However, since some of the components either never need to update or are always fully redrawn when updated, we'll define those not as classes but as functions that directly return a DOM node. For example, here is a component that shows the field where the user can enter their name:

```

function renderUserField(name, dispatch) {
  return elt("label", {}, "Your name: ", elt("input", {
    type: "text",
    value: name,
    onchange(event) {
      dispatch({type: "setUser", user: event.target.value});
    }
  }));
}

```

```
}
```

The `elt` function used to construct DOM elements is the one we used in Chapter 19.

A similar function is used to render talks, which include a list of comments and a form for adding a new comment.

```
function renderTalk(talk, dispatch) {
  return elt(
    "section", {className: "talk"},
    elt("h2", null, talk.title, " ", elt("button", {
      type: "button",
      onclick() {
        dispatch({type: "deleteTalk", talk: talk.title});
      }
    }, "Delete")),
    elt("div", null, "by ",
      elt("strong", null, talk.presenter)),
    elt("p", null, talk.summary),
    ...talk.comments.map(renderComment),
    elt("form", {
      onsubmit(event) {
        event.preventDefault();
        let form = event.target;
        dispatch({type: "newComment",
          talk: talk.title,
          message: form.elements.comment.value});
        form.reset();
      }
    }, elt("input", {type: "text", name: "comment"}), " ",
```

```
elt("button", {type: "submit"}, "Add comment"))));
}
```

The `"submit"` event handler calls `form.reset` to clear the form's content after creating a `"newComment"` action.

When creating moderately complex pieces of DOM, this style of programming starts to look rather messy. To avoid this, people often use a *templating language*, which allows you to write your interface as an HTML file with some special markers to indicate where dynamic elements go. Or they use *JSX*, a nonstandard JavaScript dialect that allows you to write something very close to HTML tags in your program as if they are JavaScript expressions. Both of these approaches use additional tools to preprocess the code before it can be run, which we will avoid in this chapter.

Comments are simple to render.

```
function renderComment(comment) {
  return elt("p", {className: "comment"},
    elt("strong", null, comment.author),
    ": ", comment.message);
}
```

Finally, the form that the user can use to create a new talk is rendered like this:

```
function renderTalkForm(dispatch) {
  let title = elt("input", {type: "text"});
  let summary = elt("input", {type: "text"});
  return elt("form", {
    onsubmit(event) {
```

```

        event.preventDefault();
        dispatch({type: "newTalk",
                  title: title.value,
                  summary: summary.value});
        event.target.reset();
    }
}, elt("h3", null, "Submit a Talk"),
elt("label", null, "Title: ", title),
elt("label", null, "Summary: ", summary),
elt("button", {type: "submit"}, "Submit"));
}

```

POLLING

To start the app, we need the current list of talks. Since the initial load is closely related to the long polling process—the ETag from the load must be used when polling—we'll write a function that keeps polling the server for /talks and calls a callback function when a new set of talks is available.

```

async function pollTalks(update) {
    let tag = undefined;
    for (;;) {
        let response;
        try {
            response = await fetchOK("/talks", {
                headers: tag && {"If-None-Match": tag,
                               "Prefer": "wait=90"}
            });
        }
    }
}

```

```

    } catch (e) {
        console.log("Request failed: " + e);
        await new Promise(resolve => setTimeout(resolve, 500));
        continue;
    }
    if (response.status == 304) continue;
    tag = response.headers.get("ETag");
    update(await response.json());
}
}

```

This is an `async` function so that looping and waiting for the request is easier. It runs an infinite loop that, on each iteration, retrieves the list of talks—either normally or, if this isn't the first request, with the headers included that make it a long polling request.

When a request fails, the function waits a moment and then tries again. This way, if your network connection goes away for a while and then comes back, the application can recover and continue updating. The promise resolved via `setTimeout` is a way to force the `async` function to wait.

When the server gives back a 304 response, that means a long polling request timed out, so the function should just immediately start the next request. If the response is a normal 200 response, its body is read as JSON and passed to the callback, and its ETag header value is stored for the next iteration.

THE APPLICATION

The following component ties the whole user interface together:

```
class SkillShareApp {
  constructor(state, dispatch) {
    this.dispatch = dispatch;
    this.talkDOM = elt("div", {className: "talks"});
    this.dom = elt("div", null,
      renderUserField(state.user, dispatch),
      this.talkDOM,
      renderTalkForm(dispatch));
    this.syncState(state);
  }

  syncState(state) {
    if (state.talks !== this.talks) {
      this.talkDOM.textContent = "";
      for (let talk of state.talks) {
        this.talkDOM.appendChild(
          renderTalk(talk, this.dispatch));
      }
      this.talks = state.talks;
    }
  }
}
```

When the talks change, this component redraws all of them. This is simple but also wasteful. We'll get back to that in the exercises.

We can start the application like this:

```
function runApp() {
  let user = localStorage.getItem("userName") || "Anon";
  let state, app;
  function dispatch(action) {
    state = handleAction(state, action);
    app.syncState(state);
  }

  pollTalks(talks => {
    if (!app) {
      state = {user, talks};
      app = new SkillShareApp(state, dispatch);
      document.body.appendChild(app.dom);
    } else {
      dispatch({type: "setTalks", talks});
    }
  }).catch(reportError);
}
```

runApp();

If you run the server and open two browser windows for <http://localhost:8080> next to each other, you can see that the actions you perform in one window are immediately visible in the other.

EXERCISES

The following exercises will involve modifying the system defined in this chapter. To work on them, make sure you've downloaded the code

(<https://eloquentjavascript.net/code/skillsharing.zip>), installed Node (<http://nodejs.org>) and installed the project's dependency with `npm install`.

DISK PERSISTENCE

The skill-sharing server keeps its data purely in memory. This means that when it crashes or is restarted for any reason, all talks and comments are lost.

Extend the server so that it stores the talk data to disk and automatically reloads the data when it is restarted. Don't worry about efficiency—do the simplest thing that works.

COMMENT FIELD RESETS

The wholesale redrawing of talks works pretty well because you usually can't tell the difference between a DOM node and its identical replacement. But there are exceptions. If you start typing something in the comment field for a talk in one browser window and then, in another, add a comment to that talk, the field in the first window will be redrawn, removing both its content and its focus.

When multiple people are adding comments at the same time, this would be annoying. Can you come up with a way to solve it?

EXERCISE HINTS

The hints below might help when you are stuck with one of the exercises in this book. They don't give away the entire solution, but rather try to help you find it yourself.

PROGRAM STRUCTURE

LOOPING A TRIANGLE

You can start with a program that prints out the numbers 1 to 7, which you can derive by making a few modifications to the `even number printing` example given earlier in the chapter, where the `for` loop was introduced.

Now consider the equivalence between numbers and strings of hash characters. You can go from 1 to 2 by adding 1 (`+= 1`). You can go from `"#"` to `"##"` by adding a character (`+= "#"`). Thus, your solution can closely follow the number-printing program.

FIZZBUZZ

Going over the numbers is clearly a looping job, and selecting what to print is a matter of conditional execution. Remember the trick of using the remainder (%) operator for checking whether a number is divisible by another number (has a remainder of zero).

In the first version, there are three possible outcomes for every number, so you'll have to create an `if/else if/else` chain.

The second version of the program has a straightforward solution and a clever one. The simple solution is to add another conditional "branch" to precisely test the given condition. For the clever solution, build up a string containing the word or words to output and print either this word or the number if there is no word, potentially by making good use of the `||` operator.

CHESSBOARD

You can build the string by starting with an empty one ("") and repeatedly adding characters. A newline character is written `"\n"`.

To work with two dimensions, you will need a loop inside of a loop. Put braces around the bodies of both loops to make it easy to see where they start and end. Try to properly indent these bodies. The order of the loops must follow the order in which we build up the string (line by line, left to right, top to bottom). So the outer loop handles the lines, and the inner loop handles the characters on a line.

You'll need two bindings to track your progress. To know whether to put a space or a hash sign at a given position, you could test whether

the sum of the two counters is even (`% 2`).

Terminating a line by adding a newline character must happen after the line has been built up, so do this after the inner loop but inside the outer loop.

FUNCTIONS

MINIMUM

If you have trouble putting braces and parentheses in the right place to get a valid function definition, start by copying one of the examples in this chapter and modifying it.

A function may contain multiple `return` statements.

RECURSION

Your function will likely look somewhat similar to the inner `find` function in the recursive `findSolution` example in this chapter, with an `if/else if/else` chain that tests which of the three cases applies. The final `else`, corresponding to the third case, makes the recursive call. Each of the branches should contain a `return` statement or in some other way arrange for a specific value to be returned.

When given a negative number, the function will recurse again and again, passing itself an ever more negative number, thus getting further and further away from returning a result. It will eventually run out of stack space and abort.

BEAN COUNTING

Your function will need a loop that looks at every character in the string. It can run an index from zero to one below its length (`< string.length`). If the character at the current position is the same as the one the function is looking for, it adds 1 to a counter variable. Once the loop has finished, the counter can be returned.

Take care to make all the bindings used in the function *local* to the function by properly declaring them with the `let` or `const` keyword.

DATA STRUCTURES: OBJECTS AND ARRAYS

THE SUM OF A RANGE

Building up an array is most easily done by first initializing a binding to `[]` (a fresh, empty array) and repeatedly calling its `push` method to add a value. Don't forget to return the array at the end of the function.

Since the end boundary is inclusive, you'll need to use the `<=` operator rather than `<` to check for the end of your loop.

The step parameter can be an optional parameter that defaults (using the `=` operator) to 1.

Having *range* understand negative step values is probably best done by writing two separate loops—one for counting up and one for counting down—because the comparison that checks whether the loop is finished needs to be `>=` rather than `<=` when counting downward.

It might also be worthwhile to use a different default step, namely, -1, when the end of the range is smaller than the start. That way,

`range(5, 2)` returns something meaningful rather than getting stuck in an infinite loop. It is possible to refer to previous parameters in the default value of a parameter.

REVERSING AN ARRAY

There are two obvious ways to implement `reverseArray`. The first is to simply go over the input array from front to back and use the `unshift` method on the new array to insert each element at its start. The second is to loop over the input array backward and use the `push` method. Iterating over an array backward requires a (somewhat awkward) *for* specification, like `(let i = array.length - 1; i >= 0; i--)`.

Reversing the array in place is harder. You have to be careful not to overwrite elements that you will later need. Using `reverseArray` or otherwise copying the whole array (`array.slice()` is a good way to copy an array) works but is cheating.

The trick is to *swap* the first and last elements, then the second and second-to-last, and so on. You can do this by looping over half the length of the array (use `Math.floor` to round down—you don't need to touch the middle element in an array with an odd number of elements) and swapping the element at position `i` with the one at position `array.length - 1 - i`. You can use a local binding to briefly hold onto one of the elements, overwrite that one with its mirror image, and then put the value from the local binding in the place where the mirror image used to be.

A LIST

Building up a list is easier when done back to front. So `arrayToList` could iterate over the array backward (see the previous exercise) and, for each element, add an object to the list. You can use a local binding to hold the part of the list that was built so far and use an assignment like `list = {value: x, rest: list}` to add an element.

To run over a list (in `listToArray` and `nth`), a `for` loop specification like this can be used:

```
for (let node = list; node; node = node.rest) {}
```

Can you see how that works? Every iteration of the loop, `node` points to the current sublist, and the body can read its `value` property to get the current element. At the end of an iteration, `node` moves to the next sublist. When that is `null`, we have reached the end of the list, and the loop is finished.

The recursive version of `nth` will, similarly, look at an ever smaller part of the “tail” of the list and at the same time count down the index until it reaches zero, at which point it can return the `value` property of the node it is looking at. To get the zeroth element of a list, you simply take the `value` property of its head node. To get element $N + 1$, you take the N th element of the list that’s in this list’s `rest` property.

DEEP COMPARISON

Your test for whether you are dealing with a real object will look something like `typeof x == "object" && x != null`. Be careful to compare

properties only when *both* arguments are objects. In all other cases you can just immediately return the result of applying `==`.

Use `Object.keys` to go over the properties. You need to test whether both objects have the same set of property names and whether those properties have identical values. One way to do that is to ensure that both objects have the same number of properties (the lengths of the property lists are the same). And then, when looping over one of the object’s properties to compare them, always first make sure the other actually has a property by that name. If they have the same number of properties and all properties in one also exist in the other, they have the same set of property names.

Returning the correct value from the function is best done by immediately returning `false` when a mismatch is found and returning `true` at the end of the function.

HIGHER-ORDER FUNCTIONS

EVERYTHING

Like the `&&` operator, the `every` method can stop evaluating further elements as soon as it has found one that doesn’t match. So the loop-based version can jump out of the loop—with `break` or `return`—as soon as it runs into an element for which the predicate function returns `false`. If the loop runs to its end without finding such an element, we know that all elements matched and we should return `true`.

To build `every` on top of `some`, we can apply *De Morgan’s laws*, which state that `a && b` equals `!(!a || !b)`. This can be generalized to arrays,

where all elements in the array match if there is no element in the array that does not match.

DOMINANT WRITING DIRECTION

Your solution might look a lot like the first half of the `textScripts` example. You again have to count characters by a criterion based on `characterScript` and then filter out the part of the result that refers to uninteresting (script-less) characters.

Finding the direction with the highest character count can be done with `reduce`. If it's not clear how, refer to the example earlier in the chapter, where `reduce` was used to find the script with the most characters.

THE SECRET LIFE OF OBJECTS

A VECTOR TYPE

Look back to the `Rabbit` class example if you're unsure how `class` declarations look.

Adding a getter property to the constructor can be done by putting the word `get` before the method name. To compute the distance from (0, 0) to (x, y), you can use the Pythagorean theorem, which says that the square of the distance we are looking for is equal to the square of the x-coordinate plus the square of the y-coordinate. Thus, $\sqrt{x^2 + y^2}$ is the number you want. `Math.sqrt` is the way you compute a square root in JavaScript and `x ** 2` can be used to square a number.

GROUPS

The easiest way to do this is to store an array of group members in an instance property. The `includes` or `indexOf` methods can be used to check whether a given value is in the array.

Your class's constructor can set the member collection to an empty array. When `add` is called, it must check whether the given value is in the array or add it otherwise, possibly using `push`.

Deleting an element from an array, in `delete`, is less straightforward, but you can use `filter` to create a new array without the value. Don't forget to overwrite the property holding the members with the newly filtered version of the array.

The `from` method can use a `for/of` loop to get the values out of the iterable object and call `add` to put them into a newly created group.

ITERABLE GROUPS

It is probably worthwhile to define a new class `GroupIterator`. Iterator instances should have a property that tracks the current position in the group. Every time `next` is called, it checks whether it is done and, if not, moves past the current value and returns it.

The `Group` class itself gets a method named by `Symbol.iterator` that, when called, returns a new instance of the iterator class for that group.

PROJECT: A ROBOT

MEASURING A ROBOT

You'll have to write a variant of the `runRobot` function that, instead of logging the events to the console, returns the number of steps the robot took to complete the task.

Your measurement function can then, in a loop, generate new states and count the steps each of the robots takes. When it has generated enough measurements, it can use `console.log` to output the average for each robot, which is the total number of steps taken divided by the number of measurements.

ROBOT EFFICIENCY

The main limitation of `goalOrientedRobot` is that it considers only one parcel at a time. It will often walk back and forth across the village because the parcel it happens to be looking at happens to be at the other side of the map, even if there are others much closer.

One possible solution would be to compute routes for all packages and then take the shortest one. Even better results can be obtained, if there are multiple shortest routes, by preferring the ones that go to pick up a package instead of delivering a package.

PERSISTENT GROUP

The most convenient way to represent the set of member values is still as an array, since arrays are easy to copy.

When a value is added to the group, you can create a new group with a copy of the original array that has the value added (for example, using `concat`). When a value is deleted, you filter it from the array.

The class's constructor can take such an array as its argument and store it as the instance's (only) property. This array is never updated.

To add the `empty` property to the constructor, you can declare it as a static property.

You need only one `empty` instance because all empty groups are the same and instances of the class don't change. You can create many different groups from that single empty group without affecting it.

BUGS AND ERRORS

RETRY

The call to `primitiveMultiply` should definitely happen in a `try` block. The corresponding `catch` block should rethrow the exception when it is not an instance of `MultiplicatorUnitFailure` and ensure the call is retried when it is.

To do the retrying, you can either use a loop that stops only when a call succeeds—as in the `look` example earlier in this chapter—or use recursion and hope you don't get a string of failures so long that it overflows the stack (which is a pretty safe bet).

THE LOCKED BOX

This exercise calls for a `finally` block. Your function should first unlock the box and then call the argument function from inside a `try` body. The `finally` block after it should lock the box again.

To make sure we don't lock the box when it wasn't already locked, check its lock at the start of the function and unlock and lock it only when it started out locked.

REGULAR EXPRESSIONS

QUOTING STYLE

The most obvious solution is to replace only quotes with a nonletter character on at least one side—something like `/\P{L}''\P{L}/u`. But you also have to take the start and end of the line into account.

In addition, you must ensure that the replacement also includes the characters that were matched by the `\P{L}` pattern so that those are not dropped. This can be done by wrapping them in parentheses and including their groups in the replacement string (`$1`, `$2`). Groups that are not matched will be replaced by nothing.

NUMBERS AGAIN

First, do not forget the backslash in front of the period.

Matching the optional sign in front of the number, as well as in front of the exponent, can be done with `[+\-]?` or `(\+|-|)` (plus, minus, or nothing).

The more complicated part of the exercise is the problem of matching both `"5."` and `".5"` without also matching `".."`. For this, a good solution is to use the `|` operator to separate the two cases—either one or more digits optionally followed by a dot and zero or more digits *or* a dot followed by one or more digits.

Finally, to make the *e* case insensitive, either add an *i* option to the regular expression or use `[eE]`.

MODULES

A MODULAR ROBOT

Here's what I would have done (but again, there is no single *right* way to design a given module):

The code used to build the road graph lives in the `graph.js` module. Because I'd rather use `dijkstrajs` from NPM than our own pathfinding code, we'll make this build the kind of graph data that `dijkstrajs` expects. This module exports a single function, `buildGraph`. I'd have `buildGraph` accept an array of two-element arrays, rather than strings containing hyphens, to make the module less dependent on the input format.

The `roads.js` module contains the raw road data (the `roads` array) and the `roadGraph` binding. This module depends on `./graph.js` and exports the road graph.

The `VillageState` class lives in the `state.js` module. It depends on the `./roads.js` module because it needs to be able to verify that a given road exists. It also needs `randomPick`. Since that is a three-line

function, we could just put it into the `state.js` module as an internal helper function. But `randomRobot` needs it too. So we'd have to either duplicate it or put it into its own module. Since this function happens to exist on NPM in the `random-item` package, a reasonable solution is to just make both modules depend on that. We can add the `runRobot` function to this module as well, since it's small and closely related to state management. The module exports both the `VillageState` class and the `runRobot` function.

Finally, the robots, along with the values they depend on, such as `mailRoute`, could go into an `example-robots.js` module, which depends on `./roads.js` and exports the robot functions. To make it possible for `goalOrientedRobot` to do route-finding, this module also depends on `dijkstra.js`.

By offloading some work to NPM modules, the code became a little smaller. Each individual module does something rather simple and can be read on its own. Dividing code into modules also often suggests further improvements to the program's design. In this case, it seems a little odd that the `VillageState` and the robots depend on a specific road graph. It might be a better idea to make the graph an argument to the state's constructor and make the robots read it from the state object—this reduces dependencies (which is always good) and makes it possible to run simulations on different maps (which is even better).

Is it a good idea to use NPM modules for things that we could have written ourselves? In principle, yes—for nontrivial things like the pathfinding function you are likely to make mistakes and waste time writing them yourself. For tiny functions like `random-item`, writ-

ing them yourself is easy enough. But adding them wherever you need them does tend to clutter your modules.

However, you should also not underestimate the work involved in *finding* an appropriate NPM package. And even if you find one, it might not work well or may be missing some feature you need. On top of that, depending on NPM packages means you have to make sure they are installed, you have to distribute them with your program, and you might have to periodically upgrade them.

So again, this is a trade-off, and you can decide either way depending on how much a given package actually helps you.

ROADS MODULE

Since this is an ES module, you have to use `import` to access the graph module. That was described as exporting a `buildGraph` function, which you can pick out of its interface object with a destructuring `const` declaration.

To export `roadGraph`, you put the keyword `export` before its definition. Because `buildGraph` takes a data structure that doesn't precisely match `roads`, the splitting of the road strings must happen in your module.

CIRCULAR DEPENDENCIES

The trick is that `require` adds the interface object for a module to its cache *before* it starts loading the module. That way, if any `require` call made while it is running tries to load it, it is already known, and the current interface will be returned, rather than starting to load the

module once more (which would eventually overflow the stack).

ASYNCHRONOUS PROGRAMMING

QUIET TIMES

You will need to convert the content of these files to an array. The easiest way to do that is to use the `split` method on the string produced by `textFile`. Note that for the logfiles, that will still give you an array of strings, which you have to convert to numbers before passing them to new `Date`.

Summarizing all the time points into a table of hours can be done by creating a table (array) that holds a number for each hour in the day. You can then loop over all the timestamps (over the logfiles and the numbers in every logfile) and for each one, if it happened on the correct day, take the hour it occurred in, and add one to the corresponding number in the table.

Make sure you use `await` on the result of asynchronous functions before doing anything with it, or you'll end up with a `Promise` where you expected a string.

REAL PROMISES

The most straightforward approach to writing this function is to use a chain of `then` calls. The first promise is produced by reading the list of logfiles. The first callback can split this list and map `textFile` over it to get an array of promises to pass to `Promise.all`. It can return the

object returned by `Promise.all`, so that whatever that returns becomes the result of the return value of this first `then`.

We now have a promise that returns an array of logfiles. We can call `then` again on that, and put the timestamp-counting logic in there. Something like this:

```
function activityTable(day) {
  return textFile("camera_logs.txt").then(files => {
    return Promise.all(files.split("\n").map(textFile));
  }).then(logs => {
    // analyze...
  });
}
```

Or you could, for even better work scheduling, put the analysis of each file inside of the `Promise.all`, so that that work can be started for the first file that comes back from disk, even before the other files come back.

```
function activityTable(day) {
  let table = []; // init...
  return textFile("camera_logs.txt").then(files => {
    return Promise.all(files.split("\n").map(name => {
      return textFile(name).then(log => {
        // analyze...
      });
    }));
  }).then(() => table);
}
```

This shows that the way you structure your promises can have a real effect on the way the work is scheduled. A simple loop with `await` in it will make the process completely linear—it waits for each file to load before proceeding. `Promise.all` makes it possible for multiple tasks to conceptually be worked on at the same time, allowing them to make progress while files are still being loaded. This can be faster, but it also makes the order in which things will happen less predictable. In this case, we're only going to be incrementing numbers in a table, which isn't hard to do in a safe way. For other kinds of problems, it may be a lot more difficult.

When a file in the list doesn't exist, the promise returned by `textFile` will be rejected. Because `Promise.all` rejects if any of the promises given to it fail, the return value of the callback given to the first `then` will also be a rejected promise. That makes the promise returned by `then fail`, so the callback given to the second `then` isn't even called, and a rejected promise is returned from the function.

BUILDING PROMISE.ALL

The function passed to the `Promise` constructor will have to call `then` on each of the promises in the given array. When one of them succeeds, two things need to happen. The resulting value needs to be stored in the correct position of a result array, and we must check whether this was the last pending promise and finish our own promise if it was.

The latter can be done with a counter that is initialized to the length of the input array and from which we subtract 1 every time a promise succeeds. When it reaches 0, we are done. Make sure you take into

account the situation where the input array is empty (and thus no promise will ever resolve).

Handling failure requires some thought but turns out to be extremely simple. Just pass the `reject` function of the wrapping promise to each of the promises in the array as a `catch` handler or as a second argument to `then` so that a failure in one of them triggers the rejection of the whole wrapper promise.

PROJECT: A PROGRAMMING LANGUAGE

ARRAYS

The easiest way to do this is to represent Egg arrays with JavaScript arrays.

The values added to the top scope must be functions. By using a `rest` argument (with triple-dot notation), the definition of `array` can be very simple.

CLOSURE

Again, we are riding along on a JavaScript mechanism to get the equivalent feature in Egg. Special forms are passed the local scope in which they are evaluated so that they can evaluate their subforms in that scope. The function returned by `fun` has access to the `scope` argument given to its enclosing function and uses that to create the function's local scope when it is called.

This means that the prototype of the local scope will be the scope in which the function was created, which makes it possible to access bindings in that scope from the function. This is all there is to implementing closure (though to compile it in a way that is actually efficient, you'd need to do some more work).

COMMENTS

Make sure your solution handles multiple comments in a row, with whitespace potentially between or after them.

A regular expression is probably the easiest way to solve this. Write something that matches “whitespace or a comment, zero or more times”. Use the `exec` or `match` method and look at the length of the first element in the returned array (the whole match) to find out how many characters to slice off.

FIXING SCOPE

You will have to loop through one scope at a time, using `Object.getPrototypeOf` to go to the next outer scope. For each scope, use `Object.hasOwn` to find out whether the binding, indicated by the `name` property of the first argument to `set`, exists in that scope. If it does, set it to the result of evaluating the second argument to `set` and then return that value.

If the outermost scope is reached (`Object.getPrototypeOf` returns `null`) and we haven't found the binding yet, it doesn't exist, and an error should be thrown.

THE DOCUMENT OBJECT MODEL

BUILD A TABLE

You can use `document.createElement` to create new element nodes, `document.createTextNode` to create text nodes, and the `appendChild` method to put nodes into other nodes.

You'll want to loop over the key names once to fill in the top row and then again for each object in the array to construct the data rows. To get an array of key names from the first object, `Object.keys` will be useful.

To add the table to the correct parent node, you can use `document.getElementById` or `document.querySelector` with “`#mountains`” to find the node.

ELEMENTS BY TAG NAME

The solution is most easily expressed with a recursive function, similar to the `talksAbout` function defined earlier in this chapter.

You could call `byTagName` itself recursively, concatenating the resulting arrays to produce the output. Or you could create an inner function that calls itself recursively and that has access to an array binding defined in the outer function, to which it can add the matching elements it finds. Don't forget to call the inner function once from the outer function to start the process.

The recursive function must check the node type. Here we are interested only in node type 1 (`Node.ELEMENT_NODE`). For such nodes, we

must loop over their children and, for each child, see whether the child matches the query while also doing a recursive call on it to inspect its own children.

THE CAT'S HAT

`Math.cos` and `Math.sin` measure angles in radians, where a full circle is 2π . For a given angle, you can get the opposite angle by adding half of this, which is `Math.PI`. This can be useful for putting the hat on the opposite side of the orbit.

HANDLING EVENTS

BALLOON

You'll want to register a handler for the "keydown" event and look at `event.key` to figure out whether the up or down arrow key was pressed.

The current size can be kept in a binding so that you can base the new size on it. It'll be helpful to define a function that updates the size—both the binding and the style of the balloon in the DOM—so that you can call it from your event handler, and possibly also once when starting, to set the initial size.

You can change the balloon to an explosion by replacing the text node with another one (using `replaceChild`) or by setting the `textContent` property of its parent node to a new string.

MOUSE TRAIL

Creating the elements is best done with a loop. Append them to the document to make them show up. To be able to access them later to change their position, you'll want to store the elements in an array.

Cycling through them can be done by keeping a counter variable and adding 1 to it every time the "mousemove" event fires. The remainder operator (`% elements.length`) can then be used to get a valid array index to pick the element you want to position during a given event.

Another interesting effect can be achieved by modeling a simple physics system. Use the "mousemove" event only to update a pair of bindings that track the mouse position. Then use `requestAnimationFrame` to simulate the trailing elements being attracted to the position of the mouse pointer. At every animation step, update their position based on their position relative to the pointer (and, optionally, a speed that is stored for each element). Figuring out a good way to do this is up to you.

TABS

One pitfall you might run into is that you can't directly use the node's `childNodes` property as a collection of tab nodes. For one thing, when you add the buttons, they will also become child nodes and end up in this object because it is a live data structure. For another, the text nodes created for the whitespace between the nodes are also in `childNodes` but should not get their own tabs. You can use `children` instead of `childNodes` to ignore text nodes.

You could start by building up an array of tabs so that you have easy access to them. To implement the styling of the buttons, you could store objects that contain both the tab panel and its button.

I recommend writing a separate function for changing tabs. You can either store the previously selected tab and change only the styles needed to hide that and show the new one, or you can just update the style of all tabs every time a new tab is selected.

You might want to call this function immediately to make the interface start with the first tab visible.

PROJECT: A PLATFORM GAME

PAUSING THE GAME

An animation can be interrupted by returning `false` from the function given to `runAnimation`. It can be continued by calling `runAnimation` again.

So we need to communicate the fact that we are pausing the game to the function given to `runAnimation`. For that, you can use a binding that both the event handler and that function have access to.

When finding a way to unregister the handlers registered by `trackKeys`, remember that the *exact* same function value that was passed to `addEventListener` must be passed to `removeEventListener` to successfully remove a handler. Thus, the `handler` function value created in `trackKeys` must be available to the code that unregisters the handlers.

You can add a property to the object returned by `trackKeys`, containing either that function value or a method that handles the unregistering

directly.

A MONSTER

If you want to implement a type of motion that is stateful, such as bouncing, make sure you store the necessary state in the actor object—include it as a constructor argument and add it as a property.

Remember that `update` returns a *new* object rather than changing the old one.

When handling collision, find the player in `state.actors` and compare its position to the monster's position. To get the *bottom* of the player, you have to add its vertical size to its vertical position. The creation of an updated state will resemble either `Coin's collide` method (removing the actor) or `Lava's` (changing the status to "lost"), depending on the player position.

DRAWING ON CANVAS

SHAPES

The trapezoid (1) is easiest to draw using a path. Pick suitable center coordinates and add each of the four corners around the center.

The diamond (2) can be drawn the straightforward way, with a path, or the interesting way, with a `rotate` transformation. To use rotation, you will have to apply a trick similar to what we did in the `flipHorizontally` function. Because you want to rotate around the center of your rectangle and not around the point (0, 0), you must first

`translate` to there, then rotate, and then translate back.

Make sure you reset the transformation after drawing any shape that creates one.

For the zigzag (3) it becomes impractical to write a new call to `lineTo` for each line segment. Instead, you should use a loop. You can have each iteration draw either two line segments (right and then left again) or one, in which case you must use the evenness (`% 2`) of the loop index to determine whether to go left or right.

You'll also need a loop for the spiral (4). If you draw a series of points, with each point moving farther along a circle around the spiral's center, you get a circle. If, during the loop, you vary the radius of the circle on which you are putting the current point and go around more than once, the result is a spiral.

The star (5) depicted is built out of `quadraticCurveTo` lines. You could also draw one with straight lines. Divide a circle into eight pieces for a star with eight points, or however many pieces you want. Draw lines between these points, making them curve toward the center of the star. With `quadraticCurveTo`, you can use the center as the control point.

THE PIE CHART

You will need to call `fillText` and set the context's `textAlign` and `textBaseline` properties in such a way that the text ends up where you want it.

A sensible way to position the labels would be to put the text on the line going from the center of the pie through the middle of the slice. You don't want to put the text directly against the side of the pie but

rather move the text out to the side of the pie by a given number of pixels.

The angle of this line is `currentAngle + 0.5 * sliceAngle`. The following code finds a position on this line 120 pixels from the center:

```
let middleAngle = currentAngle + 0.5 * sliceAngle;
let textX = Math.cos(middleAngle) * 120 + centerX;
let textY = Math.sin(middleAngle) * 120 + centerY;
```

For `textBaseline`, the value "middle" is probably appropriate when using this approach. What to use for `textAlign` depends on which side of the circle we are on. On the left, it should be "right", and on the right, it should be "left", so that the text is positioned away from the pie.

If you are not sure how to find out which side of the circle a given angle is on, look to the explanation of `Math.cos` in Chapter 14. The cosine of an angle tells us which x-coordinate it corresponds to, which in turn tells us exactly which side of the circle we are on.

A BOUNCING BALL

A box is easy to draw with `strokeRect`. Define a binding that holds its size, or define two bindings if your box's width and height differ. To create a round ball, start a path and call `arc(x, y, radius, 0, 7)`, which creates an arc going from zero to more than a whole circle. Then fill the path.

To model the ball's position and speed, you can use the `vec` class from Chapter 16. Give it a starting speed, preferably one that is not purely vertical or horizontal, and for every frame multiply that speed

by the amount of time that elapsed. When the ball gets too close to a vertical wall, invert the *x* component in its speed. Likewise, invert the *y* component when it hits a horizontal wall.

After finding the ball's new position and speed, use `clearRect` to delete the scene and redraw it using the new position.

PRECOMPUTED MIRRORING

The key to the solution is the fact that we can use a canvas element as a source image when using `drawImage`. It is possible to create an extra `<canvas>` element, without adding it to the document, and draw our inverted sprites to it, once. When drawing an actual frame, we just copy the already inverted sprites to the main canvas.

Some care would be required because images do not load instantly. We do the inverted drawing only once, and if we do it before the image loads, it won't draw anything. A "load" handler on the image can be used to draw the inverted images to the extra canvas. This canvas can be used as a drawing source immediately (it'll simply be blank until we draw the character onto it).

HTTP AND FORMS

CONTENT NEGOTIATION

Base your code on the `fetch` examples earlier in the chapter.

Asking for a bogus media type will return a response with code 406, "Not acceptable", which is the code a server should return when it can't

fulfill the Accept header.

A JAVASCRIPT WORKBENCH

Use `document.querySelector` or `document.getElementById` to get access to the elements defined in your HTML. An event handler for "click" or "mousedown" events on the button can get the `value` property of the text field and call `Function` on it.

Make sure you wrap both the call to `Function` and the call to its result in a try block so you can catch the exceptions it produces. In this case, we really don't know what type of exception we are looking for, so catch everything.

The `textContent` property of the output element can be used to fill it with a string message. Or, if you want to keep the old content around, create a new text node using `document.createTextNode` and append it to the element. Remember to add a newline character to the end so that not all output appears on a single line.

CONWAY'S GAME OF LIFE

To solve the problem of having the changes conceptually happen at the same time, try to see the computation of a generation as a pure function, which takes one grid and produces a new grid that represents the next turn.

Representing the matrix can be done with a single array of width $\times$ height elements, storing values row by row, so, for example, the third element in the fifth row is (using zero-based indexing) stored at position

$4 \times width + 2$. You can count live neighbors with two nested loops, looping over adjacent coordinates in both dimensions. Take care not to count cells outside of the field and to ignore the cell in the center, whose neighbors we are counting.

Ensuring that changes to checkboxes take effect on the next generation can be done in two ways. An event handler could notice these changes and update the current grid to reflect them, or you could generate a fresh grid from the values in the checkboxes before computing the next turn.

If you choose to go with event handlers, you might want to attach attributes that identify the position that each checkbox corresponds to so that it is easy to find out which cell to change.

To draw the grid of checkboxes, you can either use a `<table>` element (see Chapter 14) or simply put them all in the same element and put `<br>` (line break) elements between the rows.

PROJECT: A PIXEL ART EDITOR

KEYBOARD BINDINGS

The key property of events for letter keys will be the lowercase letter itself, if SHIFT isn't being held. We're not interested in key events with SHIFT here.

A "keydown" handler can inspect its event object to see whether it matches any of the shortcuts. You can automatically get the list of first letters from the `tools` object so that you don't have to write them out.

When the key event matches a shortcut, call `preventDefault` on it and dispatch the appropriate action.

EFFICIENT DRAWING

This exercise is a good example of how immutable data structures can make code *faster*. Because we have both the old and the new picture, we can compare them and redraw only the pixels that changed color, saving more than 99 percent of the drawing work in most cases.

You can either write a new function `updatePicture` or have `drawPicture` take an extra argument, which may be undefined or the previous picture. For each pixel, the function checks whether a previous picture was passed with the same color at this position and skips the pixel when that is the case.

Because the canvas gets cleared when we change its size, you should also avoid touching its `width` and `height` properties when the old picture and the new picture have the same size. If they are different, which will happen when a new picture has been loaded, you can set the binding holding the old picture to `null` after changing the canvas size because you shouldn't skip any pixels after you've changed the canvas size.

CIRCLES

You can take some inspiration from the `rectangle` tool. As with that tool, you'll want to keep drawing on the *starting* picture, rather than the current picture, when the pointer moves.

To figure out which pixels to color, you can use the Pythagorean

theorem. First figure out the distance between the current pointer position and the start position by taking the square root (`Math.sqrt`) of the sum of the square (`x ** 2`) of the difference in x-coordinates and the square of the difference in y-coordinates. Then loop over a square of pixels around the start position, whose sides are at least twice the radius, and color those that are within the circle's radius, again using the Pythagorean formula to figure out their distance from the center.

Make sure you don't try to color pixels that are outside of the picture's boundaries.

PROPER LINES

The thing about the problem of drawing a pixelated line is that it is really four similar but slightly different problems. Drawing a horizontal line from the left to the right is easy—you loop over the x-coordinates and color a pixel at every step. If the line has a slight slope (less than 45 degrees or $\frac{1}{4}\pi$ radians), you can interpolate the y-coordinate along the slope. You still need one pixel per x position, with the y position of those pixels determined by the slope.

But as soon as your slope goes across 45 degrees, you need to switch the way you treat the coordinates. You now need one pixel per y position, since the line goes up more than it goes left. And then, when you cross 135 degrees, you have to go back to looping over the x-coordinates, but from right to left.

You don't actually have to write four loops. Since drawing a line from A to B is the same as drawing a line from B to A , you can swap the start and end positions for lines going from right to left and treat

them as going left to right.

So you need two different loops. The first thing your line drawing function should do is check whether the difference between the x-coordinates is larger than the difference between the y-coordinates. If it is, this is a horizontalish line, and if not, a verticalish one.

Make sure you compare the *absolute* values of the x and y difference, which you can get with `Math.abs`.

Once you know along which axis you will be looping, you can check whether the start point has a higher coordinate along that axis than the endpoint and swap them if necessary. A succinct way to swap the values of two bindings in JavaScript uses destructuring assignment like this:

```
[start, end] = [end, start];
```

Then you can compute the slope of the line, which determines the amount the coordinate on the other axis changes for each step you take along your main axis. With that, you can run a loop along the main axis while also tracking the corresponding position on the other axis, and you can draw pixels on every iteration. Make sure you round the nonmain axis coordinates, since they are likely to be fractional and the `draw` method doesn't respond well to fractional coordinates.

NODE.JS

SEARCH TOOL

Your first command line argument, the regular expression, can be found in `process.argv[2]`. The input files come after that. You can use the `RegExp` constructor to go from a string to a regular expression object.

Doing this synchronously, with `readFileSync`, is more straightforward, but if you use `node:fs/promises` to get promise-returning functions and write an `async` function, the code looks similar.

To figure out whether something is a directory, you can again use `stat` (or `statSync`) and the `stats` object's `isDirectory` method.

Exploring a directory is a branching process. You can do it either by using a recursive function or by keeping an array of work (files that still need to be explored). To find the files in a directory, you can call `readdir` or `readdirSync`. Note the strange capitalization—Node's filesystem function naming is loosely based on standard Unix functions, such as `readdir`, that are all lowercase, but then it adds `Sync` with a capital letter.

To go from a filename read with `readdir` to a full path name, you have to combine it with the name of the directory, either putting `sep` from `node:path` between them or using the `join` function from that same package.

DIRECTORY CREATION

You can use the function that implements the `DELETE` method as a blueprint for the `MKCOL` method. When no file is found, try to create a directory with `mkdir`. When a directory exists at that path, you can return a 204 response so that directory creation requests are idempotent. If a nondirectory file exists here, return an error code. Code 400 (“bad request”) would be appropriate.

A PUBLIC SPACE ON THE WEB

You can create a `<textarea>` element to hold the content of the file that is being edited. A `GET` request, using `fetch`, can retrieve the current content of the file. You can use relative URLs like *index.html*, instead of *http://localhost:8000/index.html*, to refer to files on the same server as the running script.

Then, when the user clicks a button (you can use a `<form>` element and “submit” event), make a `PUT` request to the same URL, with the content of the `<textarea>` as the request body, to save the file.

You can then add a `<select>` element that contains all the files in the server's top directory by adding `<option>` elements containing the lines returned by a `GET` request to the URL `/`. When the user selects another file (a “change” event on the field), the script must fetch and display that file. When saving a file, use the currently selected filename.

PROJECT: SKILL-SHARING WEBSITE

DISK PERSISTENCE

The simplest solution I can come up with is to encode the whole `talks` object as `JSON` and dump it to a file with `writeFile`. There is already a method (`updated`) that is called every time the server's data changes. It can be extended to write the new data to disk.

Pick a filename, for example `./talks.json`. When the server starts, it can try to read that file with `readFile`, and if that succeeds, the server can use the file's contents as its starting data.

COMMENT FIELD RESETS

The best way to do this is probably to make the talk component an object, with a `syncState` method, so that they can be updated to show a modified version of the talk. During normal operation, the only way a talk can be changed is by adding more comments, so the `syncState` method can be relatively simple.

The difficult part is that when a changed list of talks comes in, we have to reconcile the existing list of DOM components with the talks on the new list—deleting components whose talk was deleted and updating components whose talk changed.

To do this, it might be helpful to keep a data structure that stores the talk components under the talk titles so that you can easily figure out whether a component exists for a given talk. You can then loop over the new array of talks, and for each of them, either synchronize

an existing component or create a new one. To delete components for deleted talks, you'll have to also loop over the components and check whether the corresponding talks still exist.

INDEX

! operator, 26, 46
!= operator, 25
!== operator, 29
* operator, 19, 28, 220
** operator, 45
*= operator, 50
+ operator, 19, 22, 28, 220
++ operator, 50
+= operator, 50, 295
- operator, 19, 23, 28
-- operator, 50
-= operator, 50
/ operator, 19
/= operator, 50
< operator, 24
<= operator, 25

= operator, 34, 91, 520
 as expression, 243
 for default value, 69
 in Egg, 311
== operator, 25, 28, 95, 121
=== operator, 29, 121, 173, 586
> operator, 24
>= operator, 25
?: operator, 26, 31, 309
?? operator, 30, 75, 490
[] (array), 86, 110
[] (subscript), 86, 87
% operator, 19, 49, 440, 582, 603, 606
&& operator, 25, 30, 142
|| operator, 26, 29, 142, 582

{ } (block), 42
{ } (object), 90, 96, 110
200 (HTTP status code), 463, 537, 542
204 (HTTP status code), 546, 548
2d (canvas context), 426
304 (HTTP status code), 558, 568, 577
400 (HTTP status code), 615
403 (HTTP status code), 544
404 (HTTP status code), 463, 544, 564, 567
405 (HTTP status code), 468, 542
406 (HTTP status code), 608
500 (HTTP status code), 542

a (HTML tag), 327, 347, 350, 476, 514
Abelson, Hal, 300
absolute positioning, 356, 362, 373, 379, 388
absolute value, 113, 613
abstract data type, 144, 150
abstract syntax tree, *see* syntax tree
abstraction, 8, 58, 123, 124, 126, 300, 339, 470, 521, 522

 in Egg, 300
 of the network, 323
 with higher-order functions, 123

acceleration, 416
Accept header, 492, 608
access control, 213, 559
Access-Control-Allow-Origin header, 469
action, 497, 500, 502
activeElement property, 475
actor, 396, 405, 414
add method, 173
addEntry function, 96
addEventListener method, 364, 417, 539
addition, 18, 172
address, 116, 461
address bar, 325, 461, 464
adoption, 214
ages example, 157
alert function, 329
alpha, 516
alphanumeric character, 217
alt attribute, 343
alt key, 371

- altKey property, 371
- ambiguity, 319
- American English, 220
- ampersand character, 327, 465
- analysis, 191, 198
- ancestor element, 406
- Android, 372
- angle, 358, 434, 436, 607
- angle brackets, 326, 327
- animation, 357, 378, 388, 392, 400, 456, 607
 - bouncing ball, 459
 - platform game, 409, 410, 416, 418, 419, 439, 440, 452, 604
 - spinning cat, 356, 362
- appendChild method, 342, 601
- Apple, 332
- appliance, 144
- application, 1, 495, 554
- arc, 434, 436
- arc method, 434, 435, 607
- argument, 38, 68, 75, 109, 301
- argv property, 528
- arithmetic, 18, 28, 312
- array, 88–90, 93, 119
 - as matrix, 395
 - as table, 99
 - counting, 139
 - creation, 86, 135, 499, 584, 590
 - filtering, 130
 - flattening, 142
 - in Egg, 318
 - indexing, 86, 101, 106, 584, 603
 - iteration, 101, 124, 128
 - length of, 88
 - methods, 105, 119, 128, 130–132, 136, 141, 142
 - notation, 116
 - of rest arguments, 109
 - random element, 183
 - RegExp match, 222
 - representation, 116
 - searching, 101, 105
- Array constructor, 499
- Array prototype, 149, 156
- array-like object, 160, 338, 339, 341, 377, 477, 485, 535
- Array.from function, 530
- arrays in egg (exercise), 318, 599

- arrow function, 65, 147, 295
- arrow key, 388
- artificial intelligence, 175
- artificial life, 391, 493
- assert function, 210
- assertion, 210
- assignment, 34, 50, 243, 319, 600
- assumption, 208, 211
- asterisk, 19, 220
- async function, 283–285, 295, 297, 298, 577, 596
- asynchronous programming, 269–271, 273, 283, 292, 294, 421, 597
 - in Node.js, 527, 536, 539, 545, 550
 - reading files, 486
- at sign, 393
- attribute, 327, 338, 346, 477, 501, 610
- autofocus attribute, 476
- automatic semicolon insertion, 33
- automation, 189, 196
- automaton, 175
- avatar, 391
- average function, 134
- await keyword, 283, 284, 286, 295, 298, 596, 597
- axis, 415, 428, 442, 443, 613
- Babbage, Charles, 84
- background, 391, 403, 411
- background (CSS), 388, 392, 404
- backslash character
 - as path separator, 544
 - in regular expressions, 215, 217, 236, 592
 - in strings, 21, 327
- backtick, 20, 22
- backtracking, 229, 230, 234
- ball, 459, 607
- balloon, 388
- balloon (exercise), 388, 602
- banking example, 204
- Banks, Iain, 390
- baseControls constant, 519
- baseTools constant, 519
- bean counting (exercise), 83, 584
- beforeunload event, 382
- behavior, 246
- benchmark, 349
- Berners-Lee, Tim, 321
- best practices, 4

bezierCurveTo method, 433	bitfield, 376
big ball of mud, 249, 250	bitmap graphics, 438, 459
binary data, 4, 15, 535	black, 500
binary number, 15, 17, 99, 198, 229, 484	block, 42, 46, 59, 61, 65, 91, 203, 205, 301
binary operator, 18, 23, 32	block comment, 53, 234
binding, 6, 45, 91	block element, 347, 350, 351
as	blocking, 269, 357, 385, 536
state, 45, 48	blue, 500
as state, 95, 487	blur event, 380, 381
assignment, 34, 63	blur method, 475
compilation of, 600	body (HTML tag), 326, 328, 336
definition, 34, 59, 63, 315, 319	body (HTTP), 464, 466, 468, 538, 546, 548
destructuring, 114	body property, 336, 337, 341, 468
from parameter, 60, 71	bold, 350
global, 60, 193, 251, 422, 528, 529	Book of Programming, 15, 249, 526
in Egg, 311	Boolean, 24, 41, 45, 94, 216, 310, 312
local, 60	conversion to, 29, 40, 46
model of, 35, 91, 95	Boolean function, 40
naming, 36, 37, 52, 77, 111, 194	border (CSS), 347, 351
scope of, 60	border-radius (CSS), 373
undefined, 207	bouncing, 393, 399, 410, 414, 459
visibility, 61	bound, 130
bit, 5, 15, 17, 24, 99	

boundary, 226, 242, 247, 450, 592	bubbling, <i>see</i> event propagation
box, 213, 334, 391, 392, 459, 607	Buffer class, 534, 535, 539, 540
box shadow (CSS), 406	bug, 122, 191, 198, 235, 238, 246, 254, 332
br (HTML tag), 506, 610	building Promise.all (exercise), 298, 598
braces	bundler, 261
block, 7, 42, 582	button, 363, 465, 477, 494
body, 126	button (HTML tag), 330, 365, 372, 389, 478, 488, 493, 501
class, 151	button property, 366, 376, 503
function body, 59, 65	buttons property, 376, 503
in regular expression, 221	
object, 90, 91, 96, 114	call method, 146, 156
branching, 227, 229	call stack, 66, 67, 70, 75, 90, 202, 203, 207, 292
branching recursion, 73, 445	callback function, 269, 271, 273, 276, 364, 418, 419, 502, 534, 536, 539, 568, 576
break keyword, 49, 51	camel case, 52, 352
breakpoint, 199	cancelAnimationFrame function, 384
British English, 220	canvas, 392, 424, 428–433, 437, 438, 440–442, 444, 445, 447, 454–456, 458, 608
browser, 2, 8, 270, 321, 325, 328, 330–332, 364, 392, 459, 461, 464, 470, 478, 488, 514, 521, 553	context, 426, 427
environment, 37–39, 461	
security, 469, 555	
storage, 487, 490	
window, 364	
browser wars, 332	
browsers, 12, 261	

path, 429	celery, 552
size, 426, 429	cell, 493
canvas (HTML tag), 426, 496, 502, 513, 515, 523, 611	Celsius, 162
CanvasDisplay class, 447, 449, 452	center, 408
capitalization, 52, 152, 222, 352, 361, 540	centering, 357
capture group, 223, 225, 232, 233, 561	certificate, 471
career, 390	change event, 475, 480, 507, 610, 615
caret character, 218, 226, 242, 533	character, 20, 21, 137, 138, 479
Carla the crow, 279, 280, 287, 297	character category, 219
carriage return, 241	character encoding, 534, 535
cascading, 353	characterCount function, 133
Cascading Style Sheets, <i>see</i> CSS	characterScript function, 140, 143, 588
case conversion, 88	charCodeAt method, 137
case keyword, 51	checkbox, 473, 481, 494, 610
case sensitivity, 222, 593	checked attribute, 473, 481
casual computing, 2	chess board (exercise), 582
cat's hat (exercise), 362	chessboard (exercise), 56
catch keyword, 202, 203, 207, 208, 212, 292, 591	child node, 337, 339, 341
catch method, 277	childNodes property, 339, 344, 603
CD, 15	children property, 340
	Chinese characters, 137, 140
	choice, 227
	Chrome, 332

circle, 358, 434, 435	clipping, 450
circle (SVG tag), 425	closePath method, 431
circles (exercise), 523, 611	closing tag, 327, 329
circular dependency, 267, 595	closure, 71, 318, 599, 601, 604
class, 145, 150, 151, 172, 178, 394, 498	closure in egg (exercise), 318, 599
class attribute, 342, 347, 353, 402, 405, 406	code, 11, 234, 391
class declaration, 151, 153	structure of, 32, 46, 58, 249, 262
class hierarchy, 170	code golf, 247
className property, 347	code unit, 137
cleaning up, 204	codePointAt method, 138
clearing, 424, 440, 449, 450, 608	coin, 391, 393, 415, 454
clearInterval function, 384	Coin class, 400, 415
clearRect method, 440, 608	collaboration, 321
clearTimeout function, 384, 385	collection, 8, 86, 89, 93, 119, 166
click event, 364, 365, 367, 373, 376, 501, 609	collision detection, 410, 414–416, 605, 607
client, 323, 470, 553, 570	colon character, 26, 51, 90, 351
clientHeight property, 347	color, 426, 428, 450, 496, 516
clientWidth property, 347	color (CSS), 351
clientX property, 373, 377, 504, 505	color code, 500
clientY property, 373, 377, 504, 505	color component, 500
clipboard, 331	color field, 496, 500, 508
	color picker, 496, 508, 512
	color property, 498
	ColorSelect class, 508

comma character, 301
 command key, 371, 522
 command line, 255, 526, 528, 530, 550
 comment, 53, 116, 234, 240, 319, 337, 553, 557, 566, 574
 comment field reset (exercise), 580, 616
 COMMENT_NODE code, 337
 comments in egg (exercise), 319, 600
 CommonJS, 529
 CommonJS modules, 257, 259, 267
 communication, 322, 470
 community, 527
 compareRobots function, 189
 comparison, 24, 29, 312, 584
 deep, 121, 586
 of NaN, 25
 of numbers, 24, 39
 of objects, 95
 of strings, 24
 of undefined values, 28
 compatibility, 9, 193, 321, 332, 523, 533
 compilation, 261, 315, 316, 599
 complexity, 4, 5, 122, 170, 230, 354, 521
 component, 496–498, 506, 519
 composability, 8, 134, 262
 computed property, 87, 490
 computer, 1, 3
 concat method, 106, 142, 590, 601
 concatenation, 22, 106, 601
 conditional execution, 26, 41, 50, 56, 309
 conditional operator, 26, 31, 309
 conditional request, 558
 configuration, 240
 connected graph, 188
 connection, 323, 461, 471, 554–556
 consistency, 52, 321, 338
 console.log, 7, 14, 23, 39, 66, 69, 81, 199, 528
 const keyword, 36, 61, 95, 111, 114
 constant, 36, 112, 417
 constructor, 52, 150–152, 170, 190, 193, 203, 224, 236, 589,

 591
 content negotiation (exercise), 492, 608
 Content-Length header, 464
 Content-Type header, 464, 537, 542, 544, 551
 context, 426, 427
 context menu, 369
 continuation, 272
 continue keyword, 49
 control, 506, 508, 513, 514, 519
 control flow, 40, 127
 asynchronous, 270, 284
 conditional, 41
 exceptions, 202, 204
 functions, 66
 loop, 44, 46, 48
 control key, 371, 522
 control point, 432–434
 convention, 52
 Conway's Game of Life, 493
 coordinates, 172, 358, 373, 403, 408, 411, 428, 434, 442, 443
 copy-paste programming, 78, 254
 copyright, 255
 correlation, 97, 98, 101, 103, 104
 cosine, 111, 358
 countBy function, 139, 143
 counter variable, 44, 47, 359, 582, 584, 598, 603
 CPU, 269
 crash, 207, 211, 565, 580
 createElement method, 344, 501, 601
 createReadStream function, 539, 545
 createServer function, 537, 539, 560, 561
 createTextNode method, 343, 609
 createWriteStream function, 539, 548
 crisp, 455
 cross-domain request, 469
 crying, 222
 cryptography, 471
 CSS, 351, 352, 354, 402, 404–407, 409, 424, 429, 500, 570
 ctrlKey property, 371, 522
 curl program, 548
 curly braces, *see* braces

cursor, 479, 480
 curve, 432–434
 cwd function, 544
 cycle, 336

 Dark Blue (game), 391
 data, 3, 15, 84
 data attribute, 346, 389
 data event, 539
 data flow, 497, 498, 522
 data format, 116, 338
 data loss, 580
 data structure, 84, 263, 265, 334, 494
 collection, 86
 immutable, 180
 list, 119, 166
 map, 157
 stack, 89
 tree, 301, 336, 455
 data URL, 513, 515
 dataset, 100, 128, 129
 date, 217, 221, 224
 Date class, 224, 225, 251, 257, 297, 596
 date-names package, 257
 Date.now function, 225, 297, 517

 dblclick event, 373
 De Morgan's laws, 587
 debouncing, 385
 debugger statement, 199
 debugging, 9, 191, 192, 194, 197, 199, 203, 208, 210, 246
 decentralization, 321
 decimal number, 15, 198, 229
 declaration, 351
 decodeURIComponent function, 465, 543, 561
 deep comparison, 95, 121
 deep comparison (exercise), 121, 586
 default behavior, 350, 369
 default keyword, 51
 default value, 30, 69, 429, 490, 520
 defineProperty function, 588
 degree, 434, 442
 DELETE method, 462, 464, 468, 541, 546, 564
 delete method, 173
 delete operator, 91
 dependence, 97
 dependency, 250, 252, 254, 257,

267, 329, 532, 533
 deserialization, 117
 design, 265
 destructuring, 226
 destructuring assignment, 613
 destructuring binding, 114, 258, 520, 595
 developer tools, 11, 39, 199, 200, 207
 dialect, 261
 dialog, 38
 diamond, 458, 605
 digit, 15, 17, 198, 216, 217, 220, 221, 223, 500
 Dijkstra's algorithm, 264
 Dijkstra, Edsger, 175, 264
 dijstra package, 264, 593
 dimensions, 172, 347, 390, 393, 410, 426, 582
 direct child node, 354
 direction (writing), 143
 directory, 530, 534, 535, 541, 544–546, 550, 615
 directory creation (exercise), 550, 615
 disabled attribute, 477

 discretization, 391, 410, 419
 dispatch, 50, 497, 498, 500, 506, 519, 560, 611
 display, 402, 419, 420, 447, 454, 456
 display (CSS), 351, 389
 distance, 612
 division, 19
 division by zero, 20
 do loop, 46, 184
 doctype, 326, 328
 document, 324, 325, 334, 381, 424
 document format, 470, 492
 Document Object Model, *see* DOM
 documentation, 526
 documentElement property, 336
 dollar sign, 36, 226, 232, 242
 DOM, 336, 346
 attributes, 346
 components, 496, 497
 construction, 338, 342, 345, 501
 events, 364, 372
 fields, 472, 479
 graphics, 392, 402, 404–406,

424, 426, 455	447, 451, 608
interface, 338	drawing, 334, 347, 348, 357, 425,
modification, 342	426, 428, 432, 445, 451,
querying, 341, 354	452, 454, 495, 496, 610
tree, 336	drawing program example, 373,
dom property, 498	495
domain, 325, 469, 488	drawPicture function, 503, 513,
domain-specific language, 123, 197,	523, 611
214, 317, 354	drop-down menu, 474, 483
DOMDisplay class, 402, 403, 447	duplication, 254
dominant direction (exercise), 143,	ECMAScript, 9, 10, 193
588	ECMAScript 6, 10
done property, 517	economic factors, 521
doneAt property, 517	editor, 47
dot character, <i>see</i> period charac-	efficiency, 72, 119, 135, 315, 348,
ter	392, 405, 427, 503, 523
double click, 373	efficient drawing (exercise), 523,
double-quote character, 20, 248,	611
301, 327	Egg language, 300, 301, 306, 307,
download, 11, 254, 513, 531, 548,	309, 311, 313, 314, 316,
552, 579	318, 319, 336
download attribute, 514	electronic life, 391
draggable bar example, 374	elegance, 73, 303
dragging, 374, 496, 510, 523	element, 327, 337, 340, 344
draw function, 509, 524	ELEMENT_NODE code, 337, 601
drawImage method, 438, 439, 442,	elements property, 477

ellipse, 356, 359	536, 542, 545, 573, 577
else keyword, 42	error message, 307, 493
elt function, 345, 501, 522, 574	error recovery, 200
email, 471	error response, 463, 542, 547
emoji, 22, 137, 244, 388	error tolerance, 327
empty set, 234	Error type, 203, 207, 209, 545
encapsulation, 145, 158, 170, 365,	ES modules, 251, 252, 330, 529,
402	531
encodeURIComponent function,	escape key, 422
465, 557, 572	escaping
encoding, 322	in HTML, 327, 330
encryption, 471	in regexps, 215, 218, 236
end event, 539	in strings, 21, 301
end method, 538, 539, 542	in URLs, 465, 543, 557, 561
enemies example, 240	Escher, M.C., 424
engineering, 332	ETag header, 558, 567, 577
ENOENT (status code), 545	eval, 259
enter key, 478	evaluate function, 307, 309, 311
entity, 327	evaluation, 307, 316
enum (reserved word), 37	even number, 43, 82
environment, 37, 309	event handling, 364, 366, 367, 369,
equality, 25	370, 378, 380, 382, 392,
error, 137, 192–194, 197, 200, 201,	417, 421, 422, 438, 455,
207, 209, 276	478, 480, 501, 539, 604,
error event, 486, 548	610
error handling, 191, 201, 202, 207,	event loop, 292

- event object, 366, 373, 377
- event propagation, 367, 368, 380, 382
- event type, 366
- every method, 142
- everything (exercise), 142, 587
- evolution, 214, 521, 533
- exception handling, 202–204, 207, 209, 212, 213, 276, 279, 284, 292, 609
- exception safety, 206
- exec method, 222–224, 237, 239
- execution order, 40, 64, 66
- exercises, 3, 11, 54, 198
- exit method, 528
- expectation, 369
- experiment, 4, 11, 246
- exploit, 331
- exponent, 18, 248, 592, 593
- exponentiation, 45, 48
- export keyword, 531
- exports object, 257, 260, 595
- expression, 32–34, 40, 45, 48, 63, 301, 308
- expressivity, 317
- extraction, 224
- factorial function, 13
- Fahrenheit, 162
- fallthrough, 51
- false, 24
- farm example, 77, 80, 227
- fetch function, 467, 492, 538, 572, 576, 615
- field, 372, 465, 472, 477, 482, 487, 494, 496, 580
- Fielding, Roy, 461
- file, 485, 529, 546, 616
 - access, 259, 261, 515, 534, 535
 - image, 495, 513, 514
 - resource, 462, 464, 541, 544
 - stream, 539
- file extension, 544
- file field, 473, 485
- file format, 240
- file reading, 486
- file server, 570
- file server example, 541, 544, 546–548, 550, 551, 615
- file size, 261
- File type, 486
- FileReader class, 486, 515
- files property, 485

- fileSizes function, 294, 295
- filesystem, 485, 534, 535, 541, 543, 616
- fill function, 511
- fill method, 430, 499
- filling, 428, 430, 437, 457
- fillRect method, 428, 440
- fillStyle property, 428, 437, 500
- fillText method, 437, 438, 606
- filter method, 130, 134, 140, 179, 588–590
- finally keyword, 205, 213, 592
- find method, 140
- findInStorage function, 283
- findRoute function, 187
- finish event, 548
- Firefox, 332
- firstChild property, 339
- fixed positioning, 379
- fixing scope (exercise), 319, 600
- FizzBuzz (exercise), 56, 582
- flattening (exercise), 142
- flexibility, 10
- flipHorizontally function, 452, 605
- flipHorizontally method, 443
- flipping, *see* mirroring
- floating-point number, 17, 18
- flood fill, 506, 511
- flow diagram, 228, 229
- focus, 372, 380, 475–477, 481, 482, 522, 580
- focus event, 380, 381
- focus method, 475
- fold, *see* reduce method
- font, 438
- font-family (CSS), 352
- font-size (CSS), 388
- font-weight (CSS), 353
- for attribute, 482
- for loop, 47, 49, 101, 124, 142, 208, 584, 586
- for/of loop, 102, 138, 163, 165, 168, 589
- forEach method, 128, 160
- form, 464–466, 477, 478, 551
- form (HTML tag), 472, 473, 477, 575, 615
- form property, 477
- formatDate module, 257
- fractal example, 445
- fractional number, 18, 248, 391
- frame, 440, 452, 607

framework, 80, 497
 frequency table, 97
 function, 8, 38, 58, 65, 192, 300, 301, 314
 application, 38, 40, 60, 66, 68, 72, 110, 130, 207, 301, 308
 as property, 88
 as value, 59, 63, 70, 125, 126, 130, 366, 418, 604
 body, 59, 65
 callback, *see* callback function
 declaration, 64
 definition, 59, 64, 76
 higher-order, 64, 125, 126, 130–132, 134, 233, 418
 model of, 71
 naming, 77, 78
 purity, 80
 scope, 63, 256, 318
 Function constructor, 259, 312, 316, 493, 609
 function keyword, 59, 64
 Function prototype, 149, 156
 future, 10, 37, 64, 460
 game, 390–393, 417, 421, 447

getItem method, 488, 490
 getMinutes method, 225
 getMonth method, 225
 getPrototypeOf function, 149, 152, 319, 600
 getSeconds method, 225
 getter, 161, 173, 399
 getTime method, 225
 getYear method, 225
 GitHub, 462
 global object, 193
 global scope, 60, 251, 257, 312, 383, 384, 528, 529, 600
 goalOrientedRobot function, 188
 Google, 332
 grammar, 32, 192, 241
 graph, 176, 186, 264, 456
 graphics, 392, 402, 405, 424, 426, 427, 438, 455, 456
 grave accent, *see* backtick
 gravity, 416
 greater than, 24
 greed, 234, 235
 green, 500
 grep, 550
 grid, 391, 392, 403, 411, 493, 609

Group class, 173, 174, 190, 286, 589
 groupBy function, 143
 grouping, 19, 42, 221, 223, 232, 233, 592
 groups (exercise), 173, 174, 589
 h1 (HTML tag), 326, 347
 handleAction function, 571
 hard disk, 262, 268
 hard drive, 16, 485, 488, 527, 580
 hard-coding, 341
 hardcoding, 458
 has method, 158, 173
 hash character, 319
 hash sign, 500
 hasOwn function, 159, 319, 564, 600
 head (HTML tag), 326, 328, 336
 head property, 336
 header, 463, 464, 469, 470, 538, 556
 headers property, 467, 468, 492
 height property, 523, 611
 help text example, 380
 hexadecimal number, 229, 465, 500, 517

hidden element, 351, 389	identifier, 302
higher-order function, <i>see</i> function, higher-order	identity, 94
history, 8, 521	if keyword, 41, 243
historyUpdateState function, 517	chaining, 42, 50, 582, 583
hooligan, 559	If-None-Match header, 558, 568, 576
href attribute, 327, 341, 346	image, 343, 382, 424, 464
HTML, 325, 334, 462, 487, 551	imagination, 390
notation, 326	IME, 372
structure, 334, 337	img (HTML tag), 327, 343, 350, 382, 424, 438, 440, 515
html (HTML tag), 328, 336	immediately invoked function expression, 256
HTTP, 321, 322, 324, 325, 461–464, 466, 469–471, 537, 546, 547, 551, 555, 556	immutable, 94, 180, 399, 499, 500, 511, 517, 611
client, 538, 548, 553	implements (reserved word), 37
server, 537, 541, 569	import keyword, 252, 529, 530, 560
HTTPS, 324, 471, 472	in operator, 92, 159
human language, 32	includes method, 101, 102, 589
HyperText Markup Language, <i>see</i> HTML	indentation, 46
HyperText Transfer Protocol, <i>see</i> HTTP	index, 86
hyphen character, 19, 217, 352	index property, 222
id attribute, 341, 353, 482	index.html, 570
idempotence, 547	indexOf method, 105, 107, 173, 216, 237, 589
idempotency, 615	

infinite loop, 49, 67, 208, 585	canvas, 424, 426
infinity, 20	design, 79, 214, 224, 237, 338, 429
infrastructure, 254	HTTP, 470, 556
inheritance, 149, 168–171, 209, 545	module, 250, 256, 257, 260, 262, 467, 531
INI file, 240	object, 145, 158, 160, 161, 174, 190, 396, 447, 479, 498
ini package, 255, 256, 262, 531	interface (reserved word), 37
initialization, 381	internationalization, 218
inline element, 347, 350	internet, 240, 322–324, 330
inner function, 62, 601	Internet Explorer, 331, 332
inner loop, 231	interpolation, 22
innerHeight property, 379	interpretation, 11, 307, 309, 315
innerWidth property, 379	interview question, 56
input, 200, 363, 392, 475, 527, 565	inversion, 218
input (HTML tag), 380, 472, 479, 481, 482, 485, 508, 515	IP address, 325, 461
input event, 480	isDirectory method, 546, 614
insertBefore method, 342, 343	isEven (exercise), 82, 583
installation, 254	isolation, 145, 249, 257, 330
instance, 150, 153	iterable interface, 165, 589
instanceof operator, 170, 209	iterator, 285
instruction, 5	iterator interface, 163, 165, 174
integer, 18	
integration, 214, 338	Jacques, 85
interface, 153	Java, 9
	JavaScript, 8

availability of, 2	Kernighan, Brian, 191
flexibility of, 10	key code, 417
history of, 8, 321	key property, 371, 602, 610
in HTML, 329	keyboard, 37, 363, 369, 370, 391,
syntax, 32	392, 416, 417, 422, 475,
uses of, 10	476, 480, 522
versions of, 10	keyboard bindings (exercise), 522,
weaknesses of, 9	610
JavaScript console, 11, 23, 39,	keyboard focus, <i>see</i> focus
199, 207, 493, 528	keydown event, 370, 385, 418, 522,
JavaScript Object Notation, <i>see</i>	602, 610
JSON	keyup event, 370, 418
job, 435	keyword, 34, 37, 347
join method, 141, 156, 530	Khasekhemwy, 480
journal, 85, 90, 93, 96, 102	kill process, 538
JOURNAL dataset, 100	Knuth, Donald, 58
journalEvents function, 102	
JSON, 116, 262, 468, 490, 556,	label, 438, 459
557, 565, 577, 616	label (HTML tag), 482, 508
json method, 468	labeling, 482
JSON.parse function, 117, 616	landscape example, 62
JSON.stringify function, 117	Laozi, 268
JSX, 575	Last-Modified header, 464
jump, 6	lastChild property, 339
jump-and-run game, 390	lastIndex property, 237–239
jumping, 391, 416, 417	lastIndexOf method, 105
	latency, 261

lava, 391, 393, 394, 406, 410, 414,	line, 33, 46, 241, 424, 428–433,
454	435, 436, 458, 606
Lava class, 399, 414	line break, 20, 241
layering, 323	line comment, 53, 234
layout, 347, 349, 351	line drawing, 524, 612
laziness, 348	line width, 429, 442
Le Guin, Ursula K., 3	lines of code, 313
leaf node, 336, 337	lineTo method, 430
leak, 331, 422	lineWidth property, 429
learning, 2, 10, 11	link, 327, 339, 341, 369, 372, 514
left (CSS), 356, 357, 359, 362	link (HTML tag), 409
LEGO, 250	linked list, 119, 166, 168, 586
length property	Liskov, Barbara, 144
for array, 88, 499	list (exercise), 119, 586
for string, 78, 83, 87, 109,	listen method, 537, 538
584	listening (TCP), 323, 537
LengthList class, 168	ListIterator class, 167
less than, 24	literal expression, 32, 215, 305,
let keyword, 34, 35, 61, 95, 111,	308
114, 193	live data structure, 334, 344, 355,
level, 391, 393, 394, 402, 403,	603
406, 420, 421	live view, 554, 555, 578, 616
Level class, 394	lives (exercise), 421
lexical scoping, 62, 63	load event, 381, 438, 451, 486,
library, 338, 497, 531, 532	608
license, 255	LoadButton class, 514

local binding, 70, 319, 584	Map class, 158, 161
local scope, 60, 315	map method, 131, 134, 141, 157, 179, 395, 507
localhost, 537	Marcus Aurelius, 363
localStorage object, 488, 572	match method, 223, 239
locked box (exercise), 212, 592	matching, 216, 226, 237, 247
logging, 199	algorithm, 228-230
logical and, 25	Math object, 82, 87, 111
logical operators, 25	Math.abs function, 113, 613
logical or, 26	Math.acos function, 111
long polling, 554-556, 558, 565, 567, 568, 576	Math.asin function, 111
look-ahead, 226	Math.atan function, 111
loop, 6, 7, 44, 47, 55, 56, 72, 101, 124, 125, 134, 135, 239, 582, 584, 606	Math.ceil function, 113, 411, 450
termination of, 49	Math.cos function, 111, 358, 359, 607
loop body, 46, 126	Math.floor function, 113, 183, 411, 450
lycanthropy, 85, 96	Math.max function, 39, 87, 109, 111, 450
machine code, 4, 316	Math.min function, 40, 82, 111, 450
magic, 300	Math.PI constant, 111, 434
mailRoute array, 185	Math.random function, 112, 183, 401, 494
maintenance, 255	Math.round function, 113
malicious script, 330	Math.sin function, 111, 358, 359, 401, 415
man-in-the-middle, 471	
map, 401, 478	
map (data structure), 157	

401, 415	method, 88, 146, 153, 193, 538
Math.sqrt function, 100, 111, 588	array, 105
Math.tan function, 111	HTTP, 462, 470, 538, 548, 556, 560
mathematics, 72, 126	private, 153
max example, 109	method attribute, 465
max-height (CSS), 406	method call, 146
max-width (CSS), 406	method property, 468
maximum, 39, 111, 133, 134	methods object, 541
Meadowfield, 175	Microsoft, 331, 332
measuring a robot (exercise), 189, 590	MIME type, 492, 544
media type, 470, 492, 544	mime-types package, 544
meetup, 552	mini application, 487
memory, 5, 15	minifier, 261
call	minimalism, 391
stack, 67	minimum, 40, 82, 111
organization, 16, 34, 86, 94, 116	minimum (exercise), 82, 583
persistence, 580	minus, 19, 248
speed, 268, 315	mirror, 443, 459, 608
structure	mirroring, 441, 443
sharing, 120	Miró, Joan, 495
mesh, 324	mixer example, 144
message event, 383	MKCOL method, 550, 615
meta key, 371	mkdir function, 550, 615
metaKey property, 371, 522	modification date, 546
	modifier key, 371

- modular robot (exercise), 266, 593
- modularity, 145, 497
- module, 250, 256, 266, 402, 530, 531, 560
 - design, 262
 - resolution, 252
- module loader, 529
- module object, 260
- module system, 256
- modulo operator, 19
- Mongolian vowel separator, 219
- monster (exercise), 422, 605
- Mosaic, 331
- motion, 392
- mouse, 37
- mouse button, 366, 368, 373
- mouse cursor, 373
- mouse trail (exercise), 388, 603
- mousedown event, 367, 373, 376, 502, 503, 609
- mousemove event, 374, 376, 385, 386, 389, 503, 524, 603
- mouseup event, 373, 376
- moveTo method, 430, 435
- Mozilla, 332
- multiple attribute, 483–485
- multiple choice, 474
- multiple-choice, 473, 482, 483
- multiplication, 18, 398, 414
- multiplier function, 71
- music, 390
- mutability, 91, 94, 180
- name attribute, 477, 483
- namespace, 111
- naming, 6, 9, 36
- NaN, 20, 25, 27, 192
- negation, 23, 26
- neighbor, 493, 610
- nerd, 236
- nesting
 - in regexps, 231
 - of arrays, 99
 - of expressions, 32, 303
 - of functions, 62
 - of loops, 56, 582
 - of objects, 336, 340
 - of scope, 62
- Netscape, 8, 331, 332
- network, 269, 322, 555
 - abstraction, 470
 - protocol, 322
 - security, 471

- speed, 261, 268, 527
- new operator, 151
- newline character, 20, 56, 217, 234, 241, 395, 609
- next method, 165, 286, 589
- nextSibling property, 339
- node, 336
- node program, 527, 528
- Node.js, 11, 12, 39, 257, 270, 526, 527, 529–531, 534, 536–539, 541, 544, 546–549, 553, 554, 556, 559, 579
- node:fs package, 534–536
- node:fs/promises package, 536
- node:http package, 537, 538
- node:path package, 544
- node:stream/consumers package, 565
- node_modules directory, 530, 531
- NodeList type, 338, 354
- nodeName property, 361
- nodeType property, 337, 601, 603
- nodeValue property, 341
- nonbreaking space, 219
- not a number, 20
- note-taking example, 488
- notification, 555
- NPM, 254–257, 264, 266, 530–532, 534, 544, 560, 561, 579, 594
- npm program, 531, 533, 544
- null, 27, 28, 30, 75, 87, 114, 121, 200
- number, 17, 94, 216, 248, 592
 - conversion to, 28, 40
 - notation, 17, 18
 - precision of, 18
 - representation, 17
 - special values, 20
- Number function, 40, 41, 52, 220
- number puzzle example, 73
- Number.isNaN function, 41
- object, 84, 90, 92, 170
 - as
 - module, 256
 - as map, 401
 - creation, 116, 151, 490
 - identity, 94
 - mutability, 94
 - property, 39, 87, 111, 114, 148
 - representation, 116

- Object prototype, 148, 149
- object-oriented programming, 144, 150, 159, 160, 168, 178, 263
- Object.create function, 149, 158, 313
- Object.keys function, 92, 121, 587, 601
- Object.prototype, 157
- obstacle, 410
- offsetHeight property, 347, 348
- offsetWidth property, 347
- on method, 539
- onclick attribute, 330, 365
- onclick property, 501
- OpenGL, 426
- opening tag, 327
- operator, 18, 23, 24, 29, 301, 312
 - application, 19
- optimization, 73, 81, 349, 385, 392, 405, 456, 459, 536
- option (HTML tag), 474, 475, 483, 615
- optional, 220
- optional argument, 69, 118
- optional chaining, 115
- options property, 483
- ordering, 323
- ordinal package, 257, 260
- organic growth, 249
- organization, 249
- outline, 428
- output, 23, 38, 39, 199, 200, 312, 527, 609
- overflow, 17
- overflow (CSS), 406
- overlap, 411
- overlay, 353
- overriding, 155, 159, 169, 595
- overwriting, 548, 551, 565
- p (HTML tag), 326, 347
- package, 254, 257, 533
- package (reserved word), 37
- package manager, 254
- package.json, 532, 533
- padding (CSS), 404
- page reload, 382, 478, 488
- pageX property, 373, 377
- pageXOffset property, 348
- pageY property, 373, 377
- pageYOffset property, 348, 379
- Palef, Thomas, 391

- panning, 504
- paragraph, 327
- parallelism, 269, 464
- parameter, 38, 59, 60, 65, 68, 69, 109, 114, 146, 194, 260
- parent node, 367
- parentheses, 19
 - arguments, 38, 59, 65, 126, 301
 - expression, 32
 - in regular expressions, 221, 225, 228, 243, 592
 - statement, 41, 45, 48
- parentNode property, 339
- parse function, 306
- parseApply function, 305
- parseExpression function, 303
- parseINI function, 242, 254
- parser generator, 317
- parsing, 117, 191, 242, 300, 301, 303, 306, 309, 313, 327, 334, 543, 567
- password, 471
- password field, 473
- path
 - canvas, 435
 - canvas, 429, 430, 432, 605
 - closing, 430, 431
 - filesystem, 529, 541
 - URL, 462, 467, 541, 543, 556, 560
- pathfinding, 186, 264, 511
- patience, 524
- pattern, 214–216, 218, 236
- pausing (exercise), 422, 604
- pea soup, 123
- peanuts, 104
- percent sign, 465
- percentage, 141, 379
- performance, 230, 261, 270, 298, 315, 348, 392, 456, 536
- period character, 39, 87, 109, 115, 217, 234, 248, 500
- persistence, 487, 553, 580, 616
- persistent data structure, 178, 180, 190, 197, 499, 510, 517, 605
- persistent group (exercise), 190
- persistent map (exercise), 590
- PGroup class, 190, 590
- phase, 400, 415
- phi coefficient, 97–99

phi function, 99, 113
 phone, 372
 physics, 409, 416, 603
 physics engine, 410
 pi, 18, 111, 358, 401, 434
 PI constant, 111, 358
 pick function, 512
 picture, 424, 425, 440, 456, 496, 517
 Picture class, 498, 515
 picture property, 498
 PictureCanvas class, 502, 523
 pictureFromImage function, 515
 pie chart example, 435, 436, 438, 459, 606
 pink, 500
 pipe, 323
 pipe character, 227, 592
 pipe method, 542, 548
 pipeline, 261
 pixel, 347, 359, 373, 392, 403, 425–427, 438, 440, 449, 456, 459, 495, 498, 504, 509, 511, 516, 524, 611
 pixel art, 439
 PixelEditor class, 506, 519, 522
 pizza, 97, 99
 platform game, 390, 421
 Plauger, P.J., 191
 player, 390, 391, 393, 406, 410, 415, 420, 439, 452, 454
 Player class, 398, 415
 plus character, 19, 220, 248
 Poignant Guide, 32
 pointer, 339
 pointer event, 367, 502
 pointerPosition function, 503
 polling, 363, 364
 pollTalks function, 576
 polymorphism, 159
 pop method, 89, 105
 Popper, Karl, 345
 port, 323, 461, 537, 538
 pose, 439
 position, 348
 position (CSS), 356, 362, 379, 392, 405, 406
 POST method, 464, 466, 478, 557
 postMessage method, 383
 power example, 72
 precedence, 19, 26, 353, 354
 predicate function, 130, 136, 141,

142
 Prefer header, 558, 568, 576
 premature optimization, 73
 preventDefault method, 369, 378, 380, 382, 417, 478, 504, 610
 previousSibling property, 339
 primitiveMultiply (exercise), 212, 591
 privacy, 331
 private (reserved word), 37
 private property, 213
 process object, 528, 544
 processor, 268
 profiling, 73
 program, 33, 40
 nature of, 3
 program size, 122, 247
 programming, 1
 difficulty of, 2
 history of, 5
 joy of, 1, 4
 programming language, 1, 4, 300, 338, 527
 power of, 8
 programming style, 4, 33, 46, 52
 progress bar, 378
 project chapter, 175, 300, 390, 495, 552
 promise, 298, 598
 Promise class, 273, 277, 283, 292, 293, 298, 467, 468, 486, 536, 541, 577, 596, 598
 Promise.all function, 290, 295, 298, 596, 598
 Promise.reject function, 277
 Promise.resolve function, 273
 promptDirection function, 208, 209
 promptNumber function, 200
 propagation, *see* event propagation
 proper lines (exercise), 524, 612
 property, 490
 access, 39, 87, 146, 191, 520
 assignment, 91
 definition, 90, 96, 161
 deletion, 91
 inheritance, 148, 150, 155
 model of, 91
 naming, 158, 163, 164
 private, 153
 public, 153

- testing for, 92 - protected (reserved word), 37 - protocol, 322-324, 461, 463 - prototype, 148-151, 155, 158, 169, 313, 319, 600 - diagram, 155 - prototype property, 151, 152 - pseudorandom number, 112 - public, 154 - public (reserved word), 37 - public space (exercise), 551, 615 - publishing, 534 - punch card, 5 - pure function, 79, 80, 119, 130, 263, 494, 609 - push method, 89, 102, 105, 589 - pushing data, 554 - PUT method, 462, 464, 541, 547, 557, 565, 615 - Pythagoras, 588 - Pythagorean theorem, 611 - quadratic curve, 433 - quadraticCurveTo method, 432, 606 - query string, 465, 466, 558, 567 - querySelector method, 355, 601	- querySelectorAll method, 354, 483 - question mark, 26, 220, 235, 465 - queue, 293 - quiet times (exercise), 297, 596 - quotation mark, 20, 248 - quoting - in JSON, 116 - of object properties, 90 - quoting style (exercise), 248, 592 - rabbit example, 146, 149, 151 - radian, 358, 434, 442 - radio button, 473, 482 - radius, 523, 612 - raising (exception), 202 - random number, 112, 401 - random-item package, 593 - randomPick function, 183 - randomRobot function, 183 - range, 129, 217, 221 - range function, 8, 118, 584 - Range header, 468 - ray tracer, 456 - read-eval-print loop, 528 - readability, 6, 8, 53, 72, 79, 201, 249, 309, 408, 458 - readable stream, 539, 542, 565
---	---

- readAsDataURL method, 515 - readAsText method, 486 - readdir function, 535, 546, 614 - readdirSync function, 614 - readFile function, 259, 534, 616 - readFileSync function, 536, 614 - reading code, 11, 175 - readTextFile function, 271 - real promises (exercise), 298, 596 - real-time, 363 - reasoning, 25 - recipe analogv, 123 - record, 90 - rect (SVG tag), 425 - rectangle, 392, 410, 427, 428, 458, 510 - rectangle function, 510, 611 - recursion, 67, 72, 73, 82, 120, 283, 303, 306, 309, 340, 361, 445, 583, 586, 591, 601 - red, 500 - reduce method, 132, 134, 141, 142, 507, 588 - ReferenceError type, 319 - RegExp class, 215, 236, 614	- regex golf (exercise), 247 - regular expression, 214, 216, 218, 232, 234, 235, 237, 240, 247, 305, 550, 560, 561, 600, 614 - alternatives, 227 - backtracking, 229 - boundary, 226 - creation, 215, 236 - escaping, 215, 236, 592 - flags, 222, 232, 236, 593 - global, 232, 237, 239, 240 - grouping, 221, 222, 232 - internationalization, 218 - matching, 228, 237 - methods, 216, 224, 237 - repetition, 220 - rejecting (a promise), 276, 293, 298, 598 - relative path, 329, 529, 541, 615 - relative positioning, 356, 357 - relative URL, 467 - remainder operator, 19, 49, 440, 582, 603, 606 - remote access, 541 - remote procedure call, 470
--	--

removeChild method, 342	resolution, 252, 529
removeEventListener method, 365, 604	resolve function, 544
removeItem method, 488	resolving (a promise), 273, 276, 293
rename function, 535	resource, 322, 324, 462, 464, 470, 541, 564
rendering, 427	response, 462–464, 470, 538, 543, 546
renderTalk function, 574	Response class, 467
renderTalkForm function, 575	responsiveness, 363, 527
renderUserField function, 573	rest parameter, 110
repeat method, 109, 379	restore method, 445, 447
repeating key, 371	result property, 486
repetition, 76, 220, 221, 230, 235, 384	return keyword, 60, 67, 284, 583, 587
replace method, 232, 248, 592	return value, 39, 60, 200, 587
replaceChild method, 343, 602	reuse, 80, 170, 249, 254, 530
replaceSelection function, 480	reverse method, 119
reportError function, 573	reversing (exercise), 119, 585
request, 323, 462–464, 478, 537, 548, 553	rgb (CSS), 404
request function, 538, 539	right-aligning, 361
requestAnimationFrame function, 357, 382, 384, 418, 459, 603	rmdir function, 546, 550
require function, 257, 259, 267, 529, 531	roadGraph object, 176
reserved word, 37	roads array, 175
	roads module (exercise), 267, 595
	robot, 175, 177, 182, 185, 186,

189, 266	scale constant, 502
robot efficiency (exercise), 189, 590	scale method, 441, 444
robustness, 556	scaling, 403, 439, 441, 451, 608
root, 336	scheduling, 292, 597
rotate method, 442, 447	scientific notation, 18, 248
rotation, 458, 605	scope, 60–63, 70, 260, 308, 311, 312, 318, 319, 599, 600
rounding, 113, 199, 411, 450, 613	script (HTML tag), 329, 381, 382
roundTo example, 59, 69	SCRIPTS dataset, 129, 132, 136, 139, 143
router, 555, 560	scroll event, 378, 385
Router class, 560, 561	scrolling, 369, 378, 379, 406, 407, 417, 449
routeRobot function, 185	search method, 237
row, 361	search problem, 186, 228, 230, 341, 550
rule (CSS), 352–354	search tool (exercise), 550, 614
run function, 313	section, 241
run-time error, 196, 197, 200, 211, 600	Secure HTTP, <i>see</i> HTTPS
runAnimation function, 418, 422	security, 259, 330, 331, 469, 471, 485, 488, 543, 559
runGame function, 420, 421	security camera, 297
runLevel function, 419, 422	select (HTML tag), 474, 475, 483, 488, 496, 506, 507, 615
running code, 11	selected attribute, 483
runRobot function, 182, 590	selection, 479
Safari, 332	
sandbox, 11, 84, 330, 334, 469	
save method, 445, 447	
SaveButton class, 513	

- selectionEnd property, 479
- selectionStart property, 479
- selector, 354
- self-closing tag, 327
- semantic versioning, 533
- semicolon, 33, 48, 351
- sep binding, 544
- sequence, 220
- serialization, 116, 117
- serve-static package, 561
- server, 323, 324, 461–463, 467, 469, 470, 526, 537, 538, 540, 541, 553, 559
- session, 490
- sessionStorage object, 490
- set, 216, 218, 336
- set (data structure), 173, 190
- Set class, 173, 190, 590
- set method, 158
- setAttribute method, 346, 347, 501
- setInterval function, 384, 440
- setItem method, 488
- setter, 161
- setTimeout function, 271, 292, 384, 385, 568, 577
- shape, 424, 430, 434, 438, 458
- shapes (exercise), 458, 605
- shared property, 150, 155, 156
- shift key, 371, 610
- shift method, 105
- shiftKey property, 371
- short-circuit evaluation, 31, 75, 310, 587
- SICP, 300
- side effect, 33, 34, 39, 50, 60, 80, 94, 119, 130, 238, 262, 295, 338, 342, 343, 349, 429, 445, 466, 497, 499
- sign, 17, 248, 592
- sign bit, 17
- signal, 15
- simplicity, 316
- simulation, 178, 182, 390, 398, 493, 603
- sine, 111, 358, 401, 415
- single-quote character, 20, 248, 330
- singleton, 190
- skill, 496
- skill-sharing, 552
- skill-sharing project, 552, 553, 556,

- 559, 570
- SkillShareApp class, 578
- skipSpace function, 304, 319
- slash character, 19, 53, 215, 234, 468, 544
- slice method, 106, 107, 129, 344, 585, 599
- slope, 613
- sloppy programming, 386
- smooth animation, 357
- SMTP, 323
- social factors, 521
- socket, 555
- some method, 136, 142
- sorting, 336
- source property, 237
- special form, 301, 308, 309
- special return value, 200, 201
- specialForms object, 309
- specificity, 354
- speed, 1, 3, 459, 607
- spiral, 458, 606
- split method, 108, 177, 394, 596
- spread, 109, 110, 168, 490, 500
- spread operator, 404
- sprite, 439, 440, 451, 452
- spy, 378
- square, 40
- square bracket, 165
- square brackets, 86, 87, 114, 218, 483, 490, 584
- square example, 59, 64, 65
- square root, 100, 111, 588
- src attribute, 327, 329
- stack, *see* call stack, 89
- stack overflow, 67, 72, 83, 583
- stack trace, 203
- standard, 9, 37, 52, 130, 203, 218, 521, 527, 529
- standard environment, 37
- standard output, 528
- standards, 321, 332
- star, 458, 606
- Star Trek, 433
- startPixelEditor function, 519
- startState constant, 519
- startsWith method, 543
- stat function, 535, 544, 546, 614
- state, 48, 178
 - in
 - binding, 34, 45, 50
 - iterator, 286

objects, 178, 395	notation, 20
in objects, 447	properties, 107
of application, 405, 496, 502, 517, 580	representation, 21
of canvas, 428, 445	searching, 107
persistence, 510	String function, 40, 159
transitions, 294, 497, 500	stroke method, 430, 431, 433
statement, 33, 34, 40, 45, 48, 59, 91	strokeRect method, 428, 607
static (reserved word), 37	strokeStyle property, 429
static file, 556, 561	strokeText method, 437, 438
static method, 162, 173, 395	stroking, 428, 429, 437, 457
static property, 162, 591	strong (HTML tag), 347, 350
Stats type, 546	structure, 254, 326, 334, 497
statSync function, 614	structure sharing, 120
status code, 463, 528	style, 350
status property, 467, 572	style (HTML tag), 352, 353
stoicism, 363	style attribute, 350, 351, 353, 402
stopPropagation method, 367	style sheet, <i>see</i> CSS
stream, 323, 538, 539, 542, 548	subclass, 169
strict mode, 192, 193	submit, 473, 477, 478
string, 20, 86, 88, 94, 137	submit event, 478, 479, 575, 615
indexing, 83, 106, 109, 137, 222	substitution, 80
length, 55, 137	subtraction, 19, 172
methods, 107, 223	sum function, 8, 118
	summing (exercise), 118, 584
	summing example, 6, 122, 132, 313

superclass, 169	string, 20
survey, 435	syntax tree, 302, 303, 306, 307, 336
Sussman, Gerald, 300	SyntaxError type, 305
SVG, 424, 425, 428, 455, 456	tab character, 21, 47
swapping bindings, 613	tab key, 476
swipe, 510	tabbed interface (exercise), 389, 603
switch keyword, 51	tabindex attribute, 372, 476, 522
symbol, 163	table, 99, 100, 404
Symbol function, 163	table (HTML tag), 360, 392, 404, 610
Symbol.iterator symbol, 165	table example, 601
synchronization, 578, 616	tableFor function, 100
synchronous programming, 269, 283, 536, 550	tag, 325, 327, 334, 353
syncState method, 498, 503, 507, 508, 523, 616	talk, 552, 553, 564–566
syntax	talkResponse method, 567
error, 37, 191, 192	talksAbout function, 340
expression, 32	talkURL function, 572
function, 59, 64	Tamil, 128
identifier, 37	tampering, 471
number, 17, 248	tangent, 111
object, 90	target property, 368
of Egg, 300, 301	task management example, 105
operator, 18	TCP, 323, 324, 461, 556
statement, 33, 34, 41, 44, 47, 50, 202	td (HTML tag), 361, 404

temperature example, 161	textFile function, 274, 277, 297, 596
template, 257, 580, 616	textScripts function, 140, 588
template literals, 22	th (HTML tag), 361
templating language, 575	then method, 273, 274, 276, 277, 596, 598
tentacle (analogy), 35, 91, 95	theory, 199
terminal, 528	this binding, 89, 146, 147, 151, 193
ternary operator, 26, 31, 309	thread, 269, 270, 293, 383
test method, 216	throw keyword, 202, 203, 209, 212, 591
test runners, 197	tile, 451
test suite, 196	time, 217, 221, 224, 357, 386, 409, 410, 414, 419, 452, 517
test suites, 197	time zone, 225
testing, 189, 196	timeline, 269, 292, 329, 357, 363, 382
text, 20, 325, 326, 334, 337, 437, 455, 456, 459, 479, 483, 535, 609	timeout, 384, 556, 558, 568
text field, 380, 473–475, 479–481	times method, 398
text method, 468	timestamp, 225, 297
text node, 337, 340, 344, 603	title, 570
text wrapping, 455	title (HTML tag), 326, 328
text-align (CSS), 361	toDataURL method, 513
TEXT_NODE code, 337, 603	toLowerCase method, 88, 361
textAlign property, 438, 606	
textarea (HTML tag), 385, 474, 479, 480, 488, 493, 615	
textBaseline property, 438, 606	
textContent property, 602, 609	

tool, 214, 246, 261, 496, 506, 507, 509–512, 519, 523, 524, 533	transpilation, 316
tool property, 498	trapezoid, 458, 605
ToolSelect class, 507	traversal, 228
top (CSS), 356, 357, 359, 362	tree, 302, 336, 337
top-level scope, <i>see</i> global scope	trial and error, 198, 434
toString method, 148, 156, 157, 159, 517, 540	triangle (exercise), 55, 581
touch, 376, 496	trigonometry, 112, 358
touchend event, 377	trim method, 108, 394
touches method, 410	true, 24
touches property, 377, 505	trust, 330
touchmove event, 377, 504, 524	try keyword, 203, 205, 591, 592, 609
touchstart event, 377, 502, 504	type, 16, 23, 144, 170
toUpperCase method, 88, 196, 361, 540	type attribute, 472, 478
tr (HTML tag), 361, 404	type checking, 195, 261
trackKeys function, 417, 422	type coercion, 27–29, 40
transform (CSS), 424	type property, 301, 366
transformation, 441, 442, 444, 445, 459, 605	type variable, 195
translate method, 442, 444	typeof operator, 23, 121, 586
Transmission Control Protocol, <i>see</i> TCP	TypeScript, 195
transparency, 516	typing, 385
transparent, 426, 440	typo, 191
	unary operator, 23, 32
	uncaught exception, 207, 279
	undefined, 27, 28, 30, 35, 60, 68, 87, 91, 114, 192, 193, 200

underline, 350	URL encoding, 465
underscore character, 36, 52, 226, 236	url package, 567
undo history, 517, 518	urlPath function, 543
UndoButton class, 519	usability, 370
Unicode, 21, 22, 25, 128, 137, 217–219	use strict, <i>see</i> strict mode
property, 219	user experience, 363, 477, 554, 573
unicycling, 552	user interface, 207, 497
uniform resource locator, <i>see</i> URL	UTF16, 22, 137
uniformity, 301	UTF8, 535
uniqueness, 353	validation, 200, 211, 300, 408, 479, 565
unit (CSS), 359, 379	value, 16
Unix, 546, 548, 550	value attribute, 473, 479, 483
Unix time, 225	var keyword, 36, 60, 61, 114
unlink function, 535, 546	variable, <i>see</i> binding
unshift method, 105	Vec class, 172, 395, 396, 414, 607
unwinding the stack, 202	vector (exercise), 172, 588
upcasing server example, 540	vector graphics, 438
updated method, 565, 569, 616	verbosity, 66, 270
updateState function, 500	version, 254, 326, 463, 532, 533
upgrading, 254	viewport, 406, 408, 447, 449, 454
upload, 485	VillageState class, 178
URL, 324, 330, 425, 464, 467, 471, 538, 557, 572	virtual keyboard, 372
URL class, 543	virtual world, 175, 178, 182
	virus, 330

vocabulary, 58, 123, 124	matching, 217, 218
void operator, 37	syntax, 52, 301, 304, 600
volatile data storage, 16	trimming, 108, 394
	why, 32
waitForChanges method, 568	width property, 523, 611
waiting, 271	window, 367, 376, 381
walking, 452	window object, 364
wave, 401, 415	with statement, 194
Web, <i>see</i> World Wide Web	wizard (mighty), 5
web application, 8, 487, 495	word boundary, 226
web browser, <i>see</i> browser	word character, 217, 218, 226
web page, 261	work list, 187, 512
web worker, 383	workbench (exercise), 493, 609
WebDAV, 551	world, 390
webgl (canvas context), 426	World Wide Web, 8, 116, 321, 324, 330, 331, 461
website, 330, 331, 464, 526, 551, 552	writable stream, 538, 539, 543
WebSockets, 555	write method, 538, 539
weekDay module, 251	writeFile function, 535, 539, 616
weekend project, 551	writeHead method, 537
weresquirrel example, 85, 90, 93, 96, 102, 104	writing code, 11, 175
while loop, 7, 45, 47, 78	writing system, 128
whitespace, 319	WWW, <i>see</i> World Wide Web
in HTML, 341, 507, 603	XML, 338, 425
in URLs, 557	XML namespace, 425
indentation, 46	xmlns attribute, 425

yield (reserved word), 37
yield keyword, 286
your own loop (example), 142
Yuan-Ma, 15, 249, 526

Zawinski, Jamie, 214
zero-based counting, 83, 86, 224
zeroPad function, 79
zigzag, 606
zooming, 455



JavaScript from Beginner to Professional

Learn JavaScript quickly by building fun, interactive, and dynamic web apps, games, and pages

Laurence Lars Svekis
Maalke van Putten
Rob Percival

Packt

BIRMINGHAM—MUMBAI

JavaScript from Beginner to Professional

Copyright © 2021 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the authors, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Producer: Tushar Gupta

Acquisition Editor - Peer Reviews: Divya Mudaliar

Project Editor: Namrata Katare

Content Development Editor: Edward Doney

Copy Editor: Safis Editing

Technical Editor: Karan Sonawane

Proofreader: Safis Editing

Indexer: Sejal Dsilva

Presentation Designer: Pranit Padwal

First published: December 2021

Production reference: 1081221

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham
B3 2PB, UK.

ISBN 978-1-80056-252-3

www.packt.com

Contributors

About the authors

Laurence Lars Svekiš is an innovative technology expert with a wide range of expertise and real-world experience in web development, having worked on various web development projects, both large and small, since 1999. He has been a top course instructor since 2015, and has a passion for bringing ideas to life online. Teaching and helping others has been an amazing opportunity for him, as he enjoys sharing knowledge with others. He has an enthusiasm for education and a desire to help others experience the joy of application development and web design.

Thanks to Alexis and Sebastian for all their support.

Maaïke van Putten is a software developer and trainer with a passion for software development and helping others get to the next level in their career. Some of her favorite languages are JavaScript, Java, and Python. She participates as a developer in software development projects and as a trainer in various topics, ranging from IT for dummies to advanced topics for senior software developers. Next to that, she loves to create online content for diverse platforms to help larger audiences.

Rob Percival is a highly regarded web developer and Udemy instructor with over 1.7 million students. Over 500,000 of them have taken Rob's Complete Web Developer Course 2.0, as well as his Android Developer and iOS Developer courses.

About the reviewer

Chris Minnick is a prolific author, blogger, trainer, speaker, and web developer. His company, *WatrThis?*, is dedicated to finding better ways to teach computer and programming skills to beginners.

Chris has been a full-stack developer for over 25 years and a trainer for over 10 years. He has taught web development, ReactJS, and advanced JavaScript at many of the world's largest companies, as well as at public libraries, co-working spaces, and meetups.

Minnick has authored and co-authored over a dozen technical books for adults and kids, including *React JS Foundations*, *HTML and CSS for Dummies*, *Coding with JavaScript for Dummies*, *JavaScript for Kids*, *Adventures in Coding*, and *Writing Computer Code*.

Table of Contents

Preface	xix
Chapter 1: Getting Started with JavaScript	1
Why should you learn JavaScript?	2
Setting up your environment	3
Integrated Development Environment	3
Web browser	4
Extra tools	4
Online editor	4
How does the browser understand JavaScript?	4
Using the browser console	5
Practice exercise 1.1	7
Adding JavaScript to a web page	8
Directly in HTML	8
Practice exercise 1.2	9
Linking an external file to our web page	9
Practice exercise 1.3	10
Writing JavaScript code	11
Formatting code	11
Indentations and whitespace	11
Semicolons	12
Code comments	13
Practice exercise 1.4	13
Prompt	14
Random numbers	14
Chapter project	15
Creating an HTML file and a linked JavaScript file	15
Self-check quiz	16
Summary	16

Table of Contents

Chapter 2: JavaScript Essentials	17
Variables	18
Declaring variables	18
let, var, and const	19
Naming variables	20
Primitive data types	20
String	21
Escape characters	22
Number	23
BigInt	24
Boolean	25
Symbol	25
Undefined	26
null	26
Analyzing and modifying data types	27
Working out the type of a variable	28
Converting data types	29
Practice exercise 2.1	32
Operators	32
Arithmetic operators	32
Addition	32
Subtraction	33
Multiplication	33
Division	34
Exponentiation	34
Modulus	35
Unary operators: increment and decrement	36
Combining the operators	37
Practice exercise 2.3	38
Assignment operators	38
Practice exercise 2.4	39
Comparison operators	40
Equal	40
Not equal	40
Greater than and smaller than	41
Logical operators	42
And	42
Or	43
Not	43
Chapter project	43
Miles-to-kilometers converter	43
BMI calculator	44
Self-check quiz	44
Summary	45

Chapter 3: JavaScript Multiple Values	47
Arrays and their properties	48
Creating arrays	48
Accessing elements	50
Overwriting elements	50
Built-in length property	51
Practice exercise 3.1	53
Array methods	53
Adding and replacing elements	53
Deleting elements	55
Finding elements	56
Sorting	58
Reversing	58
Practice exercise 3.2	59
Multidimensional arrays	59
Practice exercise 3.3	60
Objects in JavaScript	61
Updating objects	62
Practice exercise 3.4	63
Working with objects and arrays	64
Objects in objects	64
Arrays in objects	65
Objects in arrays	66
Objects in arrays in objects	66
Practice exercise 3.5	67
Chapter projects	68
Manipulating an array	68
Company product catalog	68
Self-check quiz	69
Summary	69
Chapter 4: Logic Statements	71
if and if else statements	72
Practice exercise 4.1	73
else if statements	74
Practice exercise 4.2	75
Conditional ternary operators	76
Practice exercise 4.3	76
switch statements	77
The default case	79
Practice exercise 4.4	80

Combining cases	81
Practice exercise 4.5	82
Chapter projects	82
Evaluating a number game	82
Friend checker game	83
Rock Paper Scissors game	83
Self-check quiz	83
Summary	86
Chapter 5: Loops	87
while loops	88
Practice exercise 5.1	90
do while loops	91
Practice exercise 5.2	92
for loops	92
Practice exercise 5.3	95
Nested loops	95
Practice exercise 5.4	97
Loops and arrays	98
Practice exercise 5.5	100
for of loop	101
Practice exercise 5.6	102
Loops and objects	102
for in loop	103
Practice exercise 5.7	104
Looping over objects by converting to an array	104
break and continue	107
break	108
continue	109
Practice exercise 5.8	111
break, continue, and nested loops	111
break and continue and labeled blocks	113
Chapter project	114
Math multiplication table	114
Self-check quiz	115
Summary	115
Chapter 6: Functions	117
Basic functions	118
Invoking functions	118
Writing functions	118

Table of Contents

Naming functions	119
Practice exercise 6.1	120
Practice exercise 6.2	120
Parameters and arguments	120
Practice exercise 6.3	122
Default or unsuitable parameters	122
Special functions and operators	123
Arrow functions	124
Spread operator	125
Rest parameter	127
Returning function values	127
Practice exercise 6.4	129
Returning with arrow functions	129
Variable scope in functions	130
Local variables in functions	130
let versus var variables	132
const scope	133
Global variables	134
Immediately invoked function expression	136
Practice exercise 6.5	137
Recursive functions	138
Practice exercise 6.6	140
Nested functions	140
Practice exercise 6.7	142
Anonymous functions	142
Practice exercise 6.8	143
Function callbacks	143
Chapter projects	145
Create a recursive function	145
Set timeout order	145
Self-check quiz	146
Summary	147
Chapter 7: Classes	149
Object-oriented programming	150
Classes and objects	150
Classes	152
Constructors	152
Practice exercise 7.1	154
Methods	154
Practice exercise 7.2	156

Table of Contents

Properties	156
Getters and setters	157
Inheritance	159
Prototypes	161
Practice exercise 7.3	163
Chapter projects	163
Employee tracking app	163
Menu items price calculator	163
Self-check quiz	164
Summary	165
Chapter 8: Built-In JavaScript Methods	167
Introduction to built-in JavaScript methods	168
Global methods	169
Decoding and encoding URIs	169
decodeUri() and encodeUri()	170
decodeURIComponent() and encodeURIComponent()	171
Encoding with escape() and unescape()	172
Practice exercise 8.1	172
Parsing numbers	172
Making integers with parseInt()	172
Making floats with parseFloat()	174
Executing JavaScript with eval()	175
Array methods	176
Performing a certain action for every item	176
Filtering an array	177
Checking a condition for all elements	178
Replacing part of an array with another part of the array	178
Mapping the values of an array	179
Finding the last occurrence in an array	179
Practice exercise 8.2	180
Practice exercise 8.3	181
String methods	181
Combining strings	181
Converting a string to an array	182
Converting an array to a string	182
Working with index and positions	183
Creating substrings	185
Replacing parts of the string	185
Uppercase and lowercase	186
The start and end of a string	187

Table of Contents

Practice exercise 8.4	188
Practice exercise 8.5	189
Number methods	190
Checking if something is (not) a number	190
Checking if something is finite	190
Checking if something is an integer	191
Specifying a number of decimals	192
Specifying precision	192
Math methods	192
Finding the highest and lowest number	193
Square root and raising to the power of	193
Turning decimals into integers	194
Exponent and logarithm	196
Practice exercise 8.6	196
Date methods	196
Creating dates	197
Methods to get and set the elements of a date	198
Parsing dates	200
Converting a date to a string	200
Practice exercise 8.7	201
Chapter projects	201
Word scrambler	201
Countdown timer	202
Self-check quiz	203
Summary	204
Chapter 9: The Document Object Model	205
HTML crash course	206
HTML elements	206
HTML attributes	210
The BOM	211
Practice exercise 9.1	214
Window history object	214
Window navigator object	215
Window location object	216
Practice exercise 9.2	217
The DOM	217
Additional DOM properties	219
Selecting page elements	220
Practice exercise 9.3	221

Table of Contents

Chapter project	222
Manipulating HTML elements with JavaScript	222
Self-check quiz	222
Summary	223
Chapter 10: Dynamic Element Manipulation Using the DOM	225
Basic DOM traversing	226
Practice exercise 10.1	228
Selecting elements as objects	229
Changing innerText	230
Changing innerHTML	230
Accessing elements in the DOM	230
Accessing elements by ID	231
Practice exercise 10.2	232
Accessing elements by tag name	232
Practice exercise 10.3	233
Accessing elements by class name	234
Practice exercise 10.4	234
Accessing elements with a CSS selector	234
Using querySelector()	235
Using querySelectorAll()	236
Element click handler	237
This and the DOM	238
Practice exercise 10.7	239
Manipulating element style	240
Changing the classes of an element	243
Adding classes to elements	243
Removing classes from elements	244
Toggling classes	246
Manipulating attributes	247
Practice exercise 10.8	249
Event listeners on elements	251
Practice exercise 10.9	253
Creating new elements	253
Practice exercise 10.10	255
Chapter projects	256
Collapsible accordion component	256
Interactive voting system	258
Hangman game	259
Self-check quiz	262
Summary	263

Chapter 11: Interactive Content and Event Listeners	265
Introducing interactive content	266
Specifying events	266
Specifying events with HTML	266
Specifying events with JavaScript	266
Practice exercise 11.1	267
Specifying events with event listeners	267
Practice exercise 11.2	268
The onload event handler	268
Practice exercise 11.3	269
Mouse event handlers	270
Practice exercise 11.4	272
The event target property	272
Practice exercise 11.5	274
DOM event flow	275
Practice exercise 11.6	278
onchange and onblur	280
Practice exercise 11.7	281
Key event handler	282
Practice exercise 11.8	284
Drag and drop elements	285
Practice exercise 11.9	287
Form submission	289
Practice exercise 11.10	291
Animating elements	292
Practice exercise 11.11	293
Chapter projects	294
Build your own analytics	294
Star rating system	295
Mouse position tracker	296
Box clicker speed test game	297
Self-check quiz	299
Summary	299
Chapter 12: Intermediate JavaScript	301
Regular expressions	302
Specifying multiple options for words	303
Character options	304
Groups	307

Practical regex	309
Searching and replacing strings	310
Email validation	312
Functions and the arguments object	314
Practice exercise 12.3	315
JavaScript hoisting	315
Using strict mode	316
Debugging	317
Breakpoints	318
Practice exercise 12.4	320
Error handling	325
Practice exercise 12.5	326
Using cookies	327
Practice exercise 12.6	329
Local storage	330
Practice exercise 12.7	332
JSON	333
Practice exercise 12.8	336
Parsing JSON	336
Practice exercise 12.9	337
Practice exercise 12.9 answers	337
Chapter projects	338
Email extractor	338
Form validator	339
Simple math quiz	341
Self-check quiz	342
Summary	343
Chapter 13: Concurrency	345
Introducing concurrency	346
Callbacks	346
Practice exercise 13.1	349
Promises	349
Practice exercise 13.2	352
async and await	353
Practice exercise 13.3	354
Event loop	355
Call stack and callback queue	355
Chapter project	359
Password checker	359
Self-check quiz	360
Summary	361

Chapter 14: HTML5, Canvas, and JavaScript	363
Introducing HTML5 with JavaScript	364
Local file reader	365
Uploading files	365
Reading files	367
Practice exercise 14.1	368
Getting position data with GeoLocation	369
HTML5 canvas	370
Practice exercise 14.2	373
Dynamic canvas	373
Adding lines and circles to the canvas	374
Practice exercise 14.3	376
Adding text to the canvas	377
Practice exercise 14.4	378
Adding and uploading images to the canvas	380
Practice exercise 14.5	383
Adding animations to the canvas	383
Practice exercise 14.6	385
Drawing on canvas with a mouse	386
Practice exercise 14.7	389
Saving dynamic images	390
Media on the page	392
Digital accessibility in HTML	394
Chapter projects	395
Create a Matrix effect	395
Countdown clock	397
Online paint app	399
Self-check quiz	401
Summary	402
Chapter 15: Next Steps	403
Libraries and frameworks	404
Libraries	405
jQuery	405
D3	408
Underscore	409
React	410
Frameworks	412
Vue.js	412
Angular	414
Learning the backend	415
APIs	416

AJAX	417
Practice exercise 15.1	418
Node.js	418
Using the Express Node.js framework	419
Next steps	420
Chapter projects	421
Working with JSON	421
List-making project	421
Self-check quiz	422
Summary	423
Appendix – Practice Exercise, Project, and Self-Check Quiz Answers	425
Chapter 1, Getting Started with JavaScript	425
Practice exercises	425
Projects	426
Creating an HTML file and a linked JavaScript file	426
Self-check quiz	427
Chapter 2, JavaScript Essentials	427
Practice exercises	427
Projects	428
Miles-to-kilometers converter	428
BMI calculator	428
Self-check quiz	429
Chapter 3, JavaScript Multiple Values	429
Practice exercises	429
Projects	431
Manipulating an array	431
Company product catalog	431
Self-check quiz	432
Chapter 4, Logic Statements	432
Practice exercises	432
Projects	434
Evaluating a number game answers	434
Friend checker game answers	435
Rock paper scissors game answers	435
Self-check quiz	436
Chapter 5, Loops	436
Practice exercises	436
Project	441
Math multiplication table	441

Table of Contents

Self-check quiz	442
Chapter 6, Functions	442
Practice exercises	442
Projects	445
Create a recursive function	445
Set timeout order	445
Self-check quiz	446
Chapter 7, Classes	446
Practice exercises	446
Projects	448
Employee tracking app	448
Menu items price calculator	448
Self-check quiz	449
Chapter 8, Built-In JavaScript Methods	449
Practice exercises	449
Projects	452
Word scrambler	452
Countdown timer	452
Self-check quiz	453
Chapter 9, The Document Object Model	453
Practice exercises	453
Projects	454
Manipulating HTML elements with JavaScript	454
Self-check quiz	455
Chapter 10, Dynamic Element Manipulation Using the DOM	456
Practice exercises	456
Projects	461
Collapsible accordion component	461
Interactive voting system	461
Hangman game	462
Self-check quiz	464
Chapter 11, Interactive Content and Event Listeners	465
Practice exercises	465
Projects	473
Build your own analytics	473
Star rater system	474
Mouse position tracker	475
Box clicker speed test game	476
Self-check quiz	477
Chapter 12, Intermediate JavaScript	477
Practice exercises	477

Table of Contents

Projects	482
Email extractor	482
Form validator	483
Simple math quiz	484
Self-check quiz	485
Chapter 13, Concurrency	486
Practice exercises	486
Projects	487
Password checker	487
Self-check quiz	488
Chapter 14, HTML5, Canvas, and JavaScript	489
Practice exercises	489
Projects	496
Create a Matrix effect	496
Countdown clock	497
Online paint app	499
Self-check quiz	500
Chapter 15, Next Steps	501
Practice exercises	501
Projects	501
Working with JSON	501
List-making project	502
Self-check quiz	504
Other Books You May Enjoy	507
Index	513

Preface

JavaScript is an amazing multi-functional language that is used a lot for web development (among other things). Any interaction that happens on web pages is JavaScript in action. In fact, all modern browsers understand JavaScript – and soon you will understand it too.

This book deals with everything you need to know to create JavaScript applications and use JavaScript on web pages. By the time you finish this book, you'll be capable of creating interactive web pages, dynamic applications, and a lot more as you progress on your professional JavaScript journey!

Who this book is for

To get started with this book, you don't need any JavaScript experience. However, if you do have some coding experience, you're likely to go through the book and exercises with a bit more ease. Basic familiarity with HTML and CSS would be of benefit. If you're a first-time programmer, we are honored to welcome you to the world of programming in this book. It may seem difficult in the beginning, but we'll guide you right through it.

What this book covers

Chapter 1, Getting Started with JavaScript, covers some fundamentals of the JavaScript language that you'll have to know to understand the rest of the book.

Chapter 2, JavaScript Essentials, deals with essentials such as variables, data types, and operators.

Chapter 3, JavaScript Multiple Values, covers how to store multiple values in one variable using arrays and objects.

Preface

Chapter 4, Logic Statements, is where the real fun starts: we are going to use logic statements to make decisions for us!

Chapter 5, Loops, accounts for situations when it is necessary to repeat a block of code, which is what we use loops for. We are using different types of loops, such as the `for` and the `while` loop.

Chapter 6, Functions, introduces a very useful block for repeating code snippets: functions! This enables us to invoke a specified code block at any time in our script to do something for us. This will help you to not repeat yourself, which is one of the fundamental principles of writing clean code.

Chapter 7, Classes, continues with building blocks of JavaScript that help us to structure our application better. We have already seen how to create objects, and with classes we learn how to create a template for objects that we can reuse anytime we need that particular type of object.

Chapter 8, Built-In JavaScript Methods, deals with some great built-in functionality. Functions are something we can write ourselves, but we'll find ourselves using the built-in JavaScript functions often whenever we need to do common tasks, such as checking whether something is a number or not.

Chapter 9, The Document Object Model, dives into the browser object model and document object model (DOM). This is going to enrich the way we can use JavaScript by a lot. We'll learn what the DOM is, and how we can affect it with JavaScript and change our websites by doing so.

Chapter 10, Dynamic Element Manipulation Using the DOM, demonstrates how to manipulate the elements of the DOM dynamically, which will enable you to create modern user experiences. We can change our website as a response to user behavior such as clicking on a button.

Chapter 11, Interactive Content and Event Listeners, takes our responses to the user to the next level. For example, we are going to learn how to respond to events such as the cursor leaving an input box and the mouse of the user moving.

Chapter 12, Intermediate JavaScript, deals with topics that you'll need to write intermediate JavaScript code, such as regular expressions, recursion, and debugging, to boost the performance of your code.

Chapter 13, Concurrency, introduces the topic of concurrency and asynchronous programming, which will allow our code to do multiple things at the same time and be truly flexible.

Chapter 14, HTML5, Canvas, and JavaScript, focuses on HTML5 and JavaScript. We'll have seen a lot of both HTML and JavaScript in the previous chapters, but here we'll be focusing on the HTML5-specific features, such as the canvas element.

Chapter 15, Next Steps, explores the next steps you could take after you've gotten all the fundamental features of JavaScript down and you are able to write nifty programs using JavaScript. We'll take a look at some of the famous JavaScript libraries and development frameworks, such as Angular, React, and Vue, and we'll have a look at Node.js to see how the backend can be written in JavaScript.

To get the most out of this book

Previous coding experience will help, but is definitely not required. If you have a computer with a text editor (such as Notepad or TextEdit, not Word!) and a browser, you can get started with this book. We encourage you to engage with the exercises and projects, and experiment continually while you go through the chapters, to ensure you are comfortable with each concept before moving on.

Download the example code files

The code bundle for the book is also hosted on GitHub at <https://github.com/PacktPublishing/JavaScript-from-Beginner-to-Professional>. We also have other code bundles from our rich catalog of books and videos available at <https://github.com/PacktPublishing/>. Check them out!

Download the color images

We also provide a PDF file that has color images of the screenshots/diagrams used in this book. You can download it here: https://static.packt-cdn.com/downloads/9781800562523_ColorImages.pdf.

Conventions used

There are a number of text conventions used throughout this book.

CodeInText: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. For example: "We also need to let the browser know what kind of document we're working on with the `<!DOCTYPE>` declaration."

A block of code is set as follows:

```
<html>
<script type="text/javascript">
  alert("Hi there!");
</script>
</html>
```

Any command-line input or output is written as follows:

```
console.log("Hello world!")
```

Bold: Indicates a new term, an important word, or words that you see on the screen, for example, in menus or dialog boxes, also appear in the text like this. For example: "If you right-click and select **Inspect** on macOS systems, you will see a screen appear, similar to the one in the following screenshot."



Warnings or important notes appear like this.



Tips and tricks appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: Email feedback@packtpub.com, and mention the book's title in the subject of your message. If you have questions about any aspect of this book, please email us at questions@packtpub.com.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book we would be grateful if you would report this to us. Please visit, <http://www.packtpub.com/submit-errata>, selecting your book, clicking on the Errata Submission Form link, and entering the details.

Piracy: If you come across any illegal copies of our works in any form on the Internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packtpub.com with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit <http://authors.packtpub.com>.

Share Your Thoughts

Once you've read *JavaScript from Beginner to Professional*, we'd love to hear your thoughts! Please [click here to go straight to the Amazon review page](#) for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

1

Getting Started with JavaScript

It appears you have decided to start learning JavaScript. Excellent choice! JavaScript is a programming language that can be used on both the server side and client side of applications. The server side of an application is the backend logic that usually runs on computers in data centers and interacts with the database, while the client side is what runs on the device of the user, often the browser for JavaScript.

It is not unlikely that you have used functionality written in JavaScript. If you have used a web browser, such as Chrome, Firefox, Safari, or Edge, then you definitely have. JavaScript is all over the web. If you enter a web page and it asks you to accept cookies and you click OK, the popup disappears. This is JavaScript in action. And if you want to navigate a website and a sub-menu opens up, that means more JavaScript. Often, when you filter products in a web shop, this involves JavaScript. And what about these chats that start talking to you after you have been on a website for a certain number of seconds? Well, you guessed it—JavaScript!

Pretty much any interaction we have with web pages is because of JavaScript; the buttons you are clicking, birthday cards you are creating, and calculations you are doing. Anything that requires more than a static web page needs JavaScript.

In this chapter, we will cover the following topics:

- Why should you learn JavaScript?
- Setting up your environment
- How does the browser understand JavaScript?

- Using the browser console
- Adding JavaScript to a web page
- Writing JavaScript code



Note: exercise, project and self-check quiz answers can be found in the *Appendix*.

Why should you learn JavaScript?

There are many reasons why you should want to learn JavaScript. JavaScript originates from 1995, and is often considered the most widely used programming language. This is because JavaScript is the language that web browsers support and understand. You have everything you need to interpret it already installed on your computer if you have a web browser and text editor. There are better setups, however, and we will discuss these later in this chapter.

It is a great programming language for beginners, and most advanced software developers will know at least some JavaScript because they will have run into it at some point. JavaScript is a great choice for beginners for a number of reasons. The first reason is that you can start building really cool apps using JavaScript sooner than you could imagine. By the time you get to *Chapter 5, Leaps*, you will be able to write quite complex scripts that interact with users. And by the end of the book, you will be able to write dynamic web pages to do all sorts of things.

JavaScript can be used to write many different types of applications and scripts. It can be used for programming for the web browser, but also the logic layer of code that we cannot see (such as communication with the database) of an application can be programmed in JavaScript, along with games, automation scripts, and a plethora of other purposes. JavaScript can also be used for different programming styles, by which we mean ways to structure and write code. How you would go about this depends on the purpose of your script. If you've never coded before, you may not quite grasp these concepts, and it's not entirely necessary to at this stage, but JavaScript can be used for (semi) object-oriented, functional, and procedural programming, which are just different programming paradigms.

There are a ton of libraries and frameworks you can use once you get the basics of JavaScript down. These libraries and frameworks will really enhance your software life and make it a lot easier and possible to get more done in less time. Examples of these great libraries and frameworks include React, Vue.js, jQuery, Angular, and Node.js. Don't worry about these for now; just see them as a bonus for later. We will cover some of them briefly at the very end of this book.

Finally, we'll mention the JavaScript community. JavaScript is a very popular programming language, and many people are using it. As a beginner in particular, there won't be a problem for which you cannot find a solution on the internet.

The community of JavaScript is huge. The popular Stack Overflow forum contains lots of help for all sorts of coding issues and has an enormous section on JavaScript. You'll find yourself running into this web page a lot while googling problems and tips and tricks.

If JavaScript is your first programming language, you are new to the whole software community and you are in for a treat. Software developers, no matter the language, love to help one another. There are forums and tutorials online and you can find answers to almost all your questions. As a beginner, it can be hard to understand all the answers though. Just hang in there, keep trying and learning, and you will understand it soon enough.

Setting up your environment

There are many ways in which you can set up a JavaScript coding environment. For starters, your computer probably already has all the minimal things you will need to code JavaScript. We recommend you make your life a little bit easier and use an IDE.

Integrated Development Environment

An **Integrated Development Environment (IDE)** is a special application that is used to write, run, and debug code. You can just open it like you would any program. For example, to write a text document, you need to open the program, select the right file, and start to write. Coding is similar. You open the IDE and write the code. If you want to execute the code, the IDE often has a special button for this. Pressing this button will run the code from inside the IDE. For JavaScript, you might find yourself opening your browser manually in certain cases though.

An IDE does do more than that though; it usually has syntax highlighting. This means that certain elements in your code will have a certain color, and you can easily see when something is going wrong. Another great feature is the autosuggest feature, where the editor helps you with the options you have at the place where you are coding. This is often called code completion. Many IDEs have special plugins so you can make working with other tools more intuitive and add features to it, for example, a hot reload in the browser.

There are many IDEs out there and they differ in what they have to offer. We use Visual Studio Code throughout the book, but that's a personal preference. Other popular ones at the time of writing include Atom, Sublime Text, and WebStorm.

There are many IDEs and they keep on appearing, so chances are the most popular one at the time you are reading is not on this list. There are many other options. You can do a quick search on the web for JavaScript IDEs. There are a few things to pay attention to when selecting an IDE. Make sure that it supports syntax highlighting, debugging, and code completion for JavaScript.

Web browser

You will also need a web browser. Most browsers are perfectly fine for this, but it's better not to use Internet Explorer, which doesn't support the latest JavaScript features. Two good options would be Chrome and Firefox. They support the latest JavaScript features and helpful plugins are available.

Extra tools

There are many extra things you can use while coding, for example, browser plugins that will help you to debug or make things easier to look at. You don't really need any of them at this point, but keep an open mind whenever you come across a tool that others are very excited about.

Online editor

It may be the case that you don't have access to a computer, perhaps just a tablet, or that you cannot install anything on your laptop. There are great online editors out there for these scenarios as well. We don't name any, since they are evolving rapidly and probably will be old by the time you are reading this. But if you do a web search for online JavaScript IDE, you will find plenty of online options where you can just start coding JavaScript and hit a button to run it.

How does the browser understand JavaScript?

JavaScript is an interpreted language, which means that the computer understands it while running it. Some languages get processed before running, this is called compiling, but not JavaScript. The computer can just interpret JavaScript on the fly. The "engine" that understands JavaScript will be called the interpreter here.

A web page isn't just JavaScript. Web pages are written in three languages: HTML, CSS, and JavaScript.

HTML determines what is on the page; the content of the page is in there. If there is a paragraph on the page, the HTML of the page contains a paragraph. And if there is a heading, HTML was used to add a heading, and so forth. HTML consists of elements, also called tags. They specify what is on the page. Here is a little sample that will create a web page with the text `Hello world` on it:

```
<html>
<body>
Hello world!
</body>
</html>
```

In *Chapter 9, The Document Object Model*, we have a little crash course in HTML, so don't worry if you have never seen it.

CSS is the layout of the web page. So for example, if the text color is blue, this is done by CSS. Font size, font family, and position on the page are all determined by CSS. JavaScript is the final piece in the puzzle, which defines what the web page can do and how it can interact with the user or the backend.

When dealing with JavaScript, you will come across the term **ECMAScript** sooner or later. This is the specification or standardization for the JavaScript language. The current standard is **ECMAScript 6** (also referred to as **ES6**). Browsers use this specification to support JavaScript (in addition to some other topics such as **Document Object Model (DOM)**, which we'll see later). JavaScript has many implementations that might differ slightly, but ECMAScript can be considered the basic specification that the JavaScript implementation will definitely include.

Using the browser console

You may have seen this already, or not, but web browsers have a built-in option to see the code that makes the web page you are on possible. If you hit **F12** on a Windows computer while you are in the web browser, or you right-click and select **Inspect** on macOS systems, you will see a screen appear, similar to the one in the following screenshot.

It might work slightly differently on your browser on your machine, but right-clicking and selecting **Inspect** generally does the trick:

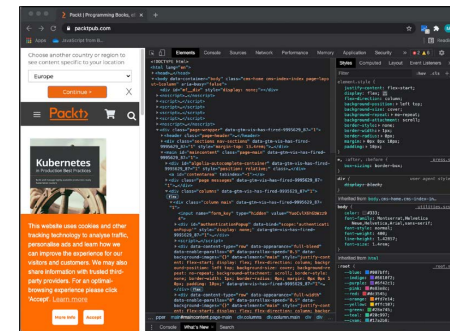


Figure 1.1: Browser console on the Packt website

This screenshot contains multiple tabs at the top. We are now looking at the **Elements** tab, which contains all the HTML and CSS (remember those?). If you click on the **Console** tab, you will find at the bottom of the panel a place where you can insert some code directly. You may see some warnings or error messages in this tab. This is not uncommon, and don't worry about it if the page is working.

The console is used by developers to log what is going on and do any debugging. Debugging is finding the problem when an application is not displaying the desired behavior. The console gives some insights as to what is happening if you log sensible messages. This is actually the first command we are going to learn:

```
console.log("Hello world!");
```

If you click on this console tab, enter the first JavaScript code above, and then hit **Enter**, this will show you the output of your code therein. It will look like the following screenshot:

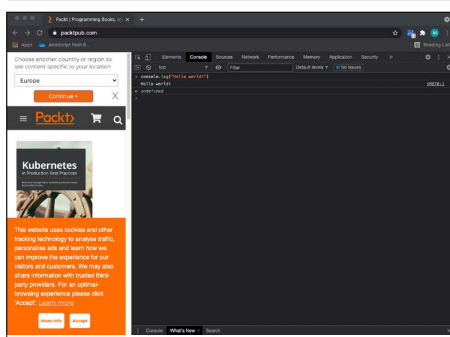


Figure 1.2: JavaScript in the browser console

You will be working with the `console.log()` statement a lot throughout the book in your code to test your code snippets and see the results. There are also other console methods, such as `console.table()`, that create a table when the inputted data can be presented as a table. Another console method is `console.error()`, which will log the inputted data, but with a styling that draws attention to the fact that it's an error.

Practice exercise 1.1

Working with the console:

1. Open the browser console, type `4 + 10`, and press `Enter`. What do you see as the response?
2. Use the `console.log()` syntax, placing a value within the rounded brackets. Try entering your name with quotes around it (this is to indicate the fact that it's a text string—we'll get to this in the next chapter).

Adding JavaScript to a web page

There are two ways to link JavaScript to a web page. The first way is to type the JavaScript directly in the HTML between two `<script>` tags. In HTML, the first tag, `<script>`, is to declare that the following script will be executed. Then we have the content that should be inside this element. Next, we close the script with the same tag, but preceded by a forward slash, `</script>`. Or you can link a JavaScript file to the HTML file using the script tag at the head of the HTML page.

Directly in HTML

Here is an example of how to write a very simple web page that will give a pop-up box saying `Hi there!`:

```
<html>
  <script type="text/javascript">
    alert("Hi there!");
  </script>
</html>
```

If you store this as a `.html` file, and open the file in your browser, you will get something like the following screenshot. We will be storing this one as `Hi.html`:



Figure 1.3: JavaScript made this popup with the text "Hi there!" appear

The `alert` command will create a popup that provides a message. This message is specified between the parentheses behind the `alert`.

Right now, we have the content directly within our `<html>` tags. This is not a best practice. We will need to create two elements inside `<html>`—`<head>` and `<body>`. In the head element, we write metadata and we also use this part later to connect external files to our HTML file. In the body, we have the content of the web page.

We also need to let the browser know what kind of document we're working on with the `<!DOCTYPE>` declaration. Since we're writing JavaScript inside an HTML file, we need to use `<!DOCTYPE html>`. Here's an example:

```
<!DOCTYPE html>
<html>

<head>
  <title>This goes in the tab of your browser</title>
</head>

<body>
  The content of the webpage
  <script>
    console.log("Hi there!");
  </script>
</body>
</html>
```

This example web page will display the following: The content of the webpage. If you look in the browser console, you'll find a surprise! It has executed the JavaScript as well and logs `Hi there!` in the console.

Practice exercise 1.2

JavaScript in an HTML page:

1. Open your code editor and create an HTML file.
2. Within your HTML file, set up the HTML tags, doctype, HTML, head, and body, and then proceed and add the script tags.
3. Place some JavaScript code within the script tags. You can use `console.log("hello world!")`.

Linking an external file to our web page

You could also link an external file to the HTML file. This is considered a better practice, as it organizes code better and you can avoid very lengthy HTML pages due to the JavaScript. In addition to these benefits, you can reuse the JavaScript on other web pages of your website without having to copy and paste. Say that you have the same JavaScript on 10 pages and you need to make a change to the script. You would only have to change one file if you did it in the way we are showing you in this example.

First, we are going to create a separate JavaScript file. These files have the postfix `.js`. I'm going to call it `ch1_alert.js`. This will be the content of our file:

```
alert("Saying hi from a different file!");
```

Then we are going to create a separate HTML file (using the postfix `.html` again). And we are going to give it this content:

```
<html>
  <script type="text/javascript" src="ch1_alert.js"></script>
</html>
```

Make sure that you put the files in the same location, or that you specify the path to the JavaScript file in your HTML. The names are case-sensitive and should match exactly.

You have two options. You can use a relative path and an absolute path. Let's cover the latter first since that is the easiest to explain. Your computer has a root. For Linux and macOS, it is `/`, and for Windows, it is often `C:/`. The path to the file starting from the root is the absolute path. This is the easiest to add because it will work on your machine. But there is a catch: on your machine, if this website folder later gets moved to a server, the absolute path will no longer work.

The second, safer option is relative paths. You specify how to get there from the file you are in at that time. So if it's in the same folder, you will only have to insert the name. If it's in a folder called "example" that is inside the folder that your file is in, you will have to specify `example/nameOfTheFile.js`. And if it's a folder up, you would have to specify `../nameOfTheFile.js`.

If you open the HTML file, this is what you should get:

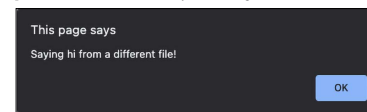


Figure 1.4: Popup created by JavaScript in a different file

Practice exercise 1.3

Linking to a JS JavaScript file:

1. Create a separate file called `app` with the extension `.js`.
2. Within the `.js` file, add some JavaScript code.
3. Link to the separate `.js` file within the HTML file you created in *Practice exercise 1.2*.
4. Open the HTML file within your browser and check to see whether the JavaScript code ran properly.

Writing JavaScript code

So, we now have lots of context, but how do you actually write JavaScript code? There are some important things to keep in mind, such as how to format the code, using the right indentation level, using semicolons, and adding comments. Let's start with formatting code.

Formatting code

Code needs to be formatted well. If you have a long file with many lines of code and you didn't stick to a few basic formatting rules, it is going to be hard to understand what you've written. So, what are the basic formatting rules? The two most important for now are indentations and semicolons. There are also naming conventions, but these will be addressed for every topic that is yet to come.

Indentations and whitespace

When you are writing code, often a line of code belongs to a certain code block (code between two curly brackets { like this }) or parent statement. If that is the case, you give the code in that block one indentation to make sure that you can see easily what is part of the block and when a new block starts. You don't need to understand the following code snippet, but it will demonstrate readability with and without indentations.

Without new lines:

```
let status = "new"; let scared = true; if (status === "new") { console.log("Welcome to JavaScript!"); if (scared) { console.log("Don't worry you will be fine!"); } else { console.log("You're brave! You are going to do great!"); } } else { console.log("Welcome back, I knew you'd like it!"); }
```

With new lines but without indentation:

```
let status = "new";
let scared = true;
if (status === "new") {
  console.log("Welcome to JavaScript!");
  if (scared) {
    console.log("Don't worry you will be fine!");
  } else {
    console.log("You're brave! You are going to do great!");
  }
} else {
  console.log("Welcome back, I knew you'd like it!");
}
```

With new lines and indentation:

```
let status = "new";
let scared = true;
if (status === "new") {
  console.log("Welcome to JavaScript!");
  if (scared) {
    console.log("Don't worry you will be fine!");
  } else {
    console.log("You're brave! You are going to do great!");
  }
} else {
  console.log("Welcome back, I knew you'd like it!");
}
```

As you can see, you can now easily see when the code blocks end. This is where the `if` has a corresponding `}` at the same indentation level. In the example without indentations, you would have to count the brackets to determine when the `if` block would end. Even though it is not necessary for working code, make sure to use indentation well. You will thank yourself later.

Semicolons

After every statement, you should insert a semicolon. JavaScript is very forgiving and will understand many situations in which you have forgotten one, but develop the habit of adding one after every line of code early. When you declare a code block, such as an `if` statement or loop, you should not end with a semicolon. It is only for the separate statements.

Code comments

With comments, you can tell the interpreter to ignore some lines of the file. They won't get executed if they are comments. It is often useful to be able to avoid executing a part of the file. This could be for the following reasons:

1. You do not want to execute a piece of code while running the script, so you comment it out so it gets ignored by the interpreter.
2. Metadata. Adding some context to the code, such as the author, and a description of what the file covers.
3. Adding comments to specific parts of the code to explain what is happening or why a certain choice has been made.

There are two ways to write comments. You can either write single-line comments or multi-line comments. Here is an example:

```
// I'm a single line comment
// console.log("single line comment, not logged");

/* I'm a multi-line comment. Whatever is between the slash asterisk and
the asterisk slash will not get executed.
console.log("I'm not logged, because I'm a comment");
*/
```

In the preceding code snippet, you see both commenting styles. The first one is single-line. This can also be an inline comment at the end of the line. Whatever comes after the `//` on the line will get ignored. The second one is multiline; it is written by starting with `/*` and ending with `*/`.

Practice exercise 1.4

Adding comments:

1. Add a new statement to your JavaScript code by setting a variable value. Since we will cover this in the next chapter, you can use the following line:

```
let a = 10;
```
2. Add a comment at the end of the statement indicating that you set a value of 10.
3. Print the value using `console.log()`. Add a comment explaining what this will do.

4. At the end of your JavaScript code, use a multiple-line comment. In a real production script, you might use this space to add a brief outline of the purpose of the file.

Prompt

Another thing we would like to show you here is also a command prompt. It works very much like an alert, but instead, it takes input from the user. We will learn how to store variables very soon, and once you know that, you can store the result of this prompt function and do something with it. Go ahead and change the `alert()` to a `prompt()` in the `Hi1.html` file, for example, like this:

```
prompt("Hi! How are you?");
```

Then, go ahead and refresh the HTML. You will get a popup with an input box in which you can enter text, as follows:

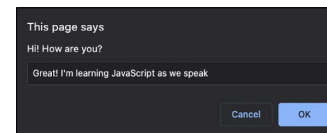


Figure 1.5: Page prompting for use input

The value you (or any other user) enter will be returned to the script, and can be used in your code! This is great for getting user input to shape the way your code works.

Random numbers

For the purpose of fun exercises in the early chapters of this book, we would like you to know how to generate a random number in JavaScript. It is absolutely fine if you don't really understand what is going on just yet; just know that this is the command to create a random number:

```
Math.random();
```

We can do it in the console and see the result appear if we log it:

```
console.log(Math.random());
```

This number will be a decimal between 0 and 1. If we want a number between 0 and 100, we can multiply it by 100, like this:

```
console.log(Math.random() * 100);
```



Don't worry, we will cover mathematic operators in *Chapter 2, JavaScript Essentials*.

If we don't want to have a decimal result, we can use the `Math.floor` function on it, which is rounding it down to the nearest integer:

```
console.log(Math.floor(Math.random() * 100));
```

Don't worry about not getting this yet. This will be explained in more detail further on in the book. In *Chapter 8, Built-In JavaScript Methods*, we will discuss built-in methods in more detail. Until then, just trust us that this does generate a random number between 0 and 100.

Chapter project

Creating an HTML file and a linked JavaScript file

Create an HTML file and create a separate JavaScript file. Then, connect to the JavaScript file from the HTML file.

1. In the JavaScript file, output your name into the console and add a multiple-line comment to your code.
2. Try commenting out the console message in your JavaScript file so that nothing shows in the console.

Self-check quiz

1. What is the HTML syntax to add an external JavaScript file?
2. Can you run JavaScript in a file with a JS extension in your browser?
3. How do you write a multiple-line comment in JavaScript?
4. What is the best way to remove a line of code from running that you might want to keep as you debug?

Summary

Nicely done! You have made a start with JavaScript! In this chapter, we have discussed a lot of context, which you will need to know before starting to code JavaScript. We saw that we can use JavaScript for many purposes, and one of the most popular use cases is the web. Browsers can work with JavaScript because they have a special part, called an interpreter, that can process JavaScript. We saw that we have multiple options for writing JavaScript on our computer. We will need an IDE, a program that we can use to write and run our code.

Adding JavaScript to a web page can be done in several ways. We saw how to include it in the script element and how to add a separate JavaScript file to a page. We ended this chapter with some important general notes on how to write well-structured, readable, and easy-to-maintain code that is well documented with comments. We also saw that we can write to the console with our `console.log()` method and ask for user input using `prompt()`. Lastly, we also saw that we can generate random numbers with the `Math.random()` function.

Next, we'll look at JavaScript's basic data types and the operators that can be used to manipulate them!

2

JavaScript Essentials

In this chapter, we will be dealing with some essential building blocks of JavaScript: variables and operators. We will start with variables, what they are, and which different variable data types exist. We need these basic building blocks to store and work with variable values in our scripts, making them dynamic.

Once we've got the variables covered, we will be ready to deal with operators. Arithmetic, assignment, and conditional and logical operators will be discussed at this stage. We need operators to modify our variables or to tell us something about these variables. This way we can do basic calculations based on factors such as user input.

Along the way, we'll cover the following topics:

- Variables
- Primitive data types
- Analyzing and modifying data types
- Operators



Note: exercise, project, and self-check quiz answers can be found in the *Appendix*.

Variables

Variables are the first building block you will be introduced to when learning most languages. Variables are values in your code that can represent different values each time the code runs. Here is an example of two variables in a script:

```
firstname = "Maaike";  
x = 2;
```

And they can be assigned a new value while the code is running:

```
firstname = "Edward";  
x = 7;
```

Without variables, a piece of code would do the exact same thing every single time it was run. Even though that could still be helpful in some cases, it can be made much more powerful by working with variables to allow our code to do something different every time we run it.

Declaring variables

The first time you create a variable, you declare it. And you need a special word for that: `let`, `var`, or `const`. We'll discuss the use of these three arguments shortly. The second time you call a variable, you only use the name of the existing variable to assign it a new value:

```
let firstname = "Maria";  
firstname = "Jacky";
```

In our examples, we will be assigning a value to our variables in the code. This is called "hardcoded" since the value of your variable is defined in your script instead of coming dynamically from some external input. This is something you won't be doing that often in actual code, as more commonly the value comes from an external source, such as an input box on a website that a user filled out, a database, or some other code that calls your code. The use of variables coming from external sources instead of being hardcoded into a script is actually the reason that scripts are adaptable to new information, without having to rewrite the code.

We have just established how powerful the variable building block is in code. Right now, we are going to hardcode variables into our scripts, and they therefore will not vary until a coder changes the program. However, we will soon learn how to make our variables take in values from outside sources.

let, var, and const

A variable definition consists of three parts: a variable-defining keyword (`let`, `var`, or `const`), a name, and a value. Let's start with the difference between `let`, `var`, or `const`. Here you can see some examples of variables using the different keywords:

```
let nr1 = 12;
var nr2 = 8;
const PI = 3.14159;
```

`let` and `var` are both used for variables that might have a new value assigned to them somewhere in the program. The difference between `let` and `var` is complex. It is related to scope.



If you understand the following sentences on scope, that is great, but it is totally fine if you do not get it. You will understand it soon enough as you keep working your way through the book.

`var` has **global scope** and `let` has **block scope**. `var`'s global scope means that you can use the variables defined with `var` in the entire script. On the other hand, `let`'s block scope means you can only use variables defined with `let` in the specific block of code in which they were defined. Remember, a block of code will always start with `{` and end with `}`, which is how you can recognize them.

On the other hand, `const` is used for variables that only get a value assigned once—for example, the value of `pi`, which will not change. If you try reassigning a value declared with `const`, you will get an error:

```
const someConstant = 3;
someConstant = 4;
```

This will result in the following output:

```
Uncaught TypeError: Assignment to constant variable.
```

We will be using `let` in most of our examples—for now, trust us that you should use `let` in most cases.

Naming variables

When it comes to naming variables, there are some conventions in place:

- Variables start with a lowercase letter, and they should be descriptive. If something holds an age, do not call it `x`, but `age`. This way, when you read your script later, you can easily understand what you did by just reading your code.
- Variables cannot contain spaces, but they can use underscores. If you use a space, JavaScript doesn't recognize it as a single variable.



We will be using camel case here. This means that when we want to use multiple words to describe a variable, we will start with a lowercase word, then use a capital for every new word after the first word—for example: `ageOfBuyer`.

Whatever the convention is in the place you are working, the key is consistency. If all naming is done in a similar format, the code will look cleaner and more readable, which makes it a lot easier to make a small change later.

The value of your variable can be anything. Let's start with the easiest thing variables can be: primitives.

Primitive data types

Now you know what variables are and why we need them in our code, it is time to look at the different types of values we can store in variables. Variables get a value assigned. And these values can be of different types. JavaScript is a loosely typed language. This means that JavaScript determines the type based on the value. The type does not need to be named explicitly. For example, if you declared a value of 5, JavaScript will automatically define it as a number type.

A distinction exists between primitive data types and other, more complex data types. In this chapter, we will cover the primitive type, which is a relatively simple data structure. Let's say for now that they just contain a value and have a type. JavaScript has seven primitives: `String`, `Number`, `BigInt`, `Boolean`, `Symbol`, `undefined`, and `null`. We'll discuss each of them in more detail below.

String

A string is used to store a text value. It is a sequence of characters. There are different ways to declare a string:

- Double quotes
- Single quotes
- Backticks: special template strings in which you can use variables directly

The single and double quotes can both be used like so:

```
let singleString = 'Hi there!';
let doubleString = "How are you?";
```



You can use the option you prefer, unless you are working on code in which one of these options has already been chosen. Again, consistency is key.

The main difference between single quotes and double quotes is that you can use single quotes as literal characters in double-quoted strings, and vice versa. If you declare a string with single quotes, the string will end as soon as a second quote is detected, even if it's in the middle of a word. So for example, the following will result in an error, because the string will be ended at the second single quote within let's:

```
let funActivity = 'Let's learn JavaScript';
```

let will be recognized as a string, but after that, the bunch of characters that follow cannot be interpreted by JavaScript. However, if you declare the string using double quotes, it will not end the string as soon as it hits the single quote, because it is looking for another double quote. Therefore, this alternative will work fine:

```
let funActivity = "Let's learn JavaScript";
```

In the same way with double quotes, the following would not work:

```
let question = "Do you want to learn JavaScript? 'Yes!'";
```

Again, the compiler will not distinguish between double quotes used in different contexts, and will output an error.

In a string using backticks, you can point to variables and the variable's value will be substituted into the line. You can see this in the following code snippet:

```
let language = "JavaScript";
let message = `Let's learn ${language}`;
console.log(message);
```

As you can see, you will have to specify these variables with a rather funky syntax—don't be intimidated! Variables in these template strings are specified between \${nameOfVariable}. The reason that it's such an intense syntax is that they want to avoid it being something you would normally use, which would make it unnecessarily difficult to do so. In our case, the console output would be as follows:

```
Let's learn JavaScript
```

As you can see, the language variable gets replaced with its value: JavaScript.

Escape characters

Say we want to have double quotes, single quotes, and backticks in our string. We would have a problem, as this cannot be done with just the ingredients we have now. There is an elegant solution to this problem. There is a special character that can be used to tell JavaScript, "do not take the next character as you normally would." This is the escape character, a backslash.

In this example, the backslash can be used to ensure your interpreter doesn't see the single or double quote marks and end either string too early:

```
let str = "Hello, what's your name? Is it \"Mike\"?";
console.log(str);
let str2 = 'Hello, what\'s your name? Is it "Mike"?';
console.log(str2);
```

This logs the following to the console:

```
Hello, what's your name? Is it "Mike"?
Hello, what's your name? Is it "Mike"?
```


As you can see, both types of quote marks inside the strings have been logged without throwing an error. This is because the backslash before the quote character gives the quote character a different meaning. In this case, the meaning is that it should be a literal character instead of an indicator to end the string.

The escape character has even more purposes. You can use it to create a line break with `\n`, or to include a backslash character in the text with `\\`:

```
let str3 = "New \nline.";
let str4 = "I'm containing a backslash: \\!";
console.log(str3);
console.log(str4);
```

The output of these lines is as follows:

```
New
line.
I'm containing a backslash: \!
```

There are some more options, but we will leave them for now. Let's get back to primitive data types by looking at the number type.

Number

The number data type is used to represent, well, numbers. In many languages, there is a very clear difference between different types of numbers. The developers of JavaScript decided to go for one data type for all these numbers: number. To be more precise, they decided to go for a 64-bit floating-point number. This means that it can store rather large numbers and both signed and unsigned numbers, numbers with decimals, and more.

However, there are different kinds of numbers it can represent. First of all, integers, for example: 4 or 89. But the number data type can also be used to represent decimals, exponentials, octal, hexadecimal, and binary numbers. The following code sample should speak for itself:

```
let intNr = 1;
let decNr = 1.5;
let expNr = 1.4e15;
let octNr = 0o10; //decimal version would be 8
let hexNr = 0x3E8; //decimal version would be 1000
let binNr = 0b101; //decimal version would be 5
```

You don't need to worry about these last three if you're not familiar with them. These are just different ways to represent numbers that you may encounter in the broader field of computer science. The takeaway here is that all the above numbers are of the number data type. So integers are numbers, like these ones:

```
let intNr2 = 3434;
let intNr3 = -111;
```

And the floating points are numbers as well, like this one:

```
let decNr2 = 45.78;
```

And binary numbers are of the number data type as well, for example, this one:

```
let binNr2 = 0b100; //decimal version would be 4
```

We have just seen the number data type, which is very commonly used. But in some special cases, you will need an even bigger number.

BigInt

The limits of the number data type are between $2^{53}-1$ and $-(2^{53}-1)$. In case you were to need a bigger (or smaller) number, BigInt comes into play. A BigInt data type can be recognized by the postfix `n`:

```
let bigNr = 90071992547409920n;
```

Let's see what happens when we start to do some calculations between our previously made integer Number, `intNr`, and `BigInt`, `bigNr`:

```
let result = bigNr + intNr;
```

The output will be as follows:

```
Uncaught TypeError: Cannot mix BigInt and other types, use explicit conversions
```

Uh-oh, a `TypeError`! It is very clear about what is going wrong. We cannot mix `BigInt` with the `Number` data type to perform operations. This is something to keep in mind for later when actually working with `BigInt`—you can only operate on `BigInt` with other `BigInts`.

Boolean

The Boolean data type can hold two values: `true` and `false`. There is nothing in between. This Boolean is used a lot in code, especially expressions that evaluate to a Boolean:

```
let bool1 = false;
let bool2 = true;
```

In the preceding example, you can see the options we have for the Boolean data type. It is used for situations in which you want to store a `true` or a `false` value (which can indicate on/off or yes/no). For example, whether an element is deleted:

```
let objectIsDeleted = false;
```

Or, whether the light is on or off:

```
let lightIsOn = true;
```

These variables suggest respectively that the specified object is not deleted, and that the specific light is on.

Symbol

Symbol is a brand new data type introduced in ES6 (we mentioned ECMA Script 6, or ES6, in *Chapter 1, Getting Started with JavaScript*). Symbol can be used when it is important that variables are not equal, even though their value and type are the same (in this case, they would both be of the symbol type). Compare the following string declarations to the symbol declarations, all of equal value:

```
let str1 = "JavaScript is fun!";
let str2 = "JavaScript is fun!";
console.log("These two strings are the same:", str1 === str2);

let sym1 = Symbol("JavaScript is fun!");
let sym2 = Symbol("JavaScript is fun!");
console.log("These two Symbols are the same:", sym1 === sym2);
```

And the output:

```
These two strings are the same: true
These two Symbols are the same: false
```

In the first half, JavaScript concludes that the strings are the same. They have the same value, and the same type. However, in the second part, each symbol is unique. Therefore, although they contain the same string, they are not the same, and output `false` when compared. These symbol data types can be very handy as properties of objects, which we will see in *Chapter 3, JavaScript Multiple Values*.

Undefined

JavaScript is a very special language. It has a special data type for a variable that has not been assigned a value. And this data type is undefined:

```
let unassigned;
console.log(unassigned);
```

The output here will be:

```
Undefined
```

We can also purposefully assign an undefined value. It is important you know that it is possible, but it is even more important that you know that manually assigning undefined is a bad practice:

```
let terribleThingToDo = undefined;
```

Alright, this can be done, but it is recommended to not do this. This is for a number of reasons—for example, checking whether two variables are the same. If one variable is undefined, and your own variable is manually set to undefined, they will be considered equal. This is an issue because if you are checking for equality, you would want to know whether two values are actually equal, not just that they are both undefined. This way, someone's pet and their last name might be considered equal, whereas they are actually both just empty values.

null

In the last example, we saw an issue that can be solved with a final primitive type, `null`. `null` is a special value for saying that a variable is empty or has an unknown value. This is case sensitive. You should use lowercase for `null`:

```
let empty = null;
```

To solve the issue we encountered with setting a variable as undefined, note that if you set it to null, you will not have the same problem. This is one of the reasons it is better to assign null to a variable when you want to say it is empty and unknown at first.

```
let terribleThingToDo = undefined;
let lastName;
console.log("Same undefined:", lastName === terribleThingToDo);

let betterOption = null;
console.log("Same null:", lastName === betterOption);
```

This outputs the following:

```
Same undefined: true
Same null: false
```

This shows that an automatically undefined variable, `lastName`, and a deliberately undefined variable, `terribleThingToDo`, are considered equal, which is problematic. On the other hand, `lastName` and `betterOption`, which was explicitly declared with a value of null, are not equal.

Analyzing and modifying data types

We have seen the primitive data types. There are some built-in JavaScript methods that will help us deal with common problems related to primitives. Built-in methods are pieces of logic that can be used without having to write JavaScript logic yourself.



We've seen one built-in method already: `console.log()`.

There are many of these built-in methods, and the ones you will be meeting in this chapter are just the first few you will encounter.

Working out the type of a variable

Especially with null and undefined, it can be hard to determine what kind of data type you are dealing with. Let's have a look at `typeof`. This returns the type of the variable. You can check the type of a variable by entering `typeof`, then either a space followed by the variable in question, or the variable in question in brackets:

```
testVariable = 1;
variableTypeTest1 = typeof testVariable;
variableTypeTest2 = typeof(testVariable);
console.log(variableTypeTest1);
console.log(variableTypeTest2);
```

As you might assume, both methods will output `number`. Brackets aren't required because technically, `typeof` is an operator, not a method, unlike `console.log`. But, sometimes you may find that using brackets makes your code easier to read. Here you can see it in action:

```
let str = "Hello";
let nr = 7;
let bigNr = 12345678901234n;
let bool = true;
let sym = Symbol("unique");
let undef = undefined;
let unknown = null;

console.log("str", typeof str);
console.log("nr", typeof nr);
console.log("bigNr", typeof bigNr);
console.log("bool", typeof bool);
console.log("sym", typeof sym);
console.log("undef", typeof undef);
console.log("unknown", typeof unknown);
```

Here, in the same `console.log()` print command, we are printing the name of each variable (as a string, declared with double quotes), then its type (using `typeof`). This will produce the following output:

```
str string
nr number
bigNr bigint
bool boolean
```

```
sym symbol
undef undefined
unknown object
```

There is an odd one out, and that is the null type. In the output you can see that `typeof null` returns `object`, while in fact, null truly is a primitive and not an object. This is a bug that has been there since forever and now cannot be removed due to backward compatibility problems. Don't worry about this bug, as it won't affect our programs—just be aware of it, since it will go nowhere anytime soon, and it has the potential to break applications.

Converting data types

The variables in JavaScript can change types. Sometimes JavaScript does this automatically. What do you think the result of running the following code snippet will be?

```
let nr1 = 2;
let nr2 = "2";
console.log(nr1 * nr2);
```

We try to multiply a variable of type `Number` with a variable of type `String`. JavaScript does not just throw an error (as many languages would), but first tries to convert the string value to a number. If that can be done, it can execute without any problem as if two numbers were declared. In this case, `console.log()` will write 4 to the console.

But this is dangerous! Guess what this code snippet does:

```
let nr1 = 2;
let nr2 = "2";
console.log(nr1 + nr2);
```

This one will log 22. The plus sign can be used to concatenate strings. Therefore, instead of converting a string to a number, it is converting a number to a string in this example, and cubbing the two strings together – “2” and “2” make “22”. Luckily, we do not need to rely on JavaScript's behavior when converting data types. There are built-in functions we can use to convert the data type of our variable.

There are three conversion methods: `String()`, `Number()`, and `Boolean()`. The first one converts a variable to type `String`. It pretty much takes any value, including undefined and null, and puts quotes around it.

The second one tries to convert a variable to a number. If that cannot be done logically, it will change the value into NaN (not a number). `Boolean()` converts a variable to a Boolean. This will be true for everything except for null, undefined, 0 (number), an empty string, and NaN. Let's see them in action:

```
let nrToStr = 6;
nrToStr = String(nrToStr);
console.log(nrToStr, typeof nrToStr);

let strToNr = "12";
strToNr = Number(strToNr);
console.log(strToNr, typeof strToNr);

let strToBool = "any string will return true";
strToBool = Boolean(strToBool);
console.log(strToBool, typeof strToBool);
```

This will log the following:

```
6 string
12 number
true boolean
```

This might seem pretty straightforward, but not all of the options are equally obvious. These, for example, are not what you might think:

```
let nullToNr = null;
nullToNr = Number(nullToNr);
console.log("null", nullToNr, typeof nullToNr);

let strToNr = "";
strToNr = Number(strToNr);
console.log("empty string", strToNr, typeof strToNr);
```

The preceding code snippet will log the following to the console:

```
null 0 number
empty string 0 number
```

As you can see, an empty string and null will both result in the number 0. This is a choice that the makers of JavaScript made, which you will have to know—it can come in handy at times when you want to convert a string to 0 when it is empty or null.

Next, enter the following snippet:

```
let strToNr2 = "hello";
strToNr2 = Number(strToNr2);
console.log(strToNr2, typeof strToNr2);
```

The result that will be logged to the console is:

```
NaN number
```

Here, we can see that anything that can't be interpreted as a number by simply removing the quotes will evaluate as NaN (not a number).

Let's continue with the following code:

```
let strToBool2 = "false";
strToBool2 = Boolean(strToBool2);
console.log(strToBool2, typeof strToBool2);

let strToBool = "";
strToBool = Boolean(strToBool);
console.log(strToBool, typeof strToBool);
```

Finally, this one will log the following:

```
true boolean
false boolean
```

This output shows that any string will return true when converted to a Boolean, even the string "false"! Only an empty string, null, and undefined will lead to a Boolean value of false.

Let's tease your brain a little bit more. What do you think this one will log?

```
let nr1 = 2;
let nr2 = "2";
console.log(nr1 + Number(nr2));
```

This one logs 4! The string gets converted to a number before it executes the plus operation, and therefore it is a mathematical operation and not a string concatenation. In the next sections of this chapter, we will discuss operators in more depth.

Practice exercise 2.1

What are the types of these variables listed below? Verify this with `typeof` and output the result to the console:

```
let str1 = 'Laurence';
let str2 = "Sveakis";
let val1 = undefined;
let val2 = null;
let myNum = 1000;
```

Operators

After seeing quite a few data types and some ways to convert them, it is time for the next major building block: operators. These come in handy whenever we want to work with the variables, modify them, perform calculations on them, and compare them. They are called operators because we use them to operate on our variables.

Arithmetic operators

Arithmetic operators can be used to perform operations with numbers. Most of these operations will feel very natural to you because they are the basic mathematics you will have come across earlier in life already.

Addition

Addition in JavaScript is very simple, we have seen it already. We use `+` for this operation:

```
let nr1 = 12;
let nr2 = 14;
let result1 = nr1 + nr2;
```

However, this operator can also come in very handy for concatenating strings. Note the added space after "Hello" to ensure the end result contains space characters:

```
let str1 = "Hello ";
let str2 = "addition";
let result2 = str1 + str2;
```

The output of printing `result1` and `result2` will be as follows:

```
26
Hello addition
```

As you can see, adding numbers and strings lead to different results. If we add two different strings, it will concatenate them into a single string.

Practice exercise 2.2

Create a variable for your name, another one for your age, and another one for whether you can code JavaScript or not.

Log to the console the following sentence, where `name`, `age` and `true/false` are variables:

```
Hello, my name is Maaike, I am 29 years old and I can code JavaScript:
true.
```

Subtraction

Subtraction works as we would expect it as well. We use `-` for this operation. What do you think gets stored in the variable in this second example?

```
let nr1 = 20;
let nr2 = 4;
let str1 = "Hi";
let nr3 = 3;
let result1 = nr1 - nr2;
let result2 = str1 - nr3;
console.log(result1, result2);
```

The output is as follows:

```
16 NaN
```

The first result is 16. And the second result is more interesting. It gives `NaN`, not an error, but just simply the conclusion that a word and a number subtracted is not a number. Thanks for not crashing, JavaScript!

Multiplication

We can multiply two numeric values with the `*` character. Unlike some other languages, we cannot successfully multiply a number and a string in JavaScript.

The result of multiplying a numeric and a non-numeric value is `NaN`:

```
let nr1 = 15;
let nr2 = 10;
let str1 = "Hi";
let nr3 = 3;
let result1 = nr1 * nr2;
let result2 = str1 * nr3;
console.log(result1, result2);
```

Output:

```
150 NaN
```

Division

Another straightforward operator is division. We can divide two numbers with the `/` character.

```
let nr1 = 30;
let nr2 = 5;
let result1 = nr1 / nr2;
console.log(result1);
```

The output is as follows:

```
6
```

Exponentiation

Exponentiation means raising a certain base number to the power of the exponent, for example, x^y . This can be read as x to the power of y . It means that we will multiply x by itself y number of times. Here is an example of how to do this in JavaScript—we use `**` for this operator.

```
let nr1 = 2;
let nr2 = 3;
let result1 = nr1 ** nr2;
console.log(result1);
```

We get the following output:

```
8
```

The result of this operation is 2 to the power of 3 ($2^2 \cdot 2$), which is 8. We're going to avoid going into a mathematics lesson here, but we can also find the root of a number by using fractional exponents: for example, the square root of a value is the same as raising it to the power of 0.5.

Modulus

This is one that often requires a little explanation. Modulus is the operation in which you determine how much is left after dividing a number by another number in its entirety. The amount of times the number can fit in the other number does not matter here. The outcome will be the remainder, or what is left over. The character we use for this operation is the % character. Here are some examples:

```
let nr1 = 10;
let nr2 = 3;
let result1 = nr1 % nr2;
console.log(`${nr1} % ${nr2} = ${result1}`);

let nr3 = 8;
let nr4 = 2;
let result2 = nr3 % nr4;
console.log(`${nr3} % ${nr4} = ${result2}`);

let nr5 = 15;
let nr6 = 4;
let result3 = nr5 % nr6;
console.log(`${nr5} % ${nr6} = ${result3}`);
```

And the output:

```
10 % 3 = 1
8 % 2 = 0
15 % 4 = 3
```

The first one is $10 \% 3$, where 3 fits 3 times into 10, and then 1 is left. The second one is $8 \% 2$. This results in 0, because 2 can fit 4 times into 8 without having anything left. The last one is $15 \% 4$, where 4 fits 3 times into 15. And then we have 3 left as a result.

This is something that would happen in your head automatically if I asked you to add 125 minutes to the current time. You will probably do two things: integer division to determine how many whole hours fit into 125 minutes, and then 125 modulo 60 (in JavaScript terms, $125 \% 60$) to conclude that you'll have to add 5 more minutes to the current time. Say our current time is 09:59, you will probably start by adding 2 hours, and get to 11:59, and then add 5 minutes, and then you will perform another modulus operation with 59 and 5, adding 1 more hour to the total and having 4 minutes left: 12:04.

Unary operators: increment and decrement

The last two operators of our arithmetic operator section are probably new to you, if you are new to programming (or only familiar with another programming language). These are the increment and decrement operators. A term we use here is **operand**. Operands are subject to the operator. So, if we say $x + y$, x and y are operands.

We only need one operand for these operators, and therefore we also call them unary operators. If we see $x++$, we can read this as $x = x + 1$. The same is true for the decrement operators: $x--$ can be read as $x = x - 1$:

```
let nr1 = 4;
nr1++;
console.log(nr1);

let nr2 = 4;
nr2--;
console.log(nr2);
```

The output is as follows:

```
5
3
```

Prefix and postfix operators

We can have the increment operator after the operand ($x++$), in which case we call this the **postfix unary operator**. We can also have it before ($++x$), which is the **prefix unary operator**. This does something different though—the next few lines might be complicated, so do not worry if you need to read it a few times and have a good look at the examples here.

The postfix gets executed after sending the variable through, and then after that, the operation gets executed. In the following example, `nr` gets incremented by 1 *after* logging. So the first logging statement is still logging the old value because it has not been updated yet. It has been updated for the second log statement:

```
let nr = 2;
console.log(nr++);
console.log(nr);
```

The output is as follows:

```
2
3
```

The prefix gets executed *before* sending the variable through, and often this is the one you will need. Have a look at the following example:

```
let nr = 2;
console.log(++nr);
```

We get the following output:

```
3
```

Alright, if you can figure out what the next code snippets logs to the console, you should really have a handle on it:

```
let nr1 = 4;
let nr2 = 5;
let nr3 = 2;
console.log(nr1++ + ++nr2 * nr3++);
```

It outputs 16. It will do the multiplication first, according to the basic mathematical order of operations. For multiplying, it uses 6 (prefix, so 5 is incremented before multiplying) and 2 (postfix, so 2 is only incremented after execution, meaning it won't affect our current calculation). This comes down to 12. And then `nr1` is a postfix operator, so this one will execute after the addition. Therefore, it will add 12 to 4 and become 16.

Combining the operators

These operators can be combined, and it works just as it does in math. They get executed in a certain order, and not necessarily from left to right. This is due to a phenomenon called operator precedence.

There is one more thing to take into account here, and that is grouping. You can group using `(` and `)`. The operations between the parentheses have the highest precedence. After that, the order of the operations takes place based on the type of operation (highest precedence first) and if they are of equal precedence, they take place from left to right:

Name	Symbol	Example
Grouping	<code>(...)</code>	<code>(x + y)</code>
Exponentiation	<code>**</code>	<code>x ** y</code>
Prefix increment and decrement	<code>--</code> , <code>++</code>	<code>--x</code> , <code>++y</code>
Multiplication, division, modulus	<code>*</code> , <code>/</code> , <code>%</code>	<code>x * y</code> , <code>x / y</code> , <code>x % y</code>
Addition and subtraction	<code>+</code> , <code>-</code>	<code>x + y</code> , <code>x - y</code>

Practice exercise 2.3

Write some code to calculate the hypotenuse of a triangle using the Pythagorean theorem when given the values of the other two sides. The theorem specifies that the relation between the sides of a right-angled triangle is $a^2 + b^2 = c^2$.



The Pythagorean theorem only applies to right-angled triangles. The sides connected to the 90-degree angle are called the adjacent and opposite sides, represented by *a* and *b* in the formula. The longest side, not connected to the 90-degree angle, is called the hypotenuse, represented by *c*.

You can use `prompt()` to get the value for *a* and *b*. Write code to get the value from the user for *a* and *b*. Then square the values of both *a* and *b* before adding them together and finding the square root. Print your answer to the console.

Assignment operators

We have seen one assignment operator already when we were assigning values to variables. The character for this basic assignment operation is `=`. There are a few others available. Every binary arithmetic operator has a corresponding assignment operator to write a shorter piece of code. For example, `x += 5` means `x = x + 5`, and `x **= 3` means `x = x ** 3` (*x* to the power of 3).

In this first example we declare a variable *x*, and set it to 2 as an initial value:

```
let x = 2;
x += 2;
```

After this assignment operation, the value of *x* becomes 4, because $x + 2$ is the same as $x = x + 2$:

In the next assignment operation, we will subtract 2:

```
x -= 2;
```

So, after this operation the value of *x* becomes 2 again ($x = x - 2$). In the next operation, we are going to multiply the value by 6:

```
x *= 6;
```

When this line has been executed, the value of *x* is no longer 2, but becomes 12 ($x = x * 6$). In the next line, we are going to use an assignment operator to perform a division:

```
x /= 3;
```

After dividing *x* by 3, the new value becomes 4. The next assignment operator we will use is exponentiation:

```
x **= 2;
```

The value of *x* becomes 16, because the old value was 4, and 4 to the power of 2 equals 16 ($4 * 4$). The last assignment operator we will talk about is the modulus assignment operator:

```
x %= 3;
```

After this assignment operation, the value of *x* is 1, because 3 can fit 5 times into 16 and then leaves 1.

Practice exercise 2.4

Create variables for three numbers: *a*, *b*, and *c*. Update these variables with the following actions using the assignment operators:

- Add *b* to *a*
- Divide *a* by *c*

- Replace the value of *c* with the modulus of *c* and *b*
- Print all three numbers to the console

Comparison operators

Comparison operators are different from the operators we have seen so far. The outcome of the comparison operators is always a Boolean, true, or false.

Equal

There are a few equality operators that determine whether two values are equal. They come in two flavors: equal value only, or equal value and data type. The first one returns true when the values are equal, even though the type is different, while the second returns true only when the value and the type are the same:

```
let x = 5;
let y = "5";
console.log(x == y);
```

The double equals operator, two equal signs, means that it will only check for equal value and not for data type. Both have the value 5, so it will log true to the console. This type of equality is sometimes called loose equality.

The triple equals operator, written as three equal signs, means that it will evaluate both the value and the data type to determine whether both sides are equal or not. They both need to be equal in order for this statement to be true, but they are not and therefore the following statement outputs false:

```
console.log(x === y);
```

This is sometimes also called strict equality. This triple equals operator is the one you should most commonly be using when you need to check for equality, as only with this one can you be sure that both variables are really equal.

Not equal

Not equal is very similar to equal, except it does the opposite—it returns true when two variables are not equal, and false when they are equal. We use the exclamation mark for not equal:

```
let x = 5;
let y = "5";
console.log(x != y);
```

This will log `false` to the console. If you are wondering what is going on here, take a look again at the double and triple equals operators, because it is the same here. However, when there is only one equals sign in a not-equal operator, it is comparing loosely for non-equality. Therefore, it concludes that they are equal and therefore not equal should result in `false`. The one with two equals signs is checking for strict non-equality:

```
console.log(x !== y);
```

This will conclude that since `x` and `y` have different data types, they are not the same, and will log `true` to the console.

Greater than and smaller than

The greater than operator returns `true` if the left-hand side is greater than the right-hand side of the operation. We use the `>` character for this. We also have a greater than or equal to operator, `>=`, which returns `true` if the left-hand side is greater than or equal to the right-hand side.

```
let x = 5;
let y = 6;
console.log(y > x);
```

This one will log `true`, because `y` is greater than `x`.

```
console.log(x > y);
```

Since `x` is not greater than `y`, this one will log `false`.

```
console.log(y > y);
```

`y` is not greater than `y`, so this one will log `false`.

```
console.log(y >= y);
```

This last one is looking at whether `y` is greater than or equal to `y`, and since it is equal to itself, it will log `true`.

It might not surprise you that we also have smaller than (`<`) and smaller than or equal to operators (`<=`). Let's have a look at the smaller than operator, as it is very similar to the previous ones.

```
console.log(y < x);
```

This first one will be `false`, since `y` is not smaller than `x`.

```
console.log(x < y);
```

So, this second one will log `true`, because `x` is smaller than `y`.

```
console.log(y < y);
```

`y` is not smaller than `y`, so this one will log `false`.

```
console.log(y <= y);
```

This last one looks at whether `y` is smaller than or equal to `y`. It is equal to `y`, so it will log `true`.

Logical operators

Whenever you want to check two conditions in one, or you need to negate a condition, the logical operators come in handy. You can use `and`, `or`, and `not`.

And

The first one we will have a look at is `and`. If you want to check whether `x` is greater than `y` and `y` is greater than `z`, you would need to be able to combine two expressions. This can be done with the `&&` operator. It will only return `true` if both expressions are `true`:

```
let x = 1;
let y = 2;
let z = 3;
```

With these variables in mind, we are going to have a look at the logical operators:

```
console.log(x < y && y < z);
```

This will log `true`, you can read it like this: if `x` is smaller than `y` and `y` is smaller than `z`, it will log `true`. That is the case, so it will log `true`. The next example will log `false`:

```
console.log(x > y && y < z);
```

Since `x` is not greater than `y`, one part of the expression is not `true`, and therefore it will result in `false`.

Or

If you want to get true if either one of the expressions is true, you use `or`. The operator for this is `||`. These pipes are used to see if either one of these two is true, in which case the whole expression evaluates to true. Let's have a look at the `or` operator in action:

```
console.log(x > y || y < z);
```

This will result in true, whereas it was false with `&&`. This is because only one of the two sides needs to be true in order for the whole expression to evaluate to true. This is the case because `y` is smaller than `z`.

When both sides are false, it will log `false`, which is the case in the next example:

```
console.log(x > y || y > z);
```

Not

In some cases you will have to negate a Boolean. This will make it the opposite value. It can be done with the exclamation mark, which reads as not:

```
let x = false;
console.log(!x);
```

This will log true, since it will simply flip the value of the Boolean. You can also negate an expression that evaluates to a Boolean, but you would have to make sure that the expression gets evaluated first by grouping it.

```
let x = 1;
let y = 2;
console.log(!(x < y));
```

`x` is smaller than `y`, so the expression evaluates to true. But, it gets negated due to the exclamation mark and prints `false` to the console.

Chapter project

Miles-to-kilometers converter

Create a variable that contains a value in miles, convert it to kilometers, and log the value in kilometers in the following format:

```
The distance of 130 kms is equal to 209.2142 miles
```

For reference, 1 mile equals 1.60934 kilometers.

BMI calculator

Set values for height in inches and weight in pounds, then convert the values to centimeters and kilos, outputting the results to the console:

- 1 inch is equal to 2.54 cm
- 22046 pounds is equal to 1 kilo

Output the results. Then, calculate and log the BMI: this is equal to weight (in kilos) divided by squared height (in meters). Output the results to the console.

Self-check quiz

1. What data type is the following variable?

```
const c = "5";
```

2. What data type is the following variable?

```
const c = 91;
```

3. Which one is generally better, line 1 or line 2?

```
let empty1 = undefined; //Line 1
let empty2 = null; //Line 2
```

4. What is the console output for the following?

```
let a = "Hello";
a = "world";
console.log(a);
```

5. What will be logged to the console?

```
let a = "world";
let b = `Hello ${a}`;
console.log(b);
```

6. What is the value of `a`?

```
let a = "Hello";
a = prompt("world");
console.log(a);
```

7. What is the value of b output to the console?

```
let a = 5;
let b = 70;
let c = "5";
b++;
console.log(b);
```

8. What is the value of result?

```
let result = 3 + 4 * 2 / 8;
```

9. What is the value of total and total2?

```
let firstNum = 5;
let secondNum = 10;
firstNum++;
secondNum--;
let total = ++firstNum + secondNum;
console.log(total);
let total2 = 500 + 100 / 5 + total--;
console.log(total2);
```

10. What is logged to the console here?

```
const a = 5;
const b = 10;
console.log(a > 0 && b > 0);
console.log(a == 5 && b == 4);
console.log(true || false);
console.log(a == 3 || b == 10);
console.log(a == 3 || b == 7);
```

Summary

In this chapter, we dealt with the first two programming building blocks: variables and operators. Variables are special fields that have a name and contain values. We declare a variable by using one of the special variable-defining words: `let`, `var`, or `const`. Variables enable us to make our scripts dynamic, store values, access them later, and change them later. We discussed some primitive data types, including strings, numbers, Booleans, and Symbols, as well as more abstract types such as undefined and null. You learned how to determine the type of a variable using the `typeof` word. And you saw how you can convert the data type by using the built-in JavaScript methods `Number()`, `String()`, and `Boolean()`.

Then we moved on and discussed operators. Operators enable us to work with our variables. They can be used to perform calculations, compare variables, and more. The operators we discussed included arithmetic operators, assignment operators, comparison operators, and logical operators.

After this chapter, you are ready to deal with more complex data types, such as arrays and objects. We'll cover these in the next chapter.

3

JavaScript Multiple Values

The basic data types have been dealt with in the previous chapter. Now it's time to look at a slightly more complicated topic: arrays and objects. In the previous chapter, you saw variables that held just a single value. To allow for more complex programming, objects and arrays can contain multiple values.

You can look at objects as a collection of properties and methods. Properties can be thought of as variables. They can be simple data structures such as numbers and strings, but also other objects. Methods perform actions; they contain a certain number of lines of code that will be executed when the method gets called. We'll explain methods in more detail later in this book and focus on properties for now. An example of an object can be a real-life object, for example, a dog. It has properties, such as name, weight, color, and breed.

We will also discuss arrays. An array is a type of object, which allows you to store multiple values. They are a bit like lists. So, you could have an array of items to buy at the grocery store, which might contain the following values: apples, eggs, and bread. This list would take the form of a single variable, holding multiple values.

Along the way, we will cover the following topics:

- Arrays and their properties
- Array methods
- Multidimensional arrays
- Objects in JavaScript
- Working with objects and arrays

Let's start with arrays.



Note: exercise, project and self-check quiz answers can be found in the *Appendix*.

Arrays and their properties

Arrays are lists of values. These values can be of all data types and one array can even contain different data types. It is often very useful to store multiple values inside one variable; for example, a list of students, groceries, or test scores. Once you start writing scripts, you'll find yourself needing to write arrays very often; for example, when you want to keep track of all the user input, or when you want to have a list of options to present to the user.

Creating arrays

You might be convinced by now that arrays are great, so let's see how we can make them. There is actually a right way and a wrong way to do it. Here are both. Which one do you think is the right one?

```
arr1 = new Array("purple", "green", "yellow");
arr2 = ["black", "orange", "pink"];
```

If you guessed the second option, using square brackets, you are right. This is the best and most readable way to create a new array. On the other hand, the first option can do unexpected things. Look at both lines of code here. What do you think they will do?

```
arr3 = new Array(10);
arr4 = [10];
```

Probably, you sense that something is up here. They do not both create an array with one value, `10`. The second one, `arr4`, does. The first option creates an array with 10 undefined values. If we log the values like this:

```
console.log(arr3);
console.log(arr4);
```

Here is what it logs:

```
[ <10 empty items> ]
[ 10 ]
```

Thanks, JavaScript! That was very helpful. So, unless that is what you need to do, please use the square brackets!

As I already mentioned, we can have mixed arrays and arrays can hold any type of variable. The values of the array won't be converted to a single data type or anything like that. JavaScript simply stores all the variables with their own data type and value in the array:

```
let arr = ["hi there", 5, true];
console.log(typeof arr[0]);
console.log(typeof arr[1]);
console.log(typeof arr[2]);
```

This will output to the console:

```
string
number
boolean
```

The last array fun fact we will go over here is what happens if you define an array using `const`. You can change the values of a constant array, but you cannot change the array itself. Here is a piece of code to demonstrate:

```
const arr = ["hi there"];
arr[0] = "new value";
console.log(arr[0]);

arr = ["nope, now you are overwriting the array"];
```

The new value for the first element of the array is going fine, but you cannot assign a new value to the full array. Here is what it will output:

```
new value
TypeError: Assignment to constant variable.
```

Accessing elements

This beautiful array we just made would become much more powerful if we could access its elements. We can do this by referencing the array's index. This is something we did not specify when we created the array, and we did not need to either. JavaScript assigns an index to every value of the array. The first value is assigned the position of 0, the second 1, the third 2, and so on. If we want to call a specific value based on its position in the array, we can use the name of our array, add square brackets to the end, and put the index we want to access between the square brackets, like this:

```
cars = ["Toyota", "Renault", "Volkswagen"];
console.log(cars[0]);
```

This log statement will write Toyota to the console because we called for the position 0 of the array, which outputs the first value in the list.

```
console.log(cars[1]);
```

Calling index position 1 is giving us the second element in the array, which is Renault. This will be logged to the console.

```
console.log(cars[2]);
```

The third element in our array has index 2, so this one will log Volkswagen. What do you think will happen if we use a negative index or an index that is higher than the number of values we get?

```
console.log(cars[3]);
console.log(cars[-1]);
```

We didn't assign a value to the negative or non-existent index, so when we ask for it, the value is undefined. As such, the log output will be undefined. JavaScript does not throw an error because of this.

Overwriting elements

The elements in an array can be overwritten. This can be done by accessing a certain element using the index and assigning a new value:

```
cars[0] = "Tesla";
console.log(cars[0]);
```

The output of this log is Tesla because it has overwritten the old value, Toyota. If we output the whole array:

```
console.log(cars);
```

It will output the following:

```
[ 'Tesla', 'Renault', 'Volkswagen' ]
```

What happens if you try to overwrite an element that does not exist?

```
cars[3] = "Kia";
```

Or even a negative index?

```
cars[-1] = "Fiat";
```

Let's see what happens when we try to write the values to the console:

```
console.log(cars[3]);
console.log(cars[-1]);
```

And the output:

```
Kia
Fiat
```

Ha! They suddenly exist. How is that you may wonder? We will discuss this in the next section. For now, just remember that this is not the right way to add values to the array. We will discuss the right way when we explain arrays in the *Array methods* section.

Built-in length property

Arrays have a very useful built-in property: `length`. This will return the number of values that the array has:

```
colors = ["black", "orange", "pink"]
booleans = [true, false, false, true];
emptyArray = [];

console.log("Length of colors:", colors.length);
console.log("Length of booleans:", booleans.length);
console.log("Length of empty array:", emptyArray.length);
```

The first `console.log` call returns 3, indicating that the colors array contains 3 values. The second one returns 4, and the last one is an empty array with a length of 0:

```
Length of colors: 3
Length of booleans: 4
Length of empty array: 0
```

So, be aware that the length is one higher than the maximum index because the index of the array starts at 0, but when determining the length, we look at the number of elements and there are four separate elements. This is why the maximum index is 3 when the length is 4. Hence, the positional value of the last element in the array will be one fewer than the total number of elements.

Take a moment and try to figure out how you can use the length to access the last element of the array:

```
lastElement = colors[colors.length - 1];
```

You get the highest index by subtracting 1 from the length because, as you know, arrays are zero-indexed. So, the positional value of the last element in the array will be one fewer than the total number of elements.

So, this might seem pretty straightforward. Remember the non-existent index position we called in the previous section? Let's see what happens in this example:

```
numbers = [12, 24, 36];
numbers[5] = 48;
console.log(numbers.length);
```

The length of the array is only counting the integer numbers starting from 0 up to the highest filled index. If there are elements in the middle of the sequence that do not have a value, they will still get counted. In this case, the length becomes 6. If we log the array, we can see why:

```
console.log("numbers", numbers);
```

The output will be as follows:

```
numbers [ 12, 24, 36, <2 empty items>, 48 ]
```

Because we added an element, 48, at index 5, it also created 2 elements at index positions 3 and 4 containing empty values. For now, let's have a look at array methods and find out the right way to add to an array.

Practice exercise 3.1

1. Create an array to use as your shopping list with 3 items: "Milk," "Bread," and "Apples."
2. Check your list length in the console.
3. Update "Bread" to "Bananas."
4. Output your entire list to the console.

Array methods

We have just seen the built-in length property. We also have a few built-in methods. Methods are functions on a certain object. Instead of holding a value, like properties, they perform actions. We will cover functions in-depth in *Chapter 6, Functions*. For now, all you need to know is that you can call methods and functions, and when you do, some code that is specified inside that function gets executed.

We just accidentally saw we could add elements using new indices. This is not the proper way to do it as it is easy to make mistakes and accidentally overwrite a certain value or skip a certain index. The right way is to do this with a special method. Similarly, we can also delete elements and sort the elements in the array.

Adding and replacing elements

We can add elements with the `push()` method:

```
favoriteFruits = ["grapefruit", "orange", "lemon"];
favoriteFruits.push("tangerine");
```

The value gets added to the end of the array. The push method returns the new length of the array, four in this case. You can store this length in a variable like this:

```
let lengthOfFavoriteFruits = favoriteFruits.push("lime");
```

The value 5 gets stored in the `lengthOfFavoriteFruits` variable. If we log our array, `favoriteFruits`, like this:

```
console.log(favoriteFruits);
```

Here is the new array:

```
[ 'grapefruit', 'orange', 'lemon', 'tangerine', 'lime' ]
```

This was easy right? But what if you would want to add elements at a certain index? You can use the `splice()` method. This one is slightly more difficult:

```
let arrOfShapes = ["circle", "triangle", "rectangle", "pentagon"];
arrOfShapes.splice(2, 0, "square", "trapezoid");
console.log(arrOfShapes);
```

After this, the output containing the array is as follows:

```
[
  'circle',
  'triangle',
  'square',
  'trapezoid',
  'rectangle',
  'pentagon'
]
```

First, let's point out the different layouts of this output. This might depend on the interpreter you are using, but at some point, it will decide it is too long for a single line and apply an automatic format to the array to make it more readable. It doesn't change the value of the array; it is just a different representation of the same values were they to be on a single line.

As you can see, the square and trapezoid get inserted on index 2. The rest of the array is shifting to the right. The `splice()` method takes multiple parameters. The first parameter, 2 in our case, is the index of the array on which we want to start inserting. The second parameter, 0 in our case, is the number of elements we want to delete starting at our previously defined starting index. The parameters after these first two, square and trapezoid in our case, are whatever should be inserted starting at the start index.

So, had we said this instead:

```
arrOfShapes.splice(2, 2, "square", "trapezoid");
console.log(arrOfShapes);
```

It would have replaced the elements `rectangle` and `pentagon` and added square and trapezoid in their place, as follows:

```
[ 'circle', 'triangle', 'square', 'trapezoid' ]
```


If you were to increase the second parameter to a number higher than our array, it would not affect the result as JavaScript would simply stop as soon as it runs out of values to delete. Try the following code:

```
arrOfShapes.splice(2, 12, "square", "trapezoid");
console.log(arrOfShapes);
```

This would also have had this output:

```
[ 'circle', 'triangle', 'square', 'trapezoid' ]
```

You can also add another array to your array. This can be done with the `concat()` method. This way, you can create a new array that consists of a concatenation of both arrays. The elements of the first array will be first, and the elements of the argument of `concat()` will be concatenated to the end:

```
let arr5 = [1, 2, 3];
let arr6 = [4, 5, 6];
let arr7 = arr5.concat(arr6);
console.log(arr7);
```

And here is the output:

```
[ 1, 2, 3, 4, 5, 6 ]
```

This `concat()` method can do even more! We can use it to add values as well. We can add a single value, or we can comma-separate multiple values:

```
let arr8 = arr7.concat(7, 8, 9);
console.log(arr8);
```

The new value of the array will be as follows:

```
[ 1, 2, 3, 4, 5, 6, 7, 8, 9 ]
```

Deleting elements

There are several ways in which you can delete elements from an array. Removing the last element is done with `pop()`:

```
arr8.pop();
```

Logging the array after executing `pop()` results in this:

```
[ 1, 2, 3, 4, 5, 6, 7, 8 ]
```

Deleting the first element can be done with `shift()`. This causes all other indices to be reduced by one:

```
arr8.shift();
```

The new array will be:

```
[ 2, 3, 4, 5, 6, 7, 8 ]
```

Remember `splice()`? This is a very special method because we can also use it to delete values. We specify the index from where we want to start deleting, and then the number of elements we want to delete.

```
arr8.splice(1, 3);
```

After this, the value of the array is as follows:

```
[ 2, 6, 7, 8 ]
```

As you can see, 3 elements starting from the second positional index have been deleted. The values 3, 4, and 5 are gone. If you do not wish to change any of the later indices, you can also use the operator `delete`. This is not a method, but you can use it to change the value of a certain position of the array to undefined:

```
delete arr8[0];
```

The array then becomes:

```
[ <1 empty item>, 6, 7, 8 ]
```

This is useful when you are relying on index or length for some reason. For example, if you are keeping user input, and you want to determine the number of user inputs based on an array that the user is pushing to, deleting would decrease the number of inputs, whereas that might not be what you would want.

Finding elements

If you want to check whether a value is present in an array, you can use the `find()` method. What will go in the `find()` method is somewhat different. It is actually a function. This function will be executed on every element in the array until it finds a match, and if it does not, then it will return undefined.

Do not worry if this is too difficult for now; it will become clear soon enough. We are writing the function in two different ways in the following code snippet. They are actually doing the same, except that the first one is checking for an element being equal to 6, and the second for an element being equal to 10:

```
arr8 = [ 2, 6, 7, 8 ];
let findValue = arr8.find(function(e) { return e === 6 });
let findValue2 = arr8.find(e => e === 10);
console.log(findValue, findValue2);
```

The log statement will log 6 and *undefined* because it can find an element that matches for 6, but not one that matches for 10.

A function can take a certain input. In this case, it takes the element of the array as an input. When the element of the array equals 6 (*findValue*) or 10 (*findValue2*), it returns the element. In *Chapter 6, Functions*, we will cover functions in much more detail. It is a lot for a beginner to take in, so you can review this a bit later if it is unclear for now.

Often, you do not only want to find the element, but you want to know what position it is on. This can be done with the *indexOf()* method. This method returns the index on which the value is found. If a value occurs in an array more than once, it will return the first occurrence. If the value is not found, it will return -1:

```
arr8 = [ 2, 6, 7, 8 ];
let findIndex = arr8.indexOf(6);
let findIndex2 = arr8.indexOf(10);
console.log(findIndex, findIndex2);
```

So, the first one will return 1 since that is the index position of 6 in the array. The second one will return -1 because the array does not contain 10.

If you want to find the next occurrence of the specified number, you can add a second argument to *indexOf()*, specifying from which position it should start searching:

```
arr8 = [ 2, 6, 7, 8 ];
let findIndex3 = arr8.indexOf(6, 2);
```

In this case, the value of *findIndex3* will be -1, because 6 cannot be found starting from index 2.

The last occurrence can also be found. This is done with the *lastIndexOf()* method:

```
let animals = ["dog", "horse", "cat", "platypus", "dog"];
let lastDog = animals.lastIndexOf("dog");
```

The value of *lastDog* will be 4 because that is the last occurrence of dog in the array.

Sorting

There is also a built-in method for sorting arrays. It sorts numbers from small to high and strings A-Z. You can call *sort()* on an array and the order of the values of the array will change to a sorted order:

```
let names = ["James", "Alicia", "Fatiha", "Maria", "Bert"];
names.sort();
```

The value of *names* after sorting is as follows:

```
[ 'Alicia', 'Bert', 'Fatiha', 'James', 'Maria' ]
```

As you can see, the array is now sorted alphabetically. For numbers, it is sorting them in ascending order, as you can see in the following code snippet:

```
let ages = [18, 72, 33, 56, 40];
ages.sort();
```

After executing this *sort()* method, the value of *ages* is:

```
[ 18, 33, 40, 56, 72 ]
```

Reversing

The elements of the array can be reversed by calling the built-in method, *reverse()*, on an array. It puts the last element first, and the first element last. It does not matter whether the array is sorted or not; it just reverses the order.

The value of *names* before reversing is as follows:

```
[ 'Alicia', 'Bert', 'Fatiha', 'James', 'Maria' ]
```

Now we are going to call the *reverse()* method:

```
names.reverse();
```

The new order will be:

```
[ 'Maria', 'James', 'Fatiha', 'Bert', 'Alicia' ]
```

Practice exercise 3.2

1. Create an empty array to use as a shopping list.
2. Add Milk, Bread, and Apples to your list.
3. Update "Bread" with Bananas and Eggs.
4. Remove the last item from the array and output it into the console.
5. Sort the list alphabetically.
6. Find and output the index value of Milk.
7. After Bananas, add Carrots and Lettuce.
8. Create a new list containing Juice and Pop.
9. Combine both lists, adding the new list twice to the end of the first list.
10. Get the last index value of Pop and output it to the console.
11. Your final list should look like this:

```
[ "Bananas", "Carrots", "Lettuce", "Eggs", "Milk", "Juice",  
  "Pop", "Juice", "Pop" ]
```

Multidimensional arrays

Earlier, we established already that arrays can contain any data type. This means that arrays can also contain other arrays (which, in turn, can contain... other arrays!). This is called a multidimensional array. It sounds complicated, but it is just an array of arrays: a list of lists:

```
let someValues1 = [1, 2, 3];  
let someValues2 = [4, 5, 6];  
let someValues3 = [7, 8, 9];  
  
let arrOfArrays = [someValues1, someValues2, someValues3];
```

So, we can create an array of already existing arrays. This is called a two-dimensional array. We can write it like this:

```
let arrOfArrays2 = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];
```

If you want to access elements of the inner arrays, you will have to specify an index twice:

```
let value1 = arrOfArrays[0][1];
```

The statement will grab the first array because it has an index position of 0. From this first array, it will take the second value, because it has an index position of 1. Then it stores this value in value1. That means the value of value1 will be 2. Can you figure out what the value of the next one will be?

```
let value2 = arrOfArrays[2][2];
```

It takes the third array, and from this third array, it takes the third value. Thus, 9 will be stored in value2. And it does not stop here; it can go many levels deep. Let's show that by creating an array of our array of arrays. We are simply going to store this array three times in another array:

```
arrOfArraysOfArrays = [arrOfArrays, arrOfArrays, arrOfArrays];
```

This is what the array looks like in terms of values:

```
{  
  [ [ 1, 2, 3 ], [ 4, 5, 6 ], [ 7, 8, 9 ] ],  
  [ [ 1, 2, 3 ], [ 4, 5, 6 ], [ 7, 8, 9 ] ],  
  [ [ 1, 2, 3 ], [ 4, 5, 6 ], [ 7, 8, 9 ] ]  
}
```

Let's get the middle element of this array, which is the value 5, belonging to the second array of arrays. It is done like this:

```
let middleValue = arrOfArraysOfArrays[1][1][1];
```

The first step is to get the second array of arrays, so index 1. Then we need to get the second array of this one, which again is index 1. Now we reach the level of the values, and we need the second value, so again we use index 1. This is useful in many situations, for example, when you want to work with matrices.

Practice exercise 3.3

1. Create an array containing three values: 1, 2, and 3.
2. Nest the original array into a new array three times.
3. Output the value 2 from one of the arrays into the console.

Objects in JavaScript

Now it is time to have a look at another complex data structure that can contain more than one value: objects! Objects are very useful and can be used to describe real-life objects as well as more complex abstract concepts that allow for more flexibility in your code.

Secretly, you have just been introduced to objects already, because arrays are a very special type of object. Arrays are objects with indexed properties. All the other objects, and also the objects we will see here, are objects with named properties. This means that instead of an automatically generated index number, we will give it a custom descriptive name.

As we can tell from the following code, arrays are defined by JavaScript as being of the object type:

```
let arr = [0, 1, 2];
console.log(typeof arr);
```

The output of the preceding code is as follows:

```
Object
```

Objects are not too dissimilar to real-world objects. They *have* properties and they *can* perform actions, methods. Here, we will only deal with the properties. We will cover methods in *Chapter 7, Classes*, after we have seen functions. An object is a chance to group multiple variables into one. This is done with curly brackets: { and }. Let's have a look at this object of a dog here:

```
let dog = {
  dogName: "JavaScript",
  weight: 2.4,
  color: "brown",
  breed: "chihuahua",
  age: 3,
  burglarBiter: true
};
```

We created a variable, `dog`, and we gave this an object as a value. We can recognize that this is an object by seeing the { and }. In between the curly braces, we see a bunch of properties and their values.

If you have ever wondered whether something should be a property, just try the following template sentence in your head:

"objectname" has a(n) "property name"

For example, a dog has a name, a dog has a color, and a dog has a weight. This is slightly different for the Boolean properties, for which you can use "is" or "is not" instead of "has".

We can access the properties of this object in a very similar way as we would with the array. This time, we are not using the index number, but the name of the property, to get the value:

```
let dogColor1 = dog["color"];
```

There is another way to do this. Instead of the square brackets, the property name can also be added to the object name with a dot in between:

```
let dogColor2 = dog.color;
```

This might look familiar. Do you remember how we got the length of an array with the built-in property `length`? Yes – the same way! The difference between properties and methods is the lack of parentheses for properties.

Updating objects

We can change the value of the properties of the objects. Again, this is similar to an array because an array is an object as well, but for properties, we have two options:

```
dog["color"] = "blue";
dog.weight = 2.3;
```

This has changed the properties of our chihuahua JavaScript. The color gets updated to blue and it has lost a little bit of weight since the new weight is 0.1 lower. So if we log our dog:

```
console.log(dog);
```

We will get the following:

```
{
  dogName: 'JavaScript',
  weight: 2.3,
  color: 'blue',
```

```

    breed: "chihuahua",
    age: 3,
    burglarBiter: true
  }

```

It's useful to note that if we change the data type of one of our properties, for example:

```
dog["age"] = "three";
```

This is not a problem. JavaScript will just change the whole value and data type to the new situation.

Another element to note is that we are now using the literal string values to refer to the object's properties, but we can also work with variables to achieve this. So, for example:

```

let variable = "age";
console.log(dog[variable]);

```

This will still output three, as we just changed the value of age to three. If we change the value of the variable to another dog property, we will be accessing another property, like this:

```

variable = "breed";
console.log(dog[variable]);

```

This will print chihuahua. And when we update the value accessing this way, it is the exact same as when we would have accessed it with the literal string:

```

dog[variable] = "dachshund";
console.log(dog["breed"]);

```

So, this will log dachshund to the console.

Practice exercise 3.4

1. Create a new `myCar` object for a car. Add some properties, including, but not limited to, `make` and `model`, and values for a typical car or your car. Feel free to use booleans, strings, or numbers.

2. Create a variable that can hold the string value `color`. This variable containing a string value `color` can now be used to reference the property name within `myCar`. Then, use the variable within the square bracket notation to assign a new value to the `color` property in `myCar`.
3. Use that same variable and assign a new property string value to it, such as `forSale`. Use the bracket notation once again to assign a new value to the `forSale` property to indicate whether the car is available for purchase.
4. Output `make` and `model` into the console.
5. Output the value of `forSale` into the console.

Working with objects and arrays

When working with objects and arrays, you will see these often combined. In the last section of this chapter, we will deal with combining objects and arrays, and also objects inside objects.

Objects in objects

Let's say we want to have an object for a company. This company will have an address. And an address is another object. If we give our company an address, we are using an object inside an object:

```

let company = {
  companyName: "Healthy Candy",
  activity: "food manufacturing",
  address: {
    street: "2nd street",
    number: "123",
    zipcode: "33116",
    city: "Miami",
    state: "Florida"
  },
  yearOfEstablishment: 2021
};

```

As you can see, our company object has an address object with values. This can go very many levels deep if necessary.

To access or modify one of the properties of the address here, we can use two approaches:

```
company.address.zipcode = "33117";
company["address"]["number"] = "100";
```

As you can see, this is very similar to the array. We first need to select the address and then do the same thing to access the property we want to change.

Arrays in objects

Our company might have a range of activities instead of one. We can simply replace the activity from our previous sample with an array:

```
company = { companyName: "Healthy Candy",
  activities: ["food manufacturing",
    "improving kids' health", "manufacturing toys"],
  address: {
    street: "2nd street",
    number: "123",
    zipcode: "33116",
    city: "Miami",
    state: "Florida"
  },
  yearOfEstablishment: 2021
};
```

We have now used an array in our company object. You can simply use an array with the square braces after the property. Retrieving the individual values is very similar. The second value of the activities array can be fetched using this statement:

```
let activity = company.activities[1];
```

Here, we call the object we're interested in, `company`, then the relevant array, `activities`, with reference to the index position of the variable we're looking for within the array, which is 1.

Objects in arrays

It is very possible that instead of one address, our company has a list of addresses. We can accomplish this by creating an array of address objects. In this case, we will create an array of two:

```
let addresses = [{
  street: "2nd street",
  number: "123",
  zipcode: "33116",
  city: "Miami",
  state: "Florida"
},
{
  street: "1st West avenue",
  number: "5",
  zipcode: "75001",
  city: "Addison",
  state: "Texas"
}];
```

So, the arrays can be recognized by the square brackets and the objects by the curly brackets. The street name of the first object can be fetched using this statement:

```
let streetName = addresses[0].street;
```

Here, we call the array we're interested in, `addresses`, with reference to the index position of the object we're looking for within the array, 0, and then the required variable from within the object, which is `street`. This may seem complicated, but you may notice that this simply reverses the syntax required to retrieve a variable from an array inside an object from the previous section. It's worth practicing calling variables from nested arrays and objects until you're comfortable with it!

Objects in arrays in objects

Just to show that this can go as many levels as we would need, we are going to give our company object an array of address objects. So, let's add this array of address objects to our company object. This way, our company has an array of addresses:

```
company = { companyName: "Healthy Candy",
  activities: [ "food manufacturing",
    "improving kids' health",
```

```
"manufacturing toys"],
  address: [{
    street: "2nd street",
    number: "123",
    zipcode: "33116",
    city: "Miami",
    state: "Florida"
  }],
  {
    street: "1st West avenue",
    number: "5",
    zipcode: "75001",
    city: "Addison",
    state: "Texas"
  }],
  yearOfEstablishment: 2021
};
```

To access elements of increasingly nested objects and arrays, we simply extend the same logic you have seen in the previous sections. To access the street name of Healthy Candy's first address, we can use the following code:

```
let streetName = company.address[0].street;
```

As you see, we can stack object and array element requests indefinitely.

We will not make it any more complicated than this for now. Whenever you need a list of something, you will be using an array. Whenever you want to represent something with properties that have descriptive names, it is better to use an object. Just remember that object properties can be of any type.

Practice exercise 3.5

1. Create an object named `people` that contains an empty array that is called `friends`.
2. Create three variables, each containing an object, that contain one of your friend's first names, last names, and an ID value.
3. Add the three friends to the `friends` array.
4. Output it to the console.

Chapter projects

Manipulating an array

Take the following array:

```
const theList = ['Laurence', 'Svekis', true, 35, null, undefined,
  {test: 'one', score: 55}, ['one', 'two']];
```

Manipulate your array using various methods, such as `pop()`, `push()`, `shift()`, and `unshift()`, and transform it into the following:

```
["FIRST", "Svekis", "MIDDLE", "hello World", "LAST"]
```

You can take the following steps, or adopt your own approach:

- Remove the first item and the last item.
- Add `FIRST` to the start of the array.
- Assign `hello World` to the fourth item value.
- Assign `MIDDLE` to the third index value.
- Add `LAST` to the last position in the array.
- Output it to the console.

Company product catalog

In this project, you will implement a data structure for a product catalog and create queries to retrieve data.

1. Create an array to hold an inventory of store items.
2. Create three items, each having the properties of name, model, cost, and quantity.
3. Add all three objects to the main array using an array method, and then log the inventory array to the console.
4. Access the quantity element of your third item, and log it to the console. Experiment by adding and accessing more elements within your data structure.

Self-check quiz

1. Can you use `const` and update values within an array?
2. Which property in an array gives the number of items contained in the array?
3. What is the output in the console?

```
const myArr1 = [1,3,5,6,8,9,15];
console.log(myArr1.indexOf(0));
console.log(myArr1.indexOf(3));
```

4. How do you replace the second element in an array `myArr = [1,3,5,6,8,9,15]` with the value 4?
5. What is the output in the console?

```
const myArr2 = [];
myArr2[10] = 'test';
console.log(myArr2);
console.log(myArr2[2]);
```

6. What is the output in the console?

```
const myArr3 = [3,6,8,9,3,55,553,434];
myArr3.sort();
myArr3.length = 0;
console.log(myArr3[0]);
```

Summary

So, in this chapter, we have seen arrays and objects. Arrays are a list of values. These could be values of the same type, but also values of different types. Every element of the array gets an index. The index of the first element is 0. We can access the elements of the array using this index. We can also use this index to change and delete the element.

We then saw that it is also possible to have arrays containing other arrays; these are multidimensional arrays. To access the elements of a multidimensional array, you would need to use as many indices as you have nested arrays.

Then, we covered objects and learned that arrays are a special kind of object. Objects contain properties and methods. We looked at the properties of objects and saw that these properties are given a name and can be accessed and modified using this name.

We ended this module by looking at how arrays can contain objects, and how objects can contain arrays and more. This enables us to create complex object structures, which will be of great use in designing real-life applications.

4

Logic Statements

Up to this point, our code has been rather static. It will do the same thing every time we execute it. In this chapter, that is all going to change. We will be dealing with logical statements. Logical statements allow us to make multiple paths in our code. Depending on the outcome of a certain expression, we will follow one code path or another.

There are different logic statements, and we will go over them in this chapter. We will start with `if` and `if else` statements. After that, we will be dealing with the ternary operator, and the final one we will be dealing with is the `switch` statement.

Along the way, we will cover the following topics:

- `if` and `if else` statements
- `else if` statements
- Conditional ternary operators
- `switch` statements



Note: exercise, project and self-check quiz answers can be found in the Appendix.

if and if else statements

We can make decisions in our code using `if` and `if else` statements. It is very much like this template:

if "some condition is true", then "a certain action will happen", else "another action will happen"

For example, *if* it is raining then, I will take my umbrella, *else* I will leave my umbrella at home. It is not that much different in code:

```
let rain = true;

if(rain){
  console.log("** Taking my umbrella when I need to go outside **");
} else {
  console.log("** I can leave my umbrella at home **");
}
```

In this case, the value of `rain` is `true`. And therefore, it will log to the console:

```
** Taking my umbrella when I need to go outside **
```

But let's first take a step back and look at the syntax. We start with the word "`if`." After this, we get something within parentheses. Whatever is between these parentheses will be translated to a Boolean. If the value of this Boolean is `true`, it will execute the block of code associated with `if`. You can recognize this block by the curly braces.

The next block is optional; it is an `else` block. It starts with the word "`else`" and is only executed in case of the Boolean having the value `false`. If there is no `else` block and the condition evaluates to `false`, the program will just skip ahead to the code underneath the `if`.

Only one of these two blocks will be executed; the `if` block when the expression is `true`, and the `else` block when the expression is `false`:

```
if(expression) {
  // code associated with the if block
  // will only be executed if the expression is true
} else {
  // code associated with the else block
  // we don't need an else block, it is optional
  // this code will only be executed if the expression is false
}
```

Here is another example. If the age is below 18, log to the console that access is denied, otherwise log to the console that the person is allowed to come in:

```
if(age < 18) {
  console.log("We're very sorry, but you can't get in under 18");
} else {
  console.log("Welcome!");
}
```

There is a common coding mistake related to `if` statements. I have made it in the following code snippet. Can you see what this code does?

```
let hobby = "dancing";

if(hobby == "coding"){
  console.log("** I love coding too! **");
} else {
  console.log("** Can you teach me that? **");
}
```

It will log the following:

```
** I love coding too! **
```

That might surprise you. The problem here is the single equal sign in the `if` statement. Instead of evaluating the condition, it is assigning coding to hobby. And then it is converting coding to a Boolean, and since it is not an empty string, it will become true, so the `if` block will be executed. So, always remember to use the double equal sign in this case.

Let's test our knowledge with a practice exercise.

Practice exercise 4.1

1. Create a variable with a Boolean value.
2. Output the value of the variable to the console.
3. Check whether the variable is true and if so, output a message to the console, using the following syntax:

```
if(myVariable){
  //action
}
```

4. Add another `if` statement with an `!` in front of the variable to check whether the condition is *not* true, and create a message that will be printed to the console in that instance. You should have two `if` statements, one with an `!` and the other without. You could also use an `if` and an `else` statement instead—experiment!
5. Change the variable to the opposite to see how the result changes.

else if statements

A variation of the `if` statement is an `if` statement with multiple `else if` blocks. This can be more efficient in certain situations because you are always only going to execute one or zero blocks. If you have many `if else` statements underneath one another, they are going to be evaluated and possibly executed even though one of the ones above already had a condition evaluate to true and proceeded to execute the associated code block.

Here is the written template:

If "a value falls into a certain category", then "a certain action will happen", else if "the value falls into a different category than the previous statement", then "a certain action will happen", else if "the value falls into a different category than either of the previous brackets", then "a certain action will happen"

For example, take this statement, to determine what the ticket price should be. If a person is younger than 3, then access is free, else if a person is older than 3 and younger than 12, then access is 5 dollars, else if a person is older than 12 and younger than 65, then access is 10 dollars, else if a person is 65 or older, then access is 7 dollars:

```
let age = 10;
let cost = 0;
let message;
if (age < 3) {
  cost = 0;
  message = "Access is free under three.";
} else if (age >= 3 && age < 12) {
  cost = 5;
  message = "With the child discount, the fee is 5 dollars";
} else if (age >= 12 && age < 65) {
  cost = 10;
  message = "A regular ticket costs 10 dollars.";
}
```

```

} else {
  cost = 7;
  message = "A ticket is 7 dollars.";
}

console.log(message);
console.log("Your Total cost "+cost);

```

Chances are that you will think the code is easier to read than the written template. In that case, nicely done! You are really starting to think like a JavaScript developer already.

The code gets executed top to bottom, and only one of the blocks will be executed. As soon as a true expression is encountered, the other ones will be ignored. This is why we can also write our sample like this:

```

if(age < 3){
  console.log("Access is free under three.");
} else if(age < 12) {
  console.log("With the child discount, the fee is 5 dollars");
} else if(age < 65) {
  console.log("A regular ticket costs 10 dollars.");
} else if(age >= 65) {
  console.log("A ticket is 7 dollars.");
}

```

Practice exercise 4.2

1. Create a prompt to ask the user's age
2. Convert the response from the prompt to a number
3. Declare a message variable that you will use to hold the console message for the user
4. If the input age is equal to or greater than 21, set the message variable to confirm entry to a venue and the ability to purchase alcohol
5. If the input age is equal to or greater than 19, set the message variable to confirm entry to the venue but deny the purchase of alcohol
6. Provide a default else statement to set the message variable to deny entry if none are true
7. Output the response message variable to the console

Conditional ternary operators

We did not actually discuss this very important operator in our section on operators in *Chapter 2, JavaScript Essentials*. This is because it helps to understand the if else statement first. Remember that we had a unary operator that was called a unary operator because it only had one operand? This is why our ternary operator has its name; it has three operands. Here is its template:

```
operand1 ? operand2 : operand3;
```

operand1 is the expression that is to be evaluated. If the value of the expression is true, operand2 gets executed. If the value of the expression is false, operand3 gets executed. You can read the question mark as "then" and the colon as "else" here:

```
expression ? statement for true : statement associated with false;
```

The template for saying it in your head should be:

if "operand1", then "operand2", else "operand3"

Let's have a look at a few examples:

```
let access = age < 18 ? "denied" : "allowed";
```

This little code snippet will assign a value to access. *If* age is lower than 18, *then* it will assign the value *denied*, *else* it will assign the value *allowed*. You can also specify an action in a ternary statement, like this:

```
age < 18 ? console.log("denied") : console.log("allowed");
```

This syntax can be confusing at first. The template of what to say in your head while reading it can really come to the rescue here. You can only use these ternary operators for very short actions, so it's best practice to use the ternary operator in these instances as it makes code easier to read. However, if the logic contains multiple comparison arguments, you'll have to use the regular if-else.

Practice exercise 4.3

1. Create a Boolean value for an ID variable
2. Using a ternary operator, create a message variable that will check whether their ID is valid and either allow a person into a venue or not
3. Output the response to the console

switch statements

If else statements are great for evaluating Boolean conditions. There are many things you can do with them, but in some cases, it is better to replace them with a switch statement. This is especially the case when evaluating more than four or five values.

We are going to see how switch statements can help us and what they look like. First, have a look at this if else statement:

```
if(activity === "Get up") {
  console.log("It is 6:30AM");
} else if(activity === "Breakfast") {
  console.log("It is 7:00AM");
} else if(activity === "Drive to work") {
  console.log("It is 8:00AM");
} else if(activity === "Lunch") {
  console.log("It is 12:00PM");
} else if(activity === "Drive home") {
  console.log("It is 5:00PM");
} else if(activity === "Dinner") {
  console.log("It is 6:30PM");
}
```

It is determining what the time is based on what we are doing. It would be better to implement this using a switch statement. The syntax of a switch statement looks like this:

```
switch(expression) {
  case value1:
    // code to be executed
    break;
  case value2:
    // code to be executed
    break;
  case value-n:
    // code to be executed
    break;
}
```

You can read it in your head as follows: If the expression equals value1, do whatever code is specified for that case. If the expression equals value2, do whatever code is specified for that case, and so on.

Here is how we can rewrite our long if else statement in a much cleaner manner using a switch statement:

```
switch(activity) {
  case "Get up":
    console.log("It is 6:30AM");
    break;
  case "Breakfast":
    console.log("It is 7:00AM");
    break;
  case "Drive to work":
    console.log("It is 8:00AM");
    break;
  case "Lunch":
    console.log("It is 12:00PM");
    break;
  case "Drive home":
    console.log("It is 5:00PM");
    break;
  case "Dinner":
    console.log("It is 6:30PM");
    break;
}
```

If our activity has the value Lunch it will output the following to the console:

```
It is 12:00PM
```

What's up with all these breaks, you may be wondering? If you do not use the command `break` at the end of a case, it will execute the next case as well. This will be done from the case where it has a match, until the end of the switch statement or until we encounter a break statement. This is what the output of our switch statement would be without breaks for the Lunch activity:

```
It is 12:00PM
It is 5:00PM
It is 6:30PM
```

One last side note. `switch` uses strict type checking (the triple equals strategy) to determine equality, which checks for both a value and a data type.

The default case

There is one part of switch that we have not worked with yet, and that is a special case label, namely, default. This works a lot like the else part of an if else statement. If it does not find a match with any of the cases and a default case is present, then it will execute the code associated with the default case. Here is the template of a switch statement with a default case:

```
switch(expression) {
  case value1:
    // code to be executed
    break;
  case value2:
    // code to be executed
    break;
  case value-n:
    // code to be executed
    break;
  default:
    // code to be executed when no cases match
    break;
}
```

The convention is to have the default case as the last case in the switch statement, but the code will work just fine when it is in the middle or the first case. However, we recommend you stick to the conventions and have it as a last case, since that is what other developers (and probably your future self) will expect when dealing with your code later.

Let's say our long if statement has an else statement associated with it that looks like this:

```
if(_) {
  // omitted to avoid making this unnecessarily long
} else {
  console.log("I cannot determine the current time.");
}
```

The switch statement would then look like this:

```
switch(activity) {
  case "Get up":
    console.log("It is 6:30AM");
}
```

```
break;
case "Breakfast":
  console.log("It is 7:00AM");
  break;
case "Drive to work":
  console.log("It is 8:00AM");
  break;
case "Lunch":
  console.log("It is 12:00PM");
  break;
case "Drive home":
  console.log("It is 5:00PM");
  break;
case "Dinner":
  console.log("It is 6:30PM");
  break;
default:
  console.log("I cannot determine the current time.");
  break;
}
```

If the value of the activity was to be something that is not specified as a case, for example, "Watch Netflix," it would log the following to the console:

```
I cannot determine the current time.
```

Practice exercise 4.4

As discussed in *Chapter 1, Getting Started with JavaScript*, the JavaScript function `Math.random()` will return a random number in the range of 0 to less than 1, including 0 but not 1. You can then scale it to the desired range by multiplying the result and using `Math.floor()` to round it down to the nearest whole number; for example, to generate a random number between 0 and 9:

```
// random number between 0 and 1
let randomNumber = Math.random();
// multiply by 10 to obtain a number between 0 and 10
randomNumber = randomNumber * 10;
// removes digits past decimal place to provide a whole number
RandomNumber = Math.floor(randomNumber);
```

In this exercise, we'll create a Magic 8-Ball random answer generator:

1. Start by setting a variable that gets a random value assigned to it. The value is assigned by generating a random number 0-5, for 6 possible results. You can increase this number as you add more results.
2. Create a prompt that can get a string value input from a user that you can repeat back in the final output.
3. Create 6 responses using the switch statement, each assigned to a different value from the random number generator.
4. Create a variable to hold the end response, which should be a sentence printed for the user. You can assign different string values for each case, assigning new values depending on the results from the random value.
5. Output the user's original question, plus the randomly selected case response, to the console after the user enters their question.

Combining cases

Sometimes, you would want to do the exact same thing for multiple cases. In an if statement, you would have to specify all the different `or (||)` clauses. In a switch statement, you can simply combine them by putting them on top of each other like this:

```
switch(grade){
  case "F":
  case "D":
    console.log("You've failed!");
    break;
  case "C":
  case "B":
    console.log("You've passed!");
    break;
  case "A":
    console.log("Nice!");
    break;
  default:
    console.log("I don't know this grade.");
}
```

For the values F and D, the same thing is happening. This is also true for C and B. When the value of `grade` is either C or B, it will log the following to the console:

```
You've passed!
```

This is more readable than the alternative if-else statement:

```
if(grade === "F" || grade === "D") {
  console.log("You've failed!");
} else if(grade === "C" || grade === "B") {
  console.log("You've passed!");
} else if(grade === "A") {
  console.log("Nice!");
} else {
  console.log("I don't know this grade.");
}
```

Practice exercise 4.5

1. Create a variable called `prize` and use a prompt to ask the user to set the value by selecting a number between 0 and 10
2. Convert the prompt response to a number data type
3. Create a variable to use for the output message containing the value `"My Selection: "`
4. Using the switch statement (and creativity), provide a response back regarding a prize that is awarded depending on what number is selected
5. Use the switch break to add combined results for prizes
6. Output the message back to the user by concatenating your prize variable strings and the output message string

Chapter projects

Evaluating a number game

Ask the user to enter a number and check whether it's greater than, equal to, or less than a dynamic number value in your code. Output the result to the user.

Friend checker game

Ask the user to enter a name, using the switch statement to return a confirmation that the user is a friend if the name selected is known in the case statements. You can add a default response that you don't know the person if it's an unknown name. Output the result into the console.

Rock Paper Scissors game

This is a game between a player and the computer, where both will make a random selection of either Rock, Paper, or Scissors (alternatively, you could create a version using real player input). Rock will beat out Scissors, Paper will beat out Rock, and Scissors will beat out Paper. You can use JavaScript to create your own version of this game, applying the logic with an if statement. Since this project is a little more difficult, here are some suggested steps:

1. Create an array that contains the variables Rock, Paper, and Scissors.
2. Set up a variable that generates a random number 0-2 for the player and then do the same for the computer's selection. The number represents the index values in the array of the 3 items.
3. Create a variable to hold a response message to the user. This can show the random results for the player and then also the result for the computer of the matching item from the array.
4. Create a condition to handle the player and computer selections. If both are the same, this results in a tie.
5. Use conditions to apply the game logic and return the correct results. There are several ways to do this with the condition statements, but you could check which player's index value is bigger and assign the victory accordingly, with the exception of Rock beating Scissors.
6. Add a new output message that shows the player selection versus the computer selection and the result of the game.

Self-check quiz

1. What will be outputted to the console in this instance?

```
const q = '1';
switch (q) {
  case '1':
    answer = "one";
```

```
    break;
  case 1:
    answer = 1;
    break;
  case 2:
    answer = "this is the one";
    break;
  default:
    answer = "not working";
}
console.log(answer);
```

2. What will be outputted to the console in this instance?

```
const q = 1;

switch (q) {
  case '1':
    answer = "one";
  case 1:
    answer = 1;
  case 2:
    answer = "this is the one";
    break;
  default:
    answer = "not working";
}
console.log(answer);
```

3. What will be outputted to the console in this instance?

```
let login = false;
let outputHolder = "";
let userOkay = login ? outputHolder = "logout" : outputHolder = "login";
console.log(userOkay);
```

4. What will be outputted to the console in this instance?

```
const userNames = ["Mike", "John", "Larry"];
const userInput = "John";
let htmlOutput = "";
if (userNames.indexOf(userInput) > -1) {
  htmlOutput = "Welcome, that is a user";
```

```

    } else {
      htmlOutput = "Denied, not a user ";
    }
    console.log(htmlOutput + ": " + userInput);
  }

```

5. What will be outputted to the console in this instance?

```

let myTime = 9;
let output;
if (myTime >= 8 && myTime < 12) {
  output = "Wake up, it's morning";
} else if (myTime >= 12 && myTime < 13) {
  output = "Go to lunch";
} else if (myTime >= 13 && myTime <= 16) {
  output = "Go to work";
} else if (myTime > 16 && myTime < 20) {
  output = "Dinner time";
} else if (myTime >= 22) {
  output = "Time to go to sleep";
} else {
  output = "You should be sleeping";
}
console.log(output);

```

6. What will be outputted to the console in this instance?

```

let a = 5;
let b = 10;
let c = 20;
let d = 30;
console.log(a > b || b > a);
console.log(a > b && b > a);
console.log(d > b || b > a);
console.log(d > b && b > a);

```

7. What will be outputted to the console in this instance?

```

let val = 100;
let message = (val > 100) ? `${val} was greater than 100` :
`${val} was LESS or Equal to 100`;
console.log(message);
let check = (val % 2) ? `Odd` : `Even`;
check = `${val} is ${check}`;
console.log(check);

```

Summary

Now, let's wrap things up. In this chapter, we have covered conditional statements. We started with if else statements. Whenever the condition associated with the if is true, the if block gets executed. If the condition is false and there is an else block present, that will be executed. We have also seen ternary operators and the funky syntax they bring to the table. It is a short way of writing an if-else statement if you only need one statement per block.

And lastly, we have seen switch statements and how they can be used to optimize our conditional code. With the switch statement, we can compare one condition with many different cases. When they are equal (value and type), the code associated with the case gets executed.

In the next chapter, we are going to add loops to the mix! This is going to help us write more efficient code and algorithms.

5

Loops

We are starting to get a good basic grasp of JavaScript. This chapter will focus on a very important control flow concept: loops. Loops execute a code block a certain number of times. We can use loops to do many things, such as repeating operations a number of times and iterating over data sets, arrays, and objects. Whenever you feel the need to copy a little piece of code and place it right underneath where you copied it from, you should probably be using a loop instead.

We will first discuss the basics of loops, then continue to discuss nesting loops, which is basically using loops inside loops. Also, we will explain looping over two complex constructs we have seen, arrays and objects. And finally, we will introduce two keywords related to loops, `break` and `continue`, to control the flow of the loop even more.



There is one topic that is closely related to loops that is not in this chapter. This is the built-in `foreach` method. We can use this method to loop over arrays, when we can use an arrow function. Since we won't discuss these until the next chapter, `foreach` is not included here.

These are the different loops we will be discussing in this chapter:

- `while` loop
- `do while` loop
- `for` loop
- `for in`
- `for of` loop



Note: exercise, project, and self-check quiz answers can be found in the *Appendix*.

while loops

The first loop we will discuss is the **while** loop. A `while` loop executes a certain block of code as long as an expression evaluates to `true`. The snippet below demonstrates the syntax of the `while` loop:

```
while (condition) {  
  // code that gets executed as long as the condition is true  
}
```

The `while` loop will only be executed as long as the condition is `true`, so if the condition is `false` to begin with, the code inside will be skipped.

Here is a very simple example of a `while` loop printing the numbers 0 to 10 (excluding 10) to the console:

```
let i = 0;  
while (i < 10) {  
  console.log(i);  
  i++;  
}
```

The output will be as follows:

```
1  
2  
3  
4  
5  
6  
7  
8  
9
```

These are the steps happening here:

1. Create a variable, `i`, and set its value to zero
2. Start the `while` loop and check the condition that the value of `i` is smaller than 10

3. Since the condition is true, the code logs 1 and increases `i` by 1
4. The condition gets evaluated again; 1 is still smaller than 10
5. Since the condition is true, the code logs 1 and increases `i` by 1
6. The logging and increasing continues until `i` becomes 10
7. 10 is not smaller than 10, so the loop ends

We can have a `while` loop that looks for a value in an array, like this:

```
let someArray = ["Mike", "Antal", "Marc", "Emir", "Louiza", "Jacky"];
let notFound = true;

while (notFound && someArray.length > 0) {
  if (someArray[0] === "Louiza") {
    console.log("Found her!");
    notFound = false;
  } else {
    someArray.shift();
  }
}
```

It checks whether the first value of the array is a certain value, and when it is not, it deletes that value from the array using the `shift` method. Remember this method? It removes the first element of the array. So, by the next iteration, the first value has changed and is checked again. If it stumbles upon the value, it will log this to the console and change the Boolean `notFound` to `false`, because it has found it. That was the last iteration and the loop is done. It will output:

```
Found her!
false
```

Why do you think the `&& someArray.length > 0` is added in the `while` condition? If we were to leave it out, and the value we were looking for was not in the array, we would get stuck in an infinite loop. This is why we make sure that we also end things if our value is not present, so our code can continue.

But we can also do more sophisticated things very easily with loops. Let's see how easy it is to fill an array with the Fibonacci sequence using a loop:

```
let nr1 = 0;
let nr2 = 1;
let temp;
```

```
fibonacciArray = [];

while (fibonacciArray.length < 25) {
  fibonacciArray.push(nr1);
  temp = nr1 + nr2;
  nr1 = nr2;
  nr2 = temp;
}
```

In the Fibonacci sequence, each value is the sum of the two previous values, starting with the values 0 and 1. We can do this in a `while` loop as stated above. We create two numbers and they change every iteration. We have limited our number of iterations to the length of the `fibonacciArray`, because we don't want an infinite loop. In this case the loop will be done as soon as the length of the array is no longer smaller than 25.

We need a temporary variable that stores the next value for `nr2`. And every iteration we push the value of the first number to the array. If we log the array, you can see the numbers getting rather high very quickly. Imagine having to generate these values one by one in your code!

```
[
  0,    1,    1,    2,    3,
  5,    8,   13,   21,   34,
  55,   89,  144,  233,  377,
  610,  987, 1597, 2584, 4181,
  6765, 10946, 17711, 28657, 46368
]
```

Practice exercise 5.1

In this exercise we will create a number guessing game that takes user input and replies based on how accurate the user's guess was.

1. Create a variable to be used as the max value for the number guessing game.
2. Generate a random number for the solution using `Math.random()` and `Math.floor()`. You will also need to add 1 so that the value is returned as 1-[whatever the set max value is]. You can log this value to the console for development to see the value as you create the game, then when the game is complete you can comment out this console output.

3. Create a variable that will be used for tracking whether the answer is correct or not and set it to a default Boolean value of `false`. We can update it to be `true` if the user guess is a match.
4. Use a `while` loop to iterate a prompt that asks the user to enter a number between 1 and 5, and convert the response into a number in order to match the data type of the random number.
5. Inside the `while` loop, check using a condition to see if the prompt value is equal to the solution number. Apply logic such that if the number is correct, you set the status to `true` and break out of the loop. Provide the player with some feedback as to whether the guess was high or low, and initiate another prompt until the user guesses correctly. In this way we use the loop to keep asking until the solution is correct, and at that point we can stop the iteration of the block of code.

do while loops

In some cases, you really need the code block to be executed at least once. For example, if you need valid user input, you need to ask at least once. The same goes for trying to connect with a database or some other external source: you will have to do so at least once in order for it to be successful. And you will probably need to do so as long as you did not get the result you needed. In these cases, you can use a **do while loop**.

Here is what the syntax looks like:

```
do {
  // code to be executed if the condition is true
} while (condition);
```

It executes what is within the `do` block, and then after that it evaluates the `while`. If the condition is `true`, it will execute what is in the `do` block again. It will continue to do so until the condition in the `while` changes to `false`.

We can use the `prompt()` method to get user input. Let's use a `do while` loop to ask the user for a number between 0 and 100.

```
let number;
do {
  number = prompt("Please enter a number between 0 and 100: ");
} while (!(number >= 0 && number < 100));
```

Here is the output; you will have to enter the number in the console yourself here.

```
Please enter a number between 0 and 100: > -50
Please enter a number between 0 and 100: > 150
Please enter a number between 0 and 100: > 34
```



Everything behind the `>` is user input here. The `>` is part of the code; it is added by the console to make the distinction between console output (*Please enter a number between 0 and 100*) and the console input (*-50, 150, and 34*) clearer.

It asks three times, because the first two times the number was not between 0 and 100 and the condition in the `while` block was `true`. With 34, the condition in the `while` block became `false` and the loop ended.

Practice exercise 5.2

In this exercise, we will create a basic counter that will increase a dynamic variable by a consistent step value, up to an upper limit.

1. Set the starting counter to 0
2. Create a variable, `step`, to increase your counter by
3. Add a `do while` loop, printing the counter to the console and incrementing it by the step amount each loop
4. Continue to loop until the counter is equal to 100 or more than 100

for loops

for loops are special loops. The syntax might be a little bit confusing at first, but you will find yourself using them soon, because they are very useful.

Here is what the syntax looks like:

```
for (initialize variable; condition; statement) {
  // code to be executed
}
```

Between the parentheses following the `for` statement, there are three parts, separated by semi-colons. The first one initializes the variables that can be used in the `for` loop. The second one is a condition: as long as this condition is true, the loop will keep on iterating. This condition gets checked after initializing the variables before the first iteration (this will only take place when the condition evaluates to true). The last one is a statement. This statement gets executed after every iteration. Here is the flow of a `for` loop:

1. Initialize the variables.
2. Check the condition.
3. If the condition is true, execute the code block. If the condition is false, the loop will end here.
4. Perform the statement (the third part, for example, `i++`).
5. Go back to step 2.

This is a simple example that logs the numbers 0 to 10 (excluding 10) to the console:

```
for (let i = 0; i < 10; i++) {
  console.log(i);
}
```

It starts by creating a variable, `i`, and sets this to 0. Then it checks whether `i` is smaller than 10. If it is, it will execute the `log` statement. After this, it will execute `i++` and increase `i` by one.



If we don't increase `i`, we will get stuck in an infinite loop, since the value of `i` would not change and it would be smaller than 10 forever. This is something to look out for in all loops!

The condition gets checked again. And this goes on until `i` reaches a value of 10. 10 is not smaller than 10, so the loop is done executing and the numbers 0 to 9 have been logged to the console.

We can also use a `for` loop to create a sequence and add values to an array, like this:

```
let arr = [];
for (let i = 0; i < 100; i++) {
  arr.push(i);
}
```

This is what the array looks like after this loop:

```
[
  0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
  12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23,
  24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35,
  36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47,
  48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59,
  60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71,
  72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83,
  84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95,
  96, 97, 98, 99
]
```

Since the loop ran the block of code 100 times, starting with an initial value of 0 for `i`, the block of code will add the incrementing value into the array at the end of the array. This results in an array that has a count of 0-99 and a length of 100 items. Since arrays start with an index value of zero, the values in the array will actually match up with the index values of the items in the array.

Or we could create an array containing only even values:

```
let arr = [];
for (let i = 0; i < 100; i = i + 2) {
  arr.push(i);
}
```

Resulting in this array:

```
[
  0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20,
  22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42,
  44, 46, 48, 50, 52, 54, 56, 58, 60, 62, 64,
  66, 68, 70, 72, 74, 76, 78, 80, 82, 84, 86,
  88, 90, 92, 94, 96, 98
]
```

Most commonly, you will see `i++` as the third part of the `for` loop, but please note that you can write any statement there. In this case, we are using `i = i + 2` to add 2 to the previous value every time, creating an array with only even numbers.

Practice exercise 5.3

In this exercise we will use a `for` loop to create an array that holds objects. Starting with creating a blank array, the block of code within the loop will create an object that gets inserted into the array.

1. Setup a blank array, `myWork`.
2. Using a `for` loop, create a list of 10 objects, each of which is a numbered lesson (e.g. Lesson 1, Lesson 2, Lesson 3...) with an alternating `true/false` status for every other item to indicate whether the class will be running this year. For example:

```
name: 'Lesson 1', status: true
```
3. You can specify the status by using a ternary operator that checks whether the modulo of the given lesson value is equal to zero and by setting up a Boolean value to alternate the values each iteration.
4. Create a lesson using a temporary object variable, containing the name (lesson with the numeric value) and predefined status (which we set up in the previous step).
5. Push the objects to the `myWork` array.
6. Output the array to the console.

Nested loops

Sometimes it can be necessary to use a loop inside a loop. A loop inside a loop is called a nested loop. Often it is not the best solution to the problem. It could even be a sign of poorly written code (sometimes called "code smell" among programmers), but every now and then it is a perfectly fine solution to a problem.

Here is what it would look like for `while` loops:

```
while (condition 1) {
  // code that gets executed as long as condition 1 is true
  // this loop depends on condition 1 being true
  while (condition 2) {
    // code that gets executed as long as condition 2 is true
  }
}
```

Nesting can also be used with `for` loops, or with a combination of both `for` and `while`, or even with all kinds of loops; they can go several levels deep.

An example in which we might use nested loops would be when we want to create an array of arrays. With the outer loop, we create the top-level array, and with the inner loop we add the values to the array.

```
let arrOfArrays = [];
for (let i = 0; i < 3; i++){
  arrOfArrays.push([]);
  for (let j = 0; j < 7; j++) {
    arrOfArrays[i].push(j);
  }
}
```

When we log this array like this:

```
console.log(arrOfArrays);
```

We can see that the output is an array of arrays with values from 0 up to 6.

```
[
  [
    0, 1, 2, 3, 4, 5, 6
  ],
  [
    0, 1, 2, 3, 4, 5, 6
  ],
  [
    0, 1, 2, 3, 4, 5, 6
  ]
]
```

We used the nested loops to create an array in an array, meaning we can work with rows and columns after having created this loop. This means nested loops can be used to create tabular data. We can show this output as a table using the `console.table()` method instead, like so:

```
console.table(arrOfArrays);
```

This will output:

(index)	0	1	2	3	4	5	6
0	0	1	2	3	4	5	6
1	0	1	2	3	4	5	6
2	0	1	2	3	4	5	6

Let's put this into practice in the next exercise.

Practice exercise 5.4

In this exercise we will be generating a table of values. We will be using loops to generate rows and also columns, which will be nested within the rows. Nested arrays can be used to represent rows in a table. This is a common structure in spreadsheets, where each row is a nested array within a table and the contents of these rows are the cells in the table. The columns will align as we are creating an equal number of cells in each row.

1. To create a table generator, first create an empty array, `myTable`, to hold your table data.
2. Set variable values for the number of rows and columns. This will allow us to dynamically control how many rows and columns we want within the table. Separating the values from the main code helps make updates to the dimensions easier.
3. Set up a counter variable with an initial value of 0. The counter will be used to set the content and count the values of the cells within the table.
4. Create a `for` loop with conditions to set the number of iterations, and to construct each row of the table. Within it, set up a new temporary array (`tempTable`) to hold each row of data. The columns will be nested within the rows, generating each cell needed for the column.
5. Nest a second loop within the first to count the columns. Columns are run within the row loop so that we have a uniform number of columns within the table.

6. Increment the main counter each iteration of the inner loop, so that we track a master count of each one of the cells and how many cells are created.
7. Push the counter values to the temporary array, `tempTable`. Since the array is a nested array representing a table, the values of the counter can also be used to illustrate the cell values next to each other in the table. Although these are separate arrays representing new rows, the value of the counter will help illustrate the overall sequence of cells in the final table.
8. Push the temporary array to the main table. As each iteration builds a new row of array items, this will continue to build the main table in the array.
9. Output into the console with `console.table(myTable)`. This will show you a visual representation of the table structure.

Loops and arrays

If you are not convinced of how extremely useful loops are by now, have a look at loops and arrays. Loops make life with arrays a lot more comfortable.

We can combine the `length` property and the condition part of the `for` loop or `while` loop to loop over arrays. It would look like this in the case of a `for` loop:

```
let arr = [some array];
for (initialize variable; variable smaller than arr.length; statement)
{
  // code to be executed
}
```

Let's start with a simple example that is going to log every value of the array:

```
let names = ["Chantal", "John", "Maxime", "Bobbi", "Jair"];
for (let i = 0; i < names.length; i++){
  console.log(names[i]);
}
```

This will output:

```
Chantal
John
Maxime
Bobbi
Jair
```

We use the `length` property to determine the maximum value of our index. The index starts counting at 0, but the length does not. The index is always one smaller than the length. Hence, we loop over the values of the array by increasing the length.

In this case we aren't doing very interesting things yet; we are simply printing the values. But we could be changing the values of the array in a loop, for example, like this:

```
let names = ["Chantal", "John", "Maxime", "Bobbi", "Jair"];
for (let i = 0; i < names.length; i++){
  names[i] = "hello " + names[i];
}
```

We have concatenated `hello` with the beginnings of our names. The array is changed in the loop and the array will have this content after the loop has executed:

```
[
  'hello Chantal',
  'hello John',
  'hello Maxime',
  'hello Bobbi',
  'hello Jair'
]
```

The possibilities are endless here. When an array comes in somewhere in the application, data can be sent to the database per value. Data can be modified by value, or even filtered, like this:

```
let names = ["Chantal", "John", "Maxime", "Bobbi", "Jair"];
for (let i = 0; i < names.length; i++){
  if(names[i].startsWith("M")){
    delete names[i];
    continue;
  }
  names[i] = "hello " + names[i];
}
console.log(names);
```

The `startsWith()` method just checks whether the string starts with a certain character. In this case it checks whether the name starts with the string `M`.



Don't worry, we will cover this function and many more in detail in *Chapter 8, Built-in JavaScript Methods*.

The output is:

```
[
  'hello Chantal',
  'hello John',
  <1 empty item>,
  'hello Bobbi',
  'hello Jair'
]
```

You'll have to be careful here though. If we were to remove the item instead of deleting it and leaving an empty value, we would accidentally skip the next value, since that value gets the index of the recently deleted one and `i` is incremented and moves on to the next index.

What do you think this one does:

```
let names = ["Chantal", "John", "Maxime", "Bobbi", "Jair"];
for (let i = 0; i < names.length; i++){
  names.push("...")
}
```

Your program gets stuck in an infinite loop here. Since a value gets added every iteration, the length of the loop grows with every iteration and `i` will never be bigger than or equal to `length`.

Practice exercise 5.5

Explore how to create a table grid that contains nested arrays as rows within a table. The rows will each contain the number of cells needed for the number of columns set in the variables. This grid table will dynamically adjust depending on the values for the variables.

1. Create a grid array variable.
2. Set a value of 64 for the number of cells.
3. Set a counter to 0.

4. Create a global variable to be used for the row array.
5. Create a loop that will iterate up to the number of cells you want in the array, plus one to include the zero value. In our example, we would use $64 \div 1$.
6. Add an outer `if` statement, which uses modulo to check if the main counter is divisible by 8 or whatever number of columns you want.
7. Inside the preceding `if` statement, add another `if` statement to check if the row is undefined, indicating whether it is the first run or whether the row is complete. If the row has been defined, then add the row to the main grid array.
8. To finish off the outer `if` statement, if the counter is divisible by 8, clear the row array – it has already been added to the grid by the inner `if` statement.
9. At the end of the `for` loop, increment of the main counter by 1.
10. Set up a temporary variable to hold the value of the counter and push it to the row array.
11. Within the loop iteration, check if the value of the counter is equal to the total number of columns you want; if it is, then add the current row to the grid.
12. Please note that the extra cell will not be added to the grid since there aren't enough cells to make a new row within the condition that adds the rows to the grid. An alternative solution would be to remove the `+1` from the loop condition and add `grid.push(row)` after the loop is completed, both of which will provide the same solution output.
13. Output the grid into the console.

for of loop

There is another loop we can use to iterate over the elements of an array: the **for of loop**. It cannot be used to change the value associated with the index as we can do with the regular loop, but for processing values it is a very nice and readable loop.

Here is what the syntax looks like:

```
let arr = [some array];
for (let variableName of arr) {
  // code to be executed
  // value of variableName gets updated every iteration
  // all values of the array will be variableName once
}
```

So you can read it like this: "For every value of the array, call it `variableName` and do the following." We can log our `names` array using this loop:

```
let names = ["Chantal", "John", "Maxime", "Bobbi", "Jair"];
for (let name of names){
  console.log(name);
}
```

We need to specify a temporary variable; in this case we called it `name`. This is used to put the value of the current iteration in, and after the iteration, it gets replaced with the next value. This code results in the following output:

```
Chantal
John
Maxime
Bobbi
Jair
```

There are some limitations here: we cannot modify the array, but we could write all the elements to a database or a file, or send it somewhere else. The advantage of this is that we cannot accidentally get stuck in an infinite loop or skip values.

Practice exercise 5.6

This exercise will construct an array as it loops through the incrementing values of `x`. Once the array is done, this exercise also will demonstrate several ways to output array contents.

1. Create an empty array
2. Run a loop 10 times, adding a new incrementing value to the array
3. Log the array into the console
4. Use the `for` loop to iterate through the array (adjust the number of iterations to however many values are in your array) and output into the console
5. Use the `for of` loop to output the value into the console from the array

Loops and objects

We have just seen how to loop over the values of an array, but we can also loop over the properties of an object. This can be helpful when we need to go over all the properties but don't know the exact properties of the object we are iterating over.

Looping over an object can be done in a few ways. We can use the `for in` loop to loop over the object directly, or we can convert the object to an array and loop over the array. We'll consider both in the following sections.

for in loop

Manipulating objects with loops can also be done with another variation of the `for` loop, the **for in loop**. The `for in` loop is somewhat similar to the `for of` loop. Again here, we need to specify a temporary name, also referred to as a key, to store each property name in. We can see it in action here:

```
let car = {
  model: "Golf",
  make: "Volkswagen",
  year: 1999,
  color: "black",
};

for (let prop in car){
  console.log(car[prop]);
}
```

We need to use the `prop` of each loop iteration to get the value out of the `car` object. The output then becomes:

```
Golf
Volkswagen
1999
black
```

If we just logged the `prop`, like this:

```
for (let prop in car){
  console.log(prop);
}
```

This is what our output would look like:

```
model
make
year
color
```

As you can see, all the names of the properties get printed, and not the values. This is because the `for in` loop is getting the property names (keys) and not the values. The `for of` is doing the opposite; it is getting the values and not the keys.

This `for in` loop can also be used on arrays, but it is not really useful. It will only return the indices, since these are the "keys" of the values of the arrays. Also, it should be noted that the order of execution cannot be guaranteed, even though this is usually important for arrays. It is therefore better to use the approaches mentioned in the section on loops and arrays.

Practice exercise 5.7

In this exercise, we will experiment with looping over objects and internal arrays.

1. Create a simple object with three items in it.
2. Using the `for in` loop, get the properties' names and values from the object and output them into the console.
3. Create an array containing the same three items. Using either the `for` loop or the `for in` loop, output the values from the array into the console.

Looping over objects by converting to an array

You can use any loop on objects, as soon as you convert the object to an array. This can be done in three ways:

- Convert the keys of the object to an array
- Convert the values of the object to an array
- Convert the key-value entries to an array (containing arrays with two elements: object key and object value)

Let's use this example:

```
let car = {
  model: "Golf",
  make: "Volkswagen",
  year: 1999,
  color: "black",
};
```

If we want to loop over the keys of the object, we can use the `for...in` loop, as we saw in the previous section, but we can also use the `for...of` loop if we convert it to an array first. We do so by using the `Object.keys(nameOfObject)` built-in function. This takes an object and grabs all the properties of this object and converts them to an array.

To demonstrate how this works:

```
let arrKeys = Object.keys(car);
console.log(arrKeys);
```

This will output:

```
[ 'model', 'make', 'year', 'color' ]
```

We can loop over the properties of this array like this using the `for...of` loop:

```
for(let key of Object.keys(car)) {
  console.log(key);
}
```

And this is what it will output:

```
model
make
year
color
```

Similarly, we can use the `for...of` loop to loop over the values of the object by converting the values to an array. The main difference here is that we use `Object.values(nameOfObject)`:

```
for(let key of Object.values(car)) {
  console.log(key);
}
```

You can loop over these arrays in the same way you loop over any array. You can use the length and index strategy like this in a regular `for` loop:

```
let arrKeys = Object.keys(car);
for(let i = 0; i < arrKeys.length; i++) {
  console.log(arrKeys[i] + ": " + car[arrKeys[i]]);
}
```

And this will output:

```
model: Golf
make: Volkswagen
year: 1999
color: black
```

More interesting is how to loop over both arrays at the same time using the `for...of` loop. In order to do so, we will have to use `Object.entries()`. Let's demonstrate what it does:

```
let arrEntries = Object.entries(car);
console.log(arrEntries);
```

This will output:

```
[
  [ 'model', 'Golf' ],
  [ 'make', 'Volkswagen' ],
  [ 'year', 1999 ],
  [ 'color', 'black' ]
]
```

As you can see, it is returning a two-dimensional array, containing key-value pairs. We can loop over it like this:

```
for (const [key, value] of Object.entries(car)) {
  console.log(key, ":", value);
}
```

And this will output:

```
model : Golf
make : Volkswagen
year : 1999
color : black
```

Alright, you have seen many ways to loop over objects now. Most of them come down to converting the object to an array. We can imagine that at this point you could use a `break`. Or maybe you'd just like to `continue`?

break and continue

break and **continue** are two keywords that we can use to control the flow of execution of the loop. **break** will stop the loop and move on to the code below the loop. **continue** will stop the current iteration and move back to the top of the loop, checking the condition (or in the case of a **for** loop, performing the statement and then checking the condition).

We will be using this array of car objects to demonstrate **break** and **continue**:

```
let cars = [
  {
    model: "Golf",
    make: "Volkswagen",
    year: 1999,
    color: "black",
  },
  {
    model: "Picanto",
    make: "Kia",
    year: 2020,
    color: "red",
  },
  {
    model: "Peugeot",
    make: "208",
    year: 2021,
    color: "black",
  },
  {
    model: "Fiat",
    make: "Punto",
    year: 2020,
    color: "black",
  }
];
```

We will first have a closer look at **break**.

break

We have already seen **break** in the **switch** statement. When **break** was executed, the **switch** statement ended. This is not very different when it comes to loops: when the **break** statement is executed, the loop will end, even when the condition is still true.

Here is a silly example to demonstrate how **break** works:

```
for (let i = 0; i < 10; i++) {
  console.log(i);
  if (i === 4) {
    break;
  }
}
```

It looks like a loop that will log the numbers 0 to 10 (again excluding 10) to the console. There is a catch here though: as soon as *i* equals 4, we execute the **break** command. **break** ends the loop immediately, so no more loop code gets executed afterward.

We can also use **break** to stop looping through the array of cars when we have found a car that matches our demands.

```
for (let i = 0; i < cars.length; i++) {
  if (cars[i].year >= 2020) {
    if (cars[i].color === "black") {
      console.log("I have found my new car:", cars[i]);
      break;
    }
  }
}
```

As soon as we run into a car with the year 2020 or later and the car is black, we will stop looking for other cars and just buy that one. The last car in the array would also have been an option, but we did not even consider it because we found one already. The code snippet will output this:

```
I have found my new car: { model: 'Peugeot', make: '208', year: 2021,
color: 'black' }
```

However, often it is not a best practice to use **break**. If you can manage to work with the condition of the loop to break out of the loop instead, this is a much better practice. It prevents you getting stuck in an infinite loop, and the code is easier to read.

If the condition of the loop is not an actual condition, but pretty much a run-forever kind of statement, the code gets hard to read.

Consider the following code snippet:

```
while (true) {
  if (superLongArray[0] !== 42 && superLongArray.length > 0) {
    superLongArray.shift();
  } else {
    console.log("Found 42!");
    break;
  }
}
```

This would be better to write without break and without something terrible like `while(true)`; you could do it like this:

```
while (superLongArray.length > 0 && notFound) {
  if (superLongArray[0] !== 42) {
    superLongArray.shift();
  } else {
    console.log("Found 42!");
    notFound = false;
  }
}
```

With the second example, we can see the conditions of the loop easily, namely the length of the array and a `notFound` flag. However, with `while(true)` we are kind of misusing the while concept. You want to specify the condition, and it should evaluate to `true` or `false`; this way your code is nice to read. If you say `while(true)`, you're actually saying forever, and the reader of your code will have to interpret it line by line to see what is going on and when the loop is ended by a workaround break statement.

continue

break can be used to quit the loop, and **continue** can be used to move on to the next iteration of the loop. It quits the current iteration and moves back up to check the condition and start a new iteration.

Here you can see an example of continue:

```
for (let car of cars){
  if(car.color !== "black"){
```

```
    continue;
  }
  if (car.year >= 2020) {
    console.log("we could get this one:", car);
  }
}
```

The approach here is to just skip every car that is not black and consider all the others that are not older than make year 2020 or later. The code will output this:

```
we could get this one: { model: 'Peugeot', make: '200', year: 2021,
color: 'black' }
we could get this one: { model: 'Fiat', make: 'Punto', year: 2020,
color: 'black' }
```

Be careful with `continue` in a while loop. Without running it, what do you think the next code snippet does?

```
// Let's only log the odd numbers to the console
let i = 1;
while (i < 50) {
  if (i % 2 === 0){
    continue;
  }
  console.log(i);
  i++;
}
```

It logs 1, and then it gets you stuck in an infinite loop, because `continue` gets hit before the value of `i` changes, so it will run into `continue` again, and again, and so on. This can be fixed by moving the `i++` up and subtracting 1 from `i`, like this:

```
let i = 1;
while (i < 50) {
  i++;
  if ((i-1) % 2 === 0){
    continue;
  }
  console.log(i-1);
}
```

But again, there is a better way without `continue` here. The chance of error is a lot smaller:

```
for (let i = 1; i < 50; i = i + 2) {
  console.log(i);
}
```

And as you can see it is even shorter and more readable. The value of `break` and `continue` usually comes in when you are looping over large data sets, possibly coming from outside your application. Here you'll have less influence to apply other types of control. Using `break` and `continue` is not a best practice for simple basic examples, but it's a great way to get familiar with the concepts.

Practice exercise 5.8

This exercise will demonstrate how to create a string with all the digits as it loops through them. We can also set a value to skip by adding a condition that will use `continue`, skipping the matching condition. A second option is to do the same exercise and use the `break` keyword.

1. Set up a string variable to use as output.
2. Select a number to skip, and set that number as a variable.
3. Create a `for` loop that counts to 10.
4. Add a condition to check if the value of the looped variable is equal to the number that should be skipped.
5. If the number is to be skipped in the condition, `continue` to the next number.
6. As you iterate through the values, append the new count value to the end of the main output variable.
7. Output the main variable after the loop completes.
8. Reuse the code, but change the `continue` to `break` and see the difference. It should now stop at the skip value.

break, continue, and nested loops

`break` and `continue` can be used in nested loops as well, but it is important to know that when `break` or `continue` is used in a nested loop, the outer loop will not break.

We will use this array of arrays to discuss `break` and `continue` in nested loops:

```
let groups = [
  ["Martin", "Daniel", "Keith"],
  ["Margot", "Marina", "Ali"],
  ["Helen", "Jonah", "Sambikos"],
];
```

Let's break down this example. We are looking for all the groups that have two names starting with an `M`. If we find such a group, we will log it.

```
for (let i = 0; i < groups.length; i++) {
  let matches = 0;

  for (let j = 0; j < groups[i].length; j++) {
    if (groups[i][j].startsWith("M")) {
      matches++;
    } else {
      continue;
    }
    if (matches === 2) {
      console.log("Found a group with two names starting with an M:");
      console.log(groups[i]);
      break;
    }
  }
}
```

We first loop over the top-level arrays and set a counter, `matches`, with a start value of `0`, and for each of these top-level arrays, we are going to loop over the values. When a value starts with an `M`, we increase `matches` by one and check whether we have found two matches already. If we find two `M`s, we break out of the inner loop and continue in our outer loop. This one will move on to the next top-level array, since nothing is happening after the inner loop.

If the name does not start with an `M`, we do not need to check for `matches` being `2`, and we can continue to the next value in the inner array.

Take a look at this example: what do you think it will log?

```
for (let group of groups) {
  for (let member of group) {
    if (member.startsWith("M")) {
      console.log("Found one starting with M:", member);
    }
  }
}
```

```

        break;
    }
}

```

It will loop over the arrays, and for every array it will check the value to see if it starts with an M. If it does, the inner loop will break. So, if one of the arrays in the array contains multiple values starting with M, only the first one will be found, since the iteration over that array breaks and we continue to the next array.

This one will output:

```

found one starting with M: Martin
found one starting with M: Margot

```

We can see that it finds Margot, the first one from the second array, but it skips Marina, because it is the second one in the array. And it breaks after having found one group, so it won't loop over the other elements in the inner array. It will continue with the next array, which doesn't contain names starting with an M.

If we wanted to find groups that have a member with a name that starts with an M, the previous code snippet would have been the way to go, because we are breaking the inner loop as soon as we find a hit. This can be useful whenever you want to make sure that an array in a data set contains at least one of something. Because of the nature of the `for...of` loop, it won't give the index or place where it found it. It will simply break, and you have the value of the element of the array to use. If you need to know more, you can work with counters, which are updated every iteration.

If we want to see whether only one of all the names in the array of arrays starts with an M, we would have to break out of the outer loop. This is something we can do with labeled loops.

break and continue and labeled blocks

We can break out of the outer loop from inside the inner loop, but only if we give a label to our loop. This can be done like this:

```

outer:
for (let group of groups) {
  inner:
  for (let member of group) {
    if (member.startsWith("M")) {
      console.log("found one starting with M:", member);
    }
  }
}

```

```

        break outer;
    }
}

```

We are giving our block a label by putting a word and a colon in front of a code block. These words can be pretty much anything (in our case, "outer" and "inner"), but not JavaScript's own reserved words, such as `for`, `if`, `break`, `else`, and others.

This will only log the first name starting with an M:

```

found one starting with M: Martin

```

It will only log one, because it is breaking out of the outer loop and all the loops end as soon as they find one. In a similar fashion you can continue the outer loop as well.

Whenever you want to be done as soon as you find one hit, this is the option to use. So, for example, if you want check for errors and quit if there aren't any, this would be the way to go.

Chapter project

Math multiplication table

In this project, you will create a math multiplication table using loops. You can do this using your own creativity or by following some of the following suggested steps:

1. Set up a blank array to contain the final multiplication table.
2. Set a value variable to specify how many values you want to multiply with one another and show the results for.
3. Create an outer `for` loop to iterate through each row and a `temp` array to store the row values. Each row will be an array of cells that will be nested into the final table.
4. Add an inner `for` loop for the column values, which will push the multiplied row and column values to the `temp` array.
5. Add the temporary row data that contains the calculated solutions to the main array of the final table. The final result will add a row of values for the calculations.

Self-check quiz

1. What is the expected output for the following code?

```
let step = 3;

for (let i = 0; i < 1000; i += step) {
  if (i > 10) {
    break;
  }
  console.log(i);
}
```

2. What is the final value for `myArray`, and what is expected in the console?

```
const myArray = [1,5,7];
for(e1 in myArray){
  console.log(Number(e1));
  e1 = Number(e1) + 5;
  console.log(e1);
}
console.log(myArray);
```

Summary

In this chapter we introduced the concept of loops. Loops enable us to repeat a certain block of code. We need some sort of condition when we loop, and as long as that condition is true, we'll keep looping. As soon as it changes to false, we end our loop.

We have seen the `while` loop, in which we just insert a condition, and as long as that condition is true we keep looping. If the condition is never true, we won't even execute the loop code once.

This is different for the `do while` loop. We always execute the code once, and then we start to check a condition. If this condition is true, we execute the code again and do so until the condition becomes false. This can be useful when working with input from outside, such as user input. We would need to request it once, and then we can keep on requesting it again until it is valid.

Then we saw the `for` loop, which has a slightly different syntax. We have to specify a variable, check a condition (preferably using that variable, but this is not mandatory), and then specify an action to be executed after every iteration. Again, it's preferable for the action to include the variable from the first part of the `for` loop. This gives us code that is to be executed as long as a condition is true.

We also saw two ways to loop over arrays and objects, `for in` and `for of`. The `for in` loop loops over keys and `for of` loops over values. They go over every element in a collection. The advantage of these loops is that JavaScript controls the execution: you can't miss an element or get stuck in an infinite loop.

Lastly, we saw `break` and `continue`. We can use the `break` keyword to end a loop immediately and the `continue` keyword to end the current iteration and go back to the top and start the next iteration, if the condition is still true, that is.

In the next chapter we are going to be adding a really powerful tool to our JavaScript toolbox: functions! They allow us to take our coding skills to the next level and structure our code better.

6

Functions

You have seen quite a lot of JavaScript already, and now you are ready for functions. Soon you will see that you have been using functions already, but now it is time to learn how to start writing your own. Functions are a great building block that will reduce the amount of code you will need in your app. You can call a function whenever you need it, and you can write it as a kind of template with variables. So, depending on how you've written it, you can reuse it in many situations.

They do require you to think differently about the structure of your code and this can be hard, especially in the beginning. Once you have got the hang of this way of thinking, functions will really help you to write nicely structured, reusable, and low-maintenance code. Let's dive into this new abstraction layer!

Along the way, we will cover the following topics:

- Basic functions
- Function arguments
- Return
- Variable scope in functions
- Recursive functions
- Nested functions
- Anonymous functions
- Function callbacks



Note: exercise, project and self-check quiz answers can be found in the *Appendix*.

Basic functions

We have been calling functions for a while already. Remember `prompt()`, `console.log()`, `push()`, and `sort()` for arrays? These are all functions. Functions are a group of statements, variable declarations, loops, and so on that are bundled together. Calling a function means an entire group of statements will get executed.

First, we are going to have a look at how we can invoke functions, and then we will see how to write functions of our own.

Invoking functions

We can recognize functions by the parentheses at the end. We can invoke functions like this:

```
nameOfTheFunction();  
functionThatTakesInput("the input", 5, true);
```

This is invoking a function called `nameOfTheFunction` with no arguments, and a function called `functionThatTakesInput` with three required arguments. Let's have a look at what functions can look like when we start writing them.

Writing functions

Writing a function can be done using the `function` keyword. Here is the template syntax to do so:

```
function nameOfTheFunction() {  
  //content of the function  
}
```

The above function can be called like this:

```
nameOfTheFunction();
```

Let's write a function that asks for your name and then greets you:

```
function sayHello() {  
  let you = prompt("What's your name? ");  
  console.log("Hello", you + "!",);  
}
```


We add a space after the question mark to ensure the user starts typing their answer one space away from the question mark, rather than directly afterward. We call this function like this:

```
sayHello();
```

It will prompt:

```
What's your name? >
```

Let's go ahead and enter our name. The output will be:

```
Hello Maaike!
```

Take a moment to consider the relationship between functions and variables. As you have seen, functions can contain variables, which shape how they operate. The opposite is also true: variables can contain functions. Still with me? Here you can see an example of a variable containing a function (`varContainingFunction`) and a variable inside a function (`varInFunction`):

```
let varContainingFunction = function() {
  let varInFunction = "I'm in a function.";
  console.log("hi there!", varInFunction);
};

varContainingFunction();
```

Variables contain a certain value and *are* something; they do not *do* anything. Functions are actions. They are a bundle of statements that can be executed when they get called. JavaScript will not run the statements when the functions do not get invoked. We will return to the idea of storing functions in variables, and consider some of the benefits, in the *Anonymous functions* section, but for now let's move on to look at the best way to name your functions.

Naming functions

Giving your function a name might seem like a trivial task, but there are some best practices to keep in mind here. To keep it short:

- Use camelCase for your functions: this means that the first word starts with a lowercase letter and new words start with a capital. That makes it a lot easier to read and keeps your code consistent.
- Make sure that the name describes what the function is doing: it's better to call a number addition function `addNumbers` than `myFunc`.

- Use a verb to describe what the function is doing: make it an action. So instead of `hiThere`, call it `sayHi`.

Practice exercise 6.1

See if you can write a function for yourself. We want to write a function that adds two numbers.

1. Create a function that takes two parameters, adds the parameters together, and returns the result.
2. Set up two different variables with two different values.
3. Use your function on the two variables, and output the result using `console.log`.
4. Create a second call to the function using two more numbers as arguments sent to the function.

Practice exercise 6.2

We are going to create a program that will randomly describe an inputted name.

1. Create an array of descriptive words.
2. Create a function that contains a prompt asking the user for a name.
3. Select a random value from the array using `Math.random`.
4. Output into the console the prompt value and the randomly selected array value.
5. Invoke the function.

Parameters and arguments

You may have noticed that we are talking about parameters and arguments. Both terms are commonly used to mean the information that is passed into a function:

```
function tester(para1, para2){
  return para1 + " " + para2;
}

const arg1 = "argument 1";
const arg2 = "argument 2";
tester(arg1, arg2);
```

A parameter is defined as the variable listed inside the parentheses of the function definition, which defines the scope of the function. They are declared like so:

```
function myFunc(param1, param2) {
  // code of the function;
}
```

A practical example could be the following, which takes *x* and *y* as parameters:

```
function addTwoNumbers(x, y) {
  console.log(x + y);
}
```

When called, this function will simply add the parameters and log the result. However, to do this, we can call the function with arguments:

```
myFunc("arg1", "arg2");
```

We have seen various examples of arguments; for example:

```
console.log("this is an argument");
prompt("argument here too");

let arr = [];
arr.push("argument");
```

Depending on the arguments you are calling with the function, the outcome of the function can change, which makes the function a very powerful and flexible building block. A practical example using our `addTwoNumbers()` function looks like this:

```
addTwoNumbers(3, 4);
addTwoNumbers(12, -90);
```

This will output:

```
7
-78
```

As you can see, the function has a different outcome for both calls. This is because we call it with different arguments, which take the place of *x* and *y*, that are sent to the function to be used within the function scope.

Practice exercise 6.3

Create a basic calculator that takes two numbers and one string value indicating an operation. If the operation equals `add`, the two numbers should be added. If the operation equals `subtract`, the two numbers should be subtracted from one another. If there is no option specified, the value of the option should be `add`.

The result of this function needs to be logged. Test your function by invoking it with different operators and no operator specified.

1. Set up two variables containing number values.
2. Set up a variable to hold an operator, either `+` or `-`.
3. Create a function that retrieves the two values and the operator string value within its parameters. Use those values with a condition to check if the operator is `+` or `-`, and add or subtract the values accordingly (remember if not presented with a valid operator, the function should default to addition).
4. Within `console.log()`, call the function using your variables and output the response to the console.
5. Update the operator value to be the other operator type—either plus or minus—and call to the function again with the new updated arguments.

Default or unsuitable parameters

What happens if we call our `addTwoNumbers()` function without any arguments? Take a moment and decide what you think this should do:

```
addTwoNumbers();
```

Some languages might crash and cry, but not JavaScript. JavaScript just gives the variables a default type, which is `undefined`. And `undefined + undefined` equals:

```
NaN
```

Instead, we could tell JavaScript to take different default parameters. And that can be done like this:

```
function addTwoNumbers(x = 2, y = 3) {
  console.log(x + y);
}
```

If you call the function with no arguments now, it will automatically assign 2 to `x` and 3 to `y`, unless you override them by calling the function with arguments. The values that are used for invoking are prioritized over hardcoded arguments. So, given the above function, what will the output of these function calls be?

```
addTwoNumbers();
addTwoNumbers(6, 6);
addTwoNumbers(10);
```

The output will be:

```
5
12
13
```

The first one has the default values, so `x` is 2 and `y` is 3. The second one assigns 6 to both `x` and `y`. The last one is a bit less obvious. We are only giving one argument, so which one will be given this value? Well, JavaScript does not like to overcomplicate things. It simply assigns the value to the first parameter, `x`. Therefore, `x` becomes 10 and `y` gets its default value 3, and together that makes 13.

If you call a function with more arguments than parameters, nothing will happen. JavaScript will just execute the function using the first arguments that can be mapped to parameters. Like this:

```
addTwoNumbers(1,2,3,4);
```

This will output:

```
3
```

It is just adding 1 and 2 and ignoring the last two arguments (3 and 4).

Special functions and operators

There are a few special ways of writing functions, as well as some special operators that will come in handy. We are talking about arrow functions and the spread and rest operators here. Arrow functions are great for sending functions around as parameters and using shorter notations. The spread and rest operators make our lives easier and are more flexible when sending arguments and working with arrays.

Arrow functions

Arrow functions are a special way of writing functions that can be confusing at first. Their use looks like this:

```
(param1, param2) => body of the function;
```

Or for no parameters:

```
() => body of the function;
```

Or for one parameter (no parentheses are needed here):

```
param => body of the function;
```

Or for a multiline function with two parameters:

```
(param1, param2) => {
  // Line 1;
  // any number of lines;
};
```

Arrow functions are useful whenever you want to write an implementation on the spot, such as inside another function as an argument. This is because they are a shorthand notation for writing functions. They are most often used for functions that consist of only one statement. Let's start with a simple function that we will rewrite to an arrow function:

```
function doingStuff(x) {
  console.log(x);
}
```

To rewrite this as an arrow function, you will have to store it in a variable or send it in as an argument if you want to be able to use it. We use the name of the variable to execute the arrow function. In this case we only have one parameter, so it's optional to surround it with parentheses. We can write it like this:

```
let doingArrowStuff = x => console.log(x);
```

And invoke it like this:

```
doingArrowStuff("Great!");
```

This will log Great! to the console. If there is more than one argument, we will have to use parentheses, like this:

```
let addTwoNumbers = (x, y) => console.log(x + y);
```

We can call it like this:

```
addTwoNumbers(5, 3);
```

And then it will log 8 to the console. If there are no arguments, you must use the parentheses, like this:

```
let sayHi = () => console.log("hi");
```

If we call sayHi(), it will log hi to the console.

As a final example, we can combine the arrow function with certain built-in methods. For example, we can use the forEach() method on an array. This method executes a certain function for every element in the array. Have a look at this example:

```
const arr = ["squirrel", "alpaca", "buddy"];
arr.forEach(e => console.log(e));
```

It outputs:

```
squirrel
alpaca
buddy
```

For every element in the array, it takes the element as input and executing the arrow function for it. In this case, the function is to log the element. So the output is every single element in the array.

Using arrow functions combined with built-in functions is very powerful. We can do something for every element in the array, without counting or writing a complicated loop. We'll see more examples of great use cases for arrow functions later on.

Spread operator

The spread operator is a special operator. It consists of three dots used before a referenced expression or string, and it spreads out the arguments or elements of an array.

This might sound very complicated, so let's look at a simple example:

```
let spread = ["so", "much", "fun"];
let message = ["JavaScript", "is", ...spread, "and", "very",
               "powerful"];
```

The value of this array becomes:

```
['JavaScript', 'is', 'so', 'much', 'fun', 'and', 'very', 'powerful']
```

As you can see, the elements of the spread operator become individual elements in the array. The spread operator spreads the array to individual elements in the new array. It can also be used to send multiple arguments to a function, like this:

```
function addTwoNumbers(x, y) {
  console.log(x + y);
}
let arr = [5, 9];
addTwoNumbers(...arr);
```

This will log 14 to the console, since it is the same as calling the function with:

```
addTwoNumbers(5, 9);
```

This operator avoids having to copy a long array or string into a function, which saves time and reduces code complexity. You can call a function with multiple spread operators. It will use all the elements of the arrays as input. Here's an example:

```
function addFourNumbers(x, y, z, a) {
  console.log(x + y + z + a);
}
let arr = [5, 9];
let arr2 = [6, 7];
addFourNumbers(...arr, ...arr2);
```

This will output 27 to the console, calling the function like this:

```
addFourNumbers(5, 9, 6, 7);
```

Rest parameter

Similar to the spread operator, we have the rest parameter. It has the same symbol as the spread operator, but it is used inside the function parameter list. Remember what would happen if we were to send an argument too many times, as here:

```
function someFunction(param1, param2) {
  console.log(param1, param2);
}
someFunction("hi", "there!", "How are you?");
```

That's right. Nothing really: it would just pretend we only sent in two arguments and log `hi there!`. If we use the rest parameter, it allows us to send in any number of arguments and translate them into a parameter array. Here is an example:

```
function someFunction(param1, ...param2) {
  console.log(param1, param2);
}
someFunction("hi", "there!", "How are you?");
```

This will log:

```
hi [ 'there!', 'How are you?' ]
```

As you can see, the second parameter has changed into an array, containing our second and third arguments. This can be useful whenever you are not sure what number of arguments you will get. Using the rest parameter allows you to process this variable number of arguments, for example, using a loop.

Returning function values

We are still missing a very important piece to make functions as useful as they are: the return value. Functions can give back a result when we specify a return value. The return value can be stored in a variable. We have done this already – remember `prompt()`?

```
let favoriteSubject = prompt("What is your favorite subject?");
```

We are storing the result of our `prompt()` function in the variable `favoriteSubject`, which in this case would be whatever the user specifies. Let's see what happens if we store the result of our `addTwoNumbers()` function and log that variable:

```
let result = addTwoNumbers(4, 5);
console.log(result);
```

You may or may not have guessed it – this logs the following:

```
9
undefined
```

The value 9 is written to the console because `addTwoNumbers()` contains a `console.log()` statement. The `console.log(result)` line outputs `undefined`, because nothing is inserted into the function to store the result, meaning our function `addTwoNumbers()` does not send anything back. Since JavaScript does not like to cause trouble and crash, it will assign `undefined`. To counter this, we can rewrite our `addTwoNumbers()` function to actually return the value instead of logging it. This is much more powerful because we can store the result and continue working with the result of this function in the rest of our code:

```
function addTwoNumbers(x, y) {
  return x + y;
}
```

`return` ends the function and sends back whatever value comes after `return`. If it is an expression, like the one above, it will evaluate the expression to one result and then return that to where it was called (the result variable, in this instance):

```
let result = addTwoNumbers(4, 5);
console.log(result);
```

With these adjustments made, the code snippet logs 9 to the terminal.

What do you think this code does?

```
let resultsArr = [];

for(let i = 0; i < 10; i++){
  let result = addTwoNumbers(i, 2*i);
  resultsArr.push(result);
}

console.log(resultsArr);
```

It logs an array of all the results to the screen. The function is being called in a loop. The first iteration, `i`, equals 0. Therefore, the result is 0. The last iteration, `i`, equals 9, and therefore the last value of the array equals 27. Here are the results:

```
[
  0, 3, 6, 9, 12,
  15, 18, 21, 24, 27
]
```

Practice exercise 6.4

Modify the calculator that you made in *Practice exercise 6.2* to return added values instead of printing them. Then, call the function 10 or more times in a loop, and store the results in an array. Once the loop finishes, output the final array into the console.

1. Set up an empty array to store the values that will be calculated within the loop.
2. Create a loop that runs 10 times, incrementing by 1 each time, creating two values each iteration. For the first value, multiply the value of the loop count by 5. For the second value, multiply the value of the loop counter by itself.
3. Create a function that returns the value of the two parameters passed into the function when it is called. Add the values together, returning the result.
4. Within the loop, call the calculation function, passing in the two values as arguments into the function and storing the returned result in a response variable.
5. Still within the loop, push the result values into the array as it iterates through the loop.
6. After the loop is complete, output the value of the array into the console.
7. You should see the values [0, 6, 14, 24, 36, 50, 66, 84, 104, 126] for the array in the console.

Returning with arrow functions

If we have a one-line arrow function, we can return without using the keyword `return`. So if we want to rewrite the function, we can write it like this to make an arrow function out of it:

```
let addTwoNumbers = (x, y) => x + y;
```

And we can call it and store the result like this:

```
let result = addTwoNumbers(12, 15);
console.log(result);
```

This will then log 27 to the console. If it's a multiline function, you will have to use the keyword `return` as demonstrated in the previous section. So, for example:

```
let addTwoNumbers = (x, y) => {
  console.log("Adding...");
  return x + y;
}
```

Variable scope in functions

In this section, we will discuss a topic that is often considered challenging. We will talk about scope. Scope defines where you can access a certain variable. When a variable is *in scope*, you can access it. When a variable is *out of scope*, you cannot access the variable. We will discuss this for both local and global variables.

Local variables in functions

Local variables are only in scope within the function they are defined. This is true for `let` variables and `var` variables. There is a difference between them, which we will touch upon here as well. The function parameters (they do not use `let` or `var`) are also local variables. This might sound very vague, but the next code snippet will demonstrate what this means:

```
function testAvailability(x) {
  console.log("Available here:", x);
}

testAvailability("Hi!");
console.log("Not available here:", x);
```

This will output:

```
Available here: Hi!
ReferenceError: x is not defined
```

When called inside the function, `x` will be logged. The statement outside of the function fails, because `x` is a local variable to the function `testAvailability()`. This is showing that the function parameters are not accessible outside of the function.

They are out of scope outside the function and in scope inside the function. Let's have a look at a variable defined inside a function:

```
function testAvailability() {
  let y = "Local variable!";
  console.log("Available here:", y);
}

testAvailability();
console.log("Not available here:", y);
```

This shows the following on the console:

```
Available here: Local variable!
ReferenceError: y is not defined
```

Variables defined inside the function are not available outside the function either.

For beginners, it can be confusing to combine local variables and `return`. Right now, we're telling you the local variables declared inside a function are not available outside of the function, but with `return` you can make their values available outside the function. So if you need their values outside a function, you can return the values. The key word here is *values*! You cannot return the variable itself. Instead, a value can be caught and stored in a different variable, like this:

```
function testAvailability() {
  let y = "I'll return";
  console.log("Available here:", y);
  return y;
}

let z = testAvailability();
console.log("Outside the function:", z);
console.log("Not available here:", y);
```

So, the returned value `I'll return` that was assigned to local variable `y` gets returned and stored in variable `z`.



This variable `z` could actually also have been called `y`, but that would have been confusing since it still would have been a different variable.

The output of this code snippet is as follows:

```
Available here: I'll return
Outside the function: I'll return
ReferenceError: y is not defined
```

let versus var variables

The difference between `let` and `var` is that `var` is function-scoped, which is the concept we described above. `let` is actually not function-scoped but block-scoped. A block is defined by two curly braces `{ }`. The code within those braces is where `let` is still available.

Let's see this distinction in action:

```
function doingStuff() {
  if (true) {
    var x = "local";
  }
  console.log(x);
}

doingStuff();
```

The output of this snippet will be:

```
local
```

If we use `var`, the variable becomes function-scoped and is available anywhere in the function block (even before defining with the value `undefined`). Thus, after the `if` block has ended, `x` can still be accessed.

Here is what happens with `let`:

```
function doingStuff() {
  if (true) {
    let x = "local";
  }
  console.log(x);
}

doingStuff();
```

This will produce the following output:

```
ReferenceError: x is not defined
```

Here we get the error that `x` is not defined. Since `let` is only block-scoped, `x` goes out of scope when the `if` block ends and can no longer be accessed after that.

A final difference between `let` and `var` relates to the order of declaration in a script. Try using the value of `x` before having defined it with `let`:

```
function doingStuff() {
  if (true) {
    console.log(x);
    let x = "local";
  }
}

doingStuff();
```

This will give a `ReferenceError` that `x` is not initialized. This is because variables declared with `let` cannot be accessed before being defined, even within the same block. What do you think will happen for a `var` declaration like this?

```
function doingStuff() {
  if (true) {
    console.log(x);
    var x = "local";
  }
}

doingStuff();
```

This time, we won't get an error. When we use a `var` variable before the `define` statement, we simply get `undefined`. This is due to a phenomenon called *hoisting*, which means using a `var` variable before it's been declared results in the variable being `undefined` rather than giving a `ReferenceError`.



Hoisting, and how to negate its effects if needed, are more complex topics that we will cover in *Chapter 12, Intermediate JavaScript*.

const scope

Constants are block-scoped, just like `let`. This is why the scope rules here are similar to those for `let`. Here is an example:

```
function doingStuff() {
  if (true) {
```

```
const X = "local";
}
console.log(X);
}

doingStuff();
```

This will produce the following output:

```
ReferenceError: X is not defined
```

Using a `const` variable before having defined it will also give a `ReferenceError`, just as it does for a `let` variable.

Global variables

As you might have guessed, global variables are variables declared outside a function and not in some other code block. Variables are accessible in the scope (either function or block) where they're defined, plus any "lower" scopes. So, a variable defined outside of a function is available within the function as well as inside any functions or other code blocks inside that function. A variable defined at the top level of your program is therefore available everywhere in your program. This concept is called a global variable. You can see an example here:

```
let globalVar = "Accessible everywhere!";
console.log("Outside function:", globalVar);

function creatingNewScope(x) {
  console.log("Access to global vars inside function.", globalVar);
}

creatingNewScope("some parameter");

console.log("Still available:", globalVar);
```

This will output:

```
Outside function: Accessible everywhere!
Access to global vars inside function. Accessible everywhere!
Still available: Accessible everywhere!
```


As you can see, global variables are accessible from everywhere because they are not declared in a block. They are *always* in scope after they have been defined – it doesn't matter where you use them. However, you can hide their accessibility inside a function by specifying a new variable with the same name inside that scope; this can be done for `let`, `var`, and `const`. (This is not changing the value of the `const` variable; you are creating a new `const` variable that is going to override the first one in the inner scope.) In the same scope, you cannot specify two `let` or two `const` variables with the same name. You can do so for `var`, but you shouldn't do so, in order to avoid confusion.

If you create a variable with the same name inside a function, that variable's value will be used whenever you refer to that variable name within the scope of that particular function. Here you can see an example:

```
let x = "global";

function doingStuff() {
  let x = "local";
  console.log(x);
}

doingStuff();
console.log(x);
```

This will output:

```
local
global
```

As you can see, the value of `x` inside the `doingStuff()` function is `local`. However, outside the function the value is still `global`. This means that you'll have to be extra careful about mixing up names in local and global scopes. It is usually better to avoid this.

The same is also true for parameter names. If you have the same parameter name as a global variable, the value of the parameter will be used:

```
let x = "global";

function doingStuff(x) {
  console.log(x);
}

doingStuff("param");
```

This will log `param`.

There is a danger in relying on global variables too much. This is something you will come across soon when your applications grow. As we just saw, local variables override the value of global variables. It is best to work with local variables in functions; this way, you have more control over what you are working with. This might be a bit vague for now, but it will become clear when coding in the wild as things get bigger and more lines and files of code get involved.

There is only one more very important point to be made about scopes for now. Let's start with an example and see if you can figure out what this should log:

```
function confuseReader() {
  x = "Guess my scope...";
  console.log("Inside the function:", x);
}

confuseReader();
console.log("Outside of function:", x);
```

Answer ready? Here is the output:

```
Inside the function: Guess my scope...
Outside of function: Guess my scope...
```

Do not close the book – we'll explain what is going on. If you look carefully, the `x` in the function gets defined without the keyword `let` or `var`. There is no declaration of `x` above the code; this is all the code of the program. JavaScript does not see `let` or `var` and then decides, "this must be a global variable." Even though it gets defined inside the function, the declaration of `x` within the function gets global scope and can still be accessed outside of the function.

We really want to emphasize that this is a terrible practice. If you need a global variable, declare it at the top of your file.

Immediately invoked function expression

The **immediately invoked function expression** (IIFE) is a way of expressing a function so that it gets invoked immediately. It is anonymous, it doesn't have a name, and it is self-executing.

This can be useful when you want to initialize something using this function. It is also used in many design patterns, for example, to create private and public variables and functions.

This has to do with where functions and variables are accessible from. If you have an IIFE in the top-level scope, whatever is in there is not accessible from outside even though it is top level.

Here is how to define it:

```
(function () {
  console.log("IIFE!");
})();
```

The function itself is surrounded by parentheses, which makes it create a function instance. Without these parentheses around it, it would throw an error because our function does not have a name (this is worked around by assigning the function to a variable, though, where the output can be returned to the variable).

() executes the unnamed function—this must be done immediately following a function declaration. If your function were to require a parameter, you would pass it in within these final brackets.

You could also combine IIFE with other function patterns. For example, you could use an arrow function here to make the function even more concise:

```
((()=>{
  console.log("run right away");
})();
```

Again, we use () to invoke the function that you created.

Practice exercise 6.5

Use IIFE to create a few immediately invoked functions and observe how the scope is affected.

1. Create a variable value with 1et and assign a string value of 1000 to it.
2. Create an IIFE function and within this function scope assign a new value to a variable of the same name. Within the function, print the local value to the console.
3. Create an IIFE expression, assigning it to a new result variable, and assign a new value to a variable of the same name within this scope. Return this local value to the result variable and invoke the function. Print the result variable, along with the variable name you've been using: what value does it contain now?

4. Lastly, create an anonymous function that has a parameter. Add logic that will assign a passed-in value to the same variable name as the other steps, and print it as part of a string sentence. Invoke the function and pass in your desired value within the rounded brackets.

Recursive functions

In some cases, you want to call the same function from inside the function. It can be a beautiful solution to rather complex problems. There are some things to keep in mind though. What do you think this will do?

```
function getRecursive(nr) {
  console.log(nr);
  getRecursive(--nr);
}
```

```
getRecursive(3);
```

It prints 3 and then counts down and never stops. Why is it not stopping? Well, we are not saying when it should stop. Look at our improved version:

```
function getRecursive(nr) {
  console.log(nr);
  if (nr > 0) {
    getRecursive(--nr);
  }
}
```

```
getRecursive(3);
```

This function is going to call itself until the value of the parameter is no longer bigger than 0. And then it stops.

What happens when we call a function recursively is that it goes one function deeper every time. The first function call is done last. For this function it goes like this:

- getRecursive(3)
 - getRecursive(2)
 - getRecursive(1)
 - getRecursive(0)
 - done with getRecursive(0) execution

- done with `getRecursive(1)` execution
- done with `getRecursive(2)` execution
- done with `getRecursive(3)` execution

The next recursive function will demonstrate that:

```
function logRecursive(nr) {
  console.log("Started function:", nr);
  if (nr > 0) {
    logRecursive(nr - 1);
  } else {
    console.log("done with recursion");
  }
  console.log("Ended function:", nr);
}

logRecursive(3);
```

It will output:

```
Started function: 3
Started function: 2
Started function: 1
Started function: 0
done with recursion
Ended function: 0
Ended function: 1
Ended function: 2
Ended function: 3
```

Recursive functions can be great in some contexts. When you feel the need to call the same function over and over again in a loop, you should probably consider recursion. An example could also be searching for something. Instead of looping over everything inside the same function, you can split up inside the function and call the function repeatedly from the inside.

However, it must be kept in mind that in general, the performance of recursion is slightly worse than the performance of regular iteration using a loop. So if this causes a bottleneck situation that would really slow down your application, then you might want to consider another approach.

Have a look at calculating the factorial using recursive functions in the following exercise.

Practice exercise 6.6

A common problem that we can solve with recursion is calculating the factorial.



Quick mathematics refresher about factorials:

The factorial of a number is the product of all positive integers bigger than 0, up to the number itself. So for example, the factorial of seven is $7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$. You can write this as 7!.

How are recursive functions going to help us calculate the factorial? We are going to call the function with a lower number until we reach 0. In this exercise, we will use recursion to calculate the factorial result of a numeric value set as the argument of a function.

1. Create a function that contains a condition within it checking if the argument value is 0.
2. If the parameter is equal to 0, it should return the value of 1. Otherwise, it should return the value of the argument multiplied by the value returned from the function itself, subtracting one from the value of the argument that is provided. This will result in running the block of code until the value reaches 0.
3. Invoke the function, providing an argument of whatever number you want to find the factorial of. The code should run whatever number is passed initially into the function, decreasing all the way to 0 and outputting the results of the calculation to the console. It could also contain a `console.log()` call to print the current value of the argument in the function as it gets invoked.
4. Change and update the number to see how it affects the results.

Nested functions

Just as with loops, if statements, and actually all other building blocks, we can have functions inside functions. This phenomenon is called nested functions:

```
function doOuterFunctionStuff(nr) {
  console.log("Outer function");
  doInnerFunctionStuff(nr);
  function doInnerFunctionStuff(x) {
    console.log(x + 7);
  }
}
```

```

    console.log("I can access outer variables:", nr);
  }
}
doOuterFunctionStuff(2);

```

This will output:

```

Outer function
9
I can access outer variables: 2

```

As you can see, the outer function is calling its nested function. This nested function has access to the variables of the parent. The other way around, this is not the case. Variables defined inside the inner function have function scope. This means they are accessible inside the function where they are defined, which is in this case the inner function. Thus, this will throw a `ReferenceError`:

```

function doOuterFunctionStuff(nr) {
  doInnerFunctionStuff(nr);
  function doInnerFunctionStuff(x) {
    let z = 10;
  }
  console.log("Not accessible:", z);
}

doOuterFunctionStuff(2);

```

What do you think this will do?

```

function doOuterFunctionStuff(nr) {
  doInnerFunctionStuff(nr);
  function doInnerFunctionStuff(x) {
    let z = 10;
  }
}

doInnerFunctionStuff(3);

```

This will also throw a `ReferenceError`. Now, `doInnerFunctionStuff()` is defined inside the outer function, which means that it is only in scope inside `doOuterFunctionStuff()`. Outside this function, it is out of scope.

Practice exercise 6.7

Create a countdown loop starting at a dynamic value of `10`.

1. Set the `start` variable at a value of `10`, which will be used as the starting value for the loop.
2. Create a function that takes one argument, which is the countdown value.
3. Within the function, output the current value of the countdown into the console.
4. Add a condition to check if the value is less than `1`; if it is, then return the function.
5. Add a condition to check if the value of the countdown is not less than `1`, then continue to loop by calling the function within itself.
6. Make sure you add a decrement operator on the countdown so the preceding condition eventually will be true to end the loop. Every time it loops, the value will decrease until it reaches `0`.
7. Update and create a second countdown using a condition if the value is greater than `0`. If it is, decrease the value of the countdown by `1`.
8. Use `return` to return the function, which then invokes it again and again until the condition is no longer true.
9. Make sure, when you send the new countdown value as an argument into the function, that there is a way out of the loop by using the `return` keyword and a condition that continues the loop if met.

Anonymous functions

So far, we have been naming our functions. We can also create functions without names if we store them inside variables. We call these functions anonymous. Here is a non-anonymous function:

```

function doingStuffAnonymously() {
  console.log("Not so secret though.");
}

```

Here is how to turn the previous function into an anonymous function:

```

function () {
  console.log("Not so secret though.");
};

```

As you can see, our function has no name. It is anonymous. So you may wonder how you can invoke this function. Well actually, you can't like this!

We will have to store it in a variable in order to call the anonymous function; we can store it like this:

```
let functionVariable = function () {
  console.log("Not so secret though.");
};
```

An anonymous function can be called using the variable name, like this:

```
functionVariable();
```

It will simply output `Not so secret though..`

This might seem a bit useless, but it is a very powerful JavaScript construct. Storing functions inside variables enables us to do very cool things, like passing in functions as parameters. This concept adds another abstract layer to coding. This concept is called callbacks, and we will discuss it in the next section.

Practice exercise 6.8

1. Set a variable name and assign a function to it. Create a function expression with one parameter that outputs a provided argument to the console.
2. Pass an argument into the function.
3. Create the same function as a normal function declaration.

Function callbacks

Here is an example of passing a function as an argument to another function:

```
function doFlexibleStuff(executeStuff) {
  executeStuff();
  console.log("Inside doFlexibleStuffFunction.");
}
```

If we call this new function with our previously made anonymous function, `functionVariable`, like this:

```
doFlexibleStuff(functionVariable);
```

It will output:

```
Not so secret though.
Inside doFlexibleStuffFunction.
```

But we can also call it with another function, and then our `doFlexibleStuff` function will execute this other function. How cool is that?

```
let anotherFunctionVariable = function() {
  console.log("Another anonymous function implementation.");
}

doFlexibleStuff(anotherFunctionVariable);
```

This will produce the following output:

```
Another anonymous function implementation.
Inside doFlexibleStuffFunction.
```

So what happened? We created a function and stored it in the `anotherFunctionVariable` variable. We then sent that in as a function parameter to our `doFlexibleStuff()` function. And this function is simply executing whatever function gets sent in.

At this point you may wonder why the writers are so excited about this callback concept. It probably looks rather lame in the examples you have seen so far. Once we get to asynchronous functions later on, this concept is going to be of great help. To still satisfy your need for a more concrete example, we will give you one.

In JavaScript, there are many built-in functions, as you may know by now. One of them is the `setTimeout()` function. It is a very special function that is executing a certain function after a specified amount of time that it will wait first. It is also seemingly responsible for quite a few terribly performing web pages, but that is definitely not the fault of this poor misunderstood and misused function.

This code is really something you should try to understand:

```
let youGotThis = function () {
  console.log("You're doing really well, keep coding!");
};

setTimeout(youGotThis, 1000);
```

It is going to wait for 1000ms (one second) and then print:

```
You're doing really well, keep coding!
```

If you need more encouragement, you can use the `setInterval()` function instead. It works very similarly, but instead of executing the specified function once, it will keep on executing it with the specified interval:

```
setInterval(youGotThis, 1000);
```

In this case, it will print our encouraging message every second until you kill the program.

This concept of the function executing the function after having been called itself is very useful for managing asynchronous program execution.

Chapter projects

Create a recursive function

Create a recursive function that counts up to 10. Invoke the function with different start numbers as the arguments that are passed into the function. The function should run until the value is greater than 10.

Set timeout order

Use the arrow format to create functions that output the values one and two to the console. Create a third function that outputs the value three to the console, and then invokes the first two functions.

Create a fourth function that outputs the word four to the console and also use `setTimeout()` to invoke the first function immediately and then the third function.

What does your output look like in the console? Try to get the console to output:

```
Four
Three
One
Two
One
```

Self-check quiz

1. What value is output into the console?

```
let val = 10;
function tester(val){
  val += 10;
  if(val < 100){
    return tester(val);
  }
  return val;
}
tester(val);
console.log(val);
```

2. What will be output into the console by the below code?

```
let testFunction = function(){
  console.log("Hello");
}();
```

3. What will be output to the console?

```
(function () {
  console.log("Welcome");
})();
(function () {
  let firstName = "Laurence";
})();
let result = (function () {
  let firstName = "Laurence";
  return firstName;
})();
console.log(result);
(function (firstName) {
  console.log("My Name is " + firstName);
})("Laurence");
```

4. What will be output to the console?

```
let test2 = (num) => num + 5;
console.log(test2(14));
```

5. What will be output to the console?

```
var addFive1 = function addFive1(num) {  
  return num + 2;  
};  
let addFive2 = (num) => num + 2;  
console.log(addFive1(14));
```

Summary

In this chapter, we have covered functions. Functions are a great JavaScript building block that we can use to reuse lines of code. We can give our functions parameters, so that we can change the code depending on the arguments a function gets invoked with. Functions can return a result; we do so using the `return` keyword. And we can use `return` at the place where we call a function. We can store the result in a variable or use it in another function, for example.

We then met with variable scopes. The scope entails the places from where variables are accessible. Default `let` and `const` variables can be accessed inside the block where they're defined (and the inner blocks of that block) and `var` is just accessible from the line where it was defined.

We can also use recursive functions to elegantly solve problems that can be solved recursively by nature, such as calculating the factorial. Nested functions were the next topic we studied. They are not a big deal, just functions inside functions. Basic functions inside functions are not considered very pretty, but anonymous functions and arrow functions are not uncommon to see. Anonymous functions are functions without a name and arrow functions are a special case of anonymous functions, where we use an arrow to separate the parameters and the body.

In the next chapter, we'll consider classes, another powerful programming construct!

7

Classes

In this chapter, we are going to discuss JavaScript classes. We have seen JavaScript objects already, and classes are a blueprint or template for object creation. So, many of the things discussed here should not sound too unfamiliar or revolutionary.

Classes enable object-oriented programming, which was one of the most important design advancements in software development. This development reduced the complexity of applications and increased maintainability by a huge margin.

So, object-oriented programming and classes are of great importance for computer science in general. This is not necessarily the case when we apply it to JavaScript though. JavaScript classes are something special compared to other programming languages. Beneath the surface, classes are wrapped in some sort of special function. This means that they are actually an alternative syntax for defining objects using a constructor function. In this chapter, we will learn what classes are and how we can create and use them. Along the way, we will cover the following topics:

- Object-oriented programming
- Classes and objects
- Classes
- Inheritance
- Prototypes



Note: exercise, project, and self-check quiz answers can be found in the *Appendix*.

Object-oriented programming

Before we start diving right into the fun of classes, let's briefly say something about **object-oriented programming (OOP)**. OOP is a very important programming paradigm wherein code is structured in objects, leading to more maintainable and reusable code. Working with OOP teaches you to really try to think of all sorts of topics in objects, by bundling properties in such a way that they can be wrapped in a blueprint called a class. This in turn might be inheriting properties from a parent class.

For example, if we are thinking of an animal, we can come up with certain properties: name, weight, height, maximum speed, colors, and a lot more. And then if we think of a specific species of fish, we can reuse all the properties of "animal" and add a few fish-specific properties in there as well. The same for dogs; if we then think of a dog, we can reuse all the properties of "animal" and add a few dog-specific ones to it. This way we have reusable code of our animal class. And when we realize we forgot a very important property for the many animals in our application, we only need to add it to the animal class.

This is very important for Java, .NET, and other classic object-oriented ways of writing code. JavaScript doesn't necessarily revolve around objects. We will need them and we will use them, but they are not the star of our code, so to speak.

Classes and objects

As a quick refresher, objects are a collection of properties and methods. We saw them in *Chapter 3, JavaScript Multiple Values*. The properties of an object should have sensible names. So for example, if we have a person object, this object could have properties called `age` and `lastName` that contain values. Here is an example of an object:

```
let dog = { dogName: "JavaScript",
            weight: 2.4,
            color: "brown",
            breed: "chihuahua"
};
```

Classes in JavaScript encapsulate data and functions that are part of that class. If you create a class, you can later create objects using that class using the following syntax:

```
class ClassName {
  constructor(prop1, prop2) {
    this.prop1 = prop1;
  }
}
```



```

    this.prop2 = prop2;
  }
}

let obj = new ClassName("arg1", "arg2");

```

This code defines a class with `ClassName` as a name, declares an `obj` variable, and initializes this with a new instance of the object. Two arguments are provided. These arguments will be used by the constructor to initialize the properties. As you can see, the parameters of the constructor and the properties of the class (`prop1` and `prop2`) have the same name. The properties of the class can be recognized by the `this` keyword in front of them. The `this` keyword refers to the object it belongs to, so it is the first property of the instance of `ClassName`.

Remember we said that classes are just some special function beneath the surface. We could create the object with a special function like this:

```

function Dog(dogName, weight, color, breed) {
  this.dogName = dogName;
  this.weight = weight;
  this.color = color;
  this.breed = breed;
}

let dog = new Dog("Jacky", 30, "brown", "labrador");

```

The dog example could have been made using a class syntax as well. It would have looked like this:

```

class Dog {
  constructor(dogName, weight, color, breed) {
    this.dogName = dogName;
    this.weight = weight;
    this.color = color;
    this.breed = breed;
  }
}

let dog = new Dog("JavaScript", 2.4, "brown", "chihuahua");

```

This results in an object with the same properties. If we do some logging as follows, we will be able to see it:

```

console.log(dog.dogName, "is a", dog.breed, "and weighs", dog.weight,
"kg.");

```

This will output:

```
JavaScript is a chihuahua and weighs 2.4 kg.
```

In the next section, we will dive into all the parts of classes.

Classes

You may wonder, if classes do the exact same thing as simply defining an object, why do we even need classes? The answer is that classes are essentially blueprints for object creation. This means that we need to do much less typing if we need to create 20 dogs when we have a dog class. If we have to create the objects, we will have to specify all the properties' names each time. And it would be easy to make a typo and misspell a property name. Classes come in handy in these sorts of situations.

As shown in the previous section, we use the `class` keyword to tell JavaScript we want to create a class. Next, we give the class a name. It is the convention to start class names with a capital letter.

Let's have a look at all the different elements of a class.

Constructors

The constructor method is a special method that we use to initialize objects with our class blueprint. There can only be one constructor in a class. This constructor contains properties that will be set when initiating the class.

Here you can see an example of a constructor in a `Person` class:

```

class Person {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }
}

```

Beneath the surface, JavaScript creates a special function based on this constructor. This function gets the class name, and it will create an object with the given properties. With this special function, you can create instances (objects) of the class.

Here is how you can create a new object from the `Person` class:

```
let p = new Person("Maaïke", "van Putten");
```

The new word is what tells JavaScript to look for the special constructor function in the `Person` class and create a new object. The constructor gets called and returns an instance of the person object with the specified properties. This object gets stored in the `p` variable.

If we use our new `p` variable in a logging statement, you can see that the properties are really set:

```
console.log("Hi", p.firstname);
```

This outputs:

```
Hi Maaïke
```

What do you think will happen when we create a class without all of the properties? Let's find out:

```
let p = new Person("Maaïke");
```

Many languages would crash, but not JavaScript. It just sets the remaining properties to `undefined`. You can see what happens by logging it:

```
console.log("Hi", p.firstname, p.lastname);
```

This results in:

```
Hi Maaïke undefined
```

You can specify default values in constructor. You would do it like this:

```
constructor(firstname, lastname = "Doe") {
  this.firstname = firstname;
  this.lastname = lastname;
}
```

This way, it would not have printed `Hi Maaïke undefined`, but `Hi Maaïke Doe`.

Practice exercise 7.1

Take the following steps to create a person class, and print instances of friends' names:

1. Create a class for `Person` including the constructor for `firstname` and `lastname`.
2. Create a variable and assign a value of the new `Person` object using your friend's first and last names.
3. Now add a second variable with another friend's name using their first name and last name.
4. Output both friends into the console with a greeting of hello.

Methods

In a class, we can specify functions. This means that our object can start doing things using the object's own properties – for example, printing a name. Functions on a class are called methods. When defining these methods, we don't use the `function` keyword. We start directly with the name:

```
class Person {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }

  greet() {
    console.log("Hi there! I'm", this.firstname);
  }
}
```

We can call the `greet` method on a `Person` object like this:

```
let p = new Person("Maaïke", "van Putten");
p.greet();
```

It will output:

```
Hi there! I'm Maaïke
```

You can specify as many methods on a class as you want. In this example, we are using the `firstname` property. We do so by saying `this.property`. If we had a person with a different value for the `firstname` property, for example, Rob, it would have printed.

```
Hi there! I'm Rob
```

Just like functions, methods can also take parameters and return results:

```
class Person {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }

  greet() {
    console.log("Hi there!");
  }

  compliment(name, object) {
    return "That's a wonderful " + object + ", " + name;
  }
}
```

The `compliment` function does not output anything itself, so we are logging it

```
let compliment = p.compliment("Harry", "hat");
console.log(compliment);
```

The output will be:

```
That's a wonderful hat, Harry
```

In this case we are sending parameters into our method, because you don't usually compliment your own properties (that's a nice sentence, Maaikel!). However, whenever the method doesn't require external input but only the properties of the object, no parameters will work and the method can use its object's properties. Let's do an exercise and then move on to using the properties of classes outside the class.

Practice exercise 7.2

Get your friend's full name:

1. Using the `Person` class from *Practice exercise 7.1*, add a method called `fullname`, which returns the concatenated value of `firstname` and `lastname` when invoked.
2. Create values for `person1` and `person2` using two friends' first and last names.
3. Using the `fullname` method within the class, return the full name of one or both people.

Properties

Properties, sometimes also called fields, hold the data of the class. We have seen one kind of property already, when we created them in our constructors:

```
class Person {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }
}
```

Here, the `Person` class gets two properties from the constructor: `firstname` and `lastname`. Properties can be added or removed just like we did for objects. These properties can be accessed from outside the class, as we saw when we logged them outside the class by accessing them on the instance:

```
let p = new Person("Maaiké", "van Putten");
console.log("Hi", p.firstname);
```

Often, it is not desirable to provide direct access to our properties. We want our class to be in control of the values of properties for several reasons—perhaps we want to do validation on a property to assure it has a certain value. For example, imagine wanting to validate an age as not being lower than 18. We can achieve this by making direct access to the property from outside the class impossible.

This is how to add properties that aren't accessible from outside. We prefix them with a `#` symbol:

```
class Person {
  #firstname;
  #lastname;
```

```

constructor(firstname, lastname) {
  this.#firstname = firstname;
  this.#lastname = lastname;
}

```

Right now, the `firstname` and `lastname` properties cannot be accessed from outside the class. This is done by adding `#` in front of the property. If we try it:

```

let p = new Person("Maria", "Saga");
console.log(p.firstname);

```

We'll get:

```
undefined
```

If we wanted to make sure we could only create objects with names starting with an "M," we could modify our constructor a bit:

```

constructor(firstname, lastname) {
  if(!firstname.startsWith("M")){
    this.#firstname = firstname;
  } else {
    this.#firstname = "M" + firstname;
  }
  this.#lastname = lastname;
}

```

Now when you try to create a person that has a `firstname` value that doesn't start with an "M," it will add an `M` in front. So for example, the value of the following first name is `Mkay`:

```
let p = new Person("kay", "Moon");
```

This is a very silly example of validation. At this point, we cannot access it from outside the class at all after the constructor. We can only access it from inside the class. This is where getters and setters come into play.

Getters and setters

Getters and setters are special properties that we can use to get data from a class and to set data fields on the class. Getters and setters are computed properties. So, they are more like properties than they are like functions. We call them accessors. They do look a bit like functions, because they have `()` behind them, but they are not!

These accessors start with the `get` and `set` keywords. It is considered good practice to make fields private as much as possible and provide access to them using getters and setters. This way, the properties cannot be set from outside without the object itself being in control. This principle is called **encapsulation**. The class encapsulates the data, and the object is in control of its own fields.

Here is how to do it:

```

class Person {
  #firstname;
  #lastname;
  constructor(firstname, lastname) {
    this.#firstname = firstname;
    this.#lastname = lastname;
  }

  get firstname() {
    return this.#firstname;
  }

  set firstname(firstname) {
    this.#firstname = firstname;
  }

  get lastname() {
    return this.#lastname;
  }

  set lastname(lastname) {
    this.#lastname = lastname;
  }
}

```

The `getter` is used to get the property. Therefore, it doesn't take any parameters, but simply returns the property. The `setter` is the other way around: it takes a parameter, assigns this new value to the property, and returns nothing. The `setter` can contain more logic, for example, some validation, as we'll see below. The `getter` can be used outside the object as if it were a property. The properties are no longer directly accessible from outside the class, but can be accessed via the `getter` to get the value and via the `setter` to update the value. Here is how to use it outside the class instance:

```

let p = new Person("Maria", "Saga");
console.log(p.firstname);

```

This will output:

```
Maria
```

We have created a new `Person` object with a first name of `Maria` and last name of `Saga`. The output is showing the first name, which is only possible because we have a getter accessor. We can also set the value to something else, because there is a setter. Here is how to update the first name, so the name is no longer `Maria`, but `Adnane`.

```
p.firstname = "Adnane";
```

At this point, nothing special is happening in the setter. We could do a similar validation as in the constructor before, like this:

```
set firstname(firstname) {
  if(firstname.startsWith("M")){
    this.#firstname = firstname;
  } else {
    this.#firstname = "M" + firstname;
  }
}
```

This will check whether `firstname` starts with an `M`, and if it does it will update the value to whatever the `firstname` parameter is. If it doesn't, it will concatenate an `M` in front of the parameter.

Please note that we do not access `firstname` as if it was a function. If you put two parentheses `()` after it, you would actually get an error telling you that it is not a function.

Inheritance

Inheritance is one of the key concepts of OOP. It is the concept that classes can have child classes that inherit the properties and methods from the parent class. For example, if you needed all sorts of vehicle objects in your application, you could specify a class named `Vehicle` in which you specify some shared properties and methods of vehicles. You would then go ahead and create the specific child classes based on this `Vehicle` class, for example, `boat`, `car`, `bicycle`, and `motorcycle`.

This could be a very simple version of the `Vehicle` class:

```
class Vehicle {
  constructor(color, currentSpeed, maxSpeed) {
    this.color = color;
  }
}
```

```
this.currentSpeed = currentSpeed;
this.maxSpeed = maxSpeed;
}

move() {
  console.log("moving at", this.currentSpeed);
}

accelerate(amount) {
  this.currentSpeed += amount;
}
}
```

Here we have two methods in our `Vehicle` class: `move` and `accelerate`. And this could be a `Motorcycle` class inheriting from this class using the `extends` keyword:

```
class Motorcycle extends Vehicle {
  constructor(color, currentSpeed, maxSpeed, fuel) {
    super(color, currentSpeed, maxSpeed);
    this.fuel = fuel;
  }
  dowheelie() {
    console.log("Driving on one wheel!");
  }
}
```

With the `extends` keyword we specify that a certain class is the child of another class. In this case, `Motorcycle` is a child class of `Vehicle`. This means that we'll have access to properties and methods from `Vehicle` in our `Motorcycle` class. We have added a special `dowheelie()` method. This is not something that makes sense to add to the `Vehicle` class, because this is an action that is specific to certain vehicles.

The `super` word in the constructor is calling the constructor from the parent, the `Vehicle` constructor in this case. This makes sure that the fields from the parent are set as well and that the methods are available without having to do anything else: they are automatically inherited. Calling `super()` is not optional, you must do it when you are in a class that is inheriting from another class, else you will get a `ReferenceError`.

Because we have access to the fields of `Vehicle` in `Motorcycle`, this will work:

```
let motor = new Motorcycle("Black", 0, 250, "gasoline");
console.log(motor.color);
```

```
motor.accelerate(50);
motor.move();
```

And this is what it will output:

```
Black
moving at 50
```

We cannot access any `Motorcycle` specific properties or methods in our `Vehicle` class. This is because not all vehicles are motorcycles, so we cannot be sure that we would have the properties or methods from a child.

Right now, we don't use any getters and setters here, but we clearly could. If there are getters and setters in the parent class, they are inherited by the child class as well. This way we could influence which properties could be fetched and changed (and how) outside our class. This is generally a good practice.

Prototypes

A prototype is the mechanism in JavaScript that makes it possible to have objects. When nothing is specified when creating a class, the objects inherit from the `Object.prototype` prototype. This is a rather complex built-in JavaScript class that we can use. We don't need to look at how this is implemented in JavaScript, as we can consider it the base object that is always on top of the inheritance tree and therefore always present in our objects.

There is a prototype property available on all classes, and it is always named "prototype." We can access it like this:

```
ClassName.prototype
```

Let's give an example of how to add a function to a class using the prototype property. In order to do so, we'll be using this `Person` class:

```
class Person {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }

  greet() {
    console.log("Hi there!");
  }
}
```

And here is how to add a function to this class using prototype:

```
Person.prototype.introduce = function () {
  console.log("Hi, I'm", this.firstname);
};
```

prototype is a property holding all the properties and methods of an object. So, adding a function to prototype is adding a function to the class. You can use prototype to add properties or methods to an object, like we did in the above example in our code with the `introduce` function. You can also do this for properties:

```
Person.prototype.favoriteColor = "green";
```

And then you can call them from instances of `Person`:

```
let p = new Person("Maria", "Saga");
console.log(p.favoriteColor);
p.introduce();
```

It will output:

```
green
Hi, I'm Maria
```

And it will be as if you had defined the class with a favorite color holding a default value, and a function, `introduce`. They have been added to the class and are available for all instances and future instances.

So the methods and properties defined via prototype are really as if they were defined in the class. This means that overwriting them for a certain instance doesn't overwrite them for all instances. For example, if we were to have a second `Person` object, this person could overwrite the `favoriteColor` value and this wouldn't change the value for our object with `firstname` as `Maria`.

This is something you should not be using when you have control over the class code and you want to change it permanently. In that case, just change the class. However, you can expand existing objects like this and even expand existing objects conditionally. It is also important to know that the JavaScript built-in objects have prototypes and inherit from `Object.prototype`. However, be sure not to modify this prototype since it will affect how our JavaScript works.

Practice exercise 7.3

Create a class that contains properties for different animal species and the sound that each species makes, and create two (or more) animals:

1. Create a method that prints a given animal and its sound.
2. Add a prototype with another action for the animal.
3. Output an entire animal object into the console.

Chapter projects

Employee tracking app

Create a class to track the employees of a company:

1. Use first names, last names, and the number of years worked as values in the constructor.
2. Create two or more people with values for their first names, last names, and the number of years they've worked at the company. Add the people into an array.
3. Set up a prototype to return the details of the person's first and last names and how long they've worked at the company.
4. Iterate the contents of the array to output the results into the console, adding some text to make the output a full sentence.

Menu items price calculator

Create a class which will allow you to work out the combined price of a number of items, and interact with it to work out the total cost of different orders.

1. Create a class that contains the prices of two menu items as private field declarations.
2. Use the constructor in the class to get the argument values (how many of each item are being bought).
3. Create a method to calculate and return the total cost depending on how many of each item the user selects.
4. Use a getter property to grab the value output by the calculation method.
5. Create two or three objects with different combinations of menu selections, and output the total cost in the console.

Self-check quiz

1. What is the keyword used to create a class?
2. How would you set up a class for a person's first and last names that could include `first` and `last` as initial properties?
3. What is the concept of one thing gaining the properties and behaviors of another thing called?
4. Which of the following are true about the constructor method?
 - It gets executed automatically when a new object is created.
 - It should only be added afterward.
 - It has to include the `constructor` keyword.
 - It is used to initialize object properties.
 - It can be used when you have multiple values.
5. Troubleshoot the following code so that the prototype outputs the first and last name of the `Person` into the console. Which is the correct syntax for the `Person` prototype?

```
function Person(first,last) {
  this.first = first;
  this.last = last;
}
// What should go here: A, B, or C?
const friend1 = new Person("Laurence", "Svekis");
console.log(friend1.getName());
```

A)

```
Person.prototype.getName = (first,last) {
  return this.first + " " + this.last;
};
```

B)

```
Person.prototype.getName = function getName() {
  return this.first + " " + this.last;
};
```

C)

```
Person.prototype = function getName() {
  return this.first + " " + this.last;
};
```

Summary

In this chapter, we introduced you to the concept of OOP. This means that we structure our code in such a way that objects are the central players of the logic. Classes are blueprints for objects. We can make a template for an object and create an instance easily by using the *new* keyword.

We then saw that classes can inherit from each other by using the *extends* keyword. Classes that extend from another class will have to call the constructor of this class with *super()* and will then automatically have access to all the properties and methods of the parent. This is great for reusable and highly maintainable code.

Lastly, we encountered prototypes. This is the built-in JavaScript concept that makes classes possible. By adding properties and methods to a class using *prototype*, we can modify the blueprint of that class.

In the next chapter, we will consider some of JavaScript's built-in methods, which can be used to manipulate and add complexity to your code!

8

Built-In JavaScript Methods

We have just covered most of the basic building blocks in JavaScript. Now it's time to look at some powerful built-in methods that will make your life easier that we haven't seen yet. Built-in methods are functionality that we get out of the box with JavaScript. We can use these methods without having to code them first. This is something we have done a lot already, for example, `console.log()` and `prompt()`.

Many built-in methods belong to built-in classes as well. These classes and their methods can be used at any time because JavaScript has already defined them. These classes exist for our convenience, since they are very common things to need, such as the `Date`, `Array`, and `Object` classes.

The ability to harness the capabilities that are already built into JavaScript can improve the effectiveness of the code, save time, and comply with various best practices for developing solutions. We are going to address some of the common uses for such functions, such as manipulating text, mathematical computations, dealing with date and time values, interactions, and supporting robust code. Here are the topics covered in this chapter:

- Global JavaScript methods
- String methods
- Math methods
- Date methods
- Array methods
- Number methods



Note: exercise, project and self-check quiz answers can be found in the Appendix.

Introduction to built-in JavaScript methods

We have seen many built-in JavaScript methods already. Any method that we didn't define ourselves is a built-in method. Some examples include `console.log()`, `Math.random()`, `prompt()`, and many more – think about methods on arrays for example. The difference between a method and a function is that a function is defined anywhere in the script, and a method is defined inside a class. So methods are pretty much functions on classes and instances.

Methods can often be chained as well; this is only true for methods returning a result. The next method will then be performed on the result. So for example:

```
let s = "Hello";
console.log(
  s.concat(" there!")
    .toUpperCase()
    .replace("THERE", "you")
    .concat(" You're amazing!")
);
```

We create a variable, `s`, and we store `Hello` in there on the first line. Then we want to be logging something. This code has been divided over different lines for readability, but it's actually one statement. We first perform a `concat()` method on our `s` variable, which appends a string to our string. So after that first operation the value is `Hello there!`. Then we transform this to uppercase with the next method. At that point the value is `HELLO THERE!`. Then we proceed to replace `THERE` with `you`. After that, the value becomes `HELLO you!`. We then append a string to it again and finally the value will be logged:

```
HELLO you! You're amazing!
```

We need to log or store the output in this example, because the original string value will not be updated by just calling methods on a string.

Global methods

The global JavaScript methods can be used without referring to the built-in object they are part of. This means that we can just use the method name as if it is a function that has been defined inside the scope we are in, without the "object" in front of it. For example, instead of writing:

```
let x = 7;
console.log(Number.isNaN(x));
```

You can also write:

```
console.log(isNaN(x));
```

So, the `Number` can be left out, because `isNaN` is made globally available without referring to the class it belongs to (in this instance, the `Number` class). In this case, both of these `console.log` statements will log `false` (they are doing the exact same thing), because `isNaN` returns true when it isn't a number. And 7 is a number, so it will log `false`.

JavaScript has been built to have these available directly, so to achieve this, some magic is going on beneath the surface. The JavaScript creators chose the methods that they thought were most common. So the reasons why some of them are available as global methods and others are not might seem a bit arbitrary. It's just the choice of some very bright developers at a certain point in time.

We'll address the most common global methods below. We start with decoding and encoding URIs, escaped and unescaped, followed by parsing numbers, and finally evaluate.

Decoding and encoding URIs

Sometimes you will need to encode or decode a string. Encoding is simply converting from one shape to another. In this case we will be dealing with percent encoding, also called URL encoding. Before we start, there might be some confusion on the URL and URI meaning. A **URI (uniform resource identifier)** is an identifier of a certain resource. **URL (uniform resource locator)** is a subcategory of URI that is not only an identifier, but also holds the information on how to access it (location).

Let's talk about encoding and decoding these URIs (and also URLs, since they are a subset). An example of when you'd need this is when you are sending variables over the URL using the `get` method in a form. These variables that you are sending via the URL are called query parameters.

If something contains a space, this will be decoded, because you cannot use spaces in your URL. They will be converted to `%20`. The URL might look something like:

```
www.example.com/submit?name=maaike%20van%20putten&love=coding
```

All characters can be converted to some %-starting format. However, this is not necessary in most cases. URIs can contain a certain number of alphanumeric characters. The special characters need to be encoded. An example, before encoding, is: `https://www.example.com/submit?name=maaike van putten`

The same URL after encoding is:

```
https://www.example.com/submit?name=maaike%20van%20putten
```

There are two pairs of encode and decode methods. We will discuss them and their use cases here. You cannot have a URI with spaces, so working with these methods is crucial in order to work with variables containing spaces.

decodeUri() and encodeUri()

The `decodeUri()` and `encodeUri()` are actually not really encoding and decoding, they are more so fixing broken URIs. It is like the previous example with the spaces. This method pair is really good at fixing broken URIs and decoding them back into a string. Here you can see them in action:

```
let uri = "https://www.example.com/submit?name=maaike van putten";
let encoded_uri = encodeURI(uri);
console.log("Encoded:", encoded_uri);
let decoded_uri = decodeURI(encoded_uri);
console.log("Decoded:", decoded_uri);
```

And here is the output:

```
Encoded: https://www.example.com/submit?name=maaike%20van%20putten
Decoded: https://www.example.com/submit?name=maaike van putten
```

As you can see, it has replaced the spaces in the encoded version and removed them again in the decoded version. All the other characters get to stay the same—this encode and decode do not take special characters into account, and therefore leave them in the URI. Colons, question marks, equal signs, slashes, and ampersands can be expected.

This is great for fixing broken URIs, but it's actually a bit useless whenever you need to encode strings that contain any of these characters: `/`, `?`, `:`, `@`, `&`, `+`, `$`, `#`. These can be used in URIs as part of the URI and are therefore skipped. This is where the next two built-in methods come in handy.

decodeURIComponent() and encodeURIComponent()

So, the methods `decodeURIComponent()` and `encodeURIComponent()` can be very useful to fix a broken URL, but they are useless when you only want to encode or decode a string that contains a character with a special meaning, such as `=` or `&`. Take the following example:

```
https://www.example.com/submit?name=this&that=some thing&code=love
```

Weird value, we can agree on that, but it will demonstrate our problem. Using `encodeURIComponent` on this will leave us with:

```
https://www.example.com/submit?name=this&that=some%20thing&code=love
```

There are actually 3 variables in here according to URI standards:

- name (value is this)
- that (value is some thing)
- code (value is love)

While we intended to send in one variable, name, with the value `this&that=some thing&code=love`.

In this case, you will need `decodeURIComponent()` and `encodeURIComponent()`, because you would need the `=` and `&` in the variable part encoded as well. Right now, this is not the case and it will actually cause problems in interpreting the query parameters (the variables after the `?`). We only wanted to send in one parameter: name. But instead we sent in three.

Let's have a look at another example. Here is what the example of the previous section would have done with this component encoding:

```
let uri = "https://www.example.com/submit?name=maaike van putten";
let encoded_uri = encodeURIComponent(uri);
console.log("Encoded:", encoded_uri);
let decoded_uri = decodeURIComponent(encoded_uri);
console.log("Decoded:", decoded_uri);
```

The resulting output is as follows:

```
Encoded: https%3A%2F%2Fwww.example.com%2Fsubmit%3Fname%3Dmaaike%20
van%20putten
Decoded: https://www.example.com/submit?name=maaike van putten
```

Clearly, you don't want this as your URI, but the component methods are useful to encode, for example, a URL variable. If the URL variable were to contain a special character, like `=` and `&`, this would change the meaning and break the URI if these characters don't get encoded.

Encoding with escape() and unescape()

These are still global methods available to do something similar to encode (`escape`) and decode (`unescape`). Both methods are strongly discouraged to use and they might actually disappear from future JavaScript versions or may not be supported by browsers for good reasons.

Practice exercise 8.1

Output the `decodeURIComponent()` for the string `How's%20it%20going%3F` to the console. Also, encode the string `How's it going?` to be output into the console.

Create a web URL and encode the URI:

1. Add the strings as variables in the JavaScript code
2. Using `encodeURIComponent()` and `decodeURIComponent()` output the results into the console
3. Create a web URI with request parameters `http://www.basescripts.com/?=Hello World`;
4. Encode and output the web URI into the console

Parsing numbers

There are different ways to parse strings to numbers. In many situations you will have to translate a string to a number, for example reading input boxes from an HTML web page. You cannot calculate with strings, but you can with numbers. Depending on what exactly you need to do, you will need either one of these methods.

Making integers with parseInt()

With the method `parseInt()` a string will be changed to an integer. This method is part of the `Number` class, but it is global and you can use it without the `Number` in front of it. Here you can see it in action:

```
let str_int = "6";
let int_int = parseInt(str_int);
console.log("Type of ", int_int, "is", typeof int_int);
```

We start off with a string containing a 6. Then we convert this string to an integer using the `parseInt` method, and when we log the result, we will get in the console:

```
Type of 6 is number
```

You can see that the type has changed from `string` to `number`. At this point, you may wonder what will happen if `parseInt()` tries to parse other types of numbers, like string versions of floats or binary numbers. What do you think will happen when we do this?

```
let str_float = "7.6";
let int_float = parseInt(str_float);
console.log("Type of", int_float, "is", typeof int_float);

let str_binary = "0b101";
let int_binary = parseInt(str_binary);
console.log("Type of", int_binary, "is", typeof int_binary);
```

This will log:

```
Type of 7 is number
Type of 0 is number
```

Can you figure out the logic here? First of all, JavaScript doesn't like crashing or using errors as a way out, so it is trying to make it work to the best of its abilities. The `parseInt()` method simply stops parsing when it runs into a non-numeric character. This is the specified behavior, and you need to keep that in mind while working with `parseInt()`. In the first case, it stops parsing as soon as it finds the dot, so the result is 7. And in the binary number case, it stops parsing as soon as it hits the b, and the result is 0. By now you can probably figure out what this does:

```
let str_nan = "hello!";
let int_nan = parseInt(str_nan);
console.log("Type of", int_nan, "is", typeof int_nan);
```

Since the first character is non-numeric, JavaScript will convert this string to `NaN`. Here is the result that you will get in the console:

```
Type of NaN is number
```

So `parseInt()` can be a bit quirky, but it's very valuable. In the real world, it is used a lot to combine the input of users via web pages and calculations.

Making floats with `parseFloat()`

Similarly, we can use `parseFloat()` to parse a string to a float. It works exactly the same, except it can also understand decimal numbers and it doesn't quit parsing as soon as it runs into the first dot:

```
let str_float = "7.6";
let float_float = parseFloat(str_float);
console.log("Type of", float_float, "is", typeof float_float);
```

This will log:

```
Type of 7.6 is number
```

With the `parseInt()`, this value became 7, because it would stop parsing as soon as it finds a non-numeric character. However, `parseFloat()` can deal with one dot in the number, and the numbers after that are interpreted as decimals. Can you guess what happens when it runs into a second dot?

```
let str_version_nr = "2.3.4";
let float_version_nr = parseFloat(str_version_nr);
console.log("Type of", float_version_nr, "is", typeof float_version_nr);
```

This will log:

```
Type of 2.3 is number
```

The strategy is similar to the `parseInt()` function. As soon as it finds a character it cannot interpret, a second dot in this case, it will stop parsing and just return the result so far. Then one more thing to note. It is not going to append a `.0` to integers, so 6 is not going to become `6.0`. This example:

```
let str_int = "6";
let float_int = parseFloat(str_int);
console.log("Type of", float_int, "is", typeof float_int);
```

Will log:

```
Type of 6 is number
```

Lastly, the behavior for binary numbers and strings is the same. It is going to stop parsing as soon as it runs into a character it cannot interpret:

```
let str_binary = "0b101";
let float_binary = parseFloat(str_binary);
console.log("Type of", float_binary, "is", typeof float_binary);

let str_nan = "hello!";
let float_nan = parseFloat(str_nan);
console.log("Type of", float_nan, "is", typeof float_nan);
```

This will log:

```
Type of 0 is number
Type of NaN is number
```

You will use the `parseFloat()` whenever you need a decimal number. However, it will not work with binary, hexadecimal, and octal values, so whenever you really need to work with these values or integers you'll have to use `parseInt()`.

Executing JavaScript with `eval()`

This global method executes the argument as a JavaScript statement. This means that it will just do whatever JavaScript is inserted in there, as if that JavaScript were written directly on the spot instead of `eval()`. This can be convenient for working with injected JavaScript, but injected code comes with great risks. We'll deal with these risks later; let's first explore an example. Here is a fabulous website:

```
<html>
<body>
  <input onchange="go(this)"></input>
  <script>
    function go(e) {
      eval(e.value);
    }
  </script>
</body>
</html>
```

This is a basic HTML web page with an input box on it.



You'll learn more about HTML in *Chapter 9, The Document Object Model*.

Whatever you insert in the input box will get executed. If we were to write this in the input box:

```
document.body.style.backgroundColor = "pink";
```

The website background would change to pink. That looks like fun, right? However, we cannot stress enough how careful you should be using `eval()`. They might as well have called it *evil* according to many developers. Can you reason why this might be?

The answer is security! Yes, this is probably the worst thing security-wise you can do in most situations. You are going to execute external code. This code could be malicious. It is a method for supporting code injection. The well-respected **OWASP (Open Web Application Security Project)** Foundation creates top 10s for security threats every 3 years. Code injection has been on it since their first top 10 and it is still in the OWASP top 10 security threats now. Running it server side can cause your server to crash and your website to go down, or worse. There are almost always better solutions to what you want to do than using `eval()`. Next to the security risks, it is terrible performance-wise. So just for this reason already you might want to avoid using it.

Alright, so one last note on this. If you know what you are doing you might want to use it in very specific cases. Even though it is "evil", it has a lot of power. It can be okay to use in certain cases, for example when you are creating template engines, your own interpreter, and all other JavaScript core tools. Just beware of the danger and control access to this method carefully. And one last bonus tip, when you feel like you really have to use `eval`, do a quick search on the web. Chances are that you will find a better approach.

Array methods

We have seen arrays already—they can contain multiple items. We have also seen quite a few built-in methods on arrays, like `shift()` and `push()`. Let's look at a few more in the following sections.

Performing a certain action for every item

There is a reason we are starting with this method. You might be thinking of loops at this point, but there is a built-in method that you can use to execute a function for every element in the array. This is the `forEach()` method. We mentioned this briefly in *Chapter 6, Functions*, but let's consider it in some more detail. It takes the function that needs to be executed for every element as input. Here you can see an example:

```
let arr = ["grapefruit", 4, "hello", 5.6, true];

function printStuff(element, index) {
  console.log("Printing stuff:", element, "on array position:", index);
}

arr.forEach(printStuff);
```

This code snippet will write to the console:

```
Printing stuff: grapefruit on array position: 0
Printing stuff: 4 on array position: 1
Printing stuff: hello on array position: 2
Printing stuff: 5.6 on array position: 3
Printing stuff: true on array position: 4
```

As you can see, it called the `printStuff()` function for every element in the array. And we can also use the index, it is the second parameter. We don't need to control the flow of the loop here and we cannot get stuck at a certain point. We just need to specify what function needs to be executed for every element. And the element will be input for this function. This is used a lot, especially for a more functional programming style in which many methods get chained, for example, to process data.

Filtering an array

We can use the built-in `filter()` method on an array to alter which values are in the array. The filter method takes a function as an argument, and this function should return a Boolean. If the Boolean has the value `true`, the element will end up in the filtered array. If the Boolean has the value `false`, the element will be left out. You can see how it works here:

```
let arr = ["squirrel", 5, "Tjed", new Date(), true];

function checkString(element, index) {
  return typeof element === "string";
}

let filterArr = arr.filter(checkString);
console.log(filterArr);
```

This will log to the console:

```
[ 'squirrel', 'Tjed' ]
```

It is important to realize that the original array has not changed, the `filter()` method returns a new array with the elements that made it through the filter. We capture it here in the variable `filterArr`.

Checking a condition for all elements

You can use the `every()` method to see whether something is true for all elements in the array. If that is the case, the `every()` method will return `true`, else it will return `false`. We are using the `checkString()` function and array from the previous example here:

```
console.log(arr.every(checkString));
```

This will log `false`, since not all elements are of type `string` in the array.

Replacing part of an array with another part of the array

The `copyWithin()` method can be used to replace a part of the array with another part of the array. In the first example we specify 3 arguments. The first one is the target position, to which the values get copied. The second one is the start of what to copy to the target position and the last one is the end of the sequence that will be copied to the target position; this last index is not included. Here we are only going to override position 0 with whatever is in position 3:

```
arr = ["grapefruit", 4, "hello", 5.6, true];
arr.copyWithin(0, 3, 4);
```

`arr` becomes:

```
[ 5.6, 4, 'hello', 5.6, true ]
```

If we specify a range with length 2, the first two elements after the starting position get overridden:

```
arr = ["grapefruit", 4, "hello", 5.6, true];
arr.copyWithin(0, 3, 5);
```

And now `arr` becomes:

```
[ 5.6, true, 'hello', 5.6, true ]
```

We can also not specify an end at all; it will take the range to the end of the string:

```
let arr = ["grapefruit", 4, "hello", 5.6, true, false];
arr.copyWithin(0, 3);
console.log(arr);
```

This will log:

```
[ 5.6, true, false, 5.6, true, false ]
```

It is important to keep in mind that this function changes the *content* of the original array, but will never change the *length* of the original array.

Mapping the values of an array

Sometimes you'll need to change all the values in an array. With the array `map()` method you can do just that. This method will return a new array with all the new values. You'll have to say how to create these new values. This can be done with the arrow function. It is going to execute the arrow function for every element in the array, so for example:

```
let arr = [1, 2, 3, 4];
let mapped_arr = arr.map(x => x + 1);
console.log(mapped_arr);
```

This is what the console output with the new mapped array looks like:

```
[ 2, 3, 4, 5 ]
```

Using the arrow function, the `map()` method has created a new array, in which each of the original array values has been increased by 1.

Finding the last occurrence in an array

We can find occurrences with `indexOf()` as we have seen already. To find the last occurrence, we can use the `lastIndexOf()` method on an array, just as we did for string.

It will return the index of the last element with that value, if it can find it at all:

```
let bb = ["so", "bye", "bye", "love"];
console.log(bb.lastIndexOf("bye"));
```

This will log 2, because the index 2 holds the last bye variable. What do you think you'll get when you ask for the last index of something that's not there?

```
let bb = ["so", "bye", "bye", "love"];
console.log(bb.lastIndexOf("hi"));
```

That's right (hopefully)! It's -1.

Practice exercise 8.2

Remove duplicates from the array using `filter()` and `indexOf()`. The starting array is:

```
[ "Laurence", "Mike", "Larry", "Kim", "Joanne", "Laurence", "Mike",
  "Laurence", "Mike", "Laurence", "Mike" ]
```

Using the array `filter()` method, this will create a new array using the elements that pass the test condition implemented by the function. The final result will be:

```
[ 'Laurence', 'Mike', 'Larry', 'Kim', 'Joanne' ]
```

Take the following steps:

1. Create an array of names of people. Make sure you include duplicates. The exercise will remove the duplicate names.
2. Using the `filter()` method, assign the results of each item from the array as arguments within an anonymous function. Using the value, index, and array arguments, return the filtered result. You can set the return value to true temporarily as this will build the new array with all the results in the original array.
3. Add a `console.log` call within the function that will output the index value of the current item in the array. Also add the value so you can see the results of the item value that has the current index number and the first matching result from the array's index value.

- Using `indexOf()` the current value returns the index value of the item and applies the condition to check to see if it matches the original index value. This condition will only be true on the first result so all subsequent duplicates will be false and not get added to the new array. `false` will not return the value into the new array. The duplicates will all be false since the `indexOf()` only gets the first match in the array.
- Output the new, unique value array onto the console.

Practice exercise 8.3

Using the array `map()` method, update an array's contents. Take the following steps:

- Create an array of numbers.
- Using the array `map` method and an anonymous function, return an updated array, multiplying all the numbers in the array by 2. Output the result into the console.
- As an alternative method, use the arrow function format to multiply each element of the array by 2 with the array `map()` method in one line of code.
- Log the result onto the console.

String methods

We have worked with strings already and chances are that you have run into some of the methods on strings by now. There are a few we didn't address specifically just yet and we are going to discuss them in this section.

Combining strings

When you want to concatenate strings, you can use the `concat()` method. This does not change the original string(s); it returns the combined result as a string. You will have to capture the result in a new variable, else it will get lost:

```
let s1 = "Hello ";
let s2 = "JavaScript";
let result = s1.concat(s2);
console.log(result);
```

This will log:

```
Hello JavaScript
```

Converting a string to an array

With the `split()` method we can convert a string to an array. Again, we will have to capture the result; it is not changing the original string. Let's use the previous result containing `Hello JavaScript`. We will have to tell the `split` method on what string it should split. Every time it encounters that string, it will create a new array item:

```
let result = "Hello JavaScript";
let arr_result = result.split(" ");
console.log(arr_result);
```

This will log:

```
[ 'Hello', 'JavaScript' ]
```

As you can see, it creates an array of all the elements separated by a space. We can split by any character, for example a comma:

```
let favoriteFruits = "strawberry,watermelon,grapefruit";
let arr_fruits = favoriteFruits.split(",");
console.log(arr_fruits);
```

This will log:

```
[ 'strawberry', 'watermelon', 'grapefruit' ]
```

It has now created an array with 3 items. You can split on anything, and the string you are splitting on is left out of the result.

Converting an array to a string

With the `join()` method you can convert an array to a string. Here is a basic example:

```
let letters = ["a", "b", "c"];
let x = letters.join();
console.log(x);
```


This will log:

```
a,b,c
```

The type of `x` is `string`. If you want something else other than a comma, you can specify that, like this:

```
let letters = ["a", "b", "c"];
let x = letters.join('-');
console.log(x);
```

This will use the `-` instead of the comma. This is the result:

```
a-b-c
```

This can be nicely combined with the `split()` method that we covered in the previous section, which does the reverse and converts a string into an array.

Working with index and positions

Being able to find out what index a certain substring is at within your string is very useful. For example, when you need to search for a certain word through the user input of a log file and create a substring starting at that index. Here is an example of how to find the index of a string. The `indexOf()` method returns the index, a single number, of the first character of the substring:

```
let poem = "Roses are red, violets are blue, if I can do JS, then you can too!";
let index_re = poem.indexOf("re");
console.log(index_re);
```

This is logging 7 to the console, because the first occurrence of `re` is in `are`, and the `re` begins at index 7. When it can't find an index, it will return `-1`, like this example:

```
let indexNotFound = poem.indexOf("python");
console.log(indexNotFound);
```

It is logging `-1` to indicate that the string we are searching for doesn't occur in the target string. Often you will write an `if` check to see whether it's `-1` before dealing with the result. For example:

```
if(poem.indexOf("python") !== -1) {
  // do stuff
}
```

An alternative way of searching for a particular substring within a string is to use the `search()` method:

```
let searchStr = "When I see my fellow, I say hello";
let pos = searchStr.search("lo");
console.log(pos);
```

This will log 17, because that is the index of `lo` in `fell`ow. Much like `indexOf()`, if it cannot find it, it will return `-1`. This is the case for this example:

```
let notFound = searchStr.search("JavaScript");
console.log(notFound);
```

`search()` will accept a regex format as input, whereas `indexOf()` just takes a string. `indexOf()` is faster than the `search()` method, so if you just need to look for a string, use `indexOf()`. If you need to look for a string pattern, you'll have to use the `search()` method.



Regex is a special syntax for defining string patterns, with which you can replace all occurrences, but we'll deal with that in *Chapter 12, Intermediate JavaScript*.

Moving on, the `indexOf()` method is returning the index of the first occurrence, but similarly, we also have a `lastIndexOf()` method. It returns the index where the argument string occurs last. If it cannot find it, it returns `-1`. Here is an example:

```
let lastIndex_re = poem.lastIndexOf("re");
console.log(lastIndex_re);
```

This returns 24; this is the last time `re` appears in our poem. It is the second `are`.

Sometimes you will have to do the reverse; instead of looking for what index a string occurs at, you will want to know what character is at a certain index position. This is where the `charAt(index)` method comes in handy, where the specified index position is taken as an argument:

```
let pos1 = poem.charAt(10);
console.log(pos1);
```

This is logging `r`, because the character at index 10 is the `r` of `red`. If you are asking for the position of an index that is out of the range of the string, it will return an empty string, as is happening in this example:

```
let pos2 = poem.charAt(1000);
console.log(pos2);
```

This will log an empty line to the screen, and if you ask for the type of `pos2`, it will return `string`.

Creating substrings

With the `slice(start, end)` method we can create substrings. This does not alter the original string, but returns a new string with the substring. It takes two parameters, the first is the index at which it starts and the second is the end index. If you leave out the second index it will just continue until the end of the string from the start. The end index is not included in the substring. Here is an example:

```
let str = "Create a substring";
let substr1 = str.slice(5);
let substr2 = str.slice(0,3);
console.log("1:", substr1);
console.log("2:", substr2);
```

This will log:

```
1: e a substring
2: Cre
```

The first one only has one argument, so it starts at index 5 (which holds an `e`) and grabs the rest of the string from there. The second one has two arguments, 0 and 3. `C` is at index 0 and `a` is at index 3. Since the last index is not included in the substring, it will only return `Cre`.

Replacing parts of the string

If you need to replace a part of the string, you can use the `replace(old, new)` method. It takes two arguments, one string to look for in the string and one new value to replace the old value with. Here is an example:

```
let hi = "Hi buddy";
let new_hi = hi.replace("buddy", "Pascal");
console.log(new_hi);
```

This will log to the console `Hi Pascal`. If you don't capture the result, it is gone, because the original string will not get changed. If the string you are targeting doesn't appear in the original string, the replacement doesn't take place and the original string will be returned:

```
let new_h12 = h1.replace("not there", "never there");
console.log(new_h12);
```

This logs `Hi buddy`.

One last note here, it is only changing the first occurrence by default. So this example will only replace the first `hello` in the new string:

```
let s3 = "hello hello";
let new_s3 = s3.replace("hello", "oh");
console.log(new_s3);
```

This logs `oh hello`. If we wanted to replace all the occurrences, we could use the `replaceAll()` method. This will replace all occurrences with the specified new string, like this:

```
let s3 = "hello hello";
let new_s3 = s3.replaceAll("hello", "oh");
console.log(new_s3);
```

This logs `oh oh`.

Uppercase and lowercase

We can change the letters of a string with the `toUpperCase()` and `toLowerCase()` built-in methods on `string`. Again, this is not changing the original string, so we'll have to capture the result:

```
let low_bye = "bye!";
let up_bye = low_bye.toUpperCase();
console.log(up_bye);
```

This logs:

```
BYE!
```

It converts all the letters to uppercase. We can do the opposite with `toLowerCase()`:

```
let caps = "HI HOW ARE YOU?";
let fixed_caps = caps.toLowerCase();
console.log(fixed_caps);
```

This will log:

```
hi how are you?
```

Let's make it a bit more complicated and say that we'd like the first letter of the sentence to be capitalized. We can do this by combining some of the methods we have seen already right now:

```
let caps = "HI HOW ARE YOU?";
let fixed_caps = caps.toLowerCase();
let first_capital = fixed_caps.charAt(0).toUpperCase().concat(fixed_
caps.slice(1));
console.log(first_capital);
```

We are chaining the methods here; we first grab the first character of `fixed_caps` with `charAt(0)` and then make it uppercase by calling `toUpperCase()` on it. We then need the rest of the string and we get it by concatenating `slice(1)`.

The start and end of a string

Sometimes you would want to check what a string starts or ends with. You've guessed it, there are built-in methods for this on `string`. We can imagine this chapter is tough to work through, so here is a little encouragement and an example at the same time:

```
let encouragement = "You are doing great, keep up the good work!";
let bool_start = encouragement.startsWith("You");
console.log(bool_start);
```

This will log `true` to the console, because the sentence starts with `You`. Careful here, because it is case sensitive. So the following example will log `false`:

```
let bool_start2 = encouragement.startsWith("you");
console.log(bool_start2);
```

If you don't care about uppercase or lowercase, you can use the previously discussed `toLowerCase()` method here, so that it will not take uppercase or lowercase into account:

```
let bool_start3 = encouragement.toLowerCase().startsWith("you");
console.log(bool_start3);
```

We are now converting the string to lowercase first, so we know we are only working with lowercase characters here. However, an important side note here is that this will affect performance for huge strings.



Again, a more performance-friendly alternative is to use `regex`. Getting excited for *Chapter 12, Intermediate JavaScript*, yet?

To end this section, we can do the same thing for checking whether a string ends with a certain string. You can see it in action here:

```
let bool_end = encouragement.endsWith("Something else");
console.log(bool_end);
```

Since it doesn't end with `Something else`, it will return `false`.

Practice exercise 8.4

Using string manipulation, create a function that will return a string with the first letter of all the words capitalized and the rest of the letters in lowercase. You should transform the sentence `this will be capiTalized for each word` into `This Will Be Capitalized For Each Word`:

1. Create a string with several words that have letters with different cases, a mix of upper and lowercase words.
2. Create a function that gets a string as an argument, which will be the value that we will manipulate.
3. In the function first transform everything to lowercase letters.
4. Create an empty array that can hold the values of the words when we capitalize them.
5. Convert the phrase into words in an array using the `split()` method.
6. Loop through each one of the words that are now in the new array, so you can select each one independently. You can use `forEach()` for this.

- Using `slice()` isolate the first letter in each word, then transform it to uppercase. Again using `slice()`, get the remaining value of the word without including the first letter. Then concatenate the two together to form the word that is now capitalized.
- Add the new capitalized word into the blank array that you created. By the end of the loop you should have an array with all the words as separate items in the new array.
- Take the array of updated words and using the `join()` method, transform them back into a string with spaces between each word.
- Return the value of the newly updated string with capitalized words that can then be output into the console.

Practice exercise 8.5

Using the `replace()` string method, complete this vowel replacer exercise by replacing the vowels in a string with numbers. You can start with this string:

```
1 I love JavaScript
```

And turn it into something like the following:

```
2 I3v1 j0v0scr2pt
```

Take the following steps:

- Create the previously specified string, and convert it to lowercase.
- Create an array containing the vowels: a, e, i, o, u.
- Loop through each letter you have in the array, and output the current letter into the console so that you can see which letter will be converted.
- Within the loop, using `replaceAll()` update each vowel substring with the index value of the letter from the vowel array.



Using `replace()` will only replace the first occurrence; if you use `replaceAll()` this will update all matching results.

- Once the loop completes output the result of the new string into the console.

Number methods

Let's move on to some built-in methods on the `Number` object. We have seen a few already, these are so popular that some of them have been made into global methods.

Checking if something is (not) a number

This can be done with `isNaN()`. We have seen this already when we talked about global methods, we can use this method without `Number` in front of it. Often you will want to do the opposite, you can negate the function with an `!` in front of it:

```
let x = 34;
console.log(isNaN(x));
console.log(!isNaN(x));
let str = "hi";
console.log(isNaN(str));
```

This will log to the console:

```
false
true
true
```

Since `x` is a number, `isNaN` will be `false`. But this result negated becomes `true`, since `x` is a number. The string `hi` is not a number, so it will become `false`. And this one?

```
let str1 = "5";
console.log(isNaN(str1));
```

Some funky stuff is going on here, even though `5` is between quotation marks, JavaScript still sees that it's a `5` and it will log `false`. At this point, I'm sure you'd wish your partner, family, and coworkers are as understanding and forgiving as JavaScript.

Checking if something is finite

By now you might be able to guess the name of the method on `Number` that checks whether something is finite. It is a very popular one and has been made into a global method as well, and its name is `isFinite()`. It returns `false` for `NaN`, `Infinity`, and `undefined`, and `true` for all other values:

```
let x = 3;
let str = "finite";
console.log(isFinite(x));
console.log(isFinite(str));
console.log(isFinite(Infinity));
console.log(isFinite(10 / 0));
```

This will log:

```
true
false
false
false
```

The only finite number in this list is `x`. The others are not finite. A string is a not a number and is therefore not finite. `Infinity` is not finite and `10` divided by `0` returns `Infinity` (not an error).

Checking if something is an integer

Yes, this is done with `isInteger()`. Unlike `isNaN()` and `isFinite()`, `isInteger()` has not been made global and we will have to refer to the `Number` object to use it. It really does what you think it would: it returns `true` if the value is an integer and `false` when it's not.

```
let x = 3;
let str = "integer";
console.log(Number.isInteger(x));
console.log(Number.isInteger(str));
console.log(Number.isInteger(Infinity));
console.log(Number.isInteger(2.4));
```

This will log:

```
true
false
false
false
```

Since the only integer in the list is `x`.

Specifying a number of decimals

We can tell JavaScript how many decimals to use with the `toFixed()` method. This is different from the rounding methods in `Math`, since we can specify the number of decimals here. It doesn't change the original value, so we'll have to store the result:

```
let x = 1.23456;
let newX = x.toFixed(2);
```

This will only leave two decimals, so the value of `newX` will be `1.23`. It rounds the number normally; you can see this when we ask for one more decimal:

```
let x = 1.23456;
let newX = x.toFixed(3);
console.log(x, newX);
```

This logs `1.23456 1.235` as output.

Specifying precision

There is also a method to specify precision. Again this is different from the rounding methods in the `Math` class, since we can specify the total number of numbers to look at. This comes down to JavaScript looking at the total number of numbers. It is also counting the ones before the dot:

```
let x = 1.23456;
let newX = x.toPrecision(2);
```

So the value of `newX` will be `1.2` here. And also here, it is rounding the numbers:

```
let x = 1.23456;
let newX = x.toPrecision(4);
console.log(newX);
```

This will log `1.235`.

Now, let's move on and talk about some related mathematical methods!

Math methods

The `Math` object has many methods that we can use to do calculations and operations on numbers. We will go over the most important ones here. You can see all the available ones when you use an editor that shows suggestions and options during typing.

Finding the highest and lowest number

There is a built-in method `max()` to find the highest number among the arguments. You can see it here:

```
let highest = Math.max(2, 56, 12, 1, 233, 4);
console.log(highest);
```

It logs 233, because that's the highest number. In a similar way, we can find the lowest number:

```
let lowest = Math.min(2, 56, 12, 1, 233, 4);
console.log(lowest);
```

This will log 1, because that is the lowest number. If you try to do this with non-numeric arguments, you will get `NaN` as a result.

```
let highestOfWords = Math.max("hi", 3, "bye");
console.log(highestOfWords);
```

It is not giving 3 as output, because it is not ignoring the text but concluding that it cannot determine whether hi should be higher or lower than 3. So it returns `NaN` instead.

Square root and raising to the power of

The method `sqrt()` is used to calculate the square root of a certain number. Here you can see it in action:

```
let result = Math.sqrt(64);
console.log(result);
```

This will log 8, because the square root of 64 is 8. This method works just like the mathematics you learned in school. In order to raise a number to a certain power ($\text{base}^{\text{exponent}}$, for example 2^3), we can use the `pow(base, exponent)` function. Like this:

```
let result2 = Math.pow(5, 3);
console.log(result2);
```

We are raising 5 to the power of 3 here (5^3), so the result will be 125, which is the result of $5 \times 5 \times 5$.

Turning decimals into integers

There are different ways to turn decimals into integers. Sometimes you will want to round a number. This you can do with the `round()` method:

```
let x = 6.78;
let y = 5.34;

console.log("X:", x, "becomes", Math.round(x));
console.log("Y:", y, "becomes", Math.round(y));
```

This will log:

```
X: 6.78 becomes 7
Y: 5.34 becomes 5
```

As you can see it is using normal rounding here. It is also possible that you don't want to round down, but up. For example, if you need to calculate how many wood boards you need and you conclude that you need 1.1, 1 is not going to be enough to do the job. You'll need 2. In this case, you can use the `ceil()` method (referring to ceiling):

```
console.log("X:", x, "becomes", Math.ceil(x));
console.log("Y:", y, "becomes", Math.ceil(y));
```

This will log:

```
X: 6.78 becomes 7
Y: 5.34 becomes 6
```

The `ceil()` method is always rounding up to the first integer it encounters. We have used this before when we were generating random numbers! Careful with negative numbers here, because -5 is higher than -6. This is how it works, as you can see in this example:

```
let negativeX = -x;
let negativeY = -y;

console.log("negativeX:", negativeX, "becomes", Math.ceil(negativeX));
console.log("negativeY:", negativeY, "becomes", Math.ceil(negativeY));
```

This will log:

```
negativeX: -6.78 becomes -6
negativeY: -5.34 becomes -5
```

The `floor()` method is doing the exact opposite of the `ceil()` method. It rounds down to the nearest integer number, as you can see here:

```
console.log("X:", x, "becomes", Math.floor(x));
console.log("Y:", y, "becomes", Math.floor(y));
```

This will log:

```
X: 6.78 becomes 6
Y: 5.34 becomes 5
```

Again, careful with negative numbers here, because it can feel counterintuitive:

```
console.log("negativeX:", negativeX, "becomes", Math.floor(negativeX));
console.log("negativeY:", negativeY, "becomes", Math.floor(negativeY));
```

This logs:

```
negativeX: -6.78 becomes -7
negativeY: -5.34 becomes -6
```

And then one last method, `trunc()`. This gives the exact same result as `floor()` for positive numbers, but it gets to these results differently. It is not rounding down, it is simply only returning the integer part:

```
console.log("X:", x, "becomes", Math.trunc(x));
console.log("Y:", y, "becomes", Math.trunc(y));
```

This will log:

```
X: 6.78 becomes 6
Y: 5.34 becomes 5
```

When we use negative numbers for `trunc()` we can see the difference:

```
negativeX: -6.78 becomes -6
negativeY: -5.34 becomes -5
```

So whenever you need to round down, you'll have to use `floor()`, if you need the integer part of the number, you'll need `trunc()`.

Exponent and logarithm

The exponent is the number to which a base is being raised. We use `e` (Euler's number) a lot in mathematics, this is what the `exp()` method in JavaScript does. It returns the number to which `e` must be raised to get the input. We can use the `exp()` built-in method of `Math` to calculate the exponent, and the `log()` method to calculate the natural logarithm. You can see an example here:

```
let x = 2;
let exp = Math.exp(x);
console.log("Exp:", exp);
let log = Math.log(exp);
console.log("Log:", log);
```

This will log:

```
Exp: 7.38905609893065
Log: 2
```

Don't worry if you can't follow along mathematically at this point. You'll figure this out whenever you'll need it for your programming.

Practice exercise 8.6

Experiment with the `Math` object with these steps:

1. Output the value of `PI` into the console using `Math`.
2. Using `Math` get the `ceil()` value of 5.7, get the `floor()` value of 5.7, get the round value of 5.7, and output it into the console.
3. Output a random value into the console.
4. Use `Math.floor()` and `Math.random()` to get a number from 0 to 10.
5. Use `Math.floor()` and `Math.random()` to get a number from 1 to 10.
6. Use `Math.floor()` and `Math.random()` to get a number from 1 to 100.
7. Create a function to generate a random number using the parameters of `min` and `max`. Run that function 100 times, returning into the console a random number from 1 to 100 each time.

Date methods

In order to work with dates in JavaScript we use the built-in `Date` object. This object contains a lot of built-in functions to work with dates.

Creating dates

There are different ways to create a date. One way to create dates is by using the different constructors. You can see some examples here:

```
let currentDateTime = new Date();
console.log(currentDateTime);
```

This will log the current date and time, in this case:

```
2021-06-05T14:21:45.625Z
```

But, this way we are not using the built-in method, but the constructor. There is a built-in method, `now()`, that returns the current date and time, similar to what the no argument constructor does:

```
let now2 = Date.now();
console.log(now2);
```

This will log the current time, represented in seconds since January 1st 1970. This is an arbitrary date representing the Unix epoch. In this case:

```
1622902938507
```

We can add 1,000 milliseconds to the Unix epoch time:

```
let milliDate = new Date(1000);
console.log(milliDate);
```

It will log:

```
1970-01-01T00:00:01.000Z
```

JavaScript can also convert many string formats to a date. Always mind the order in which days and months of dates are presented in the date format and the interpreter of JavaScript. This can vary depending on the region:

```
let stringDate = new Date("Sat Jun 05 2021 12:40:12 GMT+0200");
console.log(stringDate);
```

This will log:

```
2021-06-05T10:40:12.000Z
```

And lastly, you can also specify a certain date using the constructor:

```
let specificDate = new Date(2022, 1, 10, 12, 10, 15, 100);
console.log(specificDate);
```

This will log:

```
2022-02-10T12:10:15.100Z
```

Please mind this very important detail here, the second parameter is the month. 0 is for January and 11 is for December.

Methods to get and set the elements of a date

Now we have seen how to create dates, we'll learn how to get certain parts of dates. This can be done with one of the many get methods. Which you will use depends on the part you need:

```
let d = new Date();
console.log("Day of week:", d.getDay());
console.log("Day of month:", d.getDate());
console.log("Month:", d.getMonth());
console.log("Year:", d.getFullYear());
console.log("Seconds:", d.getSeconds());
console.log("Milliseconds:", d.getMilliseconds());
console.log("Time:", d.getTime());
```

This will log right now:

```
Day of week: 6
Day of month: 5
Month: 5
Year: 2021
Seconds: 24
Milliseconds: 171
Time: 1622903604171
```

The time is so high because it's the number of milliseconds since January 1st 1970. You can change the date in a similar way with a set method. Important to note here is that the original date object gets changed with these set methods:

```
d.setFullYear(2010);
console.log(d);
```


We have changed the year of our date object to 2010. This will output:

```
2010-06-05T14:29:51.481Z
```

We can also change the month. Let's add the below snippet to our change of the year code. This will change it to October. Please mind that while I'm doing this, I run the code again and again, so the minutes and smaller units of time will vary in the examples when I haven't set these yet:

```
d.setMonth(9);
console.log(d);
```

It will log:

```
2010-10-05T14:30:39.501Z
```

This is a weird one, in order to change the day, we have to call the `setDate()` method and not the `setDay()` method. There is no `setDay()` method, since the day of the week is deducted from the specific date. We cannot change that September 5th 2021 is a Sunday. We can change the number of days of the month though:

```
d.setDate(10);
console.log(d);
```

This will log:

```
2010-10-10T14:34:25.271Z
```

We can also change the hours:

```
d.setHours(10);
console.log(d);
```

Now it will log:

```
2010-10-10T10:34:54.518Z
```

Remember how JavaScript doesn't like to crash? If you call `setHours()` with a number higher than 24, it will roll over to the next date (1 per 24 hours) and after using the modulo operator, whatever is left over from hours % 24 will be the hours. The same process applies for minutes, seconds, and milliseconds.

The `setTime()` actually overrides the complete date with the inserted epoch time:

```
d.setTime(162289770682);
console.log(d);
```

This will log:

```
2021-06-05T10:42:50.682Z
```

Parsing dates

With the built-in `parse()` method we can parse epoch dates from strings. It accepts many formats, but again you will have to be careful with the order of days and months:

```
let d1 = Date.parse("June 5, 2021");
console.log(d1);
```

This will log:

```
1622851200000
```

As you can see it ends with many zeros, because no time or seconds are specified in our string. And here is another example of a completely different format:

```
let d2 = Date.parse("6/5/2021");
console.log(d2);
```

This will also log:

```
1622851200000
```

The input for the `parse` is ISO formats of dates. Quite a few formats can be parsed to string, but you'll have to be careful. The result might depend on the exact implementation. Make sure that you know what the format of the incoming string is, so that you don't confuse months and days, and make sure that you know the behavior of the implementations. This can only be done reliably if you know what the string format is. So for example when you need to convert data coming from your own database or website's date picker.

Converting a date to a string

We can also convert dates back to strings. For example with these methods:

```
console.log(d.toString());
```

This will log the day in written format:

```
Sat Jun 05 2021
```

This is another method that converts it differently:

```
console.log(d.toLocaleDateString());
```

It will log:

```
6/5/2021
```

Practice exercise 8.7

Output the date with the full month name into the console. When converting to or from arrays, remember that they are zero-based:

1. Set up a date object, which can be any date in the future or past. Log the date out into the console to see how it is typically output as a date object.
2. Set up an array with all the named months of the year. Keep them in sequential order so that they will match the date month output.
3. Get the day from the date object value, using `getDate()`.
4. Get the year from the date object value, using `getFullYear()`.
5. Get the month of the date object value, using `getMonth()`.
6. Set up a variable to hold the date of the date object and output the month using the numeric value as the index for the array month name. Due to arrays being zero-based and the month returning a value of 1-12, the result needs to be subtracted by one.
7. Output the result into the console.

Chapter projects

Word scrambler

Create a function that returns a value of a word and scrambles the letter order with `Math.random()`:

1. Create a string that will hold a word value of your choice.
2. Create a function that can intake a parameter of the string word value.

3. Just like an array, strings also have a length by default. You can use this length to set the loop maximum value. You will need to create a separate variable to hold this value as the length of the string will be decreasing as the loop continues.
4. Create an empty temporary string variable that you can use to hold the new scrambled word value.
5. Create a for loop that will iterate the number of letters within the string parameter starting at 0 and iterating until the original length value of the string is reached.
6. Create a variable that will randomly select one letter using its index value, with `Math.floor()` and `Math.random()` multiplied by the current length of the string.
7. Add the new letter to the new string and remove it from the original string.
8. Using `console.log()` output the newly constructed string from the random letters and output to the console both the original string and the new one as the loop continues.
9. Update the original string by selecting the substring from the index value and adding it to the remaining string value from the index plus one onward. Output the new original string value with the removed characters.
10. As you loop through the content you will see a countdown of the remaining letters, the new scrambled version of the word as it is built, and the decreasing letters in the original word.
11. Return the final result and invoke the function with the original string word as an argument. Output this to the console.

Countdown timer

Create code for a countdown timer that can be executed in the console window, and will show the total milliseconds, days, hours, minutes, and seconds remaining until a target date is reached:

1. Create an end date that you want to count down to. Format it in a date type format within a string.
2. Create a countdown function that will parse the `endDate()` and subtract the current date from that end date. This will show the total in milliseconds. Using `Date.parse()` you can convert a string representation of a date to a numeric value as a number of milliseconds since January 1, 1970, 00:00:00 UTC.

3. Once you have the total milliseconds, to get the days, hours, minutes, and seconds you can take the following steps:
 - To get days you can divide the number of milliseconds in a date, removing the remainder with `Math.floor()`.
 - To get hours you can use modulus to capture just the remainder once the total days are removed.
 - To get minutes you can use the value of milliseconds in a minute and using the modulus capture the remainder.
 - Do the same for seconds by dividing the number by seconds in milliseconds and getting the remainder. If you use `Math.floor()` you can round down removing any remaining decimal places that will be shown in the lower values.
4. Return all the values within an object with property names indicating what the unit of time the values refer to.
5. Create a function to use a `setTimeout()` to run the `update()` function every second (1,000 milliseconds). The `update()` function will create a variable that can temporarily hold the object return values of `countdown()`, and create an empty variable that will be used to create the output values.
6. Within the same function, using the `for` loop get all the properties and values of the `temp` object variable. As you iterate through the object update the output string to contain the property name and property value.
7. Using `console.log()`, print the output result string into the console.

Self-check quiz

1. Which method will decode the following?

```
var c = "http://www.google.com?id=1000&n=500";
var e = encodeURIComponent(c);
```

- a. `decodeURIComponent(e)`
- b. `e.decodeURIComponent()`
- c. `decoderURIComponent(c)`
- d. `decoderURIComponent(e)`

2. What will be output into the console from the following syntax?

```
const arr = ["hi", "world", "hello", "hi", "hi", "hi World", "Hi"];
console.log(arr.lastIndexOf("hi"));
```

3. What is the result of the below code in the console?

```
const arr = ["Hi", "world", "hello", "Hi", "hi", "hi World", "Hi"];
arr.copyWithin(0, 3, 5);
console.log(arr);
```

4. What is the result of the below code in the console?

```
const arr = ["Hi", "world", "hello", "Hi", "hi", "hi World", "Hi"];
const arr2 = arr.filter((element, index) => {
  const ele2 = element.substring(0, 2);
  return (ele2 == "hi");
});
console.log(arr2);
```

Summary

In this chapter we have dealt with many built-in methods. These are methods that are handed to us by JavaScript and that we can use for things that we'll often need. We went over the most used global built-in methods, which are so common they can be used without being prepended by the object they belong to.

We also discussed array methods, string methods, number methods, math methods, and date methods. You'll find yourself using these methods a lot and chaining them (whenever they return a result) when you get more comfortable with JavaScript.

Now we've become familiar with many of JavaScript's core features, we'll spend the next couple of chapters diving into how it works alongside HTML and the browser to bring web pages to life!

9

The Document Object Model

The **Document Object Model (DOM)** is a lot more exciting than it may sound at first. In this chapter, we will introduce you to the DOM. This is a fundamental concept you will need to understand before working with JavaScript on web pages. It grabs an HTML page and turns it into a logical tree. If you do not know any HTML, no worries. We start with an HTML crash course section that you can skip if you are familiar with HTML.

Once we are sure that we are on the same page with HTML knowledge, we will introduce you to the **Browser Object Model (BOM)**. The BOM holds all the methods and properties for JavaScript to interact with the browser. This is information related to previous pages visited, the size of the window of the browser, and also the DOM.

The DOM contains the HTML elements on the web page. With JavaScript, we can select and manipulate parts of the DOM. This leads to interactive web pages instead of static ones. So, long story short, being able to work with the DOM means you're able to create interactive web pages!

We will cover the following topics:

- HTML crash course
- Introducing the BOM
- Introducing the DOM
- Types of DOM elements
- Selecting page elements

We can imagine you cannot wait to get started, so let's dive into it.



Note: exercise, project and self-check quiz answers can be found in the *Appendix*.

HTML crash course

Hyper-Text Markup Language (HTML) is the language that shapes the content of web pages. Web browsers understand HTML code and represent it in the format we are used to seeing: web pages. Here is a little very basic HTML example:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Tab in the browser</title>
  </head>
  <body>
    <p>Hello web!</p>
  </body>
</html>
```

This is what this basic web page looks like:

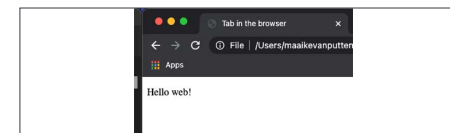


Figure 9.1: Basic website

HTML code consists of elements. These elements contain a tag and attributes. We will explain these fundamental concepts in the coming sections.

HTML elements

As you can see, HTML consists of words between `<angle brackets>`, or elements. Any element that gets opened needs to be closed. We open with `<elementname>` and we close with `</elementname>`.

Everything in between that is part of the element. There are a few exceptions with regards to the closing, but you will run into them at your own pace. In the previous example we had multiple elements, including these two. It is an element with the tag `body` and an inner element with the tag `p`.

```
<body>
  <p>Hello web!</p>
</body>
```

So elements can contain inner elements. Elements can only be closed if all inner elements have been closed. Here is an example to demonstrate that. Here is the right way:

```
<outer>
  <sub>
    <inner>
      </inner>
    </sub>
  </outer>
```

And here is the wrong way:

```
<outer>
  <sub>
    <inner>
      </sub>
    </inner>
  </outer>
```

Please note, these are just made-up element names. In the last example, we close `sub` before we have closed the `inner` element. This is wrong; you must always close the inner elements before closing the outer element. We call inner elements child elements, and outer elements parent elements. Here is some correct HTML:

```
<body>
  <div>
    <p>
      </p>
    </div>
  </body>
```

This isn't correct HTML, because the `div` is closed before its inner element `p`:

```
<body>
  <div>
    <p>
  </div>
  </p>
</body>
```

The different elements represent different pieces of layout. The `p` we just saw represents paragraphs. And another common one is `h1`, which represents a big title. What is more important is to know the three major building elements of every HTML page. The HTML element, the head element, and the body element.

In the HTML element, all the HTML takes place. You can only have one of these in your HTML page. It is the outer element, and all other elements are housed in it. It contains the other two top-level elements: `head` and `body`. If you are ever confused about the order of `head` and `body`, just think of a human; the head is on top of the body.

In the `head` element, we arrange a lot of things that are meant for the browser and not for the user. You can think of certain metadata, such as which JavaScript scripts and which stylesheets need to be included, and what the searching engine should use as a description on the search result page. We will not really be doing a lot with the `head` element as JavaScript developers, other than including scripts.

Here's an example of a basic `head` element:

```
<head>
  <title>This is the title of the browser tab</title>
  <meta name="description" content="This is the preview in google">
  <script src="included.js"></script>
</head>
```

The `body` element is mostly the content that will appear on the web page. There can only be one `body` element in the HTML element. Titles, paragraphs, images, lists, links, buttons, and many more are all elements that we can come across in the `body`. They have their own tag, so for example, `img` for image and `a` for a link. Here is a table including common tags for in the `body`. It is definitely not an exhaustive list.

Tag to open	Tag to end	Description
<p>	</p>	Used to create a paragraph.
<h1>	</h1>	Used to create a header; smaller headers are h2 to h6.
		Generic inline container for content that needs to be separated, for example, for layout purposes.
<a>		Used for hyperlinks.
<button>	</button>	Used for buttons.
<table>	</table>	Creates a table.
<tr>	</tr>	Creates a table row, must be used inside a table.
<td>	</td>	Creates a table data cell inside a row.
		Unordered lists, with bullet points, for example.
		Ordered lists with numbers.
		List items for inside ordered and unordered lists.
<div>	</div>	Section inside the HTML page. It is often used as a container for other styles or sections and can easily be used for special layouts.
<form>	</form>	Creates an HTML form.
<input>	</input>	Creates an input field in which the user can enter information. These can be textboxes, checkboxes, buttons, passwords, numbers, dropdowns, radio buttons, and much more.
<input />	None	Same as input, but written without a closing tag, the / at the end makes it self-closing. This is only possible for a few elements.
 	None	Used to make a line break (go to a new line). It does not need an end tag and is therefore an exception.

Can you figure out what this HTML example does:

```
<html>
<head>
  <title>Awesome</title>
</head>

<body>
  <h1>Language and awesomeness</h1>
  <table>
```

```
<tr>
  <th>Language</th>
  <th>Awesomeness</th>
</tr>
<tr>
  <td>JavaScript</td>
  <td>100</td>
</tr>
<tr>
  <td>HTML</td>
  <td>100</td>
</tr>
</table>
</body>
</html>
```

It creates a web page, with **Awesome** in the tab title. And on the page, it has a big header saying **Language** and **awesomeness**. Then there is a table with three rows and two columns. The first row contains the headers **Language** and **Awesomeness**. The second row holds the values **JavaScript** and **100**, and the third row holds the values **HTML** and **100**.

HTML attributes

The last part of HTML that we will discuss in this crash course is HTML attributes. Attributes influence the element they are specified on. They exist inside the element they are specified on and are assigned a value using an equal sign. For example, the attribute of **a** (which indicates a hyperlink) is the **href**. This specifies where the link is redirecting to:

```
<a href="https://google.com">Ask Google</a>
```

This displays a link with the text **Ask Google**. And when you click it, you will be sent to Google, which can be told by the value of the **href** attribute. This modifies the **a** element. There are many attributes out there, but for now you just need to know that they modify the element they are specified on.

Here is a table with an overview of the most important attributes to get started with HTML and JavaScript. Why these are important will unfold somewhere in the next chapter.

Attribute name	Description	Can be used on which element?
id	Gives an element a unique ID, such as age.	All of them
name	Used to give a custom name to an element.	input, button, form, and quite a few we haven't seen yet
class	Special metadata that can be added to an element. This can result in a certain layout or JavaScript manipulation.	Almost all of them inside body
value	Sets the initial value of the element it is added to.	button, input, li, and a few we haven't seen yet
style	Gives a specified layout to the HTML element it is added to.	All of them

We will introduce you to other attributes when you will need them for practicing your JavaScript magic.

Okay, this has been one of the more brief HTML crash courses out there. There are many great resources to find more information. If you need more information or explanation at this point, create and open an HTML file like the following and take it from there!

```
<!DOCTYPE html >
<html>

<body>

  <a href="https://google.com">Ask google</a>

</body>

</html>
```

We will now go ahead and have a look at the BOM and the different parts of the BOM.

The BOM

The BOM, sometimes also called the **window browser object**, is the amazing "magic" element that makes it possible for your JavaScript code to communicate with the browser.

The window object contains all the properties required to represent the window of the browser, so for example, the size of the window and the history of previously visited web pages. The window object has global variables and functions, and these can all be seen when we explore the window object. The exact implementation of the BOM depends on the browser and the version of the browser. This is important to keep in mind while working your way through these sections.

Some of the most important objects of the BOM we will look into in this chapter are:

- History
- Navigator
- Location

As well as the preceding useful objects, we will also consider the DOM in more detail. But first, we can explore the BOM and see the objects of it with the command `console.dir(window)`. We will enter this in the console of our browser. Let's discuss how to get there first.

We can access the HTML elements and the JavaScript if we go to the inspection panel of our browser. It differs a bit in how you get there, but often the `F12` button while in the browser will do the trick, or else a right-click on the website you want to see the console for and clicking on **Inspect element** or **Inspect** on a macOS device.

You should see a side panel (or if you have changed your settings, a separate window) pop up.

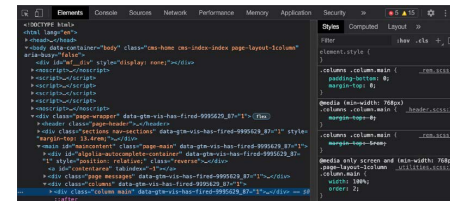


Figure 9.2: Inspecting a page in the browser

Navigate to the **Console** tab, which is next to the **Elements** tab in the image above. You can type the following command and press *Enter* to get information about the window object:

```
console.dir(window);
```

This command will produce a view like the following:

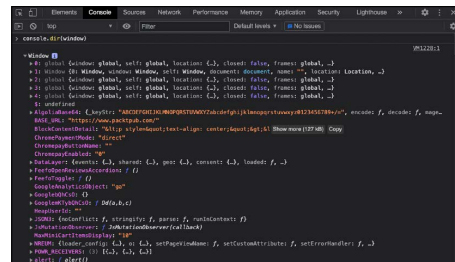


Figure 9.3: Part of the output of console.dir(window) showing the window browser object

The `console.dir()` method shows a list of all the properties of the specified object. You can click on the little triangles to open the objects and inspect them even more.

The BOM contains many other objects. We can access these like we saw when we dealt with objects, so for example, we can get the length of the history (in my browser) accessing the history object of the window and then the length of the history object, like this:

```
window.history.length;
```

After the exercise, we will learn more about the history object.

Practice exercise 9.1

1. Go back to the website you were just viewing and execute the command `console.dir(window)`.
2. Can you find the document object that is nested within the `window` object? Under the root of the `window` object in the console, you can navigate down to the object that is named `document`.
3. Can you find the height and width (in pixels) of your window? You can return the inner and outer window.

Window history object

The window browser object also contains a history object. This object can actually be written without the prefix of window because it has been made globally available, so we can get the exact same object by using the `console.dir(window.history)` or simply the `console.dir(history)` command in the console:

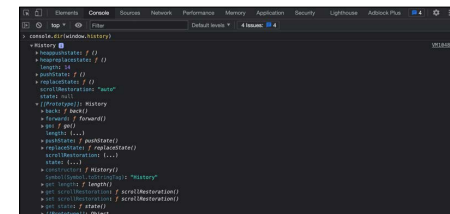


Figure 9.4: History object

This object is actually what you can use to go back to a previous page. It has a built-in function for that called `go`. What happens when you execute this command?

```
window.history.go(-1);
```

Go ahead and try it for yourself in the console of your browser (make sure that you did visit multiple pages in that tab).

Window navigator object

In the window object that we just saw, there is a navigator property. This property is particularly interesting because it contains information about the browser we are using, such as what browser it is and what version we are using, and what operating system the browser is running on.

This can be handy for customizing the website for certain operating systems. Imagine a download button that will be different for Windows, Linux, and macOS.

You can explore it using this command in the console:

```
console.dir(window.navigator);
```

As you can see, we start with accessing the window, because navigator is an object of the window object. So it is a property of the window object, which we specify with the dot in between. In other words, we access these window objects in the same way we do any other object. But, in this case, as navigator is also globally available, we can also access this without window in front of it with this command:

```
console.dir(navigator);
```

Here is what the navigator object might look like:

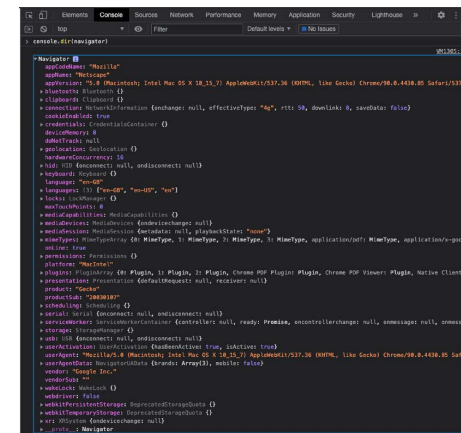


Figure 9.5: The navigator object

Window location object

Another rather interesting and unique property of window is the location object. This contains the URL of the current web page. If you override (parts of) that property, you force the browser to go to a new page! How to do this exactly differs per browser, but the next exercise will guide you through this.

The `location` object consists of a few properties. You can see them by using the command `console.dir(window.location)` or `console.dir(location)` in the console. Here is what the output will look like:

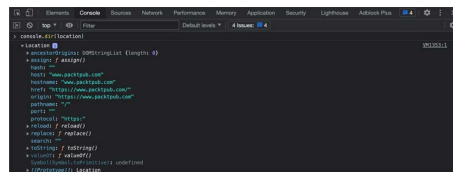


Figure 9.6: The location object

There are many objects on the `location` object, just as with the others we have seen. We can access the nested objects and properties using dot notation (like for other objects we have seen). So, for example, in this browser I can enter the following:

```
location.ancestorOrigins.length;
```

This will get the length of the `ancestorOrigins` object, which represents how many browsing contexts our page is associated with. This can be useful to determine whether the web page is framed in an unexpected context. Not all browsers have this object though; again, this BOM and all the elements of it vary per browser.

Follow the steps in the practice exercise to do such magic yourself.

Practice exercise 9.2

Travel through the `window` object to get to the `location` object, then output the values of the `protocol` and `href` properties of the current file, into the console.

The DOM

The DOM is actually not very complicated to understand. It is a way of displaying the structure of an HTML document as a logical tree. This is possible because of the very important rule that inner elements need to be closed before outer elements get closed.

Here is an HTML snippet:

```
<html>
<head>
<title>Tab in the browser</title>
</head>
<body>
<h1>DOM</h1>
<div>
<p>Hello web!</p>
<a href="https://google.com">Here's a link!</a>
</div>
</body>
</html>
```

And here is how we can translate it to a tree:

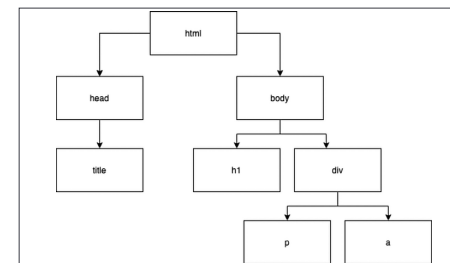


Figure 9.7: Tree structure of the DOM of a very basic web page

As you can see, the most outer element, **html**, is at the top of the tree. The next levels, **head** and **body**, are its children. **head** has only one child: **title**. **body** has two children: **h1** and **div**. And **div** has two children: **p** and **a**. These are typically used for paragraphs and links (or buttons). Clearly, complex web pages have complicated trees. This logical tree and a bunch of extra properties make up a web page's DOM.

The DOM of a real web page wouldn't fit on a page in this book. But if you can draw trees like these in your head, it will be of great help soon.

Additional DOM properties

We can inspect the DOM in a similar fashion as we did the others. We execute the following command in the console of our website (again, the `document` object is globally accessible, so accessing it through the `window` object is possible but not necessary):

```
console.dir(document);
```

In this case, we want to see the `document` object, which represents the DOM:

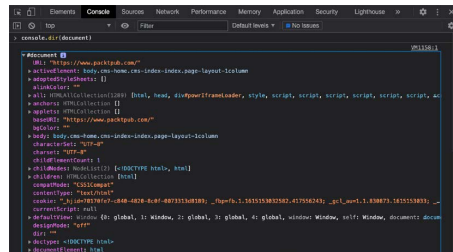


Figure 9.8: The DOM

You really do not need to understand everything you are seeing here, but it is showing many things, among which are the HTML elements and JavaScript code.

Great, right now you have got the basics of the BOM down, and its child object that is most relevant to us right now, the DOM. We saw many properties of the DOM earlier already. For us, it is most relevant to look at the HTML elements in the DOM. The DOM contains all the HTML elements of a web page.

These basics of DOM elements, combined with some knowledge of manipulating and exploring the DOM, will open up so many possibilities.

In the next chapter, we will focus on traversing the DOM, finding elements in the DOM, and manipulating the DOM. The code we will be writing there will really start to look like proper projects.

Selecting page elements

The `document` object contains many properties and methods. In order to work with elements on the page, you'll first have to find them. If you need to change the value of a certain paragraph, you'll have to grab this paragraph first. We call this selecting the paragraph. After selecting, we can start changing it.

To select page elements to use within your JavaScript code and in order to manipulate elements, you can use either the `querySelector()` or `querySelectorAll()` method. Both of these can be used to select page elements either by tag name, ID, or class.

The `document.querySelector()` method will return the first element within the document that matches the specified selectors. If no matching page elements are found, the result `null` is returned. To return multiple matching elements, you can use the method `document.querySelectorAll()`.

The `querySelectorAll()` method will return a static `NodeList`, which represents a list of the document's elements that match the specified group of selectors. We will demonstrate the usage of both `querySelector()` and `querySelectorAll()` with the following HTML snippet:

```
<!doctype html>
<html>
  <head>
    <title>JS Tester</title>
  </head>
  <body>
    <h1 class="output">Hello World</h1>
    <div class="output">Test</div>
  </body>
</html>
```

We are going to select the `h1` element with `querySelector()`. Therefore, if there is more than one, it will just grab the first:

```
const ele1 = document.querySelector("h1");
console.dir(ele1);
```

If you want to select multiple elements, you can use `querySelectorAll()`. This method is going to return all the elements that match the selector in an array. In this example, we are going to look for instances of the output class, which is done by prepending the class name with a dot.

```
const eles = document.querySelectorAll(".output");
console.log(eles);
```

After selecting, you can start using the dynamic features of the DOM: you can manipulate the elements using JavaScript. Content can be changed in the same way a variable's contents can be, elements can be removed or added, and styles can be adjusted. This can all be done with JavaScript and the way the user interacts with the page can affect this. We have seen the two most common methods to select in the DOM here, `querySelector()` and `querySelectorAll()`. You can actually select any element you might need with these. There are lots more, which you'll encounter in the next chapter, along with many of the ways the DOM can be manipulated.

Practice exercise 9.3

Select a page element and update the content, change the style, and add attributes. Create an HTML file containing a page element with a class of output using the following code template:

```
<!DOCTYPE html >
<html>

<div class="output"></div>
<script>
</script>

</html>
```

Within the script tags, make the following changes to the output element:

1. Select the page element as a JavaScript object.
2. Update the `textContent` property of the selected page element.
3. Using the `classList.add` object method, add a class of `red` to the element.
4. Update the `id` property of the element to `tester`.
5. Through the `style` object, add a `backgroundColor` property of `red` to the page element.

6. Get the document URL via `document.URL` and update the text of the output element to contain the value of the document URL. You can log it in the console first to ensure you have the correct value.

Chapter project

Manipulating HTML elements with JavaScript

Take the HTML code below:

```
<div class="output">
  <h1>Hello</h1>
  <div>Test</div>
  <ul>
    <li id="one">One</li>
    <li class="red">Two</li>
  </ul>
  <div>Test</div>
</div>
```

Take the following steps (and experiment further) to understand how HTML elements can be manipulated with JavaScript code.

1. Select the element with the class `output`.
2. Create another JavaScript object called `mainList` and select only the `ul` tag that is within the output element. Update the ID of that `ul` tag to `mainList`.
3. Search for the `tagName` of each `div`, and output them into the console as an array.
4. Using a `for` loop, set the ID of each of the `div` tags to `id` with a numeric value of the order they appear within output. Still within the loop, alternate the color of the contents of each element in output to be red or blue.

Self-check quiz

1. Go to your favorite website and open the browser console. Type `document.body`. What do you see in the console?
2. As we know, with objects, we can write to the property value and assign a new value with the assignment operator. Update the `textContent` property of the `document.body` object on a web page of your choosing to contain the string `Hello World`.

3. Use what we learned about objects to list out BOM object properties and values. Try it on the `document` object.
4. Now do the same for the `window` object.
5. Create an HTML file with an `h1` tag. Use JavaScript and select the page element with the `h1` tag and assign the element into a variable. Update the `textContent` property of the variable to `Hello World`.

Summary

We started this chapter with the basics of HTML. We learned that HTML consists of elements and that these elements can contain other elements. Elements have a tag that specifies the type of element they are and they can have attributes that alter the element or add some metadata to the element. These attributes can be used by JavaScript.

We then had a look at the BOM, which represents the window of the browser that is being used for the web page and contains other objects, such as the `history`, `location`, `navigator`, and `document` objects. The `document` object is referred to as the DOM, which is what you are most likely to be working with. The document contains the HTML elements of the web page.

We also started to consider how we can select document elements and use these to manipulate the web page. This is what we'll continue exploring in the next chapter!

10

Dynamic Element Manipulation Using the DOM

Learning the difficult concepts of the previous chapter will be rewarded in this chapter. We will take our DOM knowledge one step further and learn how to manipulate the DOM elements on the page with JavaScript. First, we need to learn how to navigate the DOM and select the elements we want. We will learn how we can add and change attributes and values, and how to add new elements to the DOM.

You will also learn how to add style to elements, which can be used to make items appear and disappear. Then we will introduce you to events and event listeners. We will start easy, but by the end of this chapter you will be able to manipulate web pages in many ways, and you will have the knowledge to create basic web apps. The sky is the limit after getting this skill down.

Along the way, we will cover the following topics:

- Basic DOM traversing
- Accessing elements in the DOM
- Element click handler
- This and the DOM
- Manipulating element style
- Changing the classes of an element
- Manipulating attributes
- Event listeners on elements
- Creating new elements



Note: exercise, project and self-check quiz answers can be found in the Appendix.

We have learned a lot about the DOM already. In order to interact with our web page and create a dynamic web page, we have to connect our JavaScript skills to the DOM.

Basic DOM traversing

We can traverse the DOM using the document object that we saw in the previous chapter. This document object contains all the HTML and is a representation of the web page. Traversing over these elements can get you to the element you need in order to manipulate it.

This is not the most common way to do it, but this will help understand how it works later. And sometimes, you might actually find yourself needing these techniques as well. Just don't panic: there are other ways to do it, and they will be revealed in this chapter!

Even for a simple HTML piece there are already multiple ways to traverse the DOM. Let's go hunting for treasure in our DOM. We start with this little HTML snippet:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Let's find the treasure</h1>
    <div id="forest">
      <div id="tree1">
        <div id="squirrel"></div>
        <div id="flower"></div>
      </div>
      <div id="tree2">
        <div id="shrubbery">
          <div id="treasure"></div>
        </div>
        <div id="mushroom">
          <div id="bug"></div>
        </div>
      </div>
    </div>
  </body>
</html>
```

We now want to traverse the DOM of this snippet to find the treasure. We can do this by stepping into the document object and navigating our way from there onwards. It is easiest to do this exercise in the console in the browser, because that way you'll get direct feedback about where in the DOM you are.

We can start by using the `body` property from the document. This contains everything that's inside the `body` element. In the console, we'll type:

```
console.dir(document.body);
```

We should get a really long object. There are a few ways from this object to get to our treasure. To do so, let's discuss the `children` and `childNodes` property.



`childNodes` is more a complete term than `children`. `Children` just contain all the HTML elements, so are really the nodes. `childNodes` also contain text nodes and comments. With `children`, however, you can use the ID, and therefore they are easier to use.

To get to the treasure using `children` you would have to use:

```
console.dir(document.body.children.forest.children.tree2.children.shrubbery.children.treasure);
```

As you can see, on every element we select, we have to select the `children` again. So, first, we grab the `children` from the `body`, then we select `forest` from these `children`. Then from `forest`, we want to grab its `children` again, and from these `children` we want to select `tree2`. From `tree2` we want to grab the `children` again, from these `children` we need `shrubbery`. And then finally, we can grab the `children` from `shrubbery` and select `treasure`.

To get to the treasure using `childNodes` you would have to use your console a lot because text and comment nodes are also in there. `childNodes` is an array, so you will have to select the right index to select the right child. There is one advantage here: it is a lot shorter because you won't need to select the name separately.

```
console.dir(document.body.childNodes[3].childNodes[3].childNodes[1].childNodes[1]);
```

You could also combine them:

```
console.dir(document.body.childNodes[3].childNodes[3].childNodes[1].children.treasure);
```

There are many ways to traverse the document. Depending on what you need, you might have to use one specific way. For tasks that require DOM traversing, it is usually the case that if it works, it is a good solution.

So far, we have seen how we can move down the DOM, but we can also move up. Every element knows its parent. We can use the `parentElement` property to move back up. For example, if we use the treasure HTML sample and type this into the console:

```
document.body.children.forest.children.tree2.parentElement;
```

We are back at `forest`, since that is the parent element of `tree2`. This can be very useful, in particular when combined with functions such as `getElementById()`, which we will see later in more detail.

Not only can we move up and down, we can also move sideways. For example, if we select `tree2` like this:

```
document.body.children.forest.children.tree2;
```

We can get to `tree1` using:

```
document.body.children.forest.children.tree2.previousElementSibling;
```

And from `tree1` we can get to `tree2` using:

```
document.body.children.forest.children.tree1.nextElementSibling;
```

As an alternative to `nextElementSibling`, which returns the next node that is an element, you could use `nextSibling`, which will return the next node of any type.

Practice exercise 10.1

In this exercise, experiment with traversing the DOM hierarchy. You can use this sample HTML website:

```
<!doctype html>
<html><head><title>Sample Webpage</title></head>
<body>
  <div class="main">
    <div>
      <ul>
        <li>One</li>
        <li>Two</li>
```

```

        <li>Three</li>
      </ul>
    </div>
    <div>blue</div>
    <div>green</div>
    <div>yellow</div>
    <div>Purple</div>
  </div>
</body>
</html>

```

Take the following steps:

1. Create and open the above sample web page, or visit your favorite website, and open the document body in the console with `console.dir(document)`.
2. In the `body.children` property, select some of the child elements. View how they match the page content.
3. Navigate to and output the next nodes or elements into the console.

Selecting elements as objects

Now we know how to traverse the DOM, we can make changes to the elements. Instead of using `console.dir()`, we can just type in the path to the element we want to change. We now have the element as a JavaScript object, and we can make changes to all its properties. Let's use a simpler HTML page for this one.

```

<!DOCTYPE html>
<html>
  <body>
    <h1>Welcome page</h1>
    <p id="greeting">
      Hi!
    </p>
  </body>
</html>

```

We can traverse to the `p` element, for example, by using this code:

```
document.body.children.greeting;
```

This gives us the power to manipulate the properties of the element, and the element itself, directly! Let's execute this newly gained power in the next section.

Changing innerText

The `innerText` property focuses on the text between the opening and closing of the element, like so:

```
<element>here</element>
```

The retrieved value would be here as plain text. For example, if we go to the console and we type:

```
document.body.children.greeting.innerText = "Bye!";
```

The message that is displayed on the page changes from Hi! to Bye! immediately. `innerText` returns the content of the element as plain text, which is not a problem in this case because there is only text in there. However, if there is any HTML inside the element you need to select, or if you want to add HTML, you cannot use this method. It will interpret the HTML as text and just output it on the screen. So if we executed this:

```
document.body.children.greeting.innerText = "<p>Bye!</p>";
```

It will output to the screen `<p>Bye!</p>`, with the HTML around it, as if it was intended as a text string. To get around this, you need to use `innerHTML`.

Changing innerHTML

If you did not only want to work with plain text, or perhaps specify some HTML formatting with your value, you could use the `innerHTML` property instead. This property doesn't just process be plain text, it can also be inner HTML elements:

```
document.body.children.greeting.innerHTML = "<b>Bye!</b>";
```

This will display **Bye!** in bold on the screen, having taken the `b` element into account rather than just printing it as if it were a single string value.

You were already promised that you could access elements in a more convenient way than traversing the DOM. Let's see how exactly in the next section.

Accessing elements in the DOM

There are multiple methods to select elements from the DOM. After getting the elements, we are able to modify them. In the following sections, we will discuss how to get elements by their ID, tag name, and class name, and by CSS selector.

Instead of traversing it step by step as we just did, we are going to use built-in methods that can go through the DOM and return the elements that match the specifications.

We are going to use the following HTML snippet as an example:

```
<!DOCTYPE html>
<html>
<body>
  <h1>Just an example</h1>
  <div id="one" class="example">Hi!</div>
  <div id="two" class="example">Hi!</div>
  <div id="three" class="something">Hi!</div>
</body>
</html>
```

Let's start by accessing elements by ID.

Accessing elements by ID

We can grab elements by ID with the `getElementById()` method. This returns one element with the specified ID. IDs should be unique, as only one result will be returned from the HTML document. There are not a lot of rules for valid IDs; they cannot contain spaces and must be at least one character. As with the conventions for naming variables, it is a best practice to make it descriptive and avoid special characters.

If we want to select the element with an ID of `two` right away, we could use:

```
document.getElementById("two");
```

This would return the full HTML element:

```
<div id="two" class="example">Hi!</div>
```

To reiterate, if you have more than one element with the same ID, it will just give you back the first one it encounters. You should avoid this situation in your code though.

This is what the full file looks like with the JavaScript inside the HTML page, instead of simply querying the browser console:

```
<html>
<body>
```

```
<h1 style="color:pink;">Just an example</h1>
<div id="one" class="example">Hi!</div>
<div id="two" class="example">Hi!</div>
<div id="three" class="something">Hi!</div>
</body>
<script>
  console.log(document.getElementById("two"));
</script>
</html>
```

In this case, it would log the full HTML `div` with `id="two"` to the console.

Practice exercise 10.2

Try experimenting with getting elements by their IDs:

1. Create an HTML element and assign an ID in the element attribute.
2. Select the page element using its ID.
3. Output the selected page element into the console.

Accessing elements by tag name

If we ask for elements by tag name, we get an array as a result. This is because there could be more than one element with the same tag name. It will be a collection of HTML elements, or `HTMLCollection`, which is a special JavaScript object. It's basically just a list of nodes. Execute the following command in the console:

```
document.getElementsByTagName("div");
```

It will give back:

```
HTMLCollection(3) [div#one.example, div#two.example, div#three.
something, one: div#one.example, two: div#two.example, three:
div#three.something]
```

As you can see, all the elements in the DOM with the `div` tag are returned. You can read what the ID is and what the class is from the syntax. The first ones in the collection are the objects: `div` is the name, `#` specifies the ID, and `.` specifies the class. If there are multiple dots, there are multiple classes. Then you can see the elements again (`namedItems`), this time as key-value pairs with their ID as the key.

We can access them using the `item()` method to access them by index, like this:

```
document.getElementsByTagName("div").item(1);
```

This will return:

```
<div id="two" class="example">Hi!</div>
```

We can also access them by name, using the `namedItem()` method, like this:

```
document.getElementsByTagName("div").namedItem("one");
```

And this will return:

```
<div id="one" class="example">Hi!</div>
```

When there is only one match, it will still return an `HTMLCollection`. There is only one `h1` tag, so let's demonstrate this behavior:

```
document.getElementsByTagName("h1");
```

This will output:

```
HTMLCollection [h1]
```

Since `h1` doesn't have an ID or class, it is only `h1`. And since it doesn't have an ID, it is not a `namedItem` and is only in there once.

Practice exercise 10.3

Use JavaScript to select page elements via their tag name:

1. Start by creating a simple HTML file.
2. Create three HTML elements using the same tag.
3. Add some content within each element so you can distinguish between them.
4. Add a script element to your HTML file, and within it select the page elements by tag name and store them in a variable as an array.
5. Using the index value, select the middle element and output it into the console.

Accessing elements by class name

We can do something very similar for class names. In our example HTML, we have two different class names: `example` and `something`. If you get elements by class name, it gives back an `HTMLCollection` containing the results. The following will get all the elements with the class `example`:

```
document.getElementsByClassName("example");
```

This returns:

```
HTMLCollection(2) [div#one.example, div#two.example, one: div#one.example, two: div#two.example]
```

As you can see, it only returned the `div` tags with the `example` class. It left out the `div` with the `something` class.

Practice exercise 10.4

Select all matching page elements using the class name of the element.

1. Create a simple HTML file to work on.
2. Add three HTML elements adding the same class to each. You can use different tags as long as the same element class is included. Add some content within each element so you can distinguish between them.
3. Add a script element to your file, and within it select the page elements by class name. Assign the resulting `HTMLCollection` values to a variable.
4. You can use an index value to select the individual `HTMLCollection` items, just as you would for array items. Starting with an index of 0, select one of the page elements with the class name and output the element into the console.

Accessing elements with a CSS selector

We can also access elements using a CSS selector. We do this with `querySelector()` and `querySelectorAll()`. We then give the CSS selector as an argument, and this will filter the items in the HTML document and only return the ones that satisfy the CSS selector.

The CSS selector might look a bit different than you might think at first. Instead of looking for a certain layout, we use the same syntax as we use when we want to specify a layout for certain elements. We haven't discussed this yet, so we will cover it here briefly.

If we state `p` as a CSS selector, it means all the elements with tag `p`. This would look like this:

```
document.querySelectorAll("p");
```

If we say `p.example`, it means all the `p` tag elements with `example` as the class. They can also have other classes; as long as `example` is in there, it will match. We can also say `#one`, which means select all with an ID of `one`.

This method is the same result as `getElementById()`. Which one to use is a matter of taste when all you really need to do is select by ID — this is great input for a discussion with another developer. `querySelector()` allows for more complicated queries, and some developers will state that `getElementById()` is more readable. Others will claim that you might as well use `querySelector()` everywhere for consistency. It doesn't really matter at this point, but try to be consistent.

Don't worry too much about all these options for now; there are many, and you'll figure them out when you need them. This is how you can use the CSS selectors in JavaScript.

Using `querySelector()`

This first option will select the first element that matches the query. So, enter the following in the console, still using the HTML snippet introduced at the start of the section:

```
document.querySelector("div");
```

It should return:

```
<div id="one" class="example">Hi!</div>
```

It only returns the first `div`, because that's the first one it encounters. We could also ask for an element with the class `something`. If you recall, we select classes using dot notation like this:

```
document.querySelector(".something");
```

This returns:

```
<div id="three" class="something">Hi!</div>
```

With this method, you can only use valid CSS selectors: elements, classes, and IDs.

Practice exercise 10.5

Use `querySelector()` to enable single element selection:

1. Create another simple HTML file.
2. Create four HTML elements adding the same class to each. They can be different tag names as long as they have the class within the element attribute.
3. Add some content within each element so you can distinguish between them.
4. Within a script element, use `querySelector()` to select the first occurrence of the elements with that class and store it in a variable. If there is more than one matching result in `querySelector()`, it will return the first one.
5. Output the element into the console.

Using `querySelectorAll()`

Sometimes it is not enough to return only the first instance, but you want to select all the elements that match the query. For example when you need to get all the input boxes and empty them. This can be done with `querySelectorAll()`:

```
document.querySelectorAll("div");
```

This returns:

```
NodeList(3) [div#one.example, div#two.example, div#three.something]
```

As you can see, it is of object type `NodeList`. It contains all the nodes that match the CSS selector. With the `item()` method we can get them by index, just as we did for the `HTMLCollection`.

Practice exercise 10.6

Use `querySelectorAll()` to select all matching elements in an HTML file:

1. Create an HTML file and add four HTML elements, adding the same class to each one.
2. Add some content within each element so you can distinguish between them.
3. Within a script element, use `querySelectorAll()` to select all the matching occurrences of the elements with that class and store them in a variable.
4. Output all the elements into the console, first as an array and then looping through them to output them one by one.

Element click handler

HTML elements can do something when they are clicked. This is because a JavaScript function can be connected to an HTML element. Here is one snippet in which the JavaScript function associated with the element is specified in the HTML:

```
<!DOCTYPE html>
<html>
<body>
  <div id="one" onclick="alert('Ouch! Stop it!')">Don't click here!
</div>
</body>
</html>
```

Whenever the text in the `div` gets clicked, a pop up with the text `Ouch! Stop it!` opens. Here, the JavaScript is specified directly after `onclick`, but if there is JavaScript on the page, you can also refer to a function that's in that JavaScript like this:

```
<!DOCTYPE html>
<html>
<body>
  <script>
    function stop(){
      alert("Ouch! Stop it!");
    }
  </script>
  <div id="one" onclick="stop()">Don't click here!</div>
</body>
</html>
```

This code is doing the exact same thing. As you can imagine, with bigger functions this would be a better practice. The HTML can also refer to scripts that get loaded into the page.

There is also a way to add a click handler using JavaScript. We select the HTML element we want to add the click handler to, and we specify the `onclick` property.

Here is a HTML snippet:

```
<!DOCTYPE html>
<html>
<body>
```

```
<div id="one">Don't click here!</div>
</body>
</html>
```

This code is at the moment not doing anything if you click it. If we want to dynamically add a click handler to the `div` element, we can select it and specify the property via the console:

```
document.getElementById("one").onclick = function () {
  alert("Auch! Stop!");
}
```

As it's been added in the console, this functionality will be gone when you refresh the page.

This and the DOM

The `this` keyword always has a relative meaning; it depends on the exact context it is in. In the DOM, the special `this` keyword refers to the element of the DOM it belongs to. If we specify an `onclick` to send `this` in as an argument, it will send in the element the `onclick` is in.

Here is a little HTML snippet with JavaScript in the script tag:

```
<!DOCTYPE html>
<html>
<body>
  <script>
    function reveal(el){
      console.log(el);
    }
  </script>
  <button onclick="reveal(this)">Click here!</button>
</body>
</html>
```

And this is what it will log:

```
<button onclick="reveal(this)">Click here!</button>
```

As you can see, it is logging the element it is in, the button element.

We can access the parent of this with a function like this:

```
function reveal(e1){
  console.log(e1.parentElement);
}
```

In the above example, the body is the parent of the button. So if we click the button with the new function, it will output:

```
<body>
<script>
  function reveal(e1.parentElement){
    console.log(e1);
  }
</script>
<button onclick="reveal(this)">Click here!</button>
</body>
```

We could output any other property of the element the same way; for example, `console.log(e1.innerText)`; would print the inner text value as we saw in the *Changing innerText* section.

So, the `this` keyword is referring to the element, and from this element we can traverse the DOM like we just learned. This can be very useful, for example, when you need to get the value of an input box. If you send `this`, then you can read and modify the properties of the element that triggered the function.

Practice exercise 10.7

Create a button within a basic HTML document and add the `onClick` attribute. The example will demonstrate how you can reference object data with this:

1. Create a function to handle a click within your JavaScript code. You can name the function `message`.
2. Add this to the `onClick` function parameters sending the current element object data using `this`.
3. Within the `message` function, use `console.dir()` to output in the console the element object data that was sent to the function using `onClick` and `this`.
4. Add a second button to the page also invoking the same function on the click.

5. When the button is clicked, you should see the element that triggered the click in the console, like so:

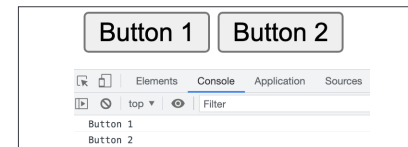


Figure 10.1: Implementing the onclick attribute

Manipulating element style

After selecting the right element from the DOM, we can change the CSS style that applies to it. We can do this using the `style` property. This is how to do it:

1. Select the right element from the DOM.
2. Change the right property of the style property of this element.

We are going to make a button that will toggle the appearing and disappearing of a line of text. To hide something using CSS, we can set the `display` property of the element to `none`, like this for a `p` (paragraph) element:

```
p {
  display: none;
}
```

And we can toggle it back to visible using:

```
p {
  display: block;
}
```

We can add this style using JavaScript as well. Here is a little HTML and JavaScript snippet that will toggle the displaying of a piece of text:

```
<!DOCTYPE html>
<html>
```

```

<body>
<script>
function toggleDisplay(){
  let p = document.getElementById("magic");
  if(p.style.display === "none") {
    p.style.display = "block";
  } else {
    p.style.display = "none";
  }
}
</script>
<p id="magic">I might disappear and appear.</p>
<button onclick="toggleDisplay()">Magic!</button>
</body>
</html>

```

As you can see, in the if statement we are checking for whether it is currently hiding, if it is hiding, we show it. Otherwise, we hide it. If you click the button and it is currently visible, it will disappear. If you click the button when the text is gone, it will appear.

You can do all sorts of fun things using this style element. What do you think this does when you click the button?

```

<!DOCTYPE html>
<html>
<body>
<script>
function rainbowify(){
  let divs = document.getElementsByTagName("div");
  for(let i = 0; i < divs.length; i++) {
    divs[i].style.backgroundColor = divs[i].id;
  }
}
</script>
<style>
div {
  height: 30px;
  width: 30px;
  background-color: white;
}
</style>

```

```

<div id="red"></div>
<div id="orange"></div>
<div id="yellow"></div>
<div id="green"></div>
<div id="blue"></div>
<div id="indigo"></div>
<div id="violet"></div>
<button onclick="rainbowify()">Make me a rainbow</button>
</body>
</html>

```

This is what you see when you first open the page:



Figure 10.2: A button that will do wonderful things when it is clicked

And when you click the button:

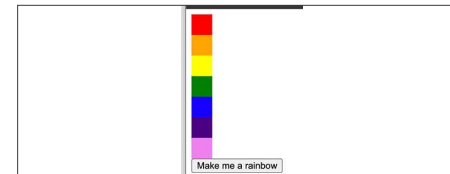


Figure 10.3: Beautiful rainbow made by JavaScript at the click of a button

Let's go over this script to see how works. First of all, there are a few `div` tags in the HTML that all have the ID of a certain color. There is a `style` tag specified in HTML, which gives a default layout to these `div` tags of 30px by 30px and a white background.

When you click the button, the `rainbowify()` JavaScript function is executed. In this function the following things are happening:

1. All the `div` elements get selected and stored in an array, `divs`.
2. We loop over this `divs` array.
3. For every element in the `divs` array, we are setting the `backgroundColor` property of style to the ID of the element. Since all the IDs represent a color, we see a rainbow appear.

As you can imagine, you can really have a lot of fun playing around with this. With just a few lines of code, you can make all sorts of things appear on the screen.

Changing the classes of an element

HTML elements can have classes, and as we have seen, we can select elements by the name of the class. As you may remember, classes are used a lot for giving elements a certain layout using CSS.

With JavaScript, we can change the classes of HTML elements, and this might trigger a certain layout that is associated with that class in CSS. We are going to have a look at adding classes, removing classes, and toggling classes.

Adding classes to elements

This might sound a bit vague, so let's have a look at an example where we are going to add a class to an element, which in this case will add a layout and make the element disappear.

```
<!DOCTYPE html>
<html>
  <body>
    <script>
      function disappear(){
        document.getElementById("shape").classList.add("hide");
      }
    </script>
```

```
<style>
.hide {
  display: none;
}

.square {
  height: 100px;
  width: 100px;
  background-color: yellow;
}

.square.blue {
  background-color: blue;
}
</style>
<div id="shape" class="square blue"></div>

<button onclick="disappear()">Disappear!</button>
</body>
</html>
```

In this example, we have some CSS specified in the `style` tag. Elements with the `hide` class have a `display: none` style, meaning they are hidden. Elements with the `square` class are 100 by 100 pixels and are yellow. But when they have both the `square` and `blue` class, they are blue.

When we click on the **Disappear!** button, the `disappear()` function gets called. This one is specified in the script tag. The `disappear()` function changes the classes by getting the `classList` property of the element with the ID `shape`, which is the square we are seeing. We are adding the `hide` class to the `classList` and because of this, the elements get the `display: none` layout and we can no longer see it.

Removing classes from elements

We can also remove a class. If we remove the `hide` class from the `classList`, for example, we could see our element again because the `display: none` layout no longer applies.

In this example, we are removing another class. Can you figure out what will happen if you press the button by looking at the code?

```

<!DOCTYPE html>
<html>
<body>
  <script>
    function change(){
      document.getElementById("shape").classList.remove("blue");
    }
  </script>
  <style>
    .square {
      height: 100px;
      width: 100px;
      background-color: yellow;
    }

    .square.blue {
      background-color: blue;
    }
  </style>
  <div id="shape" class="square blue"></div>

  <button onclick="change()">Change!</button>
</body>
</html>

```

When the button gets pressed, the change function gets triggered. This function removes the blue class, which removes the blue background color from the layout, leaving us with the yellow background color and the square will turn yellow.

You may wonder why the square was blue in the first place since it had two layouts for background-color assigned to it with the CSS. This happens with a points system. When a styling is more specific, it gets more points. So, specifying two classes with no space in between means that it applies to elements with these two classes. This is more specific than pointing at one class.



Referring to an ID in CSS, `#nameId`, gets even more points and would be prioritized over class-based layouts. This layering allows for less duplicate code, but it can become messy, so always make sure to combine the CSS and the HTML well to get the desired layout.

Toggleing classes

In some cases, you would want to add a class when it doesn't already have that particular class, but remove it when it does. This is called toggling. There is a special method to toggle classes. Let's change our first example to toggle the hide class so the class will appear when we press the button the second time, disappear the third time, and so on. The blue class was removed to make it shorter; it's not doing anything in this example other than making the square blue.

```

<!DOCTYPE html>
<html>
<body>
  <script>
    function changeVisibility(){
      document.getElementById("shape").classList.toggle("hide");
    }
  </script>
  <style>
    .hide {
      display: none;
    }

    .square {
      height: 100px;
      width: 100px;
      background-color: yellow;
    }
  </style>
  <div id="shape" class="square"></div>

  <button onclick="changeVisibility()">Magic!</button>
</body>
</html>

```

Pressing the Magic! button will add the class to the `classList` when it isn't there and remove it when it is there. This means that you can see the result every time you press the button. The square keeps appearing and disappearing.

Manipulating attributes

We have seen already that we can change the class and style attributes, but there is a more general method that can be used to change any attribute. Just a quick reminder, attributes are the parts in HTML elements that are followed by equals signs. For example, this HTML link to Google:

```
<a id="friend" class="fancy boxed" href="https://www.google.com">Ask my friend here.</a>
```

The attributes in this example are `id`, `class`, and `href`. Other common attributes are `src` and `style`, but there are many others out there.

With the `setAttribute()` method, we can add or change attributes on an element. This will change the HTML of the page. If you inspect the HTML in the browser you will see that the changed attributes are visible. You can do this from the console and see the result easily, or write another HTML file with this built in as a function. In this HTML snippet, you will see it in action:

```
<!DOCTYPE html>
<html>
<body>
<script>
function changeAttr(){
let el = document.getElementById("shape");
el.setAttribute("style", "background-color:red;border:1px solid black");
el.setAttribute("id", "new");
el.setAttribute("class", "circle");
}
</script>
<style>
div {
height: 100px;
width: 100px;
background-color: yellow;
}
</style>
```

```
.circle {
border-radius: 50%;
}
</style>
<div id="shape" class="square"></div>

<button onclick="changeAttr()">Change attributes...</button>
</body>
</html>
```

This is the page before clicking the button:

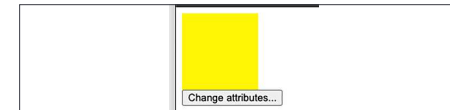


Figure 10.4: Page with a yellow square div

After clicking the button, the HTML of the div becomes:

```
<div id="new" class="circle" style="background-color:red;border:1px solid black"></div>
```

As you can see, the attributes are changed. The `id` has changed from `shape` to `new`. The class has changed from `square` to `circle` and a style has been added. It will look like this:

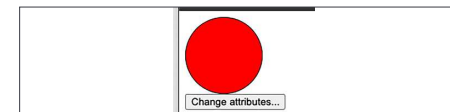


Figure 10.5: Page with a red circle with a black line around it

This is a very powerful tool that can be used to interact with the DOM in very many ways. Think, for example, of a tool that can be used to create images, or even postcards. Beneath the surface, there is a lot of manipulating going on.



It is important to note here that JavaScript interacts with the DOM and not with the HTML file—therefore, the DOM is the one that gets changed. If you click the button again, you'll get an error message in the console because no element with `id="shape"` is found in the DOM, and as a result we try to call a method on a null value.

Practice exercise 10.8

Creating custom attributes: using an array of names, the following code will update the element's HTML, adding HTML code using the data from the array. The items within the array will be output to the page as HTML code. The user will be able to click the page elements and they will display the page element attribute values.

My Friends Table

Laurence	1
Mike	2
John	3
Larry	4
Kim	5
Joanne	6
Lisa	7
Janet	8
Jane	9

Figure 10.6: Creating custom attributes with an array of names

As the HTML will start getting more complex from now on, and we're only trying to test your JavaScript, we will provide HTML templates to use where needed. You can use the following HTML template and provide your answer as the completed script element:

```
<!DOCTYPE html>
<html>
<head>
  <title>Complete JavaScript Course</title>
</head>
<body>
```

```
<div id="message"></div>
<div id="output"></div>
<script>

</script>
</body>
</html>
```

Take the following steps:

1. Create an array of names. You can add as many as you want—all the string values will be output onto the page within a table.
2. Select the page elements as JavaScript objects.
3. Add a function and also invoke that function within the JavaScript code. The function can be called `build()` as it will be building the page content. Within the `build` function, you will be setting up the HTML in a table.
4. Create a table named `html`, and within the tags, loop through the contents of the array and output the results into the `html` table.
5. Add a class called `box` to one of the cells that has the index value of the item from the array, adding the same class to the elements for each additional row.
6. As you create the HTML for the elements within the `tr` element, create an attribute called `data-row` in the main row element that includes the index value of the item from the array. In addition, add another attribute within the element called `data-name` that will contain the text output.
7. Within the attribute of the same `tr` element, also add `onclick` to invoke a function named `getData` passing the current element object as `this` into the function parameter.
8. Add the table of HTML code to the page.
9. Create a function named `getData` that will be invoked once the HTML `tr` elements are clicked. Once the `tr` element is clicked, use `getAttribute` to get the attribute values of the row value and the contents of the text output and store them in different variables.
10. Using the values in the attributes stored in the preceding step, output the values into the `message` element on the page.
11. Once the user clicks the element on the page, it will display the details coming from the element attributes within the element with the `id` of `message`.

Event listeners on elements

Events are things that happen on a web page, like clicking on something, moving the mouse over an element, changing an element, and there are many more. We have seen how to add an `onClick` event handler already. In the same way, you can add an `onchange` handler, or an `onmouseover` handler. There is one special condition, though: one element can only have one event handler as an HTML attribute. So, if it has an `onClick` handler, it cannot have an `onmouseover` handler as well. At this point, we have only seen how to add event listeners using HTML attributes like this:

```
<button onClick="addRandomNumber()">Add a number</button>
```

There is a way to register event handlers using JavaScript as well. We call these event listeners. Using event listeners, we can add multiple events to one element. This way, JavaScript is constantly checking, or listening, for certain events to the elements on the page. Adding event listeners is a two-step process:

1. Select the element you want to add an event to
2. Use the `addEventListener("event", function)` syntax to add the event

Even though it is two steps, it can be done in one line of code:

```
document.getElementById("square").addEventListener("click",
  changeColor);
```

This is getting the element with the ID `square` and adding the `changeColor` function as the event for whenever it gets clicked. Note that when using event listeners, we remove the `on` prefix from the event type. For example, `click` here references the same event type as `onClick`, but we have removed the `on` prefix.

Let's consider another way to add an event listener (don't worry, we will review these methods in detail in *Chapter 11, Interactive Content and Event Listeners*) by setting the event property of a certain object to a function.



There is a fun fact here—event listeners often get added during other events!

We could reuse our trusty `onClick` listener in this context, but another common one is when the web page is done loading with `onload`:

```
window.onload = function() {
  // whatever needs to happen after loading
  // for example adding event listeners to elements
}
```

This function will then be executed. This is common for `window.onload`, but less common for many others, such as `onClick` on a `div` (it is possible though). Let's look at an example of the first event listener we looked at on a web page. Can you figure out what it will be doing when you click on the square?

```
<!DOCTYPE html>
<html>
<body>
<script>
  window.onload = function() {
    document.getElementById("square").addEventListener("click",
      changeColor);
  }
  function changeColor(){
    let red = Math.floor(Math.random() * 256);
    let green = Math.floor(Math.random() * 256);
    let blue = Math.floor(Math.random() * 256);
    this.style.backgroundColor = `rgb(${red}, ${green}, ${blue})`;
  }
</script>
<div id="square" style="width:100px;height:100px;background-
color:grey;">Click for magic</div>
</body>
</html>
```

The web page starts with a gray square with the text `Click for magic` in it. After the web page is done loading, an event gets added for this square. Whenever it gets clicked, the `changeColor` function will be executed. This function uses random variables to change the color using RGB colors. Whenever you click the square, the color gets updated with random values.

You can add events to all sorts of elements. We have only used the `click` event so far, but there are many more. For example, `focus`, `blur`, `focusin`, `focusout`, `mouseout`, `mouseover`, `keydown`, `keypress`, and `keyup`. These will be covered in the next chapter, so keep going!

Practice exercise 10.9

Try an alternative way to implement similar logic to *Practice exercise 10.7*. Use the following HTML code as a template for this exercise, and add the contents of the script element:

```
<!doctype html>
<html>
<head>
  <title>JS Tester</title>
</head>
<body>
  <div>
    <button>Button 1</button>
    <button>Button 2</button>
    <button>Button 3</button>
  </div>
  <script>

  </script>
</body>
</html>
```

Take the following steps:

1. Select all the page buttons into a JavaScript object.
2. Loop through each button, and create a function within the button scope called `output`.
3. Within the `output()` function, add a `console.log()` method that outputs the current object's `textContent`. You can reference the current parent object using the `this` keyword.
4. As you loop through the buttons attach an event listener that when clicked invokes the `output()` function.

Creating new elements

In this chapter, you have seen so many cool ways to manipulate the DOM already. There is still an important one missing, the creation of new elements and adding them to the DOM. This consists of two steps, first creating new elements and second adding them to the DOM.

This is not as hard as it may seem. The following JavaScript does just that:

```
let el = document.createElement("p");
el.innerText = Math.floor(Math.random() * 100);
document.body.appendChild(el);
```

It creates an element of type `p` (paragraph). This is a `createElement()` function that is on the `document` object. Upon creation, you need to specify what type of HTML element you would want to create, which in this case is a `p`, so something like this:

```
<p>innerText here</p>
```

And as `innerText`, it is adding a random number. Next, it is adding the element as a new last child of the body. You could also add it to another element; just select the element you want to add it to and use the `appendChild()` method.

Here, you can see it incorporated in a HTML page. This page has a button, and whenever it gets pressed, the `p` gets added.

```
<!DOCTYPE html>
<html>
<body>
  <script>
    function addRandomNumber(){
      let el = document.createElement("p");
      el.innerText = Math.floor(Math.random() * 100);
      document.body.appendChild(el);
    }
  </script>
  <button onclick="addRandomNumber()">Add a number</button>
</body>
</html>
```

Here is a screenshot of this page after having pressed the button five times.

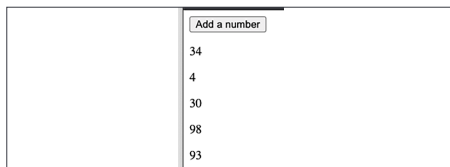


Figure 10.7: Random numbers after pressing the button five times

Once we refresh the page, it's empty again. The file with the source code doesn't change and we're not storing it anywhere.

Practice exercise 10.10

Shopping list: Using the following HTML template, update the code to add new items to the list of items on the page. Once the button is clicked, it will add a new item to the list of items:

```
<!DOCTYPE html>
<html>
<head>
  <title>Complete JavaScript Course</title>
  <style>
  </style>
</head>
<body>
  <div id="message">Complete JavaScript Course</div>
  <div>
    <input type="text" id="addItem">
    <input type="button" id="addNew" value="Add to List"> </div>
```

```
<div id="output">
  <h1>Shopping List</h1>
  <ol id="sList"> </ol>
</div>
<script>

</script>
</body>
</html>
```

Take the following steps:

1. Select the page elements as JavaScript objects.
2. Add an `onClick` event listener to the add button. Once the button is clicked, it should add the contents of the input field to the end of the list. You can call the function `addOne()`.
3. Within `addOne()`, create 11 elements to append to the main list on the page. Add the input value to the list item text content.
4. Within the `addOne()` function, get the current value of the `addItem` input field. Use that value to create a `TextNode` with that value, adding it to the list item. Append the `TextNode` to the list item.

Chapter projects

Collapsible accordion component

Build a collapsing and expanding accordion component that will open page elements, hiding and showing content when the title tab is clicked. Using the following HTML as a template, add the completed script element and create the desired functionality with JavaScript:

```
<!doctype html>
<html>
<head>
  <title>JS Tester</title>
  <style>
    .active {
      display: block !important;
    }
  </style>
```

```

        .myText {
            display: none;
        }
        .title {
            font-size: 1.5em;
            background-color: #ddd;
        }
    }
</style>
</head>
<body>
    <div class="container">
        <div class="title">Title #1</div>
        <div class="myText">Just some text #1</div>
        <div class="title">Title #2</div>
        <div class="myText">Just some text #2</div>
        <div class="title">Title #3</div>
        <div class="myText">Just some text #3</div>
    </div>
    <script>

    </script>
</body>
</html>

```

Take the following steps:

1. Using `querySelectorAll()`, select all the elements with a class of `title`.
2. Using `querySelectorAll()`, select all the elements with a class of `myText`. This should be the same number of elements as the `title` elements.
3. Iterate through all the `title` elements and add event listeners that, once clicked, will select the next element siblings.
4. Select the element on the click action and toggle the `classList` of the element with the class of `active`. This will allow the user to click the element and hide and show the below content.
5. Add a function that will be invoked each time the elements are clicked that will remove the class of `active` from all the elements. This will hide all the elements with `myText`.

Interactive voting system

The below code will create a dynamic list of people that can be clicked, and it will update the corresponding value with the number of times that name was clicked. It also includes an input field that will allow you to add more users to the list, each of which will create another item in the list that can be interacted with the same as the default list items.

	Maaike	Add Friend
1	Laurence	4
2	Mike	6
3	John	1
4	Larry	1
5	Maaike	4

Figure 10.8: Creating an interactive voting system

Use the following HTML code as a template to add JavaScript to, and provide your answer as the completed script element.

```

<!DOCTYPE html>
<html>
<head>
    <title>Complete JavaScript Course</title>
</head>
<body>
    <div id="message">Complete JavaScript Course</div>
    <div>
        <input type="text" id="addFriend">
        <input type="button" id="addNew" value="Add Friend">
    </div>
    <table id="output"></table>
    <script>

    </script>
</body>
</html>

```

Take the following steps:

1. Create an array of people's names called `myArray`. This will be the default original list of names.
2. Select the page elements as JavaScript objects so they can easily be selected within the code.
3. Add event listener to the **Add Friend** button. Once clicked, this will get the value from the input field and pass the values to a function that will add the friend list to the page. Additionally, add the new friend's name into the people's names array you created. Get the current value in the input field and push that value into the array so the array matches the values on the page.
4. Run a function to build the content on the page, using the `forEach()` loop get all the items within the array and add them to the page. Include 0 as a default for the vote count, as all individuals should start on zero votes.
5. Create a main function that will create the page elements, starting with the parent table row, `tr`. Then create three table cell, `td`, elements. Add content to the table cells, including the vote count in the last column, the person name in the middle, and the index plus 1 in the first column.
6. Append the table cells to the table row and append the table row to the output area on the page.
7. Add an event listener that will increase the vote counter for that row when the user clicks.
8. Get the text content from the last column in the row. It should be the value of the current counter. Increment the counter by one and make sure the datatype is a number so you can add to it.
9. Update the last column with the new click counter.

Hangman game

Create a Hangman game using arrays and page elements. You can use the following HTML template:

```
<!doctype html>
<html><head>
  <title>Hangman Game</title>
  <style>
    .gameArea {
```

```
      text-align: center;
      font-size: 2em;
    }
    .box,
    .box0 {
      display: inline-block;
      padding: 5px;
    }
    .boxE {
      display: inline-block;
      width: 40px;
      border: 1px solid #ccc;
      border-radius: 5px;
      font-size: 1.5em;
    }
  }
</style>
</head>
<body>
  <div class="gameArea">
    <div class="score"> </div>
    <div class="puzzle"></div>
    <div class="letters"></div>
    <button>Start Game</button>
  </div>
</script>
</body>
</html>
```

Take the following steps:

1. Set up an array that contains some words or phrases that you want to use in the game.
2. In JavaScript, create a main game object containing a property to contain the current word solution, and another property to contain the array of letters of the solution. It should also create an array to contain the page elements and correspond with the values of the index values of each letter from the solution, and finally add a property to count the number of letters left to solve and end the game when needed.
3. Select all the page elements into variables so they are easier to access in the code.

4. Add an event listener to the **Start Game** button, making it clickable, and when it gets clicked it should launch a function called `startGame()`.
5. Within `startGame()`, check if the `words` array has any words left. If it does, then hide the button by setting the `.display` object to `none`. Clear the game contents and set the total to 0. Within the current word in the game object, assign a value, which should be the response of `shift()` from the array containing the in-game words.
6. In the game solution, convert the string into an array of all the characters in the word solution using `split()`.
7. Create a function called `builder()` that can be used to build the game board. Invoke the function within the `startGame()` function once all the game values are cleared and set.
8. Create a separate function that you can use to generate page elements. In the parameters, get the type of element, the parent that the new element will be appended to, the output content for the new element, and a class to add to the new element. Using a temporary variable, create the element, add the class, append to the parent, set the `textContent`, and return the element.
9. In the `builder()` function, which will also get invoked once `startGame()` is run, clear the `innerHTML` from the letters and puzzle page elements.
10. Iterate through the game solution array, getting each letter of the solution. Use the `builder()` function to generate page elements, add an output value of `-`, set a class, and append it to the main puzzle page element.
11. Check if the value is blank, and if it is, clear `textContent` and update the border to white. If it's not blank, increment the total so that it reflects the total number of letters that must be guessed. Push the new element into the game puzzle array.
12. Create a new function to update the score so that you can output the current number of letters left. Add it to the `builder()` function.
13. Create a loop to represent the 26 letters of the alphabet. You can generate the letter by using an array containing all the letters. The string method `fromCharCode()` will return the character from the numeric representation.
14. Create elements for each letter, adding a class of box and appending it to the letters page element. As each element gets created, add an event listener that runs a function called `checker()`.
15. Once the letter gets clicked, we need to invoke the `checker()` function, which will remove the main class, add another class, remove the event listener, and update the background color. Also invoke a function called `checkLetter()`, passing the value of the clicked letter into the argument.

16. The `checkLetter()` function will loop through all the solution letters. Add a condition to check if the solution letter is equal to the letter selected by the player. Make sure to convert the inputted letter to uppercase so that you can match the letters accurately. Update the matching letters in the puzzle using the game puzzle array and the index from the letter in the solution. The index values will be the same on each, which provides an easy way to match the visual representation with what is in the array.
17. Subtract one from the game global object that tracks the total letters left to be solved, invoke the `updatescore()` function to check if the game is over, and update the score. Set the `textContent` of the puzzle to the letter removing the original dash.
18. In the `updatescore()` function, set the score to the number of letters left. If the total left is less than or equal to zero, the game is over. Show the button so that the player has an option for the next phrase.

Self-check quiz

1. What output will the following code produce?

```
<div id="output">Complete JavaScript Course </div>
<script>
  var output = document.getElementById('output');
  output.innerText = "Hello <br> World";
</script>
```

2. What output will be seen within the browser page?

```
<div id="output">Complete JavaScript Course </div>
<script>
  document.getElementById('output').innerHTML = "Hello
<br> World";
</script>
```

3. What will be seen in the input field from the following code?

```
<div id="output">Hello World</div>
<input type="text" id="val" value="JavaScript">
<script>
  document.getElementById('val').value = document.
getElementById('output').innerHTML;
</script>
```


4. In the following code, what is output into the console when the element with the word *three* gets clicked? What is the output when the element with the word *one* gets clicked?

```
<div class="holder">
  <div onClick="output('three')">Three
  <div onClick="output('two')">Two
  <div onClick="output('one')">One</div>
</div>
</div>
<script>
function
output(val) {
  console.log(val);
}
</script>
```

5. What line of code needs to be added to remove the event listener when the button is clicked in the following code?

```
<div class="btn">Click Me</div>
<script>
const btn = document.querySelector(".btn");
btn.addEventListener("click", myFun);
function myFun() {
  console.log("clicked");
}
</script>
```

Summary

In this chapter, we really took our web skills to the next level. Manipulating the DOM allows all kinds of interactions with the web page, meaning that the web page is no longer a static event.

We started off by explaining the dynamic web and how to traverse the DOM. After having walked over the elements manually, we learned that there's an easier way to select elements in the DOM with the `getElementBy_()` and the `querySelector()` methods. After having selected them, we had the power to modify them, add new elements to them, and do all sorts of things using the elements we selected. We started with some more basic HTML handlers, and we could assign a function to, for example, the `onClick` attribute of the HTML element.

We also accessed the clicked element using the `this` argument that was sent in as a parameter, and we could modify it in different ways, for example, by changing the style property. We also saw how to add classes to an element, create new elements, and add them to the DOM. And finally, we worked with event listeners on elements that really took our dynamic web pages to the next level. With event listeners, we can specify more than one event handler for a certain element. All these new skills allow us to create amazing things in the web browser. You can actually create complete games now!

The next chapter will take your event handler skills to the next level and will enhance your ability to create interactive web pages even further (and make it a bit easier as well!).

11

Interactive Content and Event Listeners

You are now familiar with the basic manipulation of the **Document Object Model (DOM)**. In the last chapter, we had a look at events, and we saw that event listeners are constantly monitoring for whether a certain event is taking place; when it does occur, the specified event (a function) gets invoked.

In this chapter, we are going to take this knowledge one step further and use event listeners to create interactive web content. This chapter is really going to complete your DOM knowledge. We are going to cover the following topics:

- Interactive content
- Specifying events
- The `onload` event handler
- The mouse event handler
- The event target property
- DOM event flow
- `onchange` and `onblur`
- The key event handler
- Drag and drop elements
- Form submission
- Animating elements



Note: exercise, project and self-check quiz answers can be found in the *Appendix*.

Introducing interactive content

Interactive content is content that responds to the actions of a user. Think, for example, of a web app in which you can create postcards dynamically, or play a game on a website in a web browser.

This interactive content is made possible by changing the DOM based on user interactions. These interactions could be anything: entering text in an input field, clicking somewhere on the page, hovering over a certain element with the mouse, or a certain input with the keyboard. All these are called events. We have seen events already. But there is actually more to it!

Specifying events

There are three ways to specify events. We have seen each of these in the previous chapter, but let's run through them again now. One is HTML-based and the other two are JavaScript-based. For this example, we are going to use the `click` event as an example.

Specifying events with HTML

First, the HTML way:

```
<p id="unique" onclick="magic()">Click here for magic!</p>
```

The great thing about specifying events this way is that it's quite easy to read this code and predict what it is going to do. As soon as you click on the paragraph, the function `magic()` will be triggered. There are some downsides too: you can only specify one event this way, and you cannot change the event dynamically either.

Specifying events with JavaScript

Here is the first way to do it using JavaScript.

```
document.getElementById("unique").onclick = function() { magic(); };
```

What is happening here is that we are getting the property that represents the selected event and assigning our function to it. So, in this case, we are selecting the `p` shown in the previous section by its attribute value `unique`, grabbing the `onclick` property, and assigning the `magic()` function to it by wrapping it in an anonymous function. We could also specify the exact function on the spot here. We can overwrite this with another function anytime, making the event that can be fired more dynamic.

We can also specify different events now, which we cannot do with HTML. So we could also give it a `keyup`, `keydown`, or `mouseover` event, for example—we will consider each of these event types in this chapter.



If we wanted to specify event triggers for all the elements of the page, we could do so in a loop for a cleaner coding style.

Practice exercise 11.1

Personalize your web pages. Allow users to change the theme of the page display between regular mode and dark mode.

1. Within a simple HTML document, set up a Boolean value variable to use that will toggle the color modes.
2. Use `window.onclick` to set up a function that outputs a message in the console when clicked. You can use the value of the Boolean variable.
3. Within the function, add a condition that checks whether the `darkMode` variable is true or false.
4. If false, then update the page style to a background color of black and a font color of white.
5. Add an `else` response that changes the color of the background to white and the color of the text to black. Also, update the value of the `darkMode` variable accordingly.

Specifying events with event listeners

The last method is using the `addEventListener()` method to add an event to an element. With this, we can specify multiple functions for the same event, for example, when an element gets clicked.

What is striking for both methods we have looked at—using HTML events and assigning to properties—is that the event gets prefixed with `on`. For example, `onclick`, `onload`, `onfocus`, `onblur`, `onchange`, etc. This is not the case when we use the `addEventListener()` method, where we specify the event type within the event listener *without* the `on` prefix, as here, with an alternative to `onclick`:

```
document.getElementById("unique").addEventListener("click", magic);
```

Please note that we are leaving out the parentheses behind the magic function here. We cannot send in parameters like this. If that is something you have to do, you'd have to wrap the functionality in an anonymous function, like this:

```
document.getElementById("unique").addEventListener("click", function() {
  magic(arg1, arg2) });
```

In this chapter, we may use any of these ways to specify an event. We will mostly be using one of the JavaScript options, though.

Practice exercise 11.2

Create several divs with color names in `textContent`. Add JavaScript to add `click` event listeners to each element, and as each element is clicked, update the background color of the body to match the color name in the div.

The onload event handler

We briefly saw this event handler in the previous chapter. The `onload` event gets fired after a certain element is loaded. This can be useful for a number of reasons. For example, if you want to select an element using `getElementById`, you'll have to be sure this element is loaded in the DOM already. This event is most commonly used on the `window` object, but it can be used on any element. When you use it on `window`, this event gets started when the window object is done loading. Here is how to use it:

```
window.onload = function() {
  // whatever needs to happen after the page loads goes here
}
```

`onload` is similar, but it's different for the `window` and `document` objects. The difference depends on the web browser you are using. The `load` event fires at the end of the document loading process. Therefore, you will find that all the objects in the document are in the DOM and the assets have finished loading.

You can also use the `addEventListener()` method on any element to handle any event. And it can also be used for the event that all the content in the DOM is loaded. There is a special built-in event for this: `DOMContentLoaded()`. This event can be used to handle the event of the DOM loading, which will get fired immediately after the DOM for the page has been constructed when the event is set. Here is how to set it:

```
document.addEventListener("DOMContentLoaded", (e) => {
  console.log(e);
});
```

This will log to the console when all the DOM content has been loaded. As an alternative, you will also often see it in the body tag, like this:

```
<body onload="unique()"></body>
```

This is assigning a function called `unique()` to the body, and it will fire off when the body is done loading. You cannot combine `addEventListener()` and the HTML by using them together. One will overwrite the other, depending on the order of the web page. If you need two events to happen when the DOM is loaded, you will need two `addEventListener()` calls in your JavaScript.

Practice exercise 11.3

Using a basic HTML file, the below exercise will demonstrate the order of loading for the window object and the document object using `DOMContentLoaded`, which is an event that fires once the document object content is loaded in the browser. The window object will load afterward, even if the `window.onload` statement comes first.

1. Within a basic HTML file, create a function named `message` that requires two parameters, the first one being a string value for the message and the second an event object. Within the function, output into the console using `console.log` the event and the message.
2. Using the window object, attach an `onload` function to the event object. Invoke the function, passing a string value of `Window Ready` and the event object to the `message` function for output.
3. Create a second function to capture the DOM content loading, and add an event listener listening for `DOMContentLoaded` to the document object. Once that event is triggered, pass the event object and a string value of `Document Ready` to the `message` output function.
4. Change the order of the event listeners, placing the document event statement prior to the window `onload`: does it make a difference in the output?

5. Using the document object, add the `DOMContentLoaded` event listener, which will send to the function the arguments of `Document Ready` and the event object that was triggered.
6. Run the script and see which event is triggered first; change the order of the events to see if the output sequence changes.

Mouse event handlers

There are different mouse event handlers. Mouse events are actions of the mouse. These are the ones we have:

- `ondblclick`: when the mouse is double-clicked
- `onmousedown`: when the mouse clicks on top of an element without the click being released
- `onmouseup`: when the mouse click on top of an element is released
- `onmouseenter`: when the mouse moves onto an element
- `onmouseleave`: when the mouse leaves an element and all of its children
- `onmousemove`: when the mouse moves over an element
- `onmouseout`: when the mouse leaves an individual element
- `onmouseover`: when the mouse hovers over an element

Let's have a look at one of these in practice. What do you think this does?

```
<!doctype html>
<html>
<body>
  <div id="divvy" onmouseover="changeColor()" style="width: 100px;
height: 100px; background-color: pink;">
  <script>
    function changeColor() {
      document.getElementById("divvy").style.backgroundColor =
"blue";
    }
  </script>
</body>
</html>
```

If you go with your mouse over the pink square (the `div` with `id divvy`), it turns blue immediately. This is because `onmouseover` is added in the HTML and points to the JavaScript function that changes the color of the square.

Let's look at a similar slightly more complicated example.

```
<!doctype html>
<html>
<body>
<div id="divvy" style="width: 100px; height: 100px; background-
color: pink;">
<script>
window.onload = function donenow() {
  console.log("hi");
  document.getElementById("divvy").addEventListener("mousedown",
function() { changeColor(this, "green"); });
  document.getElementById("divvy").addEventListener("mouseup",
function() { changeColor(this, "yellow"); });
  document.getElementById("divvy").addEventListener("dblclick",
function() { changeColor(this, "black"); });
  document.getElementById("divvy").addEventListener("mouseout",
function() { changeColor(this, "blue"); });
}
  console.log("hi2");

  function changeColor(e1, color) {
    e1.style.backgroundColor = color;
  }
</script>
</body>
</html>
```

We are still starting with the pink square. There are four event listeners connected to this div:

- **mousedown**: when the button is pressed on the mouse but is not yet released, the square will turn green.
- **mouseup**: as soon as the mouse button gets released, the square will turn yellow.
- **dblclick**: this is a favorite. What do you think will happen upon a double-click? A double-click contains two **mousedown** events and two **mouseup** events. Before the second **mouseup**, it is not a double-click. So, the square will get the colors green, yellow, green, black (and then stay black until another event is fired).
- **mouseout**: when the mouse leaves the square, it turns blue and stays blue until one of the three above events is fired again.

This allows for a lot of interaction. You can do so many things with this. Just to give you an example of a thing you can do, say you want to have a very dynamic, mouseover-driven, product-help decision tool. It will consist of four columns, and the content of the last three columns (from the right) is dynamic content. The first column is for categories. The second column contains some more specific product categories for each category. The third column consists of individual products, and the fourth column shows product information. This requires a lot of event listeners, and lots of deletion and addition of listeners.

Practice exercise 11.4

Our aim is to change the background color of the element on the page as the various mouse events occur. On **mousedown** over the element, the element will turn green.

When the mouse is over the element, it will turn red. As the mouse moves out of the element boundaries, the color will turn yellow. When the mouse is clicked, the color will go green, and when the mouse is released, it will change to blue. The actions also be logged in the console.

1. Create a blank element on the page and assign a class to it.
2. Select the element using its class name.
3. Assign a variable to the element object from the page.
4. Update the content of the element to say hello world.
5. Using the style properties of the element, update the height and width, then add a default background color to it.
6. Create a function to handle two arguments, the first being a color value as a string and the second being the event object of the trigger.
7. In the function, output the color value to the console, and for the event, output the event type to the console.
8. Add event listeners to the element **mousedown**, **mouseover**, **mouseout**, and **mouseup**. For each of these events, send two arguments to the function that you created: a color value and the event object.
9. Run the code and try it in your browser.

The event target property

Whenever an event gets fired, an event variable becomes available. It has many properties, and you can check it out by using this command in the function that gets fired for the event:

```
console.dir(event);
```

This will show many properties. One of the most interesting properties for now is the `target` property. The target is the HTML element that fired the event. So, we can use it to get information from a web page. Let's look at a simple example.

```
<!doctype html>
<html>
<body>
  <button type="button" onclick="triggerSomething()">Click</button>
  <script>
    function triggerSomething() {
      console.dir(event.target);
    }
  </script>
</body>
</html>
```

In this case, `event.target` is the button element. In the console, the button element and all its properties will be logged, including potential siblings and parents.

A use case where parent properties can come in handy is in the case of HTML forms, where there are multiple input fields and a button. A button in a form would often have the form as its direct parent. Via this parent, data from the input fields can be fetched. This is demonstrated in the following example:

```
<!doctype html>
<html>
<body>
  <div id="welcome">Hi there!</div>
  <form>
    <input type="text" name="firstname" placeholder="First name" />
    <input type="text" name="lastname" placeholder="Last name" />
    <input type="button" onclick="sendInfo()" value="Submit" />
  </form>
  <script>
    function sendInfo() {
      let p = event.target.parentElement;
      message("Welcome " + p.firstname.value + " " + p.lastname.value);
    }

    function message(m) {
      document.getElementById("welcome").innerHTML = m;
    }
  </script>
</body>
</html>
```

```
</script>
</body>
</html>
```

This results in a little form, like this:

Figure 11.1: Basic HTML form

And once you enter data in the fields and hit **Submit**, this is what it looks like:

Figure 11.2: Data entered in the basic HTML form with a dynamic welcome message

With this command, `event.target[CIT]` is the HTML button:

```
let p = event.target.parentElement;
```

The parent element of the button, the form in this case, gets stored in the `p` variable. `p` is the parent and it represents the form element. So, this command will get the value of the input box:

```
p.firstname.value;
```

Similarly, `p.lastname.value` gets the last name. We haven't seen this yet, but with the `value` property, you can get the values of input elements.

Next, both input values are combined and sent to the `message()` function. This function changes the inner HTML of the div to a personalized welcome message, which is why **Hi there!** changes to **Welcome Maaike van Putten**.

Practice exercise 11.5

Change the text in a div element on the page. This exercise will demonstrate how you can get the value from an input field and place it within a page element. It also covers tracking button clicks and details about the event target. You can use the following HTML document as a template, to which you can add JavaScript:

```
<!doctype html>
<html>
<head>
```

```

<title>JS Tester</title>
</head>
<body>
  <div class="output"></div>
  <input type="text" name="message" placeholder="Your Message">
  <button class="btn1">Button 1</button>
  <button class="btn2">Button 2</button>
  <div>
    <button class="btn3">Log</button>
  </div>
  <script>
    // ...
  </script>
</body>
</html>

```

Take the following steps:

1. Using the above HTML as a template, add the JavaScript code, selecting each page element, including the `div`, the input field, and the button element. Assign these element objects to variables in your code.
2. Create an empty array called `log`, which will be used to track and log all the events.
3. Create a function that will capture the event object details in an object, adding it to the `log` array. Get the event target and create an object, adding it to the array that stores the input value at the time, the type of event, the class name of the target element, and the tag name of the target element.
4. Within the logging function, get the value of the content within the input and assign that value to the `textContent` of the `div`.
5. Clear the `div` content after the information is added to the `log` array.
6. Add an event listener to both of the first two buttons that sends the event object to the tracking function created in the earlier steps.
7. Attach an event listener to the third button that outputs the log content to the console.

DOM event flow

Let's go over what happens when you click on an element that has multiple elements associated with it.

We are going to create nested `div` elements. In order to illustrate this, there is a style added to the body. It is actually better to add this style in a head tag, and even better to have a separate CSS file, but this is a bit shorter to read. This is what the nested `div` elements will look like:

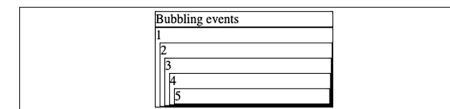


Figure 11.3: Event bubbling in web page

And below is the code associated with it. The script is on the bottom and will be executed when the parts on top are done. It is going to add event listeners to every `div`, and what it will do is log the `innerText`. For the outermost element of the nested `div` elements, this would be 12345, with a new number on every line.

So the question here is, how will it trigger the events? Say we click on the 5, what will be executed? The event of all of the nested `div` elements, or only the one from 5? And if it is going to execute all of them, will it execute them from inner to outer event or the other way around?

```

<!DOCTYPE html>
<html>
  <body>
    <style>
      div {
        border: 1px solid black;
        margin-left: 5px;
      }
    </style>
    <div id="message">Bubbling events</div>
    <div id="output">
      1
      <div>
        2
        <div>
          3
          <div>
            4

```

```

        <div>5</div>
      </div>
    </div>
  </div>
</div>
<script>
  function bubble() {
    console.log(this.innerText);
  }
  let divs = document.getElementsByTagName("div");
  for (let i = 0; i < divs.length; i++) {
    divs[i].addEventListener("click", bubble);
  }
</script>
</body>
</html>

```

In this case, it has the default behavior. It will execute all five events, so each one of every nested div. And it will execute them inside out. So it will start with the innerText of only 5, then 45, until the last one, **12345**:

```

1
2
3
4
5
4
5
3
4
5
2
3
4
5
1
2
3
4
5
>

```

Figure 11.4: Console output event bubbling

This is called **event bubbling**. It's what happens when you trigger the handlers on an element. It runs its own events first, then its parents, and so on. It is called bubbling because it goes up from the inner event to the outer, like water bubbles going up.

You can alter this behavior by adding `true` as a third argument when you add the event listener like this:

```
divs[i].addEventListener("click", bubble, true);
```

This would be the result:

```

1
2
3
4
5
2
3
4
5
3
4
5
4
5
5
>

```

Figure 11.5: Console output event capturing

This moving from the outer element to the inner element is called **event capturing**. It is not used a lot anymore nowadays, but if you do need to implement it, you can use the `useCapture` argument of `addEventListener()` (the third argument) and set it to `true`. It is `false` by default.

The event capturing and bubbling allows us to apply a principle called **event delegation**. Event delegation is the concept where instead of adding event handlers to every element in a certain block of HTML, we define a wrapper and add the event to this wrapper element, and it then applies to all the child elements as well. You'll apply this principle in the next exercise.

Practice exercise 11.6

This example will demonstrate event capturing and the delegation of page elements. By adding event listeners to the parent and children within the main element, this example will order the console messages according to the event capture properties.

All of the `div` elements with a `class` of `box` will have the same event object. We can add the event target, `textContent`, as well into the console so that we can tell which element was clicked.

Use the following template:

```
<!doctype html>
<html>
<head>
  <title>JS Tester</title>
  <style>
    .box {
      width: 200px;
      height: 100px;
      border: 1px solid black
    }
  </style>
</head>
<body>
  <div class="container">
    <div class="box" id="box0">Box #1</div>
    <div class="box" id="box1">Box #2</div>
    <div class="box" id="box2">Box #3</div>
    <div class="box" id="box3">Box #4</div>
  </div>
  <script>
  </script>
</body>
</html>
```

Take the following steps:

1. In the JavaScript code, select all the elements with the classes, and separately select the main container element.
2. Add event listeners to the main container, outputting to the console value of 4 for the `useCapture` argument set to `false`, and 1 for the `useCapture` argument set to `true`.
3. For each of the nested elements, add `click` event listeners with a console `log()` value of 3 for the `useCapture` argument set to `false`, and 2 for `useCapture` set to `true`.

4. Click the page elements to see the event delegation and order of output on the page.
5. Within the box elements on both `click` events, add to the console an output for the event target's `textContent` value.

onchange and onblur

Two other events that are often combined with input boxes are `onchange` and `onblur`. `onchange` gets triggered when an element changes, for example, when the value of an input box changes. `onblur` gets triggered when an object goes out of focus; for example, when you have the cursor in one input box and the cursor goes to another input box, the `onblur` event of the first input box will get triggered.

Here is an example of both in practice; it's an HTML snippet of the body of a web page. The inputs contain `onblur` and `onchange`, and there is an extra function.

```
<!DOCTYPE html>
<html>
<body>
  <div id="welcome">Hi there!</div>
  <form>
    <input type="text" name="firstname" placeholder="First name"
    onchange="logEvent()" />
    <input type="text" name="lastname" placeholder="Last name"
    onblur="logEvent()" />
    <input type="button" onclick="sendInfo()" value="Submit" />
  </form>
  <script>
    function logEvent() {
      let p = event.target;
      if (p.name == "firstname") {
        message("First Name Changed to " + p.value);
      } else {
        message("Last Name Changed to " + p.value);
      }
    }

    function sendInfo() {
      let p = event.target.parentElement;
      message("Welcome " + p.firstname.value + " " + p.lastname.value);
    }
  </script>
</body>
</html>
```

```

function message(m) {
    document.getElementById("welcome").innerHTML = m;
}
</script>
</body>
</html>

```

The `firstname` input box has an `onchange` event. If the value of the data in the input box gets changed, this event gets triggered as soon as the input box loses focus. If the input box loses focus when the value has not changed, nothing happens for `onchange`. This is not true for `onblur`, which is assigned to the `lastname` input box. Even when the value hasn't changed, the event will get triggered.

Another event that is often used with input boxes is `onfocus`, or simply `focus` when used in combination with an event listener. This event is associated with the cursor entering the input box, and it gets fired when the input box gets focused on by the cursor and input can be entered.

Practice exercise 11.7

With two input fields on the page, JavaScript will listen for changes to the content in the input field. Once the input field is not in focus, if the value has been changed, the `change` event will be invoked. `blur` and `focus` are also added to the input fields and will get logged to the console as those events occur. Both input elements will have the same event listeners, and as you change the content of the input fields and remove focus, the output text content will update using the values of the input field that triggered the event.

Use the following HTML template:

```

<!doctype html>
<html>
<head>
  <title>JS Tester</title>
</head>
<body>
  <div class="output1">
    <input type="text" placeholder="First Name" name="first"><br>
    <input type="text" placeholder="Last Name" name="last"><br>
    <script>

```

```

</script>
</body>
</html>

```

Now take the following steps:

1. Within the JavaScript code, put the HTML output element into a variable object that you can use to display content on the page.
2. Select both input fields. You can use `querySelector()` and `"input[name='first']"`, which will allow you to make your selection using the input field name.
3. Add an event listener to the first input and the second input. The event listener should be a `change` event to track changed values. This will only be invoked if the value in the field is changed and you click off the input field.
4. Create a separate function to handle the output of the content to the page, updating the `textContent` of the output element.
5. Send the values of the input fields as they get changed to the output element `textContent`.
6. Add four additional event listeners and listen for `blur` and `focus` on each input. Once the event gets triggered, output in the console the value of the event type.

Key event handler

There are several key events. One of them is `onkeypress`. `onkeypress` gets triggered, well, you may have guessed this, whenever a key gets pressed. Pressing means here when the button is pressed and released. If you want an event to happen as soon as the button is pressed (so before releasing), you can use the `onkeydown` event. If you want the event to happen on release, you can use the `onkeyup` event.

There are many things we can do with key events. For example, we can restrict what characters can be entered in an input box. Every time a key gets pressed, we can check the character and decide whether it gets to stay.

We can get the key that triggered the event using:

```
event.key;
```

The following HTML code has two input boxes. Can you see what is happening here?

```
<!doctype html>
<html>
<body>
<body>
<div id="wrapper">JavaScript is fun!</div>
<input type="text" name="myNum1" onkeypress="numCheck()">
<input type="text" name="myNum2" onkeypress="numCheck2()">
<script>
function numCheck() {
    message("Number: " + isNaN(event.key));
    return !isNaN(event.key);
}

function numCheck2() {
    message("Not a number: " + isNaN(event.key));
    return isNaN(event.key);
}

function message(m) {
    document.getElementById('wrapper').innerHTML = m;
}
</script>
</body>
</html>
```

The first one checks if a value is a number, and if it is a number, it will write Number: true at the top; else, it will write Number: false at the top. The second one is checking for if a value is not a number; if it is not a number, it will write Not a number: true; else, it will write Not a number: false.

So this is one way of using the onkeypress event, but we can do even more. We can add a return statement to our onkeypress event, like this:

```
onkeypress="return numCheck2()";
```

If true is returned, the key value gets added to the input box; if false is returned, the key value is not added.

The following code snippet only allows numbers to be entered in the input box. Whenever the user tries to type something else, the function restricts it.

```
<!doctype html>
<html>
<body>
<body>
<div id="wrapper">JavaScript is fun!</div>
<input type="text" name="myNum1" onkeypress="return numCheck()"
onpaste="return false">
<input type="text" name="myNum2" onkeypress="return numCheck2()"
onpaste="return false">
<script>
function numCheck() {
    message(!isNaN(event.key));
    return !isNaN(event.key);
}

function numCheck2() {
    message(isNaN(event.key));
    return isNaN(event.key);
}

function message(m) {
    document.getElementById("wrapper").innerHTML = m;
}
</script>
</body>
</html>
```

As you can see, return gets included in onkeypress to ensure that only numbers can be entered. One other thing might have caught your eye: onpaste="return false". This is to deal with smart people who copy and paste numbers to a non-numeric field or other characters to a numeric field and still manage to get illegal characters in there.

Practice exercise 11.8

By recognizing key presses and detecting the values of characters as key presses occur with the element in focus, we can also detect if content is pasted into an input field.

1. Create two input fields within your HTML. Add an element to output content too.
2. Using JavaScript, select the page elements. You can assign a variable called `output` to the element with a class of `output`. Create another variable, `eles`, and select all the input fields (using `querySelectorAll()`) as its value. This way, we can loop through the node list and assign the same events to all the matching elements.
3. Using `forEach()`, iterate through all the input elements from the page. Add the same event listeners to all of them.
4. Add a `keydown` event listener and check if the value is a number. If it is a number, then add it to the output area.
5. On `keyup`, output to the console the value of the key.
6. Check if there was a paste in the input field; if there was, then you can output the word `paste` to the console.

Drag and drop elements

There are also special event handlers for dragging and dropping. We need a starting point to be able to drag and drop something. Let's create the CSS and HTML for a dragging and dropping area.

```
<!doctype>
<html>
<head>
<style>
  .box {
    height: 200px;
    width: 200px;
    padding: 20px;
    margin: 0 50px;
    display: inline-block;
    border: 1px solid black;
  }

  #dragne {
    background-color: red;
  }
</style>
</head>
```

```
<body>
<div class="box">1</div>
<div class="box">2</div>
</body>
</html>
```

Now we are also going to include an element that is going to be dragged and dropped. In order to mark an element as something that can be dragged, we need to add the attribute `draggable`. This is the code we are going to include in our second div, with the first div around it:

```
<div class="box"> 1
<div id="dragne" draggable="true">
  Drag Me Please!
</div>
</div>
```

Then we need to decide what we are going to do when we drop the draggable element. We need to specify this within the box it can be dragged to. We are going to add the functionality to both boxes, so it can be dragged over to one and back to the other.

```
<div class="box" ondrop="dDrop()" ondragover="nDrop()">
  1
<div id="dragne" ondragstart="dStart()" draggable="true">
  Drag Me Please!
</div>
</div>
<div class="box" ondrop="dDrop()" ondragover="nDrop()">2</div>
```

And here is the script that will be added to the end of the body:

```
<script>
let holderItem;

function dStart() {
  holderItem = event.target;
}

function nDrop() {
  event.preventDefault();
}
```

```
function dDrop() {
  event.preventDefault();
  if (event.target.className == "box") {
    event.target.appendChild(holderItem);
  }
}
</script>
```

We start by specifying a variable in the script for the item that we want to hold when dragging. When the `ondragstart` event gets triggered, we are going to store the element that is being dragged in the `holderItem` variable. Normally, when you drag, dropping is not allowed by the design of HTML. In order to allow the drop, you'll have to prevent the default event that means the item you want to drop cannot be dropped. You can do this with:

```
event.preventDefault();
```

Usually, before you prevent the default behavior, you would do some checks to see whether the element that is being dropped can be accepted at that place. In the example above, we check whether the class name of the element that it is being dropped to is `box`. If that's the case, we append `holderItem` as a child to the `box`.

We have created a page that allows moving an HTML element from one box to another. If you try to release it anywhere else, the element goes back instead to its previous location.

Practice exercise 11.9

This will be an "I'm not a robot" check. Drag and drop can be used to ensure that it's a live user that is acting on a page rather than a robot. This exercise will demonstrate how to create a visual dragging effect on an active element, in which the user clicks the mouse down to create a drag action, and once the mouse button is released, the drop event occurs. The successful actions are logged to the console.

You can use the following template:

```
<!doctype html>
<html>
<head>
  <title>JS Tester</title>
  <style>
    .box {
```

```
width: 100px;
height: 100px;
border: 1px solid black;
background-color: white;
}
.red {
  background-color: red;
}
</style>
</head>
<body>
  <div class="box">1
    <div id="dragne" draggable="true">
      Drag Me Please!
    </div>
  </div>
  <div class="box">2</div>
</script>
</body>
</html>
```

The preceding HTML creates styles for an element that will be used for dropping, and sets a width, height, and border. It creates another class called `red` and adds a red background to the active element so that it shows as active, along with two `div` elements that will have classes of the `box` element for dropoff. Finally, we create a `div` nested in one of the boxes that has an `id` of `dragne` and an attribute of `draggable` set to `true`, with some instructive text added to aid the user. Complete the script by taking the following steps:

1. Select the draggable element as an object in your JavaScript code.
2. Add an event listener of `dragstart`, where it updates the draggable element to 0.5 opacity.
3. Add another event listener of `dragend` that removes the value for the opacity.
4. Using `querySelectorAll()`, select all the dropoff boxes.
5. Add event listeners to all the dropoff boxes, setting things such that the `red` class is added to an element whenever the user triggers the `dragerenter` event. This will indicate to the user that the action is taking place.
6. Set `dragerover`, adding a `preventDefault()` method to the element to disable any actions that might already exist.

7. On `dragleave`, remove the red class.
8. Adding the event listener of `drop` to the box, append the `draggable` element to the event target.
9. To work across all elements in the same way, remove the default action of the element. You can use the `preventDefault()` method to this event to disable any actions that might already exist.
10. You can add console log messages to any of these events to better track them.

Form submission

When a form gets submitted, an event can be triggered. This can be achieved in different ways, and one of them is to add to the `form` element the `onsubmit` attribute.

```
<form onsubmit="doSomething()">
```

The function that is listed there will get triggered whenever input of type `submit` is submitted.

```
<input type="submit" value="send">
```

We can do more with the HTML of the `form` element; for example, we can redirect to another page. We have to specify the way we want to send the form values using the `method` attribute and the location page using the `action` attribute.

```
<form action="anotherpage.html" method="get" onsubmit="doStuff()">
```

Don't worry about the `get` for now; this just means that values get sent via the URL. URLs look like this when you use `get`:

```
www.example.com/anotherpage.html?name=edward
```

After the question mark, the variables that are sent along are shown in key-value pairs. This is the form that created the URL when `edward` was inserted for `name`.

```
<!doctype html>
<html>
<body>
  <form action="anotherpage.html" method="get">
    <input type="text" placeholder="name" name="name" />
    <input type="submit" value="send" />
  </form>
</body>
</html>
```

`anotherpage.html` can use the variables from the URL. This can be done in the JavaScript of `anotherpage.html` doing something like this:

```
<!doctype html>
<html>
<body>
  <script>
    let q = window.location.search;
    let params = new URLSearchParams(q);
    let name = params.get("name");
    console.log(name);
  </script>
</body>
</html>
```

So far we have been submitting forms using the `action` and `onsubmit` attributes. `action` redirects to another location. This could be the API endpoint of a different page. `onsubmit` specifies an event that is fired when the form gets submitted.

There are more cool things we can do with the `onsubmit` event of forms. Remember the use of `return` for `onkeypress`? We can do something similar here! If we make the called function return a Boolean, the form will only get submitted when the Boolean is true.

This comes in very handy if we want to do some form validation before sending it. Have a look at this code and see if you can figure out when it can be submitted.

```
<!doctype html>
<html>
<body>
  <div id="wrapper"></div>
  <form action="anotherpage.html" method="get" onsubmit="return
    valForm()">
    <input type="text" id="firstName" name="firstName"
      placeholder="First name" />
    <input type="text" id="lastName" name="lastName"
      placeholder="Last name" />
    <input type="text" id="age" name="age" placeholder="Age" />
    <input type="submit" value="submit" />
  </form>
  <script>
    function valForm() {
      var p = event.target.children;
```

```

    if (p.firstName.value == "") {
      message("Need a first name!!");
      return false;
    }
    if (p.lastName.value == "") {
      message("Need a last name!!");
      return false;
    }
    if (p.age.value == "") {
      message("Need an age!!");
      return false;
    }
    return true;
  }

  function message(m) {
    document.getElementById("wrapper").innerHTML += m;
  }
}
</script>
</body>
</html>

```

This form contains three input fields and one input button. The fields are for last name, first name, and age. When one of them is missing, the form will not submit, because the function will return `false`. A message will also be added to the div above the form, explaining what went wrong.

Practice exercise 11.10

This will be about creating a form validator. In this exercise, you will be checking to ensure that the desired values are entered into the input fields. The code will check the input values that are entered by the user to match predetermined conditions for those field values.

1. Set up a form, adding three input fields inside: First, Last, and Age. Add a submit button.
2. Within the JavaScript code, select the form as an element object.
3. Add an submit event listener to the form.
4. Set up the default value for error as `false`.
5. Create a function named `checker()`, which will check to see the length of a string and output the string length to the console.

6. Add conditions to each of the field values, first checking if the value is there and then returning an error if the response is `false`, before changing the error variable to `true`.
7. Use `console.log()` to log the details about the error.
8. For the age input value, check the value to see if the age provided is 19 or over, otherwise cause an error.
9. At the end of the validation, check if `error` is `true`; if it is, use `preventDefault()` to stop the form submission. Log the errors to the console.

Animating elements

Lastly, we want to show you that you can animate using HTML, CSS, and JavaScript. This allows us to do even cooler things with our web page. For example, we can trigger an animation as an event. This can be used for many different purposes, for example, to illustrate an explanation, to draw the user's eyes to a certain location, or to play a game.

Let's show you a very basic example. We can use the `position` key and set it to `absolute` in CSS. This makes the position of the element relative to its nearest positioned parent. Here, that would be the body. This is the code for a purple square that moves from left to right when a button is clicked.

```

<!doctype html>
<html>
<style>
  div {
    background-color: purple;
    width: 100px;
    height: 100px;
    position: absolute;
  }
</style>
<body>
  <button onclick="toTheRight()">Go right</button>
  <div id="block"></div>

  <script>
    function toTheRight() {
      let b = document.getElementById("block");
      let x = 0;
      setInterval(function () {

```

```

        if (x === 600) {
            clearInterval();
        } else {
            x++;
            b.style.left = x + "px";
        }
    }, 2);
}
</script>
</body>
</html>

```

We need to give the `div` block an absolute position, because we rely on the CSS `left` property to make it move. In order to be to the left of something, that something needs to be absolute, else the `left` property cannot be positioned relative to it. In this case, we need to be a certain number of pixels to the left of the `div`; this is why we need the position of the `div` to be absolute, so the position of the moving box can be relative to that of its parent.

When we click the button, the function `toTheRight()` gets triggered. This function grabs `block` and stores it in `b`. It sets `x` to 0. Then we use a very powerful built-in JavaScript function: `setInterval()`. This function keeps on evaluating an expression until `clearInterval()` gets called. This is done when `x`, the measure of how far we are to the left, reaches 600. It repeats it every 2 milliseconds, which gives it the sliding look.

You can at the same time also set different positions, like `style.top`, `style.bottom`, and `style.right`, or append new elements to create a snow effect, or show constantly driving cars. With this in your toolbox, the sky is the limit.

Practice exercise 11.11

Here we will click the purple square and watch it move on the page. This exercise will demonstrate creating the events for a simple interactive element on the page. The purple square will move every time it's clicked; once it reaches the boundaries of the page, it will change direction from left to right and right to left, depending on what side it hits.

1. Set up styling for an element, setting `height` and `width` before setting `position` to `absolute`.
2. Create an element that you want to move on the page.

3. Select and store the element using JavaScript.
4. Set up an object with values for `speed`, `direction`, and `position`.
5. Add an event listener for if the element is clicked on.
6. Set a default value of 30 for the value of the interval counter.
7. If the counter is less than 1, then end the interval and clear it.
8. Once the interval has run 30 times using the value of `x`, the element will stand still and wait to be clicked again.
9. While in motion, check if the position value is greater than 800 or less than 0, which means it needs to change direction. The `direction` value will provide the direction of movement. If the movement takes the box outside the boundaries of the container, we need to send it in the other direction. This can be done by multiplying by negative one. If the value is positive, it will become negative, sending the element to the left. If the value is negative, it will become positive, sending the element to the right.
10. Update the `style.left` position value of the element, assigning the value of the position that was updated. Add `px`, as the assigned value of the style is a string.
11. Output the result to the console.

Chapter projects

Build your own analytics

Figure out which elements are clicked on in a page and record their IDs, tags, and class name.

1. Create a main container element within your HTML.
2. Add four elements inside the main element, each having a class of `box` and a unique ID with unique text content.
3. Set up your JavaScript code to contain an array that you can use for tracking, adding details from each click into it.
4. Select the main container element as a variable object in your code.
5. Add an event listener to capture clicks on the element.
6. Create a function to handle the clicks. Get the target element from the event object.

7. Check if the element has an ID, so that you don't track clicks on the main container.
8. Set up an object to track the values; include the element `textContent`, `id`, `tagName`, and `className`.
9. Add the temporary object to your tracking array.
10. Output the values captured in your tracking array to your console.

Star rating system

Create a star rating component that is fully interactive and dynamic using JavaScript.



Figure 11.6: Creating a star rating system

You can use the following HTML and CSS as a starting template. Provide the completed script element as your answer.

```
<!DOCTYPE html>
<html>
<head>
  <title>Star Rater</title>
  <style>
    .stars ul {
      list-style-type: none;
      padding: 0;
    }
    .star {
      font-size: 2em;
      color: #ddd;
      display: inline-block;
    }
    .orange {
      color: orange;
    }
    .output {
      background-color: #ddd;
    }
  </style>
</head>
<body>
  <div>
    <ul class="stars">
      <li class="star"></li>
      <li class="star"></li>
      <li class="star"></li>
      <li class="star"></li>
      <li class="star"></li>
    </ul>
    <div class="output"></div>
  </div>
</body>
</html>
```

```
</style>
</head>
<body>
  <ul class="stars">
    <li class="star"></li>
    <li class="star"></li>
    <li class="star"></li>
    <li class="star"></li>
    <li class="star"></li>
  </ul>
  <div class="output"></div>
</script>
</body>
</html>
```

Take the following steps:

1. Select all the stars within the `ul` that have a class of `stars` into an object, and create another object for the output element.
2. Create another object to contain the results of calling `querySelectorAll()` on elements with a class of `star`.
3. Loop through the resulting node list, adding to the element object the value of the index plus 1, and adding an event listener listening for a click. Attach a function called `starRate()` to the `click` event of each star element.
4. Within the `starRate()` function, add to the output the value of the star using the event target and the element object's star value, which was set in the previous step.
5. Loop through all the stars using `forEach()` to check if the index value of the star element is less than the star value; if it is, apply a class of `orange`. Otherwise, remove the class of `orange` from the `classList` element.

Mouse position tracker

Track your mouse's `x` and `y` position within an element. As you move the mouse within the element, the `x` position and `y` position values will update.

1. Create a page element and add sizing to it, including height and width. Create a class style named `active` that has a `background-color` property of `red`. Finally, create an output element that will contain your text.

2. Select the main container element and add event listeners to it. Listen for `mouseover`, `mouseout`, and `mousemove`.
3. On `mouseover`, add the active class, and on `mouseout`, remove the active class.
4. On `mousemove`, call a function that tracks the event element `clientX` and `clientY` positions, embeds them in a human-readable sentence, and outputs it into the output element.

Box clicker speed test game

The objective here is to click on the red box as quickly as possible once it appears. The box will be randomly placed within a container and positioned with random values. The box will have an event listener that will track the start and click times to calculate the duration of the click events. You can use the following template, as the HTML gets a little complex here—just add the `<script>` element to make the HTML interactive!

```
<!DOCTYPE html>
<html>

<head>
  <title>Click Me Game</title>
  <style>
    .output {
      width: 500px;
      height: 500px;
      border: 1px solid black;
      margin: auto;
      text-align: center;
    }

    .box {
      width: 50px;
      height: 50px;
      position: relative;
      top: 50px;
      left: 20%;
      background-color: red;
    }

    .message {
      text-align: center;
    }
  </style>
</head>
<body>
  <div class="output"></div>
  <div class="message"></div>
</body>
</html>
```

```
padding: 10px;
font-size: 1.3em;
}
</style>
</head>

<body>
  <div class="output"></div>
  <div class="message"></div>
</script>

</script>
</body>
</html>
```

Work with the above HTML code using JavaScript.

1. There are two `div` elements, one with a class of `output` for the gameplay area and another with a class of `message` to provide instructions to the player. Select those main elements as objects using JavaScript.
2. Using JavaScript, create another element within the output element, and create a `div` that will be the main clickable object. Attach a style called `box` to the new element and append it to the output element.
3. Using JavaScript, add to the message area instructions for the user: `Press to start`. They will need to click the newly created `div` with a class of `box` to start the game.
4. Create a global object called `game` to track the timer and the start time values. This is to be used to calculate the duration in seconds between when the element is shown and when the player clicks it. Set `start` to `null`.
5. Create a function that will generate a random number and return a random value, with the argument being the maximum value you want to use.
6. Add an event listener to the box element. Once clicked, this should start the gameplay. Set the display of the box element to `none`. Using the `setTimeout()` method, invoke a function called `addBox()` and set the timeout to a random millisecond value. You can adjust as needed; this will be the time between the click object box being shown and it disappearing. If the start value is `null`, then add text content to the message area of loading.
7. If `start` has a value, then get the Unix time value using `getTime()` of the current date object, subtract the game start time from the current time value in milliseconds, and then divide by 1,000 to get the value in seconds. Output the result to the message element for the player to see their score.

8. Create a function to handle the clicks, adding the box once the timer is up. Update the message text content to say Click it... Set the game start value to the current time in milliseconds. Apply the style of block to the element so it shows on the page.
9. From the available space (500 total container width minus the 50 box width) set a random position above and to the left of the element using the `Math.random()`.
10. Play the game and update the styling as needed.

Self-check quiz

1. Where can you find `window.innerHeight` and `window.innerWidth`?
2. What does `preventDefault()` do?

Summary

In this chapter, we have dealt with quite a few topics to increase the interactivity of web pages. We saw the different ways to specify events and then we dived into some different event handlers in more detail. The `onload` event handler gets fired when the element it was specified on, often the document object, is fully loaded. This is great to wrap other functionality in, because it avoids selecting DOM content that is not there yet.

We also saw the mouse event handlers, for responding to all the different things that can be done with a mouse on a web page. The use of all these event handlers is very similar, but they each enable a different type of interaction with the user. We also saw that we can access the element that fired an event by calling `event.target`. This property holds the element that fired an event.

We also dived into `onchange`, `onblur`, and the key event handlers in more detail. After that, we saw how to trigger interaction when forms are submitted. We looked at the HTML `action` attribute, which redirects the handling of submission, and the `onsubmit` event, which deals with form submission. We then saw some things that we can do with all these events, such as specifying a drag and drop on a page and animating elements.

In the next chapter, we will move on to some more advanced topics that will take your existing JavaScript skills to the next level!

12

Intermediate JavaScript

The concepts and solution approaches presented up to this point in the book are not the only way to think about solving issues. In this chapter, we will challenge you to look a little deeper, be curious, and practice the good habit of optimizing solutions.

In previous chapters, you were promised great things about this chapter because the optimal use of some built-in methods require knowledge of regular expressions, which we will cover in this chapter. There is a lot more fun to be had though—here is a list of topics that we'll cover:

- Regular expressions
- Functions and the arguments object
- JavaScript hoisting
- Strict mode
- Debugging
- Using cookies
- Local storage
- JSON

As you can see, a selection of diverse topics, but all advanced and fun. The sections in this chapter are not as related to each other as you might have gotten used to by now. They are mostly individual topics that can help to really enhance your understanding and improve your JavaScript knowledge a lot.



Note: exercise, project and self-check quiz answers can be found in the Appendix.

Regular expressions

Regular expressions, also known as **regex**, are simply ways to describe text patterns. You can consider them next-level strings. There are different regex implementations. This means that depending on the interpreter, regex might differ a bit in the way they're written. However, they are somewhat standardized, so you write them (almost) the same for all versions of regex. We are going to use regex for JavaScript.

Regex can be very useful in many situations, for example when you need to look for errors in a large file or retrieve the browser agent a user is using. They can also be used for form validation, as with regex you can specify valid patterns for field entries such as email addresses or phone numbers.

Regex is not only useful for finding strings, but can also be used for replacing strings. By now you might think, *so regex is amazing, but is there a catch?* And yes, unfortunately, there is a catch. At first, regex might kind of look like your neighbor's cat walked over your keyboard and just typed some random characters by accident. This regex checks for a valid email, for example:

```
/([a-zA-Z0-9._-]+@[a-zA-Z0-9._-]+\.[a-zA-Z0-9._-]+)/g
```

Fear not, after this section, you will be able to decipher the secret patterns within the regex. We are not going to go through everything there is to say about regex, but we will establish a solid level of familiarity that will allow you to work with them and expand your knowledge as you go.

Let's start off easy. The regex pattern is specified between two slashes. This is a valid regex expression:

```
/JavaScript/
```

The above expression will match if a given string contains the word `JavaScript`. When it matches, this means the result is positive. And this can be used to do many things.

We can use the JavaScript built-in `match()` function for this. This function returns the regex match on the result (if there is one) in the form of the substring that matched the starting position of this string and the input string.

There are actually other built-in functions that use regex, but we will see them later. `match()` is just a convenient function to demonstrate how regex works. You can see it in action here:

```
let text = "I love JavaScript!";
console.log(text.match(/javascript/));
```

This logs null because it is case-sensitive by default and therefore is not a match. If we had looked for `/ava/` or simply `/a/`, it would have matched because it contains `ava` and `a`. If you want it to be case-insensitive, you can specify this using an `i` after the slash. In this case-insensitive example, the expression will match the previous string:

```
console.log(text.match(/javascript/i));
```

This will actually log the result, because it is now case-insensitive, and from that point of view, our string does contain `javascript`. Here is the result:

```
{
  'JavaScript',
  index: 7,
  input: 'I love JavaScript!',
  groups: undefined
}
```

The result is an object, containing the found match and the index it started on, as well as the input that was looked through. The groups are undefined. You can create groups with round parentheses, as you'll see when we get to the section on groups.

You can often find regex in JavaScript in combination with the built-in search and replace method on strings, which we'll cover next.

Specifying multiple options for words

In order to specify a certain range of options, we can use this syntax:

```
let text = "I love JavaScript!";
console.log(text.match(/javascript[nodejs|react|j]/));
```

Here, the expression matches either `javascript`, `nodejs`, or `react`. At this point, we are only matching for the first encounter and then we quit. So this is not going to find two or more matches right now—it will output the same thing as before:

```
let text = "I love React and JavaScript!";
console.log(text.match(/javascript[nodejs|react|j]/));
```

It logs this:

```
{
  'React',
  index: 7,
  input: 'I love React and JavaScript!',
  groups: undefined
}
```

If we wanted to find all matches, we could specify the global modifier, `g`. It is very similar to what we did for case-insensitive searches. In this example, we are checking for all matches, and it is case-insensitive. All the modifiers are behind the last slash. You can use multiple modifiers at the same time as we do below, or you could decide to only use `g`:

```
let text = "I love React and JavaScript!";
console.log(text.match(/javascript[nodejs|react|j]g));
```

This returns both `React` and `JavaScript` as a result:

```
[ 'React', 'JavaScript' ]
```

As you can see, the result looks very different now. As soon as you specify `g`, the match function will just return an array of the matching words. This is not too exciting in this case, since these are the words we asked for. But it can be more of a surprise with a more complex pattern. This is exactly what we'll learn next.

Character options

So far, our expressions are quite readable, right? The character options are where things start to look, well, intense. Say we want to search for a string of only one character equal to `a`, `b`, or `c`. We would write it like this:

```
let text = "d";
console.log(text.match(/[abc]/));
```

This will return null because `d` is not `a`, `b`, or `c`. We can include `d` like this:

```
console.log(text.match(/[abcd]/));
```

This will log:

```
[ 'd', index: 0, input: 'd', groups: undefined ]
```

Since this is a range of characters, we can write it shorter, like this:

```
let text = "d";
console.log(text.match(/[a-d]/));
```

And if we wanted any letter, lowercase or uppercase, we would write this:

```
let text = "t";
console.log(text.match(/[a-zA-Z]/));
```

We could actually also use the case-insensitive modifier to achieve the same thing, but this would apply to the regex pattern as a whole, and you might only need it to apply for the specific character:

```
console.log(text.match(/[a-z]/i));
```

We would get a match on both of the preceding options. If we wanted to include numbers as well, we would write:

```
console.log(text.match(/[a-zA-Z0-9]/));
```

As you can see, we can just concatenate ranges to specify one character, much like we could concatenate possible options for that specific character, like `[abc]`. The example above specifies three possible ranges. It will match any lowercase or uppercase letter from a to z and all numeric characters as well.

It doesn't mean that it can only match a one-character string by the way; it will just match the first matching character in this case because we didn't add the global modifier. However, these special characters won't match:

```
let text = "æ!";
console.log(text.match(/[a-zA-Z0-9]/));
```

To address the difficulty of complex characters not matching an expression, the dot functions as a special wildcard character in regex that can match any character. So what do you think this does?

```
let text = "Just some text.";
console.log(text.match(/./g));
```

Since it has the global modifier, it is going to match any character. This is the result:

```
[
  'J', 'u', 's', 't',
  ' ', 's', 'o', 'm', 'e',
  ' ', 't', 'e', 'x', 't',
  '.',
]
```

```
'e', ' ', 't', 'e',
'x', 't', ' ',
]
```

But what if you only wanted to find a match for the dot character itself? If you want a special character (one that is used in regex to specify a pattern) to have a normal meaning, or a normal character to have a special meaning, you can escape it using the backslash:

```
let text = "Just some text.";
console.log(text.match(/\./g));
```

In this example, we escape the dot by adding a preceding backslash. Therefore, it doesn't function as a wildcard and it is going to look for a literal match. This is what it will return:

```
[ '. ' ]
```

There are some normal characters that get a special meaning by adding a backslash before them. We are not going to cover them in depth, but let's have a look at some examples:

```
let text = "I'm 29 years old.";
console.log(text.match(/\d/g));
```

If we escape the `d`, `\d`, it matches any digit. We are doing a global search so it will specify any digit. This is the result:

```
[ '2', '9' ]
```

We can also escape the `s`, `\s`, which matches all whitespace characters:

```
let text = "Coding is a lot of fun!";
console.log(text.match(/\s/g));
```

The above example will just return a few spaces, but tabs and other types of whitespace are also included:

```
[ ' ', ' ', ' ', ' ', ' ', ' ', ' ' ]
```

A very useful one is `\b`, which matches text only when it's at the beginning of a word. So, in the following example, it is not going to match the instances of `in` in `beginning`:

```
let text = "In the end or at the beginning?";
console.log(text.match(/\bin/gi));
```

This is what it will end up logging:

```
[ 'In' ]
```

Even though you can check for characters being numbers, the `match()` method belongs to the `string` object, so you implement it on numeric variables. For example, try the following:

```
let nr = 357;
console.log(nr.match(/3/g));
```

You should receive a `TypeError` saying that `nr.match()` is not a function.

Groups

There are many reasons to group your regex. Whenever you want to match a group of characters, you can surround them with parentheses. Have a look at this example:

```
let text = "I love JavaScript!";
console.log(text.match(/(love|dislike)\s(javascript|spiders)/g));
```

Here it is going to look for either `love` or `dislike`, followed by a whitespace character, followed by `javascript` or `spiders`, and it will do so for all occurrences while ignoring whether they are in uppercase or lowercase. This is what it will log:

```
[ 'love JavaScript' ]
```

Let's just say we can match on roughly four combinations here. Two of them seem to make more sense to me personally:

- Love spiders
- Dislike spiders
- Love JavaScript
- Dislike JavaScript

Groups are very powerful when we know how to repeat them. Let's see how to do that. Very often, you'll find yourself in need of repeating a certain regex piece. We have several options for this. For example, if we want to match any four alphanumeric characters in a sequence, we could just write this:

```
let text = "I love JavaScript!";
console.log(text.match(/[a-zA-Z0-9][a-zA-Z0-9][a-zA-Z0-9][a-zA-Z0-9]/g));
```

This will produce the following as output:

```
[ 'love', 'Java', 'Scri' ]
```

This is a terrible way to go about repeating a block: let's look for better options. If we only want it to be present 0 or 1 times, we can use the question mark. So this is for optional characters, for example:

```
let text = "You are doing great!";
console.log(text.match(/n?g/g));
```

This looks for a `g` character that may or not be preceded by an `n`. Therefore, this will log:

```
[ 'ng', 'g' ]
```

Arguably, one time is not really an example of repeating. Let's look at getting more repetitions. If you want something at least once, but optionally more often, you can use the plus sign: `+`. Here is an example:

```
let text = "123123123";
console.log(text.match(/(123)+/));
```

This is going to match for the group `123` one or more times. And since this string is present, it will find a match. This is what will be logged:

```
[ '123123123', '123', index: 0, input: '123123123', groups: undefined ]
```

It matches the whole string in this case, since it is just `123` repeated. There are also situations where you want to have a certain piece of regex match any number of times, which can be indicated with the asterisk: `*`. Here is an example regex pattern:

```
/(123)*a/
```

It will match with any `a` preceded by `123` any number of times. So it will match on the following, for example:

- 123123123a
- 123a
- a
- ba

The last thing to note about repeating is that we can be more specific as well. We do this using this syntax `{min, max}`. Here is an example:

```
let text = "abcabcabc";
console.log(text.match(/(abc){1,2}/));
```

This will log:

```
[ 'abcabc', 'abc', index: 0, input: 'abcabcabc', groups: undefined ]
```

It does this because it will match on `abc` both once and twice. As you can see, we have been using groups, but `groups` is still undefined in the output. In order to specify groups, we'll have to name them. Here's an example of how to do it:

```
let text = "I love JavaScript!";
console.log(text.match(/(?<language>javascript)/i));
```

This will output:

```
[
  'JavaScript',
  'JavaScript',
  index: 7,
  input: 'I love JavaScript!',
  groups: [Object: null prototype] { language: 'JavaScript' }
]
```

There is more to say about regex, but this should already enable you to do quite a lot of cool things with it. Let's have a look at some practical examples.

Practical regex

Regex can be of great use in many situations—anywhere you need to match certain string patterns, regex will come in handy. We are going to discuss how you can use regex in combination with other string methods, and how you can use it to validate email addresses and IPv4 addresses.

Searching and replacing strings

In *Chapter 8, Built-In JavaScript Methods*, we saw the search and replace methods on strings. We would have liked our search to be case-insensitive, though. Guess what—we can use regex for this!

```
let text = "That's not the case.";
console.log(text.search(/Case/i));
```

Adding the `i` modifier here ignores the distinction between uppercase and lowercase. This code returns 15, which is the starting index position of the match. This cannot be done using the normal string input.

How do you think we can alter the behavior of the replace method using regex in such a way that we can replace all instances rather than the first instance of a string? Again, with a modifier! We use the global modifier (`g`) for this. To get a feel for the difference, look at this expression without `g`:

```
let text = "Coding is fun. Coding opens up a lot of opportunities.";
console.log(text.replace("Coding", "JavaScript"));
```

This is what it outputs:

```
JavaScript is fun. Coding opens up a lot of opportunities.
```

Without regex, it only replaces the first encounter. This time, let's see it with the `g` global modifier:

```
let text = "Coding is fun. Coding opens up a lot of opportunities.";
console.log(text.replace(/Coding/g, "JavaScript"));
```

The result is as follows:

```
JavaScript is fun. JavaScript opens up a lot of opportunities.
```

As you can see, all occurrences are replaced.

Practice exercise 12.1

Find and replace strings. The following exercise involves replacing characters in a specified string value. The first input field will indicate which character string will be replaced, and the second input field will indicate which characters will replace them once the button is clicked.

Use the HTML below as a template, and add the JavaScript needed to complete the task:

```
<!doctype html>
<html>

<head>
  <title>Complete JavaScript Course</title>
</head>

<body>
  <div id="output">Complete JavaScript Course</div>
  Search for:
  <input id="sText" type="text">
  <br> Replace with:
  <input id="rText" type="text">
  <br>
  <button>Replace</button>
</script>
</body>
</html>
```

Take the following steps:

1. Select each of the three page elements using JavaScript and assign the element objects as variables so that they can be easily referenced in your code.
2. Add an event listener to the button to invoke a function when clicked.
3. Create a function named `lookup()` that will find and replace the text in the output element. Assign the output element's text content to a variable named `s`, and then assign the value of the input we are replacing to another variable named `rt`.
4. Create a new regex with the value of the first input field, which will allow you to replace the text. Using the regex, check for a match with the `match()` method. Wrap this with a condition that will execute a block of code if matches are found.
5. If the match is found, use `replace()` to set the new value.
6. Update the output area with the newly created and updated text output.

Email validation

In order to create a regex pattern, we need to be able to describe the pattern with words first. Email addresses consist of five parts, in the form of `[name]@[domain].[extension]`.

Here are the five parts explained:

1. `name`: One or more alphanumerical characters, underscores, dashes, or dots
2. `@`: Literal character
3. `domain`: One or more alphanumerical characters, underscores, dashes, or dots
4. `..`: Literal dot
5. `extension`: One or more alphanumerical characters, underscores, dashes, or dots

So, let's do the steps for regex:

1. `[a-zA-Z0-9._-]+`
2. `@`
3. `[a-zA-Z0-9._-]+`
4. `\.` (remember, the dot is a special character in regex, so we need to escape it)
5. `[a-zA-Z0-9._-]+`

Putting it all together:

```
/([a-zA-Z0-9._-]+@[a-zA-Z0-9._-]+\.[a-zA-Z0-9._-]+)/g
```

Let's look at this regex in action:

```
let emailPattern = /([a-zA-Z0-9._-]+@[a-zA-Z0-9._-]+\.[a-zA-Z0-9._-]+)/g;
let validEmail = "maaike_1234@email.com";
let invalidEmail = "maaike@email.com";
console.log(validEmail.match(emailPattern));
console.log(invalidEmail.match(emailPattern));
```

We tested the pattern on both a valid and an invalid email address, and this is the output:

```
{ 'maaike_1234@email.com' }
null
```

As you can see, it returns a result for the valid email and it returns null (no match) for the invalid email.

Practice exercise 12.2

Create an application that uses JavaScript to check whether the string value of an input is a validly formatted email using regex. Look at the following template:

```
<!doctype html>
<html>
<head>
  <title>JavaScript Course</title>
</head>
<body>
  <div class="output"></div>
  <input type="text" placeholder="Enter Email">
  <button>Check</button>
  <script>

  </script>
</body>
</html>
```

Take the following steps:

1. Use the above template code to start creating your application. Within the JavaScript code, select the input, output, and button elements from the page as JavaScript objects.
2. Add an event listener to the button to run a block of code when clicked that will get the current value in the input field. Create a blank response value that will populate the output div element contents.
3. Add a test with the string value from the input field and the expression for email format. If the test result is *false*, update the response output to say *Invalid Email* and change the output color to red.
4. If the condition of the test returns *true*, add a response that confirms the email format is correct and change the text color of output to green.
5. Output the response value into the output element.

Functions and the arguments object

JavaScript deals with arguments in functions by adding them to a custom object called arguments. This object works a lot like an array, and we can use it instead of using the name of the parameter. Consider the following code:

```
function test(a, b, c) {
  console.log("first:", a, arguments[0]);
  console.log("second:", b, arguments[1]);
  console.log("third:", c, arguments[2]);
}

test("fun", "js", "secrets");
```

This outputs:

```
first: fun fun
second: js js
third: secrets secrets
```

When you update one of the parameters, the argument gets changed accordingly. The same goes for the other way around:

```
function test(a, b, c) {
  a = "nice";
  arguments[1] = "JavaScript";
  console.log("first:", a, arguments[0]);
  console.log("second:", b, arguments[1]);
  console.log("third:", c, arguments[2]);
}

test("fun", "js", "secrets");
```

This is going to change both arguments[0] and b, as they are related to a and arguments[1], respectively, as you can see in the output:

```
first: nice nice
second: JavaScript JavaScript
third: secrets secrets
```

If the function is called with more arguments than were declared in the function signature, this is the way to access them. However, the modern way is to use the rest parameter (*...param*) instead of the arguments object.



In case you've forgotten what the rest parameter is, you can revisit the rest parameter in *Chapter 6, Functions*.

Practice exercise 12.3

This exercise will demonstrate using the array-like `arguments` object and extracting values from it. Using the `arguments` length property, we will iterate through the items in the `arguments` and return the last item in the list:

1. Create a function without any parameters. Create a loop to iterate through the length of the `arguments` object. This will allow an iteration of each item of the `arguments` in the function.
2. Set up a variable called `lastOne` with a blank value.
3. As you loop through the `arguments`, set `lastOne` to the current value of the argument using the index of `i` to return the argument value. The argument will have an index value that can be used to reference the value as you iterate through the `arguments` object.
4. Return the value of `lastOne`, which should only return the last argument value as the response.
5. Output the response from the function, pass a number of arguments into the function, and console log the response result. You should see only the last item in the list. If you want to see each one, you can output them separately to the console as you look through the values, or construct an array that can then be returned, adding each one as you go through the arguments.

JavaScript hoisting

In *Chapter 6, Functions*, we discussed that we have three different variables, `const`, `let`, and `var`, and we highly recommended that you should use `let` instead of `var` because of their different scopes. JavaScript **hoisting** is why. Hoisting is the principle of moving declarations of variables to the top of the scope in which they are defined. This allows you to do things that you cannot do in many other languages, and for good reason by the way. This should look normal:

```
var x;
x = 5;
console.log(x);
```

It just logs 5. But thanks to hoisting, so does this:

```
x = 5;
console.log(x);
var x;
```

If you try to do this with `let`, you'll get a `ReferenceError`. This is why it is better to use `let`. Because clearly, this behavior is very hard to read, unpredictable, and you don't really need it.

The reason this happens is that the JavaScript interpreter moves all the `var` declarations to the top of the file before processing the file. Only the declarations, not the initializations. This is why you get a result of `undefined` if you use it before having initialized it. And this is why it should be initialized before it has been declared. It was designed this way for memory allocation, but the side effects are undesirable.

However, there is a way to turn this behavior off. Let's see how we can do so in the next section!

Using strict mode

We can change the understanding and forgiving behavior of JavaScript to some extent using strict mode. You can switch on strict mode with the following command in your code. This needs to be the first command of your code:

```
"use strict";
```

Here is something that works when we don't use strict mode:

```
function sayHi() {
  greeting = "Hello!";
  console.log(greeting);
}
```

```
sayHi();
```

We forgot to declare `greeting`, so JavaScript did it for us by adding a `greeting` variable to the top level and it will log `Hello!`. If we enable strict mode, however, this will give an error:

```
"use strict";

function sayHi() {
```

```
greeting = "Hello!";
console.log(greeting);
}

sayHi();
```

The error:

```
ReferenceError: greeting is not defined
```

You can also use strict mode only in a particular function: simply add it to the top of the function and it gets enabled for that function only. Strict mode alters a few other things too; for example, when using strict mode, there are fewer words that can be used as names for your variables and functions because they are likely to become reserved keywords in the future that JavaScript will need for its own language.

Using strict mode is a great way of getting used to using JavaScript in the setting of frameworks or even for writing TypeScript later. It is typically considered a good practice nowadays, so we would encourage you to use this in your own code when you have the chance. This is often not an (easy) option when working with existing older code though.

Now we have seen strict mode, it's time to dive into a whole different mode: debug mode! Debug mode is for when you are not busy writing or running your application, but are running it in a special way to spot the locations of any errors.

Debugging

Debugging is a delicate art. In the beginning, it usually is very hard to spot what's wrong with your code. If you are using JavaScript in the browser and it is not behaving as you would expect, step 1 is always to open the console in the browser. Often it will contain errors that can help you further.

If that doesn't solve it, you can log to the console in every step of your code, and also log the variables. This will give you some insight as to what is going on. It might just be that you are relying on a certain variable that happens to be undefined. Or perhaps you are expecting a certain value from a mathematical computation, but you've made an error and the result is something completely different from what you thought. Using `console.log()` during development to see what's happening is rather common.

Breakpoints

A more professional way to go about debugging is to use breakpoints. This can be done from most browsers and IDEs. You click on the line before your code (in the **Sources** panel in Chrome, but this may be different for different browsers), and a dot or arrow will appear. When your application is running, it will pause at this point to give you the opportunity to inspect the values of variables and walk through the code line by line from there.

This way, you will get a good clue of what is going on and how to fix it. Here is how to use breakpoints in Chrome, and most other browsers have something like this. In Chrome, go to the **Sources** tab of the **Inspect** panel. Select the file you want to set a breakpoint in. Then you just click on the line number and it sets the breakpoint.

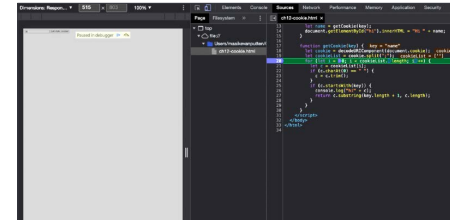


Figure 12.1: Breakpoints in the browser

Then try to trigger the line of code, and when it gets triggered, it pauses. On the very right of the screen I can inspect all the variables and values:

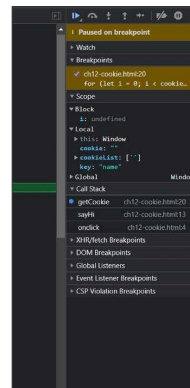


Figure 12.2: Inspecting breakpoint variables

You can now go through your code with a fine-toothed comb: with the play icon on top, you can resume script execution (until it hits the next breakpoint or runs in to the same breakpoint again). With the round arrow icon at the top, I can go to the next line and inspect the values on the next line again.



There are many options with breakpoints that we don't have space to cover here. For more detail on how you can debug your code with breakpoints, look in your chosen code editor's documentation or check the relevant Google Chrome documentation here: <https://developer.chrome.com/docs/devtools/javascript/breakpoints/>.

Practice exercise 12.4

Variable values can be tracked in the editor while debugging. The following exercise will demonstrate how to use the editor's breakpoints to check a value of a variable at a certain point in the running of the script. This is a simple example, but the same process can be used to find out information about larger scripts at specific points during execution, or establish where a problem may lie.



There are minor differences and nuances in the way breakpoints operate in different editors, so please refer to the documentation of your environment for more of a detailed walk-through—this is intended to give you an idea of what breakpoints offer when it comes to debugging.

You can use the following short script as an example:

```
let val = 5;
val += adder();
val += adder();
val += adder();
console.log(val);
function adder(){
  let counter = val;
  for(let i=0;i<val;i++){
    counter++;
  }
  return counter;
}
```



Remember to add `<script>` tags and open the script as an HTML document if you're testing this in your browser console.

This exercise has been tested in a desktop editor but it is equally relevant to browser consoles and other environments. Take the following steps:

1. Open your script in your chosen editor, or the **Sources** tab of your browser's **Inspect** panel. Click to the left of the line of code where you want to add a breakpoint. A dot or other indicator will appear to indicate the breakpoint is set:

```

1  let val = 5;
2  val += adder();
3  val += adder();
4  val += adder();
5  console.log(val);
6  function adder(){
7      let counter = val;
8      for(let i=0;i<val;i++){
9          counter++;
10         }
11     return counter ;
12 }

```

Figure 12.3: Setting breakpoints

2. Run the code with your new breakpoints: I have selected **Run | Start Debugging**, but this will vary depending on your editor. You can simply reload the web page if you're using the browser console to rerun the code with your new breakpoints accounted for.

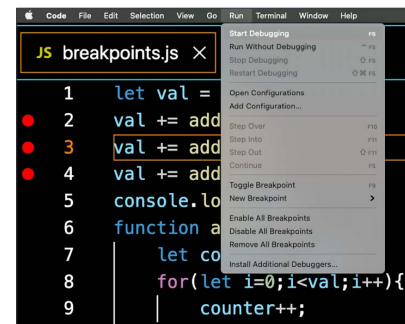


Figure 12.4: Running code with breakpoints added

3. You should now see the debugging console. There will be a tab that lists the variables in the code and the current values at the first breakpoint. It's called **VARIABLES** in my editor, but it's the **Scope** tab in the Chrome browser console.
4. You can use the menu options to move to the next breakpoint, stop debugging, or restart the breakpoint sequence. Press the play icon to move to the next breakpoint. It will update to have a value of 5, as specified by line 1, and pause at the first breakpoint. Note that the highlighted line has not been run yet:

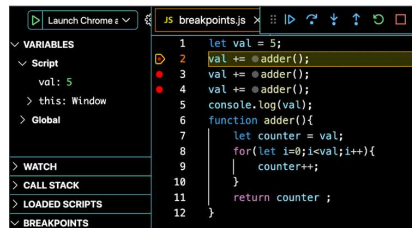


Figure 12.5: Viewing variables in the console

5. Press the play icon once more, and the script will run until it hits the next breakpoint, at which point the value of the variable will update as a result of the code on line 2:

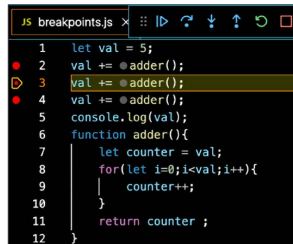


Figure 12.6: Progressing through breakpoints in a script

6. Press play again to move to the next breakpoint, which increases the value of val once more:

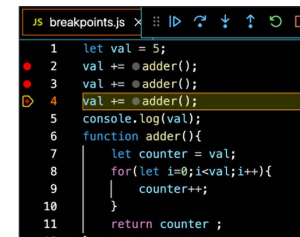


Figure 12.7: The final breakpoint

7. Once the last breakpoint is reached you will only see options to restart or stop the debugger. If you press stop, it will end the debugging process:

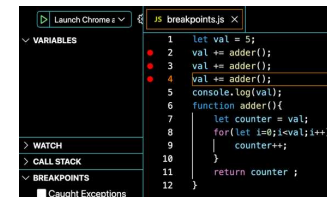


Figure 12.8: Breakpoints in the browser

The final value of val after the third breakpoint was revealed as 135. Write down the values of val after the first and second calls to the adder() function, which were revealed to you by using breakpoints.



This was a basic exercise, but we invite you to test out using breakpoints on some larger scripts and get more comfortable in your understanding of how your code works during runtime.

Error handling

We have seen a lot of errors appear already. Until now, we let the program crash when it encountered an error. There are other ways to deal with errors. When we are dealing with code that depends on some sort of outside input, such as an API, user input, or a file we will need to deal with the errors that this input can cause.

If we expect a certain piece of code to throw an error, we can surround this code with a catch block. The error it might throw will be caught in this block.



You have to be careful not to use this too much, and you usually don't want to do this when you can just write better code to avoid the error in the first place.

Here is an example of a piece of code that throws an error, and is surrounded with a try and catch block. Let's assume the `somethingVeryDangerous()` function might throw errors:

```
try {
  somethingVeryDangerous();
} catch (e) {
  if (e instanceof TypeError) {
    // deal with TypeError exceptions
  } else if (e instanceof RangeError) {
    // deal with RangeError exceptions
  } else if (e instanceof EvalError) {
    // deal with EvalError exceptions
  } else {
    // deal with all other exceptions
    throw e; //rethrow
  }
}
```

If it throws an error, it will end up in the catch block. Since `Error` could mean many different errors, we are going to check for the exact error we are dealing with and write a specific handling of this error. We check the exact error class with the `instanceof` operator. After the error handling, the rest of the code will continue to execute normally.

You can do one more thing with a try catch block, and that is add a `finally` block. This `finally` block gets executed irrespective of whether errors are thrown. This is great for cleanup purposes. Here is a simple example:

```
try {
  trySomething();
} catch (e) {
  console.log("Oh oh");
} finally {
  console.log("Error or no error, I will be logged!");
}
```

We don't know the output of this code, since `trySomething()` is not defined. If it were to throw an error, it would log `Oh oh` to the console and then `Error or no error, I will be logged!`. If `trySomething()` didn't throw an error, it would only log the last part.

Lastly, if, for whatever reason, you need to throw an error, you can do so with the `throw` keyword, like this:

```
function somethingVeryDangerous() {
  throw RangeError();
}
```

This can be of great use whenever you need to deal with things out of your control, such as an API response, user input, or input from reading a file. If unexpected things happen, sometimes you'll have to throw an error to deal with it appropriately.

Practice exercise 12.5

1. Using `throw`, `try`, and `catch`, check if the value is a number, and if it's not, then create a custom error.
2. Create a function with one argument called `val`.
3. Use `try`, and within it add a condition that checks whether `val` is a number using `isNaN`. If true, then throw an error that states that it is not a number. Otherwise, output `Got a number to the console`.

4. Use `catch` to catch any errors and output the error values to the console.
5. Add `finally` to run and output the value, and when the function has completed, also include the value of `val`.
6. Create one request to the function with a string argument and another with a number. See the results in the console.

Using cookies

Cookies are small data files that are stored on your own computer and used by websites. Cookies were invented to store things about the user of the website. Cookies are strings with a special pattern. They contain key-value pairs, and these key-value pairs are separated by semi-colons.

You can create a cookie and use it again later. Here is how you can create a cookie:

```
document.cookie = "name=Maaikje;favoriteColor=black";
```

This does not work in all browsers when you run it on the client side (such as in your `<script>` tag). In Chrome, for example, you cannot set the cookies from the client side. You have to run the code from a server. (I have used Safari instead to do this here, but there are no guarantees about future support.) An alternative is the web storage API.

It is also possible to start Chrome from the command line with certain settings enabled, or to enable the cookies in the settings under privacy preferences. Careful to turn this off afterward if you don't want it, though. This is how you can read from the cookie:

```
let cookie = decodeURIComponent(document.cookie);
let cookieList = cookie.split(";");
for (let i = 0; i < cookieList.length; i++) {
  let c = cookieList[i];
  if (c.charAt(0) == " ") {
    c = c.trim();
  }
  if (c.startsWith("name")) {
    alert(c.substring(5, c.length));
  }
}
```

This example gets all the cookies using `decodeURIComponent()`, and then splits them on the `;`. This leaves us with an array, `cookieList`, with key-value pairs as strings. Next, we loop over all the key-value pairs. Trim them (remove the whitespace in front and at the back), and see whether they start with `name`. This was the name of our cookie key.

If we want to get the value, we have to start reading after the key, so at least the length of the key, which is 4 (`name`) in this case. This brings us to index 3 already. We also want to skip the equal sign on index 4, so we start at index 5. In this case, we are adding an alert to the name. Here is an example of a simple website that uses a cookie to greet the user:

```
<!DOCTYPE html>
<html>
  <body>
    <input onchange="setCookie(this)" />
    <button onclick="sayHi('name')">Let's talk, cookie!</button>
    <p id="hi"></p>

    <script>
      function setCookie(e) {
        document.cookie = "name=" + e.value + ";";
      }

      function sayHi(key) {
        let name = getCookie(key);
        document.getElementById("hi").innerHTML = "Hi " + name;
      }

      function getCookie(key) {
        let cookie = decodeURIComponent(document.cookie);
        let cookieList = cookie.split(";");
        for (let i = 0; i < cookieList.length; i++) {
          let c = cookieList[i];
          if (c.charAt(0) == " ") {
            c = c.trim();
          }
          if (c.startsWith(key)) {
            console.log("hi" + c);
            return c.substring(key.length + 1, c.length);
          }
        }
      }
    </script>
  </body>
</html>
```

```

</script>
</body>
</html>

```

If you are writing a new website, you probably should not be using this. However, whenever you need to work with older code, chances are you'll come across this. And now you know what it means and how to adjust it. Good for you!

Practice exercise 12.6

Let's make a cookie builder. Create several functions that will allow you to interact with page cookies, including reading a cookie value by name, creating a new cookie using a name and setting it for a set number of days, and deleting a cookie. You can use the following HTML template to get you started:

```

<!doctype html>
<html>
<head>
  <title>Complete JavaScript Course</title>
</head>
<body>
  <script>

</script>
</body>
</html>

```

Take the following steps:

1. Set up your webpage, and in the JavaScript code, output the value of `document.cookie`. It should be blank.
2. Create a function that will take the parameters for `cookieName`, `cookieValue`, and the number of days you want to set the cookie.
3. Check if `days` is valid, and within the block of valid code, get the current date. Set a `setTime` value for the cookie to expire in milliseconds by multiplying the days into milliseconds.
4. Change the date object of milliseconds until the cookie expires to a UTC string value.
5. Set `document.cookie` to `cookieName = cookieValue`, plus add the expiry details and lastly specify `path=.`.

6. Create a function to create a test cookie with a value and expiry set after a number of days. Create a second cookie the same way, and when you refresh your page, you should see at least two cookies in the console.
7. Create a second function to read a cookie value, set the value as `false`, and then create an array of the cookies split by semi-colons.
8. Loop through all the cookies and split again where the equal signs are. This will give you the first item with index 0 as the name of the cookie. Add a condition to check if the name is equal to the name that was requested in the function parameters. If it matches, assign the value of the second item in the index, which will be the value of the cookie with the selected name. Return `cookieValue` in the function.
9. Add two console log messages using the function to read both cookies you have set earlier. Output the values of the cookies in the console.
10. To delete a cookie, you need to set a date prior to the current date. You can create a cookie with a -1 date and send the cookie with its selected name to be deleted by invoking the cookie creation function.
11. Try deleting a cookie by name.

Local storage

We have looked at cookies as a way to save user data, but there is actually a more modern way to do this: **local storage**. Local storage is an amazing fun topic that will add to your ability to make smart websites. With local storage, we can save key-value pairs in our web browser and use them again in a new session (when the browser is opened again later). The information is typically stored in a folder on the computer of the user, but this differs a bit by browser.

This allows the website to store some information and retrieve it later, even after refreshing the page or closing the browser. The advantage of local storage over cookies is that they don't need to be passed around with every HTTP request, which is the case with cookies. Local storage just lives there and waits to be accessed.

The `localStorage` object is a property of the `window` object that we have seen before. There are a few methods on the `localStorage` object that we need to know to use it effectively. First of all, we need to be able to get and set key-value pairs on local storage. We use `setItem()` whenever we want to save something and `getItem()` whenever we want to retrieve the value later. Here is how to do it:

```

<!DOCTYPE html>
<html>
<body>

```

```

<div id="stored"></div>
<script>
  let message = "Hello storage!";
  localStorage.setItem("example", message);

  if (localStorage.getItem("example")) {
    document.getElementById("stored").innerHTML =
      localStorage.getItem("example");
  }
</script>
</body>
</html>

```

This code snippet outputs `Hello storage!` on the page. You can add items to storage by specifying a key and a value with the `setItem` method. You can access `localStorage` directly or via the `window` object. Here we specify `example` as the key and `Hello storage!` as the value and save it to local storage. Then we check whether the `example` key is set in local storage and output the data by writing it to the `innerHTML` of the `div` with the ID stored.

If you go back to your code and turn off the `setItem()` line before loading the page a second time, it still will output that value, since the information was stored when running the script the first time and never got deleted. Local storage doesn't expire, though it can be manually deleted.

We can also retrieve a key using the index. This is useful whenever we need to loop through the key-value pairs and we don't know the names of the keys. This is how to retrieve a key by index:

```
window.localStorage.key(0);
```

In this case, the key is `name`. In order to get the associated value, we can do this:

```
window.localStorage.getItem(window.localStorage.key(0));
```

We can also remove key-value pairs like this:

```
window.localStorage.removeItem("name");
```

And we can remove all the key-value pairs from the local storage in one call:

```
window.localStorage.clear();
```

So, with local storage you can save values even after closing the browser. This allows for a lot of "smart" behavior, since your app is now able to remember things, such as what you've entered in a form, which settings you've toggled on a website, and what you've looked at previously.

Please don't see this as an alternative that you can use to bypass the problems with cookies and privacy. Local storage raises the exact same issues as cookies, it's just less known. You will still have to mention on your website that you are tracking users and storing information, just like you need to do for cookies.

Practice exercise 12.7

Let's create a local storage shopping list that will store values in the browser's local storage. This is an example of using JavaScript to convert from strings to useable JavaScript objects and back to strings that can be stored in local storage. You can use the following template:

```

<!doctype html>
<html>
<head>
  <title>JavaScript</title>
  <style>
    .ready {
      background-color: #ddd;
      color: red;
      text-decoration: line-through;
    }
  </style>
</head>
<body>
  <div class="main">
    <input placeholder="New Item" value="test item" maxlength="30">
    <buttonAdd</button>
  </div>
  <ul class="output">
  </ul>
</script>

</script>
</body>
</html>

```

Take the following steps:

1. In the JavaScript code, select all the page elements as JavaScript objects.
2. Create a `tasks` array with a value of the local `tasklist` storage if it exists, otherwise set the `tasks` array to an empty array. Using `JSON.parse`, you can convert the string value to a useable object in JavaScript.
3. Loop through all the items in the `tasklist` array; they will be stored as objects, with a name and a Boolean value for their checked status. Create a separate function to build the task item, adding it to the page from the list.
4. In the task generation function, create a new list item and a `textNode`. Append `textNode` to the list item. Append the list item to the page output area. If the task is marked complete with a Boolean value of `true`, then add the `style` class of `ready`.
5. Add an event listener to the list item that will toggle the `ready` class when clicked. Every time there is a change to any list item, you will also need to store that to the local storage. Create a task builder function that will store and ensure the visual list is the same as the local storage list. You will need to clear the current task list array and rebuild from the visual data, so create a function to handle the list building.
6. The task builder function will clear the current `tasks` array, and select all the 11 elements on the page. Loop through all the list items, getting the text value from the element, and checking if it contains the class of `ready`. If it contains the `ready` class, then mark the checked condition as `true`. Add the results to the `tasks` array, and this will rebuild the array to ensure it matches with what the user sees visually. Send to a save tasks function to save the `tasks` array in local storage, so if the page is refreshed, you will see the same list.
7. In the save tasks function, set the `localStorage` item to the `tasks` array. You will need to stringify the object so that it can go into the string parameter of local storage.
8. Now, when you refresh the page, you will see the list of tasks. They can be crossed out by clicking them and new items can be added in the input field by pressing the button to submit new items.

JSON

JSON stands for **JavaScript Object Notation**, which is nothing more than a data format. We saw this notation when we were creating our objects in JavaScript; however, JSON doesn't mean JavaScript objects, it's just a way of representing data using a similar format as JavaScript objects. It can also be easily converted to a JavaScript object.

JSON is a standard used to communicate with APIs, including APIs that aren't written in JavaScript! APIs can accept data, for example, the data from a form on a website, in JSON format. And nowadays, APIs almost always send data back in JSON. Sending data from an API happens, for example, when you enter a web shop—the products typically come from a call to an API that is connected to a database. This data gets converted to JSON and is sent back to the website. Here is an example of JSON:

```
{
  "name" : "Malika",
  "age" : 50,
  "profession" : "programmer",
  "languages" : ["JavaScript", "C#", "Python"],
  "address" : {
    "street" : "Some street",
    "number" : 123,
    "zipcode" : "3850AA",
    "city" : "Utrecht",
    "country" : "The Netherlands"
  }
}
```

This is an object that seems to describe a person. It has key-value pairs. The keys always have to be between quotes, but the values only have to be between quotes when they are strings. So, the first key is `name` and the first value is `Malika`.

Lists of values (or JavaScript arrays) are indicated with `[]`. The JSON object contains a list of `languages`, which has the square brackets, and another object, `address`. You can tell this by the curly brackets.

There are actually only a few flavours in JSON:

- Key-value pairs with values of the following types: string, number, Boolean, and null
- Key-value pairs with lists, which have `[` and `]` that contain the items in the list
- Key-value pairs with other objects, which have `{` and `}` that contain other JSON elements

These three options can be combined, so an object can contain other objects and a list can contain other lists. We saw this already in the above example. Our object contained a nested address object.

But this can be nested even further. A list can also contain objects, which can contain lists with objects, with lists, and so on. This might sound a bit complicated and that's exactly the point. Even though it is very simple, nesting all these options can still complicate JSON a bit. There is a reason we've placed it in our advanced topic chapter.

Let's now have a look at a slightly more complex example:

```
{
  "companies": [
    {
      "name": "JavaScript Code Dojo",
      "addresses": [
        {
          "street": "123 Main street",
          "zipcode": 12345,
          "city": "Scott"
        },
        {
          "street": "123 Side street",
          "zipcode": 35401,
          "city": "Tuscaloosa"
        }
      ]
    },
    {
      "name": "Python Code Dojo",
      "addresses": [
        {
          "street": "123 Party street",
          "zipcode": 68863,
          "city": "Nebraska"
        },
        {
          "street": "123 Monty street",
          "zipcode": 33306,
          "city": "Florida"
        }
      ]
    }
  ]
}
```

This is a list of companies, with two company objects on it. The companies have two key-value pairs: a name and an address list. Each of the address lists contains two addresses, and each address consists of three key-value pairs: `street`, `zipcode` and `city`.

Practice exercise 12.8

This exercise will demonstrate how you can create a valid JSON object that can be used as a JavaScript object. You will create a simple list of names and statuses that can be looped through and output the results to the console. You will load JSON data to JavaScript and output the results of the object's contents:

1. Create a JavaScript object that contains JSON formatted data. The object should contain at least two items and each item should be an object with at least two paired values.
2. Create a function that can be invoked that will loop through each item in the JavaScript JSON object and output the result to the console. Output each item of data to the console using `console.log`.
3. Invoke the function and launch the JavaScript code.

Parsing JSON

There are many libraries and tools available for parsing a JSON string into an object. A JavaScript string can be converted to a JSON object using the `JSON.parse()` function. Data that is received from another place is always of value `string`, so in order to treat it as an object, it needs to be converted. This is how to do it:

```
let str = '{"name": "Maaike", "age": 30}';
let obj = JSON.parse(str);
console.log(obj.name, "is", obj.age);
```

After parsing, it can be treated as an object. Therefore it will log `Maaike is 30` to the console.

The other way around is also necessary sometimes. Objects can be converted to a JSON string using the `JSON.stringify()` method. It converts the object or value from JavaScript to a JSON string. You can see it in action here:

```
let dog = {
  "name": "wiesje",
  "breed": "dachshund"
};
```

```
let strdog = JSON.stringify(dog);
console.log(typeof strdog);
console.log(strdog);
```

The type of `strdog` becomes a string because it is being stringified. And it no longer has the properties `name` and `breed`. These will be undefined. This code snippet will log the following to the console:

```
string
{"name":"wiesje","breed":"dachshund"}
```

This can be useful for storing JSON data directly in a database, for example.

Practice exercise 12.9

This exercise will demonstrate the use of JSON methods to parse JSON and convert string values to JSON. Using JSON methods with JavaScript, convert a JSON formatted string value to a JavaScript object and convert a JavaScript object into a string representation of the JSON object:

1. Create a JSON object with several items and objects. You can use the JSON object from the previous lesson.
2. Using the JSON `stringify()` method, convert the JSON JavaScript object into a string version and assign it to a variable named `newStr` `[{"name": "Learn JavaScript", "status": true}, {"name": "Try JSON", "status": false}]`.
3. Using `JSON.parse()`, convert the `newStr` value back into an object and assign it to a variable named `newObj`.
4. Iterate through the items in the `newObj` and output the results to the console.

Practice exercise 12.9 answers

```
let myList = [{
  "name": "Learn JavaScript",
  "status": true
},
{
  "name": "Try JSON",
  "status": false
}
];
```

```
const newStr = JSON.stringify(myList);
const newObj = JSON.parse(newStr);
newObj.forEach((el) => {
  console.log(el);
});
```

Chapter projects

Email extractor

Use the following HTML as a starter template and add the JavaScript code to make an email extractor function:

```
<!doctype html>
<html>
<head>
  <title>Complete JavaScript Course</title>
</head>
<body>
  <textarea name="txtarea" rows=2 cols=50></textarea> <button>Get
Emails</button>
  <textarea name="txtarea2" rows=2 cols=50></textarea>
<script>
</script>
</body>
</html>
```

Take the following steps:

1. In JavaScript, select both text areas and the button and set them as JavaScript objects.
2. Add an event listener to the button that will invoke a function that gets the content of the first textarea and filters it to only accept email addresses.
3. Within the extracting function, get the content of the first input field. Using `match()`, return an array of the email addresses that were matched from within the content from the first textarea.
4. To remove any duplicates, create a separate array that will hold only unique values.

5. Loop through all the email addresses found and check whether each one is already in the `holder` array, and if not, add it.
6. Using the `join()` array method, you can now join together the results of the email addresses found within the content and output it into the second `textarea`.

Form validator

This project is an example of a typical form structure where you check the values inputted into the form and validate them before the content gets submitted. A response is returned to the user if the values do not meet the validation criteria in the code. Use the following HTML and CSS as a starting template:

```
<!doctype html>
<html>
<head>
  <title>JavaScript Course</title>
  <style>
    .hide {
      display: none;
    }
    .error {
      color: red;
      font-size: 0.8em;
      font-family: sans-serif;
      font-style: italic;
    }
    input {
      border-color: #ddd;
      width: 400px;
      display: block;
      font-size: 1.5em;
    }
  </style>
</head>
<body>
  <form name="myform"> Email :
    <input type="text" name="email"> <span class="error hide"></
span>
```

```
<br> Password :
<input type="password" name="password"> <span class="error
hide"></span>
<br> User Name :
<input type="text" name="userName"> <span class="error hide"></
span>
<br>
<input type="submit" value="Sign Up"> </form>
<script>

</script>
</body>
</html>
```

Take the following steps:

1. Using JavaScript, select all the page elements and set them as JavaScript objects so they are easier to select within the code. Also select all the page elements that have the `error` class as an object.
2. Add an event listener to submit and capture the click, preventing the default form action.
3. Loop through all the page elements that have a class `error` and add the `hide` class, which will remove them from view since this is a new submission.
4. Using the regular expression for valid emails, test the results against the input value of the email field.
5. Create a function to respond to errors, which removes the `hide` class from the element next to the element that triggered the event. Apply focus to that element within the function.
6. If there is an error that an input does not match the desired regex, pass the parameters to the error handling function you just created.
7. Check the password field input value to ensure only letters and numbers are used. Also check the length to ensure that it is 3-8 characters. If either are false, then add the error with the error function and create a message for the user. Set the error Boolean to true.
8. Add in an object to track the form data creation and add values to the object by looping through all the inputs, setting the property name to be the same as the input name, and the value the same as the input value.
9. Before the end of the validation function, check if an error is still present, and if it is not, submit the form object.

Simple math quiz

In this project, we will create a math quiz that will allow the user to respond to math questions. The application will check the responses and score the accuracy of the user's answers to the questions. You can use the following HTML template:

```
<!doctype html>
<html>
<head>
<title>Complete JavaScript Course</title>
</head>
<body>
<span class="val1"></span> <span>+</span> <span class="val2"></span>
</span> = <span>
  <input type="text" name="answer"></span><button>Check</button>
  <div class="output"></div>
</body>
</html>
```

Take the following steps:

1. In JavaScript, wrap the code within a function, `app`. Within the `app` function, create variable objects to contain all the page elements so they can be used in the script, and create a blank object called `game`.
2. Add a `DOMContentLoaded` event listener that invokes the `app` initialization once the page loads.
3. Within an `init()` function, add an event listener to the button, listen for a click, and track the event into a function called `checker`. Also within the `init` function, load another function called `loadQuestion()`.
4. Create a function to load the questions, and another function that can generate a random number from min and max values in the arguments.
5. In the `loadQuestion()` function, generate two random values and add them to the `game` object. Calculate the result of both values added together and assign that value within the `game` object as well.
6. Assign and update the `textContent` of the page elements that require the dynamic number values for the calculation question.
7. When the button is clicked, use a ternary operator to determine whether the answer to the question was correct or incorrect. Set the color to green for correct, and to red for incorrect.

8. Create a page element to output all the questions and keep track of the results. Within the `checker()` function, append a new element to the HTML with a style color to indicate a correct or incorrect response. Display the first and second values as well as the answer, and show the user's response within brackets.
9. Clear the input field and load the next question.

Self-check quiz

1. What will the following regex expression return from the following words?

```
Expression / ([a-e])\w+/g
"Hope you enjoy JavaScript"
```

2. Are cookies part of the document object?

3. What will the following code do to a JavaScript cookie?

```
const mydate = new Date();
mydate.setTime(mydate.getTime() - 1);
document.cookie = "username=; expires=" + mydate.toGMTString();
```

4. What is the output in the console from the following code?

```
const a = "hello world";
(function () {
  const a = "JavaScript";
})();
console.log(a);
```

5. What is the output in the console from the following code?

```
<script>
"use strict";
myFun();
console.log(a);
function myFun() {
  a = "Hello World";
}
</script>
```


6. What is the output of the following code?

```
console.log("a");
setTimeout(() => {
  console.log("b");
}, 0);
console.log("c");
```

Summary

In this chapter, we had some important, more advanced topics that we still had to cover, but that you were probably not ready for earlier in the book. After this chapter, you should have deepened your understanding of JavaScript in several areas, first and foremost, regular expressions. With regex, we can specify patterns of strings and we can use these to search other strings for matches to our patterns.

We also considered functions and the arguments object, with which we can access arguments by their index. We continued with a look at JavaScript hoisting and strict mode, which enables us to use JavaScript with a few more rules. Getting used to JavaScript in strict mode is generally a good practice and is great preparation for working with JavaScript frameworks.

Debugging and tweaking were also discussed: we can use breakpoints or log our output to the console to get an idea of what is going on. Handling errors well can prevent unnecessary crashes of our program. Finally, we looked at JavaScript cookie creation and the use of local storage, along with the use of JSON, a syntax for sending data around. We saw the different types of key-value pairs and how to parse JSON. We also saw how to store key-value pairs in the `localStorage` object of `window`.

This chapter has deepened our understanding of JavaScript, and we learned some new things we need to know for modern JavaScript, but also a lot for when dealing with old (legacy) code. In the next chapter, we'll dive into an even more advanced topic: concurrency. This topic is about multitasking with your JavaScript code.

13

Concurrency

It's time for a more advanced topic. You're ready! We are going to deal with the topic of asynchronous code and some options for multitasking with code. This concept is called concurrency. Don't worry if you find this chapter a bit of a struggle; this is programming in JavaScript at a high level. These are the topics we'll be dealing with:

- Concurrency
- Callbacks
- Promises
- `async/await`
- Event loop

Yes, this is tough, but understanding how to leverage concurrency can really enhance the performance of your program by speeding up the process, which is more than enough reason to dive into this advanced topic!



Note: exercise, project, and self-check quiz answers can be found in the *Appendix*.

Introducing concurrency

Concurrency is whenever things are happening "at the same time" or *in parallel*. To give a non-code example, let's talk about managing my household. When I come home on a Friday night, I have a series of tasks: the kids need to eat, to shower, and to be brought to bed, the laundry needs to be folded and laundry needs to be put in the machine, and to be fair, a lot more, but this is enough to illustrate the example.

If I were to do this without being able to do multiple things at once, it would be a very tough night and get very late. I would first make dinner—put a pizza in the oven and wait next to it—feed the kids, shower them afterward, then bring them to bed, and then fold the laundry afterward, turn the machine on again, and wait until it's done. Luckily, I can multitask, so it looks more like this: I put the pizza in the oven, in the meantime, I turn the washing machine on and maybe fold a few pieces of laundry, then I feed the kids, do the rest of the laundry while they shower, and I'm done a lot quicker.

This is the same for your computer and the applications that you use. If it weren't able to do multiple things at the same time, you would probably be very annoyed. You wouldn't be able to check your mail while you are writing code, you wouldn't be able to listen to music while writing code, and a lot more. This is your computer switching between different tasks. The same thing can happen at the application level. For example, we can do a call to some API and not wait for the reply but do something useful in the meantime instead. We can do so using the concept of **concurrency**.

There are three strategies in JavaScript that you'll need to know of when working with concurrency: **callbacks**, **Promises**, and the **async** and **await** keywords.

Callbacks

Callbacks are the first thing we should understand when we are talking about concurrency. The good news is that the callback principle is not too hard to understand. It is just a function that takes another function as an argument, which is then called when the rest of the initial function has finished. In other words, it's just a function calling a function, like this:

```
function doSomething(callback) {  
  callback();  
}  
  
function sayHi() {
```

```

    console.log("Hi!");
  }

  doSomething(sayHi);

```

The `doSomething()` function, which is created with the parameter callback, is just calling whatever function is being passed in as an argument. We call it using the `sayHi()` function as an argument, so this code snippet is just a very complicated way to get `Hi!` printed to the console.

Here is an example of the callback principle actually doing something:

```

function judge(grade) {
  switch (true) {
    case grade == "A":
      console.log("You got an", grade, ": amazing!");
      break;
    case grade == "B":
      console.log("You got a", grade, ": well done!");
      break;
    case grade == "C":
      console.log("You got a", grade, ": alright.");
      break;
    case grade == "D":
      console.log("You got a", grade, ": hmm...");
      break;
    default:
      console.log("An", grade, "! What?!");
  }
}

function getGrade(score, callback) {
  let grade;
  switch (true) {
    case score >= 90:
      grade = "A";
      break;
    case score >= 80:
      console.log(score);
      grade = "B";
      break;
    case score >= 70:

```

```

    grade = "C";
    break;
  case score >= 60:
    grade = "D";
    break;
  default:
    grade = "F";
  }
  callback(grade);
}

getGrade(85, judge);

```

There are two functions here: `judge()` and `getGrade()`. We call the function `getGrade()` with two arguments: 85 and the function `judge()`. Note that when calling the function as an argument, we do not include the parentheses. The `judge()` function gets stored in a callback. After determining the grade, the function that is stored in a callback (`judge()` in this case) gets called with the grade.

This could also be another function that does something more useful than judging, for example, sending a certain email based on test results. If we wanted that, we actually wouldn't need to change the `getGrade()` function; we just need to write a new function to do this and call `getGrade()` with the new function as a second argument.

You might be very disappointed right now, because this is not too exciting. Callbacks become really valuable in an asynchronous context, for example, when one function is still waiting for the results of a call to the database before calling the callback function that is going to process the data.

Some JavaScript built-in functions work with this callback principle, for example, the `setTimeout()` and `setInterval()` functions. They will take a function that is executed after a certain time in the case of a timeout and every certain amount of time for the specified interval. We have seen these already, but just as a reminder:

```

setInterval(encourage, 500);

function encourage() {
  console.log("You're doing great, keep going!");
}

```

The functions that are inserted as arguments are called callbacks here. Understanding concurrency really starts with callbacks, but multiple nested callbacks make code difficult to read.

When this is all written as one function with anonymous functions inside, this gets very indented as well. We call this **callback hell** or the **Christmas tree problem** (because the code gets nested so much, it looks like a Christmas tree on its side).

Callbacks are a great concept, but they can create ugly code very fast. There is often a better solution, we promise.

Practice exercise 13.1

This exercise will demonstrate how to use a callback function, creating a way to pass a value from one function to another by invoking a callback function. We will create a greeting callback using a full name in a string.

1. Create a function named `greet()` that takes one argument, `fullName`. That argument should be an array. Output the items of the array into the console interpolated into a greeting message string.
2. Create a second function that has two arguments: the first one is a string for the user's full name, and the second is the `callback` function.
3. Split the string into an array using the `split()` method.
4. Send the full-name array to the `greet()` function created in the first step.
5. Invoke the process of the `callback` function.

Promises

With **Promises**, we can organize the sequence of our code in a slightly easier-to-maintain way. A Promise is a special object that connects code that needs to produce a result and the code that needs to use this result in the next step.

When we create a Promise, we give it a function. In the following example, we use a convention that we have seen a lot: we are creating a function on the spot. So, inside the argument list we are defining the function, often done using arrow functions as well. This function needs two parameters, and these parameters are callbacks. We have called them `resolve` and `reject` here.



You can call these parameters anything you want, but `resolve` or `res` and `reject` or `rej` are most common.

When `resolve()` is called, the Promise is presumed to be successful and whatever is between the arrows is returned and used as input for the `then` method on the Promise object. If `reject()` is called, the Promise failed and the `catch()` method on the Promise object (if present) is executed with the argument of the `reject()` function.

This is a lot of information that can be hard to understand at first, so here is an example of a Promise to help you:

```
let promise = new Promise(function (resolve, reject) {
  // do something that might take a while
  // Let's just set x instead for this example
  let x = 20;
  if (x > 10) {
    resolve(x); // on success
  } else {
    reject("Too low"); // on error
  }
});

promise.then(
  function (value) {
    console.log("Success:", value);
  },
  function (error) {
    console.log("Error:", error);
  }
);
```

We first create a Promise. When creating a Promise, we don't know what the value of the Promise is going to be. This value is whatever is sent as an argument to the `resolve` function. It is a sort of placeholder.

So when we call `then` on the Promise, we basically say: figure out what the value of the Promise is, and when you know, execute one function if the Promise was resolved or a different function if it was rejected. When a Promise is neither resolved nor rejected, we say that the Promise is pending.

`then()` is a Promise itself, so when it returns we can use the result for the next `then()` instance. This means we can chain the `then()` instances, which can look like this:

```
const promise = new Promise((resolve, reject) => {
  resolve("success!");
});
```

```

.then(value => {
  console.log(value);
  return "we";
})
.then(value => {
  console.log(value);
  return "can";
})
.then(value => {
  console.log(value);
  return "chain";
})
.then(value => {
  console.log(value);
  return "promises";
})
.then(value => {
  console.log(value);
})
.catch(value => {
  console.log(value);
})

```

This will log:

```

success!
we
can
chain
promises

```

The resolve functions are implemented with an arrow function. The return statement is the value input for the next function. You can see that the last block is a `catch()` function. If any of the functions were to result in a rejection and the Promise were therefore rejected, this `catch()` block would be executed and print whatever the `reject()` function sent to the `catch()` method. For example:

```

const promise = new Promise((resolve, reject) => {
  reject("oops... ");
})
.then(value => {
  console.log(value);
  return "we";
})

```

```

.then(value => {
  console.log(value);
  return "can";
})
.then(value => {
  console.log(value);
  return "chain";
})
.then(value => {
  console.log(value);
  return "promises";
})
.then(value => {
  console.log(value);
})
.catch(value => {
  console.log(value);
})

```

This will just log oops... because the first Promise was rejected instead of resolved. This is great for creating asynchronous processes that need to wait till another process is complete. We can try to do a certain set of actions and when something goes wrong, use a `catch()` method to deal with it.

Practice exercise 13.2

In this exercise, you will create a counter that will output the values in sequence using Promises.

1. Set up a Promise that resolves with a value of `Start Counting`.
2. Create a function named `counter()` that has one argument that gets the value and outputs it into the console.
3. Set up the next function in the Promise with four `then()` instances, which should output a value into the counter function, and return a value which will provide input for the subsequent `then()` instance. The returned values should be one, then two, then three. The screen output in the console should be the following:

```

Start Counting
One
Two
Three

```

async and await

We have just seen the `Promise` syntax. With the `async` keyword, we can make a function return a `Promise`. This makes the Promises nicer to read and look a lot like synchronous (non-concurrent) code. We can use this `Promise` just like we learned in the previous section, or we can use the more powerful `await` keyword to wait until the `Promise` is done. `await` only works in an asynchronous function.

In an asynchronous context, we can await other Promises as well, as can be seen in this example:

```
function saySomething(x) {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve("something" + x);
    }, 2000);
  });
}

async function talk(x) {
  const words = await saySomething(x);
  console.log(words);
}

talk(2);
talk(4);
talk(8);
```

Can you figure out what this code does? We call the asynchronous function `talk()` three times in a row with no break. Each of these function calls is awaiting the `saySomething()` function. The `saySomething()` function contains a new `Promise` that is being resolved with a `setTimeout()` function that waits two seconds before resolving with the value of `something` plus `x`. So after two seconds, the three functions are done at the same time (or so it seems to the human eye).

If the `talk()` function were not asynchronous, it would throw a `SyntaxError` because of the `await` keyword. `await` is only valid in asynchronous functions, so `talk()` must be asynchronous. Without the `async` and the `await` in this example, it would store the result of the function `saySomething()`, a pending `Promise`, in words and log that once for every function call:

```
Promise { <pending> }
Promise { <pending> }
Promise { <pending> }
```

We have now seen the basic building blocks of concurrency. This should prepare you for working with concurrency in real life. Concurrency really is an advanced topic; debugging it is troublesome, but it is really worth while in terms of performance when applied at the right moment.

Practice exercise 13.3

This exercise will demonstrate how to use `await` to wait on a `Promise` inside an `async` function. Using `await` and `async`, create a counter with `timeout()`, and increment a global counter value.

1. Create a global value for a counter.
2. Create a function that takes one argument. Return the result of a new `Promise`, setting a `setTimeout()` function that will contain the resolve instance.
3. Increment the counter within `setTimeout()`, adding one every second. Resolve the `Promise` with the value of the counter and the value of the variable that was in the function argument.
4. Create an asynchronous function that outputs into the console the value of the global counter and the value of the argument of the function.
5. Create a variable to capture the returned resolve value from the `await` function. Output that result into the console.
6. Create a loop to iterate 10 times, incrementing the value and invoking the `async` function, passing the value of the increment variable as the parameter in the function.

The results should look like the following:

```
ready 1 counter:0
ready 2 counter:0
ready 3 counter:0
x value 1 counter:1
x value 2 counter:2
x value 3 counter:3
```

Event loop

We would like to end this chapter by explaining how JavaScript handles asynchrony and concurrency under the hood. JavaScript is a single-threaded language. A thread in this context means a *path of execution*. If there is only a single path, this means that tasks will have to wait for one another and only one thing can happen at a time.

This single executor is the **event loop**. It's a process that executes the actual work. You may wonder about this, because you've just learned about concurrency and doing things asynchronously and at the same time. Well, even though JavaScript is single-threaded, it doesn't mean that it cannot outsource some tasks and wait for them to come back. This is exactly how JavaScript manages to do things in a multithreaded manner.

Call stack and callback queue

JavaScript works with a **call stack**, and all the actions that it has to execute are queued up here. The event loop is a process that is constantly monitoring this call stack, and whenever there are tasks to do, the event loop does them one by one. The tasks on top get executed first.

Here's a tiny script:

```
console.log("Hi there");
add(4,5);

function add(x, y) {
  return x + y;
}
```

Here's a visualization of the call stack and event loop for this script.

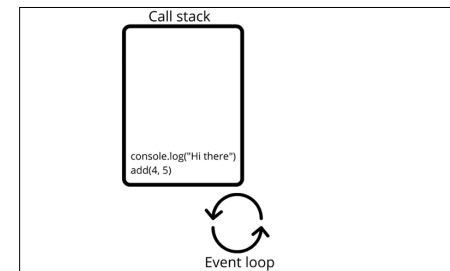


Figure 13.1: Visualization of the event loop and the call stack

No multithreading is going on here. But it is here:

```
console.log("Hi there");
setTimeout(() => console.log("Sorry I'm late"), 1000);
console.log(add(4, 5));

function add(x, y) {
  return x + y;
}
```

The `setTimeout()` task gets outsourced to the browser's web API (more on APIs in *Chapter 15, Next Steps*). When it's done, this appears in a special place: the **callback queue**. When the call stack is empty (and only then!), the event loop will check the callback queue for work to do. If there are any callbacks waiting, they'll be executed, one by one. After every action, the event loop will check the call stack for work first.

Here's a visualization of the situation with the outsourcing of `setTimeout()`:

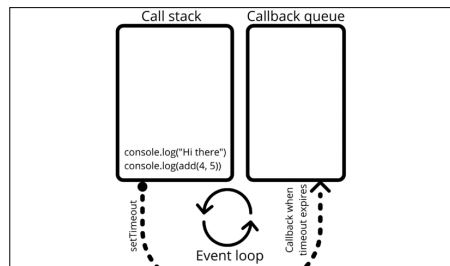


Figure 13.2: Visualization of the `setTimeout` being outsourced

When `setTimeout()` expires, the event loop will have done whatever was on the **call stack** already, and will check the callback queue and execute any tasks on there:

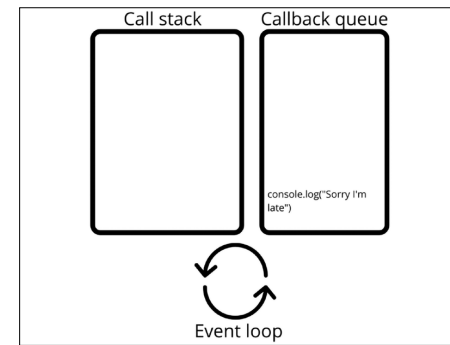


Figure 13.3: Visualization of the task on the callback queue

And this is what it will output:

```
Hi there
9
Sorry I'm late
```

Let's see if you read the above text well. What do you think will happen when we set the timer to 0, like here?

```
console.log("Hi there");
setTimeout(() => console.log("Sorry I'm late"), 0);
console.log(add(4,5));
```



```
function add(x, y) {
  return x + y;
}
```

This will output the exact same thing. `setTimeout()` will also be outsourced when the timer is at 0. The callback is placed in the callback queue right away, but the event loop won't even check the callback queue until the callstack is empty. So it will still print Sorry I'm Late after 9, even though the timer is at 0.

Chapter project

Password checker

Using an array of allowed passwords, this exercise will create an application to check if one of these password string values exists in an array that lists all the accepted passwords. Set a Promise to check if the password is valid, and upon the result either resolve with the status of true or reject with the status of false. Return the check results.

1. Create an array of allowed passwords.
2. Create a login function that will check if the argument is a value that is included in the passwords array. You can use `indexOf()` or the `includes()` method to check the array for a value and return a Boolean value of the result.



The `includes()` method is an array method that can check whether a certain value is included among the items in the array. It will return a Boolean value depending on the result.

3. Add a function that returns a Promise. Using `resolve` and `reject`, return a JavaScript object with the Boolean of `true` or `false` to indicate the password validity status.
4. Create a function that checks the password, sending it to the login function, and using `then()` and `catch()`, outputs the result of either the rejected password or the resolved password.
5. Send several passwords, some within the array, others not, to the checker function.

Self-check quiz

1. Fix the error in the following code to use the callback function:

```
function addOne(val){
  return val + 1;
}
function total(a, b, callback){
  const sum = a + b;
  return callback(sum);
}
console.log(total(4, 5, addOne()));
```

2. Write down the result of the following code:

```
function checker(val) {
  return new Promise((resolve, reject) => {
    if (val > 5) {
      resolve("Ready");
    } else {
      reject(new Error("Oh no"));
    }
  });
}
checker(5)
  .then((data) => {console.log(data); })
  .catch((err) => {console.error(err); });
```

3. What line(s) of code need to be added to the preceding function so that there is always a result after the function runs that ensures the word `done` is output into the console?
4. Update the below code to make the function return a Promise:

```
function myFun() {
  return "Hello";
}
myFun().then(
  function(val) { console.log(val); },
  function(err) { console.log(err); }
);
```

Summary

In this chapter, we've discussed concurrency. Concurrency enables our code to do multiple things at the same time and we can determine the order of things using callbacks, Promises, and the `async` and `await` keywords. Implementing these in your applications and pages will improve the user experience a lot! Users are quite demanding nowadays; if a website isn't loading fast enough, they bounce (go back to, for example, Google). Concurrency helps to deliver results faster.

The next two chapters are about using JavaScript for modern web development and will be dealing with HTML5 and JavaScript and modern JavaScript frameworks that are real game-changers.

14

HTML5, Canvas, and JavaScript

HTML5 was released in 2012 and became standardized in 2014, which caused browsers to support all sorts of new features. The introduction of HTML5 impacted the realm of possibilities that are available through JavaScript. The options for graphics, videos, interaction with graphics, and a lot more with JavaScript have increased tremendously since the introduction of HTML5, and have been so revolutionary that in fact, they led to the end of support of Flash by web browsers.

HTML5 allows web page(s) to be better structured by adding new elements, such as `<header>`. And also the DOM has improved quite a bit, which has led to increased performance. There are quite a few other additions and you'll see some of them in the chapter. Another fun (and useful) addition worth mentioning here is the `<canvas>` element, which we'll cover in this chapter as well.

JavaScript gives us a lot of amazing features already, but together with HTML5 there is a lot more possible when it comes to creating dynamic interactive web apps. This combination enables us to level up our content presentation game. We can work with files in the browser now, as well as drawing on the HTML5 canvas and adding images and text to it.

In this chapter, we will be examining some of the amazing things HTML5 has brought us. The topics don't all relate to one another directly, but they have in common that they were made possible by the powerful team of HTML5 and JavaScript, and of course, that they are all fun and useful. They will allow you to create an even more dynamic and interactive experience for the users of your app.

These are the topics that will be covered in this chapter:

- Introducing HTML5 with JavaScript
- Local file reader
- GeoLocation
- HTML5 canvas
- Dynamic canvas
- Drawing on the canvas with the mouse
- Saving dynamic images
- Media on the page
- Digital accessibility



Note: exercise, project and self-check quiz answers can be found in the *Appendix*.

Introducing HTML5 with JavaScript

HTML5 is formally a version of HTML. It is a huge step up compared to its predecessor and enables us to make full applications in the web browser that are even accessible offline. When you read HTML5 in a job description, it often means more than just HTML. Usually, the combination of HTML5 with JavaScript, CSS, JSON, and others is included here as well.

Since HTML5 the structure of our page has improved. We have new elements such as `<header>`, `<nav>`, and `<article>`. And we can play videos with the `<video>` element, which means we no longer need Flash since HTML5. And as we already mentioned we can work with the `<canvas>` element to create visuals on the page or to represent visuals such as animation, graphs, and others. Some things that had to be done with JavaScript in the past can now be done solely with HTML, such as adding video and audio to a webpage.

Changes to the DOM also improved the loading time of web page(s). We are going to dive into some of the HTML5-specific features in this chapter. Let's start with accessing files from the browser.

Local file reader

Since HTML5 we can finally interact with local files using the JavaScript that runs in our browser, which is really an amazing feature. Using this feature, we can upload files from our device to our web app and read from them in our app. This means that we can attach files to forms for example, which is great in many cases whenever we need to upload some sort of file for whatever purpose, for example, adding a résumé to your online job application.

Let's first make sure that the browser you are using supports this. We can run a simple script to check whether it does:

```
<!DOCTYPE html>
<html>
<body>
  <div id="message"></div>
  <script>
    let message = document.getElementById("message");
    if (window.FileReader) {
      message.innerText = "Good to go!";
    } else {
      message.innerText = "No FileReader :( ";
    }
  </script>
</body>
</html>
```

If you open this file in your browser it should say **Good to go!** when your browser supports file reading. Try updating your browser or using another one if it says **No FileReader** :(Browsers that will work are, for example, Chrome and Firefox.

Uploading files

Uploading files is actually easier than you might think. We indicate we want to upload a file by adding an input of type file. Here is a basic script that does just that:

```
<!DOCTYPE html>
<!DOCTYPE html>
<html>
<body>
  <input type="file" onchange="uploadFile(this.files)" />
  <div id="message"></div>
```

```
<script>
let message = document.getElementById("message");

function uploadFile(files) {
  console.log(files[0]);
  message.innerText = files[0].name;
}
</script>
</body>
</html>
```

It gives a blank HTML page with a **Choose file** button and the **No file chosen** comment behind it. Clicking on the button pops up the filesystem and you can select a file. After selecting the file, the JavaScript gets triggered. And as you can see, we are sending in the property files that are active in our body. This is a list of files. Therefore, we are grabbing the 0th index, the first element in the list. Files are represented as objects.

The file object gets logged to the console here, which enables you to see all the properties and associated values. Some of the important properties are the *name*, *size*, *type*, and *lastModified*, but there are many more.

We are putting the name of our file in the *innerText* of our div message. So, on the screen, you will see the name of the file appear in the div. We can do something similar for multiple files. Here is how to upload multiple files at the same time:

```
<html>
<body>
  <input type="file" multiple onchange="uploadFile(this.files)" />
  <div id="message"></div>
  <script>
    let message = document.getElementById("message");

    function uploadFile(files) {
      for (let i = 0; i < files.length; i++) {
        message.innerHTML += files[i].name + "<br>";
      }
    }
  </script>
</body>
</html>
```

We have added the multiple attribute to our input element. This changes the text on the button; instead of **Choose file** it now says **Choose files**, and we can select more than one file as a result.

We have changed our upload function a bit as well by adding a loop. And instead of `innerText`, we are now using `innerHTML`, because then we could insert a break using the HTML break. It will output the names of all the selected files below the input box on the screen.

Reading files

There is a special JavaScript object for reading files. It has a very suitable name: `FileReader`. Here is how we can use it to read a file.

```
<!DOCTYPE html>
<html>
<body>
  <input type="file" onchange="uploadAndReadFile(this.files)" />
  <div id="message"></div>
  <script>
    let message = document.getElementById("message");

    function uploadAndReadFile(files) {
      let fr = new FileReader();
      fr.onload = function (e) {
        message.innerHTML = e.target.result;
      };
      fr.readAsText(files[0]);
    }
  </script>
</body>
</html>
```

As you can see, we have to specify what needs to happen in order to connect our HTML and JavaScript to a file. We do this by adding the `onload` event as an anonymous function that is sending on the event data.

Reading the data can then be done using one of the `readAs()` methods on the `FileReader` object. We have used `readAsText()` here, because we are dealing with a text file. This triggers the actual reading and the `onload` function that comes with it gets triggered when it's done, adding the result of the reading to our message. This accepts all file types, but not all file types will make sense.

In order to see something sensible, we will have to upload something that contains plain text, such as `.txt`, `.json`, and `.xml`. With this we can also send a file to the server or process the contents of a log file.

Practice exercise 14.1

This exercise will demonstrate the process of uploading and displaying local image files in your webpage. Use the following HTML and CSS as a starting template:

```
<!doctype html>
<html>
<head>
  <title>Complete JavaScript Course</title>
  <style>
    .thumb {
      max-height: 100px;
    }
  </style>
</head>
<body>
  <input type="file" multiple accept="image/*" />
  <div class="output"></div>
  <script>
  </script>
</body>
</html>
```

Take the following steps to complete the script element:

1. Select your page elements as values within variable objects in your JavaScript code.
2. Add an event listener to the input field. The event trigger should be changed so that it immediately invokes a reader function.
3. Create a function to handle the reading of the selected files.
4. Using the event object, select the target element that triggered the event. Get the files selected within that input and assign them to the `files` variable.
5. Loop through all the files that were selected.
6. Set the files by index within the loop to a variable named `f11e`.

7. Set the image file as the file within the loop selected from the user input field files.
8. Add the newly created `img` tag to the page, create an area on the page that you can output the content to, and append the new page element to it.
9. Create a new `FileReader` object.
10. Add an `onload` event listener to the `fileReader` object to create and invoke an anonymous function that sets the source of the image as the result from the target element. Pass in the image object you just created as an argument into the function.
11. Using `readAsDataURL()`, get the current file object and pass it into the file reader object so that it can be used once the `onload` completes and is added to the page.
12. You can now select multiple image files from your computer and have them show on your webpage.

Getting position data with `Geolocation`

We are going to look at the window object `navigator` now to see whether we can locate the user of the browser. This can be useful for many things, for example, suggesting restaurant locations nearby the user. We can have a look at the `Geolocation` by inspecting `navigator.geolocation`. This is one way to do it:

```
<!DOCTYPE html>
<html>
<body>
<script>
  window.onload = init;

  function init() {
    console.dir(navigator.geolocation);
  }
</script>
</body>
</html>
```

If you check out the log, you can see what the `Geolocation` object contains, and one of these methods is to get the current position of the user. Here is how to use it:

```
<!DOCTYPE html>
<html>
<body>
<script>
  window.onload = init;

  function init() {
    navigator.geolocation.getCurrentPosition(showGeoPosition);
  }

  function showGeoPosition(data) {
    console.dir(data);
  }
</script>
</body>
</html>
```

This might look a bit more complicated than you'd expect, and this is because the `getCurrentPosition()` method takes another method as an argument. The position data gets sent to this function and that function will use the data as input. Therefore, we will have to wrap `console.dir()` in an external function (called `showGeoPosition()`) that takes a parameter and outputs this data so that we can see the data in the console. We can then send this function to the `getCurrentPosition()` function and see the data.

If you run this, you should get a `GeolocationPosition` object, with a `coords` property containing your latitude and longitude. The browser might prompt whether you're okay with sharing your location. And if it doesn't show anything, make sure the preferences and settings of your computer allow the browser to use your location.

Using this, you can get the location of the user and show personalized content based on it or gather data about their location for other purposes, such as analyzing where visitors are located or displaying suggestions for the user based on their location.

HTML5 canvas

Did we already mention that the `<canvas>` element is new in HTML5? This is an amazing tool that will help you create dynamic web apps. Here is how to set up the canvas:

```
<!DOCTYPE html>
<html>
  <body>
    <canvas id="c1"></canvas>
    <script></script>
  </body>
</html>
```

And when you open this page, you will see nothing. Why? Well, the canvas element is, by default, a white rectangle that you cannot see against the white background. You could add some CSS to add a border to the canvas or a background color to the body and your canvas will be revealed.

But, we probably want to put something on it and we need JavaScript to make that happen. Let's create a "drawing" on it using JavaScript:

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      canvas {
        border: 1px solid black;
      }
    </style>
  </head>
  <body>
    <canvas id="c1"></canvas>
    <script>
      let canvas = document.getElementById("c1");
      let ctx = canvas.getContext("2d");
      canvas.width = 500; //px
      canvas.height = 500; //px
      ctx.fillRect(20, 40, 100, 100);
    </script>
  </body>
</html>
```

The context of the canvas is read and stored in the `ctx` variable (a common shortened version of context). We need this to be able to draw on the canvas. We change the dimensions of the canvas to 500 by 500 pixels. This is not the same as using CSS for width and height; this adds the HTML attributes' width and height.

With the `fillRect()` method on the context of the canvas, we can draw a rectangle on the canvas. It takes four parameters. The first two are the *x* and *y* coordinates of where the figure should be added to the canvas. The last two are the width and height of the rectangle. In our case, it's a square. Here is what the result looks like:

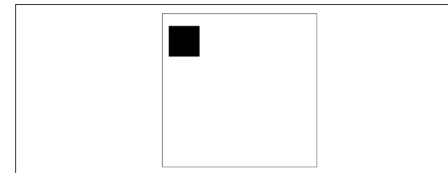


Figure 14-1: Result of the `fillRect()` method on our 500 px by 500 px canvas

We can also change the color we are drawing with. You can get a pink square instead by replacing the JavaScript of the previous HTML document with the following:

```
<script>
  let canvas = document.getElementById("c1");
  let ctx = canvas.getContext("2d");
  canvas.width = 500; //px
  canvas.height = 500; //px
  ctx.fillStyle = "pink";
  ctx.fillRect(20, 40, 100, 100);
</script>
```



We have now just used the word *pink*, but you can also work with hexadecimal color codes for the `fillStyle` property, which could look like this for pink: `#FFC0CB`. The first two characters specify the amount of red (FF here), the third and fourth the amount of green (C0), and the last two the amount of blue (CB). The values differ from 00 to FF (0 to 255 in decimal numbers).

There are more things you can do with the canvas than just drawing. Let's have a look at adding text to our canvas.

Practice exercise 14.2

We will be implementing shapes and using the HTML5 canvas element to draw on a webpage with JavaScript. Draw a rectangle using JavaScript. The output will resemble the following:



Figure 14.2: Exercise outcome

Take the following steps:

1. Add the canvas element to the page.
2. Set the width and height to 640 px and, using CSS, add a 1 pc border to the element.
3. Within the JavaScript, select the canvas element and set the Context to 2d.
4. Set the fill style to red.
5. Create an output of the shape by using a rectangle.
6. Set the outline of the rectangle.
7. Clear the rectangle inside to make it transparent and the color of the background.

Dynamic canvas

We can draw more advanced shapes, add images, and add text. This enables us to take our canvas skills to the next level.

Adding lines and circles to the canvas

Here we will see how to draw a line and a circle. Here is a piece of sample code that draws a line:

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      #canvas1 {
        border: 1px solid black;
      }
    </style>
  </head>
  <body>
    <canvas id="canvas1"></canvas>
    <script>
      let canvas = document.getElementById("canvas1");
      let ctx = canvas.getContext("2d");
      canvas.width = 100;
      canvas.height = 100;
      ctx.lineWidth = 2;
      ctx.moveTo(0, 20);
      ctx.lineTo(50, 100);
      ctx.stroke();
    </script>
  </body>
</html>
```

The line width is set to 2 pixels. This first puts the focus to 0 (x) and 20 (y). This means it is at the very left edge of the canvas, 20 pixels from the top. This canvas is smaller; it is 100 by 100 pixels. The second point is at 50 (x) and 100 (y). This is what the line looks like:

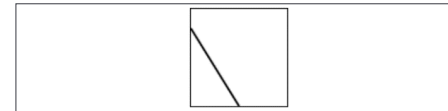


Figure 14.3: Outcome of drawing a line to the canvas

And before we move over to text, here is how to draw a circle.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    #canvas1 {
      border: 1px solid black;
    }
  </style>
</head>
<body>
  <canvas id="canvas1"></canvas>
  <script>
    let canvas = document.getElementById("canvas1");
    let ctx = canvas.getContext("2d");
    canvas.width = 150;
    canvas.height = 200;
    ctx.beginPath();
    ctx.arc(75, 100, 50, 0, Math.PI * 2);
    ctx.stroke();
  </script>
</body>
</html>
```

We use the `arc()` method to create a curve or a circle. It takes five parameters:

- start position x on canvas
- start position y on canvas
- radius of the circle
- starting angle in radians
- ending angle in radians

So, if we don't want a circle, but a semicircle, for example, we'll have to specify a different starting and end angle in radians. This time we used the `stroke()` method to do the actual drawing instead of the `fill()` method:

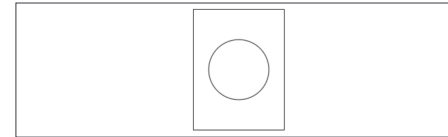


Figure 14.4: Outcome of drawing a circle using the `arc()` method

`Stroke()` is only drawing the line, whereas `fill()` colors the full shape.

In the canvas, the shapes and lines will be added on top of each other, based on the order in which they're drawn. The first one you draw is underneath the latter ones. Exactly what happens when you paint on a real canvas. You will be seeing this in the next practice exercise.

Practice exercise 14.3

In this exercise, you will be drawing a stick person using canvas:

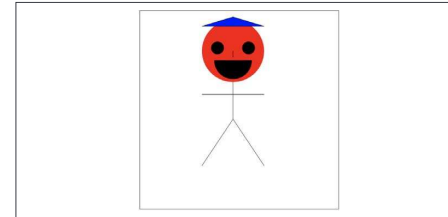


Figure 14.5: Exercise result within the web browser canvas element

1. Create the page elements and prepare to draw on the canvas.
2. Begin the path with an arc roughly at the top center of your canvas object.
3. Using `arc()`, set a position for the left eye, roughly at the top left of the center of the arc you just drew, then add another arc for the right eye. Create a half arc for the mouth (the radian angle for a semicircle is π) and fill all.
4. Move the draw position to the center and draw a line for the nose.
5. Draw the body with a line down from the center of the arc, create the left arm, and then move the draw position to do the right arm, which will be twice the width of the left arm. Move back to the center and continue down to draw the left leg, move back to the center, and draw the line for the right leg.
6. Move to the top, set the colour to blue, and draw a triangle for a hat.

Adding text to the canvas

We can add text to the canvas in a similar fashion as well. In this example, we set a font and a font size, and then write our text to the canvas:

```
<!DOCTYPE html>
<html>
<head>
<style>
#canvas1 {
border: 1px solid black;
}
</style>
</head>
<body>
<canvas id="canvas1"></canvas>
<script>
let canvas = document.getElementById("canvas1");
let ctx = canvas.getContext("2d");
canvas.width = 200;
canvas.height = 200;
ctx.font = "24px Arial";
let txt = "Hi canvas!";
ctx.fillText(txt, 10, 35);
</script>
</body>
</html>
```

The `fillText()` method is used to add text. We have to specify three parameters: the text, the x position, and the y position. Here is the result:



Figure 14.6: Result of using the `fillText()` method

We have specified 35 px from the top for the text to start. We can specify other aspects of the text, for example, like this:

```
ctx.textAlign = "center";
```

Here, we use the `textAlign` property on the canvas to specify how the text should be aligned.

Practice exercise 14.4

We will work with text and add text to your canvas element. The following exercise will demonstrate how to dynamically add text and position it within your canvas element. The result from the exercise code will look similar to this diagram:



Figure 14.7: Exercise outcome

Take the following steps:

1. Create a simple HTML document, and add the canvas element to your page. Set the height and width to 640, and add a 1 px border to the element so you can see it on the page.
2. Select the page elements as values within JavaScript variables.
3. Create a string variable with the message `Hello World`.
4. Set a font style using the `font` property and a blue fill color using the `fillStyle` property. You can also align the text to the left.
5. Add the text to the canvas with `fillText` and set the *x* and *y* positions of the text.
6. Set a new font and color of red.
7. Create a loop and, using the value of the loop variable, add text to the page canvas element.

Adding and uploading images to the canvas

We can add an image to the canvas. We can simply get an image from our page, and add it to our canvas:

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      canvas {
        border: 1px solid black;
      }
    </style>
  </head>
  <body>
    <canvas id="c1"></canvas>
    
    <script>
      window.onload = function () {
        let canvas = document.getElementById("c1");
        canvas.height = 300;
        canvas.width = 300;
        let ctx = canvas.getContext("2d");
        let myImage = document.getElementById("flower");
        ctx.drawImage(myImage, 10, 10);
      };
    </script>
  </body>
</html>
```

We wrap it all in an `onload` event listener here because we want to be sure that the image is loaded before getting it from the DOM, else the canvas will remain empty. We use the `drawImage()` method to add an image to the canvas. It takes three arguments: the *image*, the *x* position, and the *y* position.

We can use one canvas inside another canvas as well. We do this exactly like we did when we were using the image. This is a very powerful feature, because it enables us to use a part of the drawing from the user input, for example. Let's look at an example of how to do this:

```
<!DOCTYPE html>
<html>
  <head>
    <style>
```

```

    canvas {
      border: 1px solid black;
    }
  </style>
</head>
<body>
  <canvas id="canvas1"></canvas>
  <canvas id="canvas2"></canvas>
  <canvas id="canvas3"></canvas>
  <script>
    let canvas1 = document.getElementById("canvas1");
    let ctx1 = canvas1.getContext("2d");
    ctx1.strokeRect(5, 5, 150, 100);

    let canvas2 = document.getElementById("canvas2");
    let ctx2 = canvas2.getContext("2d");
    ctx2.beginPath();
    ctx2.arc(50, 50, 20, 0, 2 * Math.PI);
    ctx2.stroke();

    let canvas3 = document.getElementById("canvas3");
    let ctx3 = canvas3.getContext("2d");
    ctx3.drawImage(canvas1, 10, 10);
    ctx3.drawImage(canvas2, 10, 10);
  </script>
</body>
</html>

```

We create three canvases, to two we add shapes, and the third one is a combination of the first two. Here is what it looks like:



Figure 14.8: The result: three canvases with shapes

We also can upload images to the canvas. This can be of great use when you want to show a preview to your user of something that was just uploaded, for example, a profile picture. This is very similar to grabbing the `<img>` element from the webpage and using that element, but this time we need to read our data from the uploaded file, create a new image element, and then draw that image to the canvas.

The below code does just that:

```

<html>
<head>
  <style>
    canvas {
      border: 1px solid black;
    }
  </style>
</head>
<body>
  <input type="file" id="imgLoader" />
  <br>
  <canvas id="canvas"></canvas>
  <script>
    let canvas = document.getElementById("canvas");
    let ctx = canvas.getContext("2d");
    let imgLoader = document.getElementById("imgLoader");
    imgLoader.addEventListener("change", upImage, false);
    function upImage() {
      let fr = new FileReader();
      fr.readAsDataURL(event.target.files[0]);
      fr.onload = function (e) {
        let img = new Image();
        img.src = event.target.result;
        img.onload = function () {
          canvas.width = img.width;
          canvas.height = img.height;
          ctx.drawImage(img, 0, 0);
        };
        console.log(fr);
      };
    }
  </script>
</body>
</html>

```

Every time the input of the input field changes, the `upImage()` method gets executed. This method does a few things, so let's break them down. First of all, we create a new `FileReader` and add the uploaded file. There is only one in this case, so we use index `0`. Instead of `readAsText()` that we have already seen, we are now using `readAsDataURL()`, which we can use to read images.

This will trigger the `onload` event. And in our case, this creates a new image that can be added to the canvas later. As a source, we add the result of our read action and when the image is loaded, we change the size of our canvas to the size of our picture and then add the picture in there.

These new skills will enable you to work with images on canvases, draw your own images, upload images from elsewhere, and even re-use the ones on the webpage. This can come in handy for many situations, for example, to create basic animation, or to create the functionality to upload a new profile picture to a user's profile.

Practice exercise 14.5

We will practice uploading a local image to the canvas. The following exercise will demonstrate how to upload images from your local computer and have them displayed within the canvas element within your browser.

1. Set up the page elements and add an input field to upload an image. Add the canvas element to the page.
2. In JavaScript, select the input field and the canvas elements as JavaScript objects.
3. Add an event listener to invoke an upload function once there is a change in the input field contents.
4. Create the aforementioned function to handle the upload of the image to the canvas. Using `FileReader`, create a new `FileReader` object. In the `reader.onload` event, create a new image object.
5. Add an `onload` listener to the image object so that when the image is loaded, set the canvas height and width to match half the image size height and width. Using `ctx.drawImage()`, add the image to the canvas.
6. Set the `img` source to the result from the input value.
7. Use the reader object and invoke `readAsDataURL()` to convert the file input value to a readable format of base64 image data that can be used within the canvas.

Adding animations to the canvas

With the methods we have seen so far, we can already start creating animations. We do this by using loops and recursion, combined with `timeout()`. These drawings with (short) time intervals result in an animation. Let's start with a basic animation:

```
<!DOCTYPE html>
<html>
```

```
<head>
<style>
  canvas {
    border: 1px solid black;
  }
</style>
</head>

<body>
<canvas id="canvas"></canvas>
<script>
  window.onload = init;
  var canvas = document.getElementById("canvas");
  var ctx = canvas.getContext("2d");
  canvas.height = 500;
  canvas.width = 500;
  var pos = {
    x: 0,
    y: 50,
  };

  function init() {
    draw();
  }

  function draw() {
    pos.x = pos.x + 5;
    if (pos.x > canvas.width) {
      pos.x = 0;
    }
    if (pos.y > canvas.height) {
      pos.y = 0;
    }

    ctx.fillRect(pos.x, pos.y, 100, 100);
    window.setTimeout(draw, 50);
  }
</script>
</body>
</html>
```

This will start drawing a square at position 5, 50. And after 50 ms, it will draw another square at position 10, 50, and after that at 15, 50. And it will keep on changing this *x* value by 5 up to the point that *x* gets bigger than the width of the canvas, when it is then set to zero. This way, the last bit of white canvas on that line gets colored black too.

Right now, it is more creating a line, and not a moving square. This is because we keep on adding the colored part to the canvas, but not resetting it to the previous color. We can do this with the `clearRect()` method. This method takes four parameters. The first two are the starting point to draw the rectangle to be cleared (so *x*, *y*). The third one is the width of the rectangle to be cleared and the last one is the height. In order to clear the full canvas, we'll have to write:

```
ctx.clearRect(0, 0, canvas.width, canvas.height);
```

Adding this to the beginning of the draw function in our previous example results in a moving square instead of a fat line being drawn because the previous square is not kept, but the canvas resets every time and the square gets drawn from scratch.

Practice exercise 14.6

We will practice animating shapes and moving objects on the page. This exercise will demonstrate how to move an object on the page using the HTML5 canvas element and JavaScript.

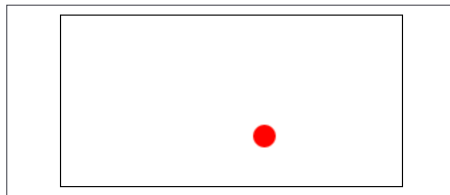


Figure 14.9: Red circle moving within the boundaries of the canvas object

Take the following steps to create a red circle that will then be moved within the canvas boundaries, appearing to be bouncing off the sides:

1. Create the canvas and apply a border of 1 px to it.
2. Select the canvas page elements with JavaScript and prepare to draw on the canvas.
3. Create variables to track the *x* and *y* positions, as well as the *x*-direction speed and the *y*-direction speed. You can set these as a default of 1 and the *x* and *y* starting positions can be half of the canvas dimensions.
4. Create a function to draw the ball. This will draw the ball as a red ball arc in the *x* and *y* positions. Also, the size for the ball should be set as a variable so that the boundaries can be calculated from it. Fill and close the path.
5. Create a function to move the ball and set the interval on that function to 10 milliseconds.
6. In the aforementioned movement function, clear the current rectangle and draw the ball using the draw ball function.
7. Check the position of the ball. If the ball is outside the canvas boundaries, you need to change direction. This can be done by multiplying the direction by -1. Update the *x* and *y* positions with the new values.

Drawing on canvas with a mouse

We have all the ingredients already to create a canvas on which we can draw with our mouse. Let's walk you through it. We'll start by setting up the canvas:

```
<!DOCTYPE html>
<html>
<head>
<style>
  canvas {
    border: 1px solid black;
  }
</style>
</head>
<body>
  <canvas id="canvas"></canvas>
  <input type="color" id="bgColor" />
</body>
```

```

<script>
  let canvas = document.getElementById("canvas");
  let ctx = canvas.getContext("2d");
  canvas.width = 700;
  canvas.height = 700;
</script>
</body>
</html>

```

In our script element, we are going to add a method for when the window has loaded. When the window has loaded, we need to add some event listeners:

```

window.onload = init; // add this line to the start of the script

function init() {
  canvas.addEventListener("mousemove", draw);
  canvas.addEventListener("mousemove", setPosition);
  canvas.addEventListener("mouseenter", setPosition);
}

```

We want to draw when the mouse is moving, and we want to change the current position on the canvas when the mouse is moving. This is also something we want to do on `mouseenter`. Let's write the code for setting the position. This will be added to the script element as well. We will also have to add the position variable, which again should be declared at the start of the script:

```

let pos = {
  x: 0,
  y: 0,
};

```

And the function for setting the position:

```

function setPosition(e) {
  pos.x = e.pageX;
  pos.y = e.pageY;
}

```

This function gets triggered on `mousemove` and on `mouseenter`. The event that triggers this has a `pageX` and a `pageY` property we can use to get the current position of the mouse.

The last must-have ingredient for drawing on the canvas is the `draw()` method. Here is what it could look like:

```

function draw(e) {
  if (e.buttons !== 1) return;
  ctx.beginPath();
  ctx.moveTo(pos.x, pos.y);
  setPosition(e);
  ctx.lineTo(pos.x, pos.y);
  ctx.lineWidth = 10;
  ctx.lineCap = "round";
  ctx.stroke();
}

```

We start with something that might look strange, but it is a great trick to make sure that the mouse is actually being clicked. We don't want to be drawing when no button on the mouse is clicked. This method prevents that by returning from the method if it is not being clicked.

Then we start to begin a path. We always have a current `x` and `y`, so they are set as coordinate one, and then we set them again and use these new coordinates for the line. We give it a round linecap to achieve smooth lines and a line width of 10. Then we draw the line, and as long as the mouse is moving, the `draw()` function gets called again.

The app can now be opened and used as a functioning drawing tool. We can also give the user more options here, for example, adding a color picker to change the color that the user is drawing with. In order to do that, we'll have to add a color picker to the HTML, like this:

```

<input type="color" id="bgColor" />

```

And change the selected color in JavaScript by adding an event listener for when the value of that input box changes:

```

let bgColor = "pink";
let bgC = document.getElementById("bgColor");
bgC.addEventListener("change", function () {
  bgColor = event.target.value;
});

```

We start with the color pink, and overwrite it with whatever the user selects in the color picker.

Practice exercise 14.7

We will create an online drawing board, and include a dynamic value for width, color, and ability to erase the current drawing. Use the following HTML as a template for this project to add JavaScript code to:

```
<!doctype html>
<html>
<head>
  <style>
    canvas {
      border: 1px solid black;
    }
  </style>
</head>
<body>
  <div class="controls">
    <button class="clear">Clear</button> <span>Color
    <input type="color" value="#ffff00" id="penColor"></span>
  <span>Width
    <input type="range" min="1" max="20" value="10"
    id="penWidth"></span> </div>
  <div>
    <canvas id="canvas"></canvas>
  </div>
</body>
</html>
```

Take the following steps:

1. Select the page elements as variable objects in JavaScript. Get the input field and select the button as an object.
2. Add an event listener to the button that will run a function to clear the current canvas. Within the clear function, use the `confirm()` method to check if the user wants to erase the canvas drawing. If they then confirm using `clearRect()`, delete the contents of the canvas element.
3. Set a global position object for *x* and *y* and by adding event listeners to the mouse events, update the position. If the mouse move is triggered, invoke a draw function. Set the position to update the mouse position, setting the global position values to that of the mouse *x* and *y*.

4. Within the draw function, check if the mouse button is pressed, and if not, then add `return`. If it is pressed, we can then draw on the canvas. Set the new path and move to the positions *x* and *y*. Start a new line, get the `strokestyle` value from the color input field, and set the `linewidth` value from the input width value. Add the `stroke()` method to add the new line to the page.

Saving dynamic images

We can convert the canvas to an image, and this image can then be saved as a next step. In order to convert it to an image, we need to add the following to our script element:

```
let dataURL = canvas.toDataURL();
document.getElementById("imageId").src = dataURL;
```

We are changing our canvas to a data URL, which then becomes the source of our image. We want this to happen whenever a save button gets clicked. Here is the button:

```
<input type="button" id="save" value="save" />
```

And the event listener:

```
document.getElementById("save").addEventListener("click", function () {
  let dataURL = canvas.toDataURL();
  document.getElementById("holder").src = dataURL;
});
```

Now whenever the save button gets clicked, it is going to update the image with the generated data URL from the canvas. Whatever content is within the canvas element will be turned into a base64 data image value and added to the page within an `img` tag.

In the following example, there is a canvas of 200 by 200 pixels and an empty image of the same size. When a color gets selected, a square of 100 by 100 pixels in that color is drawn on the canvas. When the save button gets clicked, this canvas gets converted to an image. This image can then be saved. Here is the code for the example:


```

<!doctype html>
<html>
<head>
  <style>
    canvas {
      border: 1px solid black;
    }
  </style>
</head>
<body>
  <canvas id="canvas"></canvas>
  <input type="color" id="squareColor" />
  <br>
  <img src="" width="200" height="200" id="holder" />
  <input type="button" id="save" value="save" />
  <script>
    const canvas = document.getElementById("canvas");
    const ctx = canvas.getContext("2d");
    canvas.width = 200;
    canvas.height = 200;
    const penColor = document.getElementById("squareColor");
    penColor.addEventListener("change", function () {
      color = event.target.value;
      draw(color);
    });
    document.getElementById("save").addEventListener("click",
function () {
  let dataURL = canvas.toDataURL();
  document.getElementById("holder").src = dataURL;
});
function draw(color) {
  ctx.fillStyle = color;
  ctx.fillRect(70, 70, 100, 100);
}
  </script>
</body>
</html>

```

Here is what it looks like after saving the image:

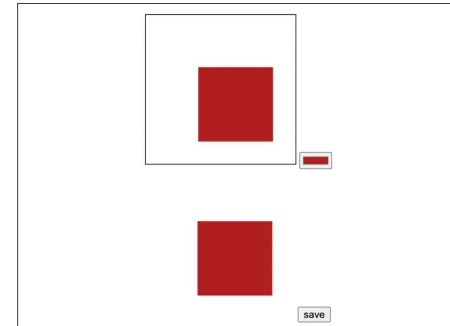


Figure 14.10: Result of saving the image

Media on the page

There are special elements for media on the page. We are going to show you how to add audio and video and how to embed YouTube on a webpage.

Adding an audio player to a page is very simple:

```

<!DOCTYPE html>
<html>
<body>
  <audio controls>
    <source src="sound.ogg" type="audio/ogg">
    <source src="sound.mp3" type="audio/mpeg">
  </audio>
</body>
</html>

```

You specify the `controls` attribute if you want the user to be able to control pause and play and the volume. If you want it to start automatically, you'll have to add the attribute `autoplay`. With the `source` element, you specify the audio files that can be played. The browser will choose only one and will choose the first one (from top to bottom) that it supports.

Adding a video to a webpage is very similar to adding audio. Here's how to do it:

```
<video width="1024" height="576" controls>
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.ogg" type="video/ogg">
</video>
```

Often you would want to link to YouTube instead. Here's how to do that:

```
<iframe
  width="1024"
  height="576"
  src="https://www.youtube.com/embed/v6TV7czb1Y"
>
</iframe>
```

You will have to use the `iframe` element. This is a special element that allows another webpage inside the current webpage. And you can then add the YouTube embed link as a source. The last code after `embed` comes from the video URL.

The height and width attributes of the video can be changed to make the video bigger or smaller. If you want to show it fullscreen, you can change the width and height like this:

```
<iframe
  width="100%"
  height="100%"
  src="https://www.youtube.com/embed/v6TV7czb1Y"
>
</iframe>
```

If you want it to be only a part of the screen, you can adjust the width and height attributes accordingly.

You can autoplay these as well with the `autoplay` attribute. If you use `autoplay` on more than one, none of them will autoplay to protect the visitor from getting all that noise from the webpage. It is typically considered annoying if your video starts making noise in the browser. Adding the attribute `muted` will avoid this.

Digital accessibility in HTML

Digital accessibility is of huge importance for visually impaired people or those unable to use a mouse. In order to use the internet with little or no vision, screen readers are in place. This is a special piece of software that reads what is on the screen or converts it to braille using special devices connected to the computer. People that cannot use a mouse will often rely on speech to give the computer instructions.

Early web applications were terrible in terms of accessibility. Luckily, WAI-ARIA created a technical specification of how to make the internet digitally accessible. Dynamic parts can be recognized if implemented correctly, and by adding semantics and metadata to the HTML, it's better useable for external tooling.

Semantics might be one of the most important parts here. This comes down to using the right HTML element for the right purpose. If something should be clicked, it is best to make it a `<button>` element and not a `<span>`, for example. If it is a button, it is possible to navigate to it with the *Tab* key and click it using *Enter*.

The same goes for headers. You can create something that looks like a header using a special class and give it a layout, but the screen readers are looking for `h1`, `h2`, and `h3`. You should always use the header elements for headers. This helps the screen readers and improves the accessibility of your website. And as a bonus, it helps you rank higher in Google as well because bots also check out the headers to see what is important on your site.

It is also important to use labels and link text that is descriptive. If the link part is only **Click here**, that is not helpful. Something like **Click here to sign up for the summer event** is much better.

Throughout this book, we have also done something wrong with our input boxes. In order to make input fields accessible, you'll have to add a label element. This will make it easier for screen readers to pick up on what the input box is about. So this is generally bad practice:

```
<input type="text" id="address" />
```

And this is much better, because now screen readers can read it too (and therefore visually impaired people can understand it):

```
<label for="address">Address:</label>
<input type="text" id="address" />
```

One last one that you may know already is the `alt` attribute for images. If the screen reader encounters an image, it will read the `alt` description. So make sure that these are descriptive, even if the image is not important. Since there is clearly no way to know it's not important if you cannot see the image, all you'll know is that you cannot see some picture. Here is how to add `alt` text:

```

```

These tips are not that important for practicing and testing purposes, but they are of great use when you are going to create professional apps. Taking accessibility into account will make your app more accessible for everyone. And as I said, Google will (currently) reward this good behavior by ranking you more highly and your app will be more profitable since more people can use it!

Chapter projects

Create a Matrix effect

This exercise will create a continuous animation of text moving from top to bottom. The final effect produced will show characters moving down the screen within the canvas element and appearing to disappear and fade as they approach the bottom of the screen as more new characters will be added to the canvas in their place. The random character can be either a 0 or 1, and will be in place in the position according to the number, which will represent the vertical position of where the character is drawn.

The canvas will be filled with a black background, which is going to use opacity to create the fading effect once it's redrawn:



Figure 14.11: Matrix effect desired outcome

Take the following steps:

1. Create a simple HTML document, and in JavaScript create a canvas element and add the `getContext` element as 2d.
2. Select that canvas element and set the attribute height and width to 500x400. Prepend it to the body of your document.
3. Create an empty array named `colVal` and create a loop to add a number of items into the array that will have a value of 0. The number of items you need to add to the array can be determined by dividing the width by ten, which should be the width between each column. The values in the array will be the starting vertical position of the content for the `fillText()` method that you will set up.
4. Create the main Matrix function to run at 50 millisecond intervals.
5. Set the `fillStyle` to be black with .05 opacity, so that when it layers on top of the existing elements it will produce a fading effect.
6. Set the canvas font color to be green.
7. Using an array map, iterate all the current items in the `colVal` array, which holds the vertical position for the output text.

8. Within the map, set the characters to display. We want it to alternate between 0 and 1 so, using `Math.random()`, generate a value of either 0 or 1 for the text output. You can use a ternary operator to do this.
9. Set the position of `x` using the index value multiplied by 10, which is the start of each new letter. Using the index from the `colVal` array, this will create separate columns of moving characters.
10. Create the character within the canvas using the `ctx.fillText()` method, setting the output character to the random 0 or 1 value, using `posX` for the column `x` position, and `posY`, which is the value in the `colVal` array for the item, as the position of the `y` axis for the output.
11. Add a condition that checks if the position of `y` is greater than 100 plus a random value of 0-300. The larger the number, the longer the number will fall on the `y` position. This is random so not all numbers end at the same spot. This will create a staggered effect after the initial drop.
12. If the position of `y` is not past the random value and 100, increment the value of the index item by 10. Assign this value of `y` back to the item in the `colVal` array, which can then be used in the next iteration. This will move the letter down 10 pixels on the canvas within the next draw round.

Countdown clock

This exercise will produce a real-time countdown clock that will display the amount of time in days, hours, minutes, and seconds left until the date value within the input date field. Adjusting the input date field will update the countdown clock. It will also use local storage to capture and save the value in the input field, so if the page is refreshed, the input field will still retain the date value and the countdown clock can continue to count down to that date value from the input field. You can use the following HTML template:

```
<!doctype html>
<html>
<head>
  <title>JavaScript</title>
  <style>
    .clock {
      background-color: blue;
      width: 400px;
      text-align: center;
      color: white;
      font-size: 1em;
    }
  </style>
</head>
<body>
  <div>
    <input type="date" name="endDate">
    <div class="clock">
      <span><span class="days">0</span> Days</span>
      <span><span class="hours">0</span> Hours</span>
      <span><span class="minutes">0</span> Minutes</span>
      <span><span class="seconds">0</span> Seconds</span>
    </div>
  </div>
</body>
</html>
```

```
.clock>span {
  padding: 10px;
  border-radius: 10px;
  background-color: black;
}
.clock>span>span {
  padding: 5px;
  border-radius: 10px;
  background-color: red;
}
input {
  padding: 15px;
  margin: 20px;
  font-size: 1.5em;
}
</style>
</head>
<body>
  <div>
    <input type="date" name="endDate">
    <div class="clock">
      <span><span class="days">0</span> Days</span>
      <span><span class="hours">0</span> Hours</span>
      <span><span class="minutes">0</span> Minutes</span>
      <span><span class="seconds">0</span> Seconds</span>
    </div>
  </div>
</body>
</html>
```

We have created page elements including input with a type of date, a main clock container, and added spans for `days`, `hours`, `minutes`, and `seconds`. They have been labeled and CSS applied as needed.

You can take the following steps:

1. Select the page elements as JavaScript objects, as well as selecting the main clock output area as the value of a JavaScript object.

2. Create variables for the `timeInterval` and a global Boolean value that can be used to stop the clock timer.
3. Check the local storage if there is an item for countdown already set. If there is, use that value.
4. Create a condition and function to start the clock as the saved value and set the input field date value to the local storage saved value.
5. Add an event listener to invoke a function if the value of the input field is changed. Clear the interval if it has changed and set the new `endDate` value in the local storage.
6. Start the clock with the start clock function from that new `endDate` input value.
7. Create a function to start the clock that is used to start the counter. Within that function, you can create a function that updates the counter and outputs the new clock time values into the page clock container area.
8. Within this function, check if `timeLeft` is less than the counter time. Create a separate function to handle this. If it's less, stop the timer.
9. If the time left is more and has a value within the object, then output the object by property names and match the property names you use in the time left function object to the class names you use in your webpage elements so they match and you can save time rewriting them. Loop through all the object values and assign the values within the `innerHTML` page element.
10. In the time left function, get the current date. Using `Date.parse()`, parse the date and calculate the total milliseconds left until the counter ends. Return the values of the total days, hours, minutes, and seconds as a response object to use in the update function.
11. If the counter is false and has passed the end time, then clear the interval. If the counter is still valid, set the interval to run the update function every 1,000 milliseconds.

Online paint app

Create a drawing application where the user can draw using their mouse in the canvas element. When the user is within the canvas element and clicks down on the mouse button, holding the button down will add lines, producing a drawing effect within the canvas element. The color and width of the drawing pencil can be changed dynamically for more functionality. In addition, this app will include a button to save and download the image from the canvas element, as well as clearing the current canvas content.

You can use the following template and add the JavaScript code:

```
<!doctype html>
<html>
<head>
  <title>Canvas HTML5</title>
  <style>
    #canvas {
      border: 1px solid black;
    }
  </style>
</head>
<body>
  <canvas id="canvas" width="600" height="400"></canvas>
  <div>
    <button class="save">Save</button>
    <button class="clear">Clear</button>
    <span>Color: <input type="color" value="#ffff00"
id="penColor"></span>
    <span>Width: <input type="range" min="1" max="20" value="10"
id="penWidth"></span>
  </div>
  <div class="output"></div>
  <script>

</script>
</body>
</html>
```

We have created a button to save and a button to clear, an input for color using the HTML5 color type, and the range type to get a numeric value for the pen width. We have also added page elements for the canvas and an output area.

Take the following steps:

1. Using JavaScript, select all the page elements as JavaScript objects and set up the canvas element to draw into.
2. Set a variable to track the location of the pen.
3. On the canvas, add an event listener to track mouse movement. Update the pen position to the `lastX` and `lastY` positions, and then set the location position to `clientX` and `clientY`. Create a function to draw at the pen position and invoke the draw function.

4. For mousedown, set draw to true, and for mouseup and mouseout, set draw to false.
5. Within the draw function, begin the move path at the pen location values and set the stroke style to the pen color and stroke width to the pen width. These can be changed by clicking the inputs and updating their HTML values. Add the stroke and close the drawing path.
6. Add an event listener to the clear button. If clicked, create a function that confirms that the user wants to remove and clear the drawing, and then if true, invoke `clearRect()` to clear the canvas contents.
7. Add another event listener to save the image. When clicked, it should invoke a function that gets the canvas object using `toDataURL` as base64 image data. You can log it into the console to see what it looks like.
8. Create an `img` element and prepend it to the output area element. Set the `src` path to the `dataURL` value.
9. To set a download of the image, create an anchor tag, append it to anywhere within the HTML page elements, and create a filename. You can generate a unique filename with `Math.random()`. Set the hyperlink to the download attribute and the `href` path to the `dataURL` path, and trigger a click with the `click()` method. Once clicked, remove the link element.

Self-check quiz

1. Which statements below are the correct way to prepare to draw?

```
const canvas = document.getElementById("canvas");
const ctx = canvas.getContext("2d");
var canvas = document.getElementById("canvas");
var ctx = getContext("canvas");
var ctx = canvas.getContext("canvas");
```

2. What will the following code do?

```
<canvas id="canvas"></canvas>
<script>
  var canvas = document.getElementById("canvas");
  var ctx = canvas.getContext("2d");
  canvas.height = 600;
  canvas.width = 500;
  ctx.beginPath();
  ctx.fillStyle = "red";
```

```
ctx.arc(50, 50, 50, 0, Math.PI * 2);
ctx.fill();
ctx.closePath();
</script>
```

- Nothing, the code has errors
 - Draws a red square
 - Draws a red circle
 - Draws half a circle
3. What are all three methods required to draw a line within the canvas element, and in what order?

Summary

We have discussed a lot of great additions to our JavaScript toolbox using HTML5 in this chapter. These new skills will really enhance our capabilities to build interactive web apps. We started off with the local file reader, which enabled us to upload and read files using several methods, such as the `readAsText()` method. Then we saw how to get the `Geolocation` of a user. This can be great to personalize suggestions, for example, for restaurants or parking spots.

The canvas was yet another amazing addition to what we can do with web page(s). Canvases allow us to draw, write text, add images (by drawing and uploading), and create complete animations. This all can be done using the methods on the canvas.

We then had a look at media on the page and how to add audio and video. Finally, we discussed the topic of digital accessibility and how to make sure your website is accessible for everybody, with and without a screen reader.

And at this point, we can say, you did it! You've worked your way through so many basic and advanced web development topics. In the final chapter, we are going to be exploring the next steps for you to take your skills to the next level beyond pure JavaScript, which is what this book has focused on.

15

Next Steps

You've come very far already! At this point, you should have the building blocks of JavaScript down. And you are able to create apps, write clever scripts, and read a lot of code. This is a great foundation for some serious next steps. In this chapter, we will be taking what you've learned to the next step by practicing and figuring out what interests you out of the endless possibilities that JavaScript offers.

We won't go into too much detail about all the topics here. The details will be outdated soon and there is an endless supply of very well-crafted tutorials and information on the internet for each one of them. Chances are that by the time you are reading this, the frameworks and libraries we are recommending are hopelessly old. The good news is that the likelihood that the next big thing will use the same concepts is huge.

This chapter will serve as a starting point for your next steps with JavaScript. We will cover the following topics:

- Libraries and frameworks
- Learning the backend
- Next steps



Note: exercise, project and self-check quiz answers can be found in the *Appendix*.

Libraries and frameworks

Let's start with libraries and frameworks. Libraries are basically pre-programmed JavaScript modules that you can use to speed up your development process. They typically do one specific thing for you. Frameworks are very similar, they are also pre-programmed, but instead of doing only one thing for you, they arrange a whole list of things. This is why it is called a framework, it really is providing you a solid place to start from and usually demands a certain structure for your files in order to do so. A framework is often a bundle of libraries that provide an all-in-one solution. Or at least a many-in-one. You'll eventually even find yourself using external libraries on top of the frameworks.

To give a non-code example, if we started building a car, we could do so from scratch and make every single piece of this car ourselves. This is pretty much what we've been doing in this book so far. With libraries, we get ready-made parts—in our car example, we could get fully built chairs that we only would have to install onto the car frame we've built. If we used a framework to make a car, we would get the skeleton of the car itself, with all the essential parts in it already, and it would probably be capable of driving already. We would only need to focus on customizing the car and making sure it includes all the special things for our wants and needs. While doing that, we would have to keep in mind the skeleton of the car we already have and continue in that style.

As you can imagine, we would be done with our car project a lot faster using libraries and frameworks. Also, we would run into less trouble using libraries and frameworks, since the pre-made parts would have been well tested by many others already. If we were to make our own car chairs from scratch, chances are that after a year of driving they are no longer comfortable, whereas the standard solution has been thoroughly checked already.

So, libraries and frameworks don't just speed up the process, they also provide you with a more stable and better-tested solution. Are there no downsides? Well, of course there are. The most important one is probably flexibility, as you will have to stick to the structure of the framework you are using. To some extent, this could also be an advantage because it usually requires a well-structured coding style from you, which will improve the code quality.

Another downside is that you'll have to keep on updating your app whenever the framework or library you are using is updated. This is very important, especially when the updates are fixes to security issues. On the one hand, frameworks and libraries are very reliable, but because they're so commonly used, it is not unusual for hackers to find weaknesses. If they find one, this will give them opportunities on many apps, including your own. On the other hand, your own code is probably weaker than an average framework, by a lot.

However, in many cases, hacking your custom app might be too costly. For example, when you just have a hobby project online, you are probably not going to pay a huge amount of ransom money to hackers and the data in your app also won't be worth the hackers' effort. Whereas a script that just tries to exploit a weakness of an often-used framework for apps on a random number of websites is common. To minimize the risk, update your dependencies often and keep an eye out for reported weaknesses by the owner of your library or framework.

Libraries

Technically, we cannot do anything more with frameworks and libraries than we can do without them. That is, if you leave time out of the equation. Frameworks and libraries allow us to develop to a higher quality a lot faster, and this is why they are so popular.

We will be discussing a few of the most popular libraries here. This is definitely not an exclusive list, and it is also very dynamic, so other libraries or frameworks might be more popular in a year's time. This is why we are not going to be covering full tutorials and how to get started here. We will just explain the basic principles and show some code snippets. However, this is still a solid foundation for the next big step in your development career.

Many of the libraries can be included in a page by adding a script tag to the head of the HTML, like this:

```
<script src="https://linktolibrary.com/librarycode.js"></script>
```

We will start by discussing a few common libraries.

jQuery

jQuery is arguably the most famous JavaScript library. It was great to use in the past, when it would be compiled into the latest version of JavaScript for the specific browser. Nowadays, it is just a different way of writing some of the things we have seen in the book. You can recognize jQuery easily by the amount of dollar signs in the code. You can also tell if a website is using jQuery if you type `$` or `jQuery` into the console of the website, and it returns the jQuery object. The jQuery library is mainly focused on selecting HTML elements from the DOM and interacting with and manipulating them. It roughly looks like this:

```
$(selector).action();
```

With the dollar sign you indicate that you want to start jQuery, and with the selector you can select the element in HTML. The signs here are a bit like CSS:

- Just a simple string value targets an HTML element: `$("p")`
- A period before a word or phrase indicates you want to select all elements with a certain class: `$(".special")`
- A hashtag targets an element with a certain ID: `$("#unique")`
- You can also use any other CSS selector, including the more complicated chained ones

Here is an example where the jQuery library is imported in the `script` element starting on line 3:

```
<html>
<head>
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/
jquery.min.js"></script>
</head>
<body>
  <p>Let's play a game!</p>
  <p>Of hide and seek...</p>
  <p class="easy">I'm easy to find!</p>
  <button id="hidebutton">Great idea</button>
  <button id="revealbutton">Found you!</button>
  <script>
    $(document).ready(function () {
      $("#hidebutton").click(function () {
        $("p").hide();
      });
      $("#revealbutton").click(function () {
        $(".easy").show();
      });
    });
  </script>
</body>
</html>
```


This is what the page looks like:



Figure 15.1: Page with a simple jQuery script

When you click the **Great idea** button, all the paragraphs will be hidden. This is done inside the event that's been added using jQuery. First, we selected the button with the ID `hidebutton`, next we call the `click` function on it, which specifies what will happen on click. In that function, we state that we'll select all `p` elements and hide them. `hide` is a special jQuery function that adds the `display:none` style to the HTML element.

So, after clicking, all the paragraphs are gone. When we click on **Found you!**, only one comes back, the last one reading **I'm easy to find**. This is because when the button with the ID `revealbutton` gets clicked, it selects all elements with class `easy` and removes the `display:none` from the style using the jQuery `show` function.

This is what jQuery really comes down to:

- Getting the selectors down
- Knowing some extra or differently named functions to manipulate the elements

You can use jQuery in your code, but this won't expand your possibilities to do more with JavaScript. It will just allow you to do the same thing with fewer characters of code. The reason jQuery was so popular is that it added a lot of value when browsers were less standardized, in which case using jQuery would actually provide the solution to standardizing JavaScript across multiple browsers. This is of little use nowadays, and if you are going to write new code, you would be better just using JavaScript. However, whenever you are working on older code, it is very likely you'll run into jQuery so knowing how it works will definitely help you in these cases.



At the time of writing, you can find the jQuery docs here:
<https://api.jquery.com/>.

D3

D3 stands for three Ds: **data-driven documents**. It is a JavaScript library that helps manipulate documents based on data and it can be used to visualize data using HTML, SVG, and CSS. It comes in very handy for dashboards that need to contain any sort of data representation.

You can make pretty much any kind of graph you could want with a lot of features using D3. It can look rather intimidating, because all the settings for the graph figure need to be set. Diving into it and breaking it up in pieces will ensure you'll overcome any hurdles. Below you'll find a very basic example to add three spheres to an SVG using D3:

```
<!DOCTYPE html>
<html>
<head>
  <script src="https://d3js.org/d3.v7.min.js"></script>
  <style>
    svg {
      background-color: lightgrey;
    }
  </style>
</head>
<body>
  <svg id="drawing-area" height=100 width=500></svg>
  <script>
    let svg = d3.select("#drawing-area");
    svg.append("circle")
      .attr("cx", 100).attr("cy", 50).attr("r", 20).style("fill",
"pink");
    svg.append("circle")
      .attr("cx", 200).attr("cy", 20).attr("r", 20).style("fill",
"black");
    svg.append("circle")
      .attr("cx", 300).attr("cy", 70).attr("r", 20).style("fill",
"grey");
  </script>
</body>
</html>
```

```

</script>
</body>

</html>

```

The D3 library gets imported in the first script tag. And the `svg` variable gets created using the `d3.select` method on the `svg` with ID `drawing-area`.

We are not doing the possibilities of D3 any justice—in this case, this isn't a lot more useful than just doing this with a canvas. However, you can make beautiful animations of the data, such as a zoom effect, a sortable bar graph, a spin effect on a sphere, and so much more. That code would take up multiple pages of the book though.



At the time of writing, you can find the full documentation here: <https://devdocs.io/d3~4/>.

Underscore

Underscore is a JavaScript library that can be summarized as a toolkit for functional programming. Functional programming can be considered a programming paradigm, it revolves around using descriptive functions in a sequence rather than separate examples. **Object-oriented programming (OOP)** is also a programming paradigm, which is all about objects and their state, and the data can be encapsulated and hidden from the outside code. In functional programming the functions are very important, but there is less state to be concerned about. These functions do the same thing with different arguments all the time, and they can be easily chained.

The Underscore library offers a lot of functions for everyday programming, such as `map`, `filter`, `invoke`, and functions for testing. Here is a little code snippet showing some Underscore, which makes an alert pop-up box for all the items in the array—in this case, it is making a pop-up for 1, 2, and 3:

```

<!DOCTYPE html>
<html>

<head>
  <script src="https://cdn.jsdelivr.net/npm/underscore@1.13.1/underscore-umd-min.js"></script>

```

```

</head>

<body>
  <script>
    _.each([1, 2, 3], alert);
  </script>
</body>

</html>

```

There are many other functions for filtering, grouping elements, transforming elements, getting a random value, getting the current time, and a lot more.

This snippet probably explains the name as well, since we access Underscore functions using an `underscore`. You will have to install Underscore first though, else the interpreter won't understand the syntax.



At the time of writing, you can find the full documentation here: <https://devdocs.io/underscore/>.

React

React is the last frontend library we are going to discuss. If you would rather say React is a framework you are not completely wrong, but not right either. The reason that we consider React a library is that you'll need to use some other libraries to get to the point where it feels like a framework.

React is used to build beautiful and dynamic user interfaces. It splits up pages into different components and the data gets sent and updated between components as it changes. Here is a very basic example that only scratches the very surface of what React can do. This HTML will give this sentence on the page: **Hi Emile, what's up?**:

```

<div id="root"></div>

```

It will do this when the following JavaScript is associated with it:

```

ReactDOM.render(
  <p> Hi Emile, what's up?</p>,
  document.getElementById('root');
);

```

This will only work when the React library is available. And it will render the DOM, replacing the `innerHTML` of the `div` with the first argument of the `render` function. We can do this by adding React in a `script` element in the header and not installing anything on our system. The completed script looks like this:

```
<!DOCTYPE html>
<html>

<head>
  <script src="https://unpkg.com/react@17/umd/react.development.js"
    crossorigin></script>
  <script src="https://unpkg.com/react-dom@17/umd/react-dom.
    development.js" crossorigin></script>
</head>

<body>
  <div id="root"></div>
  <script>
    let p = React.createElement("p", null, "Hi Emile, what's up?");
    ReactDOM.render(
      p,
      document.getElementById("root");
    );
  </script>
</body>
</html>
```

This will write **Hi Emile, what's up?** to the page using React elements created manually in the `script` tag. This is not something you should be doing for large projects though. It is way more valuable to set up React and everything you need using a package manager such as **Node Package Manager (NPM)**. This will allow you to easily manage all the dependencies and keep your code organized.



At the time of writing, more can be found here: <https://reactjs.org/docs/getting-started.html>.

Frameworks

The frameworks are more complex and usually you'll have to install them on your computer. How to do this can be found in the online documentation of the specific framework. And whenever you are done coding and you want to run your code, you'll have to run a command that will process your code into something the browser will understand. We are "serving" the application when we do this.

Vue.js

Vue.js is a lightweight JavaScript framework. It can be used to build user interfaces and **single-page applications (SPAs)**. The way user interfaces are written with Vue.js can be hard to get your head around the first time you encounter it. Have a look at this code sample:

```
<!DOCTYPE html>
<html>
  <script src="https://cdn.jsdelivr.net/npm/vue"></script>

  <body>
    <div id="app">
      <p v-if="!hide">
        Let's play hide and seek. <br />
        Go to the console and type: <br />
        obj._data.hide = true <br />
      </p>
    </div>

    <script>
      let obj = new Vue({
        el: "#app",
        data: {
          hide: false,
        },
      });
    </script>
  </body>
</html>
```

This is a simple HTML page, importing a JavaScript link from Vue. There is something weird going on in the HTML of the `<p>` tag: there is a `v-if` element. This element will only be displayed when the condition in that `v-if` is true.

In this case, it is looking at the `hide` property of our data object in our Vue instance. If you change the value of this `hide` to `true`, the negated `hide` statement will become `false`, and the element will disappear. This is something that we could have done without Vue as well, but we would then have specified a JavaScript event for the change of the value and used JavaScript to edit the CSS to hide the paragraph.

You can even see HTML elements that are new to you. That is because these are not regular HTML elements, but rather from Vue, which lets you define your own elements. You can run into HTML that looks like this:

```
<div id="custom-component">
  <maaike></maaike>
</div>
```

And when you open the webpage associated with it, it shows:

```
Maaike says: good job!
```

This is not because HTML knows how to do that. This is because there is a snippet that defines the `maaike` component. Here is the snippet:

```
<script>
  Vue.component("maaike", {
    template: "<p>Maaike says: good job!</p>",
  });
  new Vue({ el: "#app" });
</script>
```

In the preceding code, a new Vue component is created, and it can actually hold data and have a function too, but this one is very basic and just to illustrate we can add HTML templates in the `template` property. There is a paragraph specified. When the webpage gets loaded, the `<maaike>` component will be replaced with whatever is in the template.

The content of one page can come from many files. Usually these components all have their own file. There is a lot more official Vue tooling that you will get to know once you dive into Vue.js. It is actually a great framework for beginners with frameworks, as it is rather clear what is going on and is a great starting point for comprehending frameworks in general.



At the time of writing, you can find the full Vue docs here:
<https://v3.vuejs.org/guide/introduction.html>.

Angular

Angular is a framework that originates from and is (currently) maintained by Google. Angular is a lot heavier than Vue.js, but it can be considered a complete package. This means that Angular takes up more disk space, and more disk space usually means it is slower to compile and install. Looking at Angular code isn't really that much different from Vue.js. However, Angular uses TypeScript instead of JavaScript. TypeScript is a superset of JavaScript and gets transpiled to JavaScript, but it is stricter and has a different syntax as well.

Angular can be recognized by the `ng` attributes in the HTML. We are not going to show a full example, but here is the HTML that will show all the tasks on a to-do list (when the code around it is set correctly):

```
<ul>
  <li ng-repeat="task in tasks">
    {{task}} <span ng-click="deleteTask($index)">Done</span>
  </li>
</ul>
```

The `ng-repeat` attribute is specifying the repeat action that for every task on the task list, it should create a `<li>` element. And `task` can be used as a variable inside `<li>` as well, as indicated by `{{ task }}`.

There's one more Angular-specific thing going on, `ng-click`, which tells Angular what to do when an element gets clicked. This is similar to the `onClick` event of JavaScript, but it can now be dynamically binded. This means that when writing the code, you don't need to know about `onClick` yet. Clearly, you can achieve the same thing in JavaScript by specifying events that will lead to changes of the `onClick` attribute (and the complete element if necessary), but this is a lot more code that needs to be written. This goes for anything in Angular: it can be done with just JavaScript but it is a lot more work (and that might actually be an understatement, depending on the complexity of the situation).



At the time of writing, you can find the full docs here: <https://angular.io/docs>.

Learning to work with libraries and frameworks such as React, Angular, or Vue is a very logical and even must-have next step if you seek to be a frontend developer. In the authors' view, the difficulty of these options doesn't really differ that much. Which one is the best choice depends on the place you want to work and the region that you are in, because there are regional preferences for these frameworks and libraries.

Learning the backend

So far, we have only been dealing with the frontend. The frontend is the part that is running on the client side, which could be any device that the user is using, such as a phone, laptop, or tablet. In order for websites to do interesting stuff, we also need a backend. For example, if you want to log on to a website, this website somehow needs to know whether this user exists.

This is the job of the server-side code, the backend. This is code that is running not on the device of the user, but on some sort of server elsewhere, which is often owned or leased by the company hosting the website. Hosting the website usually means that they make it available to the world wide web by placing it on a server that can take outside requests via a URL.

The code on the server does many things, all related to deeper logic and data. For example, an e-commerce store has a bunch of items in the shop that come from a database. The server gets the items from the database, parsing the HTML template and sending the HTML, CSS, and JavaScript over to the client.

The same goes for logging in: when you enter your username and password on a website and you click on login, the code on the server gets triggered. This code is going to verify the details you entered with those in the database. If you have the correct details, it will send you back the page of your portal for logged-in users. And if you have entered incorrect details, it will send back the error to the client.

In this section, we will cover the basics of communication between the frontend and backend, and we will show you how you can use JavaScript to write backend code as well using Node.js.

APIs

An **API (Application Programming Interface)** is essentially an interface for code, written with more code. A request can be made to an API using (for example) a URL. This will trigger a certain piece of code and this piece of code will give a certain response back.

This is all very abstract, so let's use an example. If we had a website for a hotel, it would make sense for people to be able to make bookings online. This would require us to have some sort of API. Whenever a user has filled out all the fields and clicks on **Submit booking**, the API will get triggered by calling the URL and sending all the data that the user has entered to that endpoint (a specific URL), for example: `www.api.hotelname.com/rooms/book`. This API will process and validate our data and when everything is fine, it will store the room booking in our database and probably send a confirmation mail to our guest.

Whenever one of the hotel clerks goes to check out the reservations, another API call will be made using one of the endpoints. It could be an endpoint that looks like this for example: `www.api.hotelname.com/reservations`. This would first go ahead and check whether our employee is logged in with the right role, and if so, it will fetch all the reservations for the selected date range from the database and send the page with the results back to our employee, who can then see all the bookings. So APIs are the connection points between the logic, database, and frontend.

APIs work with **Hypertext Transfer Protocol (HTTP)** calls. HTTP is just a protocol for communication between two parties: a client and a server, or a server and another server (in which the requesting server acts like the client). This means that it has to stick to certain conventions and rules that the other party expects, and the other party will respond in a certain way. For example, this means using a specific format to specify headers, using GET methods for getting information, using POST methods for creating new information on the server, and using PUT methods to change information on the server.



There can be more done with APIs, for example, your computer and printer communicate via an API as well. However, this is not too relevant from a JavaScript point of view.

You will see how to consume these APIs in the AJAX section. You can also write your own APIs, and the ultimate basics of how to do this can be found in the *Node.js* section.

AJAX

AJAX stands for **Asynchronous JavaScript and XML**, which is a misnomer, because nowadays it is more common to use JSON instead of XML. We use it to make calls from the frontend to the backend, without refreshing the page (asynchronously). AJAX is not a programming language or a library, it is a combination of the built-in XMLHttpRequest object in the browser and the JavaScript language.

You probably won't be using plain AJAX in your day-to-day life as a frontend developer nowadays, but it is being used beneath the surface so it won't hurt to know how it works. Here is an example of calling the backend using AJAX:

```
let xhttp = new XMLHttpRequest();
let url = "some valid url";
xhttp.load = function () {
  if (this.status == 200 && this.readyState == 4) {
    document.getElementById("content").innerHTML = this.responseText;
  }
};
xhttp.open("GET", url, true);
xhttp.send();
```

This is not a working example, because there is no valid URL, but it demonstrates how AJAX works. It sets up what it needs to do when the request has been loaded, in this case, replacing the HTML inside the element with ID content with whatever the link returns. This could be a link to a file, or to some API that calls a database. It can give different responses when there is other (or no) data in the database. This response is in JSON, but it could also be in XML. This depends on how the server was coded.

More common now is the use of the **Fetch API** for AJAX requests. This is similar to what we can do with XMLHttpRequest but it provides a more flexible and powerful set of features. For example, in the following code we get the data from the URL, convert it to JSON with the `json()` method, and output it to the console:

```
let url = "some valid url";
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data));
```

The Fetch API works with promises, which should look familiar at this point. So after the promise is resolved, a new one gets created with `then`, and when that one is resolved, the next `then` gets executed.



At the time of writing, more information can be found here:
https://developer.mozilla.org/en-US/docs/Web/Guide/AJAX/Getting_Started.

Practice exercise 15.1

Create a JSON file and using `fetch`, return the results as a usable object into your JavaScript code:

1. Create a JSON object and save it in a file called `list.json`.
2. Using JavaScript, assign the filename and path to a variable named `url`.
3. Using `fetch`, make the request to the file URL. Return the results as JSON.
4. Once the response object is ready, iterate through the data and output the results into the console of each item in the JSON file.

Node.js

We can write APIs in JavaScript using Node.js. Node.js is a very clever runtime environment that has taken the Google JavaScript engine, expanded it, and made it possible to run JavaScript on servers, interacting with the filesystem protocol and HTTP using JavaScript. Because of this, we can use JavaScript for the backend. This means that you can write both the backend and the frontend with only one language (along with HTML and CSS). Without Node.js, you would have to use another language such as PHP, Java, or C# for the backend.

In order to run Node.js, you first have to set it up and then run the `node filename.js` command. You can find out how to set it up on your system in the official Node.js documentation. Often it requires downloading and installing something, and then you are done.



At the time of writing, the download instructions are accessible at
<https://nodejs.org/en/download/>.

Here is an example of some code that will receive HTTP calls that can be written for Node.js:

```
const http = require("http");

http.createServer(function(req, res){
  res.writeHead(200, {'Content-Type': 'text/html'}); //header status
  let name = "Rob";
  res.write('Finally, hello ${name}'); //body
  res.end();
}).listen(8080); //listen to port 8080

console.log("Listening on port 8080... ");
```

We start by importing the `http` module. This is an external code file that needs to be imported in order to run. The `http` module comes with Node.js, but other modules might need to be installed. You will use a package manager for this, such as NPM, which will help to install all the dependencies and be able to manage all the different versions of the external modules.

The code above sets up a server that is listening to port `8080`, and whenever it gets accessed, it will return `Finally, hello Rob`. We create the server with the `createServer` method on the imported `http` module. We then say what needs to happen for a call to our server. We respond with a `200` status (indicating "OK") and write `Finally, hello Rob` to the response. We then specify the default port `8080` as the listening port.

This example uses the built-in `http` module for Node.js, which is very powerful for creating APIs. This is definitely something that's worth having some experience with. Being able to write your own APIs will enable you to write full applications yourself. This gets even easier when we add Express to the mix.

Using the Express Node.js framework

Node.js is not a framework, nor a library. It is a runtime environment. This means that it can run and interpret the JavaScript code written. There are frameworks for Node.js and currently Express is the most popular one.

Here is a very basic Express application – again, you will have to set up Node.js first, then add the Express module (if you are using NPM, `npm install express` will do) and run it using the `node nameOfRootFile.js` command:

```
const express = require('express');
const app = express();

app.get('/', (request, response) => {
```

```
response.send('Hello Express!');
});

app.listen(3000, () => {
  console.log('Express app at http://localhost:3000');
});
```

After running this and going to `localhost:3000` (assuming you are running it on `localhost`), you will get the message **Hello Express!** in your browser. In the terminal where you are running your Node app, it will print the console log message after loading.



You can find more in the Node.js documentation, which at the time of writing is at the following address: <https://nodejs.org/en/docs/>.

For the Express module, you can go to <https://expressjs.com/en/5x/api.html>.

Next steps

You have learned a lot about JavaScript in this book and with this chapter you should have an idea of possible next steps you can take. This chapter hasn't taught you all of these topics in depth, as entire books could be (and have been) written about each of them, but you should have a good idea of where to look for your next steps and what to consider while deciding which next step to take.

The best way to learn is by doing. So we highly recommend you just come up with a fun project idea and go ahead and try to make it. Alternatively, with this knowledge, you may feel ready for an entry-level JavaScript position! You can also do tutorials online or even work in a project team as a junior, using freelance platforms such as Upwork or Fiverr to get a project. These are hard to find though, and we can imagine that you'll learn a framework or get a bit more experience with Node.js first. However, this is often possible on the job if you can show your skills and potential in the hiring process.

Chapter projects

Working with JSON

Create a JSON file locally, connect to the JSON and data, and output the data from the JSON file into your console:

1. Create a file with the extension JSON name it `people.json`.
2. Within `people.json` create an array that contains multiple objects. Each item in the array should be an object that has the same structure, using `first`, `last`, and `topic` as the property names. Make sure you use double quotes around the property names and values as this is the proper JSON syntax.
3. Add three or more entries into the array using the same object structure for each item.
4. Create an HTML file and add a JavaScript file. Within the JavaScript file use `people.json` as the URL. Using `fetch` connect to the URL and retrieve the data. Since this is a JSON-formatted file, once you get the response data it can be formatted into JSON using the `.json()` method in `fetch`.
5. Output the full contents of the data into the console.
6. Using `foreach`, loop through the items in the data and output the values into the console. You can use a template literal and output each value.

List-making project

Create a list that saves to local storage so even if the page is refreshed, the data will persist within the browser. If the local storage is empty on the first load of the page, set up a JSON file that will be loaded to the local storage and saved as a default list to start the list:

1. Set up an HTML file, adding a `div` to output the list results to, and an input field with a button that can be clicked.
2. Using JavaScript, add the page elements as objects that can be used within the code.

3. Create your default JSON file (which can be empty) and add the path to the file into your JavaScript code using a variable called `url`.
4. Add an event listener to the button element that will run a function called `addToList()`.
5. In `addToList()`, check if the value of the input field has a length of 3 or more. If it does then create an object with a name and the value of the input field. Create a global variable named `myList` to hold the list, and within `addToList()` push the new object data into `myList`.
6. Create a function called `maker()`, which will create the page element and add text into the element, appending it to the output element. Invoke `maker()` to add the new item within the `addToList()` function.
7. Also, save the item to local storage so that the visual content of `myList` will be synced with the local storage saved value. To do this, create a function called `saveToStorage()` and invoke it every time you update the `myList` in the script.
8. Within the `saveToStorage()` function, set the value of `myList` into `localStorage` using `setItem`. You will need to convert `myList` into a string value to save it into `localStorage`.
9. Add to the code `getItem()` to retrieve a value of `myList` from `localStorage`. Set up a global variable for the `myList` array.
10. Add an event listener to listen for `DOMContentLoaded`. Within that function, check if the `localStorage` loaded a value. If it did then get `myList` from local storage and convert it from a string to a JavaScript object. Clear the output element contents. Loop through the items in `myList` and add them to the page with the `maker()` function created earlier.
11. If `localStorage` does not have content, load the JSON file with the default values using `fetch`. Once the data is loaded, assign it to the global `myList` value. Loop through the items in `myList` and using `maker()`, output them to the page. Don't forget to invoke `saveToStorage()` after so that the storage will contain the same list items as are visible on the page.

Self-check quiz

1. What are JavaScript libraries and frameworks?
2. How can you tell if a webpage is using the jQuery library?
3. Which library contains a lot of functionality for manipulating data?
4. When Node.js is installed, how can you run a Node.js file?

Summary

In this chapter, we have explored a few of the possibilities to continue your JavaScript journey and keep on improving yourself. We started off by discussing the frontend and what libraries and frameworks are. Libraries and frameworks are both pre-made code that you can use in your project, but libraries typically solve one problem while frameworks provide a standard solution that usually controls the way you structure your application and will come with some limitations. On the other hand, frameworks are great fits for very many things you might want to do with your web apps.

We then moved on to looking at the backend. The backend is the code that runs on the server and we can write this code in JavaScript when we use Node.js. Node.js is a runtime engine that can process JavaScript and has some additional features for JavaScript that we don't have when using JavaScript in the browser.

And that's it. You have a very solid understanding of JavaScript at this point. You have seen all the major building blocks, and had a lot of practice with smaller exercises and bigger projects. There are a few things for certain: you'll never be done with learning as a JavaScript programmer, and you will keep on amazing yourself with the things you can make as you keep on progressing.

Don't forget to have fun!

Appendix – Practice Exercise, Project, and Self-Check Quiz Answers

Chapter 1, Getting Started with JavaScript

Practice exercises

Practice exercise 1.1

```
4 + 10
14

console.log("Laurence");
Laurence
undefined
```

Practice exercise 1.2

```
<!DOCTYPE html>
<html>

<head>
  <title>Tester</title>
</head>

<body>
```

Appendix

```
<script>
  console.log("hello world");
</script>
</body>

</html>
```

Practice exercise 1.3

```
<!DOCTYPE html>
<html>

<head>
  <title>Tester</title>
</head>

<body>
  <script src="app.js"></script>
</body>

</html>
```

Practice exercise 1.4

```
let a = 10; // assign a value of 10 to variable a
console.log(a); // This will output 10 into the console
/*
This is a multi-line
Comment
*/
```

Projects

Creating an HTML file and a linked JavaScript file

```
<!doctype html>
<html>
  <head>
    <title>JS Tester</title>
  </head>
```

```
<body>
<script src="myJS.js"></script>
</body>
</html>

// console.log("Laurence");
/*
This is my comment
Laurence Svektis
*/
```

Self-check quiz

1. <script src="myJS.js"></script>.
2. No.
3. By opening and closing it with /* and */.
4. Comment out the line with //.

Chapter 2, JavaScript Essentials

Practice exercises

Practice exercise 2.1

```
console.log(typeof(str1));
console.log(typeof(str2));
console.log(typeof(val1));
console.log(typeof(val2));
console.log(typeof(myNum));
```

Practice exercise 2.2

```
const myName = "Maaike";
const myAge = 29;
const coder = true;
const message = "Hello, my name is " + myName + ", I am " + myAge +
years old and I can code JavaScript: " + coder + ".";
console.log(message);
```

Practice exercise 2.3

```
let a = window.prompt("Value 1?");
let b = window.prompt("Value 2?");
a = Number(a);
b = Number(b);
let hypotenuseVal = ((a * a) + (b * b))**0.5;
console.log(hypotenuseVal);
```

Practice exercise 2.4

```
let a = 4;
let b = 11;
let c = 21;
a = a + b;
a = a / c;
c = c % b;
console.log(a, b, c);
```

Projects

Miles-to-kilometers converter

```
//Convert miles to kilometers.
//1 mile equals 1.60934 kilometers.
let myDistanceMiles = 130;
let myDistanceKM = myDistanceMiles * 1.60934;
console.log("The distance of " + myDistanceMiles + " miles is equal to "
+ myDistanceKM + " kilometers");
```

BMI calculator

```
//1 inch = 2.54 centimetres.
//2.2046 pounds in a kilo
let inches = 72;
let pounds = 180;
let weight = pounds / 2.2046; // in kilos
let height = inches * 2.54; // height in centimetres
console.log(weight, height);
let bmi = weight/(height/100*height/100);
console.log(bmi);
```

Self-check quiz

1. String
2. Number
3. Line 2
4. world
5. Hello world!
6. Whatever the user enters in
7. 71
8. 4
9. 16 and 536
10. true
false
true
true
false

Chapter 3, JavaScript Multiple Values

Practice exercises

Practice exercise 3.1

```
const myList = ["Milk", "Bread", "Apples"];
console.log(myList.length);
myList[1] = "Bananas";
console.log(myList);
```

Practice exercise 3.2

```
const myList = [];
myList.push("Milk", "Bread", "Apples");
myList.splice(1, 1, "Bananas", "Eggs");
const removeLast = myList.pop();
console.log(removeLast);
```

```
myList.sort();
console.log(myList.indexOf("Milk"));
myList.splice(1, 0, "Carrots", "Lettuce");
const myList2 = ["Juice", "Pop"];
const finalList = myList.concat(myList2, myList2);
console.log(finalList.lastIndexOf("Pop"));
console.log(finalList);
```

Practice exercise 3.3

```
const myArr = [1, 2, 3];
const bigArr = [myArr, myArr, myArr];
console.log(bigArr[1][1]);
console.log(bigArr[0][1]);
console.log(bigArr[2][1]);
```

Practice exercise 3.4

```
const myCar = {
  make: "Toyota",
  model: "Camry",
  tires: 4,
  doors: 4,
  color: "blue",
  forSale: false
};

let propColor = "color";
myCar[propColor] = "red";
propColor = "forSale";
myCar[propColor] = true;
console.log(myCar.make + " " + myCar.model);
console.log(myCar.forSale);
```

Practice exercise 3.5

```
const people = {friends: []};
const friend1 = {first: "Laurence", last: "Svekis", id: 1};
const friend2 = {first: "Jane", last: "Doe", id: 2};
const friend3 = {first: "John", last: "Doe", id: 3};
people.friends.push(friend1, friend2, friend3);
console.log(people);
```

Projects

Manipulating an array

```
theList.pop();
theList.shift();
theList.unshift("FIRST");
theList[3] = "hello World";
theList[2] = "MIDDLE";
theList.push("LAST");
console.log(theList);
```

Company product catalog

```
const inventory = [];
const item3 = {
  name: "computer",
  model: "imac",
  cost: 1000,
  qty: 3
}
const item2 = {
  name: "phone",
  model: "android",
  cost: 500,
  qty: 11
}
const item1 = {
  name: "tablet",
  model: "ipad",
  cost: 650,
  qty: 1
}
inventory.push(item1, item2, item3);
console.log(inventory);
console.log(inventory[2].qty);
```

Self-check quiz

1. Yes. You can reassign values within an array declared with `const`, but cannot redeclare the array itself.
2. Length
3. The outputs are as follows:

```
-1
1
```
4. You can do the following:

```
const myArr = [1,2,5,6,8,9,15];
myArr.splice(1,1,4);
console.log(myArr);
```
5. The output is as follows:

```
[empty × 10, "test"]
undefined
```
6. The output is as follows:

```
undefined
```

Chapter 4, Logic Statements

Practice exercises

Practice exercise 4.1

```
const test = false;
console.log(test);
if(test){
  console.log("It's True");
}
if(!test){
  console.log("False now");
}
```

Practice exercise 4.2

```
let age = prompt("How old are you?");
age = Number(age);
let message;
if(age >= 21){
  message = "You can enter and drink.";
}else if(age >= 19){
  message = "You can enter but not drink.";
}else{
  message = "You are not allowed in!";
}
console.log(message);
```

Practice exercise 4.3

```
const id = true;
const message = (id) ? "Allowed In" : "Denied Entry";
console.log(message);
```

Practice exercise 4.4

```
const randomNumber = Math.floor(Math.random() * 6);
let answer = "Something went wrong";
let question = prompt("Ask me anything");
switch (randomNumber) {
  case 0:
    answer = "It will work out";
    break;
  case 1:
    answer = "Maybe, maybe not";
    break;
  case 2:
    answer = "Probably not";
    break;
  case 3:
    answer = "Highly likely";
    break;
  default:
```

```
    answer = "I don't know about that";
  }
  let output = "You asked me " + question + ". I think that " + answer;
  console.log(output);
```

Practice exercise 4.5

```
let prize = prompt("Pick a number 0-10");
prize = Number(prize);
let output = "My Selection: ";
switch (prize){
  case 0:
    output += "Gold ";
  case 1:
    output += "Coin ";
    break;
  case 2:
    output += "Big ";
  case 3:
    output += "Box of ";
  case 4:
    output += "Silver ";
  case 5:
    output += "Bricks ";
    break;
  default:
    output += "Sorry Try Again";
}
console.log(output);
```

Projects**Evaluating a number game answers**

```
let val = prompt("What number?");
val = Number(val);
let num = 100;
let message = "nothing";
if (val > num) {
  message = val + " was greater than " + num;
} else if (val === num) {
```

```

    message = val + " was equal to " + num;
  } else {
    message = val + " is less than " + num;
  }
  console.log(message);
  console.log(message);

```

Friend checker game answers

```

let person = prompt("Enter a name");
let message;
switch (person) {
  case "John" :
  case "Larry" :
  case "Jane" :
  case "Laurence" :
    message = person + " is my friend";
    break;
  default :
    message = "I don't know " + person;
}
console.log(message);

```

Rock paper scissors game answers

```

const myArr = ["Rock", "Paper", "Scissors"];
let computer = Math.floor(Math.random() * 3);
let player = Math.floor(Math.random() * 3);
let message = "Player " + myArr[player] + " vs computer " +
myArr[computer] + " ";
if (player === computer) {
  message += "it's a tie";
} else if (player > computer) {
  if (computer == 0 && player == 2) {
    message += "Computer Wins";
  } else {
    message += "Player Wins";
  }
} else {
  if (computer == 2 && player == 0) {
    message += "Player Wins";
  } else {

```

```

    message += "Computer Wins";
  }
  console.log(message);
}

```

Self-check quiz

- one
- this is the one
- login
- Welcome, that is a user: John
- Wake up, it's morning
- Result:
 - true
 - false
 - true
 - true
- Result:

```

100 was LESS or Equal to 100
100 is Even

```

Chapter 5, Loops

Practice exercises

Practice exercise 5.1

```

const max = 5;
const ranNumber = Math.floor(Math.random() * max) + 1;
//console.log(ranNumber);
let correct = false;
while (!correct) {
  let guess = prompt("Guess a Number 1 - " + max);
  guess = Number(guess);
  if (guess === ranNumber) {

```

```

    correct = true;
    console.log("You got it " + ranNumber);
  } else if (guess > ranNumber) {
    console.log("Too high");
  } else {
    console.log("Too Low");
  }
}

```

Practice exercise 5.2

```

let counter = 0;
let step = 5;
do {
  console.log(counter);
  counter += step;
} while (counter <= 100);

```

Practice exercise 5.3

```

const myWork = [];
for (let x = 1; x < 10; x++) {
  let stat = x % 2 ? true : false;
  let temp = {
    name: `Lesson ${x}`, status: stat
  };
  myWork.push(temp);
}
console.log(myWork);

```

Practice exercise 5.4

```

const myTable = [];
const rows = 4;
const cols = 7;
let counter = 0;
for (let y = 0; y < rows; y++) {
  let tempTable = [];
  for (let x = 0; x < cols; x++) {
    counter++;
  }
}

```

```

tempTable.push(counter);
}
myTable.push(tempTable);
}
console.table(myTable);

```

[[index]]	0	1	2	3	4	5	6
0	1	2	3	4	5	6	7
1	8	9	10	11	12	13	14
2	15	16	17	18	19	20	21
3	22	23	24	25	26	27	28

▼ Array(4) 0
 • 0: (7) [1, 2, 3, 4, 5, 6, 7]
 • 1: (7) [8, 9, 10, 11, 12, 13, 14]
 • 2: (7) [15, 16, 17, 18, 19, 20, 21]
 • 3: (7) [22, 23, 24, 25, 26, 27, 28]
 length: 4
 • [[Prototype]]: Array(0)
 console.log

Practice exercise 5.5

```

const grid = [];
const cells = 64;
let counter = 0;
let row;
for (let x = 0; x < cells + 1; x++) {
  if (counter % 8 == 0) {
    if (row != undefined) {
      grid.push(row);
    }
    row = [];
  }
  counter++;
  let temp = counter;
  row.push(temp);
}
console.table(grid);

```


[Index]	0	1	2	3	4	5	6	7
0	1	2	3	4	5	6	7	8
1	9	10	11	12	13	14	15	16
2	17	18	19	20	21	22	23	24
3	25	26	27	28	29	30	31	32
4	33	34	35	36	37	38	39	40
5	41	42	43	44	45	46	47	48
6	49	50	51	52	53	54	55	56
7	57	58	59	60	61	62	63	64

Practice exercise 5.6

```
const myArray = [];
for (let x = 0; x < 10; x++) {
  myArray.push(x + 1);
}
console.log(myArray);

for (let i = 0; i < myArray.length; i++) {
  console.log(myArray[i]);
}

for (let val of myArray) {
  console.log(val);
}
```

Practice exercise 5.7

```
const obj = {
  a: 1,
  b: 2,
  c: 3
};
```

```
for (let prop in obj) {
  console.log(prop, obj[prop]);
}

const arr = ["a", "b", "c"];
for (let w = 0; w < arr.length; w++) {
  console.log(w, arr[w]);
}

for (el in arr) {
  console.log(el, arr[el]);
}
```

Practice exercise 5.8

```
let output = "";
let skipThis = 7;
for (let i = 0; i < 10; i++) {
  if (i === skipThis) {
    continue;
  }
  output += i;
}
console.log(output);
```

Alternatively, the following code could be used, replacing `continue` with `break`:

```
let output = "";
let skipThis = 7;
for (let i = 0; i < 10; i++) {
  if (i === skipThis) {
    break;
  }
  output += i;
}
console.log(output);
```

Project

Math multiplication table

```
const myTable = [];  
const numm = 10;  
for(let x=0; x<numm; x++){  
  const temp = [];  
  for(let y = 0; y<numm; y++){  
    temp.push(x*y);  
  }  
  myTable.push(temp);  
}  
  
console.table(myTable);
```

(index)	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7	8	9
2	0	2	4	6	8	10	12	14	16	18
3	0	3	6	9	12	15	18	21	24	27
4	0	4	8	12	16	20	24	28	32	36
5	0	5	10	15	20	25	30	35	40	45
6	0	6	12	18	24	30	36	42	48	54
7	0	7	14	21	28	35	42	49	56	63
8	0	8	16	24	32	40	48	56	64	72
9	0	9	18	27	36	45	54	63	72	81

*Array(10) 0
*0: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
*1: [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
*2: [0, 3, 6, 9, 12, 15, 18, 21, 24, 27]
*3: [0, 4, 8, 12, 16, 20, 24, 28, 32, 36]
*4: [0, 5, 10, 15, 20, 25, 30, 35, 40, 45]
*5: [0, 6, 12, 18, 24, 30, 36, 42, 48, 54]
*6: [0, 7, 14, 21, 28, 35, 42, 49, 56, 63]
*7: [0, 8, 16, 24, 32, 40, 48, 56, 64, 72]
*8: [0, 9, 18, 27, 36, 45, 54, 63, 72, 81]
length: 10
__proto__: Array(10)
= undefined

Self-check quiz

1. Result:

0
3
6
9

2. Result:

0
5
1
6
2
7
[1, 5, 7]

Chapter 6, Functions

Practice exercises

Practice exercise 6.1

```
function adder(a, b) {  
  return a + b;  
}  
  
const val1 = 10;  
const val2 = 20;  
console.log(adder(val1, val2));  
console.log(adder(20, 30));
```

Practice exercise 6.2

```
const adj = ["super", "wonderful", "bad", "angry", "careful"];  
  
function myFun() {  
  const question = prompt("What is your name?");  
  const nameAdj = Math.floor(Math.random() * adj.length);  
  console.log(adj[nameAdj] + " " + question );  
}
```

```

}
myFun();

```

Practice exercise 6.3

```

const val1 = 10;
const val2 = 5;
let operat = "-";
function cal(a, b, op) {
  if (op == "-") {
    console.log(a - b);
  } else {
    console.log(a + b);
  }
}
cal(val1, val2, operat);

```

Practice exercise 6.4

```

const myArr = [];

for(let x=0; x<10; x++){
  let val1 = 5 * x;
  let val2 = x * x;
  let res = cal(val1, val2, "+");
  myArr.push(res);
}

console.log(myArr);
function cal(a, b, op) {
  if (op == "-") {
    return a - b;
  } else {
    return a + b;
  }
}

```

Practice exercise 6.5

```

let val = "1000";

(function () {
  let val = "100"; // Local scope variable

```

```

    console.log(val);
  })();

  let result = (function () {
    let val = "Laurence";
    return val;
  })();
  console.log(result);
  console.log(val);

  (function (val) {
    console.log(`My name is ${val}`);
  })("Laurence");

```

Practice exercise 6.6

```

function calcFactorial(nr) {
  console.log(nr);
  if (nr === 0) {
    return 1;
  }
  else {
    return nr * calcFactorial(--nr);
  }
}
console.log(calcFactorial(4));

```

Practice exercise 6.7

```

let start = 10;
function loop1(val) {
  console.log(val);
  if (val < 1) {
    return;
  }
  return loop1(val - 1);
}
loop1(start);
function loop2(val) {
  console.log(val);
  if (val > 0) {

```

```

    val--;
    return loop2(val);
  }
  return;
}
loop2(start);

```

Practice exercise 6.8

```

const test = function(val){
  console.log(val);
}
test('hello 1');

function test1(val){
  console.log(val);
}
test1("hello 2");

```

Projects

Create a recursive function

```

const main = function counter(i) {
  console.log(i);
  if (i < 10) {
    return counter(i + 1);
  }
  return;
}
main(0);

```

Set timeout order

```

const one = () => console.log('one');
const two = () => console.log('two');
const three = () => {
  console.log('three');
  one();
  two();
}

```

```

const four = () => {
  console.log('four');
  setTimeout(one, 0);
  three();
}
four();

```

Self-check quiz

- 10
- Hello
- Answer:
Welcome
Laurence
My Name is Laurence
- 19
- 16

Chapter 7, Classes

Practice exercises

Practice exercise 7.1

```

class Person {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }
}

let person1 = new Person("Maaike", "van Putten");
let person2 = new Person("Laurence", "Sveakis");
console.log("hello " + person1.firstname);
console.log("hello " + person2.firstname);

```

Practice exercise 7.2

```
class Person {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }
  fullname(){
    return this.firstname + " " + this.lastname;
  }
}
let person1 = new Person("Masike", "van Putten");
let person2 = new Person("Laurence", "Svekis");
console.log(person1.fullname());
console.log(person2.fullname());
```

Practice exercise 7.3

```
class Animal {
  constructor(species, sounds) {
    this.species = species;
    this.sounds = sounds;
  }
  speak() {
    console.log(this.species + " " + this.sounds);
  }
}
Animal.prototype.eat = function () {
  return this.species + " is eating";
}
let cat = new Animal("cat", "meow");
let dog = new Animal("dog", "bark");
cat.speak();
console.log(dog.eat());
console.log(dog);
```

Projects**Employee tracking app**

```
class Employee {
  constructor(first, last, years) {
    this.first = first;
    this.last = last;
    this.years = years;
  }
}
const person1 = new Employee("Laurence", "Svekis", 10);
const person2 = new Employee("Jane", "Doe", 5);
const workers = [person1, person2];

Employee.prototype.details = function(){
  return this.first + " " + this.last + " has worked here " +
    this.years + " years";
}

workers.forEach((person) => {
  console.log(person.details());
});
```

Menu items price calculator

```
class Menu {
  #offer1 = 10;
  #offer2 = 20;
  constructor(val1, val2) {
    this.val1 = val1;
    this.val2 = val2;
  }
  calTotal(){
    return (this.val1 * this.#offer1) + (this.val2 * this.#offer2);
  }
  get total(){
    return this.calTotal();
  }
}
```

```
const val1 = new Menu(2,0);
const val2 = new Menu(1,3);
const val3 = new Menu(3,2);
console.log(val1.total);
console.log(val2.total);
console.log(val3.total);
```

Self-check quiz

1. class
2. Using the following syntax:

```
class Person {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }
}
```

3. Inheritance
4. Answers:
 - True
 - False
 - True
 - True
 - False
5. B

Chapter 8, Built-In JavaScript Methods

Practice exercises

Practice exercise 8.1

```
const secretMes1 = "How's%20it%20going%3F";
const secretMes2 = "How's it going?";
const decodedComp = decodeURIComponent(secretMes1);
console.log(decodedComp);
```

```
const encodedComp = encodeURIComponent(secretMes2);
console.log(encodedComp);
const uri = "http://www.basescripts.com?Hello World";
const encoded = encodeURIComponent(uri);
console.log(encoded);
```

Practice exercise 8.2

```
const arr = ["Laurence", "Mike", "Larry", "Kim", "Joanne", "Laurence",
"Mike", "Laurence", "Mike", "Laurence", "Mike"];
const arr2 = arr.filter( (value, index, array) => {
  console.log(value, index, array.indexOf(value));
  return array.indexOf(value) === index;
});
console.log(arr2);
```

Practice exercise 8.3

```
const myArr = [1,4,5,6];
const myArr1 = myArr.map(function(ele){
  return ele * 2;
});
console.log(myArr1);

const myArr2 = myArr.map((ele)=> ele*2);
console.log(myArr2);
```

Practice exercise 8.4

```
const val = "thIs will be capItalized for each word";
function wordsCaps(str) {
  str = str.toLowerCase();
  const tempArr = [];
  let words = str.split(" ");
  words.forEach(word => {
    let temp = word.slice(0, 1).toUpperCase() + word.slice(1);
    tempArr.push(temp);
  });
  return tempArr.join(" ");
}
console.log(wordsCaps(val));
```

Practice exercise 8.5

```
let val = "I love JavaScript";
val = val.toLowerCase();
let vowels = ["a","e","i","o","u"];
vowels.forEach((letter,index) =>{
  console.log(letter);
  val = val.replaceAll(letter,index);
});
console.log(val);
```

Practice exercise 8.6

```
console.log(Math.ceil(5.7));
console.log(Math.floor(5.7));
console.log(Math.round(5.7));
console.log(Math.random());
console.log(Math.floor(Math.random()*11)); // 0-10
console.log(Math.floor(Math.random()*10)+1); // 1-10;
console.log(Math.floor(Math.random()*100)+1); // 1-100;
function ranNum(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
for (let x = 0; x < 100; x++) {
  console.log(ranNum(1, 100));
}
```

Practice exercise 8.7

```
let future = new Date(2025, 5, 15);
console.log(future);
const months = ["January", "February", "March", "April", "May", "June",
  "July", "August", "September", "October", "November", "December"];
let day = future.getDate();
let month = future.getMonth();
let year = future.getFullYear();
let myDate = `${months[month-1]} ${day} ${year}`;
console.log(myDate);
```

Projects**Word scrambler**

```
let str = "JavaScript";

function scramble(val) {
  let max = val.length;
  let temp = "";
  for(let i=0;i<max;i++){
    console.log(val.length);
    let index = Math.floor(Math.random() * val.length);
    temp += val[index];
    console.log(temp);
    val = val.substr(0, index) + val.substr(index + 1);
    console.log(val);
  }
  return temp;
}
console.log(scramble(str));
```

Countdown timer

```
const endDate = "Sept 1 2022";

function countdown() {
  const total = Date.parse(endDate) - new Date();
  const days = Math.floor(total / (1000 * 60 * 60 * 24));
  const hrs = Math.floor((total / (1000 * 60 * 60)) % 24);
  const mins = Math.floor((total / 1000 / 60) % 60);
  const secs = Math.floor((total / 1000) % 60);
  return {
    days,
    hrs,
    mins,
    secs
  };
}

function update() {
  const temp = countdown();
```

```

let output = "";
for (const property in temp) {
  output += ` ${property}: ${temp[property]} `;
}
console.log(output);
setTimeout(update, 1000);
}

update();

```

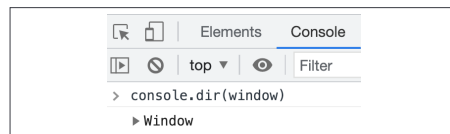
Self-check quiz

1. decodeURIComponent(e)
2. 4
3. ["Hii", "hi", "hello", "Hii", "hi", "hi World", "Hi"]
4. ["hi", "hi World"]

Chapter 9, The Document Object Model

Practice exercises

Practice exercise 9.1



Practice exercise 9.2

```

console.log(window.location.protocol);
console.log(window.location.href);

```

Practice exercise 9.3

```

<script>
  const output = document.querySelector('.output');
  output.textContent = "Hello World";
  output.classList.add("red");
  output.id = "tester";
  output.style.backgroundColor = "red";
  console.log(document.URL);
  output.textContent = document.URL;
</script>

```

Projects

Manipulating HTML elements with JavaScript

```

const output = document.querySelector(".output");
const mainList = output.querySelector("ul");
mainList.id = "mainList";

```



```

console.log(mainList);
const eles = document.querySelectorAll("div");
for (let x = 0; x < eles.length; x++) {
  console.log(eles[x].tagName);
  eles[x].id = "id" + (x + 1);
  if (x % 2) {
    eles[x].style.color = "red";
  } else {
    eles[x].style.color = "blue";
  }
}

```

Self-check quiz

1. You should see an object representing the list of elements contained within body object of the HTML page.

```

> document.body
< <body>
  > <ul></ul>
  > <div class="output"></div>
  > <script></script>
  <!-- Code injected by live-server -->
  > <script type="text/javascript"></script>
</body>

```

2. `document.body.textContent = "Hello World";`

3. The code is as follows:

```

for (const property in document) {
  console.log(`${property}: ${document[property]}`);
}

```

4. The code is as follows:

```

for (const property in window) {
  console.log(`${property}: ${document[window]}`);
}

```

5. The code is as follows:

```

<!doctype html>
<html>
<head>

```

```

<title>JS Tester</title>
</head>
<body>
  <h1>Test</h1>

  <script>
    const output = document.querySelector('h1');
    output.textContent = "Hello World";
  </script>
</body>
</html>

```

Chapter 10, Dynamic Element Manipulation Using the DOM

Practice exercises

Practice exercise 10.1

```

> console.dir(document)
  > #document
  < undefined
  > document.body.children
  < <HTMLCollection [div.main] >
    > 0: div.main
    length: 1
    > [[Prototype]]: HTMLCollection
  > document.body.children[0].children[0]
  < > <div></div>
  > document.body.children[0].children[0].nextSibling
  < > #text

```

Practice exercise 10.2

```
<!doctype html>
<html>
<head>
  <title>Canvas HTML5</title>
</head>
<body>
  <div id="one">Hello World</div>
  <script>
    const myEle = document.getElementById("one");
    console.log(myEle);
  </script>
</body>
</html>
```

Practice exercise 10.3

```
<!doctype html>
<html>
<head>
  <title>Dynamic event manipulation</title>
</head>
<body>
  <div>Hello World 1</div>
  <div>Hello World 2</div>
  <div>Hello World 3</div>
  <script>
    const myEles = document.getElementsByTagName("div");
    console.log(myEles[1]);
  </script>
</body>
</html>
```

Practice exercise 10.4

```
<!doctype html>
<html>
<head>
  <title>Canvas HTML5</title>
</head>
<body>
```

```
<body>
  <h1 class="ele">Hello World</h1>
  <div class="ele">Hello World 1</div>
  <div class="ele">Hello World 3</div>
  <script>
    const myEles = document.getElementsByClassName("ele");
    console.log(myEles[0]);
  </script>
</html>
```

Practice exercise 10.5

```
<!doctype html>
<html>
<head>
  <title>Canvas HTML5</title>
</head>
<body>
  <div>
    <h1 class="ele">Hello World</h1>
    <div class="ele">Hello World 1</div>
    <div class="ele">Hello World 3</div>
    <p class="ele">Hello World 4</p>
  <script>
    const myEle = document.querySelector(".ele");
    console.log(myEle);
  </script>
</html>
```

Practice exercise 10.6

```
<!doctype html>
<html>
<head>
  <title>JS Tester</title>
</head>
<body>
  <div class="container">
    <div class="myEle">One</div>
    <div class="myEle">Two</div>
    <div class="myEle">Three</div>
```

```

    <div class="myEle">Four</div>
    <div class="myEle">Five</div>
  </div>
</script>
const eles = document.querySelectorAll(".myEle");
console.log(eles);
eles.forEach((el) => {
  console.log(el);
});
</script>
</body>
</html>

```

Practice exercise 10.7

```

<!doctype html>
<html>
<head>
  <title>JS Tester</title>
</head>
<body>
  <div>
    <button onclick="message(this)">Button 1</button>
    <button onclick="message(this)">Button 2</button>
  </div>
  <script>
    function message(el) {
      console.dir(el.textContent);
    }
  </script>
</body>
</html>

```

Practice exercise 10.8

```

<script>
const message = document.querySelector("#message");
const myArray = ["Laurence", "Mike", "John", "Larry", "Kim",
  "Joanne", "Lisa", "Janet", "Jane"];
build();
//addClicks();

```

```

function build() {
  let html = "<h1>My Friends Table</h1><table>";
  myArray.forEach((item, index) => {
    html += "<tr class='box' data-row='${index+1}'"
      data-name='${item}' onclick='getData(this)'>
      <td>${item}</td>";
    html += "<td >${index + 1}</td></tr>";
  });
  html += "</table>";
  document.getElementById("output").innerHTML = html;
}

function getData(el) {
  let temp = el.getAttribute("data-row");
  let tempName = el.getAttribute("data-name");
  message.innerHTML = `${tempName } is in row #${temp}`;
}
</script>

```

Practice exercise 10.9

```

<script>
const btns = document.querySelectorAll("button");
btns.forEach((btn) => {
  function output() {
    console.log(this.textContent);
  }
  btn.addEventListener("click", output);
});
</script>

```

Practice exercise 10.10

```

<script>
document.getElementById("addNew").onclick = function () {
  addOne();
}
function addOne() {
  var a = document.getElementById("addItem").value;
  var li = document.createElement("li");
  li.appendChild(document.createTextNode(a));
}

```

```

    }
    document.getElementById("slist").appendChild(li);
  }
</script>

```

Projects

Collapsible accordion component

```

<script>
const menus = document.querySelectorAll(".title");
const openText = document.querySelectorAll(".myText");
menus.forEach((el) => {
  el.addEventListener("click", (e) => {
    console.log(el.nextElementSibling);
    remover();
    el.nextElementSibling.classList.toggle("active");
  })
})
function remover() {
  openText.forEach((ele) => {
    ele.classList.remove("active");
  })
}
}
</script>

```

Interactive voting system

```

<script>
window.onload = build;
const myArray = ["Laurence", "Mike", "John", "Larry"];
const message = document.getElementById("message");
const addNew = document.getElementById("addNew");
const newInput = document.getElementById("addFriend");
const output = document.getElementById("output");
addNew.onclick = function () {
  const newFriend = newInput.value;
  adder(newFriend, myArray.length, 0);
  myArray.push(newFriend);
}
function build() {

```

```

myArray.forEach((item, index) => {
  adder(item, index, 0);
});
});
}
function adder(name, index, counter) {
  const tr = document.createElement("tr");
  const td1 = document.createElement("td");
  td1.classList.add("box");
  td1.textContent = index + 1;
  const td2 = document.createElement("td");
  td2.textContent = name;
  const td3 = document.createElement("td");
  td3.textContent = counter;
  tr.appendChild(td1);
  tr.appendChild(td2);
  tr.appendChild(td3);
  tr.onclick = function () {
    console.log(tr.lastChild);
    let val = Number(tr.lastChild.textContent);
    val++;
    tr.lastChild.textContent = val;
  }
  output.appendChild(tr);
}
}
</script>

```

Hangman game

```

<script>
const game = { cur: "", solution: "", puzz: [], total: 0 };
const myWords = ["learn Javascript", "learn html",
  "learn css"];
const score = document.querySelector(".score");
const puzzle = document.querySelector(".puzzle");
const letters = document.querySelector(".letters");
const btn = document.querySelector("button");
btn.addEventListener("click", startGame);
function startGame() {
  if (myWords.length > 0) {
    btn.style.display = "none";
    game.puzz = [];

```

```

        game.total = 0;
        game.cur = myWords.shift();
        game.solution = game.cur.split("");
        builder();
    } else {
        score.textContent = "No More Words.";
    }
}

function createElements(elType, parentEle, output, cla) {
    const temp = document.createElement(elType);
    temp.classList.add("boxE");
    parentEle.append(temp);
    temp.textContent = output;
    return temp;
}

function updateScore() {
    score.textContent = `Total Letters Left : ${game.total}`;
    if (game.total <= 0) {
        console.log("game over");
        score.textContent = "Game Over";
        btn.style.display = "block";
    }
}

function builder() {
    letters.innerHTML = "";
    puzzle.innerHTML = "";
    game.solution.forEach((lett) => {
        let div = createElements("div", puzzle, "-", "boxE");
        if (lett == " ") {
            div.style.borderColor = "white";
            div.textContent = " ";
        } else {
            game.total++;
        }
        game.puzz.push(div);
        updateScore();
    })
    for (let i = 0; i < 26; i++) {
        let temp = String.fromCharCode(65 + i);
        let div = createElements("div", letters, temp, "box");
    }
}

```

```

        let checker = function (e) {
            div.style.backgroundColor = "#ddd";
            div.classList.remove("box");
            div.classList.add("boxD");
            div.removeEventListener("click", checker);
            checkLetter(temp);
        }
        div.addEventListener("click", checker);
    }
}

function checkLetter(letter) {
    console.log(letter);
    game.solution.forEach((ele, index) => {
        game.puzz.forEach((puzz, index) => {
            if (ele.toUpperCase() == letter) {
                game.puzz[index].textContent = letter;
                game.total--;
                updateScore();
            }
        });
    });
}
</script>

```

Self-check quiz

1. Hello
 World
2. Hello
World
3. Hello World
4. When three gets clicked, the output is three. When one gets clicked, the output is:
one
two
three
5. btn.removeEventListener("click", myFun);

Chapter 11, Interactive Content and Event Listeners

Practice exercises

Practice exercise 11.1

```
<!DOCTYPE html>
<html>

<head>
  <title>Laurence Svekis</title>
</head>

<body>
  <script>
    let darkMode = false;
    window.onclick = () => {
      console.log(darkMode);
      if (!darkMode) {
        document.body.style.backgroundColor = "black";
        document.body.style.color = "white";
        darkMode = true;
      } else {
        document.body.style.backgroundColor = "white";
        document.body.style.color = "black";
        darkMode = false;
      }
    }
  </script>
</body>
</html>
```

Practice exercise 11.2

```
<!doctype html>
<html>
<body>
  <div>red</div>
  <div>blue</div>
  <div>green</div>
  <div>yellow</div>
  <script>
    const divs = document.querySelectorAll("div");
    divs.forEach((el) => {
      el.addEventListener("click", () => {
        document.body.style.backgroundColor = el.textContent;
      });
    });
  </script>
</body>
</html>
```

Practice exercise 11.3

```
<!doctype html>
<html>
<head>
  <title>JS Tester</title>
</head>
<body>
  <script>
    document.addEventListener("DOMContentLoaded", (e) => {
      message("Document ready", e);
    });
    window.onload = (e) => {
      message("Window ready", e);
    }
    function message(val, event) {
      console.log(event);
      console.log(val);
    }
  </script>
</body>
</html>
```

Practice exercise 11.4

```
<!doctype html>
<html>
<head>
  <title>JS Tester</title>
</head>
<body>
  <div class="output"></div>
  <script>
    const output = document.querySelector(".output");
    output.textContent = "hello world";
    output.style.height = "200px";
    output.style.width = "400px";
    output.style.backgroundColor = "red";
    output.addEventListener("mousedown", function (e) {
      message("green", e);
    });
    output.addEventListener("mouseover", function (e) {
      message("red", e);
    });
    output.addEventListener("mouseout", function (e) {
      message("yellow", e);
    });
    output.addEventListener("mouseup", function (e) {
      message("blue", e);
    });
    function message(elColor, event) {
      console.log(event.type);
      output.style.backgroundColor = elColor;
    }
  </script>
</body>
</html>
```

Practice exercise 11.5

```
<script>
  const myInput = document.querySelector("input[name='message']");
  const output = document.querySelector(".output");
  const btn1 = document.querySelector(".btn1");
```

```
const btn2 = document.querySelector(".btn2");
const btn3 = document.querySelector(".btn3");
const log = [];
btn1.addEventListener("click", tracker);
btn2.addEventListener("click", tracker);
btn3.addEventListener("click", (e) => {
  console.log(log);
});
function tracker(e) {
  output.textContent = myInput.value;
  const ev = e.target;
  console.dir(ev);
  const temp = {
    message: myInput.value,
    type: ev.type,
    class: ev.className,
    tag: ev.tagName
  };
  log.push(temp);
  myInput.value = "";
}
</script>
```

Practice exercise 11.6

```
<script>
  const main = document.querySelector(".container");
  const boxes = document.querySelectorAll(".box");
  main.addEventListener("click", (e) => {
    console.log("4");
  }, false);
  main.addEventListener("click", (e) => {
    console.log("1");
  }, true);

  boxes.forEach(ele => {
    ele.addEventListener("click", (e) => {
      console.log("3");
      console.log(e.target.textContent);
    }, false);
```

```

    ele.addEventListener("click", (e) => {
      console.log("2");
      console.log(e.target.textContent);
    }, true);
  });
</script>

```

Practice exercise 11.7

```

<script>
  const output = document.querySelector(".output1");

  const in1 = document.querySelector("input[name='first']");
  const in2 = document.querySelector("input[name='last']");
  in1.addEventListener("change", (e) => {
    console.log("change");
    updater(in1.value);
  });
  in1.addEventListener("blur", (e) => {
    console.log("blur");
  });
  in1.addEventListener("focus", (e) => {
    console.log("focus");
  });
  in2.addEventListener("change", (e) => {
    console.log("change");
    updater(in2.value);
  });
  in2.addEventListener("blur", (e) => {
    console.log("blur");
  });
  in2.addEventListener("focus", (e) => {
    console.log("focus");
  });
  function updater(str) {
    output.textContent = str;
  }
</script>

```

Practice exercise 11.8

```

<!doctype html>
<html>
<head>
  <title>JS Tester</title>
</head>
<body>
  <div class="output"></div>
  <input type="text" name="myNum1">
  <input type="text" name="myNum2">
  <script>
    const eles = document.querySelectorAll("input");
    const output = document.querySelector(".output");
    eles.forEach(e1 => {
      e1.addEventListener("keydown", (e) => {
        if (!isNaN(e.key)) {
          output.textContent += e.key;
        }
      });
      e1.addEventListener("keyup", (e) => {
        console.log(e.key);
      });
      e1.addEventListener("paste", (e) => {
        console.log('pastet');
      });
    });
  </script>
</body>
</html>

```

Practice exercise 11.9

```

<script>
  const dragme = document.querySelector("#dragme");
  dragme.addEventListener("dragstart", (e) => {
    dragme.style.opacity = .5;
  });
  dragme.addEventListener("dragend", (e) => {
    dragme.style.opacity = "";
  });
  const boxes = document.querySelectorAll(".box");

```



```
boxes.forEach(box => {
  box.addEventListener("dragenter", (e) => {
    e.target.classList.add('red');
  });
  box.addEventListener("dragover", (e) => {
    e.preventDefault();
  });
  box.addEventListener("dragleave", (e) => {
    //console.log("leave");
    e.target.classList.remove('red');
  });
  box.addEventListener("drop", (e) => {
    e.preventDefault();
    console.log("dropped");
    e.target.appendChild(dragme);
  });
});
function dragStart(e) {
  console.log("Started");
}
</script>
```

Practice exercise 11.10

```
<!doctype html>
<html>
<head>
<title>JS Tester</title>
</head>
<body>
  <form action="index2.html" method="get">
    First: <input type="text" name="first">
    <br>Last: <input type="text" name="last">
    <br>Age: <input type="number" name="age">
    <br><input type="submit" value="submit">
  </form>
  <script>
    const form = document.querySelector("form");
    const email = document.querySelector("#email");
    form.addEventListener("submit", (e) => {
      let error = false;
```

```
if (checker(form.first.value)) {
  console.log("First Name needed");
  error = true;
}
if (checker(form.last.value)) {
  console.log("Last Name needed");
  error = true;
}
if (form.age.value < 19) {
  console.log("You must be 19 or over");
  error = true;
}
if (error) {
  e.preventDefault();
  console.log("please review the form");
}
});
function checker(val) {
  console.log(val.length);
  if (val.length < 6) {
    return true;
  }
  return false;
}
</script>
</body>
</html>
```

Practice exercise 11.11

```
<!doctype html>
<html>
<style>
  div {
    background-color: purple;
    width: 100px;
    height: 100px;
    position: absolute;
  }
</style>
<body>
```

```

<div id="block"></div>
<script>
  const main = document.querySelector("#block");
  let mover = { speed: 10, dir: 1, pos: 0 };
  main.addEventListener("click", moveBlock);
  function moveBlock() {
    let x = 30;
    setInterval(function () {
      if (x < 1) {
        clearInterval();
      } else {
        if (mover.pos > 800 || mover.pos < 0) {
          mover.dir *= -1;
        }
        mover.pos += x * mover.dir;
        main.style.left = mover.pos + "px";
        console.log(mover.pos);
      }
    }, 2);
  }
</script>
</body>
</html>

```

Projects

Build your own analytics

```

<!doctype html >
<html>
<head>
  <title>JS Tester</title>
  <style>.box{width:200px;height:100px;border:1px solid black}</style>
</head>
<body>
  <div class="container">
    <div class="box" id="box0">Box #1</div>
    <div class="box" id="box1">Box #2</div>
    <div class="box" id="box2">Box #3</div>
  </div>
</body>
</html>

```

```

<div class="box" id="box3">Box #4</div>
</div>
<script>
  const counter = [];
  const main = document.querySelector(".container");
  main.addEventListener("click", tracker);
  function tracker(e){
    const el = e.target;
    if(el.id){
      const temp = {};
      temp.content = el.textContent;
      temp.id = el.id;
      temp.tagName = el.tagName;
      temp.class = el.className;
      console.dir(el);
      counter.push(temp);
      console.log(counter);
    }
  }
</script>
</body>
</html>

```

Star rater system

```

<script>
  const starsUL = document.querySelector(".stars");
  const output = document.querySelector(".output");
  const stars = document.querySelectorAll(".star");
  stars.forEach((star, index) => {
    star.starValue = (index + 1);
    star.addEventListener("click", starRate);
  });
  function starRate(e) {
    output.innerHTML =
      `You Rated this ${e.target.starValue} stars`;
    stars.forEach((star, index) => {
      if (index < e.target.starValue) {
        star.classList.add("orange");
      } else {
        star.classList.remove("orange");
      }
    });
  }
</script>

```

```

    });
  }
</script>

```

Mouse position tracker

```

<!DOCTYPE html>
<html>
<head>
  <title>Complete JavaScript Course</title>
  <style>
    .holder {
      display: inline-block;
      width: 300px;
      height: 300px;
      border: 1px solid black;
      padding: 10px;
    }

    .active {
      background-color: red;
    }
  </style>
</head>
<body>
  <div class="holder">
    <div id="output"></div>
  </div>
  <script>
    const ele = document.querySelector(".holder");
    ele.addEventListener("mouseover",
      (e) => { e.target.classList.add("active"); });
    ele.addEventListener("mouseout",
      (e) => { e.target.classList.remove("active"); });
    ele.addEventListener("mousemove", coordin);
    function coordin() {
      let html = "X:" + event.clientX + " | Y:" + event.clientY;
      document.getElementById("output").innerHTML = html;
    }
  </script>
</body>
</html>

```

Box clicker speed test game

```

<script>
  const output = document.querySelector('.output');
  const message = document.querySelector('.message');
  message.textContent = "Press to Start";
  const box = document.createElement('div');
  const game = {
    timer: 0,
    start: null
  };
  box.classList.add('box');
  output.append(box);

  box.addEventListener('click', (e) => {
    box.textContent = "";
    box.style.display = 'none';
    game.timer = setTimeout(addBox, ranNum(3000));
    if (!game.start) {
      message.textContent = 'Loading....';
    } else {
      const cur = new Date().getTime();
      const dur = (cur - game.start) / 1000;
      message.textContent = `It took ${dur} seconds to click`;
    }
  });

  function addBox() {
    message.textContent = 'Click it...';
    game.start = new Date().getTime();
    box.style.display = 'block';
    box.style.left = ranNum(450) + 'px';
    box.style.top = ranNum(450) + 'px';
  }

  function ranNum(max) {
    return Math.floor(Math.random() * max);
  }
</script>

```

Self-check quiz

- 1. Window Object Model.
- 2. The preventDefault() method cancels the event if it can be canceled. The default action that belongs to the event will not occur.

Chapter 12, Intermediate JavaScript

Practice exercises

Practice exercise 12.1

```
<script>
const output = document.getElementById("output");
const findValue = document.getElementById("sText");
const replaceValue = document.getElementById("rText");
document.querySelector("button").addEventListener("click", lookup);

function lookup() {
  const s = output.textContent;
  const rt = replaceValue.value;
  const re = new RegExp(findValue.value, "gi");
  if (s.match(re)) {
    let newValue = s.replace(re, rt);
    output.textContent = newValue;
  }
}
</script>
```

Practice exercise 12.2

```
<script>
const output = document.querySelector(".output");
const emailVal = document.querySelector("input");
const btn = document.querySelector("button");
const emailExp =
  /^[A-Za-z0-9._-]+@[A-Za-z0-9._-]+\.[A-Za-z0-9]+\w+$/;
btn.addEventListener("click", (e) => {
```

```
const val = emailVal.value;
const result = emailExp.test(val);
let response = "";
if (!result) {
  response = "Invalid Email";
  output.style.color = "red";
} else {
  response = "Valid Email";
  output.style.color = "green";
}
emailVal.value = "";
output.textContent = response;
});
</script>
```

Practice exercise 12.3

```
function showNames() {
  let lastOne = "";
  for (let i = 0; i < arguments.length; i++) {
    lastOne = arguments[i];
  }
  return lastOne;
}
console.log(showNames("JavaScript", "Laurence", "Mike", "Larry"));
```

Practice exercise 12.4

15
45

Practice exercise 12.5

```
function test(val) {
  try {
    if (isNaN(val)) {
      throw "Not a number";
    } else {
      console.log("Got number");
    }
  }
```

```

    } catch (e) {
      console.error(e);
    } finally {
      console.log("Done " + val);
    }
  }
  test("a");
  test(100);

```

Practice exercise 12.6

```

<script>
  console.log(document.cookie);
  console.log(rCookie("test1"));
  console.log(rCookie("test"));
  cCookie("test1", "new Cookie", 30);
  dCookie("test2");
  function cCookie(cName, value, days) {
    if (days) {
      const d = new Date();
      d.setTime(d.getTime() + (days * 24 * 60 * 60 * 1000));
      let e = "; expires=" + d.toUTCString();
      document.cookie = cName + "=" + value + e + "; path=/";
    }
  }
  function rCookie(cName) {
    let cookieValue = false;
    let arr = document.cookie.split("; ");
    arr.forEach(str => {
      const cookie = str.split("=");
      if (cookie[0] == cName) {
        cookieValue = cookie[1];
      }
    });
    return cookieValue;
  }
  function dCookie(cName) {
    cCookie(cName, "", -1);
  }
</script>

```

Practice exercise 12.7

```

<script>
  const userTask = document.querySelector(".main input");
  const addBtn = document.querySelector(".main button");
  const output = document.querySelector(".output");
  const tasks = JSON.parse(localStorage.getItem("tasklist")) || [];
  addBtn.addEventListener("click", createListItem);
  if (tasks.length > 0) {
    tasks.forEach(task => {
      genItem(task.val, task.checked);
    });
  }
  function saveTasks() {
    localStorage.setItem("tasklist", JSON.stringify(tasks));
  }
  function buildTasks() {
    tasks.length = 0;
    const curList = output.querySelectorAll("li");
    curList.forEach(el => {
      const tempTask = {
        val: el.textContent,
        checked: false
      };
      if (el.classList.contains("ready")) {
        tempTask.checked = true;
      }
      tasks.push(tempTask);
    });
    saveTasks();
  }
  function genItem(val, complete) {
    const li = document.createElement("li");
    const temp = document.createTextNode(val);
    li.appendChild(temp);
    output.appendChild(li);
    tempTask.value = "";
    if (complete) {
      li.classList.add("ready");
    }
  }

```

```

    li.addEventListener("click", (e) => {
      li.classList.toggle("ready");
      buildTasks();
    });
    return val;
  }
  function createListItem() {
    const val = userTask.value;
    if (val.length > 0) {
      const myObj = {
        val: genItem(val, false),
        checked: false
      };
      tasks.push(myObj);
      saveTasks();
    }
  }
}
</script>

```

Practice exercise 12.8

```

let myList = [{
  "name": "Learn JavaScript",
  "status": true
},
{
  "name": "Try JSON",
  "status": false
}
];

reloader();
function reloader() {
  myList.forEach((el) => {
    console.log(`${el.name} = ${el.status}`);
  });
}

```

Practice Exercise 12.9

```

let myList = [{
  "name": "Learn JavaScript",
  "status": true
},
{
  "name": "Try JSON",
  "status": false
}
];

const newStr = JSON.stringify(myList);
const newObj = JSON.parse(newStr);
newObj.forEach((el) => {
  console.log(el);
});

```

Projects

Email extractor

```

<script>
  const firstArea = document.querySelector(
    "textarea[name='txtarea1']");
  const secArea = document.querySelector(
    "textarea[name='txtarea2']");
  document.querySelector("button").addEventListener("click", lookup);
  function lookup() {
    const rawTxt = firstArea.value;
    const eData = rawTxt.match(
      /[a-zA-Z0-9_-]+@[a-zA-Z0-9_-]+\.[a-zA-Z0-9_-]+/g);
    const holder = [];
    for (let x = 0; x < eData.length; x++) {
      if (holder.indexOf(eData[x]) == -1) {
        holder.push(eData[x]);
      }
    }
    secArea.value = holder.join(',');
  }
</script>

```

Form validator

```

<script>
const myForm = document.querySelector("form");
const inputs = document.querySelectorAll("input");
const errors = document.querySelectorAll(".error");
const required = ["email", "userName"];
myForm.addEventListener("submit", validation);
function validation(e) {
  let data = {};
  e.preventDefault();
  errors.forEach(function (item) {
    item.classList.add("hide");
  });
  let error = false;
  inputs.forEach(function (el) {
    let tempName = el.getAttribute("name");
    if (tempName != null) {
      el.style.borderColor = "#ddd";
      if (el.value.length == 0 &&
        required.includes(tempName)) {
        addError(el, "Required Field", tempName);
        error = true;
      }
      if (tempName == "email") {
        let exp = /^[A-Za-z0-9_-]+@[A-Za-z0-9_-]+\.[A-Za-z0-9]+\w+$/;
        let result = exp.test(el.value);
        if (!result) {
          addError(el, "Invalid Email", tempName);
          error = true;
        }
      }
      if (tempName == "password") {
        let exp = /[A-Za-z0-9]+$/;
        let result = exp.test(el.value);
        if (!result) {
          addError(el, "Only numbers and Letters",
            tempName);
          error = true;
        }
      }
    }
  });
}

```

```

        if (!(el.value.length > 3 &&
          el.value.length < 9)) {
          addError(el, "Needs to be between 3-8 " +
            "characters", tempName);
          error = true;
        }
      }
      data[tempName] = el.value;
    });
    if (!error) {
      myForm.submit();
    }
  }

  function addError(el, mes, fieldName) {
    let temp = el.nextElementSibling;
    temp.classList.remove("hide");
    temp.textContent = fieldName.toUpperCase() + " " + mes;
    el.style.borderColor = "red";
    el.focus();
  }
}
</script>

```

Simple math quiz

```

<!doctype html>
<html>
<head>
<title>Complete JavaScript Course</title>
</head>
<body>
<span class="val1"></span> <span>+</span>
<span class="val2"></span> = <span>
  <input type="text" name="answer"></span><button>Check</button>
<div class="output"></div>
<script>
const app = function () {
  const game = {};
  const val1 = document.querySelector(".val1");
  const val2 = document.querySelector(".val2");
  const output = document.querySelector(".output");
}

```

```

const answer = document.querySelector("input");
function init() {
  document.querySelector("button").addEventListener(
    "click", checker);
  loadQuestion();
}
function ranValue(min, max) {
  return Math.floor(Math.random() * (max - min + 1) +
    min);
}
function loadQuestion() {
  game.val1 = ranValue(1, 100);
  game.val2 = ranValue(1, 100);
  game.answer = game.val1 + game.val2;
  val1.textContent = game.val1;
  val2.textContent = game.val2;
}
function checker() {
  let bg = answer.value == game.answer ? "green" : "red";
  output.innerHTML +=
    <div style="color:${bg}">${game.val1} +
    ${game.val2} = ${game.answer} ${answer.value}</div>;
  answer.value = "";
  loadQuestion();
}
return {
  init: init
};
})();
document.addEventListener('DOMContentLoaded', app.init);
</script>
</body>
</html>

```

Self-check quiz

1. The range matched is from a to e and is case sensitive. It will return the rest of the word: enjoy aJavaScript.
2. Yes.
3. It will clear cookies from the site.
4. hello world

5. a is not defined.
6. a
c
b

Chapter 13, Concurrency

Practice exercises

Practice exercise 13.1

```

function greet(fullName){
  console.log(`Welcome, ${fullName[0]} ${fullName[1]}`)
}
function processCall(user, callback){
  const fullName = user.split(" ");
  callback(fullName);
}

processCall("Laurence Svekis", greet);

```

Practice exercise 13.2

```

const myPromise = new Promise((resolve, reject) => {
  resolve("Start Counting");
});

function counter(val){
  console.log(val);
}

myPromise
  .then(value => {counter(value); return "one"})
  .then(value => {counter(value); return "two"})
  .then(value => {counter(value); return "three"})
  .then(value => {counter(value)});

```


Practice exercise 13.3

```

let cnt = 0;
function outputTime(val) {
  return new Promise(resolve => {
    setTimeout(() => {
      cnt++;
      resolve(`x value ${val} counter:${cnt}`);
    }, 1000);
  });
}
async function aCall(val) {
  console.log(`ready ${val} counter:${cnt}`);
  const res = await outputTime(val);
  console.log(res);
}
for (let x = 1; x < 4; x++) {
  aCall(x);
}

```

Projects**Password checker**

```

const allowed = ["1234", "pass", "apple"];

function passwordChecker(pass) {
  return allowed.includes(pass);
}

function login(password) {
  return new Promise((resolve, reject) => {
    if (passwordChecker(password)) {
      resolve({
        status: true
      });
    } else {
      reject({
        status: false
      });
    }
  });
}

```

```

});
}

function checker(pass) {
  login(pass)
    .then(token => {
      console.log("Approve:");
      console.log(token);
    })
    .catch(value => {
      console.log("Reject:");
      console.log(value);
    })
  }
  checker("1234");
  checker("wrong");
}

```

Self-check quiz

1. The updated code is as follows:

```

function addOne(val){
  return val + 1;
}

function total(a, b, callback){
  const sum = a + b;
  return callback(sum);
}

console.log(total(4, 5, addOne));

```

2. The console will show the error message Error: Oh no.
3. The updated code is as follows:

```

function checker(val) {
  return new Promise((resolve, reject) => {
    if (val > 5) {
      resolve("Ready");
    } else {
      reject(new Error("Oh no"));
    }
  });
}

```

```
checker(5)
  .then((data) => { console.log(data); })
  .catch((err) => { console.error(err); })
  .finally(() => { console.log("done"); });
```

4. The updated code is as follows:

```
async function myFun() {
  return "Hello";
}
myFun().then(
  function(val) { console.log(val); },
  function(err) { console.log(err); }
```

Chapter 14, HTML5, Canvas, and JavaScript

Practice exercises

Practice exercise 14.1

```
<script>
const message = document.getElementById("message");
const output = document.querySelector(".output");
const myInput = document.querySelector("input");
myInput.addEventListener("change", uploadAndReadFile);
function uploadAndReadFile(e) {
  const files = e.target.files;
  for (let i = 0; i < files.length; i++) {
    const file = files[i];
    const img = document.createElement("img");
    img.classList.add("thumb");
    img.file = file;
    output.appendChild(img);
    const reader = new FileReader();
    reader.onload = (function (myImg) {
      return function (e) {
        myImg.src = e.target.result;
      };
    })(img);
    reader.readAsDataURL(file);
  }
}
```

```
))(img);
  reader.readAsDataURL(file);
}
}
</script>
```

Practice exercise 14.2

```
<!doctype html>
<html>
<head>
  <title>Canvas HTML5</title>
  <style>
    #canvas {
      border: 1px solid black;
    }
  </style>
</head>
<body>
  <canvas id="canvas" width="640" height="640">Not Supported</canvas>
  <script>
    const canvas = document.querySelector("#canvas");
    const ctx = canvas.getContext("2d");
    ctx.fillStyle = "red";
    ctx.fillRect(100, 100, 500, 300); //filled shape
    ctx.strokeRect(90, 90, 520, 320); // outline
    ctx.clearRect(150, 150, 400, 200); //transparent
  </script>
</body>
</html>
```

Practice exercise 14.3

```
<!doctype html>
<html>
<head>
  <title>Canvas HTML5</title>
  <style>
    #canvas {
      border: 1px solid black;
    }
  </style>
</head>
```

```
</style>
</head>
<body>
  <canvas id="canvas" width="640" height="640">Not Supported</canvas>
  <script>
    const canvas = document.querySelector("#canvas");
    const ctx = canvas.getContext("2d");
    ctx.beginPath();
    ctx.fillStyle = "red";
    ctx.arc(300, 130, 100, 0, Math.PI * 2);
    ctx.fill();
    ctx.beginPath();
    ctx.fillStyle = "black";
    ctx.arc(250, 120, 20, 0, Math.PI * 2);
    ctx.moveTo(370, 120);
    ctx.arc(350, 120, 20, 0, Math.PI * 2);
    ctx.moveTo(240, 160);
    ctx.arc(300, 160, 60, 0, Math.PI);
    ctx.fill();
    ctx.moveTo(300, 130);
    ctx.lineTo(300, 150);
    ctx.stroke();
    ctx.beginPath();
    ctx.moveTo(300, 230);
    ctx.lineTo(300, 270);
    ctx.lineTo(400, 270);
    ctx.lineTo(200, 270);
    ctx.lineTo(300, 270);
    ctx.lineTo(300, 350);
    ctx.lineTo(400, 500);
    ctx.moveTo(300, 350);
    ctx.lineTo(200, 500);
    ctx.stroke();
    ctx.beginPath();
    ctx.fillStyle = "blue";
    ctx.moveTo(200, 50);
    ctx.lineTo(400, 50);
    ctx.lineTo(300, 20);
    ctx.lineTo(200, 50);
    ctx.fill();
    ctx.stroke();
```

```
</script>
</body>
</html>
```

Practice exercise 14.4

```
<!doctype html>
<html>
<head>
  <title>Canvas HTML5</title>
  <style>
    #canvas {
      border: 1px solid black;
    }
  </style>
</head>
<body>
  <canvas id="canvas" width="640" height="640">Not Supported</canvas>
  <script>
    const canvas = document.querySelector("#canvas");
    const ctx = canvas.getContext("2d");
    ctx.beginPath();
    ctx.fillStyle = "red";
    ctx.arc(300, 130, 100, 0, Math.PI * 2);
    ctx.fill();
    ctx.beginPath();
    ctx.fillStyle = "black";
    ctx.arc(250, 120, 20, 0, Math.PI * 2);
    ctx.moveTo(370, 120);
    ctx.arc(350, 120, 20, 0, Math.PI * 2);
    ctx.moveTo(240, 160);
    ctx.arc(300, 160, 60, 0, Math.PI);
    ctx.fill();
    ctx.moveTo(300, 130);
    ctx.lineTo(300, 150);
    ctx.stroke();
    ctx.beginPath();
    ctx.moveTo(300, 230);
    ctx.lineTo(300, 270);
    ctx.lineTo(400, 270);
    ctx.lineTo(200, 270);
    ctx.lineTo(300, 270);
```

```

    ctx.lineTo(300, 350);
    ctx.lineTo(400, 500);
    ctx.moveTo(300, 350);
    ctx.lineTo(200, 500);
    ctx.stroke();
    ctx.beginPath();
    ctx.fillStyle = "blue";
    ctx.moveTo(200, 50);
    ctx.lineTo(400, 50);
    ctx.lineTo(300, 20);
    ctx.lineTo(200, 50);
    ctx.fill();
    ctx.stroke();
  </script>
</body>
</html>

```

Practice exercise 14.5

```

<!doctype html>
<html>
<head>
  <title>Canvas HTML5</title>
  <style>
    #canvas {
      border: 1px solid black;
    }
  </style>
</head>
<body>
  <div><label>Image</label>
  <input type="file" id="imgLoader" name="imgLoader">
</div>
<div><canvas id="canvas"></canvas></div>
<script>
  const canvas = document.querySelector("#canvas");
  const ctx = canvas.getContext("2d");
  const imgLoader = document.querySelector("#imgLoader");
  imgLoader.addEventListener("change", handleUpload);
  function handleUpload(e) {
    console.log(e);
  }

```

```

const reader = new FileReader();
reader.onload = function (e) {
  console.log(e);
  const img = new Image();
  img.onload = function () {
    canvas.width = img.width / 2;
    canvas.height = img.height / 2;
    ctx.drawImage(img, 0, 0, img.width / 2,
      img.height / 2);
  }
  img.src = e.target.result;
}
reader.readAsDataURL(e.target.files[0]);
}
</script>
</body>
</html>

```

Practice exercise 14.6

```

<!doctype html>
<html>
<head>
  <title>Canvas HTML5</title>
  <style>
    #canvas {
      border: 1px solid black;
    }
  </style>
</head>
<body>
  <div><canvas id="canvas"></canvas></div>
  <script>
    const canvas = document.getElementById("canvas");
    const ctx = canvas.getContext("2d");
    const ballSize = 10;
    let x = canvas.width / 2;
    let y = canvas.height / 2;
    let dirX = 1;
    let dirY = 1;
    function drawBall() {
      ctx.beginPath();

```

```

        ctx.arc(x, y, ballSize, 0, Math.PI * 2);
        ctx.fillStyle = "red";
        ctx.fill();
        ctx.closePath();
    }
    function move() {
        ctx.clearRect(0, 0, canvas.width, canvas.height);
        drawBall();
        if (x > canvas.width - ballSize || x < ballSize) {
            dirX *= -1;
        }
        if (y > canvas.height - ballSize || y < ballSize) {
            dirY *= -1;
        }
        x += dirX;
        y += dirY;
    }
    setInterval(move, 10);
</script>
</body>
</html>

```

Practice exercise 14.7

```

<script>
window.onload = init;
const canvas = document.getElementById("canvas");
const ctx = canvas.getContext("2d");
canvas.style.border = "1px solid black";
const penColor = document.querySelector("#penColor");
const penWidth = document.querySelector("#penWidth");
document.querySelector("#clear").addEventListener(
    "click", clearImg);
canvas.width = 700;
canvas.height = 700;
let pos = {
    x: 0,
    y: 0,
};
function init() {
    canvas.addEventListener("mousemove", draw);
    canvas.addEventListener("mousemove", setPosition);

```

```

        canvas.addEventListener("mouseenter", setPosition);
    }
    function draw(e) {
        if (e.buttons !== 1) return;
        ctx.beginPath();
        ctx.moveTo(pos.x, pos.y);
        setPosition(e);
        ctx.lineTo(pos.x, pos.y);
        ctx.strokeStyle = penColor.value;
        ctx.lineWidth = penWidth.value;
        ctx.lineCap = "round";
        ctx.stroke();
    }
    function setPosition(e) {
        pos.x = e.pageX;
        pos.y = e.pageY;
    }
    function clearImg() {
        const temp = confirm("Clear confirm?");
        if (temp) {
            ctx.clearRect(0, 0, canvas.offsetWidth,
                canvas.offsetHeight);
        }
    }
</script>

```

Projects

Create a Matrix effect

```

<!doctype html>
<html>
<head>
    <title>Canvas HTML5</title>
</head>
<body>
    <div class="output"></div>
<script>
    const canvas = document.createElement("canvas");
    const ctx = canvas.getContext("2d");
    canvas.setAttribute("width", "500");

```

```

canvas.setAttribute("height", "300");
document.body.prepend(canvas);
const colVal = [];
for(let x=0;x<50;x++){
  colVal.push(0);
}
function matrix() {
  ctx.fillStyle = "rgba(0,0,0,.05)";
  ctx.fillRect(0, 0, canvas.width, canvas.height);
  ctx.fillStyle = "green";
  colVal.map((posY, index) => {
    let output = Math.random()<0.5?0:1;
    let posX = (index * 10) + 10;
    ctx.fillText(output, posX, posY);
    if (posY > 100 + Math.random() * 300) {
      colVal[index] = 0;
    } else {
      colVal[index] = posY + 10;
    }
  });
}
setInterval(matrix, 50);
</script>
</body>
</html>

```

Countdown clock

```

<script>
const endDate = document.querySelector("input[name='endDate']");
const clock = document.querySelector(".clock");
let timeInterval;
let timeStop = true;
const savedValue = localStorage.getItem("countdown") || false;
if (savedValue) {
  startClock(savedValue);
  let inputValue = new Date(savedValue);
  endDate.valueAsDate = inputValue;
}
endDate.addEventListener("change", function (e) {

```

```

e.preventDefault();
clearInterval(timeInterval);
const temp = new Date(endDate.value);
localStorage.setItem("countdown", temp);
startClock(temp);
timeStop = true;
});
function startClock(d) {
  function updateCounter() {
    let tl = (timeLeft(d));
    if (tl.total <= 0) {
      timeStop = false;
    }
    for (let pro in tl) {
      let el = clock.querySelector("." + pro);
      if (el) {
        el.innerHTML = tl[pro];
      }
    }
  }
  updateCounter();
  if (timeStop) {
    timeInterval = setInterval(updateCounter, 1000);
  } else {
    clearInterval(timeInterval);
  }
}
function timeLeft(d) {
  let currentDate = new Date();
  let t = Date.parse(d) - Date.parse(currentDate);
  let seconds = Math.floor((t / 1000) % 60);
  let minutes = Math.floor((t / 1000 / 60) % 60);
  let hours = Math.floor((t / (1000 * 60 * 60)) % 24);
  let days = Math.floor(t / (1000 * 60 * 60 * 24));
  return {
    "total": t,
    "days": days,
    "hours": hours,
    "minutes": minutes,
    "seconds": seconds
  };
}

```

```
}
</script>
```

Online paint app

```
<script>
const canvas = document.querySelector("#canvas");
const ctx = canvas.getContext("2d");
const penColor = document.querySelector("#penColor");
const penWidth = document.querySelector("#penWidth");
const btnSave = document.querySelector(".save");
const btnClear = document.querySelector(".clear");
const output = document.querySelector(".output");
const mLoc = {
  draw: false,
  x: 0,
  y: 0,
  lastX: 0,
  lastY: 0
};
canvas.style.border = "1px solid black";
btnSave.addEventListener("click", saveImg);
btnClear.addEventListener("click", clearCanvas);
canvas.addEventListener("mousemove", (e) => {
  mLoc.lastX = mLoc.x;
  mLoc.lastY = mLoc.y;
  //console.log(e);
  mLoc.x = e.clientX;
  mLoc.y = e.clientY;
  draw();
});
canvas.addEventListener("mousedown", (e) => {
  mLoc.draw = true;
});
canvas.addEventListener("mouseup", (e) => {
  mLoc.draw = false;
});
canvas.addEventListener("mouseout", (e) => {
  mLoc.draw = false;
});
});
```

```
function saveImg() {
  const dataURL = canvas.toDataURL();
  console.log(dataURL);
  const img = document.createElement("img");
  output.prepend(img);
  img.setAttribute("src", dataURL);
  const link = document.createElement("a");
  output.append(link);
  let fileName = Math.random().toString(16).substr(-8) +
    ".png";
  link.setAttribute("download", fileName);
  link.href = dataURL;
  link.click();
  output.removeChild(link);
}
function clearCanvas() {
  let temp = confirm("clear canvas?");
  if (temp) {
    ctx.clearRect(0, 0, canvas.width, canvas.height);
  }
}
function draw() {
  if (mLoc.draw) {
    ctx.beginPath();
    ctx.moveTo(mLoc.lastX, mLoc.lastY);
    ctx.lineTo(mLoc.x, mLoc.y);
    ctx.strokeStyle = penColor.value;
    ctx.lineWidth = penWidth.value;
    ctx.stroke();
    ctx.closePath();
  }
}
</script>
```

Self-check quiz

1. `const canvas = document.getElementById('canvas');`
`const ctx = canvas.getContext('2d');`
2. Draws a red circle.

3. `moveTo()`, `lineTo()`, `stroke()`

```
ctx.moveTo(100, 0);
ctx.lineTo(100, 100);
ctx.stroke();
```

Chapter 15, Next Steps

Practice exercises

Practice exercise 15.1

```
[
  {
    "name": "Learn JavaScript",
    "status": true
  },
  {
    "name": "Try JSON",
    "status": false
  }
]

const url = "list.json";
fetch(url).then(rep => rep.json())
  .then((data) => {
    data.forEach((el) => {
      console.log(`${el.name} = ${el.status}`);
    });
  });
```

Projects

Working with JSON

```
<!DOCTYPE html>
<html>
  <head><title>Working with JSON Project</title></head>
  <body>
```

```
<script src="myscript.js"></script>
</body>
</html>

// myscript.js

let url = "people.json";
fetch(url)
  .then(response => response.json())
  .then(data => {
    console.log(data);
    data.forEach(person => {
      console.log(`${person.first} ${person.last} - ${person.topic}`);
    });
  });

// people.json
[
  {
    "first": "Laurence",
    "last": "Svekis",
    "topic": "JavaScript"
  },
  {
    "first": "John",
    "last": "Smith",
    "topic": "HTML"
  },
  {
    "first": "Jane",
    "last": "Doe",
    "topic": "CSS"
  }
]
```

List-making project

```
<!DOCTYPE html>
<html>
  <head>
```



```

<title>JavaScript List Project</title>
</head>
<body>
  <div class="output"></div>
  <input type="text"><button>add</button>
  <script>
    const output = document.querySelector(".output");
    const myValue = document.querySelector("input");
    const btn1 = document.querySelector("button");
    const url = "list.json";
    btn1.addEventListener("click", addToList);
    let localData = localStorage.getItem("myList");
    let myList = [];
    window.addEventListener("DOMContentLoaded", () => {
      output.textContent = "Loading.....";
      if (localData) {
        myList = JSON.parse(localStorage.getItem("myList"));
        output.innerHTML = "";
        myList.forEach((el, index) => {
          maker(el);
        });
      } else {
        reloader();
      }
    });

    function addToList() {
      if (myValue.value.length > 3) {
        const myObj = {
          "name": myValue.value
        }
        myList.push(myObj);
        maker(myObj);
        savetoStorage();
      }
      myValue.value = "";
    }

    function savetoStorage() {
      console.log(myList);

```

```

    localStorage.setItem("myList", JSON.stringify(myList));
  }

  function reloader() {
    fetch(url).then(rep => rep.json())
      .then((data) => {
        myList = data;
        myList.forEach((el, index) => {
          maker(el);
        });
        savetoStorage();
      });
  }

  function maker(el) {
    const div = document.createElement("div");
    div.innerHTML = `${el.name}`;
    output.append(div);
  }
</script>
</body>
</html>

```

Self-check quiz

1. Pre-programmed JavaScript modules that you can use to speed up your development process.
2. Open the console and type \$ or jQuery, and if you get a jQuery object as a response, the page has reference to the \$ or jQuery.
3. UnderscoreJS.
4. Type node and the filename into the terminal within the file directory.



packt.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

At www.packt.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:



Responsive Web Design with HTML5 and CSS - Third Edition

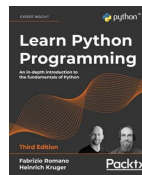
Ben Frain

ISBN: 9781839211560

- Integrate CSS media queries into your designs; apply different styles to different devices
- Load different sets of images depending upon screen size or resolution
- Leverage the speed, semantics, and clean markup of accessible HTML patterns
- Implement SVGs into your designs to provide resolution-independent images

Other Books You May Enjoy

- Apply the latest features of CSS like custom properties, variable fonts, and CSS Grid
- Add validation and interface elements like date and color pickers to HTML forms
- Understand the multitude of ways to enhance interface elements with filters, shadows, animations, and more



Learn Python Programming - Third Edition

Fabrizio Romano

Heinrich Kruger

ISBN: 9781801815093

- Get Python up and running on Windows, Mac, and Linux
- Write elegant, reusable, and efficient code in any situation
- Avoid common pitfalls like duplication, complicated design, and over-engineering
- Understand when to use the functional or object-oriented approach to programming
- Build a simple API with FastAPI and program GUI applications with Tkinter
- Get an initial overview of more complex topics such as data persistence and cryptography
- Fetch, clean, and manipulate data, making efficient use of Python's built-in data structures



Dancing with Python

Robert S. Sutor

ISBN: 9781801077859

- Explore different quantum gates and build quantum circuits with Qiskit and Python
- Write succinct code the Pythonic way using magic methods, iterators, and generators
- Analyze data, build basic machine learning models, and plot the results
- Search for information using the quantum Grover Search Algorithm
- Optimize and test your code to run efficiently

Other Books You May Enjoy

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit authors.packtpub.com and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share Your Thoughts

Now you've finished *JavaScript from Beginner to Professional*, we'd love to hear your thoughts! If you purchased the book from Amazon, please [click here](#) to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Index

A

accordion component
 building 256, 257
action
 performing, for every item 176, 177
addition operator 32
analytics
 building 294, 295
and operator 42
Angular 414
 reference link 415
anonymous functions 142, 143
API (Application Programming Interface) 416
argument 120, 121
arguments object 314
arithmetic operators 32
 addition 32
 division 34
 exponentiation 34
 modulus 35, 36
 multiplication 33
 subtraction 33
array methods 176
arrays 48, 89, 98-100
 built-in length property 51, 52
 creating 48, 49
 converting, to string 182, 183
 elements, accessing 50
 elements, adding 53-55
 elements, deleting 55, 56
 elements, finding 56, 57
 elements, overwriting 50, 51
 elements, replacing 53-55
 filtering 177, 178
 in objects 65
 manipulating 68
 methods 53
 occurrence, finding in 179, 180
 part, replacing with another
 part of array 178, 179
 reversing 58
 sorting 58
 string, converting to 182
 values, mapping 179
 working with 64
arrow functions 124, 125
 returning with 129
assignment operators 38, 39
 reference link 418
Asynchronous JavaScript and XML (AJAX) 417
async keyword 353, 354
attributes
 exercise 249, 250
 manipulating 247, 248
await keyword 353, 354

B

backend
 learning 415
BigInt 24
Boolean 25
box clicker
 speed test game 297, 298
break keyword 107-109
 in labeled blocks 113
 using, in nested loops 112, 113
breakpoints 318-320
 example 320-324
Browser Object Model (BOM) 205, 211-213
 window history object 214

 window location object 216, 217
 window navigator object 215
built-in JavaScript methods 168

C

callback queue 355-359
callbacks 346-348
call stack 355-359
canvas
 drawing, with mouse 386-389
cases
 combining 81, 82
classes 149-152
 constructors 152, 153
 methods 154
 properties 156, 157
code comments 13
code completion 3
command prompt 14
company product catalog 68
comparison operators 40
 equal 40
 greater than 41
 not equal 41
 smaller than 41
compiling 4
concat() method 55
concurrency 346
condition
 checking, for elements 178
conditional ternary operators 76
const keyword 19
constructors 152, 153
const scope 133, 134
continue keyword 107-110
 in labeled blocks 113
 using, in nested loops 112, 113
cookies
 using 327-329
copyWithin() method 178
countdown clock 397, 398
countdown timer 202, 203
CSS selector
 querySelectorAll(), using 236
 querySelector(), using 235
 used, for accessing DOM elements 234, 235

D

data-driven documents (D3) 408, 409
data position
 obtaining, with Geolocation 369, 370
data types
 analyzing 27
 converting 29-31
 modifying 27
date methods 196
dates
 converting, to string 200
 creating 197, 198
debugging 317
decimals
 turning, into integers 194, 195
decodeUri() 170
decodeURIComponent() 171, 172
decrement operator 36
default case 79, 80
default parameters 122, 123
digital accessibility
 in HTML 394, 395
division operator 34
Document Object Model (DOM) 5, 205, 217, 218
 additional properties 219
 exercise 228, 229
 overview 226-228
 page elements, selecting 220, 221
DOM elements
 accessing 230, 231
 accessing, by class name 234
 accessing, by ID 231, 232
 accessing, by tag name 232, 233
 accessing, with CSS selector 234, 235
 innerHTML, modifying 230
 innerText, modifying 230
 selecting, as objects 229
DOM Event Flow 275-278
do while loop 91, 92
dynamic canvas 373
 animations, adding to 383-385
 circles, adding to 374-376
 images, adding to 380-383
 images, uploading to 380-383
 lines, adding to 374-376

stick man, drawing 376, 377
text, adding to 377-379
dynamic images
saving 390-392

E

ECMAScript 6 (ES6) 5
element click handler 237, 238
elements
animating 292, 293
class, adding to 243, 244
class, modifying 243
class, removing 244, 245
class, toggling 246
creating 253, 254
dragging 285-287
dropping 285-287
exercise 255
mouse position, tracking 296
element style
manipulating 240-243
else-if statements 74, 75
email extractor 338
employee tracking app 163
encapsulation 158
encodeURIComponent() 170
encodeURIComponent() 171, 172
equal operator 40
error handling 325, 326
escape()
encoding with 172
escape characters 22
eval()
used, for executing JavaScript 175, 176
event bubbling 278
event capturing 278
event delegation 278
event listeners
exercise 253
on element 251, 252
event loop 355
events
specifying 266
specifying, with event listeners 267, 268
specifying, with HTML 266
specifying, with JavaScript 267

event target property 272-274
exponent 196
exponentiation operator 34
Express module
reference link 420
Express Node.js framework
using 419

F

Fetch API 417
Fibonacci sequence 90
floats
making, with parseFloat() 174, 175
foreach 87
for in loop 103
used, for looping over object 103
for loops 92-94
formatting code 11
indentations and whitespace 11, 12
semicolon 12
form submission 289-291
form validator 339, 340
for of loop 101, 102
frameworks 404, 412
Angular 414
Vue.js 412, 413
friend checker game 83
function callbacks 143, 144
functions 314
basic functions 118
invoking 118
local variables 130, 131
naming 119
return value 128
writing 118, 119
function values
returning 127

G

GeoLocation
used, for obtaining data position 369, 370
get keyword 158
getters 157-159
global JavaScript methods 169
global variables 134-136
greater than operator 41

H

Hangman game 259-261
highest number
finding 193
HTML5
with JavaScript 364
HTML5 canvas 370-372
exercise 373
HTML crash course 206
Hyper-Text Markup Language (HTML) 206
attributes 210
elements 206-210
elements, manipulating with JavaScript 222
used, for specifying events 266
Hypertext Transfer Protocol (HTTP) 416

I

if-else statements 72, 73
if statements 72, 73
immediately invoked function expression (IIFE) 136, 137
includes() method 359
increment operator 36
index
working with 183, 184
inheritance 159-161
integers
decimals, turning into 194, 195
making, with parseInt() 172, 173
Integrated Development Environment (IDE) 3
interactive content 266
interactive voting system 258, 259
interpreter 4

J

JavaScript
adding, to web page 8
executing, with eval() 175, 176
external file, linking to web page 9-11
HTML elements, manipulating with 222
in HTML 8, 9
learning, need for 2, 3
used, for specifying events 267
using, in HTML5 364

JavaScript code
code comments 13
command prompt 14
formatting code 11
random number 14
writing 11
JavaScript coding environment
Integrated Development Environment (IDE) 3
online editors 4
setting up 3
tools 4
web browser 4
JavaScript hoisting 315, 316
JavaScript Object Notation (JSON) 333, 334
parsing 336
jQuery 405-407
URL 408
JSON
working with 421

K

key event handler 282-284

L

let keyword 19
block scope 19
let variables
versus var variables 132, 133
libraries 404, 405
data-driven documents (D3) 408, 409
jQuery 405-407
React 410, 411
Underscore library 409, 410
list
creating, to save local storage 421, 422
local file reader 365
files, displaying 368, 369
files, reading 367
files, uploading 365-369
local storage 330-332
local variables
in functions 130, 131
logarithm 196
logical operators 42
and 42

not 43
or 43
loops 87
arrays 98-100
do while loop 91, 92
for a loop 101, 102
for in loop 103
for loop 92-94
for of loop 103
nested loop 95, 96
used, for creating math
multiplication table 114
while loop 88-90
lowest number
finding 193

M

Math methods 192
math multiplication table
creating, with loops 114
math quiz
creating 341, 342
Matrix effect
creating 395-397
media
adding, on page 392, 393
menu items price calculator 163
methods 154, 155
modulus operator 35, 36
mouse event handlers 270-272
multidimensional arrays 59, 60
multiplication operator 33

N

nested functions 140, 141
nested loops 95, 96
using, in break keyword 111-113
using, in continue keyword 111-113
Node.js 418, 419
reference link 418
Node Package Manager (NPM) 411
not equal operator 40
not operator 43
null 26
number 23

number game
evaluating 82
number methods 190
isFinite() 190
isInteger() 191
isNaN 190
number of decimals
specifying 192

O

object-oriented programming (OOP) 150, 409
objects 150
in arrays 66
in objects 64
looping over, by converting to array 104-106
looping over, with for in loop 103
updating 62, 63
working with 64
objects in arrays
in objects 66, 67
objects, JavaScript 61, 62
occurrence
finding, in array 179, 180
onblur 280, 281
onchange 280, 281
online editor
for JavaScript coding environment 4
online paint app 399-401
onload event handler 268, 269
operand 36
operators 32
arithmetic operators 32
assignment operators 38, 39
combining 38
comparison operators 40
logical operators 42
or operator 43
OWASP (Open Web Application Security Project) 176

P

page elements
selecting 220, 221
parameter 121
parseFloat()
used, for making floats 174, 175

parseFloat()
used, for making integers 172, 173
parsing dates 200
parsing numbers 172
password checker 359
positions
working with 183, 184
postfix unary operator 36, 37
precision
specifying 192
prefix unary operator 36, 37
primitive data types 20
BigInt 24
Boolean 25
null 26
number 23
strings 21
symbol 25
undefined 26
printStuff() 177
Promises 349-352
properties 156
getters 157-159
setters 157-159
prototype 161, 162

R

random number
generating, in JavaScript 14, 15
React 410, 411
React documentation
reference link 411
recursive functions 138, 139
creating 145
regex 184, 302
regular expressions 302, 303
character options 304-307
email validation 312
groups 307-309
multiple options, specifying
for words 303, 304
rest parameter 127
rock paper scissors game 83

S

set keyword 158
setters 157-159
setTimeout 145
single-page applications (SPAs) 412
smaller than operator 41
splice() method 54
spread operator 125, 126
star rating system
creating 295, 296
strict mode
using 316, 317
string methods 181
string parts
replacing 185, 186
strings 21
array, converting to 182, 183
combining 181
converting, to array 182
date, converting to 200
end of 187, 188
lowercase 186, 187
replacing 310
searching 310
start of 187, 188
uppercase 186, 187
substrings
creating 185
subtraction operator 33
switch statement 77, 78
default case 79, 80
symbol 25

T

this keyword 15, 238, 239
exercise 239
type of 28

U

unary operators
decrement 36
increment 36

undefined data type 26
Underscore.js
 reference link 410
Underscore library 409, 410
unescape()
 encoding with 172
uniform resource identifier (URI)s
 decoding 169, 170
 encoding 169, 170
uniform resource locator (URL) 169
unsuitable parameters 122, 123

V

values
 mapping, of array 179
variables 18
 declaring 18
 naming 20
var keyword 19
 global scope 19
var variables
 versus let variables 132, 133
Vue.js 412, 413
 reference link 414

W

web browser
 for JavaScript coding environment 4
 recognizing, JavaScript 4, 5
web browser console
 using 5-7
 working with 7
web page
 JavaScript, adding to 8
while loop 88-90
window browser object 211
window history object 214
window location object 216, 217
window navigator object 215
word scrambler 201, 202